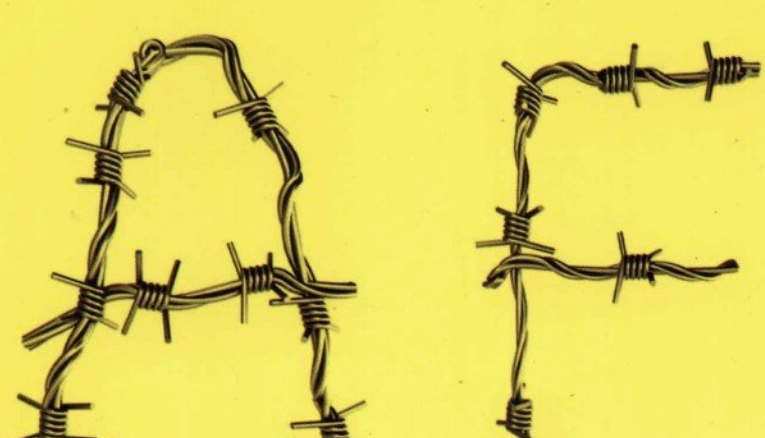
HZ BOOKS
华章科技

ELSEVIER
爱思唯尔

Android取证领域广受好评的经典著作，资深取证技术专家撰写，世界顶级取证专家审校，理论指导与实用性兼备

从Android的硬件设备、应用开发环境、系统原理多角度剖析Android系统的安全原理，结合实用的工具和案例系统讲解Android取证的技术、策略、方法和步骤

安全技术大系
SECURITY



Android Forensics
Investigation, Analysis and Mobile Security for Google Android

Android取证实战

调查、分析与移动安全

（美） Andrew Hoog 著
何泾沙 等译

机械工业出版社
China Machine Press

更多资源请访问稀酷客(www.ckook.com)

Android取证实战 调查、分析与移动安全

Android Forensics

Investigation, Analysis and Mobile Security for Google Android

如果你想学习和进行Android取证，那么本书是为你而写的，从没有一本书如此为你呈现Android操作系统与安全相关的技术细节，本书是移动取证从业人员的必备书籍。

—— **Jim Steele, 某一级无线通信运营商（Tier 1）数字取证总监**

随着采用Android系统的移动智能设备（从最初的智能手机和平板电脑到如今的汽车电子和智能机器人，等等）越来越多，Android设备用户面临的安全威胁也越来越多，针对Android智能设备的安全威胁已经成为移动互联网安全领域最严重的问题之一。与之相应的是，对Android平台上的犯罪行为进行取证的需求也逐渐增多，由于采用Android系统的设备类型和种类繁多，所以相关的取证工作也颇具难度。

本书针对目前Android取证工作者的需求给出了系统化的解决方案，首先从Android系统的硬件设备、Android应用的开发环境、Android系统的原理（文件系统和数据存储）等多角度讲解了Android的安全知识，然后结合实用的取证分析工具和案例全面讲解了Android取证的技术、原理、策略、方法和步骤。

本书译自原版Android Forensics: Investigation, Analysis and Mobile Security for Google Android并由Elsevier授权出版

ELSEVIER 爱思唯尔

客服热线：(010) 88378991 88361066
购书热线：(010) 68326294 88379649 68995259
投稿热线：(010) 88379604

数字阅读：www.hzmedia.com.cn
华章网站：www.hzbook.com
网上购书：www.china-pub.com

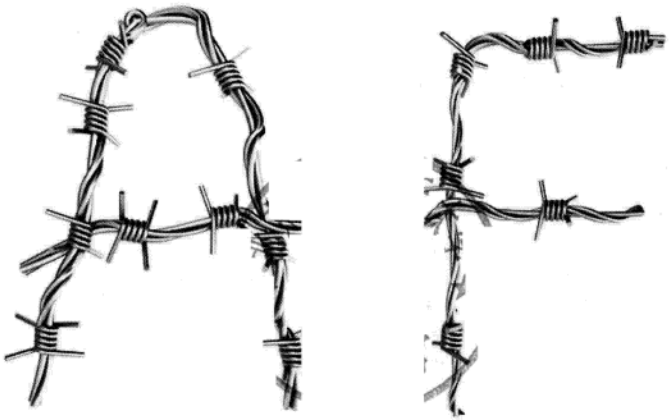
上架指导：计算机/安全

ISBN 978-7-111-42199-3

9 787111 421993 >

定价：69.00元

更多资源请访问稀酷客(www.ckook.com)



Android Forensics

Investigation, Analysis and Mobile Security for Google Android

Android取证实战

调查、分析与移动安全

（美） Andrew Hoog 著

何泾沙 等译

机械工业出版社
China Machine Press

更多资源请访问稀酷客(www.ckook.com)

图书在版编目（CIP）数据

Android 取证实战：调查、分析与移动安全 /（美）胡格（Hoog, A.）著；何泾沙等译 . —北京：机械工业出版社，2013.5
（华章程序员书库）
书名原文：Android Forensics: Investigation, Analysis and Mobile Security for Google Android

ISBN 978-7-111-42199-3

Ⅰ. A… Ⅱ. ① 胡… ② 何… Ⅲ. 移动终端 – 应用程序 – 程序设计 Ⅳ. TN929.53

中国版本图书馆 CIP 数据核字（2013）第 077825 号

版权所有 • 侵权必究
封底无防伪标均为盗版
本书法律顾问 北京市展达律师事务所

本书版权登记号：图字：01-2012-5192

本书是 Android 取证领域广受好评的经典著作，也是国内第一本关于 Android 取证的著作，由资深取证技术专家撰写，世界顶级取证专家审校，权威性毋庸置疑！从 Android 的硬件设备、应用开发环境、系统原理等多角度剖析了 Android 系统的安全原理，结合实用的取证分析工具和经典案例，讲解了 Android 取证的原理、技术、策略、方法和步骤。

全书一共 7 章：第 1 章介绍了 Android 平台的概况和特点、Linux 与 Android，Android 与取证，并讲解如何创建基于 Ubuntu 的虚拟机；第 2 章讲解了 Android 所支持的各种类型的硬件和终端设备，为取证和安全分析做好准备。第 3 章讲解了软件开发套件、Android 虚拟终端的安装，以及取证技术的一些重要概念，涵盖 Davlik 虚拟机、Android 程序调试桥、USB 调试设置等；第 4 章分析了 Android 系统的数据存储方式、涉及的内存类型，以及 Android 中常见的各类文件系统；第 5 章分析了 Android 终端设备成为泄漏数据以及用于作为主动攻击源的原因，并为个人、企业安全总监和应用开发者提供了一些非常具体的建议；第 6 章深入讲解了规避密码的几个不同策略和多种逻辑获取技术和物理获取技术（如 adb pull、备份分析、AFLogical、JTAG、芯片摘取、AFPhysical 等）；第 7 章介绍了一些具体的策略和 Android 文件的目录（文件夹）结构，并深入分析了 11 个可以用于获取 Android 终端设备中主要数据的应用程序。

Andrew Hoog: Android Forensics: Investigation, Analysis and Mobile Security for Google Android (ISBN: 9781597496513).

Copyright © 2011 by Elsevier Inc. All rights reserved.

Authorized Simplified Chinese translation edition published by the Proprietor.

Copyright © 2013 by Elsevier (Singapore) Pte Ltd. All rights reserved.

Printed in China by China Machine Press under special arrangement with Elsevier (Singapore) Pte Ltd. This edition is authorized for sale in China only, excluding Hong Kong SAR, Macau and Taiwan. Unauthorized export of this edition is a violation of the Copyright Act. Violation of this Law is subject to Civil and Criminal Penalties.

本书简体中文版由 Elsevier（Singapore）Pte Ltd. 授权机械工业出版社在中国大陆境内独家出版和发行。本版仅限在中国境内（不包括香港特别行政区、澳门特别行政区及台湾地区）出版及标价销售。未经许可之出口，视为违反著作权法，将受法律之制裁。

本书封底贴有 Elsevier 防伪标签，无标签者不得销售。

机械工业出版社（北京市西城区百万庄大街 22 号 邮政编码 100037）
责任编辑：高婧雅
三河市杨庄长鸣印刷装订厂印刷
2013 年 6 月第 1 版第 1 次印刷
186mm×240mm • 20.25 印张
标准书号：ISBN 978-7-111-42199-3
定价：69.00 元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换
客服热线：（010）88378991 88361066 投稿热线：（010）88379604
购书热线：（010）68326294 88379649 68995259 读者信箱：hzjsj@hzbook.com

更多资源请访问稀酷客(www.ckook.com)

译 者 序

从2008年10月第一部Android智能手机诞生至今，在短短不到五年的时间内，Android移动平台已经快速成长为全球最流行的、用于移动应用开发的操作系统，在2011年就已经以27%的份额占据智能手机操作系统的首位。虽然目前关于Android系统应用方面的统计数据基本上都来源于智能手机和平板电脑，然而因其在移动终端设备市场中所具有的强大竞争力以及在系统功能方面所具有的优越性，相信在不久的将来，Android操作系统将会在其他各种类型的移动终端设备中得到更加广泛的应用，诸如汽车电子、智能电视、全球定位系统（GPS）终端、游戏机、笔记本电脑以及其他电子产品。因此，学习和掌握Android操作系统及其在移动系统中的应用对跟踪当今最新信息技术的发展趋势至关重要。

伴随着信息技术的快速发展，信息已经成为人们工作和生活中的重要资源。尤其是近年来移动通信技术的飞速发展以及智能手机应用的日益普及，信息在各个层面（国家、社会、团体、个人）更加紧密地联系在一起，各类信息无所不在、触手可及。然而，以移动通信技术为代表的信息技术在给人们的工作和生活带来极大便利的同时，也不可避免地引发了各种负面问题，尤其是利用网络和信息而引发的各类非法或犯罪行为，如窃取国家机密和个人隐私信息、网络诈骗、网上非法交易、在线间谍、黑客攻击、网上刺探等犯罪行为不断出现，而且呈现出高速增长的势头。利用智能手机或其他无线移动终端设备进行犯罪是一种典型的高科技犯罪，亟待设计出相应的对策加以防范、监控和打击，以确保信息技术与应用的持续快速发展。打击利用信息技术进行犯罪所面临的挑战之一就是在犯罪实施后如何按照法律、法规的要求对犯罪事实进行认定，而认定犯罪事实的关键就在于获取有效证据。因此，计算机取证（数字取证），即从信息系统中获取与犯罪事实相关的数据，并使其成为法律上可采纳的有效证据，这在近年来已经成为信息系统安全研究领域中的一个热点问题。然而，与传统的信息安全问题不同，计算机取证不仅仅是一个技术问题，它还涉及诸如法律法规、社会道德规范、个人隐私权利等众多有关其他技术和人文因素的考虑，属于交叉学科问题，这就给计算机取证带来了一定程度的复杂性。

鉴于基于Android的无线移动终端设备应用的日益广泛，对这些终端设备中的数据进行提取、处理和分析以获得司法证据也变得越来越迫切，成为一个不可忽略的途径。公安机关、法院等各级侦探和执法部门急需在案件的侦探和审理过程中获取涉案人员的智能手机或其他移动终端设备中存有的通讯录、通话记录、来往信息、数码照片、各类文件等用户信息和数据，并尽可能对已经删除的以上信息和数据进行恢复，以推进案件的调查和审理。因此移动取证技术成为打击网络犯罪、保护合法信息的重要技术手段。然而，面向移动终端设备进行取证所涉及的信息采集和分析具有较高的技术难度，同时也受到系统和应

更多资源请访问稀酷客(www.ckook.com)

用的影响。因此，针对不同的系统，深入分析系统的内部体系结构、掌握取证的原理和方法对实现移动取证至关重要。

本书就是在这一背景下，针对上述需求而撰写完成的，在对 Android 操作系统的相关部分进行深入分析的基础上，系统地介绍了如何从基于 Android 操作系统的移动终端中获取数据的手段和技术，是一本较完整的学习和研究 Android 取证技术的技术参考书。学习和掌握 Android 取证技术，一方面能够增强人们对相关信息和数据的保护意识，并可利用更加有效的手段来保护这些信息和数据的安全（如及时进行迁移或删除而不被犯罪分子所窃取），另一方面也能够帮助各级执法部门获取网络犯罪的有效证据，打击网络犯罪，保障国家安全和个人隐私权利。

本书由北京工业大学教授、博士生导师、北京市特聘教授何泾沙博士负责翻译，北京工业大学博士研究生赵斌、国家软件产品质量监测检验中心朱娜斐博士参与了部分章节的翻译工作。北京工业大学博士研究生张玉强、中国科学院信息工程研究所徐菲博士，以及张航、张伊璇、刘公政、万雪姣、张伯雍、潘力斌等参与了翻译过程中举办的相关技术研讨，并协助了部分章节的翻译工作。全书最后由何泾沙博士进行统稿及审校。由于译者的水平有限，再加上时间上的限制，全书的翻译中难免存在不妥之处，敬请广大读者批评指正，译者在此深表谢意。

何泾沙

更多资源请访问稀酷客(www.ckook.com)

前　言

从2008年10月第一部Android手机的诞生到2011年年初，Android移动平台迅速成长为全球最流行的移动操作系统。对消费者来说，Android平台的爆炸性成长在竞争和功能方面都是一个巨大的胜利。同时，由于缺乏用于调查相应设备和手持终端的知识以及工具，取证分析师和信息安全工程师遇到了巨大的难题。本书试图解决这些问题，不仅详细介绍了Android的软件、硬件和文件系统，而且分享了计算机取证获取技术以及随后所需的设备分析工作。对于缺乏计算机取证方面知识的读者，本书将提供基于免费、开放的代码库编写的实例，而且读者也可以亲身参与到这些实例的编写中。因为免费的Android开发套件能够提供一个完整的Android仿真器，所以读者不需要实际拥有任何Android终端设备。

随着Android终端设备数量的快速增加，用户对于这些终端设备中存储着数据的意识也在相应增强。然而遗憾的是，大多数对这些数据感兴趣的却是网络犯罪团伙，他们更清楚地知道这些终端设备中存储着大量的个人信息和商业信息，而对Android平台的成功攻击会为其带来巨大的“收益”。如果要应对这种威胁，就需要深入地了解Android平台，不但对于Android系统开发者和制造商如此，对于Android应用开发者和企业中的信息安全管理者也是如此。更加安全的应用将有助于防止敏感信息泄露，也会帮助信息安全管理者为系统制订更加有效的安全策略。

目前很多关于Android的统计数据都是针对智能手机以及平板电脑的，在不久的将来，Android将会在更多的终端设备中得到应用，如汽车、电视、全球定位系统（GPS）终端、游戏机、笔记本电脑以及其他电子产品。对于取证分析师和安全工程师来说，针对Android的取证分析将会有一个很大比例的增长。最后，Android的魅力不只绽放于某些特定的国家或地区，它将会对全球的个人、企业和机构产生巨大的影响。

下面将对本书中各章的内容进行简要介绍。

第1章

该章在介绍Android平台的发展史后，还将讨论Android开放源码项目（Android Open Source Project，AOSP）、平台国际化、Android市场，同时提供有关Linux的一些简单辅导信息，并对Android取证进行简要说明。本章还将逐步讲解如何创建一个基于Ubuntu的虚拟机，其后在全书的实例中都将会用到。本书极力推荐Ubuntu虚拟机，它还可以应用到本书以外的其他Android取证实例中。

第2章

该章介绍Android所支持的各种类型的硬件和终端设备。虽然硬件的兼容性对制造商来说非常重要，最终也会使消费者受益，但是对大量硬件和终端设备的广泛支持却给取证

更多资源请访问稀酷客(www.ckook.com)

分析师和安全工程师带来了巨大的挑战。因此，对硬件组件、终端设备类型以及 Android 启动过程的清楚了解非常有助于对整个 Android 的了解，也对取证和安全分析具有很大帮助。

第 3 章

该章讨论 Android 系统的不同版本、Android 软件开发套件（Software Development Kit，SDK）、Davlik 虚拟机、Android 安全的核心组件以及其他与 Android 取证相关的重要概念，如 Android 程序调试桥（adb）、USB 调试设置等。本章还将提供一些详细的实例，包括如何在 Linux、OS X 和 Windows 上安装软件开发套件，以及如何创建一个 Android 虚拟终端用以测试取证技术。

第 4 章

该章介绍如何在 Android 终端设备上存储数据，内容包括数据存储方式（共享参考、文件、SQLite 及网络）以及 Android 终端设备所使用的内存类型，如 RAM 以及十分重要的 NAND 闪存。该章也会详细介绍读者在 Android 终端设备中经常会遇到的各类文件系统，如 YAFFS2、EXT、FAT32/FAT16 及各类底层文件系统。

第 5 章

该章讨论 Android 终端设备、数据和应用的安全。首先回顾了 Android 终端设备如何泄露数据及其如何被用做一个主动攻击源。在讨论一些重要的安全概念后，该章将给个人、企业安全管理者和应用开发者这三类主要读者提供一些非常具体的建议。随着 Android 持续快速增长，数据和信息的安全问题将变得日益严峻。因此，该章对这个重要的问题进行了深入且切合实际的快速讨论。

第 6 章

该章介绍具体且十分有用的从 Android 终端设备获得取证信息的一些技术。在介绍了不同类型的获取方法以及如何对 Android 终端设备进行操控后，本章还讨论了规避密码的 7 个不同策略。随后，本章介绍一些具体的获取 SD 卡以及任何插入的内置多媒体卡（Embedded Multi Media Card）的技术以及相对应的代码。在此基础上，该章介绍一些逻辑获取技术，包括一些已经包含在 Android 和软件开发套件（SDK）里的技术以及一个称为 AFLogical 的可以供执法者和政府机构免费使用的技术，并简单描述了 6 个商用取证软件。最后，本章详细介绍了获取 NAND 闪存物理映像的技术，包括 6 个如何获得超级用户权限的方法以及由 viaForensics 公司开发的 AFPhysical 技术。

第 7 章

该章是最后一章，将介绍一些具体的策略以及相应的应用程序，使取证分析师和安全工程师可以对得到的 Android 终端设备进行分析。虽然很多传统的取证技术仍然适用于 Android 取证分析，但是 Android 中新的文件系统以及硬件的特殊性要求更多的新技术。没有这些新技术，从 Android 终端设备中实际获取的数据只能够得到极少有价值的内容和信息。除了提供背景信息以及实用的应用软件，该章还将给出 Android 文件的目录（文件夹）结构，并深入分析 11 个可以用于获取 Android 终端设备中主要数据的应用程序。具备了以上知识，取证分析师和安全工程师可以对任何可以获得的 Android 终端设备进行取证分析。

更多资源请访问稀酷客(www.ckook.com)

网站

访问以下网站可以获得与本书相配套的其他资料，如代码、程序及随后的更新等：http://viaforensics.com/education/android-forensics-mobile-security-book/。

致谢

我现在真正理解“举全村之力”（It takes a village）的深刻含义了，它不但适用于描述抚养孩子，也同样适用于撰写书籍。因此，我要衷心感谢“整个村庄”：

- 感谢我的家庭成员：妻子和女儿们。
- 感谢 Lee Haas 为本书所做的出色的文本编辑工作，并时刻督促我按照原定计划完成此书的撰写。
- 感谢 Ted Eull 为读者提供的出色服务，时常帮助我将头脑里激荡的想法用清晰的言语和文字表达出来。同时 Ted 也是我的好朋友。衷心感谢他的妻子，为他在创业过程中长时间工作所给予的耐心和支持。
- 感谢 Chris Triplett 率先开始对 Android 的研究及其所进行的出色工作。Chris 在将通俗的英语应用到数字取证，以弥补语言描述上的欠缺，以及提供喜剧性疏解方面十分有天分。
- 感谢 Katie Strzempka 负责另外一本书的撰写（“iPhone and iOS Forensics”）。在此郑重地向读者推荐这本书。
- 感谢我的父母 Stevie 和 Al。他们从人生的开始就为我制定了一条正确的发展道路，并且能够在我稍有偏差时及时提醒。
- 感谢 Harmonee 和 Hadabogee 照顾我的女儿们、准备晚餐以及提供的其他很多方面的帮助。
- 感谢众多在地方、州、联邦政府的法律部门及其他部门中为公共事业提供服务的勇敢的人们，感谢他们为我们的社会和国家所做的一切。
- 感谢 Google（谷歌）认识到 Android 的价值，并且为移动终端设备创造了一个新的开放平台。
- 感谢苹果公司提供了另外一个开发平台。
- 最后，感谢所有的读者。我衷心希望本书可以为读者带来价值，并赞赏他们给予的任何支持。

更多资源请访问稀酷客(www.ckook.com)

推荐阅读

黑客大曝光：Web应用程序安全（原书第3版）

作者：（美）Joel Scambray 等 译者：姚军 等 ISBN：978-7-111-35662-2 定价：65.00元

黑客大曝光：恶意软件和Rootkit安全

作者：（美）Michael A. Davis 等 译者：姚军 等 ISBN：978-7-111-34034-8 定价：55.00元

黑客大曝光：无线网络安全（原书第2版）

作者：（美）Johnny Cache 等 译者：李瑞民 等 ISBN：978-7-111-37248-6 定价：69.00元

C++反汇编与逆向分析技术揭秘

作者：钱林松 等 ISBN：978-7-111-35633-2 定价：69.00元

网络扫描技术揭秘：原理、实践与扫描器的实现

作者：李瑞民 ISBN：978-7-111-36532-7 定价：79.00元

BackTrack 4：利用渗透测试保证系统安全

作者：Shakeel Ali 等 译者：陈雪斌 等 ISBN：978-7-111-36643-0 定价：59.00元

Windows PE权威指南

作者：威利 ISBN：978-7-111-35418-5 定价：89.00元

内核漏洞的利用与防范

作者：Enrico Perla 等 译者：吴世忠 等 ISBN：978-7-111-37429-9 定价：79.00元

Java加密与解密的艺术

作者：梁栋 ISBN：978-7-111-29762-8 定价：69.00元

云计算安全与隐私

作者：Tim Mather 等 译者：刘戈舟 等 ISBN：978-7-111-34525-1 定价：65.00元

安全之美

作者：Andy Oram 等 译者：徐波 等 ISBN：978-7-111-33477-4 定价：65.00元

更多资源请访问稀酷客(www.ckook.com)

目　录

译者序
前言

第 1 章　Android 与移动取证 …… 1
1.1　绪论 …… 1
1.2　Android 平台 …… 1
1.2.1　Android 的发展历程 …… 3
1.2.2　谷歌的策略 …… 8
1.3　Linux、开源软件与取证 …… 10
1.4　Android 开放源码项目 …… 24
1.4.1　AOSP 使用许可 …… 24
1.4.2　开发过程 …… 25
1.4.3　开源在取证中的价值 …… 27
1.4.4　AOSP 下载和编译 …… 28
1.5　Android 平台的国际化 …… 29
1.5.1　Unicode …… 29
1.5.2　输入键盘 …… 29
1.5.3　个性化分支 …… 30
1.6　Android 市场 …… 31
1.6.1　应用软件安装 …… 32
1.6.2　应用情况统计 …… 34
1.7　Android 取证 …… 34
1.8　本章小结 …… 35
1.9　参考资料 …… 35

第 2 章　Android 硬件平台 …… 37
2.1　绪论 …… 37
2.2　核心部件概述 …… 37
2.2.1　中央处理器 …… 37
2.2.2　基带调制解调器 / 无线电 …… 38
2.2.3　内存（RAM 与 NAND 闪存）… 38
2.2.4　GPS …… 39
2.2.5　无线（Wi-Fi 与蓝牙） …… 39
2.2.6　安全数字卡 …… 39
2.2.7　显示屏 …… 40
2.2.8　摄像机 …… 40
2.2.9　输入键盘 …… 41
2.2.10　电池 …… 41
2.2.11　通用串行总线 …… 41
2.2.12　加速仪 / 陀螺仪 …… 42
2.2.13　音响 / 麦克风 …… 42
2.3　终端设备类型概述 …… 42
2.3.1　智能手机 …… 43
2.3.2　平板电脑 …… 43
2.3.3　上网本电脑 …… 43
2.3.4　谷歌 TV …… 43
2.3.5　车辆（内置） …… 43
2.3.6　GPS 设备 …… 44
2.3.7　其他终端设备 …… 44
2.4　只读内存及启动加载程序 …… 44
2.4.1　开启电源和片上 ROM 代码执行 …… 45
2.4.2　启动加载程序（初始程序加载 / 第二阶段程序加载）…… 46
2.4.3　Linux 内核 …… 46
2.4.4　init 进程 …… 47
2.4.5　Zygote 和 Dalvik …… 49
2.4.6　系统服务器 …… 49
2.5　制造商 …… 50
2.6　Android 更新 …… 51
2.6.1　自定义用户界面 …… 52
2.6.2　售后市场中的 Android 终端设备 … 52

更多资源请访问稀酷客(www.ckook.com)

2.7 具体的终端设备 …… 53
2.7.1 T-Mobile G1 …… 53
2.7.2 摩托罗拉 Droid …… 53
2.7.3 HTC Incredible …… 53
2.7.4 谷歌 Nexus One …… 54
2.8 本章小结 …… 54
2.9 参考资料 …… 55
第 3 章 Android 软件开发套件和 Android 程序调试桥 …… 56
3.1 绪论 …… 56
3.2 Android 平台 …… 56
3.3 软件开发套件 …… 60
3.3.1 软件开发套件的发布史 …… 60
3.3.2 软件开发套件的安装 …… 61
3.3.3 Android 虚拟终端设备（仿真器） …… 68
3.3.4 Android 操作系统体系结构 … 71
3.3.5 Dalvik 虚拟机 …… 72
3.3.6 本地代码开发 …… 73
3.4 Android 安全模型 …… 73
3.5 取证与软件开发套件 …… 74
3.5.1 将 Android 终端设备与工作站进行连接 …… 75
3.5.2 USB 接口 …… 78
3.5.3 Android 程序调试桥简介 …… 83
3.6 本章小结 …… 85
3.7 参考资料 …… 85
第 4 章 Android 文件系统与数据结构 … 86
4.1 绪论 …… 86
4.2 shell 中的数据 …… 86
4.2.1 存储的数据 …… 86
4.2.2 应用数据存储目录结构 …… 87
4.2.3 数据如何存储 …… 88
4.3 内存类型 …… 105
4.4 文件系统 …… 112
4.4.1 rootfs、devpts、sysfs 和 cgroup 文件系统 …… 113
4.4.2 proc …… 115
4.4.3 tmpfs …… 116
4.4.4 扩展文件系统 …… 119
4.4.5 FAT32/VFAT …… 120
4.4.6 YAFFS2 …… 120
4.5 挂载的文件系统 …… 131
4.6 本章小结 …… 134
4.7 参考资料 …… 134
第 5 章 Android 终端设备、数据与应用安全 …… 135
5.1 绪论 …… 135
5.2 数据窃取目标和攻击向量 …… 136
5.2.1 以 Android 终端设备作为目标 … 136
5.2.2 以 Android 终端设备作为攻击向量 …… 143
5.2.3 数据存储 …… 143
5.2.4 用于记录的终端设备 …… 144
5.3 安全考虑 …… 145
5.3.1 安全的哲学原理 …… 145
5.3.2 美国的计算机犯罪法律与规定 …… 146
5.3.3 开放源码与封闭源码 …… 148
5.3.4 NAND 闪存加密 …… 149
5.4 个人安全策略 …… 150
5.5 企业安全策略 …… 152
5.5.1 安全策略 …… 152
5.5.2 密码、模式和个人识别号锁 … 152
5.5.3 终端设备远程清除 …… 153
5.5.4 升级到最新版软件 …… 154
5.5.5 终端设备的远程管理功能 …… 154
5.5.6 应用软件与终端设备审计 …… 156
5.6 应用开发安全策略 …… 157
5.6.1 移动应用安全测试 …… 157

更多资源请访问稀酷客(www.ckook.com)

5.6.2 应用安全策略 …… 158
5.7 本章小结 …… 164
5.8 参考资料 …… 164
第 6 章 Android 取证技术 …… 166
6.1 绪论 …… 166
6.1.1 取证调查的类型 …… 166
6.1.2 逻辑技术与物理技术的区别 … 167
6.1.3 修改目标终端设备 …… 168
6.2 操作 Android 终端设备的程序 … 169
6.2.1 终端设备的安全保护 …… 169
6.2.2 网络隔离 …… 170
6.2.3 如何绕过口令 …… 172
6.3 Android USB 大容量存储终端设备映像 …… 178
6.3.1 SD 卡与 eMMC …… 179
6.3.2 如何获得 SD 卡或 eMMC 的取证映像 …… 179
6.4 逻辑技术 …… 185
6.4.1 adb pull …… 185
6.4.2 备份分析 …… 186
6.4.3 AFLogical …… 187
6.4.4 供应商 …… 193
6.5 物理技术 …… 220
6.5.1 基于硬件的物理技术 …… 221
6.5.2 基于软件的物理技术和权限 … 223
6.5.3 AFPhysical 技术 …… 230
6.6 本章小结 …… 236
6.7 参考资料 …… 236
第 7 章 Android 应用与取证分析 …… 238
7.1 绪论 …… 238
7.2 分析技术 …… 238
7.2.1 时间序列分析 …… 238
7.2.2 文件系统分析 …… 241
7.2.3 文件雕复 …… 244
7.2.4 strings 命令 …… 246
7.2.5 十六进制：取证分析师的好朋友 …… 248
7.2.6 Android 目录结构 …… 254
7.3 FAT 取证分析 …… 260
7.3.1 FAT 时间序列分析 …… 261
7.3.2 更多的 FAT 分析 …… 268
7.3.3 FAT 分析师说明 …… 269
7.4 YAFFS2 取证分析 …… 272
7.4.1 YAFFS2 时间序列分析 …… 275
7.4.2 YAFFS2 文件系统分析 …… 280
7.4.3 YAFFS2 文件雕复 …… 283
7.4.4 YAFFS2 的 strings 分析 …… 285
7.4.5 YAFFS2 分析师注意事项 …… 286
7.5 Android 应用分析与参考 …… 290
7.5.1 Messaging（短信与彩信）…… 290
7.5.2 多媒体消息帮助应用 …… 292
7.5.3 浏览器 …… 292
7.5.4 联系人 …… 297
7.5.5 媒体扫描仪 …… 299
7.5.6 YouTube …… 301
7.5.7 Cooliris 多媒体展厅 …… 302
7.5.8 谷歌地图 …… 303
7.5.9 Gmail …… 307
7.5.10 Facebook …… 309
7.5.11 Adobe Reader …… 311
7.6 本章小结 …… 312
7.7 参考资料 …… 312

更多资源请访问稀酷客(www.ckook.com)

第 1 章

Android 与移动取证

本章要点

- Android 平台
- Linux、开源软件与取证
- Android 开放源码项目
- Android 平台的国际化
- Android 市场
- Android 取证

1.1 绪论

数字取证是一个令人兴奋且高速发展的技术领域，可对诸如公司内部调查分析、民事诉讼、刑事调查、情报收集以及涉及国家安全等众多方面产生巨大的影响。作为快速发展及变化的领域，移动取证给人们带来了巨大的商机以及严峻的挑战。虽然 Android 取证所涉及的从移动终端设备中获取数据并对其加以分析本身非常有趣，但是首先对 Android 平台以及取证分析需要使用的工具进行全面了解尤为重要。全面掌握此方面的知识能够使取证分析师和安全工程师成功地对 Android 终端设备进行取证分析。

> **提示：针对本书内容所做的更正、更新以及相关软件**
>
> 所有针对本书内容所做的更正、更新，甚至软件实例都将在线提供，通过以下网址进行访问：http://viaforensics.com/education/android-forensics-mobile-security-book/。希望读者经常访问以上网站，随着时间推移，它会给读者提供更多有价值的信息。除了内容上的更正与更新，本书中用到的某些软件也可以通过此网站进行下载。

1.2 Android 平台

Android 是一个基于 Linux 2.6 内核的开源移动终端设备平台，并通过电信服务提供商、移动设备和元部件制造商以及软件开发商联合成立的开放手机联盟（Open Handset Alliance）进行管理。

更多资源请访问稀酷客(www.ckook.com)

Android 已经对智能手机市场产生了巨大影响，对于取证领域来说也将如此。在第一部 Android 终端设备于 2008 年 10 月推出的两年零一个月之后，Android 就成为全球第二大智能手机平台，占领了美国 6.15 亿部智能手机市场中 26% 的份额（comScore 报告）。表 1.1 是由 comScore 公司提供的 2010 年 11 月美国智能手机市场占有份额数据。

表 1.1 2010 年 11 月美国智能手机占用份额（13 周岁及以上用户）

平　台	智能手机用户占有市场份额（%）
RIM	33.5
谷歌（Google）	26.0
苹果（Apple）	25.0
微软（Microsoft）	9.0
奔迈（Palm）	3.9

然而，Android 的影响力已不仅限于美国市场。根据高德纳咨询公司（Gartner, Inc.）的市场分析报告，Android 操作系统（OS）在 2010 年第三季度已经成为第二最受欢迎的操作系统，在全球智能手机市场占有 25.5% 的份额（Gartner says, n.d.），如表 1.2 所示。

表 1.2 2009—2010 年第三季度全球的智能手机销售量，按照操作系统分类

公　司	2010 年第三季度销售量（千台）	2010 年第三季度市场占有率（%）	2009 年第三季度销售量（千台）	2009 年第三季度市场占有率（%）
塞班（Symbian）	29 480.1	36.6	18 314.8	44.6
Android	20 500.0	25.5	1424.5	3.5
iOS	13 484.4	16.7	7040.4	17.1
Research in motion	11 908.3	14.8	8522.7	20.7
微软（Microsoft）				
Windows mobile	2247.9	2.8	3259.9	7.9
Linux	1697.1	2.1	1918.5	4.7
其他操作系统	1214.8	1.5	612.5	1.5
总计	80 532.6	100.0	41 093.3	100.0

引用 Google Investor 网站上所发布的谷歌首席执行官 Eric Schmidt 的报告，2011 年 2 月每天就有超过 350 000 部 Android 终端设备投入使用（Google investor, n.d.）。以上数据只是智能手机市场的情况，而智能手机仅仅是市场上应用 Android 系统的众多终端设备中的一类。

Android 的开放源码特性不但为工业界开辟了一个新的发展方向，而且能够让开发者、精通代码的取证分析师以及高水平的犯罪团伙（非常不幸）对终端设备进行深入透彻的了解。随着核心的平台技术快速成熟（同时可以继续免费使用），网络服务提供商及各类硬件生产商能够把更多的精力放在如何提供专门的、个性化的服务和功能上，以留住他们所拥有的客户。

更多资源请访问稀酷客(www.ckook.com)

1.2.1　Android 的发展历程

在过去 30 年间，企业在手持计算设备的研发上投入了巨资，希望培育出新的市场。与传统计算机的发展情况相似，用于制造手持计算设备的核心硬件元部件在技术上取得了巨大的进步，为手持计算设备提供了一个体积虽小但功能强大的移动平台。

对 Android 的发展起到关键作用的核心人物当属 Andy Rubin，他曾经任职于几家研发机器人的公司，还有苹果公司、WebTV 及 Danger Inc. 公司。其在任职 Danger Inc. 公司期间开发出了一款智能手机以及相应的操作系统，即人们熟知的 T-Mobile Sidekick。这个叫做 DangerOS 的操作系统使用 Java 进行开发，提供软件开发套件，并具有现代智能手机的部分功能。2004 年，Rubin 离开了 Danger Inc. 公司，在尝试了几个新的想法后，又回到了智能手机开发行业，与几个志同道合的工程师进行合作，于 2003 年成立了 Android 公司（Android, Inc.）。

在合作团队进行开发的同时，Rubin 积极地向潜在的投资人和无线通信服务提供商宣传 Android，其中就包括谷歌公司，随后谷歌公司在 2005 年 7 月收购了 Android 公司。对 Android 公司的收购以及随之而来的申请专利、提供涉及移动通信的服务以及投标获得无线通信频谱使市场推测谷歌正在开发属于自己的智能手机并有可能将公司发展成为一个无线通信服务提供商。

然而，2007 年 11 月 5 日，Andy Rubin 在谷歌官方博客上宣布了一个更加宏伟的计划（Official Google blog, n.d.）：

> Android 是第一款真正开放的综合移动开发平台，包含一个操作系统、用户界面以及应用程序，即包括运行移动电话所需要的所有软件，但却不会是一个阻碍移动创新的私有财产。Android 由我们与开放手机联盟（Open Handset Alliance）合作开发，该联盟由 30 多家高科技公司和主要移动通信服务公司组成，包括摩托罗拉公司（Motorola）、美国高通公司（Qualcomm）、宏达国际电子股份有限公司（HTC，High Tech Computer Corporation）和 T Mobile。通过与无线通信服务提供商、设备制造商、开发商及其他相关公司建立全面的合作伙伴关系，我们希望推出一个标准的、开放的移动软件平台，为移动通信界打造一个基于开放的生态系统。我们相信最终结果将会更好地加快创新的步伐，给移动通信用户提供超乎想象的应用和性能。

一周后，谷歌向开发商发布了早期版的 Android 软件开发套件（SDK）。此次发布也让谷歌在 2008 年的 1 月至 4 月间发起了第一波 Android 开发挑战（Android Developer Challenge）。谷歌为此投入了 100 万美元，奖励最具创新的 Android 应用。50 个获得此荣誉的 Android 应用登载在以下网站：http://code.google.com/android/adc/adc_gallery/。

2008 年 8 月，谷歌宣布推出 Android 市场（Android Market），任何应用开发商都可以将自己为移动终端设备开发的软件上传到 Android 市场，供移动用户浏览、下载及安

更多资源请访问稀酷客(www.ckook.com)

装。最初的版本不支持对下载的应用程序进行收费。随后，在 2009 年年初，谷歌为 Android 市场增加了收费功能。最后，2008 年 10 月，谷歌正式发布了官方版本的 Android 开放源码项目（AOSP）（Bort, n.d.），并正式向市场推出第一款 Android 智能手机 T-Mobile G1。

一经推出，Android 生态系统就经历了一个高速的增长，目前拥有来自于众多领域的贡献者。表 1.3 总结了 Android 平台的重要发展里程碑。

表 1.3 Android 的发展里程碑

时　间	事　件
2005 年 7 月 1 日	谷歌收购 Android 公司
2007 年 11 月 12 日	Android 系统正式推出
2008 年 8 月 28 日	Android 市场正式推出
2008 年 9 月 23 日	Android 1.0 平台正式推出
2008 年 10 月 21 日	Android 正式成为开放源码软件
2009 年 2 月 13 日	Android 市场：美国开始接受收费应用
2009 年 3 月 12 日	Android 市场：英国开始接受收费应用
2009 年 4 月 15 日	Android 1.5 (Cupcake) 平台正式推出
2009 年 9 月 16 日	Android 1.6 (Donut) 平台正式推出
2009 年 10 月 5 日	Android 2.0/2.1 (Eclair) 平台正式推出
2010 年 5 月 20 日	Android 2.2 (Froyo) 平台正式推出
2010 年 5 月 23 日	Android 2.2 Nexus One 版手机正式推出
2010 年 12 月 6 日	Android 2.3 (Gingerbread) 平台正式推出
2011 年 2 月 2 日	Android 3.0 (Honeycomb) 预览版正式推出

1. 开放手机联盟

开放手机联盟（Open Handset Alliance，OHA）是一个由移动技术公司组成的共同合作实体，包括无线通信服务提供商、手机和零部件制造商、软件开发商及其他技术支持商和系统集成商。此联盟成立于 2007 年 11 月 5 日，最初有 34 个成员。然而，到 2011 年 1 月，此联盟已经发展到拥有近 80 个成员。

开放手机联盟（OHA）的宗旨是“加速移动技术创新，为消费者提供功能丰富、价格低廉、服务优质的移动体验”（Alliance FAQ，n.d.），其工作重点是协调、开发及推出 Android 终端设备。谷歌是 OHA 和 AOSP 的主要幕后推动者。因此，有人抱怨说此联盟只不过是一个市场销售机制，没有为联盟成员和消费者提供任何实质性的价值。然而，在 2010 年，新的成员继续加入此联盟，OHA 毫无疑问将继续存在。表 1.4 列出截至 2011 年 2 月 3 日 OHA 成员的名单，按照移动通信服务提供商、手机制造商、半导体生产商、软件开发公司、市场推广公司进行分类（联盟成员 Alliance members，n.d.）。

更多资源请访问稀酷客(www.ckook.com)

表 1.4　开放手机联盟成员

成员类型	公　司
移动通信服务提供商	Bouygues Telecom
	中国移动通信公司
	中国电信公司
	中国联通
	日本 KDDI 公司
	日本 NTT DoCoMo 公司
	软银移动公司（Softbank Mobile Corp.）
	Sprint Nextel
	T-Mobile
	意大利电信（Telecom Italia）
	Telefo' nica
	Telus
	沃达丰（Vodafone）
手机制造商	宏碁公司（Acer Inc.）
	阿尔卡特移动电话（Alcatel Mobile Phones）
	华硕电脑（ASUSTeK Computer Inc.）
	CCI
	戴尔公司（Dell）
	富士康国际（FIH）
	海尔电信（青岛）有限责任公司
	宏达国际电子股份有限公司（HTC Corporation）
	华为科技公司
	京瓷株式会社（Kyocera）
	联想移动通信技术公司
	LG
	摩托罗拉公司（Motorola）
	日本 NEC 公司
	三星电子（Samsung）
	夏普公司（Sharp Corporation）
	索尼爱立信公司（Sony Ericsson）
	东芝公司（Toshiba Corporation）
	中兴通讯股份有限公司（ZTE Corporation）
半导体生产商	AKM Semiconductor Inc.
	听众（Audience）
	ARM
	Atheros Communications
	博通公司（Broadcom Corporation）

更多资源请访问稀酷客(www.ckook.com)

（续）

成员类型	公　司
半导体生产商	CSR Plc.
	赛普拉斯半导体公司（Cypress Semiconductor Corp.）
	飞思卡尔（Freescale Semiconductor）
	金雅拓（Gemalto）
	英特尔公司（Intel Corporation）
	Marvell Semiconductor, Inc.
	联发科技（MediaTek, Inc.）
	MIPS 科技公司（MIPS Technologies, Inc.）
	英伟达（Nvidia Corporation）
	美国高通公司（Qualcomm）
	瑞萨电子公司（Renesas Electronics Corp.）
	ST-Ericsson
	Synaptics, Inc.
	德州仪器（Texas Instruments Inc.）
	威睿电通（Via Telecom）
软件开发公司	爱可信公司（Access Co., Ltd）
	埃森德公司（Ascender Corp.）
	Cooliris, Inc.
	eBay Inc.
	谷歌公司（Google Inc.）
	LivingImage Ltd
	Myriad
	Motoya Co., Ltd
	Nuance 通信公司（Nuance Communications, Inc.）
	NXP Software
	OMRON Software Co., Ltd
	PacketVideo (PV)
	SkyPop
	SONiVOX
	SVOX
	VisualOn Inc
市场推广公司	埃森哲（Accenture）
	Aplix Corp.
	播思国际控股公司（Borqs）
	L&T Infotech
	Noser Engineering Inc.
	Sasken Communication Technologies Limited

更多资源请访问稀酷客(www.ckook.com)

（续）

成员类型	公　司
市场推广公司	SQL Start International Inc.
	TAT The Astonishing Tribe AB
	Teleca AB
	Wind River Systems
	Wipro Technologies

2. Android 的特性

我们将在第 2 章讨论各种类型的 Android 终端设备，本节先介绍几个不同类型的大多数 Android 终端设备所具有的共同特性。

首先，Android 从一开始的设计目标就是支持与互联网的连接，不论是通过无线蜂窝网络，如 GSM/CDMA（Global System for Mobile Communications/ Code Division Multiple Access）或无线通信网络（Wi-Fi）。无论采取何种方式，支持与互联网连接是所有 Android 终端设备的核心功能。不错，许多 Android 终端设备的确是智能手机，因此可以支持拨打和接听电话、发送和接收短信以及其他无线蜂窝网络所能提供的服务。用户与终端设备实现交互通常通过一个触摸界面，但是很多终端设备也可以通过连接键盘或其他外围输入设备来支持终端设备与用户之间的交互。

Android 终端设备的第二个核心特性是可以从 Android 市场上下载应用软件并进行安装。对众多的用户来说，这是一个非常重要的功能，因为此功能可以使用户无限扩展终端设备提供的服务功能。这些下载的应用软件对于取证分析师来说也将成为一个丰富的信息来源。

Android 终端设备的最后一个核心特性是允许用户在终端设备上存储他们的个人数据。这自然是本书将要详细讨论的取证工作的基础。大多数 Android 终端设备自身都带有基于闪存技术（NAND）的内存以及用于存储大量数据的便携式外接 SD 卡。宏达国际电子股份有限公司近期销售的某些终端设备还提供虚拟影射功能，可以将一个具有独立 USB 识别号的虚拟 SD 卡影射到 NAND 上而呈现为一个独立的 SD 卡加以使用。此类虚拟 SD 卡通常按照微软的 FAT32 文件系统进行格式化。

3. 对无线蜂窝网络的支持

由于 Android 终端设备中的最大一类终端是智能手机，了解 Android 目前都支持哪些类型的无线蜂窝通信技术尤为重要。

作为第一批推出的 Android 终端设备，HTC DREAM100 或 T-Mobile G1 是一款 GSM 手机。GSM 是目前世界上使用最普遍并得到最广泛支持的无线蜂窝通信系统。在美国，提供基于 GSM 无线通信服务的运营商包括 AT&T 和 T-Mobile。GSM 系统使用一个 SIM（Subscriber Identity Module）模块或 USIM（Universal Subscriber Identity Module）模块对用户进行识别。

Android 支持的第二类无线蜂窝通信系统是 CDMA（Code Division Multiple Access）。

更多资源请访问稀酷客(www.ckook.com)

CDMA 是一个 CDMA 手机用来编码，以及发送通话、数据和控制信号的技术，在美国比在世界上其他国家或地区更加流行，目前使用的技术标准是 CDMA2000。使用此技术的主要无线通信服务运营商包括 Verizon 无线公司（Verizon Wireless）、Sprint、U.S. Cellular 和 Cricket Communications。

Android 支持的最后一类无线蜂窝通信系统是 iDEN，其主要的吸引力是对美国的 Sprint Nextel 公司提供的流行的 PTT（push-to-talk）功能进行支持。在美国，目前也就只有 Sprint Nextel（同时拥有 Boost Mobile）这一家主要的无线通信服务运营商支持 iDEN。iDEN 的开发者摩托罗拉公司也是支持 iDEN 的第一款 Android 手机 Motorola i1 的生产厂商。

1.2.2 谷歌的策略

显而易见，Android 是一个强大但耗费巨资开发出的移动终端设备平台。那么，为什么谷歌会心甘情愿地提供免费的 Android 呢？

答案始于谷歌给公司清楚定义的使命（公司信息：about，n.d.）：

> 谷歌的使命是将世界上所有的信息有效地进行组织，并使这些信息在任何地方都可以被访问到而且有用。

手机已经成为眼下最流行的电子消费产品，在全世界已经超过 40 亿部。因此，通过提供一个先进且免费的移动平台，谷歌相信它正在实现其使命中“（信息）在任何地方都可以被访问到”的部分。当然，这一切对谷歌来说还必须能够实现实际的商业利益。当更多的用户连接到互联网上，就有更多的用户需要搜索信息，最终将驱动作为谷歌主要收入的广告营业额的增长。在 2009 年 3 月的一次采访中，Andy Rubin 说道：

> 谷歌有一个很好的基于广告的商业盈利模式。同时，开放源码与基于广告的商业盈利模式存在着一个自然的联系。开放源码基本上是一个发布策略，它可以从根本上去除被接纳的障碍。
>
> (Krazit, n.d.)

对 Android 的批评之一是谷歌对设备制造商发布 Android 系统的管理方式不当造成了目前市场非常混乱，充斥着不同版本以及各有差异的 Android 系统。这与其他诸如 iPhone 的终端设备情况形成了鲜明的对比，因为苹果公司完全主导了 iPhone 硬件和操作系统的开发，同时又对第三方开发的应用软件具有显著影响。但是 Rubin 为谷歌的策略进行了辩解。在同一次的采访中，Rubin 就此问题进行了以下阐述（Krazit, n.d.）：

> 对终端设备进行控制的想法虽然很好，（但是）我们面对的是 40 亿部手机。当你具有对整个终端设备进行控制的能力，由一个厂商快速进行创新的能力将变得非常有限。你仍然可以激励创新，你可以严格要求企业，可以在技术上精益求

更多资源请访问稀酷客(www.ckook.com)

> 精，也可以引领整个手机行业，但是你无论如何是无法做好所有事情的。你可以总是在某些小的方面具有特色。但是我们在这里讨论的是如何摆脱那些小的方面，而按照用户期望的方式让用户去访问互联网。我们不想在某些方面造成互联网的改变，不想只拥有部分互联网，不想站在某个角落里使用一个落后版本的互联网，我们想“站在”整个互联网上。

因此，通过推出一款能够同时满足消费者需要以及设备制造商和无线通信服务提供商需要的移动操作系统，谷歌为其基于搜索和广告的盈利模式提供了一个绝佳的发布平台。

1. 应用

除了提供一个核心移动开发栈，Android 支持创新的另一个重要方式是给予进行应用开发的第三方为 Android 开发和发布应用程序的权利。截至 2011 年 1 月，已经有超过 20 万项 Android 应用程序被开发出来。虽然此策略看起来与苹果公司的策略相似，但是在方式方法上有着本质区别。苹果严格控制着推出的应用商店，任何开发者提交应用程序后需要经历一个漫长的审核过程，而苹果公司对每一款应用程序都具有最后的审批权利。提交的应用程序会因为诸多原因被拒绝，尤其是应用程序包含有任何苹果不喜欢的内容。谷歌的不同之处在于不对提交并发布在 Android 市场上的应用程序进行任何审核。虽然谷歌拥有禁止某个开发者、从 Android 市场上删除某个应用程序，甚至从某个 Android 终端设备上进行远程卸载某个应用程序的权利，但通常采用的方式是放手。

2. Nexus 手机

2010 年 1 月，谷歌正式推出了自有品牌的智能手机 Nexus One（即 N1，见图 1.1）。N1 由宏达电子（HTC）生产，从整体上看，给手机制造商开发和生产手机树立了一个典范。N1 采用高速处理器（1 GHz），运行最新版本的 Android 系统，有很多创新之处，例如使用了 2 个对讲话筒对周边的背景噪声进行综合并与用户的话音进行有效融合以达到技术上可以实现的最佳通话。

大多无线通信服务提供商销售的是被完全封装的手机，以迫使消费者必须与其签署两年的服务合同才能以一定的折扣价买到手机。N1 由谷歌公司自行销售，出售的手机本身没有被封装住，此举被许多市场分析师认为是对前者的直接挑战。N1 同时通过 T-Mobile 以折价方式出售，只要购买者同意与该公司签署一个长期服务合同。然而，最终的结果显示，N1 的销售没有预期的那样火爆，由此引起市场猜测谷歌没有能够成功地实现 N1（参见图 1.1）。

与此同时，谷歌公司试图展示应该如何按照他们认为最好的方式推出和维护 Android 手机。让很多人跌破眼镜的是，一年之后，谷歌又推出了由三星公司（Samsung）生产的智能手机 Nexus S，如图 1.2 所示。Nexus S 有一个诱人的功能是它运行的 Android 2.3 版本支持通过互联网拨打电话（Voice over IP，VoIP）。只要一个手机通过 Wi-Fi 或者是其他方式与互联网实现连接，此手机就可以使用任何现有的 VoIP 服务拨打或接听电话。在美国，此款手机只通过 Best Buy 商店进行销售，通信服务的提供商是 T-Mobile（参见图 1.2）。

更多资源请访问稀酷客(www.ckook.com)

图 1.1 宏达电子生产的谷歌 Nexus One（N1）智能手机

图 1.2 三星生产的谷歌 Nexus S 智能手机

人们不清楚的是谷歌推出 Nexus 系列智能手机的总体目标是什么。然而，能够清楚的是谷歌试图推出自有品牌智能手机的最终目的是给消费者在购买和使用手机上提供更多的选择及灵活性。

1.3 Linux、开源软件与取证

开源软件对数字取证领域已经产生了巨大的影响。以免费、开源方式推出的软件取证工具与不公开代码的解决方案相比具有以下优势：

- 能够通过熟读代码而了解所采用的具体步骤；
- 能够改进软件能力并将改进的部分与同行们共享；
- 价格优势。

虽然许多免费的开源软件没有商业化的技术支持模型，还是有一些专门提供技术支持的公司存在。例如，红帽公司（Red Hat）通过为 Linux 操作系统（Linux OS）提供技术支持和服务已经发展成为一个有一定规模的公司。此外，许多为免费的开源软件包提供维护的人员通常都能够随时联系到并及时回复询问的问题，而且由于他们直接对软件进行维护而能够提供非常优质的服务。

Linux 操作系统（Linux OS）是最知名的一款免费开源软件。Linux 不仅是 Android 的一个关键部分，还可以作为一个功能强大的取证工具加以使用。

Linux 发展历史简介

目前已经出版了很多关于 Linux 技术的书籍，因此只是通过有限的篇幅试图对这样一个重要的操作系统进行清楚的介绍非常困难。网络上也有非常多的有关 Linux 取证方面的

更多资源请访问稀酷客(www.ckook.com)

资源，其中很多涉及如何将 Linux 作为一个取证工具进行使用。

1991 年，当时还是芬兰赫尔辛基大学本科生的 Linux Torvalds 决定开发一个终端仿真器，用以与学校的计算机系统进行连接。他专为他的个人计算机（处理器为 Intel 386）编写了代码。但是当他完成了代码开发以后，他意识到他所编写的代码可以作为一个操作系统的基础。因此，他在 Usenet 新闻小组 Usenet newsgroup comp.os.minix 上发布了以下这条著名的信息（Torvalds, 1991）：

```
Path: gmdzi!unido!mcsun!news.funet.fi!hydra!klaava!torvalds
From: torva...@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: Free minix-like kernel sources for 386-AT
Keywords: 386, preliminary version
Message-ID: <1991Oct5.054106.4647@klaava.Helsinki.FI>
Date: 5 Oct 91 05:41:06 GMT
Organization: University of Helsinki
Lines: 55

Do you pine for the nice days of minix-1.1, when men were men and wrote
their own device drivers? Are you without a nice project and just dying
to cut your teeth on a OS you can try to modify for your needs? Are you
finding it frustrating when everything works on minix? No more all-
nighters to get a nifty program working? Then this post might be just
for you :-)

As I mentioned a month(?) ago, I'm working on a free version of a
minix-lookalike for AT-386 computers.  It has finally reached the stage
where it's even usable (though may not be depending on what you want),
and I am willing to put out the sources for wider distribution.  It is
just version 0.02 (+1 (very small) patch already), but I've successfully
run bash/gcc/gnu-make/gnu-sed/compress etc under it.

<snip>

I can (well, almost) hear you asking yourselves "why?".  Hurd will be
out in a year (or two, or next month, who knows), and I've already got
minix.  This is a program for hackers by a hacker.  I've enjouyed doing
it, and somebody might enjoy looking at it and even modifying it for
their own needs.  It is still small enough to understand, use and
modify, and I'm looking forward to any comments you might have.

<snip>
```

在读到上面这条信息后，很多热心的 Linux 使用者更加渴望去了解、修改、创建，甚至去摆弄复杂系统（通常被称为黑客心理）。这条消息被 Linux 发布在专为 Minix 操作系统（Minix OS）设置的新闻小组上，此操作系统当时是很多热心人试图用于测试和开发出一个类似 UNIX 操作系统（UNIX OS）的首选。然而，Linux 想冲破 Minix 操作系统在许可证方面的限制以及克服此系统在技术上的局限性。

在过去的 20 多年间，Linux 得到了很大的完善，并且已经在数量众多的个人计算机和服务器以及当前的移动终端设备上运行。目前已经有成千上万种功能强大的工具可以使用，并为许多编程语言完善了开发环境。目前也已经推出了非常多的版本，以适合不同的

更多资源请访问稀酷客(www.ckook.com)

需要，如服务器、工作站、笔记本电脑、嵌入式设备、安全套件等。

1. 在 VirtualBox 中安装 Linux

Linux 真的是一款非常棒的操作系统，我们会将它的强大功能贯穿于本书的所有举例中，便于读者领会和完成。本书的所有举例都在一个以虚拟机（Virtual Machine，VM）方式运行的 Ubuntu 10.10 64 位台式机上完成。虽然不同开发商提供的虚拟机软件相互兼容（包括运行在 Mac OS X 上的 VMWare Fusion），本书的重点是选择免费和开放的软件。因此，VirtualBox 这款既开放又可以免费得到的软件成为我们的首选。

> **说明**
>
> 在本书的举例中会大量使用 Ubuntu 虚拟机（Ubuntu VM）。以后的章节将会在已有的基础之上加入更多的工具和脚本程序。在此建议读者建立一个 Ubuntu 虚拟机，独自重复所有的举例，以得到最大的收获。所有的 Android 取证实例也将直接使用 Ubuntu 虚拟机。

VirtualBox 目前的拥有者是 Oracle 公司，通过 GPLv2 许可证发布。甲骨文公司的官方网站上专门有回答有关如何取得许可证的问题的专题。

读者可以通过 http://www.virtualbox.org/ 下载 VirtualBox，它可以安装在包括微软视窗操作系统（Windows）、苹果 Mac 操作系统（Mac OS X）和 Linux（2.4 及 2.6）操作系统在内的很多操作系统上。在安装了 VirtualBox 后，用户会看到 Oracle VM VirtualBox Manager（见图 1.3），随后就可以创建和管理虚拟机。

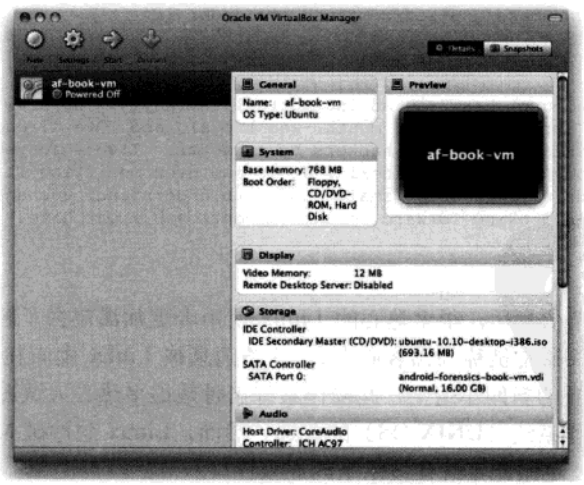


图 1.3 Oracle VM VirtualBox Manager for OS X

在创建了一个新的虚拟机后，用户必须确认磁盘上有足够的存储空间（建议至少有 20 GB）以及在内存中有尽可能多的可分配空间。对于 Android 系统，谷歌建议至少要有

更多资源请访问稀酷客(www.ckook.com)

1536 MB（1.5 GB）磁盘空间（获得 Android 源代码，n.d.）。

通过 VirtualBox Manager 图形化用户界面（Graphical User Interface，GUI）去设置一个新的虚拟机非常简单。然而，如果使用的是一台 64 位 Ubuntu Linux 工作站或服务器但是又没有运行应用软件的权限，则应该按照以下的步骤去创建、配置及运行一个新的虚拟机（VirtualBox 3.2.10）。

从一个 ssh 对话命令行开始，最好执行 screen 程序，这样即使失去了与服务器的连接，创建的虚拟机也能够保持在运行状态。然后，完成以下步骤：

```
mkdir -p ~/vbox
cd ~/vbox
wget http://releases.ubuntu.com/ubuntu-10.10 -desktop
-amd64.iso

VBoxManage createvm -name af-book-vm -ostype Ubuntu -register

VBoxManage modifyvm af-book-vm --memory 1536 --acpi on --boot1 dvd \
--nic1 bridged --usb on --usbehci on --vrdp on --vrdpport 3392 \
--clipboard bidirectional --pae on --hwvirtex on --hwvirtexexcl on
  --vtxvpid on \
--nestedpaging on --largepages on

VBoxManage modifyvm af-book-vm --bridgeadapter1 eth0

VBoxManage storagectl af-book-vm --name "IDE Controller" --add ide

VBoxManage createvdi --filename ~/vbox/af-book-vm.vdi \
--size 20000 --register

VBoxManage storageattach af-book-vm --storagectl "IDE Controller" \
--port 0 --device 0 --type hdd --medium ~/vbox/af-book-vm.vdi

VBoxManage storageattach af-book-vm --storagectl "IDE Controller" \
--port 1 --device 0 --type dvddrive --medium ~/vbox/ubuntu-10.10-desktop
-i386.iso

VBoxHeadless -startvm af-book-vm -p 3392 &

#need to eject DVD, the restart
VBoxManage storageattach af-book-vm --storagectl "IDE Controller" --port 1 \
--device 0 --type dvddrive --medium none

#restart the virtual machine
VBoxHeadless -startvm af-book-vm -p 3392
```

完成以上步骤后，虚拟机开始运行，并可以通过任何一个 RDP（Remote Desktop Protocol，远程桌面协议）浏览界面进行访问，如在微软的 Windows 上使用 Remote Desktop Connection，在 Linux 上使用 rdesktop，在苹果的 Mac 上使用 Microsoft' s Remote Desktop Connection Client for Mac。要顺利连接到虚拟机，需要与运行虚拟机服务器的 3392 端口（即 <服务器 IP 地址：3392>）进行连接。然后，按照提示命令进行操作，直至系统开机。

系统在运行时如遭遇关机或重启，本次的 VBoxHeadless 运行将结束。可以反复使用

更多资源请访问稀酷客(www.ckook.com)

下面的命令开启服务器备份。然后，RDP 就会被恢复并安装 openssh 服务器，用户进而可以使用 ssh 来取代效率较低的 RDP。

```
sudo apt-get install openssh-server
```

现在可以通过执行 ifconfig 命令，并通过找到 eth() 的“inet addr”而获得虚拟机的 IP 地址。然后就可以使用最心仪的 ssh 程序（如果使用 Windows，可以使用特别好的免费客户端 Putty）ssh 到虚拟机。

Sleuth 套件

本节介绍由 Brian Carrier 开发的一款叫做 TSK（The Sleuth Kit，TSK）的非常好用的开源取证套件。本书的很多举例中都会广泛地涉及 TSK。Brian 开发了 TSK 并继续进行维护，他为取证领域提供了大量而出色的服务。如果尚不熟悉 TSK 的话，可以访问网站 http://sleuthkit.org/ 并考虑使用此程序。此网站上有大量的关于 TSK 的信息以及许多取证方面的微博文章和书籍。如果想要深入领会本书中的举例，应该考虑使用下面的命令在 Linux 工作站上安装 TSK[⊖]：

```
sudo apt-get install sleuthkit
```

希望他人能够向 Brian 学习，踏着他的足迹，向取证社区提供类似的重要工具套件与服务。

2. 禁止 automount（自动挂载）

一个非常关键的要求是，用于进行取证的工作站不能启用 automount。这是因为，就像它的名称表达的那样，automount 会主动将所连接的终端设备上的任何文件系统自动挂载。在 Ubuntu 上，禁止 automount 必须由每一个用户自行选择完成。因此，如果工作站上存在有两个或以上的用户账号，则必须确认使用下面的命令对每一个用户账号进行以上的改变设置：

```
gconf-editor
```

然后进入 apps → nautilus → preferences，并确定“media_automount”和“media_automount_open”两个选项均没有被选取，如图 1.4 所示。

图 1.4　在 Ubuntu 上禁止 automount

⊖ 如果在安装 sleuthkit 过程中遇到问题，可先尝试运行 sudo apt-get update，以应对 apt 文件库被改变的情况。

更多资源请访问稀酷客(www.ckook.com)

此后就可以关闭 Gnome 配置编辑器（即 gconf editor）。此时 automount 已经被禁止。对于普通用户来说，也许需要做许多其他工作。但是，对于取证分析师来说，这一步是必须要求做的（同样的要求适用于硬件写阻止器）。

3. Linux 与取证——基本命令

在对取证用 Linux 工作站进行设置和配置之前，有必要对 Linux 与取证的关联性进行一下简要介绍。由于支持很多不同的文件系统、有大量的先进工具可供使用，又允许自由开发和编译源代码，Linux 工作站成为一个功能强大的取证分析工具。但是，由于很多取证分析师对 Linux 系统不太熟悉，因此，以下章节中将常用的 Linux 命令进行描述，描述的内容包括命令本身的介绍、通常用法以及一个或多个如何应用命令的实例。

（1）man

“man”命令将所提交的命令的使用手册显示在终端屏幕上。使用手册包含对命令的详细说明以及命令的用法（包括此命令所有的可用选项或“标记”）。

```
$  man [-k keywords] commands
```

下面的第一个实例用于显示 mount 命令的用户手册起始页，第二个实例用于查找所有包含字符串 grep 的用户手册，man 是一个功能强大的查找工具。

```
$ man mount
MOUNT(8)                    Linux Programmer's Manual                    MOUNT(8)

NAME
       mount - mount a filesystem

SYNOPSIS
       mount [-lhV]

       mount -a [-fFnrsvw] [-t vfstype] [-O optlist]

       mount [-fnrsvw] [-o option[,option]...]  device|dir

       mount [-fnrsvw] [-t vfstype] [-o options] device dir

DESCRIPTION
       All files accessible in a Unix system are arranged in one big tree, the
       file hierarchy, rooted at /.  These files can be spread out  over  sev-
       eral  devices.  The mount command serves to attach the filesystem found
       on some device to the big file tree. Conversely, the umount(8)  command
       will detach it again.

       The standard form of the mount command, is
<snip>

$ man -k grep
bzegrep (1)          - search possibly bzip2 compressed files for a regular
                       expression
bzfgrep (1)          - search possibly bzip2 compressed files for a regular
                       expression
bzgrep (1)           - search possibly bzip2 compressed files for a regular
                       expression
egrep (1)            - print lines matching a pattern
fgrep (1)            - print lines matching a pattern
```

更多资源请访问稀酷客(www.ckook.com)

```
git-grep (1)          - Print lines matching a pattern
grep (1)              - print lines matching a pattern
pgrep (1)             - look up or signal processes based on name and other
                        attributes
rgrep (1)             - print lines matching a pattern
xzegrep (1)           - search compressed files for a regular expression
xzfgrep (1)           - search compressed files for a regular expression
xzgrep (1)            - search compressed files for a regular expression
zegrep (1)            - search possibly compressed files for a regular
                        expression
zfgrep (1)            - search possibly compressed files for a regular
                        expression
zgrep (1)             - search possibly compressed files for a regular
                        expression
zipgrep (1)           - search files in a ZIP archive for lines matching a
                        pattern
```

（2）help

与 man 相似，help 命令将所提交的命令的有关信息显示在终端屏幕上，包括用法和实例。使用的方式为 - -help，也可以简单地使用 -h 或 -help。

```
$ mount --help

Usage: mount -V                : print version
       mount -h                : print this help
       mount                   : list mounted filesystems
       mount -l                : idem, including volume labels
So far the informational part. Next the mounting.
The command is 'mount [-t fstype] something somewhere'.
Details found in /etc/fstab may be omitted.
       mount -a [-t|-O] ...    : mount all stuff from /etc/fstab
       mount device            : mount device at the known place
       mount directory         : mount known device here
       mount -t type dev dir   : ordinary mount command
Note that one does not really mount a device, one mounts
a filesystem (of the given type) found on the device.
One can also mount an already visible directory tree elsewhere:
       mount --bind olddir newdir
or move a subtree:
       mount --move olddir newdir
One can change the type of mount containing the directory dir:
       mount --make-shared dir
       mount --make-slave dir
       mount --make-private dir
       mount --make-unbindable dir
One can change the type of all the mounts in a mount subtree
containing the directory dir:
       mount --make-rshared dir
       mount --make-rslave dir
       mount --make-rprivate dir
       mount --make-runbindable dir
A device can be given by name, say /dev/hda1 or /dev/cdrom,
or by label, using  -L label  or by uuid, using  -U uuid.
Other options: [-nfFrsvw] [-o options] [-p passwdfd].
For many more details, say man 8 mount.
```

（3）cd

此命令用于改变当前所在的目录（文件夹）。在 Linux 中，~ 是一个特殊字符，用于

更多资源请访问稀酷客(www.ckook.com)

代表当前正在使用的用户的主目录（文件夹）。例如，在 Linux 系统中，用户 ahoog 的主目录（文件夹）是 /home/ahoog。在文件系统的任何地方，都可以用 ~ 代表 /home/ahoog。这种表达方式在文档里非常好用。因此，在本书的整个篇幅中，我们都会使用 ~ 来表达。即使对于设置的其他用户名，此表达方式依然有效。

```
$ cd ~                          (进入当前用户的主目录，从文件系统的任何位置执行均有效)
$ cd ~/Desktop/Projects         (进入用户桌面（Desktop）上的 Projects 目录)
$ cd ..                         (进入当前所在目录的上一级目录，回到桌面（Desktop))
$ cd ../../                     (进入当前所在目录上两级的目录)
$ cd                            (也用于进入当前用户的主目录，从文件系统的任何位置执行均有效)
$ cd /                          (进入 root 文件系统目录，从文件系统的任何位置执行均有效)
```

（4）mkdir

除非特别指出，此命令用于在当前的目录（文件夹）中创建一个新的子目录（文件夹）。

```
$ mkdir android                          (在当前的目录（文件夹）中创建一个子目录 android)
$ mkdir -p ~/android/forensics/book      (创建一个子目录的全部路径，即使此路径先前不存在)
```

（5）rmdir/rm

根据所使用的不同选项或标记，此命令用来删除文件或目录（文件夹）。rmdir 命令只能用于删除内容为空的目录（文件夹）。如果目录（文件夹）里有文件存在，文件必须首先被删除，然后才能使用 rmdir 命令删除目录（文件夹）。rm 命令则可以用于删除文件和目录（文件夹），用户会在实际删除执行之前得到进行确认的提示。使用选项 -f 则可以越过或省略提示步骤，因此，建议用户谨慎使用，常用语就叫做 rm minus rf 或 rm -rf。

```
$ rmdir android                          (删除一个内容为空的目录（文件夹))
$ rmdir -p ~/android/forensics/book      (删除所给出路径下的一个目录)
$ rm -r android                          (删除指定的目录以及该目录中的所有内容)
$ rm -rf android                         (删除指定的目录以及该目录中的所有内容，并且在删除过
                                         程中不提供任何提示)
$ rm test.txt                            (删除指定的文件)
$ rm *.txt                               (删除当前目录中所有 .txt 类型的文件)
$ rm *                                   (删除当前目录中所有的文件)
```

（6）nano

此命令用于在终端上使用编辑器创建和修改文本文件。当需要创建一个文件时，直接输入此命令。

```
$ nano
```

输入 nano 后在终端屏幕上或 运行 ssh 期间打开文本编辑器，使用户可以输入内容，如图 1.5 所示。

更多资源请访问稀酷客(www.ckook.com)

图 1.5 使用 nano 创建文件

当完成内容输入以后，按 Control X 将终止编辑器，系统将提示用户是否需要将内容进行存储。此时，我们指定文件名为 newfile.txt。

如果需要修改一个已经存在的文件，在 nano 命令之后加上文件名，如果文件存在于不同的目录（文件夹）中，则使用完整的路径和文件名：

```
$ nano /etc/apt/sources.list
```

（7）ls

此命令用于罗列出一个目录（文件夹）中的文件以及子目录（文件夹）。当不使用任何选项时，ls 命令只罗列出当前所在目录（文件夹）中的文件 / 子目录。使用 -lh 选项将会出现很长的表，包含很多诸如访问权限、文件拥有者、文件大小以及日期和时间方面的信息。

```
ahoog@ubuntu:~/src$ ls
dc3dd-7.0.0          md5deep-3.7          viaextract-python
dc3dd-7.0.0.tar.gz   md5deep-3.7.tar.gz   yaffs2-old
download             viaextract_env       yaffs2-older.tgz

ahoog@ubuntu:~/src$ ls -lh
total 9.0M
drwxr-xr-x 10 ahoog ahoog 4.0K 2011-02-12 13:21 dc3dd-7.0.0
-rw-r--r--  1 ahoog ahoog 4.2M 2011-02-12 13:12 dc3dd-7.0.0.tar.gz
-rw-r--r--  1 ahoog ahoog 4.2M 2010-08-19 18:23 download
drwxr-xr-x  6 ahoog ahoog 4.0K 2011-02-18 07:21 md5deep-3.7
-rw-r--r--  1 ahoog ahoog 256K 2010-12-17 09:33 md5deep-3.7.tar.gz
drwxr-xr-x  6 ahoog ahoog 4.0K 2011-02-16 11:54 viaextract_env
drwxr-xr-x  6 ahoog ahoog 4.0K 2011-02-16 11:21 viaextract-python
drwxr-xr-x  8 ahoog ahoog 4.0K 2011-02-18 07:13 yaffs2-old
-rw-r--r--  1 ahoog ahoog 460K 2011-02-18 07:11 yaffs2-older.tgz
```

（8）tree ⊖

此命令用于显示出指定目录（文件夹）中子目录（子文件夹）的层次结构。如果没有使用任何参数，则当前所在的目录（文件夹）即为指定目录（文件夹）。在 Linux 中，当

⊖ 命令 tree 没有包含在 Ubuntu 正式发布版中，可以使用 sudo apt-get install tree 进行安装。

更多资源请访问稀酷客(www.ckook.com)

前所在目录（文件夹）用“.”表示，当前所在目录（文件夹）的上一层目录（文件夹）则用 .. 表示。在下面所举例的输出中，使用的是当前所在目录（文件夹），同时也正好是当前用户的主目录（文件夹）。用户可以使用 -L 标记来指定所要显示的子目录（文件夹）的层级数。在第一个实例中，只显示一层。在第二个实例中，则显示两层的所有目录（文件夹）和文件。值得提醒的是，关于任何命令的详细信息可以通过 man 页面（man tree）获得，也可以通过使用命令中的 help 参数（tree - -help）获得。

```
ahoog@ubuntu:~/src$ tree -L 1
.
├── dc3dd-7.0.0
├── dc3dd-7.0.0.tar.gz
├── download
├── md5deep-3.7
├── md5deep-3.7.tar.gz
├── viaextract_env
├── viaextract-python
├── yaffs2-old
└── yaffs2-older.tgz

5 directories, 4 files

ahoog@ubuntu:~/src$ tree -L 2 viaextract_env/
viaextract_env/
├── bin
│   ├── activate
│   ├── activate_this.py
│   ├── easy_install
│   ├── easy_install-2.6
│   ├── pilconvert.py
│   ├── pildriver.py
│   ├── pilfile.py
│   ├── pilfont.py
│   ├── pilprint.py
│   ├── pip
│   ├── pisa
│   ├── python
│   └── xhtml2pdf
├── include
│   └── python2.6 -> /usr/include/python2.6
├── lib
│   └── python2.6
└── src
    └── viaextract -> /home/ahoog/src/viaextract-python/trunk/viaextract/

7 directories, 13 files
```

（9）less

此命令用于一页一页地显示指定的文件。它通常与其他的命令一同使用来一页一页地显示输出结果。以下使用 less 命令来一屏一屏地在终端窗口显示文件 sanitize-csv.sh 的内容。当使用 less 命令显示内容后，以下是一些很重要的、值得记住的指令：

- h：读取 help 菜单；
- q：退出 help 菜单；

更多资源请访问稀酷客(www.ckook.com)

- spacebar：显示下一屏 / 一页；
- b：显示前一屏 / 一页；
- /：查找一个字符串；
- Enter：向下移动一行；
- y：向上移动一行。

还有更多的在使用 less 这个功能强大的命令的过程中可以使用的指令。因此，为了记得使用并读取 help 屏幕或 man 页面，也可以在互联网上查找丰富的帮助信息。

```
ahoog@ubuntu:~$ less sanitize-csv.sh
#!/bin/bash

# create a new directory to store the sanitized files
mkdir sanitized

#for each file ending with .csv
for f in *.csv
do
        #read the top 1 row of the file and save it to the sanitized directory
        #with the same filename followed by -1strowonly
        head -1 "$f" > sanitized/"$f"-1strowonly
done

#create a "tar gzip" archive of the file so it is easier share
tar czvf AFlogical-sanitized.tgz sanitized/*
sanitize-csv.sh (END)
```

（10）cat

此命令将一个文件的内容输出到屏幕，或者写入另外一个指定的文件（无法确保文件的原有格式）。

```
ahoog@ubuntu:~/Desktop$ cat android.txt
android forensics is so much fun.

This file contains unnecessary information used to display the workings of the
"cat" command.

The "cat" command can be used in conjunction with "less" in order to display
the contents of a
file one page at a time.
```

此命令还可以用于将多个文件合并进一个文件（通常称之为串接文件）。

```
$ cat file1.txt file2.txt file3.txt > final.txt
```

（11）find

此命令用于在一个目录（文件夹）层次结构中查找文件。以下命令将列出在指定用户的主目录（文件夹）中所有查找到的文件，同时包含这些文件的完整路径。

```
ahoog@ubuntu:~$ find ~
/home/ahoog
/home/ahoog/scalpel-1.60
/home/ahoog/scalpel-1.60/README
```

更多资源请访问稀酷客(www.ckook.com)

```
/home/ahoog/scalpel-1.60/dirname.h
/home/ahoog/scalpel-1.60/base_name.c
/home/ahoog/scalpel-1.60/dig.c
/home/ahoog/scalpel-1.60/prioque.h
/home/ahoog/scalpel-1.60/scalpel.o
/home/ahoog/scalpel-1.60/files.c
/home/ahoog/scalpel-1.60/helpers.o
<snip>
```

find 命令的输出同时也可以应用在另外一个命令中。以下实例是关于如何将 find 命令查找到的文件使用在 md5sum 命令中。此例使用了几个参数：

- find：命令；
- ~：在当前用户的主目录（文件夹）里查找文件；
- -type f：只列出普通文件（不列出目录（文件夹））；
- -exec：执行以下命令；
- sha256sum：计算一个文件的 sha256 散列值的应用包；
- {} \;：退出 shell 的方法与表示法。

```
ahoog@ubuntu:~$ find ~ -type f -exec sha256sum {} \;
1d4b6d9e7930e1bd186de982e21ad0d4dab92239920dea791dc25f2d0ffe92cb
/home/ahoog/scalpel-1.60/README
a771570f4b3ebfb212b39abaeff92cc7b4c116b920b726f6f6cabc78092b5d5b
/home/ahoog/scalpel-1.60/dirname.h
95d9d2243444b3e53eb2b449bd9052050bb4bb7d12d9d9bcaae2802de61feab1
/home/ahoog/scalpel-1.60/base_name.c
70babfb49c46a989f8d034b5e7438d5f51fe115a0ce61e6fdac6f0619d47d581
/home/ahoog/scalpel-1.60/dig.c
a5456e609810bca0730768a9d07b8c591246f1448c72b7d107d8b7ebd90f8fef
/home/ahoog/scalpel-1.60/prioque.h
f2364fd3caac10a297ec93123da6503b621cb65a98efaab6ca6fa5aeeccb6c32
/home/ahoog/scalpel-1.60/scalpel.o
d9c2351cc1fa8a8e4ea874005b756d89fa722de051d9e23dd2cd7b3988ce7122
/home/ahoog/scalpel-1.60/files.c
58c10d10cb629c95a81035fd73c553fa0c9ca2bbcacd184b9533f342285cf554
/home/ahoog/scalpel-1.60/helpers.o
<snip>
```

如果使用一个命令使用了对由很多文件得出的结果进行操作，也许会出问题。在这种情况下，应该应用一个叫做 xargs 的应用程序将 find 命令的结果通过管道的方式输出。

对于许多 Linux 命令来说，一个命令的输出结果也可以存储到一个文件中。举例来说，以上同一个命令的输出结果可以存储于用户主目录（文件夹）下的一个名称为 md5.txt 的文件中。

```
ahoog@ubuntu:~$ find~-type f -exec md5 {} \; > ~/md5.txt
```

将 find 命令使用 md5 共享程序执行的结果作为 find 命令的输入，再将此命令产生的输出重定向（存储）到用户主目录（文件夹）下的 md5.txt 文件中。

（12）chmod

此命令是“改变模式”的缩写，用于改变用户对文件或目录（文件夹）的访问权限。以下是几个应用实例。需要说明的是这些命令必须在 textfile.txt 所在目录（文件夹）为当

更多资源请访问稀酷客(www.ckook.com)

前目录（文件夹）时执行，或者需要明确给出通向这个文件的整个路径。

```
*Provides details on the file permissions for "textfile.txt"
   ahoog@ubuntu:~/Desktop$ ls -l textfile.txt
   -rw-r--r-- 1 ahoog ahoog 264 2011-03-01 12:17 textfile.txt

*Gives read, write, and execute permissions for the owner, and read and execute
permissions for group and world.
   ahoog@ubuntu:~/Desktop$ chmod 755 textfile.txt

   ahoog@ubuntu:~/Desktop$ ls -l textfile.txt
   -rwxr-xr-x 1 ahoog ahoog 264 2011-03-01 12:17 textfile.txt

*Gives read, write, and execute permissions for the owner, and execute
permissions for group and world.
   ahoog@ubuntu:~/Desktop$ chmod 711 textfile.txt

   ahoog@ubuntu:~/Desktop$ ls -l textfile.txt
   -rwx--x--x 1 ahoog ahoog 264 2011-03-01 12:17 textfile.txt

*Gives read, write, and execute permissions for the owner, and read-only
permissions for group and world.
   ahoog@ubuntu:~/Desktop$ chmod 744 textfile.txt

   ahoog@ubuntu:~/Desktop$ ls -l textfile.txt
   -rwxr--r-- 1 ahoog ahoog 264 2011-03-01 12:17 textfile.txt
```

chmod 也可用于对一组文件或一个目录（文件夹）进行操作。

```
$ chmod 755 *           (Changes permissions of all files in the current
                         directory)
$ chmod -R 444 Files/   (Changes permissions of the "Files" directory and all of
                         the files within it)
```

（13）chown

chown 命令用于改变一个指定文件或目录（文件夹）的拥有人或所隶属的组。在以下举例中，textfile.txt 文件的原来拥有人和所隶属的组均是 ahoog。使用 chown 命令将此文件的拥有人变为 root。执行此命令要求使用 sudo 指令。

```
ahoog@ubuntu:~/Desktop$ ls -l textfile.txt
-rwxr--r-- 1 ahoog ahoog 264 2011-03-01 12:17 textfile.txt

ahoog@ubuntu:~/Desktop$ sudo chown root textfile.txt
[sudo] password for ahoog:

ahoog@ubuntu:~/Desktop$ ls -l textfile.txt
-rwxr--r-- 1 root ahoog 264 2011-03-01 12:17 textfile.txt
```

（14）sudo

在任何命令之前添加 sudo 用于提升用户的访问权限，使用户可以作为超级用户或另外一个用户执行紧跟其后的命令。运行某些命令要求使用 sudo，如 apt-get（进行软件安装）、chown（如果目前不是拥有者的话，则改变为拥有者）、挂载、访问原态磁盘设备，以及很多其他命令（取决于具体被访问的文件）。使用 sudo 的方法就是在某个命令之前加

更多资源请访问稀酷客(www.ckook.com)

上 sodu，系统将会要求输入密码。然后系统会在 sodu 的日志中记录此命令。

```
ahoog@ubuntu:~$ sudo xxd /dev/sda1 | less
[sudo] password for ahoog:
```

（15）apt-get

apt-get 命令中的 apt 部分是高级包装工具（Advanced Packaging Tool）的缩写，允许用户进行软件安装及卸载、升级，甚至整个系统的升级。执行此命令要求 sodu 访问权限。

```
$ sudo apt-get install scalpel      (Installs scalpel software package)
[sudo] password for ahoog:

$ sudo apt-get remove scalpel       (Uninstalls scalpel software package)
[sudo] password for ahoog:

$ sudo apt-get update               (Updates the APT package index, which stores
                                     packages available for download)
[sudo] password for ahoog:

$ sudo apt-get upgrade -u           (Upgrades APT package versions, including
                                     security updates; should be run after
                                     update)
[sudo] password for ahoog:
```

（16）grep

此命令用于在一个文件或一组文件和目录（文件夹）中查找一个指定的词组，类似于使用 find 在一个打开的文档中查找一个词组。查找的词组区分大小写字母，因此，如果不确定一个字母是大写还是小写的话，就要使用 -i 选项（不区分大小写）。执行时间取决于文件的大小，使用 -i 选项会延长命令的完成时间。

命令的一般使用方法为：

```
$ grep keyword file.txt
```

以下是几个 grep 命令的应用实例：

```
$ grep Forensics androidBook.txt          (will search for "Forensics" in the
                                           specified file)
$ grep forensics androidBook.txt          (will search for "forensics", case
                                           insensitive, in the specified file)
$ grep "list of files" androidBook.txt    (will search the specified file for
                                           "list of files", case sensitive)
```

下面的命令用于在用户台式机上所有的文件中查找词组 unnecessary。查找的结果显示，这个词组在 textfile.txt 文件被找到，也存在于 WXP-PRO-OEM.iso 中。因为后一个文件是一个二进制文件，需要运用其他的技术来显示其内容：

```
ahoog@ubuntu:~/Desktop$ grep unnecessary *
android.txt:This file contains unnecessary information used to display the
workings of the "cat" command.
Binary file WXP-PRO-OEM.iso matches
```

更多资源请访问稀酷客(www.ckook.com)

在使用 Linux 进行取证调查和分析时，grep 命令是一个不可缺少的公用工具。

（17）管道与重定向（|和>）

管道字符|（在大多数键盘上位于回车键的上方）用于将一个命令的输出结果作为另外一个命令的参数进行进一步处理。输出结果也可以通过使用 > 重定向至另外一个文件。

以下命令将 cat file.txt 的结果通过管道的方式交由 less 命令执行，使用户可以在屏幕上逐页浏览文件的内容。

```
$ cat file.txt | less
```

以下命令首先使用 grep 命令在 ch1.xml 文件中查找字符串 android，然后从得出的结果中查找区分大小写的字符串 forensics。最终得到的查找结果再使用 less 命令一页一页地显示出来。

```
$ grep android ch1.xml | grep -i forensics | less
```

将执行一个命令得到的输出结果重定向也非常有用。以下命令得到文件 ch1.xml 的输出（使用 cat 命令），然后将其复制到用户台式机的一个文件名为 ~/Desktop/new-ch1.xml 的文件。

```
cat ch1.xml > ~/Desktop/new-ch1.xml
```

当针对一个特定的文件甚至针对整个磁盘空间映像执行 strings 命令时，重定向非常有用。我们将在第 7 章进一步讨论此应用。

1.4 Android 开放源码项目

Android 的开放策略自然导致 Android 源代码在 2008 年 10 月 28 日通过 Android 开放源码项目（Android Open Source Project，AOSP）进行发布。AOSP 网站发布的声明如下（得到 Android 源代码，n.d.）：

> 我们创造了 Android 是为了将我们在移动应用方面的经验进行共享。我们的目标就是始终要为移动通信服务提供商、原始设备制造商 [Original equipment manufacturers，OEM] 以及开发商提供一个使创新想法成为现实的开放平台。我们的目标是确保系统不存在单点失灵的情况，即允许一个厂家有能力限制或控制任何其他厂家所做的创新。我们的方案就是一个开放以及开放源码的平台。

谷歌的开发策略注重在开发一流的设备上（如 Nexus 系列智能手机），使谷歌能够吸纳一个新平台所带来的风险。同时，在 AOSP 开发下一版 Android 系统时，制造商们可以在他们生产的终端设备上使用当前最新版本的 Android 系统。

1.4.1 AOSP 使用许可

AOSP 通过使用以下两种软件使用许可进行管理：Apache 软件使用许可 2.0（Apache 2.0，即 ASL2.0）和 GNU 公共使用许可 v2(GPLv2)。GPLv2 是一个要求比较严格的使用许可，

更多资源请访问稀酷客(www.ckook.com)

它要求所有代码的贡献者使用同一使用许可免费发布所开发的新代码。由于谷歌认为此种方式将会影响 Android 的商业化的支持，GPLv2 主要涵盖在 Android 系统中的 Linux 内核部分。

Apache 2.0 使用许可更加容易被商业实体所接受，因为它附加的制约比较少，而且不要求公司开放他们开发的相关软件的源代码。以下是 AOSP 关于为什么要采用 Apache 2.0 使用许可的说明（Licenses, n.d.）：

> 有时会有人问我们为什么 Android 宁愿采用 Apache 软件使用许可 2.0。对于用户层（即非内核）软件，我们确实在事实上倾向于使用 ASL2.0（或者类似的使用许可，如 BSD、MIT 等），而不是其他方式的使用许可，如 LGPL。
>
> Android 就是倡导自由和选择。Android 的目标就是在移动世界中推崇开放，但是我们并不认为可以预测或主导安装我们的软件的人们如何去使用。因此，在鼓励每一个人去生产开放及可修改的终端设备的同时，我们并不认为我们有权利去强迫人们这么做。使用 LGPL 代码库将会强迫人们这样去做。
>
> 我们的具体顾虑如下：
>
> 1）LGPL（使用简单化的语言）要求满足以下的任意一个条件：将源代码发往具体应用；对源代码书面出价；与 LGPL 化的代码库实现动态链接，使用户可以通过手动方式对代码库进行升级及更新。由于 Android 软件通常以静态的系统镜像方式发布，遵守这些要求会制约 OEM 的设计。（例如，用户在只读闪存（read-only flash storage）上很难更新一个代码库。）
>
> 2）LGPL 要求允许用户进行修改以及为这些修改进行程序调试所做的反向工程。大多数终端设备制造商不愿意受到这些条件的约束。因此，为了尽可能地降低给这些公司带来的负担，我们尽可能在用户空间不使用 LGPL。
>
> 3）在历史上，对下游的设备开发商和应用开发商而言，LGPL 代码库是造成大量的遵守使用许可方面问题的根源。很不幸，让工程师们了解这些问题也非常困难并且是一个很漫长的过程。Android 成功的关键在于使终端设备制造商尽可能容易地遵守使用许可。
>
> 以上观点就是我们为什么宁愿对源代码使用 ASL2.0 的原因。但我们并不是要批评 LGPL 或其他使用许可。我们对此议题有很强的感受，以至于达到了最大限度地使用 ASL2.0。然而，我们喜欢所有的免费及开源的使用许可，同时也充分尊重他人的意见和选择。只是我们断定 ASL2.0 是达到我们所追求的目标的正确使用许可。

1.4.2　开发过程

AOSP 是一个非常成熟且复杂的开放源码项目，需要对全球众多开发商进行协调。因此，AOSP 为此项目的贡献者定义了一套必须严格遵守的角色和过程。以下是所定义的角色：

- 贡献者 / 开发者：为这个项目贡献代码的个人和企业；
- 验证者：对被改变的代码进行测试的个人；

更多资源请访问稀酷客(www.ckook.com)

- 审批者：具有开发经验、能够决定改变的代码是否可以被接受的个人；
- 项目负责人：通常为负责对 AOSP 项目进行管理的谷歌员工。

任何人随时都可以下载、编译及改进 AOSP 项目。开发过程如图 1.6 所示。

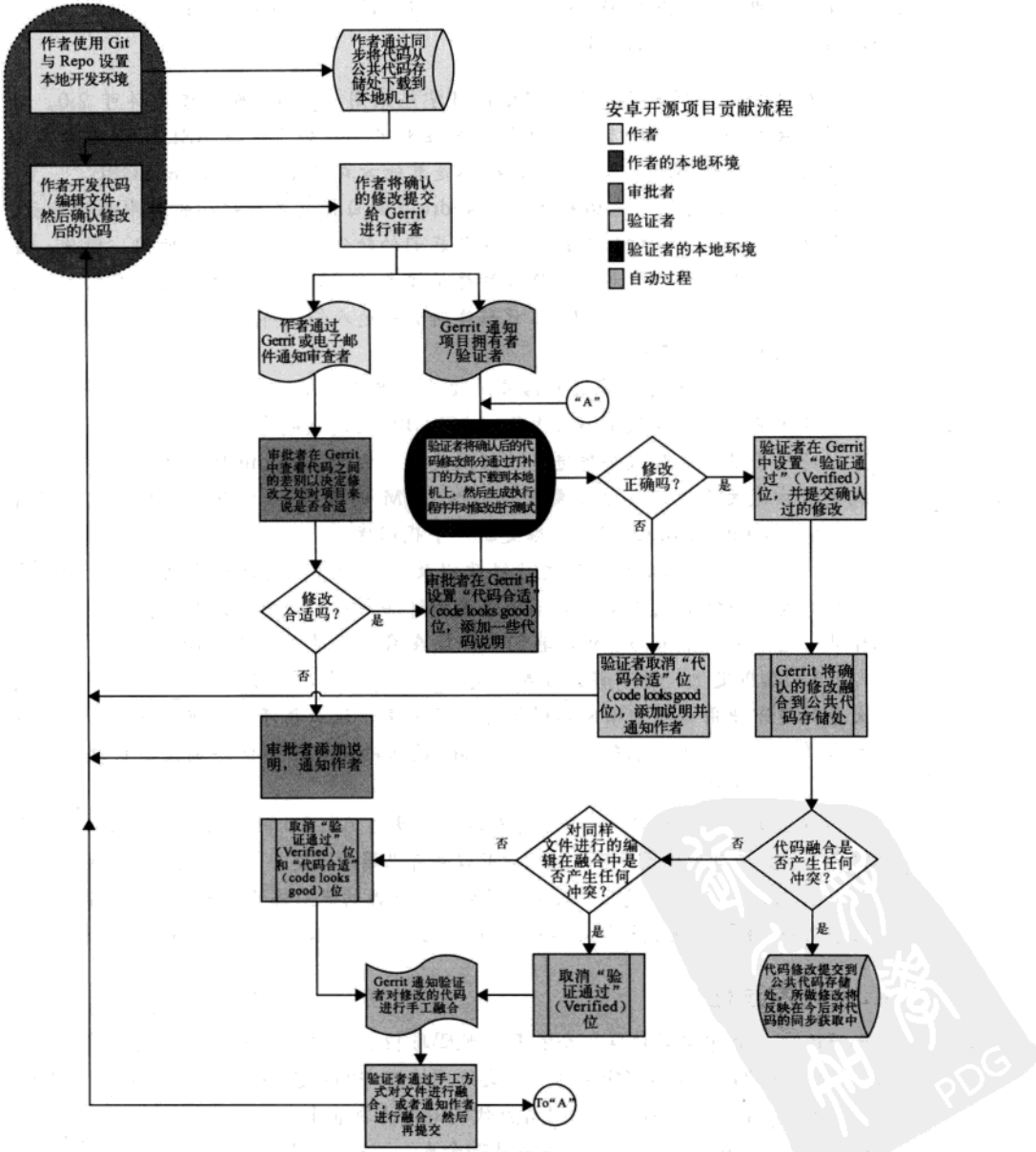


图 1.6　AOSP 开发流程

更多资源请访问稀酷客(www.ckook.com)

1.4.3　开源在取证中的价值

取证检查者不需要对 AOSP 项目直接做任何贡献。但是，下载此软件却有着巨大的价值。例如，当检查一个来自于 Android 1.5 终端设备的 Yet Another Flash File System2 (YAFFS2) 物理映像，使用字符串去提取 ASCII 内容时，发现词组 silly old name 反复出现。对于大多数文件系统来说，检查者需要推测词组 silly old name 是否适宜。但是，通过下载源代码，检查者可以快速找到此词组，获得相应代码并进行检查已获得其他信息。在以上例子中，当一个 YAFFS2 个体（比如一个文件）的物理头部被更新时，在某些情况下，名称字段被设置为 silly old name。

在 AOSP 的文件 kernel/fs/yaffs2/yaffs_guts.c 中，有一个叫做 yaffs_UpdateObjectHeader 的函数。在此函数的代码及函数头中的注释如下：

```
/* UpdateObjectHeader updates the header on NAND for an object.

 * If name is not NULL, then that new name is used.
 */
int yaffs_UpdateObjectHeader(yaffs_Object * in, const YCHAR * name, int force,
       int isShrink, int shadows)
```

定义变量 oldName，并赋予内容 silly old name：

```
yaffs_strcpy(oldName,_Y("silly old name"));
```

当先前的 ChunkId 值大于 0 时，正在被更新的客体头部中的名称字段被设置成 oldName：

```
if (prevChunkId > 0) {
 result = yaffs_ReadChunkWithTagsFromNAND(dev, prevChunkId,
     buffer, &oldTags);

 yaffs_VerifyObjectHeader(in,oh,&oldTags,0);

 memcpy(oldName, oh->name, sizeof(oh->name));
}
```

代码中还检查是否最初对函数调用时对客体使用了一个新的名字，如果函数调用没有使用新的名字，则使用数值 oldName（还是旧的名称 silly old name）：

```
if (name && *name) {
   memset(oh->name, 0, sizeof(oh->name));
   yaffs_strncpy(oh->name, name, YAFFS_MAX_NAME_LENGTH);
  } else if (prevChunkId>=0) {
   memcpy(oh->name, oldName, sizeof(oh->name));
  } else {
   memset(oh->name, 0, sizeof(oh->name));
  }
```

也许并不是每一个检查者都愿意去阅读和理解程序员写的代码（在这里是 C 语言），那么很显然，此类信息在取证检查中就变得非常有用。当然，还可以使用其他很多手段给检查者带来巨大的价值，比如如何给 SMS 消息加上时间戳、如何去实现地理标记等。

更多资源请访问稀酷客(www.ckook.com)

1.4.4 AOSP 下载和编译

希望上述的 YAFFS2 实例能够很清楚地展示出应用 Android 源代码的价值。本节重点讲述下载及编译最新版 AOSP 所需要的步骤。Android 2.2 及以前版本可以在 32 位机器上进行编译，但是最新版的 AOSP（Android 2.3）及后续版本要求使用 64 位的机器进行编译。

如果使用我们先前构建的 Ubuntu 虚拟机，我们就可以开始对 Ubuntu 的安装进行更新，以源代码为基础来构建 Android：

```
#add repository needed for sun-java6-jdk
sudo add-apt-repository "deb http://archive.canonical.com/ lucid partner"

#update installed packages
sudo apt-get update
sudo apt-get upgrade -u
sudo reboot
#install packages needed to build Android
sudo apt-get install git-core gnupg flex bison gperf libsdl1.2-dev libesd0-dev \
libwxgtk2.6-dev squashfs-tools build-essential zip curl libncurses5-dev \
zlib1g-dev sun-java6-jdk pngcrush g++-multilib lib32z1-dev lib32ncurses5-dev \
lib32readline5-dev gcc-4.3-multilib g++-4.3-multilib

#make directories and repo utility
mkdir -p ~/bin
mkdir -p ~/android
curl http://android.git.kernel.org/repo > ~/bin/repo
chmod 755 ~/bin/repo

#initialize Android git archive
cd ~/android
time ~/bin/repo init -u git://android.git.kernel.org/platform/manifest.git

#download source files
time ~/bin/repo sync

#build Android
cd ~/android
source build/envsetup.sh
lunch
time make
```

恭喜你，你已经完成了用源代码构建 Android（或者已经在开始构建，可能会需要一些时间才能完成）。

现在，如果要对任何 Android 内部进行更深入的了解，则可以对源代码进行搜索。表 1.5 为 Android 核心项目的清单，基本上对应于 Android 源代码的目录（文件夹）。AOSP 项目信息可以从以下网站获得：https://sites.google.com/a/android.com/opensource/projects，在那里有对每一个项目函数的简单介绍。

表 1.5 Android 核心项目

项 目	描 述
bionic	C 运行时：libc，libm，libdl，动态连接器
bootiosder/legacy	初始启动加载程序参考代码

更多资源请访问稀酷客(www.ckook.com)

（续）

项　目	描　述
build	构建系统
dalvik	Dalvik 虚拟机
development	高级程序开发及调试工具
frameworks/base	核心 Android 应用框架程序库
frameworks/policies/base	框架配置策略
hardware/libhardware	硬件抽象定义库
hardware/ril	无线接口层
kemel	Linux 内核
prebuilt	用于构建 Linux 和 Mac OS 系统的二进制代码
recovery	系统修复环境
system/bluetooth	蓝牙工具
system/core	最基本的初始启动环境
system/extrss	底层调试 / 检查工具
system/wian/ti	TI 1251 无线局域网驱动程序及工具

1.5　Android 平台的国际化

Android 平台广泛支持多种国际及区域语言设置。这不但使手机可以用各种语言显示菜单、网站以及用户界面中的其他部分，而且支持使用各种国际键盘格式进行输入。

1.5.1　Unicode

Android 能够支持众多语言的关键是将所有的字符都用 Unicode 格式进行编码和解码。Unicode 是一个支持超过 600 种不同语言的国际编码标准。（Languages and scripts, n.d.）。

> **提示：对楔形文字的支持**
>
> 对于好奇和追求新鲜感的读者来说，Unicode 的确支持楔形文字，虽然我们还必须等待 Android 的用户界面支持阿卡得语楔形文字那一天的到来。我们可以从网址 http://unicode.org/repos/cldr_tmp/trunk/diff/supplemental/languages_and_scripts.html 上获得关于 Unicode 目前对所有语言和脚本提供支持的信息。

1.5.2　输入键盘

Android 支持很多种不同的输入键盘，有时称为输入方式。例如，当运行一个 Android 虚拟终端设备（Android Virtual Device，AVD）时，仿真器允许你改变通过键盘输入所使用的语言，如图 1.7 所示。

对不同语言提供支持的功能 AVD 系统中得到实现。这一点对于取证调查有着非常重要的影响，考虑到某些数据可能会使用完全没有预料到的语言进行编码，取证分析师必须

更多资源请访问稀酷客(www.ckook.com)

时刻保持警惕。

以上在 AVD 已经实现的功能在很多实际的 Android 终端设备中也已经实现。例如，在美国由 Verizon Wireless 销售的 HTC Incredible 中，有一个叫做语言与键盘（Language and Keyboard）的设置。用户可以通过此设置为用户界面选择两种语言：英语和西班牙语。然后，在文本（Text）设置下，用户可以选择触摸输入（Touch Input）设置。之后，用户可以指定键盘类型（QWERTY、电话键盘（Phone Keypad）或简捷版 QWERTY（Compact QWERTY））、选择超过 20 种不同的国际键盘、具体的中文文字输入选项（繁体字或简体字）以及很多其他选项。最新版的 Android 系统（Gingerbread, 2.3）目前支持 57 种语言（Android 2.3 platform, n.d.）。

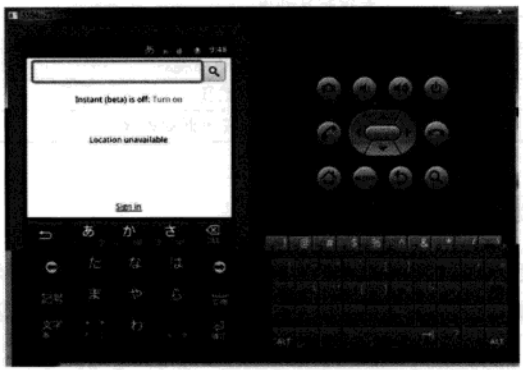


图 1.7　在 Android 虚拟终端设备上显示的中文数字键盘

最后，Android 支持用户安装的第三方键盘。有一个叫做 Swype 的日渐流行的键盘允许用户将放在键盘上的手指从一个字符连续地划向另外一个字符。键盘软件可以比较准确地确定用户敲入的字。此软件支持多种语言，是一个很好的即插即用式的键盘实例。

1.5.3　个性化分支

由于 Android 按照开放源码的方式进行发布，任何人（包括你，只要按照以上的步骤操作即可！）都可以下载并对 Android 源代码进行个性化修改。虽然很多人最终会将他们修改过的部分返回给谷歌并被融入 Android，但也有些人会开发出 Android 代码的个性化分支并发布个性化版本的 Android。

1. 售后市场固件

也许最具有典型代表性的 Android 个性化分支（也叫做 Mods、固件或 ROMs）产生于 Android 业余爱好者及追捧者社区。此社区由人数众多、背景复杂的个人组成，他们对 Android 的开发、实验或者叫做业余爱好抱有极大的热情。他们做的某些工作需要获得 Android 终端设备的超级用户访问权限、激活新的功能或者只是为了炫耀他们最新实现的个人化代码。此社区非常活跃、充满热情、富有创造力，是一个非常好的信息来源（以及

更多资源请访问稀酷客(www.ckook.com)

错误信息），他们也响应他人提出的帮助请求。严肃的 Android 研究人员绝对不应该无视此社区的存在。然而，大量的信息需要研究人员耗费更多的精力。

最流行的一个社区叫做 XDA 开发者，自称为是“最大的面向 Android 和 Windows 移动平台的智能手机追随者和开发者的互联网社区”（xda-developers, n.d.）。此社区的网站有超过 320 万注册用户，运行着一个让人印象深刻的论坛。

很多个性化的 Mods 通过 XDA 发布，开发者们通常在此互联网社区中异常活跃。最流行的售后市场固件也许要数 CyanogenMod。此固件基于 AOSP 开发，除了加入了新的功能，与由制造商和服务提供商直接提供的基于 Android 的 ROMs 相比，此固件还试图提高终端设备的性能和可靠性（CyanogenMod, n.d.）。目前 CyanogenMod 支持由 Commitva、戴尔、HTC、摩托罗拉、三星和 Viewsonic 6 个生产商生产的 17 种不同的智能手机和平板终端设备。这些售后市场固件在终端设备上允许超级用户访问权限，并正如将要在第 6 章中讨论到的那样，这是真正获取终端设备的关键。

2. OPhone OS

OPhone OS（Open Mobile Phone OS，开放移动电话 OS）是一款基于 Android 的、由一个叫做 Borqs 的中国软件开发公司开发的。OPhone OS 是由中国政府监管的，拥有超过 5 亿用户而成为全球最大的移动服务提供商的中国移动开发。2010 年 6 月，OPhone OS 2.0 在北京发布。虽然 Borqs 及 OPhone OS 在中国以外并不为人们所熟知，但是它在 Android 的生态系统中却扮演着一个日趋重要的角色。在中国，Borqs 公司首席执行官声明该公司只同一个服务提供商进行合作，也就是中国移动，因为在中国“你无法同时为两个主人服务”（China's OPhone, n.d.）。

因此，Borqs 将软件名称 OPhone OS 专门保留给中国移动使用。然而，根据该公司网站的消息，该公司还开发出另外一款叫做 Android+(Android+) 的软件（China's OPhone, n.d.）。在该公司的新闻发布中，Borqs 解释说戴尔 Aero（Dell Aero）是在 Android+ 的基础上开发出的一款软件，包含了一个类似苹果（Apple-esque）的用户界面。此软件目前由戴尔公司在巴西和墨西哥市场上发布。

3. 面向 iPhone 的 Android（及其他非 Android 终端设备）

目前最受争议的一款黑市固件也许就数能够在 iPhone 上运行的 Android。由于很多移动终端设备都使用 ARM 处理器，软件移植是可以实现的。这就能够使 Android 也运行在为其他诸如 Windows 移动、Symbian、iOS 等操作系统而设计的终端设备上。没有任何事情能够比拟于向苹果粉丝演示在漂亮的 iPhone 上也能运行 Android！

1.6　Android 市场

Android 市场是第三方开发者向任何拥有 Android 终端设备的用户发表他们所开发的应用的通道。Android 市场最早于 2008 年 8 月 28 日在谷歌开发者博客上发布，被描述为“一个开放的内容发布系统，帮助终端用户找寻、购买、下载及在他们的 Android 终端设

更多资源请访问稀酷客(www.ckook.com)

备上安装各种类型的内容”（Android developers blog, n.d.）。当 Android 市场在 2008 年 10 月首次推出时，它不支持收费的软件应用。然而，进入 2009 年，Android 市场开始在美国和英国支持对软件应用进行收费。截至 2011 年 1 月，Android 市场在 29 个国家支持对软件应用进行收费（Supported locations, n.d.）。其他一些国家，特别值得一提的是印度，可以使用 Android 市场，但是目前无法安装收费软件应用。

谷歌对 Android 市场所采用的松散的管理与苹果公司对苹果商店的严格管理形成了鲜明的对比。虽然 Android 市场面向用户有服务条例（Terms of Service）（Android Market terms, n.d.），以及面向开发者有 Android 市场开发者销售协议（Android Market Developer Distribution Agreement）（Android Market developer, n.d.），软件应用在市场上进行销售不需要经过审批流程。但是，谷歌相信，对软件应用的排名会将有问题的软件和价值低劣的软件清除出去。

开发者如果要通过 Android 市场推销软件，必须支付 25 美元注册费，并用私钥对软件应用进行签名以便能在市场中被唯一识别。当一个用户购买一个软件应用，支付款的 70% 归开发者，30% 归谷歌（有时也会涉及服务提供商）。最初，谷歌允许用户可以在购买后的 48 小时内退货。但是到 2010 年 12 月，谷歌将退货时间缩短到 15 分钟。

谷歌能够远程从 Android 市场甚至直接从 Android 终端设备上删除任何一款软件应用。这个远程应用删除特色（Remote Application Removal Feature）是 Android 的一个安全控制功能，能够将任何一个活跃的危险软件应用在大范围内迅速删除，以防止其影响更多的用户（Android developers blog: exercising, n.d.）。这项安全控制功能第一次使用是在 2010 年 6 月，删除了一款由信息安全研究者发布的概念验证型软件，但实为在终端设备上下载和安装另一款应用软件的软件（Mills, n.d.）。

按照 Android 的开放性精神，谷歌也不会禁止用户从他们所使用的手机的开发商的网站上直接下载并安装应用软件，同时也不会禁止形成另外一个与其竞争的软件应用市场。目前已经有几个可供用户选择的市场，但是相比之下，大多数的规模都很小。另外，已经有好几家大公司宣布或表示有意愿建立可供用户选择的软件应用商店，包括亚马逊（Amazon）、百思买（Best Buy）和 Verizon。

图 1.8 使用 HTC Incredible Android 2.2 智能手机访问 Android 市场

1.6.1 应用软件安装

当从 Android 市场上下载并安装一个软件应用程序，用户首先必须运行 Market 应用程序，并使用 Gmail 邮箱账号登录，如图 1.8 所示。登录账号使用户可以通过已经在谷歌设立的具有地址和信用卡信息的付款账号去购买应付费的软件应用。谷歌近期已经与

更多资源请访问稀酷客(www.ckook.com)

某些通信服务提供商达成了合作，允许用户可以将购买软件的应付款与通信服务提供商的每月账单一起进行支付，而不一定需要通过谷歌的支付信用卡付款。

谷歌的 Market 软件允许用户在 Android 市场中查找应用软件程序，并可以根据主题和流行程度进行浏览。一旦某个应用软件程序被选择，一个用户安装应用软件程序的专门页面将显示出来。这个专门页面同时还有此应用软件程序的具体信息，如软件描述、下载次数、平均排名、详细评价、其他相关的应用程序、开发者信息，最后还允许用户对此应用软件进行反馈，评估软件的适用性。

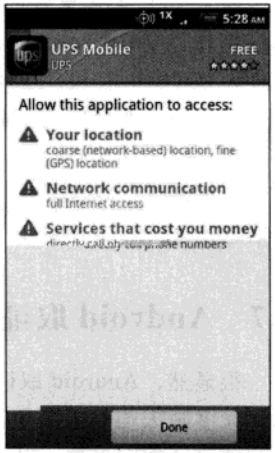


图 1.9　Android 应用软件访问权限

正如我们在后续章节中将要讨论的那样，访问权限是 Android 系统安全的核心部分。一旦某个用户决定安装一款应用软件，一个如图 1.9 所示的页面将显示并告诉用户此应用软件所要求的访问权限。此时，用户可以决定是否接受提出的访问权限要求并继续进行安装或者是决定放弃而返回到此前的显示屏幕。

在应用软件程序下载、安装后，结果会显示在终端设备顶部的通知栏中。然后，用户就可以在终端设备上运行此应用软件，或者通过应用软件程序列表对它进行访问。

如要删除一个应用软件程序，用户可以访问终端设备的设置（Settings）并进行应用（Application）设置。然后，用户就可以对应用软件进行管理（Manage）（如图 1.10 所示），屏幕会显示应用软件程序列表以及诸如已下载、正在运行、在 SD 卡上等各种特性。

通过选择某个应用软件程序，用户就可以在应用（Application Info）信息屏幕看到各种关于此应用软件程序的信息，并可以选择以下操作：强制停止（Force stop）、清理数据（Clear data）、清理缓存（Clear cashe）、卸载（Uninstall），如图 1.11 所示。

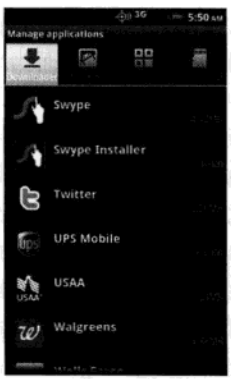


图 1.10　管理 Android 应用软件

图 1.11　应用软件信息

更多资源请访问稀酷客(www.ckook.com)

1.6.2 应用情况统计

Android 市场成长非常迅猛。在推出的 6 个月后，T-Mobile 首席技术官 Cole Brodman 评论说：用户需要更好地过滤程序来从 Android 市场近 2300 款应用软件程序中成功地选择应用软件（Lawson, n.d.）。截至 2011 年 1 月，Android 市场中已经有超过 20 万款应用软件，仅 2010 年 11 月就增加了 27 227 款（Android Market statistics, n.d.）。Android 市场中的应用软件程序的下载次数估计已超过 25 亿。

很显然，应用软件程序是安全和取证关注的一个核心领域。在第 4 章中，我们将详细探讨应用软件程序如何在 Android 终端设备中植入永久数据、信息存储于何处以及如何恢复信息并进行分析。

1.7 Android 取证

很显然，Android 取证非常必要。通常，智能手机也许是知道个人信息最多的电子产品。大多数人与智能手机几乎形影不离，包括睡觉。此终端设备内存有个人和公司信息，并存储包括文字消息、电子邮件、GPS 位置信息、照片、视频等大量的数据。人们总是对他们的智能手机比对其他人或设备更加诚实。为什么呢？因为人们感觉此终端设备是安全的，能够对他们不愿意与其他任何人共享的问题进行回答。曾经有不少取证分析师讥讽地说："谷歌是什么样，你就是什么样"，直接表达了人们对于他们所使用的智能手机的忠诚度。

面临的挑战

当然，值得做的事情都不会是很容易的，尤其是移动取证和 Android 取证具有非常多的必须克服的挑战。

数字取证的一个基本目标是预防取证检查师对被检查的终端设备进行修改。然而，移动电话缺少像传统计算机那样关闭磁盘、将其连接到一个写阻止器，并按照满足取证要求的方式进行映像的能力。与智能手机的任何交互都会在某种程度上改变此终端设备。正因为如此，取证检查师在检查移动终端设备时必须运用他们的判断，而当任何改变发生时，他们也必须解释清楚改变是如何发生的以及为什么他们做出了此选择。

有些取证检查师没有按照以上的方式去做，因而引发接连不断的争论。然而，因取证检查的需要而对目标计算机进行更改的技术已经由来已久。例如，在调查恶意软件攻击时，对内存的分析通常是必要的。同样，如果磁盘已加密，取证检查者必须在系统还在运行时就获得磁盘的映像，否则也许再也没有机会获取磁盘上的数据。再如，由于复杂的环境而必须始终与网络保持连接的系统，其具有代表性的是大公司的服务器。虽然每一个取证检查者都需要尽量不要对调查的终端设备造成任何改变，但是在移动领域中非常难做到。因此，如果不能对终端设备进行改变，剩下的选择就只能是不去检查此设备了。很显然，这种选择是不可取的，因为通过移动取证获得的证据是很多调查的重要组成部分，在

更多资源请访问稀酷客(www.ckook.com)

过去也帮助破解了很多犯罪案件。

给 Android 取证带来更多困难的是大量不同类型的终端设备、不同版本的 Android 系统以及应用。终端设备与 Android 版本的组合就已经至少达到数千种，而每一个终端设备与平台相结合又有其独特性。虽然对每一个 Android 终端设备进行逻辑分析是可以达到的，但是以上的大量的组合使对每一个 Android 终端设备进行完整的物理获取几乎是无法实现的。即使 Android 版本上一个非常微小的区别，对于关注度高的案件都需要大量的测试与验证。

1.8　本章小结

Android 是一个快速发展、特征鲜明以及令人兴奋的移动平台。特征、连结性以及流行程度相结合自然产生对 Android 取证的需求。在移动取证难度增长的同时，取证的价值也在增长。Android 的开放源码状态可以最大程度地帮助取证分析师获得所需要的对 Android 的了解，使 Android 成为一个理想的工作平台。

1.9　参考资料

Alliance, F. A. Q. (n.d.). *Open Handset Alliance*. Retrieved January 3, 2011, from http://www.openhandsetalliance.com/oha_faq.html.

Alliance Members. (n.d.). *Open Handset Alliance*. Retrieved March 9, 2011, from http://www.openhandsetalliance.com/oha_members.html.

Android developers blog: Android Market: a user-driven content distribution system. (n.d.). Retrieved January 9, 2011, from http://android-developers.blogspot.com/2008/08/android-market-user-driven-content.html.

Android developers blog: Exercising our remote application removal feature. (n.d.). Retrieved January 9, 2011, from http://android-developers.blogspot.com/2010/06/exercising-our-remote-application.html.

Android Market developer distribution agreement. (n.d.). Retrieved January 9, 2011, from http://www.android.com/us/developer-distribution-agreement.html.

Android Market terms of service. (n.d.). Retrieved January 9, 2011, from http://www.google.com/mobile/android/market-tos.html.

Android Market statistics from AndroLib, Androlib, Android applications and games directory. (n.d.). Retrieved January 9, 2011, from http://www.androlib.com/appstats.aspx.

Android 2.3 platform, & Android developers. (n.d.). Retrieved January 8, 2011, from http://developer.android.com/sdk/android-2.3.html#locs.

Bort, D. (n.d.). *Android is now available as open source*. Android Open Source Project. Retrieved January 3, 2011, from https://sites.google.com/a/android.com/opensource/posts/opensource.

China's OPhone to find its way to US as Android+. (n.d.). Retrieved January 8, 2011, from http://www.borqs.com/news.jsp.

Corporate information: About. (n.d.). Google. Retrieved January 4, 2011, from http://www.google.com/corporate/.

comScore reports November 2010 U.S. mobile subscriber market share. (n.d.). comScore. Inc. Retrieved January 9, 2011, from http://www.comscore.com/Press_Events/Press_Releases/2011/1/comScore_Reports_November_.

更多资源请访问稀酷客(www.ckook.com)

CyanogenMod, About the Rom, CyanogenMod. (n.d.). Retrieved January 8, 2011, from http://www.cyanogenmod.com/about.

Gartner says worldwide mobile phone sales grew 35 percent in third quarter 2010; smartphone sales increased 96 percent. (n.d.) Technology Research & Business Leader Insight. Gartner Inc. Retrieved March 9, 2011, from http://www.gartner.com/it/page.jsp?id=1466313.

Get Android source code, Android open source. (n.d.). Retrieved March 9, 2011, from http://source.android.com/source/download.html.

Google Investor: Google android activating 350,000 devices daily (data visualization video) "Top global smartphone platform." (n.d.). Retrieved March 9, 2011, from http://googinvestor.blogspot.com/2011/03/google-android-activations-350k-daily.html.

Krazit, T. (n.d.). *Google's Rubin: Android "a revolution." Digital Media—CNET News. Technology News—CNET News*. Retrieved January 5, 2011, from http://news.cnet.com/8301-1023_3-10245994-93.html.

Licenses. (n.d.). *Android open source*. Retrieved January 5, 2011, from http://source.android.com/source/licenses.html.

Languages and scripts. (n.d.). *Unicode Consortium*. http://unicode.org/repos/cldr-tmp/trunk/diff/supplemental/languages_and_scripts.html.

Lawson, S. (n.d.). *Android Market needs more filters, T-Mobile says*. ITworld, IT news, technology analysis and how-to resources. Retrieved January 9, 2011, from http://www.itworld.com/personal-tech/64481/android-market-needs-more-filters-t-mobile-says.

Mills, E. (n.d.). *Google remotely wipes apps off Android phones*. InSecurity Complex —CNET News. Technology News—CNET News. Retrieved January 9, 2011, from http://news.cnet.com/8301-27080_3-20008922-245.html.

Official Google Blog: Where's my Gphone? (n.d.). Retrieved January 2, 2011, from http://googleblog.blogspot.com/2007/11/wheres-my-gphone.html.

Supported locations for merchants—Android Market help. (n.d.). Retrieved January 9, 2011, from http://www.google.com/support/androidmarket/bin/answer.py?hl=en&answer=150324.

Torvalds, L. (1991, October 5). *Free minix-like kernel sources for 386-AT—comp.os.minix*. Google Groups. Retrieved March 3, 2011, from http://groups.google.com/group/comp.os.minix/msg/2194d253268b0a1b.

xda-developers. (n.d.). Retrieved January 8, 2011, from www.xda-developers.com/.

更多资源请访问稀酷客(www.ckook.com)

第 2 章

Android 硬件平台

本章要点

- 核心部件概述
- 终端设备类型概述
- 只读内存和启动加载程序
- 制造商
- 具体的终端设备

2.1 绪论

Android 的设计是为了与尽可能多的硬件实现兼容。此目标在很大程度上依赖于 Linux 内核来实现，因为 Linux 内核经过多年的发展已经能够做到支持大量不同的硬件。这是 Android 平台的一个重要特点，因为它可以让制造商有很大的自由度去设计、采购和集成最理想的 Android 终端设备的部件。此策略允许开发能力强大的、能够进行大量数据处理的双核终端设备以及入门级终端设备以满足入门级无线服务计划。虽然硬件兼容性对制造商和无线通信服务提供商以及最终对消费者都是一件好事，但硬件的多样性对取证分析师和安全工程师提出了很多挑战。了解 Android 的硬件部件、设备类型以及启动过程对于全面了解 Android 将会有很大帮助。

2.2 核心部件概述

Android 的设计就是要支持尽可能多的终端设备和制造商。因此，任何主要部件一经生产就可能已经过时。然而，在众多 Android 终端设备中，始终存在一些能够长期持续的部件值得在此讨论。以下讨论的部件是构成 Android 终端设备的核心部件。

2.2.1 中央处理器

中央处理器（CPU）对很多取证分析师来说是一个非常熟悉的术语，毋庸置疑，它在 Android 终端设备中扮演着重要的角色。CPU 负责执行操作系统（OS）和应用程序代码，并负责协调或控制系统中包括网络、存储、显示及输入设备等在内的其他核心部件。

从一开始，大多数（即使不是全部）的 Android 终端设备都使用 ARM 处理器作为其

更多资源请访问稀酷客(www.ckook.com)

CPU，对于移动平台来说已经具有足够的处理能力，而同时又是按照低能耗要求设计，是延长电池续航时间的关键。

然而，很多公司以及 Android 爱好者已经将 Android 移植到其他的平台。在公司方面，英特尔（Intel）已经将 Android 移植到它的 Atom 处理器上。同样，谷歌也已经将 Android 移植到基于 Android 的谷歌 TV 产品上。还有其他一些项目，比如 Androidx86（Android-x86, n.d.），也推出了可以运行在英特尔 x86 架构上的 Android 版本。所支持的部分平台包括很多 Eee PC 型号个人计算机以及联想 ThinkPad x61 平板电脑。

2.2.2 基带调制解调器 / 无线电

基带调制解调器以及无线电给 Android 终端设备提供连接到无线蜂窝网络的硬件和软件，使用终端设备进行语音通话和数据通信。

为了不使 CPU 被这一类的活动所占据，终端设备的设计者通常使用一个专用部件来管理复杂的无线蜂窝通信。因此，虽然 CPU 可以对终端设备内的所有活动进行直接控制，而实际上是基带调制解调器在对无线蜂窝通信进行管理。

在全书的讨论中，我们会交换地使用术语基带调制解调、基带调制解调器、无线电。虽然系统很复杂，以上的定义也可能存在某些细微差别，然而这些区别对于取证分析来说是微不足道的。

2.2.3 内存（RAM 与 NAND 闪存）

简单地说，Android 终端设备在某种意义上就是计算机，他们的运行需要各种类型的内存。目前所需要的两种内存是易失性内存（随机访问内存，RAM）和非易失性内存（NAND 闪存）。

系统使用 RAM 来装载、执行、控制操作系统、应用程序或数据的核心部分。RAM 属于易失性内存，也就是说没有电源供应就无法保住内存的状态。

然而，NAND 闪存内存（以下简称 NAND 闪存）是非易失的，因此，即使在终端设备断电后，里面的数据依然存在。NAND 闪存用来存储启动加载软件、操作系统和用户数据。因此，它成为任何取证调查的关键部件，就类似于磁盘之于笔记本电脑、台式电脑或服务器的取证调查那样重要。NAND 闪存还有一些独特的性质使它特别适合于移动终端设备，但同时也给编程人员带来很多挑战（常常也会给取证分析师带来难得的机会）。这些特性将在第 4 章中进行详细讨论。

从硬件角度来看，移动终端设备受到体积的制约。通常，RAM 和 NAND 闪存被放置在同一个叫做多芯片组装（MCP）的部件上。当对 Android 终端设备的部件进行检查时，一般来说，NAND 闪存和 RAM 会被组装成为一个 MCP。

图 2.1 显示的是一个由叫做 Hynix 的内存厂商生产的内存（用在戴尔 Streak 和其他 Android 终端设备上），它的总体架构是对 MCP 部件的一个很好的描述，它不但包含了 NAND 闪存和 RAM，而且包含了为适合不同终端设备的各种组装方式。

更多资源请访问稀酷客(www.ckook.com)

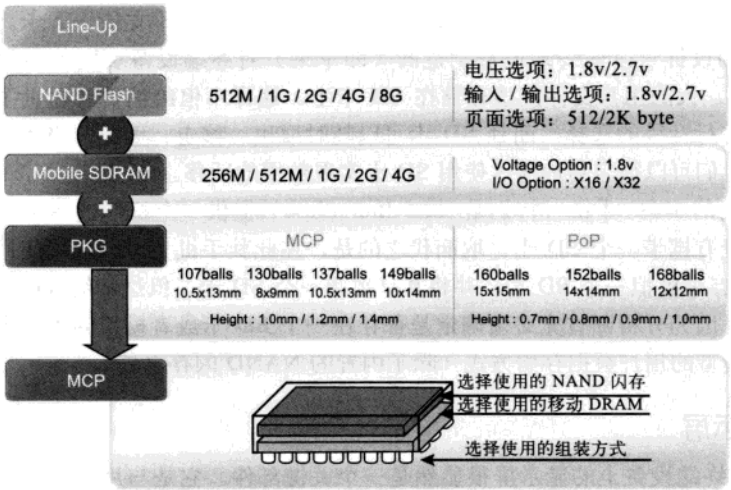


图 2.1　MCP 架构（Mobile Memory, n.d.）

2.2.4　GPS

毫无疑问，自从移动终端设备具备了无线蜂窝通信功能，最重要的创新之一就是将全球定位系统（GPS）集成到核心功能中。这个功能不但能够通过使用 GPS 卫星识别终端设备的位置，而且支持诸如点到点方向指示、基于位置的服务等应用，以及将来必将出现的更多有用的服务。

2.2.5　无线（Wi-Fi 与蓝牙）

在无线蜂窝网络之外，大多数终端设备还具有其他无线通信技术，比如用于高速通信连接的 Wi-Fi 技术以及用于与诸如耳机、键盘、打印机等外部终端设备进行连接的蓝牙技术。实际上，某些终端设备可能忽略与无线蜂窝网络的连接，这样不但可以降低终端设备的成本和复杂度，而且可以节省用户缴纳的月费。这些终端设备也许只为家居应用而设计（例如，家用电话或多媒体设备），或在没有 Wi-Fi 连接时可以离线使用（如平板电脑或电子阅读器（e-reader））。

2.2.6　安全数字卡

大多数 Android 终端设备在出厂时都有一个可取出的叫做 SD（安全数字卡）的存储卡。与在终端设备中的 NAND 闪存一样，SD 卡属于非易失类型存储，使用的是 NAND 闪存技术。然而，因为 SD 卡的设计就是为了达到移动目的，他们必须满足各种物理上和通信方面的规范，使他们能够在大多数终端设备上进行使用。

SD 卡也是大多数 Android 终端设备和流行的苹果 iPhone 在设计上的一个明显区别。

更多资源请访问稀酷客(www.ckook.com)

在设计上，iPhone 有一个内置的 4 ～ 32GB 的 NAND 闪存，不支持 SD 卡。虽然更加昂贵，但这样的设计可以使终端设备制造商（即苹果）对终端设备实施更多的控制。对于 Android 来说，大的用户文件应该存储在 SD 卡上。这样不但给消费者提供了一个廉价且容易对内存进行升级的选择，而且 SD 卡可以随时取出、移走。因此，当用户购买了一款新的手机，他们可以很容易地通过使用 SD 卡实现数据的迁移。

近期推出的 HTC 手机（特别是 HTC Incredible）提供一个标准的 SD 卡接口，但是手机出厂时没有携带一个 SD 卡。取而代之的是，在此款手机上可以设置使用一部分内置的 NAND 内存来模拟一个 SD 卡，并将其呈现为一个 SD 卡。但这却给取证分析带来了额外的复杂性，因为分析师首先必须确定是否存在一个 SD 卡或者模拟的 SD 卡，或者是否使用了其他类型的用户数据存储方式（除了内置的 NAND 闪存之外）。

2.2.7 显示屏

Android 终端设备上的显示屏很显然是一个关键部件。它是与用户实现交互的主要界面，不但通过显示屏与用户交互，而且对用户的触摸进行响应。显示屏背后的技术是目前紧张开发的核心。早期的交互技术包括一个液晶显示屏以及显示屏上的另外一层对用户输入进行检测。近期的改进包括更高的显示分辨率、更明亮的屏幕、更敏感和复杂的用户触摸交互以及低能量损耗。事实上，某些近期推出的 Android 智能手机，如三星的 Super AMOLED 技术，主要就是因为显示屏提供的功能而受到消费者的欢迎。

2.2.8 摄像机

照相机在智能手机上最初只是用来照相。这在当时是一个让人非常振奋的发展，此后，在该领域的发展又取得了重大的创新。大多数终端设备目前还可以支持摄像功能（有些还具有很高的清晰度）。当然，照相机的质量也有很大的提高，目前通常都集成了闪光灯。

近期，有些终端设备包含两个照相机。在终端设备背面的第一个照相机用于对外部进行照相和摄像。在终端设备正面的第二个照相机用于如视频会议这样的应用。

多数 Android 终端设备还把照相机功能和 GPS 功能相结合；从此，用户不但可以记录相片的日期和时间，还可以记录相片的 GPS 坐标。然后用户就可以很容易地上传并通过网络实现共享，也可以通过无线蜂窝服务提供商的多媒体消息服务（Multimedia Messaging Service，MMS）进行发送。

此领域中的一个有趣的发展是使用照相机读取条形码。某些专业化的服务利用照相机对条形码进行拍照，然后对数据进行分析。通过获得条形码可以查询产品介绍、确定最好价格或者可以让用户在餐厅的申请表上进行自动登记并根据相关记录进行分级。也许在不久的将来，这一类的应用还会允许用户对有意购买的产品进行支付。

早期的这一类应用当属谷歌推出的叫做 Goggles 的服务。用户可以对任何实物进行拍照，然后此应用试图对实物进行识别。谷歌提供的一个有趣的应用实例是观光旅客运用此应用对所处地标进行识别。

更多资源请访问稀酷客(www.ckook.com)

2.2.9　输入键盘

你也许认为在键盘上鲜有创新，然而，情况绝对不是这样。归功于触摸技术的进步，大多数 Android 终端设备都有一个在显示屏上的输入键盘。很多终端设备同时还有一个基于硬件的输入键盘。

软件输入键盘的强大功能使输入键盘的显示能够按照显示屏的方向进行自我调整（例如，如果显示屏被旋转 90 度，输入键盘也跟着旋转），并支持多种语言。

还有一些公司正在开发更加高效的文本输入技术。有一个叫做 Swype Inc. 的公司开发了一款输入键盘，用户不需要对每一个字符都选择一个键。取而代之的是，对每一个需要输入的字符，用户只要选择第一个字符，然后在输入键盘上来回移动手指（不需要离开输入键盘）去选择后续的字符，直到完成。Swype 输入键盘来决定可能的字并将其完成（或者提出建议）。这种方式已被证明是非常成功的，所以我们期待看到更多的 Swype 技术（或者是类似的创新）被集成到 Android 键盘中。

2.2.10　电池

电池的续航时间对智能手机用户一直是一个关注重点。用户也许非常喜欢他们的手机，但是却憎恨所使用的电池。人们使用终端设备以及使其功能强大的部件越多，对电池的消耗就越多。电池消耗的最小化已经受到极大的关注。然而，大多数用户仍然发现他们必须每天都对电池进行充电。

随着时间的流逝，硬件、软件和电池技术的进步将意味着对电池的充电不再需要那么频繁。目前在此领域中有一些非常有趣的研究倡议，比如对手机进行无线充电、利用人体的运动对手机连续进行充电，或者简单地去制造能量更强的电池。无论是何种改进都将受到消费者的欢迎。

对于取证分析师来说，首先需要牢记的是 SD 卡通常装配在电池的下面。因此，要获取 SD 卡（以确定终端设备的类型和身份），用户一般都需要首先将电池取出（因此会关闭终端设备）。这里有很多需要考虑的方面，我们将会在第 6 章中予以讨论。

2.2.11　通用串行总线

大多数 Android 终端设备支持通用串行总线（USB）接口，使其能够通过计算机进行访问。虽然用在不同的终端设备上的连接用电缆可能有所不同，但是一般来说，USB 接口能够使大多数操作系统实现与终端设备相连接。以下是 Android 终端设备上一些常见的对外接口：

1）充电专用：终端设备可以通过此 USB 接口进行充电；

2）磁盘接口：终端设备的部分（包括 SD 卡、模拟 SD 卡以及其他的磁盘接口）对操作系统来说可以被表现为一个大规模存储设备（Mass Storage Device），并且可以按照一个大规模存储设备进行访问；

3）厂商特设的接口：这一类接口包括定制同步协议、用以进行软件安装的模拟 CD 只读内存（ROM）驱动程序以及用于共享电话上与互联网实现连接的特殊接口；

更多资源请访问稀酷客(www.ckook.com)

4）Android 程序调试桥（ADB）：是一个让用户访问终端设备上 shell 命令行以及其他最新功能的接口。

我们将在第 3 章中讨论磁盘接口和 ADB 接口，这两类接口对于 Android 终端设备进行取证调查来说都是非常关键的部件。

2.2.12 加速仪 / 陀螺仪

根据终端设备如何被手持或被旋转，Android 能够检测并相应改变用户界面。这个功能通常是通过使用一个加速仪去检测终端设备被加速（或被定位）的尺寸和方向。通常，这个结果用于将显示屏在横向显示还是竖向显示之间进行变化。

最新的 Android 版本（本书撰写时为 2.3）支持使用比加速仪更加敏感及更加成熟的陀螺仪。陀螺仪能够对终端设备的移动进行更好的响应以及更精确的测量，这是高级游戏开发的关键。

2.2.13 音响 / 麦克风

最后，没有收听和生成音乐的功能，智能手机或平板电脑将不会那么诱人。像其他部件一样，音响和麦克风技术随着产品更新换代继续成熟。举例来说，某些 Android 终端设备拥有两到三个麦克风，再与 Android 软件相结合，具备了检测并抵消背景噪声的能力，以提供更好的音响质量。也许作为本世纪最令人震惊的技术发展，对讲电话已经发展到可以被实际应用到真实的交谈中。

2.3 终端设备类型概述

以上一节所介绍的核心部件为基础，设计者们已经创造出了大量不同类型的终端设备。当 T-Mobile G1（HTC Dream 100）在 2008 年被推向市场时，对 Android 终端设备及类型很容易进行跟踪。那时只有 G1，唯一的终端设备类型就是智能手机。当然，当时也有很多关于新类型终端设备的博客帖子，但也都是一些推测。

然而，到 2010 年年底，Android 终端设备的数量不仅大幅增长，而且种类也大幅增多。很多网站试图跟踪 Android 终端设备的增长，但是，大多数的数据都不完整。PDAdb.net 是一个还不错的可以用于了解新的 Android 终端设备的参考，跟踪并提供很多关于当前及将来的终端设备的信息。当前，超过 300 种运行 Android 的终端设备被跟踪，可以通过 PDAdb.net 的主页面进行搜索（Main Page, n.d.）。

智能手机和平板电脑目前是主要的 Android 终端设备，然而成长快速的是超薄型计算机（我们将其称为上网本电脑）以及电子阅读器（e-readers）。在技术创新方面，谷歌 TV（Google TV）终端设备（运行 Android）正在推向市场，很多汽车制造商已经宣布将在他们的媒体和导航系统中运行 Android。最后，还有很多的终端设备可以统一归为“其他”这个类，这一类终端设备也许目前还不够成熟，但将来肯定会成为主流。这一类终端设备

更多资源请访问稀酷客(www.ckook.com)

包括家用电器、游戏机、GPS 接收器、家居电话和音响设备、相框、打印机等。下一节将对其中某些类型的终端设备进行详细介绍。

2.3.1　智能手机

智能手机是最流行的 Android 终端设备。它们包含了上面介绍到的几乎所有的部件，也是知名度最高的终端设备。截至 2010 年 10 月，Android 智能手机占有美国智能手机市场 22% 的份额（NielsenWire, n.d.），并继续快速增长。Android 已经被广泛接受，将超越 iPhone 并最终成为最流行的智能手机平台。

2.3.2　平板电脑

虽然平板计算机已经存在了好几十年，看来硬件、软件、移动网络以及应用对其产生的影响也许最终会催生一个有活力的市场。虽然目前市场上已经有数款 Android 平板电脑，最新、广泛宣传的平板电脑当属三星 Galaxy Tab™。此平板电脑拥有一个 7 英寸显示屏，几乎包含了 Android 智能手机的所有部件，是一个放大版。虽然平板电脑可以支持无线通信（正如 Galaxy Tab 那样），却通常局限于数据和短信通信（MMS），不支持无线语音通信。然而，随着语音通信与数据通信的进一步融合，我们相信平板电脑很快就会支持互联网电话（VoIP）以及视频电话。

2.3.3　上网本电脑

上网本电脑是移动性更强、能耗低的笔记本电脑，是 Android 很好的候选设备。特别需要提出的是，Android 与谷歌的另外一个叫做 Chromium OS 有所不同，后者是“一个开放源码项目，目的是开发一款为花费大量时间在互联网上的用户提供快速、简单及安全的用户体验的操作系统”（Chromium OS, n.d.）。Android 在先开发，也比 Chromium OS 更加成熟。

除了拥有一个完整的硬件输入键盘以及较大的可旋转的显示屏以外，很多 Android 上网本电脑与平板电脑具有很大的相似性。通常，上网本电脑的主要数据存储介质为 NAND 闪存。然而，没有任何理由能够说明为什么不能使用更传统的磁盘驱动程序。

2.3.4　谷歌 TV

像过去很多其他的公司一样，谷歌也正在试图缩小用户观看播放的电视节目与互联网内容之间的差距。此类终端设备涵盖面从完全使用 Android 的电视机到连接到现有电视机的机顶盒。但是关键就是要使用 Android 作为基础的操作系统、将互联网与电视节目进行集成以及给开发商提供一个为新的传输媒体创造应用的框架。

2.3.5　车辆（内置）

还有一个令人期待的领域，是将 Android 集成进汽车，并作为驾驶 / 警示显示或娱乐系统的一部分。迄今为止，这一类的系统由汽车制造商各自开发，造成系统之间在功能、

更多资源请访问稀酷客(www.ckook.com)

性能、稳定性和有效性方面具有很大差异。如果汽车制造商将持续发展的 Android 操作系统的功能完全进行集成，他们就可以将主要精力放在用户体验方面，而不是放在基础的功能模块方面。用户也将从不同的汽车上和 Android 终端设备上获得一致的体验。开发商可以专心开发针对汽车需要的应用，并具有更大的销售市场。最后，还有很多其他对此感兴趣的商家，如保险公司、律师、研究机构、取证分析师等，可以从很多方面对这些系统里的信息进行分析。

荣威 350 是正在生产的第一款使用 Android 的汽车，由上海汽车工业公司在中国生产和销售。此外，很多美国的汽车生产商也已经宣布支持使用 Android，应用范围从与智能手机实现连接到完全将 Android 操作系统集成进他们所生产的汽车。

2.3.6 GPS 设备

如前所示，大多数 Android 终端设备都有内置的 GPS 硬件。当 GPS 首次推向消费者时，对其进行管理的操作系统是由生产商自行开发的。虽然很多制造商仍然坚持这种方式，但有一些已经走向 Android 操作系统。因此，取证分析师将会遇到运行 Android 的专用 GPS 终端设备。

2.3.7 其他终端设备

在数量上大幅增长的 Android 终端设备中，很多设备一经出世就面临被淘汰。Android 对于制造商来说是一个无法放弃的大好选择。Android 操作系统免费、成熟，而且允许进行个性化开发。Android 还通过自身以及第三方为应用开发提供操作机制。因此，越来越多的制造商摒弃昂贵的操作系统开发、维护和支持而转向在 Android 基础上进行开发。以下是部分其他类的 Android 终端设备：

- 家电，如洗衣机和微波炉；
- 电子阅读机，如 Barnes and Noble 图书经销商推出的 Nook；
- 多媒体播放机；
- 办公设备，如复印机；
- 家用电话、音响和视频（如相框）终端设备；
- 专用游戏机；
- 打印机。

正如我们所见，制造商有很多可以充分使用 Android 的方法，肯定也会使取证分析师的工作变得更加有趣（如果目前还不是这样的话）。

2.4 只读内存及启动加载程序

像其他计算机一样，Android 终端设备有一个比较标准的启动过程，使终端设备向内存里装载需要的固件、操作系统以及用户数据，对设备的完全运行予以支持。虽然启动过程已经定义得非常清楚，固件和 ROM 仍然随着制造商和终端设备的不同而有所不同。本

更多资源请访问稀酷客(www.ckook.com)

节的目的就是对 Android 的启动过程进行总体概述，因为本书后续章节中介绍的技术会与终端设备的不同层次相呼应。此概述只能在总体层面，因为对 Android 或 Linux 启动过程的深入介绍本身就可以形成一本完整的书籍。

本节内的大多数信息来源于一个题目为“Android 从开启电源开始的启动过程”（The Android boot process from power on）的帖子，作者是 Android 专业技能中心（Android Competence Center）的 Mattias BjoÖrnheden（BjoÖrnheden, n.d.）。在这个帖子中，Mattias 指出了 Android 启动过程需要经历的 7 个关键步骤：

1）开启电源和片上启动 ROM 代码执行；
2）启动加载程序；
3）Linux 内核；
4）init 进程；
5）Zygote and Dalvik；
6）系统服务器；
7）启动完成。

以下对上面罗列的步骤进行详细介绍。

2.4.1　开启电源和片上 ROM 代码执行

当一个 Android 终端设备的电源被开启，一段特殊的与 CPU 相配对的启动 ROM 代码被执行。1）对终端设备硬件进行初始化；2）找到启动媒介。ROM 代码与终端设备所使用的 CPU 相对应。启动过程的这一步与用于启动计算机的输入 – 输出系统相似。

例如，在硬件黑客社区中十分流行的一款叫做德州仪器（Texas Instrument）OMAP3530 ARM-compatible 计算机中的 CPU 公开发行的技术参考手册（Technical Reference Manual）就有 3444 页（Public Version of OMAP35xx, 2010）。虽然并不是人人都可以读懂此技术手册，此手册提供了非常多的关于 CPU 如何初始化并加载操作系统的细节和内幕。在第 3373 页，此手册提供了一个详细的总体启动程序的流程图。启动整个过程的 ROM 代码被硬式编码在地址 0x00014000 上，结果是当终端设备的电源开启时，CPU 可以准确地找到启动 ROM，并开始执行整个启动流程。

一旦终端设备的硬件初始化完成，ROM 代码通过扫描发现启动媒介（由 Android 终端设备保存在 NAND 闪存上），并将初始化启动加载程序（initial boot loader）复制到可读写内存（RAM）。然后，程序的执行将从启动 ROM 跳至内存中刚加载的代码，如图 2.2 所示。

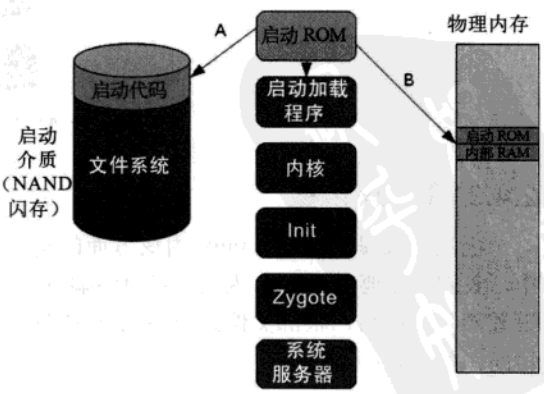


图 2.2　开机与片上启动 ROM 代码

更多资源请访问稀酷客(www.ckook.com)

2.4.2 启动加载程序（初始程序加载 / 第二阶段程序加载）

从启动媒介复制到可读写内存（RAM）的启动加载程序被执行。这一步骤与启动计算机中的 Windows、Mac 和 Linux 的启动加载程序（Boot Loader）相似。典型的启动加载程序，如 Linux 中的 GRUB，允许用户选择希望启动的操作系统，并进行相应加载。

对于 Android 终端设备，有两个独立的阶段：IPL（初始程序加载）和 SPL（第二阶段程序加载程序）。IPL 负责检测和建立起外部的 RAM，这个步骤对于启动和操作终端设备非常重要。一旦外部的 RAM 建立完成，IPL 会将第二阶段 SPL 复制到 RAM 中，并将执行转移至 SPL。

SPL 不但负责加载 Android 操作系统，而且负责对可选择的其他诸如快速启动、恢复、用于更新及调试程序或为终端设备服务的启动模式提供访问。SPL 通常由制造商提供。然而，Android 社区积极地开发了自己的 SPL（以及其他的特殊映像），以提供更多的特色和功能。在一个典型的启动场景中，SPL 将对硬件部件进行初始化，如时钟、控制台、显示器、输入键盘、基带调制解调器（baseband modem）、文件系统、虚拟内存以及其他操作终端设备所需要的功能。

然后 SPL 在启动媒介上找到 Linux 内核，并将其复制到可读写内存、加载启动参数，最后将程序执行转移到内核。图 2.3 是这个过程的图解。

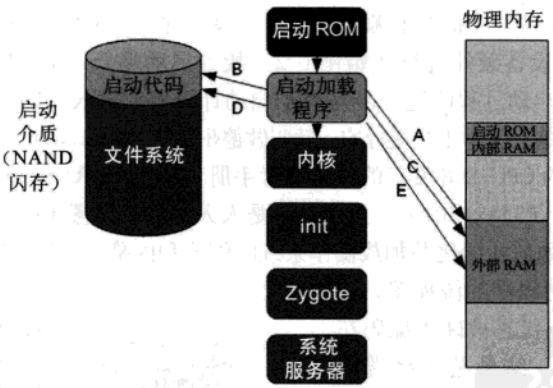


图 2.3 启动加载程序

2.4.3 Linux 内核

已经有大量的关于 Linux 内核方面的资料，而且很多都可以通过网上获取。在此书中，我们只需要对 Linux 内核已经对终端设备实施控制给予认可即可。在终端设备完成了设置额外功能后，根部文件系统被从 NAND 闪存中读出，使系统和用户可以对数据进行访问，如图 2.4 所示。

更多资源请访问稀酷客(www.ckook.com)

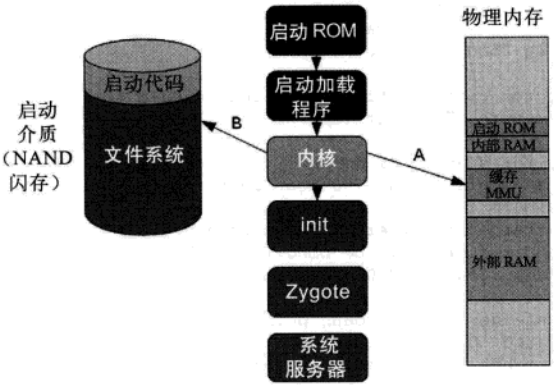


图 2.4　Linux 内核

2.4.4　init 进程

一旦内核可以对系统分区进行访问，它将处理 init 脚本，以启动关键的系统和用户进程。它与传统的 Linux 终端设备中的 /etc/init.d 脚本相似。在 Android 中，init.rc 通常处于根文件系统中，为内核提供如何启动核心服务的细节。

在运行 Android2.2 的 HTC Incredible 中，init.rc 和 init.inc.rc 文件超过 650 行，为终端设备的设置提供了大量的重要资料。以下是 /init.rc 文件的部分内容：

```
on init

sysclktz 0

loglevel 3

# setup the global environment
    export PATH /sbin:/system/sbin:/system/bin:/system/xbin
    export LD_LIBRARY_PATH /system/lib
    export ANDROID_BOOTLOGO 1
    export ANDROID_ROOT /system
    export ANDROID_ASSETS /system/app
    export ANDROID_DATA /data
    export EXTERNAL_STORAGE /mnt/sdcard
    export ASEC_MOUNTPOINT /mnt/asec

# Backward compatibility
    symlink /system/etc /etc
    symlink /sys/kernel/debug /d

# create mountpoints
    mkdir /mnt 0775 root system
    mkdir /mnt/sdcard 0000 system system

# Backwards Compat - XXX: Going away in G*
    symlink /mnt/sdcard /sdcard

    mkdir /system
```

更多资源请访问稀酷客(www.ckook.com)

```
    mkdir /data 0771 system system
    mkdir /cache 0770 system cache
    mkdir /config 0500 root root

    # Directory for putting things only root should see.
    mkdir /mnt/secure 0700 root root

    # Directory for staging bindmounts
    mkdir /mnt/secure/staging 0700 root root

    # Directory-target for where the secure container
    # imagefile directory will be bind-mounted
    mkdir /mnt/secure/asec  0700 root root

    # Secure container public mount points.
    mkdir /mnt/asec  0700 root system
    mount tmpfs tmpfs /mnt/asec mode=0755,gid=1000

    mount rootfs rootfs / ro remount

    write /proc/sys/kernel/panic_on_oops 1
    write /proc/sys/kernel/hung_task_timeout_secs 0
    write /proc/cpu/alignment 4
    write /proc/sys/kernel/sched_latency_ns 5000000
    write /proc/sys/kernel/sched_wakeup_granularity_ns 100000
    write /proc/sys/kernel/sched_min_granularity_ns 100000
    write /proc/sys/kernel/sched_compat_yield 1
    write /proc/sys/kernel/sched_child_runs_first 0

# mount mtd partitions
    # Mount /system rw first to give the filesystem a chance to save
    a checkpoint
    mount yaffs2 mtd@system /system
    mount yaffs2 mtd@system /system ro remount
    # We chown/chmod /data again so because mount is run as root + defaults
    mount yaffs2 mtd@userdata /data nosuid nodev
    chown system system /data
    chmod 0771 /data
```

从取证的角度来看，HTC Incredible 在启动进程完成后改变了浏览器存储缓存数据的方式。/bootcomplete.inc.rc 文件的内容非常有效：

```
rm -r /data/data/com.android.browser/cache
mkdir /app-cache/com.android.browser
chmod 755 /app-cache/com.android.browser
chownto /app-cache/com.android.browser /data/data/com.android.browser
mkdir /app-cache/com.android.browser/cache
chmod 755 /app-cache/com.android.browser/cache
chownto /app-cache/com.android.browser/cache /data/data/com.android.browser
ln -s /app-cache/com.android.browser/cache /data/data/com.android.browser/cache
rm -r /data/app-cache
rm -r /data/DxDrm
```

由此可见，在终端设备一旦完成了启动过程，浏览器的缓存数据存储区域从 NAND 闪存上的用户数据分区改变到一个位于 /app-cache 的临时可读写内存盘。这就意味着当终端设备被关闭时，所有写进 /app-cache 的数据都将丢失，如图 2.5 所示。

更多资源请访问稀酷客(www.ckook.com)

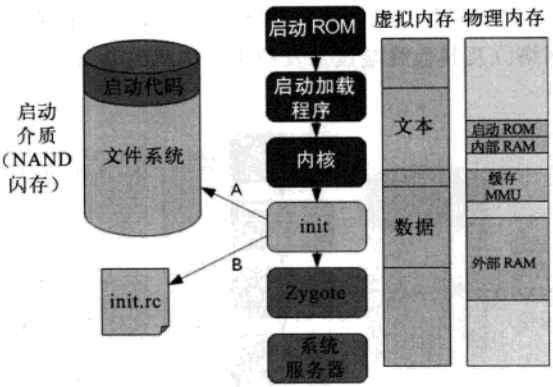


图 2.5　init 进程

总结一下，init.rc 是 Android 终端设备设置中一个最基本的步骤，需要对其进行仔细学习，以理解如何对任何一款 Android 终端设备进行配置和操作。

2.4.5　Zygote 和 Dalvik

在第 3 章中，我们将讨论为用户应用程序建立动态沙盒的单独虚拟机的具体细节。Dalvik 虚拟机是谷歌选择的创建应用沙盒的技术。在启动时，Zygote 序列建立一个 Java 动态环境，并向系统注册一个套接字（socket）；从而使需要初始化的应用程序可以请求一个新的 Dalvik 虚拟机。没有 Zygote 提供的服务，Android 内核依然可以运行。但是，应用程序却无法运行，包括嵌入的应用，如电话、浏览器以及其他的核心功能，如图 2.6 所示。

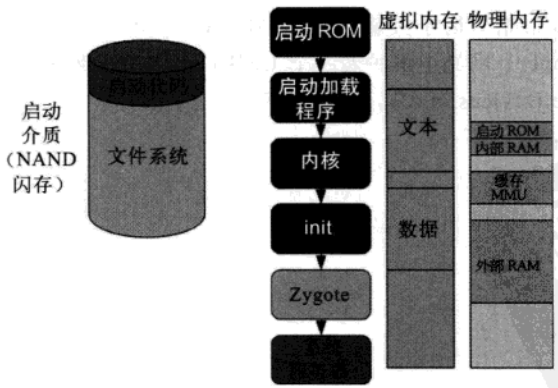


图 2.6　Zygot 与 Dalvik

2.4.6　系统服务器

以上章节中提到的终端设备中的核心功能都起源于系统服务器。一旦 Java 动态环境

更多资源请访问稀酷客(www.ckook.com)

设置完成以及 Zygote 进程开始监听，系统服务器开始运行。这样，系统服务器运行着核心功能，如电话、网络以及其他终端设备及应用所依赖的基础部件。图 2.7 是系统服务器如何运行的图解。

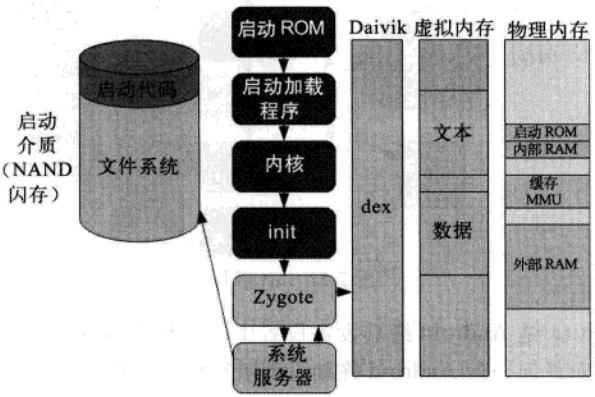


图 2.7　系统服务器

系统此时广播一个标准的叫做行动启动完成（ACTION_BOOT_COMPLETED）的消息，通知后续的进程启动过程已经完成。Android 系统此时处于完全运行状态，随时可以与用户进行交互。

2.5　制造商

谷歌的 Android 策略繁衍于大量种类不同的 Android 终端设备制造商。Android 开发者（Android Developer）网站上维护着一个 USB 厂商名称的列表，目前包含着 15 个制造商（Using Hardware Devices, n.d.）。具体如下：

- 宏碁（Acer）；
- 戴尔（Dell）；
- Foxconn；
- Garmin-Asus；
- HTC；
- 华为（Huawei）；
- Kyocera；
- LG；
- 摩托罗拉（Motorola）；
- Nvidia；
- Pantech；
- 三星（Samsung）；

更多资源请访问稀酷客(www.ckook.com)

- 夏普（Sharp）；
- ZTE。

然而，如果考虑不在以上名单里但却有在计划阶段的终端设备的厂商，目前已经有超过 50 家不同的 Android 终端设备制造商。

这自然给取证调查师和企业安全经理们带来了独特的挑战。数量众多的制造商、终端设备类型以及终端设备产生出了非常复杂的策略、程序、技术，甚至 USB 电缆。

2.6　Android 更新

Android 的更新模型是分布式的、面向终端设备的，不是谷歌的责任，而是服务提供商或终端设备制造商的责任。虽然主要受到谷歌影响的开放手机联盟（Open Handset Alliance）负责维护核心的 Android 操作系统，此联盟对任何具体的终端设备不具有任何控制能力。这个分布的方式对面向 Android 终端设备的取证和安全程序带来了以下几个方面的影响。

第一，分析师永远无法确定一个终端设备安装了哪个版本的 Android 系统。部分原因是由于公司始终追求的是利润最大化。尤其是在美国，如果一个消费者购买了一款包含两年服务合约的 Android 终端设备，无线通信服务提供商基本上就锁定了这个消费者，因为提前解约的费用一直在上升。由于此用户几乎不可能对服务进行升级或者是购买一款新手机，对服务提供商来说也就意味着固定的收入。同时，对现有 Android 终端设备和产品进行升级的开发、部署及支持费用非常高。因此，服务提供商或者是投资到新产品开发，以激发新的兴趣以及随即而来的更多的销售，或者是维护现有的终端设备，其结果是带来非常有限的额外收入，即使有的话。通常，使用旧的 Android 手机的消费者使用旧的、功能欠缺的、安全性低的 Android 版本。谷歌对此予以承认并声明将着手应对此问题。

第二，安全地获取一个 Android 终端设备的取证映像对于不同版本的 Android 以及不同类型的终端设备来说具有非常大的差别。例如，取证分析师如果对同样一款 HTC Dream 100 (T-Mobile G1) 手机进行取证分析，则使用的技术对于运行 Android1.5，内核版本为 2.6.30.4 或更早，与运行 Android1.6，内核版本为 2.6.30.4 或更新，会有非常大的差别。可以想象，如果有超过 50 家的制造商，有超过 300 款的 Android 终端设备，有 4 个主要版本及几百个次要版本，可能的组合将是巨大的。

第三，用于与不同的 Android 终端设备实现连接的硬件、驱动程序以及软件可能有所不同。将在第 3 章进行讨论的 Android 软件开发套件（SDK）会提供某种程度的一致性。然而，每一个制造商可能有自己特定的一套驱动程序和软件。例如，如果将一个三星的 Galaxy S 手机与一个运行 Windows 计算机进行连接，用户首先必须安装三星提供的专门软件。然而，很多其他的终端设备使用由谷歌通过软件开发套件（SDK）提供的标准的 USB 驱动程序。

最后，每一个制造商都会有自己的启动过程，包括硬件、启动加载程序及 ROM 固

更多资源请访问稀酷客(www.ckook.com)

件。我们将在第 6 章讨论一些如何利用不同的制造商为终端设备提供的启动过程的技术。

2.6.1 自定义用户界面

Android 的某些部分是使用 Apache 2.0 开放源码许可进行授权，而不是 GPLv2 开放源码许可。Apache 2.0 开放源码许可允许制造商和开发商对 Android 系统的某些特定部分进行个性化处理，并不要求他们将个性化的源代码贡献给整个 Android 社区进行共享。Apache 2.0 许可主要适用于终端设备中涉及制造商知识产权的特定驱动程序以及个性化的用户界面。

通过允许私有产权的用户界面，谷歌就能够使制造商在一个关键领域对锁定的用户群体进行量身订制，使他们的终端设备与竞争对手的有所差别。例如，某一款 Android 终端设备可能面向青年学生市场，因此会注重短信和社交方面的应用，而另一款终端设备可能主要面向商业用户群体。在本质上，这些终端设备在操作上非常相似。然而，对用户界面的个性化（以及硬件设计的实现）会产生独特的用户体验。表 2.1 罗列出不同的制造商所开发的个性化用户界面。

表 2.1 专用 Android 用户界面

厂　商	自定义的用户界面
摩托罗拉（Motorola）	Motoblur
HTC	Sense
三星（Samsung）	TouchWiz
索尼－爱立信（Sony Ericsson）	Rachael, UX, Nexus
宏碁（Acer）	Touch 3D
戴尔（Dell）	Stage
Viewsonic	TapnTap

2.6.2 售后市场中的 Android 终端设备

由于 Android 操作系统的开放源码性，面向在最初投向市场时运行其他操作系统的终端设备的个性化生成应运而生。作为一个名誉不佳的实例，已经有多个版本的 Android 可以安装并运行在 iPhone 上（Linux on the iPhone, n.d.）。看看苹果的追捧者在此种情境下如何反应必定是一件非常有趣的事。而更加实际的情况是，Android 已经被移植到很多款最初推出时只运行 Windows Mobile 的 HTC 手机上。还有更多的实例涉及诸如诺基亚这样的公司生产的终端设备，甚至包括智能手机以外其他种类的终端设备。

虽然可能不会经常发生，但是待取证分析的 Windows 手机（或 iPhone）实际上运行的可能是 Android 系统，考虑到这点非常重要。

更多资源请访问稀酷客(www.ckook.com)

2.7　具体的终端设备

以下的终端设备会在全书中反复使用，因此对每一款终端设备在此进行简单介绍，以便参考。某些终端设备属于第一批推向市场的 Android 智能手机，因此对其的了解已经比较清楚，用非常合理的价钱就可以购买到，对于用来对本书中的实例进行实验应该是非常好的终端设备。

2.7.1　T-Mobile G1

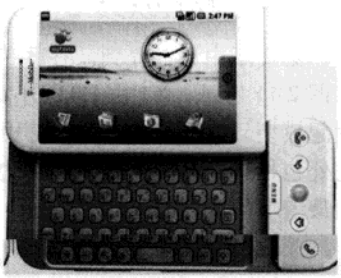


图 2.8　T-Mobile G1（DREAM 100）

图 2.8 所示的是 HTC 生产的 T-Mobile G1 智能手机，由 T-Mobile 于 2008 年 10 月在美国推出。像其他第一代终端设备一样，此款智能手机在易用性方面存在一些问题。尽管如此，在推出后的 6 个月以内，就卖出了超过 100 万台，因此可以作为一个很好的参考。

此款终端设备的数据如下：

- 生产商：HTC；
- 模型：G1（也称为 HTC Dream 100）；
- 服务提供商：T-Mobile；
- 推出时间：2008 年 10 月。

2.7.2　摩托罗拉 Droid

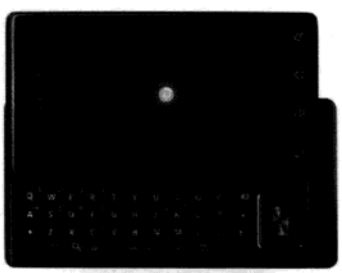


图 2.9　摩托罗拉（Motorola）Droid（A855）

图 2.9 所示的是摩托罗拉生产的摩托罗拉（Motorola）Droid 智能手机，由 Verizon 在 2009 年 11 月在美国推出。在推出后的 74 天里，就卖出了 105 万台 Droid 智能手机，比 2007 年 6 月推出的第一款 iPhone 更流行（Day 74 Sales, n.d.）。Droid 是一款非常好的可以用来作为参考的手机，如果任何用户考虑购买一台用于实验，在此强力推荐此款终端设备。

此款终端设备的数据如下：

- 生产商：摩托罗拉移动终端设备（Motorola Mobile Devices）；
- 模型：A855；
- 服务提供商：Verizon Wireless；
- 推出时间：2009 年 11 月。

2.7.3　HTC Incredible

图 2.10 所示的是面向 Verizon 网络推出的 HTC Incredible 智能手机，在美国也非常流行。此款终端设备将在此书中作为一个参考手机被大量引用。

更多资源请访问稀酷客(www.ckook.com)

此款终端设备的数据如下：

- 生产商：HTC；
- 模型：ADR6300；
- 服务提供商：Verizon Wireless；
- 推出时间：2010 年 4 月。

2.7.4 谷歌 Nexus One

第 1 章中已经提到，谷歌在 2010 年 1 月推出了自我品牌的智能手机 Nexus One(N1)，如图 2.11 所示。N1 由 HTC 开发，从各方面来说可以作为制造商应该如何开发手机的一个理想模型。内置有非常高速的中央处理器（1 GHz），运行最新版本的 Android 系统，创新之处包括有 3 个麦克风，对周边的背景噪声进行综合，然后与个人通话语音进行合并以产生最清晰的通话效果。

此款终端设备的数据如下：

- 生产商：HTC；
- 模型：HTC Passion；
- 服务提供商：T-Mobile、Verizon、Vodafone；
- 推出时间：2010 年 1 月。

图 2.10 HTC Incredible

图 2.11 谷歌（Google）Nexus One (N1)

2.8 本章小结

虽然终端设备的部件种类变化无穷，大多数的终端设备却有少量的公共核心部件。在大多数情况下，了解这些部件以及部件种类对于取证分析就足够了。然而，显而易见的是，在取证分析中，还有许多其他因素也应该进行考虑。对启动过程进行的概要介绍也为后续将要进行的深入讨论打下了基础。最后，对制造商以及终端设备的介绍对取证分析师

更多资源请访问稀酷客(www.ckook.com)

需要考虑的各种因素提供了进一步的信息。Android 市场依然是分散及多样化的，因此取证分析师需要时刻牢记，在对 Android 终端设备进行取证分析时，单一的策略和技术是无法达到预期效果的。

2.9 参考资料

Android-x86—Porting Android to x86. (n.d.). Retrieved March 9, 2011, from http://www.android-x86.org/.

Björnheden, M. (n.d.). *Enea Android Blog: The Android boot process from power on*. Retrieved December 17, 2010, from http://www.androidenea.com/2009/06/android-boot-process-from-power-on.html.

Chromium OS—The Chromium projects. (n.d.). Retrieved December 13, 2010, from http://www.chromium.org/chromium-os.

Day 74 Sales: Apple iPhone vs. Google Nexus One vs. Motorola Droid. (n.d.). The Flurry Blog—Mobile application analytics|iPhone analytics|Android analytics. Retrieved December 18, 2010, from http://blog.flurry.com/bid/31410/Day-74-Sales-Apple-iPhone-vs-Google-Nexus-One-vs-Motorola-Droid.

Krazit, T. (n.d.). *T-Mobile has sold 1 million G1 Android phones*. Wireless—CNET News. Technology news—CNET News. Retrieved December 18, 2010, from http://news.cnet.com/8301-1035_3-10226034-94.html.

Linux on the iPhone. (n.d.). Retrieved December 15, 2010, from http://linuxoniphone.blogspot.com/.

Main Page|PDAdb.net—Comprehensive database of smartphone, PDA, PDA phone, PNA, netbook & mobile device specifications. (n.d.). Retrieved November 28, 2010, from http://pdadb.net/index.php.

Mobile Memory. (n.d.). Hynix. Retrieved March 9, 2011, from http://www.hynix.com/products/mobile/mcp.jsp?menuNo=1&m=4&s=4.

Nielsen Wire. (n.d.). *U.S. smartphone battle heats up: Which is the "most desired" operating system?* Retrieved December 12, 2010, from blog.nielsen.com/nielsenwire/online_mobile/us-smartphone-battle-heats-up/.

Public Version of OMAP35xx. (2010). *Technical reference manual—Version M (SPRUF98M)* Houston, TX: Texas Instruments Incorporated. Retrieved December 17, 2010, from http://focus.ti.com/docs/prod/folders/print/omap3530.html.

Using hardware devices. (n.d.). *Android developers*. Retrieved March 9, 2011, from http://developer.android.com/guide/developing/device.html.

更多资源请访问稀酷客(www.ckook.com)

第 3 章

Android 软件开发套件和 Android 程序调试桥

本章要点

- Android 平台
- 软件开发套件
- Android 安全模型
- 取证与软件开发套件

3.1 绪论

Android 的软件开发套件（SDK）不但提供工具，创建能够在 Android 平台上运行的应用程序，而且提供文档及共享程序库，能够在很大程度上帮助对终端设备进行取证或安全分析。第 2 章中介绍的 Android 硬件在终端设备的功能上发挥主要作用，软件又将这些特点发挥到极致，最终创建出消费者希望获得的体验和功能。因此，对 Android 软件开发套件的深入了解，为我们在对数据、终端设备以及重要的共享程序库进行取证调查时提供帮助。

3.2 Android 平台

Android 在 2007 年 11 月正式对外发布，而一些重要的开发工作自 2005 年起便一直都在进行。这些开发工作与大量而种类繁多的面向 Android 的硬件相结合已经产生一个范围广阔的生态系统，对取证分析师和安全工程师来说增加了相当大的复杂性。

Android 的一个直接特点是 Android 平台的版本。平台是决定终端设备所支持功能的一个重要因素。每一个正式发布的 Android 平台都被赋予一个应用程序编程接口（API）级别，同时所有新的版本都有一个代码名。截至 2011 年 1 月的当前版本是 Android 2.3，代码名是 Gingerbread。下一个主版本的代码名是 Honeycomb，看来是瞄准着平板电脑这一类终端设备预期增长的需要。表 3.1 是一个完整的 Android 平台列表，包含了应用编程接口级别、代码名及发布日期。

表 3.1 Android 平台列表

平台	API 级别	代码名	推出时间
Android 2.3.3	10	Gingerbread	2011 年 2 月 9 日

更多资源请访问稀酷客(www.ckook.com)

（续）

平台	API 级别	代码名	推出时间
Android 2.3	9	Gingerbread	2010 年 12 月
Android 2.2	8	FroYo	2010 年 5 月 20 日
Android 2.1	7	Eclair	2010 年 1 月 11 日
Android 2.0.1	6	Eclair	2009 年 12 月 11 日
Android 2.0	5	Eclair	2009 年 10 月 5 日
Android 1.6	4	Donut	2009 年 9 月 16 日
Android 1.5	3	Cupcake	2009 年 4 月 27 日
Android 1.1	2	Petit Four	2009 年 2 月 9 日
Android 1.0	1	无	2008 年 9 月 23 日

虽然存在很多 Android 版本，在当前终端设备中使用的版本对取证分析师和安全工程师来说会有巨大的影响。图 3.1 是一个对 Android 版本使用情况的统计，基于一个在两周期间使用终端设备对 Android 市场进行访问的情况（Platform Versions, n.d.）。

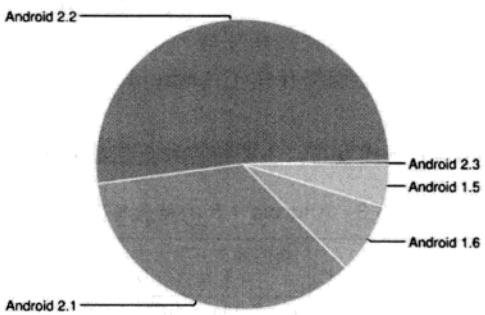


图 3.1　2011 年 1 月使用不同 Android 平台的终端设备分布

对以上的统计进一步具体化，表 3.2 罗列出的是在美国流通的基于不同 Android 版本的终端设备数量。这些数据是根据 2011 年 11 月，美国约为 1599 万台的 Android 终端设备得出的（comScore Reports, n.d.）。

表 3.2　在美国使用不同 Android 平台的终端设备数估算

平台版本	终端设备数量
Android 2.3	63 960
Android 2.2	8 282 820
Android 2.1	5 628 480
Android 1.6	1 263 210
Android 1.5	751 530

另外，谷歌也通过发布图示说明在 2010 年 8 月至 2011 年 2 月 2 日 7 个月期间 Android 版本使用的分布情况。这个数据也是基于终端设备访问 Android 市场得出，同时显示出 Android 版本更新的进展情况，如图 3.2 所示（Platform Versions, n.d.）。

虽然有些终端设备永远也不会支持最新版本的 Android 系统，但大多数最终还是会得到更新。将来的终端设备也许会支持并被快速更新到最新的版本。然而，从取证和安全的

更多资源请访问稀酷客(www.ckook.com)

角度来考虑，也不能忽视旧的版本。

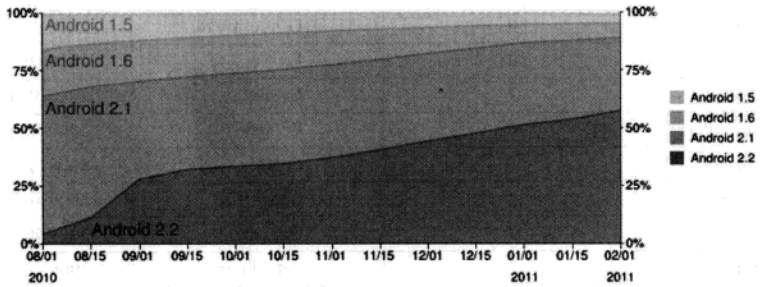


图 3.2　2010 年 8 月至 2011 年 2 月 1 日期间 Android 版本应用的分布情况

Android 平台至 2.3.3 版本之亮点（Gingerbread）

Android 是一个非常复杂且发展迅猛的平台，尝试完整地记录下所有的功能特点是此书的重要组成部分。然而，对每一个发布的主要版本进行简要介绍非常有用，能够使取证分析与终端设备可能支持的功能特点相呼应。一般来说，功能特点相互依赖，因此，Android 1.5 版本中的功能很有可能继续存在于 Android 2.3.3 版本中并得到改进。

1. Android 1.5 版本

Android 1.5 于 2009 年 4 月正式发布，主要特点和更新如表 3.3 所示（Android 1.5, n.d.）。

表 3.3　Android 1.5 的特点与亮点

新的面向用户的特点	新的面向开发者的特点、应用开发接口以及新技术	嵌入的应用
• 细化的用户界面，包括 in-call 体验、SMS/MMS 等 • 照相机、全球定位系统、浏览器及 Gmail 的性能改进 • 显示屏上的软键盘 • 主显示屏装饰物 • 视频录像与回放 • 更好的蓝牙支持与功能 • 浏览器复制与粘贴、页面查询等 • 对联系人的改进，包括对过去的通话赋予相片和日期 / 时间以及对联系人的触摸式访问方式 • 查看谷歌 Talk 中的朋友在 Contacts、SMS、MMS 和 Gmail 中的状态以及邮件的应用 • 将视频上载至 YouTube、图片上载至 Picasa	• 新的 Linux 内核（2.6.27 版本） • 对 SD 卡文件系统的自动检查与修复 • 改进的媒体架构 • 语音识别架构 • 支持 26 种地区语言	• 闹钟 • 浏览器 • 计算器 • 录像机 • 照相机 • 联系人 • 个性化的地区设定（开发者应用） • 开发工具（开发者应用） • 拨号器 • E-mail • 图库 • 日语输入法 • 短消息 • 音乐 • 设置 • 备件（开发者应用）

2. Android 1.6 版本

Android 1.6 于 2009 年 9 月正式发布，主要特点和更新如表 3.4 所示（Android 1.6, n.d.）。

更多资源请访问稀酷客(www.ckook.com)

表 3.4　Android 1.6 的特点与亮点

新的面向用户的特点	新的面向开发者的特点、应用开发接口以及新技术	嵌入的应用
• 对 Android 的快速查询框 • 更新的照相机、摄像机和图库 • 支持虚拟私有网和 802.1x • 电池使用指示灯 • Android 市场更新，包括分类、销售排行榜以及屏幕截图	• Linux 内核 2.6.29 • 扩展的搜索架构 • 文字到语音引擎 • 支持手势 • 新的访问架构 • 扩展了对显示屏密度与分辨率的支持 • 对 CDMA 电话的支持 • 新版本的 OpenCore，更好的音响处理	• 所有 Android 1.5 中的应用 • 手势构建器

3. Android 2.0 与 Android 2.1 版本

Android 2.0 和 Android 2.1 分别于 2009 年 10 月和 2010 年 1 月正式发布，主要特点和更新如表 3.5 所示（Android 2.1, n.d.）。

表 3.5　Android 2.0/2.1 的特点与亮点

新的面向用户的特点	新的面向开发者的特点、应用开发接口以及新技术	嵌入的应用
• E-mail 及 联系人中多账号之间的同步以及快速查找联系人功能 • 支持邮件的相互交换 • SMS/MMS 查找功能 • 许多照相机的增强功能，包括内置的闪光灯、数字聚焦等 • 对 Android 虚拟键盘的改进 • 浏览器中的更新，包括收藏夹中使用缩略图、二次聚焦以及对 HTML5 的支持 • 新的日历功能，如邀请客人	• 改写了图像架构，以改进硬件加速性能 • 蓝牙 2.1 • 生动的墙纸应用开发接口	• 所有 Android 1.6 中的应用

4. Android 2.2 版本

Android 2.2 于 2010 年 5 月正式发布，主要特点和更新如表 3.6 所示（Android 2.2, n.d.）。

表 3.6　Android 2.2 的特点与亮点

新的面向用户的特点	新的面向开发者的特点、应用开发接口以及新技术	嵌入的应用
• 新的主屏幕小窍门构建 • 主屏幕上电话、应用程序启动、浏览器的快捷方式 • 在扩展的交换中加入了数字密码或密钥的选择将终端设备进行解锁；远程擦除；对交换日历的支持；自动发现；全球地址表查找 • 改进的照相机和图库 • 某些终端设备可以作为可携带式的 Wi-Fi 集结点，可以与最多 8 个终端设备进行共享 • 多语言键盘 • 对浏览器、Dalvik 虚拟机、图像处理以及内核存储管理系统性能方面的改进	• Linux 内核 2.6.32 Linux（支持大于 256MB 的内存） • 新的媒体架构，可以支持本地文件播放以及 HTTP 连续播放 • 对蓝牙进行的改进，包括通过蓝牙支持语音拨号、与其他手机共享联系人以及与汽车更好的兼容性 • 面向终端设备短信的 Android 云 • Android 应用错误报告 • 外部存储器上的应用程序 • 数据备份应用编程接口 • 终端设备策略管理器	• 所有 Android 2.1 中的应用

更多资源请访问稀酷客(www.ckook.com)

5. Android 2.3 版本

Android 2.3 于 2010 年 12 月正式发布，主要特点和更新如表 3.7 所示。

表 3.7 Android 2.3 的特点与亮点

新的面向用户的特点	新的面向开发者的特点、应用开发接口以及新技术	嵌入的应用
• 在简单和速度方面对用户界面的细化 • 更快、更直观的文本输入 • 一键式文字选择及复制 / 粘贴 • 对能量管理的改进 • 支持互联网电话（VoIP） • NFC 阅读应用使用户能够阅读并与近域通信（NFC）标示进行交互 • 下载管理 • 照相机方面的改进，支持正面和背面摄像	• Linux 内核 2.6.35 • 在游戏的性能、感知、图像、音响及能量管理方面的改进 • 丰富的多媒体支持，如混合音响效果 • 在 Dalvik 动态运行和支持共享程序库方面的重大升级和改进 • 支持 57 种地方语言	• 所有 Android 2.2 中的应用 • 下载 • 搜索 • 语音录制

6. Android 2.3.3 版本

Android 2.3.3 于 2011 年 2 月正式发布，主要特点和更新如表 3.8 所示。

表 3.8 Android 2.3.3 的特点与亮点

新的面向用户的特点	新的面向开发者的特点、应用开发接口以及新技术	嵌入的应用
• 与 Android 2.3 相同	• 改进并扩展了对近域通信（NFC）的支持 • 改进了蓝牙、图像、媒体框架及语音识别 • 支持 57 种地方语言	• 所有 Android 2.3 中的应用

3.3 软件开发套件

Android 的软件开发套件（SDK）是开发 Android 应用所需要的开发资源。它包含软件共享程序库及应用编程接口、参考资料、仿真器以及其他工具。SKD 已经被很多环境支持，如 Linux、Windows、OS X，可以免费地从 http:// developer.android.com 网站上下载。

同时，Android 的软件开发套件（SDK）也是一个功能强大的取证工具，在很多情况下被取证分析师用于帮助对 Android 终端设备进行取证调查。

3.3.1 软件开发套件的发布史

在这些 Android 平台经 Android 正式发行的同时，软件开发套件也在频繁地更新。表 3.9 列出软件开发套件发行的完整历史，以帮助人们进行了解（SDK Archives, n.d.）。

表 3.9 正式记录在册的 Android 平台发布

平　台	应用编程接口（API）级别	发布日期
Android 1.6 r1	4	2009 年 9 月
Android 1.5 r3	3	2009 年 7 月

更多资源请访问稀酷客(www.ckook.com)

（续）

平　台	应用编程接口（API）级别	发布日期
Android 1.1 r1	2	2009 年 2 月
Android 1.0 r2	1	2008 年 11 月

3.3.2　软件开发套件的安装

因为软件开发套件对 Android 终端设备调查非常关键，检查者应该熟悉安装过程。以下的小节将一步一步地讲解如何在所支持的平台上安装软件开发套件。

1. 在 Linux 上安装软件开发套件

以下的安装步骤将建立在第 1 章所介绍内容的基础上，包括诸如 Java 开发套件等大多数前提，要在 Ubuntu 虚拟机上下载和编译 Android 开放源码项目（AOSP）。从任何一个终端视窗，首先安装所需要的 32 位共享程序库：

```
#install 32-bit libraries
sudo apt-get install ia32-libs
```

提示：32 位共享程序库

因为第 1 章中构建的 Ubuntu 虚拟机使用的是 64 位版本的 Ubuntu，为了安装软件开发套件（SDK），我们必须先安装 32 位的共享程序库。如果使用的是一台 32 位 Linux 工作站，就不需要此步骤。虽然 32 位的工作站可以运行软件开发套件（SDK），但是在版本 2.2 后，它就无法再用于构建 AOSP。

然后，运行 Firefox，访问 http://developer.android.com/sdk 并下载 Linux i386 平台（2011 年 1 月为 android-sdk_r08-linux_86.tgz）。默认的动作是此文件将会在文件管理器中被打开，如图 3.3 所示。

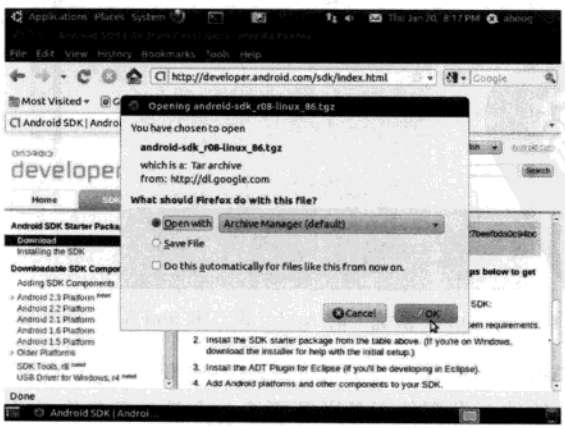


图 3.3　下载面向 Linux 操作系统的 Android 软件开发套件

更多资源请访问稀酷客(www.ckook.com)

然后，点击右键并获取此文件到主目录（文件夹），如图 3.4 所示。

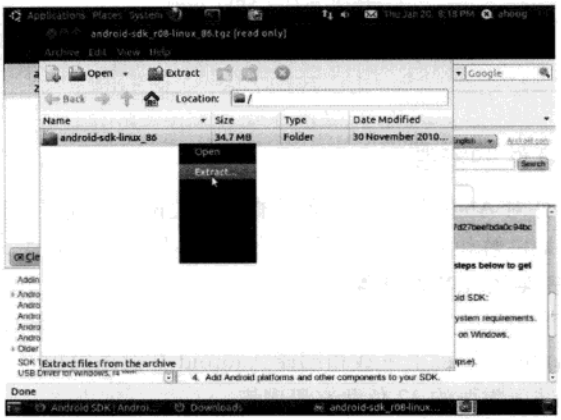


图 3.4　获取面向 Linux 操作系统的 Android 软件开发套件

然后，从终端视窗：

```
#navigate to the tools/ directory in the Android SDK
cd ~/android-sdk-linux_x86/tools

#run android
./android
```

以上操作将使 Android 软件开发套件和 Android 虚拟终端设备管理器（Android SDK and AVD Manager）开始运行，用户随后就可以下载和管理其他所需要的部分，如图 3.5 所示。

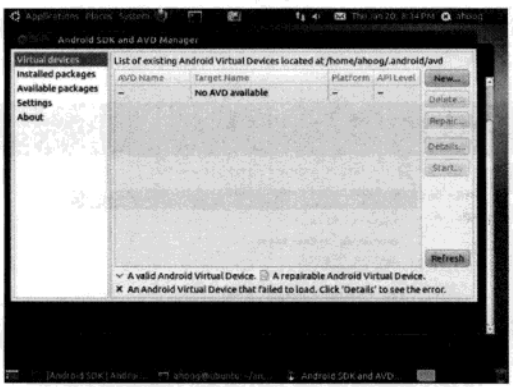


图 3.5　Linux 系统上的 Android SDK and AVD Manager

为了充分利用 Android 软件开发套件的功能，我们还要安装更多的软件部分。至少需要安装平台特定的软件开发套件工具以及至少一个软件开发套件平台（此时是 Android 2.3），使我们能够运行仿真器。为了完成安装，在左边的引导栏里去选择 Available

更多资源请访问稀酷客(www.ckook.com)

packages 以及两个额外的软件包，如图 3.6 所示。

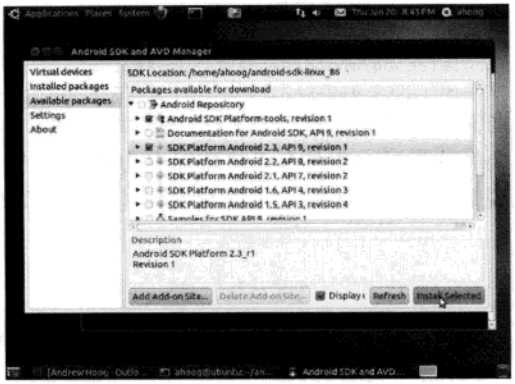


图 3.6　选择其他的 Android SDK 软件包

然后，点击 Install Selected（选择安装的包）按钮。用户将会被提示接受所有的软件包的许可，如图 3.7 所示。

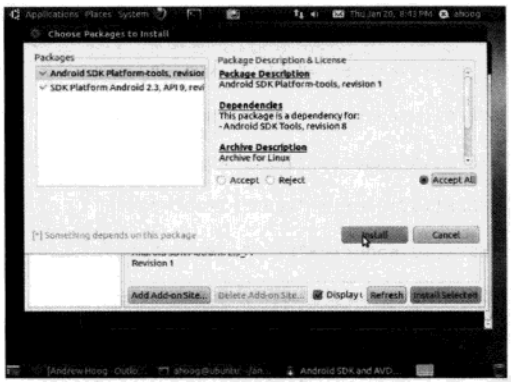


图 3.7　接受并安装 Android SDK 软件包

选择 Accept All（接受所有）（假设同意安装），然后进行安装。Android 软件开发套件和 AVD 管理器将下载并安装所有的软件部件。

用户也可以选择将二进制目录（文件夹）加入到操作系统（OS）的执行路径中，此后，用户去访问这些程序就不再需要给出完整的文件路径。在 Linux 中，执行以下命令[⊖]：

⊖ 在 .bashrc update 中，用户名被固定地设置成 ahoog，因此需要将其改变成所创建的用户名。在以下的实例中，将 USER 替换为在建立 Linux VM 时所创建的用户名。

```
export PATH=$PATH:/home/USER/android-sdk-linux_x86/tools/
export PATH=$PATH:/home/USER/android-sdk-linux_x86/platform-tools/
```

更多资源请访问稀酷客(www.ckook.com)

```
# open your .bashrc in an editor
nano -w ~/.bashrc

#add the following line, substituting your login name
export PATH=$PATH:/home/ahoog/android-sdk-linux_86/tools/
export PATH=$PATH:/home/ahoog/android-sdk-linux_86/platform-tools/
```

保存并退出，然后打开一个新的 shell（使用 Ctrl+O 组合键）。

在 Ubuntu 中必须要做的最后一步是在系统配置中为每一个 Android 终端设备厂商创建 USB 概况，具体来说就是 udev 规则。在终端上作为 root，编辑 / 创建（edit/create）udev 规则：

```
sudo nano -w /etc/udev/rules.d/51-android.rules
```

复制以下内容（厂商 ID 在 http://developer.andriod.com/guide/developing/device.html#VendorIds 中）：

```
#Acer
SUBSYSTEM=="usb", SYSFS{idVendor}=="502", MODE="0666"
#Dell
SUBSYSTEM=="usb", SYSFS{idVendor}=="413c", MODE="0666"
#Foxconn
SUBSYSTEM=="usb", SYSFS{idVendor}=="489", MODE="0666"
#Garmin-Asus
SUBSYSTEM=="usb", SYSFS{idVendor}=="091E", MODE="0666"
#HTC
SUBSYSTEM=="usb", SYSFS{idVendor}=="0bb4", MODE="0666"
#Huawei
SUBSYSTEM=="usb", SYSFS{idVendor}=="12d1", MODE="0666"
#Kyocera
SUBSYSTEM=="usb", SYSFS{idVendor}=="482", MODE="0666"
#LG
SUBSYSTEM=="usb", SYSFS{idVendor}=="1004", MODE="0666"
#Motorola
SUBSYSTEM=="usb", SYSFS{idVendor}=="22b8", MODE="0666"
#Nvidia
SUBSYSTEM=="usb", SYSFS{idVendor}=="955", MODE="0666"
#Pantech
SUBSYSTEM=="usb", SYSFS{idVendor}=="10A9", MODE="0666"
#Samsung
SUBSYSTEM=="usb", SYSFS{idVendor}=="400000000", MODE="0666"
#Sharp
SUBSYSTEM=="usb", SYSFS{idVendor}=="04dd", MODE="0666"
#Sony Ericsson
SUBSYSTEM=="usb", SYSFS{idVendor}=="0fce", MODE="0666"
#ZTE
SUBSYSTEM=="usb", SYSFS{idVendor}=="19D2", MODE="0666"
```

保存此文件。最后，确保此文件可以被所有用户读取：

```
sudo chmod a+r /etc/udev/rules.d/51-android.rules
```

现在就可以重启 udev daemon，或是简单地重启系统。

2. 在 Windows 上安装软件开发套件

最新 Windows 版的 Android 软件开发套件已经被包装成一个可执行的安装器，如图

更多资源请访问稀酷客(www.ckook.com)

3.8 所示。此安装器会首先确定是否已经安装了必要的 Java 部件，如果没有的话，将会自动下载并进行安装。然而，此安装器只会检查是否安装了 32 位的 JDK（Java 开发套件），并不会在 64 位的 Windows 7 上自动安装 JDK。如果运行的是 32 位的 Windows（如 Windows XP），此安装器也许是一个很好的选择，可从 http://developer.android.com/sdk/index.html 上下载所有软件并运行此安装器。

图 3.8　Windows 上的 Android 软件开发套件（SDK）安装器

然而，很多分析师和工程师已经在使用 64 位的操作系统。如要在 Windows 上安装 Android 软件开发套件，首先必须安装 Java SE SDK，可以从网站 http://java.sun.com/javase/downloads/ 进行下载，需要确认完整地安装了 SDK。

在安装了 SDK 后，从 http://developer.android.com/sdk/index.html 下载压缩版的 Windows Android 软件开发套件并将其解压到磁盘上。举例来说，我们可以选择 C:\ 作为目录（文件夹）进行解压，然后就会创建出目录（文件夹）C:\android-sdk-windows。

进入以上目录（文件夹）并双击 SDK Manager.exe 以开始更新过程。确认至少选择了 Android SDK Platform-tools（见图 3.9），以及一个发布的平台（此例中为 2.3）。

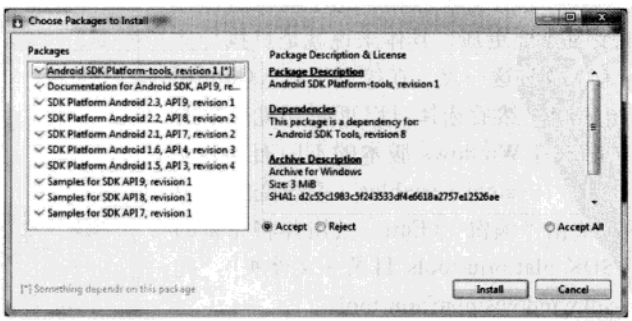


图 3.9　Windows 上的 Android 软件开发套件（SDK）管理器

对于运行 Windows 的 Android 终端设备，必须指定 USB 驱动程序。Android 软件开

更多资源请访问稀酷客(www.ckook.com)

发套件近期对如何安装驱动程序进行了升级。首先，确认正在运行 SDK Manager 并选择 Available packages（可用的安装包）。选择 Third party Add-ons → Google Inc. add-ons，最后选择 Google Usb Driver package，如图 3.10 所示。

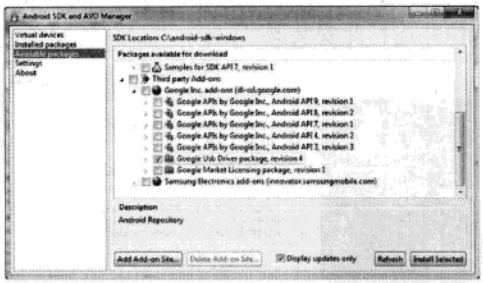


图 3.10　Windows 上的谷歌 USB 驱动软件

然后，选择接受许可（license），以进行安装，如图 3.11 所示。

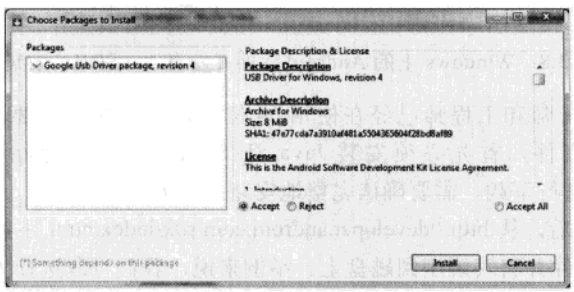


图 3.11　接受与安装许可

安装完成 USB 驱动程序后就具备了所有需要的部件。然而，对于 Android 软件开发套件，为了简化运行工具，工作站的环境变量需要更新，具体来说就是可执行文件的路径 PATH。为了做到这一点，在控制面板（Control Panel）上打开系统应用，然后选择可以更新环境变量的按钮，它所在的位置随着 Windows 版本的不同而异，如图 3.12 所示。最后，在 System variables（系统变量）中找到 Path（路径），单击“编辑”（Edit）按钮并将完整的路径附到 Android SDK platform-tools 目录（文件夹），此时即为 ;C:\android-sdkwindows\platform-tools.

“;”在此非常重要，因为它是不同路径之间的分隔符。完成了此番更新后，确认推出并等待命令提示符的出

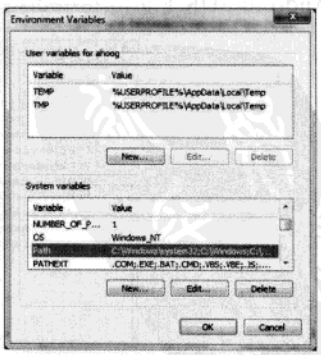


图 3.12　更新 PATH 环境变量（Windows 7 64 位）

更多资源请访问稀酷客(www.ckook.com)

现，表明新的设置已经生效。

3. 在 OS X 上安装软件开发套件

在 OS X 上安装 Android 软件开发套件，首先从 http://developer.android.com/sdk/index.html 下载安装软件包，OS X 将会自动提取相关内容。

按照屏幕引导，找到 tools 子目录（文件夹），如图 3.13 所示，双击 Android，运行 Android SDK and AVD Manager，如图 3.14 所示。

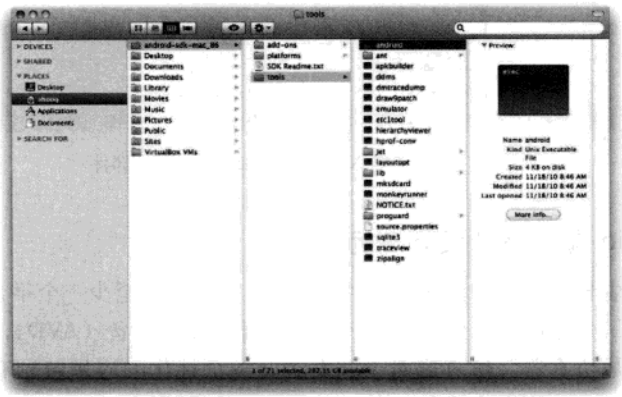


图 3.13　为 OS X 获取 Android 软件开发套件

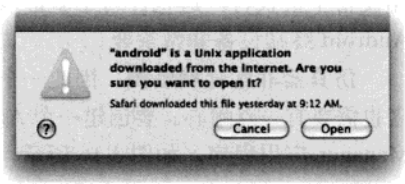


图 3.14　在 OS X 上打开 Android

在运行的 Android SDK and AVD Manager 中，选择 Available packages（可用包），点开 Android Repository（Android 仓库），然后选择 Android SDK platform-tools（Android 软件开发套件平台工具）以及至少一个 Android 平台，如图 3.15 所示。

然后选择 Accept（接受）许可，完成安装。最后，为了简化在 Android 软件开发套件上运行工具，应该更新可执行路径（PATH）。在 OS X 10.6 中，运行 Terminal（Applications → Utilities）并完成以下命令：

```
#edit your bash_profile
nano -w ~/.bash_profile

#add the following line substituting your full path to the platform-tools
directory PATH=$PATH:/Users/ahoog/android-sdk-mac_86/platform-tools

#save with Ctrl-O and then Ctrl-X to exit.  Exit Terminal
exit
```

确认完全退出 Terminal 应用程序并重新启动。在终端上，键入：

```
echo $PATH
```

以上命令将返回附有 platform-tools 的可执行路径。

更多资源请访问稀酷客(www.ckook.com)

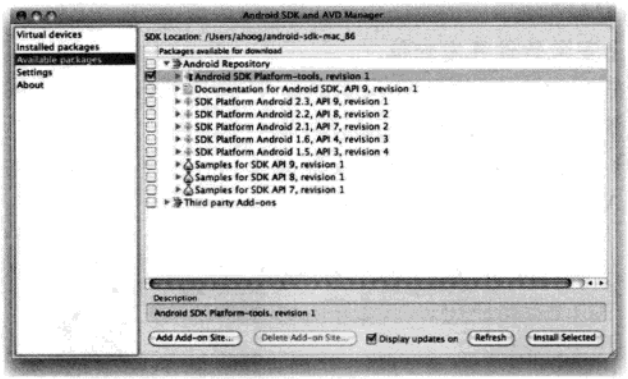


图 3.15　在 OS X 上安装 Android 软件部件

3.3.3　Android 虚拟终端设备（仿真器）

一旦在工作站上安装了 Android 软件开发套件并下载了至少一个平台版本后，接下来就可以开始创建一个在计算机上运行的 Android 虚拟终端设备（AVD），或者是虚拟移动终端设备，或者是仿真器。仿真器对于开发者开发个性化的应用特别有帮助。然而，它对取证分析师和安全工程师也有很大的价值，因为可以用来描述应用程序如何在一个终端设备上被执行。这也许对于验证取证调查中的发现或者对于测试取证工具如何影响一个 Android 终端设备非常重要。

仿真器非常耗费资源，因此一个理想的工作站应该具有足够的主机和内存资源。检查者也需要有一点耐心。要创建一个 Android 虚拟终端设备，首先运行 Android SDK and AVD Manager 应用程序，如图 3.16 所示。如果操作系统的路径已经得到更新，包含了软件开发套件中的 tools 目录（文件夹），则可以通过 shell、终端窗口或命令提示符来运行 Android。

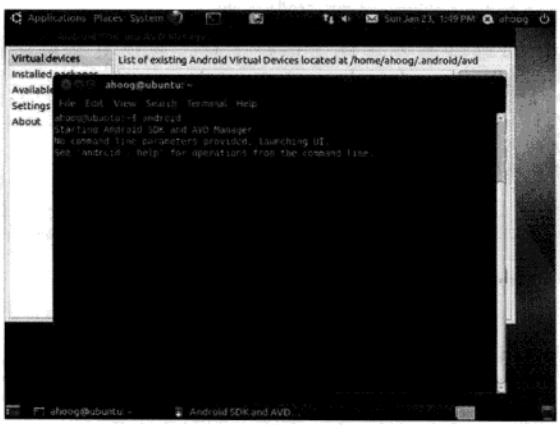


图 3.16　运行 Android SDK and AVD Manager

更多资源请访问稀酷客(www.ckook.com)

在左边的菜单，选择 Virtual devices（虚拟设备），然后单击 New（新建）按钮，如图 3.17 所示。

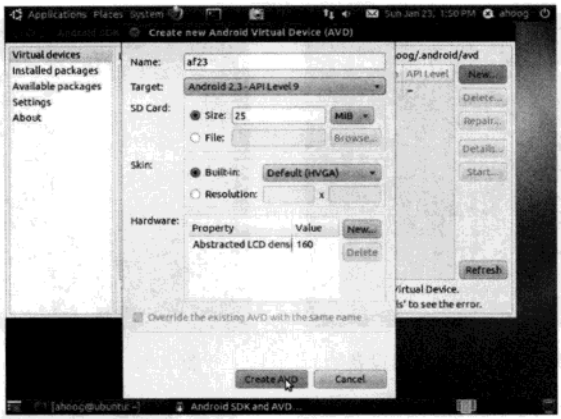


图 3.17　创建一个新的 Android 虚拟终端设备

确认填写以下字段：

- Name：虚拟终端设备的名称，例如：af23（Android Forensics 2.3）；
- Target：选择的目标平台，此次为 Android2.3 API 级别 9（Android 2.3 API level 9）；
- [可选项] SD 卡：可选地为此虚拟终端设备创建一个 SD 卡。

对虚拟终端设备还可以设置其他的特性。然而，此时所创建的是最基本的 Android 虚拟终端设备。如果需要分析的 Android 终端设备上运行着一个旧版本的平台，则可以简单地通过下载 Android 平台并使用 Android SDK and AVD manager 应用程序去创建一个运行旧版本的虚拟终端设备。当点击 Create AVD 时，虚拟终端设备将被创建，同时一个确认屏将显示，如图 3.18 所示。

在确认这个新创建的 Android 虚拟终端设备被选择后单击 Start（开始）按钮，此时你将被询问进行启动选项（Launch Options），如图 3.19 所示。

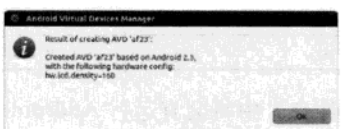


图 3.18　Android 虚拟终端设备创建确认

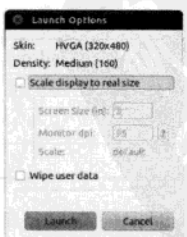


图 3.19　Android 虚拟终端设备启动选择

选择其中一个选项，然后单击 Launch（启动）按钮。此时，Android 虚拟终端设备将开始整个启动过程，可能会持续至少几分钟时间。在这个过程中，你将看到 Android 开始

更多资源请访问稀酷客(www.ckook.com)

运行，如图 3.20 所示。

最后，你面对的是一个正常运行的 Android 虚拟终端设备，如图 3.21 所示。

图 3.20 启动 Android 虚拟终端设备

图 3.21 运行 Android 虚拟终端设备

Android 虚拟终端设备功能强大，已处于正常运行状态。举例来说，你可以很容易地上互联网，浏览网站，如图 3.22 所示。你也可以配置邮件账号、向其他 Android 虚拟终端设备发送测试短信。当然，如果你是一个开发者，你也可以部署及测试你的应用程序。

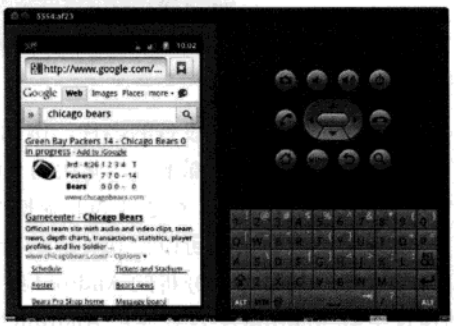


图 3.22 在 Android 虚拟终端设备上运行浏览器

当一个 Android 虚拟终端设备被创建并正常运行，所有生成的数据对于取证和安全研究来说非常有价值。所有创建的文件将在主目录（文件夹）中一个叫做 .android 的目录（文件夹）中（随平台不同而不同）（注意文件名有一个“.”作为前缀）。表 3.10 列出了具体操作系统中的路径。

表 3.10 Android 虚拟终端设备存储目录

工作站操作系统	Android 虚拟终端设备存储目录（文件夹）	举　例
Ubuntu Linux	/home/<username>/.android	/home/ahoog/.android
Mac OS X	/Users/<username>/.android	/Users/ahoog/.android
Windows 7	C:\Users\<username>\.android	C:\Users\ahoog\.android

在 Android 虚拟终端设备的 .android 目录（文件夹）中将会有运行 Android 虚拟终端设备所需要的配置文件和数据文件。

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~/.android$ tree
.
├── androidtool.cfg
├── avd
│   ├── af23.avd
│   │   ├── cache.img
│   │   ├── config.ini
│   │   ├── emulator-user.ini
│   │   ├── sdcard.img
│   │   ├── userdata.img
│   │   └── userdata-qemu.img
│   └── af23.ini
├── default.keyset
├── modem-nv-ram-5554
└── repositories.cfg

2 directories, 11 files
```

以下是取证分析和安全特别感兴趣的文件：

- cache.img：/cache 分区的磁盘映像；
- sdcard.img：SD 卡的磁盘映像（如果在对 Android 虚拟终端设备设置时进行了创建）；
- userdata-qemu.img：/data 分区的磁盘映像。

cache.img 和 userdata-qemu.img 两个文件属于 YAFFS2 文件系统，在第 4 章中将会提到，当前的取证分析软件不支持此系统的文件。然而，标准的取证分析工具可以很好地处理 sdcard.img 文件，因为它属于 FAT32 文件系统。

```
ahoog@ubuntu:~/.android/avd/af23.avd$ file sdcard.img
sdcard.img: x86 boot sector, code offset 0x5a, OEM-ID "MSWIN4.1", Media
descriptor 0xf8,
sectors 51200 (volumes > 32 MB), FAT (32 bit), sectors/FAT 397, reserved3
0x800000,
serial number 0x1d0e0817, label: "     SDCARD"
```

取证分析师和安全工程师们可以通过使用仿真器对网络、文件系统以及数据的各种呈现方式进行检查，可以学习到很多 Android 以及如何运行方面的知识。

3.3.4　Android 操作系统体系结构

对 Android 高层体系结构的理解非常重要，尤其是对安全程序以及在此之上的逻辑取证分析。

Android 的基础是 Linux 2.6 内核，它为 Android 提供了启动和管理硬件以及 Android 应用程序的最基本的软件。虽然内核提供的功能非常广泛，我们将侧重图 3.23 所标示的核心领域上。

如图 3.23 所示，底层功能包括能量管理、Wi-Fi、显示、音响驱动程序等。从取证分析的角度来说，也许最重要的就是闪存驱动程序，我们将在第 4 章中进行讨论。

在内核之上，是一系列的共享程序库，为开发者和类似终端设备拥有者的群体提供需要的核心功能。这些功能包括 WebKit 共享程序库，支持对绑定的浏览器或者是第三方开发的应用程序进行 HTML 渲染。其他的共享程序库面向字体、显示、各类媒体介质及基

更多资源请访问稀酷客(www.ckook.com)

于 SSL（Secure Socket Layers）的安全通信。最后，SQLite 共享程序库提供在 Android 上进行结构化数据存储的方法，这是取证分析师和安全工程师们需要特别关注的领域。

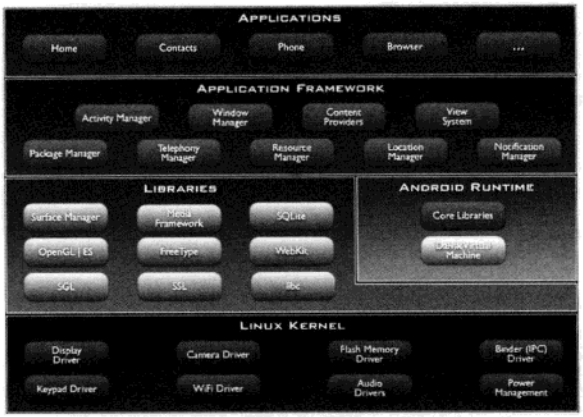


图 3.23　Android 体系结构

将核心的共享程序库与专用的 Java 虚拟机（VM）进行打包，形成 Android 动态运行环境，为应用程序提供运行环境。

最后，软件开发套件通过应用编程接口以及一个应用框架提供对这些资源的访问。应用框架支持第三方开发者之间的交互，并使他们能够通过抽象方式访问所开发的应用程序需要的关键资源。当探讨逻辑取证技术时，我们将会详细解释应用框架重要方面之一的内容提供者，因为他们提供了可从 Android 终端设备中抽取数据的主要机制。

3.3.5　Dalvik 虚拟机

为了创建一个高效、安全的移动应用环境，谷歌开发了 Dalvik 虚拟机（Dalvik VM）。

为了达到所期望的安全效果，每一个应用程序都运行在它自己的 Dalvik 虚拟机中。因此，在一台 Android 终端设备上，Dalvik 虚拟机允许很多虚拟机同时运行。Dalvik 虚拟机严重依赖于 Linux 操作系统提供的底层功能，如对核心共享程序库及硬件的访问、威胁与安全管理、存储管理等。

为了达到高效的目的，运行在一个 Dalvik 虚拟机里的应用程序使用一个特殊的文件格式，称为 Dalvik Executable（.dex）文件。开发者使用 Sun 公司的 Java 开发套件（Java Development Kit，JDK）编写和编译程序，所产生的二进制代码被转换成为 .dex 文件，以实现高效存储以及在 Dalvik 虚拟机环境中优化的运行。有一个有一定建树和知名度的 Android 黑客 JesusFreke 开展了一个很有趣的 smali/baksmali 项目。此项目可以让用户对一个 .dex 文件进行反编译，以了解相应的应用程序的功能（smali, n.d.）。

Dalvik 是 Android 的一个非常独特的组成部分，也是对终端设备进行取证和安全分析的关键部件。

更多资源请访问稀酷客(www.ckook.com)

3.3.6　本地代码开发

虽然大多数 Android 应用程序都是通过软件开发套件（SDK）使用 Java 进行开发，谷歌通过本地开发套件（Native Development Rit，NDK）提供了一个低层的开发平台。本地开发套件于 2009 年 6 月首次发布，至今已经经历了 5 次修改，最近一次发布是在 2010 年 11 月。

本地开发套件允许开发者使用 C/C++ 编写代码，然后直接为 CPU 编译。虽然这种方式为开发过程带来了一些复杂性，但是通过复用已经存在的 C/C++ 代码或者实现可以在 Dalvik 虚拟机之外进行优化的某些功能（程序），这种方式能够使某些开发者从中受益。本地开发套件不允许开发者开发的应用程序全部运行在 Dalvik 虚拟机之外；取而代之的是，C/C++ 组件被封装在应用程序的 .apk 文件内，然后从应用程序运行的虚拟机中进行调用。

到目前为止，本地开发套件支持 ARMv5TE 和 ARMv7-A 中央处理器（CPU），未来还将支持 Intel x86 中央处理器系列。当一个开发者在一个平台上编写了代码（如 Mac OS X），但是可在另外一个中央处理器进行编译，这种技术称为跨平台编译应用程序。本地开发套件极大地简化了这个过程，为开发者提供了可使用的一系列共享程序库。

从取证分析和安全的观点来说，跨平台编译是研发新技术及应用的一个重要组成部分。虽然大多数取证分析师和安全工程师并不需要编译代码，但是了解这个过程的工作原理以及它在整个过程中所发挥的作用还是十分必要的。例如，最初的 Android1.5 root 就发现了一个 Linux 内核中获得访问权限的程序错误（CVE-2009-2692）。最初的代码以源代码的方式发布，需要跨平台编译。其重要优点就在于分析师可以十分准确地描述如何利用终端设备如何并且根据需要提供源代码。

随着 Android 的日渐成熟，我们预计会看到面向本地开发套件（NDK）的进一步开发以及在本地编译的代码。

3.4　Android 安全模型

Android 平台通过一系列保护用户的控制措施来实现安全。

当一个应用程序第一次进行安装时，Android 通过检查 .apk 文件确认此应用程序拥有一个能够识别开发者的数字签名。不同于 SSL，这个数字证书不要求必须由一个证书权威机构进行签名。然而，开发者必须妥善地保管好密钥。例如，如果一个金融机构的数字签名不再有效，一个恶意的开发者可能针对银行应用发布一个更新，结果会造成关键数据被窃取。

在 .apk 文件得到验证后，Android 查验由开发者创建的表明此应用程序对系统进行访问所需要权限的专用文件。例如，一个应用程序也许会访问用户的联系人列表、SMS 短信息以及网络 / 互联网。如果此应用程序能够给 SMS 系统增加功能，这些访问权限就比较

更多资源请访问稀酷客(www.ckook.com)

合理。然而，如果此应用程序只是简单地去改变终端设备的背景图像，那么用户就应该对此访问权限进行质疑，并可以选择不安装此应用程序。在实际应用中，用户通常都快速地决定允许所有的访问权限以及应用请求，因此可能会造成恶意的应用程序被安装。

在应用程序得到验证以及用户对请求的访问权限进行授权后，应用程序可以进行安装。Android 安全模型的一个关键部分是每一个应用程序都被赋予一个唯一的 Linux 用户 ID 和组 ID，运行在自己的进程和 Dalvik 虚拟机里。在应用程序安装的过程中，系统在终端设备上创建一个专门的目录（文件夹），用于存储此应用程序的数据，并且仅允许应用程序利用 Linux 用户 ID 和组 ID 的相应访问权限对这些数据进行访问。此外，此应用程序的 Dalvik 虚拟机使用应用程序的用户 ID 运行在自己的进程中。这些关键的机制在操作系统层面上强制数据安全，因为应用程序之间不共享内存、访问权限及磁盘存储。应用程序只能在它们自己的 Dalvik 虚拟机范围内访问内存和数据。

当然，这一过程也有一些例外。第一，开发者可以使用同一个数字证书对多个应用程序进行签名，也可以声明与其他的应用程序共享同样的用户 ID、进程、内存以及数据存储空间。这种情况非常特殊，通常用于当一个开发者同时拥有一个免费版和一个收费版的应用。当一个用户升级到收费版的应用时，使用免费版时的数据可以继续使用，因此不会造成数据的丢失。

此外，Android 用户可以选择从 Android 市场以外的地方安装应用程序，也可以选择省略对数字签名的检查。此选项可以通过终端设备 Settings（设置）中的 Applications（应用）菜单进行设置。一旦选择此项，用户看到一个警告提示，如图 3.24 所示。

图 3.24 Android 可以设置成允许从不明来源安装应用程序

最常见的情况是用户通过直接下载一个 .apk 文件来从网站上进行应用程序的安装。此安装过程将省略对数字签名的检查。最近有一款 AT&T 手机（Motorola Backflip）因为将此选项从 Android 中删除而造成很多用户的不满。（Android On Lockdown, n.d.）然而，使用 Android SDK 时还是存在一些避绕的方法，我们将在第 6 章中进行讨论。

Android 中所设计的安全结构的直接结果是取证检查者没有简单的方法能够从终端设备上抽取核心的用户数据。除此以外，安全结构对于在应用程序之间分离和保护数据还是非常有效的。

3.5 取证与软件开发套件

软件开发套件对取证有什么重要作用？软件开发套件不但提供一系列对 Android 终端设备进行分析的工具和驱动程序，而且对描述应用程序以及其他取证研究也很有用。

更多资源请访问稀酷客(www.ckook.com)

3.5.1　将 Android 终端设备与工作站进行连接

对一个 Android 终端设备与一个虚拟机如何连接进行记录非常重要。迄今为止，Android 终端设备都有一个物理的 USB 接口，使它们能够相互联结、共享数据和资源，并可通过计算机或工作站进行充电。如果在终端设备上只运行一个操作系统，那么可以对与其连接的 USB 设备进行检测和访问。然而，可能需要一些额外的配置和驱动程序。但是，如果在终端设备上运行的是一台虚拟机，则需要终端设备上的操作系统直接将连接传递给虚拟机。例如，如果终端设备上运行的操作系统是 OS X，而你目前运行的是 VMWare Fusion，则需要选择菜单 Virtual Machine → USB，再选择需要连接的终端设备（此例中为 Connect High Android Phone），如图 3.25 所示。

图 3.25　在 VMWare Fusion 中连接 USB 设备到 Ubuntu 虚拟机

同样，当终端设备上的操作系统是 Linux，而你目前运行的虚拟机是甲骨文公司的 VirtualBox，首先必须确认你是 usbusers 组的成员。然后，从任何一个终端会话窗口，执行以下命令：

```
#create usbusers group
sudo addgroup usbusers

#Add your username to the userusers group:
sudo usermod -a -G usbusers ahoog
```

下一步，通过虚拟机的 Settings（设置）为终端设备增加一个 USB Filter（过滤器），如图 3.26 所示。

最后，就可以实现与 USB 终端设备的连接，如图 3.27 所示。

如果运行的是 VM headless（在第 1 章中提到过的 VirtualBox 3.2.10），按照以下步骤来做。首先，需要安装 VBox Additions，以实现共享文件夹、更好的音响、对 USB 的支持（如果下载 / 购买的是 PUEL 版）等功能。在主工作站上会显示以下信息：

更多资源请访问稀酷客(www.ckook.com)

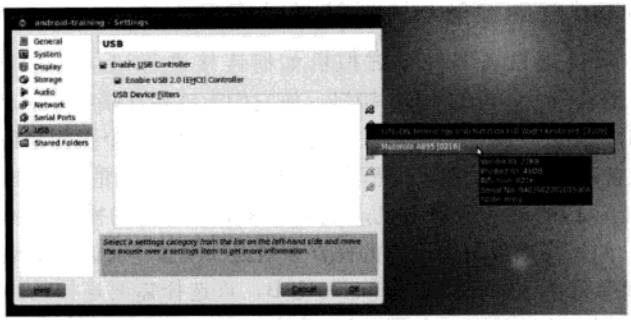


图 3.26　为在 Linux 操作系统上运行 Oracle 公司的 VirtualBox 增加一个 USB Filter（过滤器）

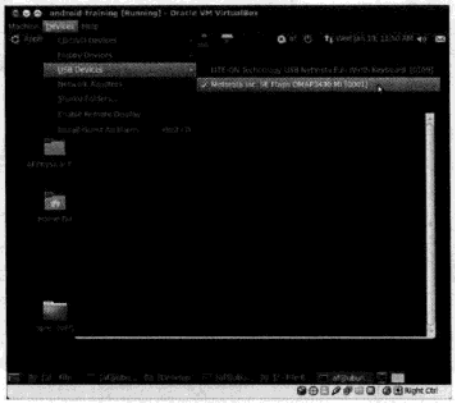


图 3.27　为在 Linux 操作系统上运行 Oracle 公司的 VirtualBox 实现 USB 终端设备连接

```
wget
http://download.virtualbox.org/virtualbox/3.2.0/VBoxGuestAdditions_3.2.0.iso

VBoxManage registerimage dvd ~/VBoxGuestAdditions_3.2.0.iso

VBoxManage storageattach af-book-vm --storagectl "IDE Controller" --port 1
--device 0 \
--type dvddrive --medium ~/VBoxGuestAdditions_3.2.0.iso
```

DVD 现在应该可以在 Ubuntu 虚拟机上使用了。远程登录虚拟机（参考第 1 章中的步骤），在使用的台式机上双击 VBOXADDITIONS_3.2.0_61806 DVD 以打开 DVD。然后双击 autorun.sh 并单击 Run 按钮。你会被要求提供密码，提供密码后安装将继续进行。图 3.28 是此步骤的图示。

现在，你已经安装好了 VBox Additions，可以将 USB 终端设备与客体机的操作系统相连接。但是，首先你必须关闭虚拟机（VM），然后执行以下步骤：

更多资源请访问稀酷客(www.ckook.com)

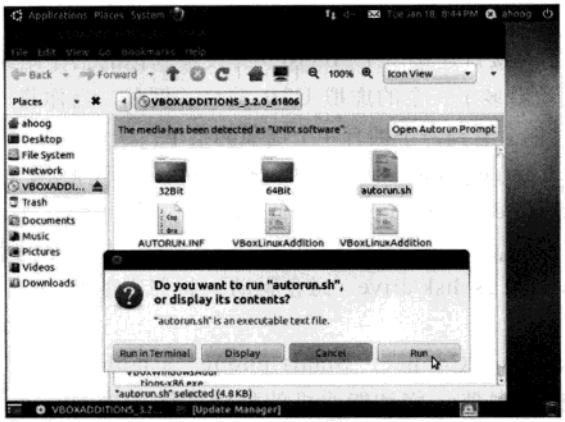


图 3.28　使用 Ubuntu 虚拟机远程协议安装 VBox 部件

```
#create usbusers group
sudo addgroup usbusers

#Add your username to the userusers group:
sudo usermod -a -G usbusers ahoog

#Determine attached USB device info
VBoxManage list usbhost

Oracle VM VirtualBox Command Line Management Interface Version 3.2.8
(C) 2005-2010 Oracle Corporation
All rights reserved.

Host USB Devices:

UUID:               b1c23004-db71-49ec-b5cb-348e2038b409
VendorId:           0x0781 (0781)
ProductId:          0x554f (554F)
Revision:           2.0 (0200)
Manufacturer:       Best Buy
Product:            Geek Squad
SerialNumber:       153563119AC07CAD
Address:            sysfs:/sys/devices/pci0000:00/0000:00:1d.0/usb2/2-1/
2-1.5//device:/dev/bus/usb/002/004
Current State:      Busy

#Create the USB filter to connect the device
VBoxManage usbfilter add 0 --target af-book-vm --vendorid 0781
--productid 554F \
--name "Geek Squad" --active yes

#Ensure USB is enabled
VBoxManage modifyvm Win2003SvrR2 --usb on

#Power on the guest (again recommended from inside screen)
VBoxHeadless -startvm af-book-vm -p 3392 &
```

通过以上实例，现在 USB 终端设备可以被转接到虚拟机了。

更多资源请访问稀酷客(www.ckook.com)

3.5.2 USB 接口

当你通过一个 USB 接口连接一个 Android 终设备到你的工作站或者一个虚拟机，硬件和 Android 通常显示出多于一个的虚拟 USB 接口。例如，当你将一部 HTC Incredible 通过 USB 进行连接，你会看到一个拥有 4 个选项的菜单：

1）Charge only Charge phone over USB（充电专用：通过 USB 对手机充电）；

2）HTC Sync Sync contacts and calendar（HTC 同步：对联系人及日历进行同步）；

3）Disk drive Mount as disk drive（磁盘驱动：按照磁盘驱动进行连接）；

4）Mobile Broadband Connect Smart phone' s mobile networks with PC（移动宽带：将智能手机的移动网络连接到 PC）。

如图 3.29 所示，默认的选择是 Charge only（充电专用）。HTC Sync 和 Mobile Broadband Connect 是 HTC 的专门选项，是此时无线通信服务商对此终端设备提供的支持。

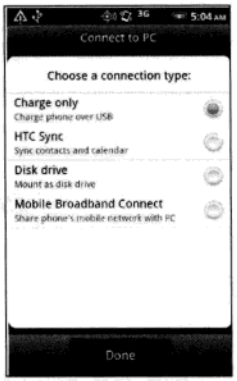


图 3.29 HTC Incredible 与 PC 相连接的选项

1. CD-ROM 接口

磁盘选项使用很普遍。这个选项将 Android 终端设备当作一个磁盘驱动连接到工作站上。这是终端设备向工作站呈现为多个 USB 设备的一个主要领域。当你将 HTC Incredible 插入计算机时，实际上它将自己注册为三类不同的驱动器：一个 CD-ROM 驱动器和两个 USB 大型存储驱动器。以下列表是在执行 dmesg 命令后 Linux 工作站内核显示的信息：

```
[  210.336135] usb 1-1: new high speed USB device using ehci_hcd and address 3
[  210.646221] scsi4 : usb-storage 1-1:1.0
[  211.649296] scsi 4:0:0:0: Direct-Access     HTC      Android Phone    0100
PQ: 0 ANSI: 2
[  211.652056] scsi 4:0:0:1: Direct-Access     HTC      Android Phone    0100
PQ: 0 ANSI: 2
[  211.654291] scsi 4:0:0:2: CD-ROM            HTC      Android Phone    0100
PQ: 0 ANSI: 2
[  211.657317] sd 4:0:0:0: Attached scsi generic sg2 type 0
[  211.658364] sd 4:0:0:1: Attached scsi generic sg3 type 0
[  211.661956] sr1: scsi3-mmc drive: 0x/0x caddy
[  211.662569] sr 4:0:0:2: Attached scsi CD-ROM sr1
[  211.662755] sr 4:0:0:2: Attached scsi generic sg4 type 5
[  211.678409] sd 4:0:0:0: [sdb] Attached SCSI removable disk
[  211.686339] sd 4:0:0:1: [sdc] Attached SCSI removable disk
```

可以看到，两个可直接访问的驱动器被分配在 4:0:0:0 和 4:0:0:1，而一个 CD-ROM 被分配在 4:0:0:2。CD-ROM 中存储着能够使此 HTC 终端设备进行同步和宽带连接的程序及驱动程序。很明显，没有任何实际的 CD-ROM 存在。然而，终端设备中的部分存储空间用来作为 CD-ROM，按照 ISO9660 被格式化。主机的操作系统就可以将此终端设备作为一个 CD-ROM 装上，在 Windows 操作系统中甚至都可以支持自动运行功能。利用 TSK

更多资源请访问稀酷客(www.ckook.com)

的 fsstat 程序，我们可以看到关于这个分区的更多信息：

```
ahoog@ubuntu:~$ sudo fsstat /dev/sr2

=== PRIMARY VOLUME DESCRIPTOR 1 ===
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: ISO9660
Volume Name: Verizon Mobile
Volume Set Size: 1
Volume Set Sequence: 1
Publisher: Publisher
Data Preparer: Publisher
Recording Application: Application
Copyright:

METADATA INFORMATION
--------------------------------------------
Path Table Location: 23-23
Inode Range: 0 - 9
Root Directory Block: 26

CONTENT INFORMATION
--------------------------------------------
Sector Size: 2048
Block Size: 2048
Total Sector Range: 0 - 2383
Total Block Range: 0 - 2383
=== SUPPLEMENTARY VOLUME DESCRIPTOR 1 ===
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: ISO9660
Volume Name:
Volume Set Size: 1
Volume Set Sequence: 1
Publisher:
Data Preparer: Publisher
Recording Application:
Copyright:


METADATA INFORMATION
--------------------------------------------
Path Table Location: 25-25
Root Directory Block: 29
Joliet Name Encoding: UCS-2 Level 1

CONTENT INFORMATION
--------------------------------------------
Sector Size: 2048
Block Size: 2048
Total Sector Range: 0 - 2383
Total Block Range: 0 - 2383
```

正像你从 Volume Name 看到的那样，这个 CD-ROM 里包含有 Verizon 提供的软件，用于使用终端设备中的其他功能。

2. SD 卡（可移走的、虚拟的）

按照取证分析的观点，从终端设备可以访问到的 SD 卡是非常重要的。存放用户文

更多资源请访问稀酷客(www.ckook.com)

件，尤其是数据量大的多媒体文件，是 Android 的一个重要策略。大多数的 Android 终端设备都有一个可插入介质的接口，能够插入迷你 SD 卡。关键的应用程序数据通常保存在终端设备上（在 /data/data 文件夹中），但是对于取证调查来说重要的文件也许存储在 SD 卡上。

根据前面一节的介绍，当一个 Android 终端设备通过 USB 进行连接时，Linux 工作站的内核信息会显示出可以访问的各类 USB 设备。这两个列出的、可取走的 SCSI 磁盘（即 sdb 和 sdc）显示出 HTC Incredible 上的 SD 卡。如果你在 Connect to PC（连接到计算机）中选择“Mount as disk drive”（作为磁盘挂载），以下更多的信息显示在内核信息上：

```
[  325.669335] sd 4:0:0:1: [sdc] 3911680 512-byte logical blocks: (2.00 GB/
1.86 GiB)
[  325.672039] sd 4:0:0:1: [sdc] Assuming drive cache: write through
[  325.678282] sd 4:0:0:1: [sdc] Assuming drive cache: write through
[  325.678294]  sdc: sdc1
[  327.671951] sd 4:0:0:0: [sdb] 13844464 512-byte logical blocks: (7.08 GB/
6.60 GiB)
[  327.674074] sd 4:0:0:0: [sdb] Assuming drive cache: write through
[  327.679387] sd 4:0:0:0: [sdb] Assuming drive cache: write through
[  327.679395]  sdb:
```

你将看到更多的关于 SD 卡的信息，如磁盘 sdc 有一个分区 sdc1，大小为 2GB。我们还可以通过执行 TSK 的 mmls 命令看到更多的关于分区的信息：

```
ahoog@ubuntu:~$ sudo mmls /dev/sdc
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot    Start        End          Length       Description
00:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
01:  -----   0000000000   0000000128   0000000129   Unallocated
02:  00:00   0000000129   0003911679   0003911551   DOS FAT16 (0x06)
```

正如你所看到的，SD 卡的文件格式是 FAT16 文件系统，但是你还经常会看到 FAT32，或许也会看到有多个文件系统，如 FAT32 以及本地 Linux 文件系统 ext3 和 ext4。

近期，终端设备还具有了拥有仿真或虚拟 SD 卡的功能，即在终端设备的 NAND 闪存上创建一个不可拔出的 SD 卡。此功能仿照的是 iPhone。在 iPhone 中，用户数据分区直接在 NAND 闪存上，不能移走。在上例中，adb 设备提供对虚拟 SD 卡的访问。与实际（物理）的 SD 卡有所不同的是，sdc 没有分区表，文件系统立即启动。要想看到重要信息，运行 TSK 的 fsstat：

```
ahoog@ubuntu:~$ sudo fsstat /dev/sdb
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: FAT32

OEM Name: BSD  4.4
Volume ID: 0xc7f80810
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory):
File System Type Label: FAT32
Next Free Sector (FS Info): 562580
```

更多资源请访问稀酷客(www.ckook.com)

```
Free Sector Count (FS Info): 13376448

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 13844463
* Reserved: 0 - 31
** Boot Sector: 0
** FS Info Sector: 1
** Backup Boot Sector: 2
* FAT 0: 32 - 1721
* FAT 1: 1722 - 3411
* Data Area: 3412 - 13844463
** Cluster Area: 3412 - 13844435
*** Root Directory: 3412 - 3475
** Non-clustered: 13844436 - 13844463
METADATA INFORMATION
--------------------------------------------
Range: 2 - 221456838
Root Directory: 2

CONTENT INFORMATION
--------------------------------------------
Sector Size: 512
Cluster Size: 32768
Total Cluster Range: 2 - 216267

FAT CONTENTS (in sectors)
--------------------------------------------
3412-3475 (64) -> EOF
3476-3539 (64) -> EOF
3540-5267 (1728) -> EOF
5268-7379 (2112) -> EOF
<snip>
```

在此特殊案例中，文件系统果真是 FAT32，同时你也会注意到此卷没有 Label(标号)，OEM Name 是 set BSD 4.4。

> **警示：自动装入 USB 设备**
>
> 在第 1 章的 Ubuntu 虚拟机配置部分，自动挂载功能被禁止，为的是防止操作系统自动检测并挂载 USB 大容量存储设备。取证分析师需要十分谨慎地去防止发生这种情况。此外，终端设备通常应该通过一个 USB 写阻止器来进行连接。

在 Ubuntu 中，如果你没有关闭 USB 自动挂载（基本上在所有情况下，你都应该关闭），SD 卡将会被自动挂载。如果终端设备依附在一个硬件的写阻止器上，或是只能读取地挂载，或是在写阻止器不需要的情况下（例如，研究与开发），在 Linux 中，你可以运行 df 命令来看到这些终端设备在哪里被挂载：

```
ahoog@ubuntu:~$ df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/sda1              19G  3.4G   15G  19% /
none                  369M  228K  369M   1% /dev
none                  375M  252K  375M   1% /dev/shm
```

更多资源请访问稀酷客(www.ckook.com)

```
none                    375M  100K  375M   1% /var/run
none                    375M     0  375M   0% /var/lock
.host:/                 931G  663G  269G  72% /mnt/hgfs
/dev/sdc1               1.9G  200M  1.7G  11% /media/E0FD-1813 (physical
                                               2GB SD Card)
/dev/sdb                6.6G  227M  6.4G   4% /media/C7F8-0810 (emmulated SD Card)
```

物理 SD 卡挂载在 /media/E0FD-1813，仿真 SD 卡挂载在 /media/C7F8-0810。

在 Android 终端设备上，以上两个 SD 卡挂载的情况如下：

```
/dev/block/vold/179:9 /mnt/sdcard vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=0702,
allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
/dev/block/vold/179:3 /mnt/emmc vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=0702,
allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
```

3. USB 调试

最后一个，同时也是十分重要的 USB 接口就是 Android 程序调试桥（Android Debug Bridge，ADB），它使开发者、取证分析师和安全工程师可以通过 USB 与终端设备进行通信，并控制此终端设备。在默认情况下，Android 虚拟终端设备（运行在仿真器中）的 USB 调试是开启的。然而，非仿真的终端设备必须明确地开启 USB 调试。要开启此功能，在终端设备的 Setting 中选择 Applications → Development，如图 3.30 所示，然后选择 USB debugging（USB 调试）。

在设置后，终端设备将在后台运行 adb 后台程序（adbd），等待 USB 连接。此后台程序将在无权力的用户账号下运行，以限制它对数据的访问。开启了 root 访问权限的 Android 虚拟终端设备和实际的终端设备将使运行的 adbd 具有 root 访问权限，可以对系统进行任何访问。进一步的细节将在第 6 章中进行讨论。

在新版本的 Android 中，在任何时候，当一个 USB 调试开启的终端设备通过 USB 连接时，它会显示一个安全警告信息，如图 3.31 所示。

图 3.30　启用 USB 调试

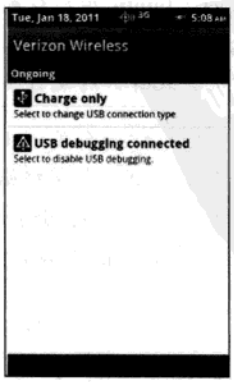


图 3.31　USB 调试警告

更多资源请访问稀酷客(www.ckook.com)

对于目前所有的 Android 取证工具，USB 调试必须被开启。虽然在终端设备没有被锁住的情况下很容易做的，但是如果终端设备上设置了密码则变得非常困难。第 6 章中将会谈到一些绕过密码的技术，但是并不是对所有的平台都适用。

3.5.3　Android 程序调试桥简介

在此书的后续章节中，我们会大量地使用 Android 程序调试桥（android debug bridge，adb），因此在这里进行介绍十分必要。Android 程序调试桥（adb）的使用涉及以下 3 个主要部件：

1）在 Android 终端设备上运行的 adbd；

2）在工作站上运行的 adbd；

3）在工作站上运行的 adb 客户端程序。

在前面谈到，当在一个 Android 终端设备上开启了 USB 调试，后台程序将会运行并会对连接进行监听。运行在终端设备上的 adbd 和运行在工作站上的 adbd 之间的通信都发生在运行在此 USB 连接之上的虚拟网络。后台程序之间使用本地端口 5555 到 5585 进行通信。当工作站的 adbd 检测到一个新的仿真器或终端设备，它将创建两个连续的端口连接。偶数端口与终端设备的控制台进行通信，而奇数端口则用于建立端口间的连接。本地的 adbd 客户端程序使用端口 5037 同本地的 adbd 进行通信。

能够使用的最基本的 adb 命令是 adb devices，用于列出已经连接的终端设备：

```
ahoog@ubuntu:~$ adb devices
List of devices attached
HT08XHJ00657    device
```

另外一个重要的命令提供停止本地 adb 服务的能力。为了达到此目的，输入以下命令行：

```
ahoog@ubuntu:~$ adb kill-server
ahoog@ubuntu:~$ adb devices
* daemon not running. starting it now on port 5037 *
* daemon started successfully *
List of devices attached
HT08XHJ00657    device
```

从上可见，如果在工作站上的 adbd 目前没有正在运行，它将被自动开启。在 Ubuntu 上，如果你看到以下信息：

```
ahoog@ubuntu:~$ adb devices
List of devices attached
???????????    no permissions
```

非常可能的是，连接的 Android 终端设备具有一个必须进行识别的制造商 ID（sudo lsusb –v），并需要被加入到 udev 规则中，正如我们在 3.3 节中讨论到的。在微软的 Windows 中，如果 Android 终端设备无法被识别，你将会得到一个警示，而必须安装合适的由谷歌或厂商提供的 USB 驱动程序。

更多资源请访问稀酷客(www.ckook.com)

所有的分析师和工程师都应该知道的另外一个功能强大的 adb 命令是 adb shell，它允许你在 Android 终端设备上开启一个 shell 与系统进行交互。对于所有探讨 Android 的人来说，这是一个重要的功能。举例来说，开启一个 Android 虚拟终端设备，按照以下的步骤来查看存储在终端设备上的应用程序的数据目录：

```
ahoog@ubuntu:~$ adb shell
# cd /data/data
# ls
com.android.sdksetup
com.android.calculator2
com.android.packageinstaller
com.android.providers.userdictionary
com.android.development
com.android.soundrecorder
com.android.providers.drm
com.android.spare_parts
com.android.providers.downloads.ui
com.android.protips
com.android.fallback
com.android.browser
com.android.providers.applications
com.android.netspeed
com.android.wallpaper.livepicker
android.tts
com.android.htmlviewer
com.android.music
com.android.certinstaller
com.android.inputmethod.pinyin
com.android.providers.subscribedfeeds
com.android.inputmethod.latin
com.android.gallery
com.android.systemui
com.android.contacts
com.android.camera
com.android.term
com.android.speechrecorder
com.android.server.vpn
com.android.quicksearchbox
com.android.defcontainer
com.svox.pico
com.android.customlocale
com.android.providers.settings
com.android.settings
com.android.providers.contacts
jp.co.omronsoft.openwnn
com.android.phone
com.android.launcher
com.android.providers.telephony
com.android.mms
com.android.providers.media
com.android.providers.downloads
com.android.deskclock
com.android.email
```

在每一个新版的 Android 软件开发套件中 adb 的功能都得到增强，它已经成为一个功能强大的工具。我们将在第 6 章中详细探讨某些重要功能，主要包括以下功能：

1）在终端设备上运行 shell 命令；

更多资源请访问稀酷客(www.ckook.com)

2）在命令行安装应用程序；

3）工作站与终端设备之间的端口转发；

4）从终端设备上来去地递归复制文件和文件夹；

5）查看终端设备的日志文件。

关于 adb 命令的所有文档都可以从以下 Android 开发者网站获得：

http://developer.android.com/guide/developing/tools/adb.html#commandsummary.

在使用工具进行取证调查之前，使用 Android 仿真器去测试工具中各式各样的命令对于理解工具是一个非常好的办法。

3.6　本章小结

Android 软件开发套件不但提供了 Android 平台深入的信息，而且从取证和安全角度均为调查一个终端设备提供了功能强大的工具。在一台取证工作站上安装了 Android 软件开发套件后，取证分析师就具备了与一部通过 USB 接口连接的 Android 终端设备进行交互的能力，只要启动了 USB 程序调试功能。我们不但可以从终端设备获取信息，而且可以安装和运行应用程序，最终实现从终端设备中抽取数据。Android 软件开发套件是一个用于取证和安全分析的重要工具。

3.7　参考资料

Android timeline. (n.d.). Android tutorials, news, views and forums, Android Academy. Retrieved March 12, 2011, from http://www.androidacademy.com/1-android-timeline.

Platform Versions, (n.d.). Android developers. Retrieved March 12, 2011, from http://developer.android.com/resources/dashboard/platform-versions.html.

comScore Reports November 2010 U.S. Mobile Subscriber Market Share—comScore, Inc. (n.d.). comScore, Inc.—Measuring the digital world. Retrieved March 12, 2011, from http://www.comscore.com/Press_Events/Press_Releases/2011/1/comScore_Reports_November_2010_.

Android 1.5 Platform. (n.d.). Android developers. Retrieved March 12, 2011, from http://developer.android.com/sdk/android-1.5.html.

Android 1.6 Platform. (n.d.). Android developers. Retrieved March 12, 2011, from http://developer.android.com/sdk/android-1.6.html.

Android 2.1 Platform. (n.d.). Android developers. Retrieved March 12, 2011, from http://developer.android.com/sdk/android-2.1.html.

SDK Archives. (n.d.). Android developers. Retrieved March 13, 2011, from http://developer.android.com/sdk/older_releases.html.

smali-Project Hosting on Google Code. (n.d.). Google code. Retrieved March 13, 2011, from http://code.google.com/p/smali/.

Android On Lockdown: AT&T Removes Best Parts of Android from Backflip (n.d.). AndroidGuys. The trusted source for Android news and opinion, Est. 2007. Retrieved March 13, 2011, from http://www.androidguys.com/2010/03/08/android-lockdown-att-removes-parts-android-backflip/.

更多资源请访问稀酷客(www.ckook.com)

第 4 章

Android 文件系统与数据结构

本章要点

- shell 中的数据
- 内存类型
- 文件系统
- 挂载的文件系统和目录结构

4.1 绪论

因为运行 Android 终端设备的底层硬件和软件都非常出色，所以取证分析师和安全工程师的主要工作重点是获取、分析及理解终端设备中所存储的数据。就像所讨论的其他问题一样，为了进行有效的分析，有很多重要的细节需要去了解，包括所存储数据的类型、数据存储的位置、以什么样的方式进行存储以及存储这些数据的媒体介质的特性等。在什么样的数据能够被恢复以及如何进行分析中，所有以上这些因素都起着主要的作用。

4.2 shell 中的数据

取证分析师关心的是他们从终端设备中能够恢复的用于调查的数据衍生品。Android 是一个广泛熟知的产品（如 Linux 中的功能）与完全崭新的产品（如 Dalvik 虚拟机和 YAFFS2 文件系统）的组合。不同制造商所采用的不同的体系架构又增加了一定的复杂度。

虽然没有任何数据让检查者能够完整地描述此方面的内容，但 Android 终端设备共享着一些基本的概念和原理。文件系统、文件及其他一些衍生品是取证分析师为了使取证调查取得最好效果而需要了解的 Android 方面的核心内容。

4.2.1 存储的数据

Android 终端设备存储着大量的数据，而且通常是个人数据和工作数据的组合。应用程序是产生这些数据的源泉，而应用程序又来源于很多不同方面，包括以下来源：

- Android 自带的应用程序；

更多资源请访问稀酷客(www.ckook.com)

- 由制造商安装的应用程序；
- 由无线移动服务提供商安装的应用程序；
- 更多的谷歌 /Android 应用程序；
- 由用户自己安装的应用程序，通常来源于 Android 市场（Android Market）。

在第 7 章（Android 应用与取证分析）中，我们会深入地去研究以上的某些应用程序，虽然试图涵盖所有可能的应用程序肯定已经超出了本书的范围。在 Android 终端设备上存在的抽样数据如下：

- 文本数据（短信 / 彩信）；
- 联系人；
- 通信日志；
- 邮件数据（Gmail、Yahoo、Exchange）；
- 短信 / 聊天；
- 全球定位系统（GPS）坐标；
- 相片 / 视频；
- 上网记录；
- 搜索记录；
- 驾驶方向；
- Facebook、Twitter 及其他社交媒体客户端；
- 存储在终端设备上的文件；
- 收藏的音乐；
- 约会日程；
- 财务信息；
- 购物记录；
- 共享文件。

4.2.2　应用数据存储目录结构

Android 应用程序主要将数据存储在两个地方：内部存储器和外部存储器，我们会在本章中详细讨论这两种情况。然而，首先对它们的存储目录结构进行一个总体了解非常有帮助。

在外部存储器区域（SD 卡及仿真 SD 卡），应用程序可以选择在任何希望的区域存储数据。但是，内部存储器通过 Android 应用编辑接口（API）进行控制。当安装一个应用程序（通过直接从市场上得来或者发送给消费者的软件）时，系统在 /data/data/ 子目录下使用这个应用程序包的名字为此应用程序建立一个内部存储器。例如，Android 浏览器程序包的默认名字为 com.android.browser，因此，数据文件将会被存储在子目录 /data/data/com.android.browser 下。虽然系统没有要求应用程序必须存储数据文件，但大多数应用都会这样做。

更多资源请访问稀酷客(www.ckook.com)

在应用程序的 /data/data 子目录中，对很多应用程序来说，有一些标准目录以及由开发者控制的目录。最常见的标准目录见表 4.1。

表 4.1 公共 /data/data/< 软件包名称 > 子目录

shared_prefs	以 XML 格式存储的共享优先目录
lib（共享程序库）	应用程序所需的自定义库文件
files（文件）	开发者存储在内部存储器上的文件
cache（缓存）	应用程序缓存文件，通常是使用 Webkit 引擎开发的 Web 浏览器或其他应用程序运行中的缓存文件
databases（数据库）	SQLite 数据库和日志文件

表 4.1 所列出的是最常见的应用程序的 /data/data 子目录中的文件。随着我们在本书中对数据进行更深入的讨论，我们会分出很多其他的文件夹及数据文件。

4.2.3 数据如何存储

Android 给开发者提供了 5 种在终端设备上存储数据的方式。取证分析师能够从至少 4 种数据格式的文件中获得数据。因此，理解这 4 种数据格式非常重要。

持久的数据存储在 NAND 闪存、SD 卡或者网络上。具体来说，这 5 种方式如下。

1）共享优先；

2）内部存储；

3）外部存储；

4）SQLite；

5）网络。

除了应用程序开发者存储的数据之外，Linux 内核和 Android 栈也通过日志、程序调试以及其他的标准信息服务提供信息。

1. 共享优先

共享优先使开发者能够将基础数据类型的变量定义 - 数值二元组以轻量级的 XML 格式进行存储。可以存储在优先文件中的基础数据类型如下：

1）boolean：正确或错误；

2）float：单精度 32 位 IEEE 754 浮点数；

3）int：32 位正负互补整数；

4）long：64 位正负互补整数；

5）strings：字符串，通常采用 UTF-8 编码。

有了这些基本的数据类型，开发者就可以创建并保存其开发的应用程序所需要的简单数值。

共享优先文件通常存储在应用程序的数据目录中的 shared_pref 文件夹中，文件的后缀是 .xml。在我们所参考的 HTC Incredible 手机上，Android 手机共享优先目录有 5 个 XML 文件：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~/data/data/com.android.phone/shared_prefs$ ls -l
total 20
-rw-r----- 1 ahoog ahoog 104 2011-01-23 18:05 cdma_msg_id.xml
-rw-r----- 1 ahoog ahoog 214 2011-01-20 09:34 com.android.phone_preferences.xml
-rw-r----- 1 ahoog ahoog 126 1980-01-06 09:42 _has_set_default_values.xml
-rw-r----- 1 ahoog ahoog 152 2010-09-10 09:46 htc_cdma_settings.xml
-rw-r----- 1 ahoog ahoog 102 2010-09-10 09:48 updateAreaCode.xml
```

com.android.phone preferences.xml 优先文件所包含的优先数据类型有 int、bllolen 和 string：

```
ahoog@ubuntu:~/data/data/com.android.phone/shared_prefs$ cat
com.android.phone_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
<int name="vm_count_key_cdma" value="0" />
<boolean name="pref_key_save_contact" value="true" />
<string name="vm_number_key_cdma">*86</string>
</map>
```

可以看到，XML 在文件的开始对字符串的编码类型进行了描述，此例中为 UTF-8。文件中有 3 个优先文件用以保存不同的设置及其性质。从取证分析的角度来看，最有趣的就是 updateAreaCode.xml：

```
ahoog@ubuntu:~/data/data/com.android.phone/shared_prefs$ cat updateAreaCode.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
<string name="MDN">312</string>
</map>
```

在此文件中，对移动目录号（MDN）提出查询并将区号存储在此文件中，推测就是在支持此功能的地区允许使用 7 位号码进行拨号。

由于很多应用程序都利用轻量级的共享优先方法来存储变量定义 - 数值二元组，它也许成为取证数据的丰富来源。当检查者能够恢复过去的或已经被删除的 XML 优先文件，这一点尤为明显。

2. 内部存储器上的文件

开发者可以在文件中存储更复杂的数据结构，并可以将文件存储在内部存储器的几个不同地方。这些文件均存储在应用程序的 /data/data 子目录中，开发者具有控制文件类型、名称和地点的能力。在默认的情况下，这些文件只能被所属的应用程序去读取，甚至终端设备的拥有者也不允许去查看这些文件的内容，除非他们拥有 root 访问权限。然而，开发者可以越过这种安全设置允许其他进程去读取，甚至去更新这些文件。

> **提示：识别专用文件**
>
> 确定应用程序的子目录中的哪些文件属于这种类型的最好方法是使用一个排除过程。原则上，任何在应用程序的 /data/data/ 子目录下，不属于 shared_prefs、lib、cache 及 databases 子目录中的文件都是由开发者创建和控制的文件。

更多资源请访问稀酷客(www.ckook.com)

作为一个很好地显示保存在内部存储器上的文件的实例，现在让我们来看看 com.google.android.apps.maps：

```
ahoog@ubuntu:~/data/data/com.google.android.apps.maps$ ls -l
total 24
drwxr-x--x 5 ahoog ahoog 4096 2011-01-18 03:42 app_
drwxr-x--x 3 ahoog ahoog 4096 2010-09-15 10:59 cache
drwxr-x--x 2 ahoog ahoog 4096 2011-01-23 10:30 databases
drwxr-x--x 2 ahoog ahoog 4096 2011-01-23 20:55 files
drwxr-xr-x 2 ahoog ahoog 4096 1980-01-06 09:41 lib
drwxr-x--x 2 ahoog ahoog 4096 2011-01-24 04:13 shared_prefs
```

此应用程序使用了大多数可用的存储方式，并将文件存储在内部存储器上的 app_ 和 files 子目录中。app_ 目录中还有几个子目录以及一个格式不明的 cache_r.m 文件：

```
ahoog@ubuntu:~/data/data/com.google.android.apps.maps$ tree app_/
app_/
├── cache
│   └── cache_r.m
├── debug
└── testdata
```

这个文件目录里存储着应用程序需要对 Google Maps 进行显示和更新的很多文件：

```
ahoog@ubuntu:~/data/data/com.google.android.apps.maps$ tree files
files
├── DA_DirOpt_en_US
├── DA_LayerInfo
├── DATA_LATITUDE_WIDGET_MODEL
├── DATA_LAYER_10
├── DATA_LAYER_2
├── DATA_LAYER_20
├── DATA_LAYER_21
├── DATA_LAYER_24
├── DATA_LAYER_25
├── DATA_LAYER_3
├── DATA_LAYER_5
├── DATA_LAYER_6
├── DATA_LAYER_7
├── DATA_LAYER_8
├── DATA_location_history
├── DATA_OptionDefinitionBlock_en
├── DATA_Preferences
├── DATA_PROTO_SAVED_CATEGORY_TREE_DB
├── DATA_PROTO_SAVED_LAYER_STATE
├── DATA_PROTO_SAVED_RECENT_LAYERS
├── DATA_RemoteStringsBlock_en
├── DATA_Restrictions
├── DATA_Restrictions_lock
├── DATA_SAVED_BGFS_3
├── DATA_SAVED_BGFS_EXTRA_3
├── DATA_SAVED_BGSF_
├── DATA_SAVED_REMOTE_ICONS_DATA_BLOCK
├── DATA_ServerControlledParametersManager.data
├── DATA_STARRING
├── DATA_SYNC_DATA
├── DATA_SYNC_DATA_LOCAL
├── DATA_TILE_HISTORY
├── DATA_Tiles
```

更多资源请访问稀酷客(www.ckook.com)

```
├── DATA_Tiles_1
├── DATA_Tiles_2
├── DATA_Tiles_3
├── event_store_driveabout
├── event_store_LocationFriendService
├── NavigationParameters.data
├── NavZoomTables.data
├── nlp_GlsPlatformKey
├── nlp_state
└── ZoomTables.data
```

这些文件很清楚地指示出取证分析师和安全工程师可能感兴趣的数据。第 7 章将更加深入地讨论如何对应用程序及所存储的数据进行数据分析。

3. 外部存储器上的文件

存储在内部存储器上的文件具有很严格的安全和位置参数，然而存储在各类外部存储器上的文件却具有极少的限制。

首先，一个使用可移走的 SD 卡的重要动机（除成本外）是可以将数据在其他终端设备上使用，尤其是升级版的 Android 终端设备。如果消费者购买了一款新的 Android 终端设备，插入了上有家庭成员相片以及视频的之前使用的 SD 卡，但是却发现无法对它们进行访问，他们会非常失望。

为了实现将 SD 卡接入台式机，以进行文件共享，SD 卡通常按照 Windows FAT32 文件系统进行了格式化。虽然此文件系统获得广泛支持，它却缺少比如 ext3、ext4、yaffs2 等文件系统中存在的细化安全机制。因此，在默认的情况下，这些文件无法强制访问控制权限。

例如，此前使用过的 com.google.android.apps.maps 应用还将数据存储在 SD 卡上的 Android/data 子目录之下。以下是存储在供参考用的 HTC Incredible 手机上的 SD 卡中的文件和目录列表，此卡挂载在 /mnt/sdcard 上：

```
ahoog@ubuntu:~/htc-inc/mnt/sdcard/Android/data$ tree com.google.android.apps.maps/
com.google.android.apps.maps/
├── cache
│   ├── cache_its.0
│   ├── cache_its.m
│   ├── cache_its_ter.m
│   ├── cache_r.0
│   ├── cache_r.1
│   ├── cache_rgts.0
│   ├── cache_rgts.m
│   ├── cache_r.m
│   ├── cache_vts.0A
│   ├── cache_vts.1
│   ├── cache_vts_GMM.0
│   ├── cache_vts_GMM.1
│   ├── cache_vts_GMM.m
│   ├── cache_vts.m
│   └── cache_vts_tran_GMM.m
├── debug
└── testdata
```

同样，HTC Incredible 售出时具有一个直接存储在 NAND 闪存上的仿真 SD 卡。以下

更多资源请访问稀酷客(www.ckook.com)

是一个存储着关于 JFIF thumbnail 专辑文件的子目录（文件夹）：

```
ahoog@ubuntu:~/htc-inc/mnt/emmc$ tree Android/data/com.android.providers.media/
└── albumthumbs
    ├── 1283015214003
    ├── 1283015215018
    ├── 1283015215425
    ├── 1283015215861
    ├── 1283015216304
    └── 1283015216711
```

正如所看到的，开发者对于存储在外部及仿真 SD 卡上的文件名、文件格式以及存储地方具有很强的控制能力。

4. SQLite

另一个基于 NAND/SD 卡的、为开发者使用的存储系统是一种具体类型的文件：SQLite 数据库。数据库用于存储结构化的数据，SQLite 是一个非常流行的数据库格式，应用于很多移动系统以及传统的操作系统中。

有很多原因使得 SQLite 非常流行。最特别的就是整个代码不但质量很高，而且是开放源码，向公共领域发布。文件格式与程序非常精炼，区区几十万行的代码就能够提供重要的功能。与传统的关系数据库管理系统（RDBMS），如 Oracle、MySQL 及微软的 SQL Server 有所不同，SQLite 的整个数据库全部包含在一个单独的跨平台文件中。

Android 软件开发套件（SDK）通过提供专门的应用编程接口（API）允许开发者在他们的应用程序中使用 SQLite 数据库。SQLite 文件通常存储在内部存储器的 /data/data/<应用程序名称 >/databases 下。但是，对于在其他地方创建数据库没有设置任何限制。

SQLite 数据库是取证数据的丰富来源。基于 WenKit 开放源码项目（Open Source Project，http://webkit.org/）开发的嵌入在 Android 中的浏览器是一个很好的实例。在 HTC Incredible 手机中，/data/data/com.android.webkit 子目录下有 28 个 SQLite 数据库。此例中，以下是 5 个子目录：

- app_icons：一个用于网页图标的数据库；
- app_cache：一个用于 Web 应用数据缓存的数据库；
- app_geolocation：两个与全球定位系统（GPS）的位置和访问权限相关的数据库；
- app_databases：21 个用于为网站提供存储本地数据库的数据库；
- databases：3 个用于浏览器和浏览器缓存的数据库。

从这些文件中恢复有价值的数据具有很大的可能性。

5. 网络

最后一个开发者可以使用的数据存储机制就是网络，也是将终端设备设计成具有网络感知能力（network aware）所得到的一个重要好处。最初，很少的应用程序选择网络作为数据存储的一个选项。然而，随着软件开发套件、应用程序以及终端设备技术日渐成熟，网络存储变得越来越普遍。

Android 开发者网站为感兴趣的人提供了非常少的关于网络存储的详细信息。整个文

更多资源请访问稀酷客(www.ckook.com)

档只不过屈屈两句话而已（Data storage, n.d.）。

你可以在网络上（如果可以访问的话）存储和读取基于 Web 服务的数据。如果要进行网络操作，使用以下软件包中的类：

- java.net.*
- android.net.*

这些在文档中提到的软件包主要在底层为开发者提供与网络和 Web 服务器等进行交互的应用编程接口。使用网络的应用程序需要更多的专用代码，虽然并不是所有的取证相关数据都存储在终端设备上，而常见的是，重要的配置和数据库数据都可以得到恢复。

举例来说（作为第 7 章的“前锋”），Dropbox 是一个非常流行的文件共享网站，上面有适用于 Android、黑莓、iOS 终端设备的应用程序。它目前的 Android 应用程序（1.0.3.0 版）已经从 Android 市场被下载超过 25 万次，并有超过 3.5 万用户对其进行推荐排名，大多数还很靠前。在安装了此应用程序后，你可以看到在一个应用程序文件夹 /data/data/com.dropbox.android 下有 4 个标准的目录：

```
ahoog@ubuntu:~/htc-inc/data/data$ tree com.dropbox.android/
com.dropbox.android/
├── databases
│   └── db.db
├── files
│   └── log.txt
├── lib
└── shared_prefs
    └── DropboxAccountPrefs.xml
```

log.txt 是一个关于所有活动的详细日志，还有其余用于参考的少数几行：

```
ahoog@ubuntu:~/com.dropbox.android$ cat ./files/log.txt
5 1296055108427 com.dropbox.android.provider.DatabaseHelper Creating new Dropbox
database.
4 1296055108459 com.dropbox.android.DropboxApplication Not authenticated, so
authenticating
4 1296055108466 com.dropbox.android.DropboxApplication No stored login token.
4 1296055108702 com.dropbox.android.DropboxApplication Not authenticated, so
authenticating
4 1296055108704 com.dropbox.android.DropboxApplication No stored login token.
4 1296055108704 com.dropbox.android.activity.SimpleDropboxBrowser Didn't
authenticate, redirecting to login
4 1296055108713 com.dropbox.android.DropboxApplication Not authenticated, so
authenticating
4 1296055108714 com.dropbox.android.DropboxApplication No stored login token.
4 1296055134550 com.dropbox.android.DropboxApplication Authenticating username:
book@viaforensics.com
4 1296055136507 com.dropbox.android.DropboxApplication Successfully
authenticated
6 1296055137501 com.dropbox.android.activity.LoginActivity Dismissed nonexistent
dialog box
4 1296055137525 com.dropbox.android.activity.LoginOrNewAcctActivity Successful
account login
4 1296055137549 com.dropbox.android.activity.delegate.MenuDelegate Successful
login
4 1296055137735 com.dropbox.android.activity.SimpleDropboxBrowser Query is:
content://com.dropbox.android.Dropbox/metadata/
```

更多资源请访问稀酷客(www.ckook.com)

```
6 1296055137742 com.dropbox.android.provider.QueryStatus Querying with query
id: DB2
4 1296055137765 com.dropbox.android.activity.SimpleDropboxBrowser Browsing
URI: content://com.dropbox.android.Dropbox/metadata/
4 1296055138208 com.dropbox.android.provider.ProviderDirSyncThread Directory
changed, going through line-by-line:
content://com.dropbox.android.Dropbox/metadata/
4 1296055161450 com.dropbox.android.activity.delegate.MenuDelegate Importing
Picture from Gallery
6 1296055170307 com.dropbox.android.provider.DropboxProvider Adding new file
(from import, probably): content://media/external/images/media/5
4 1296055170329 com.dropbox.android.taskqueue.TaskQueue Added task to queue:
content://media/external/images/media/5~/
4 1296055170333 com.dropbox.android.taskqueue.TaskQueue Starting up task queue
4 1296055170333 com.dropbox.android.taskqueue.UploadTask Uploading file from
URI: content://media/external/images/media/5
4 1296055170333 com.dropbox.android.taskqueue.DbTaskQueue Task
content://media/external/images/media/5~/ adding to task DB
6 1296055170351 com.dropbox.android.service.ServiceBinderDelegate Unbound
service!
4 1296055170352 com.dropbox.android.taskqueue.UploadTask Uploading file:
/mnt/sdcard/forensics/20110111.1618/387.jpg to / as 387.jpg
4 1296055170367 com.dropbox.android.activity.SimpleDropboxBrowser Browsing URI:
content://com.dropbox.android.Dropbox/metadata/
4 1296055170471 com.dropbox.android.service.DropboxReceiver Connectivity action:
mobile CDMA - EvDo rev. A 2GVoiceCallEnded
4 1296055170471 com.dropbox.android.service.DropboxReceiver Connectivity change!
true
4 1296055170472 com.dropbox.android.taskqueue.DbTaskQueue Adding Uploads from
stored db: 1
```

以上列表中的某些内容具有潜在价值，具体来说：

1）所有的操作都有时间戳；

2）提供成功的用户认证以及用户名；

3）从 Gallery 引入的图像；

4）SD 卡上的特定文件被上传；

5）Dropbox 服务被电话服务给中断。

此应用程序还有一个共享优先文件：

```
ahoog@ubuntu:~/htc-inc/data/data/com.dropbox.android$ cat
shared_prefs/DropboxAccountPrefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
<string name="LAST_URI">content://com.dropbox.android.Dropbox/metadata/</string>
<string name="DISPLAY_NAME">Andrew Hoog</string>
<long name="QUOTA_QUOTA" value="2147483648" />
<long name="QUOTA_NORMAL" value="1480890" />
<string name="REFERRAL_LINK">https://www.dropbox.com/referrals/NNNNAN0NnwNNN
</string>
<string name="COUNTRY"></string>
<long name="UID" value="96189742" />
<string name="EMAIL">book@viaforensics.com</string>
<string name="ACCESS_KEY">accesskeyinfohere</string>
<string name="ACCESS_SECRET">accesssecretinfohere</string>
<long name="QUOTA_SHARED" value="0" />
</map>
```

更多资源请访问稀酷客(www.ckook.com)

通过对 db.db 的快速检查可以发现使用 sqlite3 命令行程序的数据（你也可以使用一个图像界面的 SQLite 浏览器去观看数据库）：

```
ahoog@ubuntu:~/htc-inc/data/data/com.dropbox.android/databases$ sqlite3 db.db
SQLite version 3.6.22
Enter ".help" for instructions
Enter SQL statements terminated with a ";"

sqlite> .tables
android_metadata  dropbox           pending_upload

sqlite> .mode line

sqlite> select * from dropbox where _id = 2;
           _id = 2
         _data = /sdcard/dropbox/Android intro.pdf
      modified = Wed, 26 Jan 2011 15:18:40 +0000
         bytes = 176607
      revision = 10
          hash =
          icon = page_white_acrobat
        is_dir = 0
          path = /Android intro.pdf
          root =
          size = 172.5KB
     mime_type = application/pdf
  thumb_exists = 0
   parent_path = /
 _display_name = Android intro.pdf
   is_favorite =
local_modified = 1296055191000
   local_bytes = 176607
local_revision = 10
      accessed =
   sync_status = 2
```

数据库为有关 Dropbox 应用、终端设备以及用户和与其交互的其他人提供了取证分析和安全相关的重要数据。当 Android 应用程序完成安装并登录时，Dropbox 自动地将 Android intro.pdf 文件与 Dropbox 账号进行同步。当一个共享的 PDF 文件被打开时，它会被缓存在 SD 卡上。更多的关于此文件的元数据以及对其的使用都包含在数据库中。尽管 Dropbox 在应用程序中广泛地使用网络存储，我们仍然可以恢复有用的信息。

6. 内核、系统及应用日志

另外一个取证分析师和安全工程师能够找到与调查或审计相关联的文件和信息的区域就是标准的 Linux 文件系统。很不幸，这是一件非常宽泛和不可控制的事情，但是却至少为我们提供了一个获取相关信息的起点。

日志文件与程序调试是开发者和系统管理者维护系统以及应用程序的两个常用并且有效的方法，能够提供有关应用程序和运行这些应用程序的系统的内部情况。虽然并不是在所有的情况下都是如此，依然有可能通过检查各种日志和调试文件从 Android 终端设备上获得重要信息。

更多资源请访问稀酷客(www.ckook.com)

（1）Linux 内核日志

Linux 内核是 Linux 操作系统底层和抽象的界面，为终端设备中的硬件提供访问。由于内核扮演着终端设备所有功能的中心的角色，其对关键事件和活动进行日志记录的能力得到充分的利用。Linux（即 Android）终端设备中的内核日志通过使用命令 dmesg 进行访问。此命令会将所有的已有内核信息发往控制台，显示的部分信息如下：

```
ahoog@ubuntu:~$ adb shell dmesg
<6>[151434.178802] batt: SMEM_BATT: get_batt_info: batt_id=2, batt_vol=4211,
batt_temp=264, batt_current=377, eval_current=112, level=96, charging_source=1,
charging_enabled=1, full_bat=1300000, over_vchg=0 at 151435426063278
(2011-01-28 11:28:37.995086662 UTC)
<4>[151574.946685] select 7673 (ogle.android.gm), adj 15, size 4821, to kill
<4>[151574.947418] send sigkill to 7673 (ogle.android.gm), adj 15, size 4821
<4>[151575.003967] deathpending end 7673 (ogle.android.gm)
<4>[151668.188018] mmc1: Starting deferred resume
<6>[151668.195281] incrediblec_sdslot_switchvdd: Setting level to 2850 (Success)
<6>[151668.506591] mmc1: Deferred resume completed
<6>[151674.597320] [dma.c] msm_datamover_irq_handler id 8, result not
valid4000001
<6>[151734.168731] batt: M2A_RPC: level_update: 97 at 151735415412693
(2011-01-28 11:33:37.985015911 UTC)
<6>[151734.173339] batt: batt:power_supply_changed: battery at 151735422126560
(2011-01-28 11:33:37.989654582 UTC)
<6>[151734.190490] batt: SMEM_BATT: get_batt_info: batt_id=2, batt_vol=4212,
batt_temp=281, batt_current=353, eval_current=152, level=97, charging_source=1,
charging_enabled=1, full_bat=1300000, over_vchg=0 at 151735437507419
(2011-01-28 11:33:38.006744426 UTC)
<6>[152004.168853] batt: M2A_RPC: level_update: 98 at 152005415595761
(2011-01-28 11:38:07.985168460 UTC)
<6>[152004.171142] batt: batt:power_supply_changed: battery at 152005418311825
(2011-01-28 11:38:07.987457279 UTC)
<6>[152004.187622] batt: SMEM_BATT: get_batt_info: batt_id=2, batt_vol=4210,
batt_temp=288, batt_current=265, eval_current=112, level=98, charging_source=1,
charging_enabled=1, full_bat=1300000, over_vchg=0 at 152005434913389 (2011-01-28
11:38:08.003875737 UTC)
```

正如你注意到的，这些数据非常细并且很底层。然而，这些数据可以提供重要的有关时间戳、活动以及终端设备开启时的丰富信息。但是，如果终端设备近期没有被重新开启，最初的日志将不会继续存在。

执行这个命令不要求在终端设备上具有任何特别的访问权限，只要 USB 调试打开即可。如果在一个终端设备或者仿真器运行此命令，会注意到有太多的信息显示在显示屏上，那么就可以将 dmesg 的结果输送（发送）到一个叫做 wc 的程序（代表字数统计，word count）来确定日志的总行数，即使用 wc 来计算行数：

```
ahoog@ubuntu:~$ adb shell dmesg | wc -l
1859
```

因而，在所参考的 HTC Incredible 上，内核日志有 1859 行。如果需要更深入地查看信息，或者是将这些信息加入到一个报告中，则可以将 dmesg 的输出导向一个文件，如下所示：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ adb shell dmesg > dmesg.log
```

之后就可以使用一个文本编辑器或者一个显示程序打开文件 dmesg.log 来查看内核日志包含的内容。

（2）logcat

Android 还有另外几个可用的程序调试技术。其中一个叫做 logcat 的程序可以用于显示某个时刻被更新的系统和应用程序的调试信息列表。

```
ahoog@ubuntu:~$ adb shell logcat
I/HtcLocationService(  308): agent - search location by name: oak park, country:
united states, state: illinois
I/HtcLocationService(  308): agent - no location was found, total: 0
D/AutoSetting(  308): service - CALLBACK - onGetTimeZoneOffset, result: failed,
zoneId: , offset: 0
D/LocationManager(  308): removeUpdates: listener =
com.htc.htclocationservice.HtcLocationServiceAgent$7@45dfc770
V/AlarmManager(   97): Adding Alarm{463aea28 type 2 com.google.android.location}
Jan 05 05:05:25 pm
I/HtcLocationService(  308): agent - send current location notify intent, name:
Oak Park, state: Illinois, country: United States, lat: 41.8786, lng:
-87.6359,tzid:
D/AutoSetting(  308): service - CALLBACK - onSetWeatherProvider, result: success
I/WSP     (  308): [Receiver] EVENT - CURRENT LOCATION CHANGED
V/AlarmManager(   97): Adding Alarm{46265558 type 0 com.htc.htclocationservice}
Jan 28 09:12:53 am
D/AutoSetting(  308): service - wake lock release
D/LocationManager(  308): removeUpdates: listener =
com.htc.htclocationservice.HtcLocationServiceAgent$7@45dfc770
V/AlarmManager(   97): Alarm triggering: Alarm{46265558 type 0
com.htc.htclocationservice}
V/AutoSetting(  308): receiver - ***onReceive:
com.htc.app.autosetting.retrylocation
V/AutoSetting(  308): receiver - startAutosettingService, action:
com.htc.app.autosetting.retrylocation,notifyWhenNoResult:false
D/AutoSetting(  308): service - onCreate(),no SharedPreference
D/AutoSetting(  308): service - ***setupWizardIsCompleted: true
D/AutoSetting(  308): service - onStart(), id = 0
D/AutoSetting(  308): service - new wake lock
D/AutoSetting(  308): service - wake lock acquire
D/AutoSetting(  308): service - onStart(), Checking location times = 2
D/AutoSetting(  308): service - onStart(), Checking location change = false
D/AutoSetting(  308): service - onStart(), Set city info = false
D/AutoSetting(  308): service - onStart(), Set network time info = false
D/AutoSetting(  308): service - onStart(), Set network timezone info = true
D/AutoSetting(  308): service - onStart(), Set notify when no result = false
D/AutoSetting(  308): service - ***setupWizardIsCompleted: true
D/AutoSetting(  308): service - ***chkConnected,
mbReqChecking:false,mbApplyAll:true
D/AutoSetting(  308): service - ***Data call is avaiable
D/AutoSetting(  308): service - doAutoSettings(), isNetworkAvailable: true,
isUseWirelessNetworks: true, isTimeAutoState: true
D/LocationManager(  308): requestLocationUpdates: provider = network, listener =
com.htc.htclocationservice.HtcLocationServiceAgent$7@45dfc770
D/LocationManagerService(   97): CdmaCellLocation Unavailable
I/HtcLocationService(  308): agent - search location by name: oak park, country:
united states, state: illinois
V/AlarmManager(   97): Adding Alarm{45fb22a0 type 2 com.google.android.location}
```

更多资源请访问稀酷客(www.ckook.com)

```
Jan 05 05:04:51 pm
V/AlarmManager(   97): Adding Alarm{45fb0a90 type 2 com.google.android.location}
Jan 05 05:04:51 pm
```

通过快速浏览以上的简短日志，可以看到：

- 经度和纬度数据；
- 日期 / 时间信息；
- 应用程序细节。

日志非常详细，以上给出的样品只不过是实际可以得到的很小一部分样品。每一个日志信息都以一个信息类型指示符开头，在表 4.2 中加以描述。

表 4.2 日志方法类型

信息类型	描　述
V	Verbose（详细的）
D	Debug（调试）
I	Information（信息）
W	Warning（警告）
E	Error（错误）
F	Fatal（致命的）
S	Silent（静止的）

logcat 程序也可以提供来自于完整的蜂窝无线电的调试日志，可以使用以下命令来查看（只选择性地包含了部分无线电日志）：

```
ahoog@ubuntu:~$ adb shell logcat -b radio
D/CDMA     (  193): [CdmaServiceStateTracker] Set CDMA Roaming Indicator to: 128.
mCdmaRoaming = false, isPrlLoaded = true. namMatch = true , mIsInPrl = true,
mRoamingIndicator = 128, mDefaultRoamingIndicator= 64
D/CDMA     (  193): [CdmaServiceStateTracker] Poll ServiceState done:
oldSS=[0 home Verizon Wireless Verizon Wireless 31000  EvDo rev. A CSS supported
3 20RoamInd: 128DefRoamInd: 64EriInd: 1EriMode: 0RadioPowerSv: falseDefRoamInd:
64EmergOnly: false] newSS=[0 home Verizon Wireless Verizon Wireless 31000  EvDo
rev. A CSS supported 3 20RoamInd: 128DefRoamInd: 64EriInd: 1EriMode:
0RadioPowerSv: falseDefRoamInd: 64EmergOnly: false]
D/CDMA     (  193): Query NBPCD state: false
D/CDMA     (  193): will call NBPCD: 7735555555 NBPCD state: false
D/SMS      (  193): SMS send size=5time=1296219734520
D/RILJ     (  193): [23001]> RIL_REQUEST_CDMA_SEND_SMS
D/HTC_RIL (   67): ril_func_cdma_send_sms():called
D/HTC_RIL (   67): dump_cdma_sms():uTeleserviceID    = 4098
D/HTC_RIL (   67): dump_cdma_sms():bIsServicePresent = 0
D/HTC_RIL (   67): dump_cdma_sms():uServicecategory  = 0
D/HTC_RIL (   67): dump_cdma_sms():sAddress.digit_mode  = 0
D/HTC_RIL (   67): dump_cdma_sms():sAddress.number_mode = 0
D/HTC_RIL (   67): dump_cdma_sms():sAddress.number_type = 0
D/HTC_RIL (   67): dump_cdma_sms():sAddress.number_plan = 0
D/HTC_RIL (   67): dump_cdma_sms():sAddress.number_of_digits = 10
D/HTC_RIL (   67): dump_cdma_sms():sAddress.digits = [07070305050505050505]
D/HTC_RIL (   67): dump_cdma_sms():sSubAddress.subaddressType = 0
D/HTC_RIL (   67): dump_cdma_sms():sSubAddress.odd = 0
D/HTC_RIL (   67): dump_cdma_sms():sSubAddress.number_of_digits = 0
D/HTC_RIL (   67): dump_cdma_sms():sSubAddress.digits = []
D/HTC_RIL (   67): dump_cdma_sms():uBearerDataLen = 32
D/HTC_RIL (   67): dump_cdma_sms():aBearerData:
[0003202C80010D10654CBCFA20DB979F3C39F2800E0705189249CB1B00080100]
D/HTC_RIL (   67): encode_cdma_sms():encode_index = 51
D/HTC_RIL (   67): ril_func_cdma_send_sms():raw_data:
[0000021002040702 9DCD158C9880060100082000 03202C80010D10654CBCFA20DB979F3C39F2800
E0705189249CB1B00080100]
D/HTC_RIL (   67): (t=1296219734)>> AT+CMGS=51\r
D/HTC_RIL (   67): RX::> \r\n> ^M
```

更多资源请访问稀酷客(www.ckook.com)

```
D/HTC_RIL (   67): (t=1296219734)XX \r\n
D/HTC_RIL (   67): (t=1296219734)<< >
D/HTC_RIL (   67): (t=1296219734)>>
0000021002040702 9DCD158C9880060100082000032 02C80010D10654CBCFA20DB979F3C39F2800E
0705189249CB1B00080100^Z
D/RILJ    (  193): [UNSL]< UNSOL_DATA_CALL_LIST_CHANGED [DataCallState: { cid:
62, active: 1, type: IP, apn: 0, address: 10.237.127.132 }, DataCallState:
{ cid: -1, active: 0, type: , apn: , address:  }, DataCallState: { cid: -1,
active: 0, type: , apn: , address:  }]
```

虽然日志是详细的，然而通常也是神秘的，对日志进行扫描搜索可以获得如下信息：

- 事件的时间（使用 UNIX 的时间方式表达，如 t=1296218163）；
- 蜂窝调制解调器进行通信所使用的 AT 命令；
- 接收者、时间以及编码的短信消息；
- 终端设备的蜂窝 IP 地址、网络及地点信息；
- 无线通信服务提供商信息。

这些都是非常底层的信息，但是，如果这些日志得到恢复，则可以提供关于终端设备的重要信息。

> **说明：UNIX 时间**
>
> UNIX 时间是 UNIX/Linux 的系统中时间戳的通用格式。时间戳是一个正整数值，表示从 1970 年 1 月 1 日开始至今所经过的秒（或毫秒）。典型的以秒为单位的时间戳有 10 位数，而以毫秒为单位的时间戳则有 13 位数。在第 7 章中将对时间戳进行更多讨论。

logcat 的最后一个功能就是显示事件日志：

```
$adb shell logcat -b events
I/db_sample(  193):
[/data/data/com.android.providers.telephony/databases/mmssms.db,INSERT INTO
sms(body, index_on_sim, address, subject, read, type,280,,57]
I/db_sample(  193):
[/data/data/com.android.providers.telephony/databases/mmssms.db,SELECT
transport_type, _id, thread_id, address, body, date, read,170,,35]
```

上述日志仍然是很详细的。但是，系统中发生的不同事件将会产生大量的日志信息。在前一个日志片断中，可以看到在 mmssms.db 中有 INSERT 和 SELECT 语句，它们用于保存文本信息。

（3）dumpsys

下一个日志机制通过一个叫做 dumpsys 的命令进行使用。

dumpsys 提供关于服务、存储及系统其他方面很细节、很有用的信息。所提供的信息类型包括以下：

- 目前正在运行的服务；
- 针对每一项服务所转储出的信息；
- 在当前活动管理状态下的服务、播报、等待的企图、活动以及进程；

更多资源请访问稀酷客(www.ckook.com)

• 进程信息，包括内存、进程 ID 号（PID）、数据库等。

以下所列为 HTC Incredible 手机上使用 dumpsys 所产生的几个样品部分，给出了为什么这些数据对于取证分析师和安全工程师们来说非常有价值的简单解释。

首先，按照以下方式运行 dumpsys 命令：

```
$adb shell dumpsys
Currently running services:
  SurfaceFlinger
  VZW_LOCATION_SERVICE
  accessibility
  account
<snip>
```

此部分针对“account”服务所转储出的细节对获取终端设备上所使用的账号的信息有用：

```
DUMP OF SERVICE account:
Accounts: 10
  Account {name=Backup Assistant, type=com.htc.VzWBASync}
  Account {name=News, type=com.htc.newsreader}
  Account {name=Weather, type=com.htc.sync.provider.weather}
  Account {name=Stocks, type=com.htc.android.Stock}
  Account {name=book@viaforensics.com, type=com.google}
  Account {name=book@viaforensics.com, type=com.htc.android.mail.eas}
  Account {name=Andrew Hoog, type=com.htc.socialnetwork.facebook}
  Account {name=viaforensics, type=com.htc.htctwitter}
  Account {name=viaforensics, type=com.twitter.android.auth.login}
  Account {name=personal@emailaddress.com, type=com.google}
```

这样不但可以看到使用的程序，而且可以看到用户所使用的账号名。例如，以上信息显示出：

- 具有用户名 book@viaforensics.com 的用户的谷歌账号；
- 具有用户名 book@viaforensics.com 的用户的 Exchange ActiveSync (EAS) 账号（除了以上的谷歌账号以外）；
- Andrew Hoog 的 Facebook 账号；
- viaforensics 的 Twitter 账号；
- personal@emailaddress.com 的谷歌账号（终端设备上的第二个账号）

再进一步查看日志，可以同样获得最后 10 个同步的实际时间戳，如下：

```
Recent Sync History
  #1: book@viaforensics.com:com.google com.android.calendar LOCAL @ 2011-01-28
09:52:46 for 0.0s
  #2: book@viaforensics.com:com.htc.android.mail.eas htceas USER @ 2011-01-28
09:51:43 for 34.5s
  #3: book@viaforensics.com:com.google com.android.calendar LOCAL @ 2011-01-28
09:49:25 for 0.0s
  #4: Andrew Hoog:com.htc.socialnetwork.facebook com.htc.socialnetwork.facebook
SERVER @ 2011-01-28 09:48:57 for 0.5s
  #5: book@viaforensics.com:com.google com.android.calendar LOCAL @ 2011-01-28
09:45:30 for 0.0s
  #6: Andrew Hoog:com.htc.socialnetwork.facebook com.htc.socialnetwork.facebook
SERVER @ 2011-01-28 09:44:40 for 1.3s
```

更多资源请访问稀酷客(www.ckook.com)

```
  #7: book@viaforensics.com:com.htc.android.mail.eas htceas USER @ 2011-01-28
09:44:18 for 1.3s
  #8: viaforensics:com.twitter.android.auth.login
com.twitter.android.provider.TwitterProvider SERVER @ 2011-01-28 09:44:06 for
11.6s
  #9: book@viaforensics.com:com.htc.android.mail.eas htceas USER @ 2011-01-28
09:41:08 for 15.8s
  #10: Andrew Hoog:com.htc.socialnetwork.facebook com.htc.socialnetwork.facebook
SERVER @ 2011-01-28 09:37:27 for 0.1s
```

另外还有一个被幽默地称为“iphonesubinfo”的服务，但是此服务与苹果的 iPhone 没有任何关系，尽管它们有很相似的名称。

```
DUMP OF SERVICE iphonesubinfo:
Phone Subscriber Info:
  Phone Type = CDMA
  Device ID = A100001829481F
```

手机类型和终端设备 ID（被改变过）能够从此分段中获得。终端设备 ID 不是此终端设备的序号，而是它的移动设备识别号（Mobile Equipment Identifier，MEID），用于在 CDMA 网络中能够唯一地识别此终端设备。

另外一个很好的信息来源是位置服务，可以从中得知最后一次采集到的位置信息及时间：

```
DUMP OF SERVICE location:
  Last Known Locations:
    passive:
      mProvider=network mTime=1296230208384
      mLatitude=41.8786 mLongitude=-87.6359
      mHasAltitude=false mAltitude=0.0
      mHasSpeed=false mSpeed=0.0
      mHasBearing=false mBearing=0.0
      mHasAccuracy=true mAccuracy=1423.0
      mExtras=Bundle[{networkLocationType=cell, networkLocationSource=cached}]
    gps:
      mProvider=gps mTime=1296157873000
      mLatitude=41.8786 mLongitude=-87.6359
      mHasAltitude=true mAltitude=198.8000030517578
      mHasSpeed=true mSpeed=29.75
      mHasBearing=true mBearing=69.7
      mHasAccuracy=true mAccuracy=2.828427
      mExtras=Bundle[{satellites=11}]
```

在 Android 中，大多数的时间戳所表达的是自 1970 年 1 月 1 日以来的毫秒数，也就是 UNIX 使用毫秒的时间表达方式，而不是使用秒的表达方式。由于很多工具都使用 UNIX 时间表达方式中的秒，只需要直接将时间数除以 1000 即可，然后再使用标准的公式。如果使用的是一台 Ubuntu 工作站，则可以使用以下命令来实现时间的转换：

```
ahoog@ubuntu:~$ date -d @1296230208
Fri Jan 28 09:56:48 CST 2011
```

这样输出的时间对应着工作站的时区。然而，使用此命令的各种参数则可以控制时区、格式以及其他很多参数。想要了解各种可能，运行“date-help”或“man date”。

对以上 3 个缓存的位置信息进行检查，可以看到系统在以下时间对来自于全球定位系

更多资源请访问稀酷客(www.ckook.com)

统（GPS）卫星的位置和来自于无线通信基站的位置信息进行了缓存。

1）GPS: Thu Jan 27 13:51:13 CST 2011；

2）Cell: Fri Jan 28 09:56:48 CST 2011。

位置信息在所记录的时间时是准确的，因此可以用来提供终端设备位置的历史信息。网络状态部分提供其他更多信息，包括更多关于手机基站的信息：

```
network Internal State:
  location Location[mProvider=network,mTime=1296230208384,mLatitude=41.8786,
mLongitude=-87.6359,mHasAltitude=false,mAltitude=0.0,mHasSpeed=false,mSpeed=0.0,
mHasBearing=false,mBearing=0.0,mHasAccuracy=true,mAccuracy=1423.0,mExtras=Bundle
[{networkLocationSource=cached, networkLocationType=cell}]]
  Status 2
  StatusUpdateTime 385403711
  NetworkState 2
  LastCellStateChangeTime 428707868
  LastCellLockTime 0
cell state [cid: 277 lac: 3 mcc: 0 mnc: 20 radioType: 2 signalStrength: -85
 neighbors[]
 cell history
   [cid: 4671 lac: 3 mcc: 0 mnc: 20 radioType: 2 signalStrength: -103 neighors[]
   [cid: 277 lac: 3 mcc: 0 mnc: 20 radioType: 2 signalStrength: -103 neighbors[]
   [cid: 4671 lac: 3 mcc: 0 mnc: 20 radioType: 2 signalStrength: -87 neighbors[]
   [cid: 286 lac: 3 mcc: 0 mnc: 20 radioType: 2 signalStrength: -98 neighbors[]
 WifiScanFrequency 60000
 WifiEnabled 0
 WaitingForWifiScan 0
 LastNetworkQueryTime 428712899
 LastSuccessfulNetworkQueryTime 428712926
 Enabled 1
 AirplaneMode 0
 DisabledForAirplaneMode 0
```

需要指出的是，尽管涉及众多的技术细节，但要注意最后一个片断是关于内存的信息，该片断是每个进程号（PID）的输出：

```
** MEMINFO in pid 454 [com.htc.android.mail] **
                   native   dalvik    other    total    limit   bitmap nativeBmp
             size:  18048     9543      N/A    27591    32768      N/A      N/A
        allocated:  14490     4485      N/A    18975      N/A     1032        0
             free:   1341     5058      N/A     6399      N/A      N/A      N/A
            (Pss):  10644     4839     8651    24134      N/A      N/A      N/A
   (shared dirty):   1348     3924      960     6232      N/A      N/A      N/A
     (priv dirty):  10604     4724     4800    20128      N/A      N/A      N/A

Objects
           Views:        0        ViewRoots:        0
     AppContexts:        0      Activities:         0
          Assets:        3    AssetManagers:        3
   Local Binders:       50    Proxy Binders:       36
Death Recipients:        2
OpenSSL Sockets:         1

SQL
            heap:     3740       memoryUsed:     3740
pageCacheOverflo:     2185  largestMemAlloc:     1667

DATABASES
```

更多资源请访问稀酷客(www.ckook.com)

```
Pagesize   Dbsize  Lookaside  Dbname
    1024       21         38  webview.db
    1024      305         52  webviewCache.db
    1024     5662        499  mail.db
    1024        8          0    (attached) people_db
```

这不但利于确定哪些进程正在运行，而且对于确定它们访问了哪些数据库都很有用处。例如，在对一个案件的调查中，调查者需要了解当收到一封邮件时，哪些信息会被更新。从以上的列表中可以看到，邮件服务（com.htc.android.mail）不但更新 了 mail.db，而且更新与 Web 相关联的两个附在 people_db 上的数据库。此信息对于解释 Android 终端设备上的数据如何相互关联非常有用。

（4）dumpstate

另外一个程序调试命令是 dumpstate，用于将前述的部分调试与系统信息进行结合。与其他的命令相似，此命令的运行方式如下：

```
$adb shell dumpstate
/data/anr/traces.txt: Permission denied
============================================================
== dumpstate: 2011-01-28 09:56:27
============================================================

Build: FRF91
Bootloader: 0.92.0000
Radio: 2.15.00.07.28
Network: Verizon Wireless
Kernel: Linux version 2.6.32.17-g9a2fc16 (htc-kernel@u18000-Build-149)
(gcc version 4.4.0 (GCC) ) #1 PREEMPT Thu Sep 30 18:42:08 CST 2010
Command line: (unknown)
```

adbd 作为 root 运行，显示在仿真器或终端设备上的第一部分是对应用的跟踪。然而，在所用的 HTC 终端设备上，dumpstate 返回 permission denied（拒绝访问）信息。之后列出的是关于终端设备、制造、无线电、网络和具体的内核方面的基本信息。再之后列出的内容如表 4.3 所示。

表 4.3　dumpstate 分段

区　域	文件或命令
Stack traces	N/A
Device info	N/A
System	N/A
Memory info	/proc/meminfo
Cpu info	top -n 1 -d 1 -m 30 -t
Procrank	(procrank)
Virtual memory stats	/proc/vmstat
Vmalloc info	/proc/vmallocinfo
Slab info	/proc/slabinfo
Zoneinfo	/proc/zoneinfo
System log	logcat -v time -d *:v
Event log	logcat -b events -v time -d *:v

更多资源请访问稀酷客(www.ckook.com)

（续）

区　域	文件或命令
Radio log	logcat -b radio -v time -d *:v
Network interfaces	netcfg
Network routes	/proc/net/route
Arp cache	/proc/net/arp
Dump Wi-Fi firmware log	su root dhdutil -i eth0 upload/data/local/tmp/ wlan_crash.dump
System properties	N/A
Kernel log	dmesg
Kernel wakelocks	/proc/wakelocks
Kernel cpufreq	/sys/devices/system/cpu/cpu0/cpufreq/stats/time_in_state
Vold dump	vdc dump
Secure containers	vdc asec list
Processes	ps -p
Processes and threads	ps -t -p -p
Librank	librank
Binder failed transaction log	/proc/binder/failed_transaction_log
Binder transaction log	/proc/binder/transaction_log
Binder transactions	/proc/binder/transactions
Binder stats	/proc/binder/stats
Binder process state	sh -c cat/proc/binder/proc/-p
File systems and free space	df
Package settings	/data/system/packages.xml:2011-01-26 09:18:02
Package uid errors	/data/system/uiderrors.txt:2010-11-1422:52:26
Last kmsg	/proc/last_kmsg
Last radio log	parse_radio_log/proc/last_radio_log
Last panic console	/data/dontpanic/apanic_console
Last panic threads	/data/dontpanic/apanic_threads
Blocked process wait	N/A
channels	
Backlights	N/A
Dumpsys	dumpsys

（5）bugreport

最后一个程序调试命令进一步建立在以上讨论过的命令之上，它将 logcat、dumpsys 和 dumpstate 输出通过一个命令结合在一起，显示在显示屏上的内容是为了提交一个程序调试报告。此命令的运行方式如下：

```
ahoog@ubuntu:~$ adb bugreport
```

以上命令行将执行 dumpstate。在 HTC Incredible 上运行此命令时，输出被保存在一个文件中，然后对此文件内容按照行进行统计：

```
ahoog@ubuntu:~$ adb bugreport > bugreport.log
ahoog@ubuntu:~$ wc -l bugreport.log
42575 bugreport.log
```

可以看到，以上报告产生了超过 42 000 行含有时间戳、应用数据和系统信息的调试数据。对此数据进行解析将会得出非常有用的信息。但是，如果数据处理是采用手工的方

更多资源请访问稀酷客(www.ckook.com)

式进行，工作量将是令人生畏的。

4.3　内存类型

在第 2 章中讨论过，Android 终端设备主要具有两种类型的内存：易失性内存（随机访问内存，RAM）和非易失性内存（NAND 闪存）。每一类型的内存为终端设备上所存储的数据提供了不同程度的细节。

随机访问内存

系统使用随机访问内存（RAM）装载、执行及处理操作系统、应用程序和数据的关键部分，存储的内容在重新开机时不会被保存。像传统的计算机那样，RAM 可能包含应用程序用于处理数据的非常重要的信息。以下是该类信息的一些实例：

- 密码；
- 加密密钥；
- 用户名；
- 应用数据；
- 源于系统进程和服务的数据。

最近，出现了对 Android 内存进行检查的解决方案。其中一项技术由信息安全研究者 Thomas Cannon 在他的博客中有所描述，我们也将详细介绍（Android reverse engineering, n.d.）。

Android 提供了一个将应用程序所使用的内存转储进一个文件的机制，该机制的实现是通过向应用程序发送一个特别信号（SIGUSR1）。为了发送此信号，需要获得应用程序的进程识别号（PID），可以通过执行 ps 命令获得：

```
ahoog@ubuntu:~$ adb shell ps
USER      PID   PPID  VSIZE  RSS     WCHAN    PC         NAME
root      1     0     348    248   ffffffff 00000000 S /init
root      2     0     0      0     ffffffff 00000000 S kthreadd
root      3     2     0      0     ffffffff 00000000 S ksoftirqd/0
root      28    2     0      0     ffffffff 00000000 S crypto/0
root      39    2     0      0     ffffffff 00000000 S panel_on/0
keystore  72    61    1732   420   ffffffff 00000000 S /system/bin/keystore
shell     76    61    3412   196   ffffffff 00000000 S /sbin/adbd
system    97    68    288408 50100 ffffffff 00000000 S system_server
app_96    193   68    162284 28356 ffffffff 00000000 S com.swype.android.inputmethod
radio     199   68    181376 33452 ffffffff 00000000 S com.android.phone
9997      429   68    187716 34756 ffffffff 00000000 S com.htc.android.mail
app_24    568   68    187796 31064 ffffffff afd0ebd8 S com.google.android.gm
9997      732   68    161816 24492 ffffffff 00000000 S
com.htc.android.mail:directpush
app_103   756   68    150480 23036 ffffffff 00000000 S com.dropbox.android
app_43    1020  68    176632 29472 ffffffff 00000000 S com.google.android.apps.maps
app_43    1132  68    161984 27744 ffffffff 00000000 S
com.google.android.apps.maps:BackgroundFriendService
app_42    1294  68    160680 32672 ffffffff 00000000 S com.facebook.katana
app_4     1355  68    148900 23768 ffffffff 00000000 S com.htc.WeatherWallpaper
shell     1938  76    744    328   c0064900 afd0e88c S /system/bin/sh
shell     1939  1938  892    340   00000000 afd0d97c R ps
```

更多资源请访问稀酷客(www.ckook.com)

执行 ps 命令将会列出所有系统程序和应用程序的进程及父进程识别号、内存信息以及进程名。由于在一个终端设备上通常会有很多进程在同时运行，以上所显示的内容只是输入的一部分。

下一步，需要在终端设备上运行一个拥有 root 权限的交互用 shell，并设置 /data/misc 拥有足以让它对内存的转储内容进行读取和写入的访问权限：

```
ahoog@ubuntu:~$ adb shell
$ su
# chmod 777 /data/misc
```

警示：改变目录（文件夹）的访问权限

chmod 命令可以改变对文件夹 /data/misc 的访问权限，赋予系统中所有用户读、写和执行的访问权限。改变访问权限对于成功进行内存转储很有必要。然而，取证分析师应该公开地做这样的系统改变。如果分析师所做的工作是正当合法的，这一改变应该反映在报告里，在理想的情况下，访问权限应该在内存转储完成后被恢复到默认的设置。

现在，就可以发送对内存内容进行转储并显示目录内容的信号：

```
# kill -10 1294
# ls -l
drwxrwx--- dhcp     dhcp              2011-01-17 13:18 dhcp
drwxrwx--- system   system            1980-01-06 11:40 vpn
drwxrwx--- system   system            1980-01-06 11:40 bluetooth
drwx------ system   system            1980-01-06 11:40 systemkeys
drwxrwxrwx system   system            1980-01-06 11:41 lockscreen
-rw-r--r-- system   system          1 2011-01-29 15:19 screen_lock_status
-rw-rw---- compass  compass       442 2011-01-29 19:11 AK8973Prms.txt
drwxrwx--- wifi     wifi              2010-08-28 12:06 wifi
drwxrwx--- bluetooth bluetooth          1980-01-06 11:40 bluetoothd
-rw-rw-rw- app_42   app_42    3978523 2011-01-29 19:26 heap-dump-tm1296350817-
                                                      pid1294.hprof
drwx------ keystore keystore         1980-01-06 11:40 keystore
```

文件 heap-dump-tm1296350817-pid1294.hprof 内就是内存转储内容，然后我们就可以结束交互的 adb shell，使用 pull 命令在本地工作站对此文件进行分析：

```
ahoog@ubuntu:~/memdump$ adb pull /data/misc/heap-dump-tm1296351804-pid1294.hprof
facebook.hprof
1223 KB/s (3977724 bytes in 3.175s)
```

此后，我们就可以使用任何一种内存分析技术。例如，可以使用 strings 命令来抽取出 ASCII 字符串：

```
ahoog@ubuntu:~/memdump$ strings gmail.hprof > gmail.str
```

然后可以查看文件 gmail.str 中的内容。对 Gmail 的内存快速扫描可发现应用程序调用的各种加密共享程序库以及基于 HTTP 的交互情况：

更多资源请访问稀酷客(www.ckook.com)

```
POST
/proxy/gmail/a/andrewhoog.com/g/?version=25&clientVersion=25&allowAnyVersion=1
HTTP/1.1
Accept-Encoding: gzip
Content-Length: 29
Host: android.clients.google.com
Connection: Keep-Alive
User-Agent: Android-GmailProvider/156 (inc FRF91); gzip
Cookie: GXAS_SEC=andrewhoog.com=DQAAAJsAAACz6B42ndmh7G5-oRmGrv_78Q-
NxsOMxL256qXfh_Dtkj3llZ0uUir7FbGQOK8PsaCi3iXuR1GsqCtV0rOel-07_-
nrjZ7WADRPDYDJ2lIYvBfnpaZh7mbMjBpJB4iS6Kvgilgc8wRJCHhb0aaaaaaaa;
S=gmail=K1Xaaaaaaaaaamya3YypJA:gmproxy=-v0_tVIkUaaaaaaaQ;
GXAS_SEC=andrewhoog.com=DQAAAJwAAAAPCeOv_Xhali8NCBR5d6hp8wrvO79bWlcweQTUulld3sVT4
nPcw4wnUfCZAaaav8Cqp-ktu6l4gW9L2gWCUpuFkjPHGvHiEa4W7P0R-dawWSgk7_wOmaP585kz8Pkoo4
EGYzn9nbQj7X2s5qLfqRwdpRPUMWREKMqazlg6HgAG5Tsp
Cookie2: $Version=1
: application/vnd.google-x-gms-proto; charset=utf-8
Set-Cookie: XAS_SEC=andrewhoog.com=DQAAAJoAAAAUERMYMdgggggggggiwn1MqWkps31wuuOa
KKe-hKekfgyT7apv6wKpPlycE8PS7S0-gRkmJydqPyDPCgjLXBSw7SVj5Lyp;
Path=/proxy/gmail/a/; Secure
Expires: Sun, 30 Jan 2011 00:16:05 GMT
Transfer-Encoding: chunked
Date: Sun, 30 Jan 2011 00:16:05 GMT
Cache-Control: private, max-age=0
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
Server: GSE
```

Facebook 应用是使用 JSON 格式的文件对数据进行编码和传输，结果如下：

```
{"messages":{"unread":0,"most_recent":1296345224},"pokes":{"unread":0,
"most_recent":0},"shares":{"unread":0,"most_recent":0},"friend_requests":[],
"group_invites":[],"event_invites":[18191xxxxxxxxx]}!
:)","time":1296238459,"status_id":1802922removed},"pic_square":"http:\/\/profile.
ak.fbcdn.net\/hprofile-ak-snc4\/hs1323.snc4\/161426_506.jpg"},{"uid":removed,
"first_name":"removed","last_name":"removed","name":"removed
removed","status":{"message":"College friends: do you ever reflect on all the
time we spent driving past removed","time":1296194273,"status_id":10574xxxxxxxx},
{"id":7695xxxxx,"type":"user"
,"pic_square":"http:\/\/profile.ak.fbcdn.net\/hprofile-ak-
snc4\/hs712.ash1\/161111_76xxxxxx_q.jpg","name":"removed"}]},{"name":"places",
"fql_result_set":[{"page_id":1
0823xxxxx,"longitude":-
75.130002,"latitude":40.324749,"description":"","checkin_count":62,"name":
"Pocos"}]}
```

以上实例中的信息已经被清理过。然而，依然可以看到时间戳、个人简况更新、朋友信息以及登入（check-in）信息等。

对 Android 终端设备的内存进行分析能够深入了解终端设备的内部结构以及关于设备拥有者的关键信息。随着时间的推移，市场上会出现更多对 Android 内存进行分析的工具和方案。

NAND 闪存

与随机访问内存（RAM）有所不同，NAND 闪存属于不易失性存储，因此，存储的数据即使在系统断电或重启时都能完好保留。NAND 闪存不但用于存储系统文件，而且用

更多资源请访问稀酷客(www.ckook.com)

于存储大部分的用户数据。

NAND 闪存的特性与现代硬盘磁性介质的特性有很大不同。这些特性使 NAND 闪存成为在移动终端设备中使用的理想存储，同时，也给程序员带来了很多挑战，给取证分析师带来了很多机遇。

首先，NAND 闪存没有传统的磁性硬盘系统所使用的运动机械旋转部分。这样就改进了持久性，并且在体积上变得更小，也降低了终端设备的电能损耗。存储器由一个或多个芯片组成，在芯片上又经常将 NAND 闪存和随机访问内存（RAM）集成在一起（见第 2 章），最后再与终端设备的电路板直接进行集成。

NAND 闪存的密度很高，对制造商来说成本较低廉。这一点就使它非常受制造商的欢迎。一般来说，制造过程和技术的一个不足之处是 NAND 闪存在出厂时就有废块存在。制造商通常将测试作为制造流程的一部分，将检测出的废块在 NAND 闪存的具体结构中进行明确标识并在相应的文档中进行说明。与 NAND 闪存直接进行交互的软件会读取制造商标识出的废块标记，经常通过实现一个废块表，在系统中对废快进行跟踪，并在实际操作中避免使用废块。这种策略极大地提高了对废块的检测和管理效率。因此，虽然 NAND 闪存比旋转的磁盘机更加耐用，但出错率高出很多，在开发和使用中必须要进行充分考虑。

另外一个 NAND 闪存的不足是它能够被写 / 擦除的次数（生命周期）非常有限，存储块很快就无法再用于存储数据。生命周期随终端设备的不同而不同，主要决定因素是作为存储比特位构成单位的每个 NAND 闪存核中存储的数据量。如果每一个核中只存储一个比特位（单层核，SLC），NAND 闪存可以为需要保留一年的数据允许 10 万次左右的写 / 擦除。但是，由于制造商（以及消费者）在同样大小或更小的终端设备中要求有更大存储空间，在 NAND 闪存中基本上不采用单层核存储方式。在技术上已经在每一个核内实现了多层核（MLC）存储，即存储两位、三位或更多位。然而，这样不但会使制造过程更加复杂，降低了写 / 擦除速度，而且使终端设备的持久性大幅度降低。一个典型的多层核 NAND 闪存如果使用每一个核存储两比特位就将使其持久性下降 10 倍（以数据保存一年为前提进行测量），只能经历大约一万次写 / 擦除周期。随着在每一个核中存储比特密度的增加，持久性继续下降，这种情况在终端设备中必须得到重视。

与随机访问存储和 NOR 闪存（也属于闪存，通常被用于计算机中的基础输入 / 输出系统，也称 BIOS）不同，对 NAND 闪存无法进行随机访问。进而，对数据的访问是通过一个叫做页或子块的分配单元来完成，典型的分配单元大小为 512 ～ 2048 字节并通常随着 NAND 闪存空间的增加而增大。虽然 NAND 闪存的访问速度不如随机访问存储（RAM），但是依然很快，因为它没有旋转式的硬盘所使用的机械部分及运动的读写臂。

提示：页与子块

我们在本书中将页和子块不加区分地进行使用，都指 NAND 闪存中低层的数据分配单元。虽然分配单元通常被称为页，在 Android 关键的 YAFFS3 文件系统中，又通常将分配单元称为子块。

更多资源请访问稀酷客(www.ckook.com)

子块又被进一步地合在一起，形成更大的被称为块的逻辑单元，通常比传统的硬盘上使用的 512 字节的区段要大很多。在大多数 Android 终端设备中，NAND 闪存中每一个块由 64 个子块构成，每一个子块又有 2048 字节（2KB）的数据。因此，每一个块的大小为 64×2KB，即每一个块的大小为 128KB。当然，这个数字可能会随着时间的推移而发生变化，它也是由 NAND 闪存制造商控制的。当数据被写入一个块，数据就按照子块在块中的顺序先后写入。

NAND 闪存的另外一个重要特性就是能够对其进行读和写操作：

- 读（页）；
- 写（页）；
- 擦除（块）。

虽然读操作和写操作可以面向的基本单元是子块，擦除操作则以块作为其操作的基本单元。当一个块被擦除，整个块中的二进制位全部都被写入 1（即十六进制的 0xFF）。

提示：NAND 闪存中的擦除操作

擦除操作是 NAND 闪存中唯一能够将“0”改变为“1”的机制。这一点值得说明一下。在传统的硬盘中，当一个二进制值需要从“0”改变为“1”时（反过来也如此），程序将在硬盘上找到正确的位置，使用正确的电压改变并保存新的值。然而，NAND 闪存的基本架构只提供一个将“0”改变为“1”的机制，即将整块作为基本单位使用擦除函数，而不能以页作为基本单位。因此，每一页只能被写入一次。如果一个页内的任何值需要被改变，则整个块必须首先被擦除，然后此页才能被写入。

以下举一个简单的例子进行说明，为了简便，只使用一个字节：假设此字节当前的值为十进制 179，我们将其与 39 相加而得到 218。如果对十进制、十六进制和二进制之间的转换不熟悉的话，Windows、Mac OS X 和 Ubuntu Linux 都给程序员提供了一个编程模式，能够实现这些进制之间的转换。对于此例中的 179 和 218，表 4.4 提供此数在 3 种进制之间的对比。

表 4.4　整数的十进制、十六进制和二进制对比

十进制	十六进制	二进制
179	0xB3	1011 0011
218	0xDA	1101 1010

因此，179 中包含 3 个“0”，其中两个需要改变为“1”，使我们能够得到结果 218。但是，如果不将整个块擦除，NAND 闪存无法实现以上改变。也就是说，如果此字节没有被擦除，得到的结果是 146，而不是 218，说明如下：

```
1011 0011（初始字节，十六进制 0xB3 或十进制 179）
1101 1010（需要写入的新字节，十六进制 0xDA 或十进制 218）
```

更多资源请访问稀酷客(www.ckook.com)

```
......
1001 0010（结果字节，十六进制 0x92 或十进制 146）
```

由于此字节并不是所有的位都是 1（十六进制 0xFF），能够成功写入的只有原来为“1”的位数，或者仍然为“1”，或者被改写为“0”。任何时候要求写函数将“0”改变为“1”，结果都将是失败，“0”值仍然维持。因此，以上的结果将是 0x92，即十进制的 146，很明显不是正确的结果。另外一种对写函数的描述是，它只能将“1”改变为“0”，等同于这两个值的“逻辑与”。

对以上讨论进行总结，每一个子块只能被写入一次，如果需要再次被写入，则相应的整个块必须首先被擦除。

可以看出，NAND 闪存有很多限制和制约，因此，开发者对文件系统必须有充分了解，以有效地在制约范围内进行使用。不像传统的硬盘那样在出厂时就配有相应的固件用于对其进行管理（包括废块），Android 终端设备中使用的 NAND 闪存在出厂时连控制器都没有。所有对存储的管理都依赖于与 NAND 闪存进行接口的软件。纠错码（ECC）和损耗程度度量是两项重要的应用技术，都对取证和数据恢复具有重大影响。

纠错码（ECC）技术使用一个算法检测读写操作中的错误，并立刻对某些错误进行纠正。由于 NAND 闪存随着使用时间增加而逐渐退化，系统必须有能力在页或块变为不可使用时将其检测出，并将存储的数据进行恢复。当操作失败或者错误超过了一个阈值时（通常为 3 次操作失败），一个页或块将被标识为不再可用，将被加入到废块表中。

第二个对 NAND 闪存进行有效管理的重要算法是损耗程度度量代码，它将写入的数据分散在 NAND 闪存的整个空间上，以避免由于过度使用某些区域以造成这些区域中块的快速损耗。

很多使用 NAND 闪存的硬件终端设备，如可移动的 USB 驱动器和固态驱动器（SSD），在逻辑上与终端设备绑定在一起的控制器，这些设备可提供上面介绍的功能，包括废块管理、损耗程度度量以及纠错码。然而，由于 Android 终端设备在设计上是将 NAND 闪存部件进行直接集成，因此需要一个软件管理层来提供上述的这些重要功能。选择用于对 NAND 闪存软件管理的层就是内存技术设备（Memory Technology Device，MTD）系统。

开发 MTD 是为了满足 NAND 闪存及其他相似终端设备特性的需要。在 MTD 之前，Linux 主要支持的是字符设备和模块设备，都无法满足新开发出的存储设备独特性质的要求。此外，虽然 NAND 闪存严格意义说来并不属于模块设备（如传统上的硬盘驱动），但是将模块设备的特性展示给开发者能够对开发和售后支持提供有力帮助。通过使用 MTD，Android 目前已经具有了与 NAND 闪存进行有效交互所需的闪存过渡层（Flash Transition Layer，FTL）。通过这种方式，Android 就可以做到不限制制造商只使用少数供应商提供的 NAND 闪存及附属的控制器。因此，他们就可以自由地使用任何 NAND 闪存，并与能够支持大量不同类型 NAND 闪存的 MTD，“简单”地实现集成。

在 Android 中，MTD 不但提供与 NAND 闪存进行模块接口功能，还提供纠错码（ECC）、损耗程度度量及其他关键功能。纠错码（ECC）与其他的子块元数据一起被保存

更多资源请访问稀酷客(www.ckook.com)

在一个叫做 OOB 区域或空闲区域。OOB 区域在 NAND 闪存上直接位于每一个子块之后。虽然块、子块及 OOB 区域都可以进行配置，但大多数的 Android 终端设备目前使用的块为 128KB，每一个块共有 64 个 2048 字节（2KB）的块，每一个块又有一个 64 字节的离线区域，如图 4.1 所示。

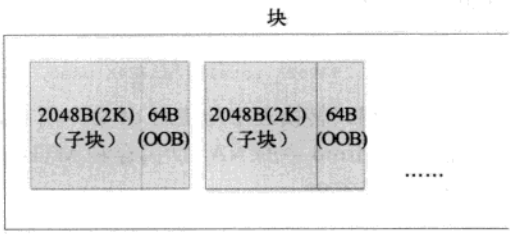


图 4.1　块（128 KB = 64×2K 子块 +OOB 区域）

OOB 区域不但用于存储 MTD 进行管理所用的信息，而且假如文件系统知晓 NAND 闪存的存在，它可以用于存储对于文件系统非常关键的元数据。当系统将存储块的大小呈现为 128KB，在加入 64 个 64 字节大小的 OOB 区域后，系统额外增加 4096 字节（4KB），最终，NAND 闪存的总字节数为 132KB。

在 Android 终端设备上，可以通过查看文件 /proc/mtd 来确定 MTD 分区。在我们所参考的 HTC Incredible 上可以看到以下内容：

```
ahoog@ubuntu:~$ adb shell cat /proc/mtd
dev:    size   erasesize  name
mtd0: 000a0000 00020000 "misc"
mtd1: 00480000 00020000 "recovery"
mtd2: 00300000 00020000 "boot"
mtd3: 0f800000 00020000 "system"
mtd4: 000a0000 00020000 "local"
mtd5: 02800000 00020000 "cache"
mtd6: 09500000 00020000 "datadata"
```

系统中一共有 7 个 MTD 分区，在本节中分别介绍它们的所在，并且对其中存在的目录和文件进行简要介绍。在前一个列表中，尺寸和擦除尺寸都是十六进制值，提供了重要的 MTD/NAND 闪存性质。擦除尺寸指出的是每一块的大小，即 0x20000 或用十进制表达为 131 072 字节或 128KB（128×1024）。这与块图是一致的，具体来说就是 64 页，每页大小为 2048 字节（2KB）。尺寸列表示分区的总大小。因此，在此例中，我们得到如表 4.5 所示的 MTD 分区。

表 4.5　MTD 分区的空间大小转换

空间大小（十六进制）	名字	空间大小（十进制，字段）	空间大小（KB）	空间大小（MB）
0xa0000	misc	655 360	640	0.6
0x480000	recovery	4 718 592	4608	4.5
0x300000	boot	3 145 728	3072	3.0
0xf800000	system	260 046 848	253 952	248.0
0xa0000	local	655 360	640	0.6
0x2800000	cache	41 943 040	40 960	40.0
0x9500000	datadata	156 237 824	152 576	149.0

更多资源请访问稀酷客(www.ckook.com)

表 4.5 中的数值也可以通过使用 df 命令进行验证。df 命令可得到挂载的文件系统列表，以及每一个文件系统的总空间、已经使用的空间和可用的空间。以下是对 /system: 使用 df 命令获得的数据：

```
ahoog@ubuntu:~$ adb shell df /system
/system: 253952K total, 243724K used, 10228K available (block size 4096)
```

可以看出，转换表中与使用 df 命令所得到的 /proc/mtd 的空间大小是一致的。

在对 Android 中的 NAND 闪存和 MTD 有了基本了解后，下面将讨论 Android 中所使用的各类文件系统。

4.4 文件系统

与很多 Linux 系统一样，Android 中使用了数个文件系统，很多用于启动和运行系统。在讨论几个文件系统的同时，我们讨论的重点将放在存储用户数据的分区上，特别是 EXT、FAT32 和 YAFFS2 文件系统。

我们可以通过查看文件 /proc/filesytem 的内容来确定 Linux 内核（也就是 Android）所支持的文件系统。在我们参考用的 HTC Incredible 上，以上文件包含以下内容：

```
ahoog@ubuntu:~$ adb shell cat /proc/filesystems
nodev   sysfs
nodev   rootfs
nodev   bdev
nodev   proc
nodev   cgroup
nodev   tmpfs
nodev   debugfs
nodev   sockfs
nodev   pipefs
nodev   anon_inodefs
nodev   inotifyfs
nodev   devpts
        ext3
        ext2
nodev   ramfs
        vfat
        yaffs
        yaffs2
```

在 HTC Incredible 支持的 18 个文件系统中，以 NAND 闪存或 SD 卡这样的物理终端设备作为支持的只有 5 个。其他的文件系统具有“nodev”性质，即它们本质上是虚拟文件系统，不被写入到任何的物理终端设备中。再进一步，只有 6 个“nodev”文件系统实际应用在终端设备中：

1）rootfs

2）tmpfs

3）cgroup

4）proc

更多资源请访问稀酷客(www.ckook.com)

5）sysfs

6）devpts

以下 3 个终端设备支持的文件系统得到实际应用：

1）ext3

2）yaffs2

3）vfat

在以下章节中将对大多数文件系统进行简要介绍，并对用于存储绝大部分用户数据的 YAFFS2 文件系统进行深入分析。

4.4.1　rootfs、devpts、sysfs 和 cgroup 文件系统

很多 Linux 中的文件系统用于对系统进行启动、操作和管理，通常也就不包含对取证调查来说有用的信息。但是，安全工程师和研究人员还是需要对这些文件系统进行认真检查，包括内核的内部运行状况，为的是努力识别系统中存在的安全漏洞以及其他弱点。我们将快速浏览一下 Android 中 4 个与基础架构相关的文件系统。

首先，rootfs 是内核在系统开启时挂载根（root）文件系统之处（整个目录树的顶级节点，用一个斜杠 / 代表）。为了能够完成启动过程，内核需要访问核心的文件和共享程序库，因此需要挂载根文件系统。随着内核逐步完成启动过程，其他的文件系统作为根文件系统的子目录被挂载。例如（本章后续讨论中还有更多实例），根文件系统挂载在 / 之下，包含有关键文件。然后，一个更加完整的将被挂载在 /system 之下。在 shell 会话行，使用“ls -l”命令可以看到根文件系统以及所包含的目录，可以看到已经挂载的文件系统以及在根文件系统中所处的目录。

devpts 文件系统用于为 Android 中端设备提供模拟终端会话界面，类似于使用 telnet 或 ssh 与一个传统的 UNIX 服务器进行连接。每当连接到一个虚拟终端，在 /dev/pts 下就会生成一个新的节点。例如，如果在 Android 终端设备有一个 adb shell 连接，则 /dev/pts 会显示以下信息：

```
ahoog@ubuntu:/dev/pts$ adb shell
$ ls -l /dev/pts
crw------- shell    shell    136,   0 2011-02-01 10:00 0
```

在下一个例子中，可以看到存在两个 adb shell 连接以及一个由安装在终端设备上的应用程序产生的终端应用连接：

```
ahoog@ubuntu:~$ adb shell ls -l /dev/pts
crw------- shell    shell    136,   2 2011-02-01 10:02 2
crw------- app_105  app_105  136,   1 2011-02-01 10:02 1
crw------- shell    shell    136,   0 2011-02-01 10:00 0
```

可以看到，最初的 /dev/pts/0 连接存在。同时，出现了另外两个连接，其中一个源于一个正在运行的具有唯一用户名的终端应用（app_105）。

sysfs 是另外一个虚拟文件系统，它包含了终端设备上的系统配置和控制文件。在

更多资源请访问稀酷客(www.ckook.com)

HTC Incredible 上，存在以下顶层目录：

```
ahoog@ubuntu:/dev/pts$ adb shell ls -l /sys
drwxr-xr-x root     root              2011-02-01 11:06 fs
drwxr-xr-x root     root              2011-01-31 15:42 devices
drwxr-xr-x root     root              2011-02-01 11:06 dev
drwxr-xr-x root     root              2011-02-01 11:06 bus
drwxr-xr-x root     root              2011-02-01 10:02 class
drwxr-xr-x root     root              2011-02-01 11:06 firmware
drwxr-xr-x root     root              2011-02-01 11:06 kernel
drwxr-xr-x root     root              2011-01-31 15:42 power
drwxr-xr-x root     root              2011-02-01 11:06 board_properties
drwxr-xr-x root     root              2011-02-01 11:06 module
drwxr-xr-x root     root              2011-02-01 11:06 block
drwxr-xr-x root     root              2011-02-01 11:06 android_touch
drwxr-xr-x root     root              2011-02-01 11:06 android_camera
drwxr-xr-x root     root              2011-02-01 11:06 camera_led_status
drwxr-xr-x root     root              2011-02-01 11:06 android_camera_awb_cal
```

如果好奇的话，可以通过对 /sys 运行 adb pull 而将文件获取到取证服务器上，然后就可以被任何用户读取。执行以下命令：

```
ahoog@ubuntu:/dev/pts$ adb pull /sys sys
pull: building file list...
<snip>
pull: /sys/camera_led_status/led_hotspot_status -> ./camera_led_status/
      led_hotspot_status
pull: /sys/camera_led_status/led_wimax_status -> ./camera_led_status/
      led_wimax_status
pull: /sys/camera_led_status/led_ril_status -> ./camera_led_status/led_ril_status
pull: /sys/android_camera_awb_cal/awb_cal -> ./android_camera_awb_cal/awb_cal
3370 files pulled. 0 files skipped.
0 KB/s (33334 bytes in 233.611s)
```

可以看到，大量的文件被获取，然后就可以使用所有的 Linux 工具对数据进行检验。虽然这些信息对于取证的价值还有待于进一步研究，但是很清楚的是，它可以提供终端设备上的底层信息，为安全研究提供帮助。例如，如果想要更多地了解和学习 NAND 终端设备，则可以查看以下目录：

```
ahoog@ubuntu:~/sysfs$ ls -l ./module/msm_nand/parameters/
total 12
-rw-r--r-- 1 ahoog ahoog 123 2011-02-01 09:11 info
-rw-r--r-- 1 ahoog ahoog   5 2011-02-01 09:11 pagesize
-rw-r--r-- 1 ahoog ahoog   8 2011-02-01 09:11 vendor
```

然后是查看每一个文件：

```
ahoog@ubuntu:~/sysfs$ cat ./module/msm_nand/parameters/vendor
Samsung

ahoog@ubuntu:~/sysfs$ cat ./module/msm_nand/parameters/info
<<  NAND INFO  >>
flash id          =5500BCEC
vendor            =Samsung
width             =16 bits
size              =512 MB
block count       =4096
```

更多资源请访问稀酷客(www.ckook.com)

```
page count        =64

ahoog@ubuntu:~/sysfs$ cat ./module/msm_nand/parameters/pagesize
2048
```

非常清楚的是，对 NAND 终端设备进行深入了解是取证与安全分析的一个重要步骤。对于存在的将近 3000 个文件，查看数据需要非常大的工作量。以下是一个快速查看文件名、路径、文件大小的方法，以便更容易地查看有关文件（试着去运行两个终端会话，并列出一个会话中的文件，然后使用复制 / 粘贴到“cat”，以将文件内容显示在另外一个终端会话中）：

```
ahoog@ubuntu:~$ find sysfs  -type f -ls | less
933783     4 -rw-r--r--    1 ahoog    ahoog           91 Feb  1 09:11
sysfs/board_properties/virtualkeys.atmel-touchscreen
933855     4 -rw-r--r--    1 ahoog    ahoog            2 Feb  1 09:11
sysfs/android_camera/node
933857     4 -rw-r--r--    1 ahoog    ahoog           22 Feb  1 09:11
sysfs/android_camera/sensor
933856     4 -rw-r--r--    1 ahoog    ahoog            2 Feb  1 09:11
sysfs/android_camera/cam_mode
933863     4 -rw-r--r--    1 ahoog    ahoog           32 Feb  1 09:11
sysfs/android_camera_awb_cal/awb_cal
933782     4 -rw-r--r--    1 ahoog    ahoog            2 Feb  1 09:11
sysfs/power/pm_trace
933779     4 -rw-r--r--    1 ahoog    ahoog            5 Feb  1 09:10
sysfs/power/wait_for_fb_wake
933781     4 -rw-r--r--    1 ahoog    ahoog            2 Feb  1 09:11
sysfs/power/pm_trace_mask
<snip>
```

提示：其他有关 sysfs 的分析

除了通过手工方式查看 sysfs 文件系统，在互联网上还有很多提供背景信息的相关资源。其中一份资源是由 Patrick Mochel 撰写的一篇论文，提供很多有帮助的背景信息（The sysfs filesystem, n.d.）。

最后一个虚拟文件系统是 cgroups，用于对 Linux 文件系统中的任务进行跟踪与聚合。在 HTC Incredible 上共产生两个 cgroups 文件系统：/dev/cpuct1 和 /acct 中各一个。虽然进一步的分析也许会得到一些结果，在取证分析中，统计数据通常没有什么用处。

4.4.2　proc

proc 文件系统在 /proc 目录下以结构化的方式提供关于内核、进程和配置参数方面的详细信息。shell 用户可以查看其中的某些文件。但是，除非拥有 root 访问权限，否则很多文件是无法被访问的。正如从前，查看 proc 文件系统的一个方法就是使用 pull 将文件从 Android 终端设备上获取到取证工作站上。此操作会耗费一些时间，并有可能造成某些文件被挂起而导致不完整的复制。

```
ahoog@ubuntu:~$ adb pull /proc proc
pull: building file list...
```

更多资源请访问稀酷客(www.ckook.com)

在 HTC Incredible 上，在复制进程 76 时，以上进程被挂起，最后只能使用 Ctrl+C 将其取消：

```
pull: /proc/76/task/5959/auxv -> proc/76/task/5959/auxv
failed to copy '/proc/76/task/5959/auxv' to 'proc/76/task/5959/auxv':
Permission denied
pull: /proc/76/task/5959/environ -> proc/76/task/5959/environ
failed to copy '/proc/76/task/5959/environ' to 'proc/76/task/5959/environ':
Permission denied
pull: /proc/76/task/6993/fd/16 -> proc/76/task/6993/fd/16
 ^C

ahoog@ubuntu:~/proc$ du -hs
25M     .
ahoog@ubuntu:~/proc$ find . -type f | wc -l
5998
```

在被取消之前，来自于将近 6000 个文件的 25MB 数据被成功地获取。与对 sysfs 进行检查相似，可以使用 find 命令去找到并罗列文件，以手动方式查看数据：

```
ahoog@ubuntu:~/proc$ find .  -type f -ls | less
934618    4 -rw-r--r--   1 ahoog    ahoog          93 Feb  1 09:48 ./cpu/alignment
413065    4 -rw-r--r--   1 ahoog    ahoog           8 Feb  1 09:50 ./25/wchan
413062    4 -rw-r--r--   1 ahoog    ahoog           2 Feb  1 09:50 ./25/oom_score
413063    4 -rw-r--r--   1 ahoog    ahoog          20 Feb  1 09:50 ./25/cgroup
413069    0 -rw-r--r--   1 ahoog    ahoog           0 Feb  1 09:51 ./25/maps
566313    4 -rw-r--r--   1 ahoog    ahoog          85 Feb  1 09:50 ./25/net/sockstat6
566315    4 -rw-r--r--   1 ahoog    ahoog         108 Feb  1 09:50 ./25/net/if_inet6
<snip>
```

另外一个方法是直接从 adb shell 上查看 /proc 目录下的某些文件，如下所示：

```
ahoog@ubuntu:~$ adb shell
$ cd /proc
$ cat cpuinfo
Processor       : ARMv7 Processor rev 2 (v7l)
BogoMIPS        : 162.54
Features        : swp half thumb fastmult vfp edsp thumbee neon
CPU implementer : 0x51
CPU architecture: 7
CPU variant     : 0x0
CPU part        : 0x00f
CPU revision    : 2

Hardware        : incrediblec
Revision        : 0002
Serial          : 0000000000000000
```

从 proc 文件系统中可以获得关于终端设备的有价值的信息。检查者应该对这些文件进行审计，并应该直接从 /proc 目录下的文件开始，而不是从任何子目录开始。

4.4.3 tmpfs

tmpfs 文件系统存储所有以随机访问内存（RAM）作为后盾的虚拟内存中的文件，还包括在终端设备上可能存在的互换（swap）或缓存（cache）文件。目前，在大多数的 Android 终端设备上没有互换空间。然而，某些售后市场上的销售固件具备这个功能。

更多资源请访问稀酷客(www.ckook.com)

tmpfs 的优点是通过使用随机访问内存（RAM），存储器的速度非常快，并且是非永久性的，在重启时不会被保存。

当然，对取证检查师来说，这变成了一个挑战。如果重要的数据存在于 tmpfs 挂载点上，对这些数据的收集必须在终端设备重启或关机前完成。它同时也提供了一个独特的机会，因为 tmpfs 通常可以被 shell 用户读取，取证程序可以被复制并在 tmpfs 中运行，也不需要对 NAND 闪存或 SD 卡做任何改变。这就使检查者在不需要对 NAND 闪存和 SD 卡做任何改变的情况下从 Android 终端设备上获得取证数据。

> **警示：调查 tmpfs**
>
> 如果感兴趣的是对终端设备的内存进行分析，不建议对 tmpfs 做任何改变，除非已经有了感兴趣部分的内存的取证副本（copy）。

在 HTC Incredible 上，标准的安装具有以下 4 个 tmpfs 挂载点：

- /dev
- /mnt/asec
- /app-cache
- /mnt/sdcard/.android secure

/dev 目录下所包含的终端设备文件允许内核对连接的诸如 NAND 闪存、SD 卡、字符设备等终端设备进行读和写。/mnt/asec 和 /mnt/sdcard/.android_secure 目录是最新增加到 Android 系统中的，允许将应用程序存储在 SD 上，而不是 /data/data 中，因为 SD 卡能够提供更大的存储空间。

有趣的是，/app-cache 目录也是新增加到 Android 中的，看起来是为了提供更多的 tmpfs 空间供应用程序使用。在 HTC Incredible 上，Web 浏览器（com.andrew.browser）在 /app-cache 中创建了一个目录，用于保存 Web 浏览的缓存文件。

```
ahoog@ubuntu:~$ adb shell ls -l / | grep app-cache
drwxrwxrwt root     root              2011-01-31 15:42 app-cache

ahoog@ubuntu:~$ adb shell
$ ls -l /app-cache
drwxr-xr-x app_12   app_12            2011-01-31 15:42 com.android.browser
```

在以上第一个列表中，可以看到 app-cache 目录的设置使所有用户都具有读、写和浏览的访问权限。此外，“rwxrwxrwt”访问权限的最后一位“t”表明只有作为此目录拥有者的 root 才可以对此目录进行删除或更改名称操作。

以上的第 2 个列表显示 app-cache 下的子目录，在此例中只有一个 com.android.browser。但是，当我们进入这个目录的更深层，就可以发现我们最感兴趣的目录和文件只允许应用程序本身（即用户名为 app_12 的 com.android.browser）或 root 访问这些文件。

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ adb shell ls -l /app-cache/com.android.browser/cache
drwxrwx--x app_12   app_12            2011-01-31 15:57 webviewCache

ahoog@ubuntu:~$ adb shell ls -l /app-cache/com.android.browser/cache/webviewCache
opendir failed, Permission denied
```

由于我们对此终端设备拥有 root 访问权限，以下是此目录包含的内容：

- 1.4 MB of data
- 64 files
- 18 ASCII files (mostly CSS and JavaScript)
- 1 empty file
- 9 GIF files
- 5 HTML files
- 11 JPEG files
- 17 PNG files
- 3 UTF-8 files

```
ahoog@ubuntu:~/app-cache/com.android.browser/cache/webviewCache$ du -hs
1.4M    .

ahoog@ubuntu:~/app-cache/com.android.browser/cache/webviewCache$ find . -type f  |
wc -l
64

ahoog@ubuntu:~/app-cache/com.android.browser/cache/webviewCache$ find . -type f |
xargs file
./1d15a326: ASCII text, with very long lines
./982785ed: HTML document text
./fe3f9f59: GIF image data, version 89a, 1 x 1
./1cdb9fc0: PNG image, 140 x 44, 8-bit colormap, non-interlaced
./1c32cdf6: JPEG image data, JFIF standard 1.01
./aacce58f: ASCII C program text
./ad01e1f2: PNG image, 100 x 66, 8-bit/color RGB, non-interlaced
./39e4b622: PNG image, 140 x 44, 8-bit colormap, non-interlaced
./fcf0e4eb: PNG image, 64 x 3, 8-bit colormap, non-interlaced
./9244746c: PNG image, 140 x 44, 8-bit colormap, non-interlaced
./13bf2ef2: ASCII C program text, with very long lines
./44e36c36: PNG image, 560 x 370, 8-bit/color RGB, non-interlaced
./ffbedd54: JPEG image data, JFIF standard 1.01
./c780272a: ASCII English text
<snip>
```

如果在 Ubuntu 工作站上浏览 webviewCache 目录，可以很容易地预览映像及其他文件（见图 4.2）。

/app-cache tmpfs 目录中有对于取证调查来说非常重要的信息。因此，需要进一步强调，为发展的需要培养一线的响应者，以确保终端设备得到恰当使用，以使取证调查的效果达到最大化。

更多资源请访问稀酷客(www.ckook.com)

图 4.2　在 Ubuntu 工作站上浏览 webviewCache

4.4.4　扩展文件系统

扩展文件系统（Extended File System）是 Linux 作为操作系统开发时事实上的标准文件系统。众所周知，Linux 支持很多不同的文件系统。但是，默认的文件系统是 EXT。自从 1992 年开发完成了 EXT 后，至今已经推出了 3 个后续的版本：EXT2、EXT3 和 EXT4。

虽然 EXT 是大多数笔记本电脑、台式机和 Linux 服务器的组成部分，它并没有出现在早期的 Android 终端设备中。然而，EXT 从 2010 年开始出现在终端设备中，在 2010 年 12 月 9 日，谷歌在其 Android 开发者博客中宣布将会有更多的 Android 终端设备从 YAFFS 文件系统过渡到 EXT 文件系统（Android developers blog, n.d.）。从 YAFFS 文件系统过渡到 EXT 文件系统看似由在网上热烈讨论（Way, n.d.）的以下几个因素决定，包括：

- 越来越多的 Android 终端设备从不够成熟的 NAND 闪存过渡到更规律的模块终端设备（eMMC）；
- EXT4 是 Linux 中的标准文件系统，完全支持 UNIX 系统中的访问权限以及语意；
- EXT4 非常稳定，并且性能很高；
- YAFFS 只有单线程，因此会在即将面市的双核系统中变成瓶颈。

第一个使用 EXT4 的 Android 终端设备是谷歌的 Nexus S 智能手机，并且预计很多运行 Android Honeycomb 版本的平板终端设备都将使用这个新的文件系统。由于目前只有一款 Android 终端设备使用 EXT4，预计随着时间的推移还会发生很多变化。当前，Nexus 在以下挂载点使用 EXT4：

- 系统映像（只读，/system）；
- 本地用户数据（可读写，/data/data）；
- 缓存分区（可读写，/cache 及其他的可能性）。

更多资源请访问稀酷客(www.ckook.com)

从取证分析的角度来看，EXT4 只不过是另外一个检查师需要了解以及取证工具需要支持的文件系统。当然，大多数取证工具不能完全（甚至在名义上）支持 EXT4，因此，这就造成了一些问题。文件雕复技术（file carving technique）肯定会有效，因此，可以预计随着时间的推移，会有更多的取证软件开始支持此文件系统。

4.4.5 FAT32/VFAT

Android 终端设备通常具有一个或多个微软的 FAT32 分区，一般在 SD 卡上和 eMMC。使用这个脆弱的文件系统的理由不是因为超级的设计，而是因为与其他操作系统的兼容性。微软的 FAT32 文件系统在包括 Mac OS X、所有的 Windows 版本（当然如此）、Linux 等大多数的操作系统中得到广泛支持。这就意味着存储在 FAT32 分区上的 Android 数据可以很容易在其他文件系统中被读取和修改，甚至被删除。

在 Linux 中，FAT32 分区的文件系统驱动程序的名称叫做 VFAT，但是千万不要与微软早期的虚拟 FAT 文件系统混为一谈，此系统通过增加支持长文件名等一系列功能给 FAT16 和 FAT32 文件系统之间建立一座桥梁。在 HTC Incredible 上，有 3 个挂载点使用 FAT32：

- /mnt/sdcard；
- /mnt/secure/asec；
- /mnt/emmc。

回顾在第 3 章中讨论的 USB 接口，在将一个 Android 终端设备连接到另外一台计算机时，一个选择就是显露出终端设备的 USB 大容量存储器（UMS）接口以进行文件共享。在 HTC Incredible 上，/mnt/sdcard 和 /mnt/emmc 分区都可以作为 UMS 终端设备而通过 USB 接口呈现给其他操作系统。

/mnt/secure/asec 分区是 SD 卡上的一个加密分区，Android 终端设备可以在此分区存储应用程序。当能够运行在 SD 卡上应用程序的功能被推出时，由于对文件的访问权限没有在 VFAT/FAT32 分区中进行维护，安全工程师们自然会担心应用数据会很容易地遭到损坏或被俘获。因此，应用程序（.apk 文件）在物理终端设备上被加密，而当需要使用时，应用程序被解密并被暂时存储在 /mnt/asec 下或者是其他一个由设计指定的位置。

正如以上讨论的那样，可移走的 SD 卡挂载在 /mnt/sdcard 点上，在上面通常存储着相片、视频、缩略图、下载的文件、文本到语音临时文件、谷歌地图导航数据以及谷歌市场应用程序产生的数据。新的 /mnt/emmc 是一个 FAT32 分区，不能够被移走，存在于终端设备内的存储器中。在目前讨论过的终端设备中，eMMC 按照 FAT32 被格式化，同样也是为了实现互操作。

4.4.6 YAFFS2

当第一款 Android 终端设备被推出时，很多人对于一个相对不知名的文件系统在系统中扮演着关键的角色感到非常吃惊。YAFFS（Yet Another Flash File System）是一个

更多资源请访问稀酷客(www.ckook.com)

开放源码的文件系统，专门面向 NAND 闪存而开发，使用 GNU 公共许可（GNU Public License，GPL），而对于不愿意签署 GNU 公共许可的人则使用商业许可协议（YAFFS licence FAQs, n.d.）。Android 终端设备使用的最新版的 YAFFS（YAFFS2）更加严格地遵守 NAND 闪存指南，目的是改进 NAND 闪存的持久性，同时进行了优化，使其能够运行在内存缺乏的移动或嵌入式终端设备上。

YAFFS2 由新西兰的 Aleph One Ltd. 公司开发。为客户的需求所驱动，Aleph One 在 2001 年 12 月开始设计 YAFFS，在 2002 年 5 月公开发布了第一版源代码。系统的主要设计者（或者肯定是知名度最高的参与者）名叫 Charles Manning，被誉为“嵌入清洁工”，已经有 20 年开发和清洁嵌入式系统的经验（YAFFS: the NAND-specific flash, n.d.）。Charles 在 YAFFS 邮件列表（mailing list）中非常活跃，是公认的 YAFFS 和 YAFFS2 的专家。

提示：更多的 YAFFS2 方面的资源

对 YAFFS2 的内部设计感兴趣的分析师和工程师应该去阅读完整的文档（以及源代码，如果那也在自己所属的专业范围内的话），网站为 http://www.yaffs.net/，需要加入邮件列表（mailing list）。

YAFFS2 特别为日渐增加的 NAND 闪存终端设备而设计，具有一系列重要的解决此存储介质中严格要求的功能。YAFFS 具有以下特点：

- 是一个具有日志结构的文件系统（在意外失去供电时也能够保护数据）；
- 提供内置的损耗程度度量和纠错功能；
- 能够处理废块；
- 速度快，并在随机访问内存（RAM）中占用很小的空间。

然而，由于在 Android 之前，YAFFS 的使用非常有限，目前还没有取证工具（无论是商业化工具，还是开放源码工具）能够支持此文件系统。这就使取证分析师没有什么选择，只能去下载 YAFFS2 源代码，捕获其中分区的取证映像，使用一个编辑器将其打开，然后进行深入挖掘。虽然随着时间的推移，有些工具会被开发出来，但是，Android 走向 EXT4 的策略会降低商业的取证公司开发此类分析工具的热情和动力。

正如我们在 NAND 闪存讨论中看到的那样，YAFFS2 以块为单元通过 MTD 子系统对内存进行访问，每一块又包含一定数量的页（在 YAFFS 的文档和代码中则被叫做子块）。当 YAFFS2 准备向 NAND 闪存写入数据时，它会将数据以及元数据结构传递给 MTD。然后，由 MTD 负责将数据和元数据写入（以及读取）NAND 闪存。

对大多数 Android 终端设备来说，MTD 子系统以块为单位访问 NAND 闪存，每一块又分成 64 个子块，每一个子块又包含 2048 字节的数据（因此，每一块为 128KB）加上一个 64 字节的离线 / 空闲区域（OOB），离线区域中存储着各类标记以及元数据。当一个块在经过分配后要被写入时，它被赋予一个序号，此序号从 1 开始，并随着每一个分配的新块而递增。

更多资源请访问稀酷客(www.ckook.com)

所有存储在 YAFFS2 中的数据结构都被称为客体（Object），客体可以是文件、目录、符号化的链接以及硬链接。每一个子块中存储着一个 yaffs_ObjectHeader（客体元数据）或者是此客体的数据。Yaffs_ObjectHeader 用于记录各类信息，包括客体的类型、父客体、客体名、用以提高查寻的速度的客体名校验和、访问权限及拥有者的信息、MAC 信息以及客体的大小（如果此客体是一个文件的话）。

在 64 字节的 OOB 区域 / 空闲区域里，YAFFS2 不但保存关于相关子块的关键信息，而且与 MTD 子系统共享此区域。以下是关键的 YAFFS2 标记：

- 1 字节：块状态（0xFF 代表可用块，其他数值代表废块）；
- 4 字节：子块 ID（0 表示此子块存储 yaffs_ObjectHeader，其他数值表示存储数据）；
- 4 字节：客体 ID（与传统的 UNIX inode 相同）；
- 2 字节：子块中数据量（除了最后一个子块外，所有的子块都被数据完全占用）；
- 4 字节：此块的序号；
- 3 字节：标记的纠错码（ECC）（在 Android 中，由 MTD 负责处理）；
- 12 字节：数据的纠错码（ECC）（在 Android 中，由 MTD 负责处理）。

当一个客体被改变时，一个新的 yaffs_ObjectHeader 会被写入闪存，这是因为 NAND 闪存里的内容在被擦除前只能被写入一次。旧的数据以及相应的 ObjectHeader 仍然存在，但是通过查看序号数值，它们就会被忽略，不再被使用。这个过程完全符合关于 NAND 闪存中的块只能被写入一次的原则（只能写入一次，然后在不再需要时被擦除）。当然，这对于数据恢复来说非常有益，因为被改变的或者被删除的数据依然存在于 NAND 闪存之中，除非块已经经过垃圾收集过程而被擦除，以准备好接受写入新的数据。

同样，在 YAFFS2 中，当一个文件被删除时，包含 ObjectHeader 的文件夹（目录）被放入一个特殊的、叫做 unlinked 的不显示的目录中。被删除的文件会继续存留在这个不显示的目录中，直到这个文件中所有的子块都被擦除。实现此功能的方式是，文件系统会一直对文件中所有的残存子块进行监测，当残存子块个数降到 0 时，表明此文件已经不复存在。此后，文件系统就不再继续在 unlinked 目录（文件夹）中监测此文件。

虽然文件系统结构可以基于 OOB 区域和 ObjectHeader 进行完全重建，但是这样并不完全充分，特别是随着 NAND 闪存空间的不断增加。因此，文件系统结构被载入并被在随机访问内存（RAM）中（需要时，会对 NAND 闪存进行写入）使用数节点结构（T-node）方式进行维护，以跟踪所有已经分配出去的子块。每一个 T-node 具有固定的 32 字节，在最低层（即第 0 层）存储着用于找到第一个子块 ID 的索引。当文件变大时，层数也相应增加，每一层都有 8 个指针指向其他的 T-node。

重建时，YAFFS2 将每一个子块按照其在块中的分配顺序读出，从最后一个子块开始，从后往前读，像在随机访问内存（RAM）中的 T-node 结构那样填充整个文件系统结构。这样做就需要对整个 NAND 进行扫描，是一个非常耗时的操作。为了解决这个问题，在 YAFFS2 中设计了检查点机制，此机制更倾向于随机访问内存（RAM）的结构，而不是被成功卸载后的 NAND 闪存（使用 10 块）。

更多资源请访问稀酷客(www.ckook.com)

为了了解 YAFFS2，还需要介绍几个关键的概念。首先，垃圾收集是形成一个队列进行等待，而当需要时，则是在每一次写入系统时完成。如果一个块中所有的子块不再继续使用，就可以对此块进行垃圾收集处理。系统也能够确定最“脏”的块，并将块内所有的子块复制到其他的块中，然后就可以对此块进行垃圾收集处理。重新使用一个块的方法就是通过写入全部的“1”（0xFF）将它擦除。

在 Android 终端设备中，查看 /proc/yaffs 文件可以获得 YAFFS2 文件系统的详细信息：

```
ahoog@ubuntu:~$ adb shell cat /proc/yaffs
YAFFS built:Sep 30 2010 18:41:07
$Id$
$Id$

Device 0 "system"
startBlock.........  0
endBlock...........  1983
totalBytesPerChunk.  2048
nDataBytesPerChunk.  2048
chunkGroupBits.....  0
chunkGroupSize.....  1
nErasedBlocks......  10
nReservedBlocks....  5
blocksInCheckpoint.  3
nTnodesCreated.....  9600
nFreeTnodes........  62
nObjectsCreated....  1100
nFreeObjects.......  20
nFreeChunks........  5690
nPageWrites........  0
nPageReads.........  551024
nBlockErasures.....  0
nGCCopies..........  0
garbageCollections.  0
passiveGCs.........  0
nRetriedWrites.....  0
nShortOpCaches.....  10
nRetireBlocks......  0
eccFixed...........  0
eccUnfixed.........  0
tagsEccFixed.......  0
tagsEccUnfixed.....  0
cacheHits..........  0
nDeletedFiles......  0
nUnlinkedFiles.....  460
nBackgroundDeletions 0
useNANDECC.........  1
isYaffs2...........  1
inbandTags.........  0

Device 1 "datadata"
startBlock.........  0
endBlock...........  1191
totalBytesPerChunk.  2048
nDataBytesPerChunk.  2048
chunkGroupBits.....  0
chunkGroupSize.....  1
nErasedBlocks......  11
nReservedBlocks....  5
blocksInCheckpoint.  0
```

更多资源请访问稀酷客(www.ckook.com)

```
nTnodesCreated.....  3700
nFreeTnodes........  119
nObjectsCreated....  3000
nFreeObjects.......  84
nFreeChunks........  50903
nPageWrites........  2368440
nPageReads.........  1028358
nBlockErasures.....  38623
nGCCopies..........  323313
garbageCollections.  18186
passiveGCs.........  2454
nRetriedWrites.....  0
nShortOpCaches.....  10
nRetireBlocks......  0
eccFixed...........  0
eccUnfixed.........  0
tagsEccFixed.......  0
tagsEccUnfixed.....  0
cacheHits..........  1017819
nDeletedFiles......  0
nUnlinkedFiles.....  643647
nBackgroundDeletions 0
useNANDECC.........  1
isYaffs2...........  1
inbandTags.........  0
```

我们可以看到很多有用的细节，比如在挂载在 /data/data 点上的 datadata YAFFS2 分区里。通过在 /proc/yaffs 列表查看此分区，我们可以得到以下信息：

1）共有 1192 个块（0 ～ 1191），每一个块由 64 个子块（每一个字块有 2048 字节）组成。因此，128KB×1192=152 576KB，我们也可以通过使用 df 命令或者查看 /proc/mtd 对此进行验证；

2）对页（子块）的读、写次数以及对块的擦除次数，告诉我们 NAND 闪存的使用情况；

3）垃圾收集的一个策略是找到基本上已经没有在使用的块，将残存的数据复制出去，然后将此块标示为可以进行垃圾收集。我们可以看到此种情况的发生率很高（323 313）；

4）我们可以看到没有检测到 ECC 错误；

5）YAFFS2 元数据报告有超过 643 000 个 unlinked 文件；

6）YAFFS2 没有使用软件版的 ECC，而是依赖于 MTD 或 NAND 闪存。

如果查看一个读、写使用率比 /data/dada 低的分区，会注意到显著的区别。查看 /proc/yaffs 文件可以提供一些用于解释纠错码、碎片数据等方面的背景信息。

更深入地了解 YAFFS2 的最好方法就是直接创建、修改并查看文件系统。这一点在本书中用于其他练习的 Ubuntu 工作站上是可以做到的。由于我们已经建立了一个虚拟机并安装了核心软件包（包括 C 语言编译程序及相关支持软件），我们现在需要安装 mtd-utils 软件包：

```
ahoog@ubuntu:~$ sudo apt-get install mtd-utils
```

下一步，下载最新的 YAFFS2 源代码：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ curl http://www.aleph1.co.uk/cgi-bin/viewvc.cgi/
yaffs2.tar.gz?view=tar > yaffs2.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  391k    0  391k    0     0   242k      0 --:--:--  0:00:01 --:--:--  277k
```

然后，对 yaffs2.tar.gz 进行解包及编译后，就可以使用内核模块：

```
ahoog@ubuntu:~$ tar xzf yaffs2.tar.gz
ahoog@ubuntu:~$ cd yaffs2/
ahoog@ubuntu:~/yaffs2$ make
make -C /lib/modules/2.6.35-25-generic/build M=/home/ahoog/yaffs2 modules
make[1]: Entering directory `/usr/src/linux-headers-2.6.35-25-generic'
  CC [M]  /home/ahoog/yaffs2/yaffs_mtdif.o
<snip>
```

下一步，载入需要的内核模块在随机访问内存（RAM）中模拟一个 MTD（除非有直接插入后可使用的 NAND 闪存），并挂载一个 YAFFS2 分区。

首先，给要挂载的文件系统在主目录下创建一个挂载点：

```
ahoog@ubuntu:~$ cd; mkdir -p ~/mnt/yaffs2
```

接下来，载入几个内核模块，支持 MTD 功能：

```
ahoog@ubuntu:~$ sudo modprobe mtd
[sudo] password for ahoog:
ahoog@ubuntu:~$ sudo modprobe mtdblock
```

接下来，使用内核模块 nandsim 来创建仿真的 NAND 闪存：

```
ahoog@ubuntu:~$ sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xa2
third_id_byte=0x00 fourth_id_byte=0x15
```

提示：关于 Linux MTD 的更多信息

Linux MTD 网站上提供更多的关于 nandsim 的详细信息。此网站提供很多背景情况介绍以及 MTD 技术支持信息，通过访问可以获得完整的细节。

以下是 Linux MTD 网站上提供的关于 nandsim 的更多详细信息，用于参考（NAND FAQ, n.d.）

NAND 仿真程序（nandsim）是一个非常有用的调试和开发工具，它可以在随机访问内存（RAM）或文件中对 NAND 闪存进行仿真。通过使用闪存的 ID 字节来选择仿真闪存的类型，闪存 ID 可以参考相关闪存的用户手册，使用“Read ID”命令获取。以下是输入参数的实例：

- *modprobe nandsim first_id_byte=0x20 second_id_byte=0x33—16MiB, 512 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0x35—32MiB, 512 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0x36—64MiB, 512 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0x78—128MiB, 512 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0x71—256MiB, 512 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0xa2 third_id_byte=0x00 fourth*

更多资源请访问稀酷客(www.ckook.com)

_id_byte=0x15—64MiB, 2048 bytes page;

- *modprobe nandsim first_id_byte=0xec second_id_byte=0xa1 third_id_byte=0x00 fourth _id_byte=0x15—128MiB, 2048 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0xaa third_id_byte=0x00 fourth _id_byte=0x15—256MiB, 2048 bytes page;*
- *modprobe nandsim first_id_byte=0x20 second_id_byte=0xac third_id_byte=0x00 fourth _id_byte=0x15—512MiB, 2048 bytes page;*
- *modprobe nandsim first_id_byte=0xec second_id_byte=0xd3 third_id_byte=0x51 fourth _id_byte=0x95—1GiB, 2048 bytes page;"*

建立了一个仿真 NAND 闪存后，与在 Android 终端设备上所做的一样，可以通过查看 /proc/mtd 对大小和分区进行验证：

```
ahoog@ubuntu:~$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 04000000 00020000 "NAND simulator partition 0"
```

以上结果显示，有一个 MTD 分区（mtd0），擦除空间大小为十六进制 0x20000 字节（128KB），总空间大小为十六进制 0x4000000 字节（65 536KB，即 64MB）。下一步，需要将 YAFFS2 内核模块载入内存：

```
ahoog@ubuntu:~$ sudo insmod ~/yaffs2/yaffs2.ko
```

在将 YAFFS2 文件系统挂载前，我们先使用一个十六进制的编辑器看一下还未初始化的仿真 NAND 闪存。如果使用类似 dd 或 xxd 的工具查看 NAND 闪存设备，OOB 区域不会显示，因为此区域在大多数工具中都不显示出来。这是为了预防混合的 64 位元数据扰乱某些程序的正确运行（想象当显示一个 PDF 文件时，在显示每个 2KB 大小的子块内的数据后又显示跟随在后的一个 64 字节的二进制 OOB 区域中的数据会产生什么样的体验）。如果要从 NAND 闪存上读取包括 OOB 区域在内的所有数据，需要使用 nanddump 程序，它属于已经安装好的 mtd-utils 软件包的一部分，读取的数据将会以二进制的格式返回。nanddump 中有以下几个选项可以使用：

```
ahoog@ubuntu:~$ nanddump --help
Usage: nanddump [OPTIONS] MTD-device
Dumps the contents of a nand mtd partition.

          --help               Display this help and exit
          --version            Output version information and exit
-a        --forcebinary        Force printing of binary data to tty
-c        --canonicalprint     Print canonical Hex+ASCII dump
-f file   --file=file          Dump to file
-i        --ignoreerrors       Ignore errors
-l length --length=length      Length
-n        --noecc              Read without error correction
-o        --omitoob            Omit oob data
-b        --omitbad            Omit bad blocks from the dump
-p        --prettyprint        Print nice (hexdump)
-q        --quiet              Don't display progress and status messages
-s addr   --startaddress=addr  Start address
```

更多资源请访问稀酷客(www.ckook.com)

在这里，我们使用 nanddump 来获取包括离线区域（OOB）在内的全部 NAND 上的数据，并将结果通过一个十六进制的编辑器（xxd）进行显示：

```
ahoog@ubuntu:~$ sudo nanddump -a /dev/mtd0ro | xxd | less
0000000: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000010: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000020: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000030: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000040: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000050: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000060: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000070: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000080: ffff ffff ffff ffff ffff ffff ffff ffff  ................
<snip>
```

需要注意的是，仿真 NAND 闪存中包含有期望在空白或被擦除的 NAND 闪存中存在的 0xFF 值。以上命令可以用于查看原始的 NAND 闪存是如何随着对其初始化及随后对文件系统的修改而发生相应改变的。

到此，我们终于可以挂载一个 YAFFS2 文件系统：

```
ahoog@ubuntu:~$ sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/yaffs2/
```

我们可以验证文件系统已经正确挂载，并可以被访问使用：

```
ahoog@ubuntu:~$ sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/yaffs2/
ahoog@ubuntu:~$ mount | grep yaffs2
/dev/mtdblock0 on /home/ahoog/mnt/yaffs2 type yaffs2 (rw)

ahoog@ubuntu:~$ ls -la ~/mnt/yaffs2/
total 8
drwxr-xr-x 1 root  root  2048 2011-02-03 11:37 .
drwxr-xr-x 3 ahoog ahoog 4096 2011-02-03 07:21 ..
drwx------ 1 root  root  2048 2011-02-03 11:37 lost+found
```

从以上结果可以看到，一个 YAFFS2 文件系统已经挂载到 ~/mnt/yaffs2，具有读 / 写（read/write）访问权限。虽然还没有任何文件，目录中有一个 lost+found 虚拟目录，用于存储那些所属目录还未定的文件及目录。再次使用 xxd 十六进制编辑器查看仿真 NAND 闪存终端设备，仍然可以看到 0xFF。但是，如果使用以下命令写入一个文件：

```
ahoog@ubuntu:~$ nano -w ~/mnt/yaffs2/book.txt
```

“Android Forensics and Mobile Security”作为此文件的内容，其后再次查看未进一步处理的 NAND 闪存时，可以看到 YAFFS2 结构中出现了 ObjectHeaders、Objects 及文件内容。以下是使用 nanddump 以及超微变化过的 xxd 命令所显示的部分十六进制的内容，使用 xxd 命令时使用了 -a 选项，以忽略那些内容为 0x00 的输入行：

```
ahoog@ubuntu:~$ sudo nanddump -a /dev/mtd0ro | xxd -a | less
0000000: 0100 0000 0100 0000 ffff 626f 6f6b 2e74  ..........book.t
0000010: 7874 0000 0000 0000 0000 0000 0000 0000  xt..............
0000020: 0000 0000 0000 0000 0000 0000 0000 0000  ................
*
00007e0: ffff ffff ffff ffff ffff ffff ffff ffff  ................
00007f0: ffff ffff ffff ffff ffff ffff ffff ffff  ................
```

更多资源请访问稀酷客(www.ckook.com)

```
0000800: ffff 0110 0000 0101 0010 0100 0080 0000  ................
0000810: 0000 2aaa aaaa 0400 0000 fbff ffff ffff  ..*.............
0000820: ffff ffff ffff ffff fff0 cfaa 5567 ffff  ............Ug..
0000830: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0000840: 416e 6472 6f69 6420 466f 7265 6e73 6963  Android Forensic
0000850: 7320 616e 6420 4d6f 6269 6c65 2053 6563  s and Mobile Sec
0000860: 7572 6974 790a 0a00 0000 0000 0000 0000  urity...........
0000870: 0000 0000 0000 0000 0000 0000 0000 0000  ................
*
0001040: ffff 0110 0000 0101 0000 0100 0000 2700  ..............'.
0001050: 0000 1900 0000 0800 0000 f7ff ffff ffff  ................
0001060: ffff ffff ffff ffff aa66 5bff ffff ffff  .........f[.....
0001070: ffff ffff ffff ffff ffff ffff ffff ffff  ................
0001080: 0100 0000 0100 0000 ffff 626f 6f6b 2e74  ..........book.t
0001090: 7874 0000 0000 0000 0000 0000 0000 0000  xt..............
00010a0: 0000 0000 0000 0000 0000 0000 0000 0000  ................
```

从移位 0x0000000 开始，可以看到一个空白的 ObjectHeader，很清楚地使用 ASCII 码显示出文件名（book.txt）。在移位 0x0000800 至移位 0x000083F 之间，可以看到首先由 YAFFS2 再由 MTD 存储的离线区域（OOB）数据，此数据为二进制数据，内容必须要确定。实际的文件内容存储在 NAND 闪存上从移位 0x0000840 开始的区域内。在移位 0x0001040 至移位 0x000107F 之间，还存在一个 OOB 区域。随后，从移位 0x0001080 开始，ObjectHeader 被再次写入到 NAND 闪存上，这是因为第一个 ObjectHeader 代表的是一个内容为空的文件。一旦加入内容并被保存，数据就被写入到 NAND 闪存上，同时一个新的 ObjectHeader 也被写入到 NAND 闪存上。新的 ObjectHeader 体现的是关于 Object 的新的元数据，包括哪些子块中包含数据、MAC（修改、访问、改变）的数据等。

需要理解的一个关键点是 YAFFS2 无法简单地更新第一个 ObjectHeader，因为它首先必须擦除整个块。因此，它写入一个新的 ObjectHeader，产生一个更大的序列数，使其成为文件的最新 ObjectHeader。可以想到，除非进行垃圾收集，否则旧的 ObjectHeader 以及数据仍然存在于 NAND 闪存中，这就给运用取证技术恢复元数据以及数据内容本身提供了一个绝好的机会。

YAFFS 实例

在关于 YAFFS2 的最后一部分，构建一个假想的实例来展示在 NAND 闪存上如何写入 ObjectHeader 和 Object。此实例源于 Charles Manning 的书《How Yaffs Works》（How YAFFS works, n.d.）。建议对 YAFFS 内部感兴趣的读者好好研究一下此书。为简单起见，在此例中，在我们使用的 NAND 闪存中，每一个块由 4 个字块组成，处于擦除状态（0xFF）。在描述了每一个改变之后，我们使用一个表来显示 NAND 闪存中的内容。

首先，我们在 NAND 闪存中创建一个空文件，如表 4.6 所示。

表 4.6　创建空文件

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
0	0	500	0	live（正常）	空文件的 ObjectHeader，文件大小为 0

更多资源请访问稀酷客(www.ckook.com)

ObjectHeader 所指的是一个空文件。下一步，我们向文件内写入 3 个子块的数据，如表 4.7 所示。

表 4.7　写入 3 个子块的数据

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
0	0	500	0	live（正常）	空文件的 ObjectHeader，文件大小为 0
0	1	500	1	live（正常）	第一个子块的数据
0	2	500	2	live（正常）	第二个子块的数据
0	3	500	3	live（正常）	第三个子块的数据

至此都很容易理解。现在，我们让数据占据整个块。

> **提示：假想的 NAND 闪存**
>
> 需要记住的是在我们的假想 NAND 闪存中，每一个块中包含 4 个子块，而不像在常见的 NAND 闪存商业产品中那样有 64 个子块。

下一步，我们将文件进行存储，这样将造成在 NAND 闪存中写入一个新的 ObjectHeader，如表 4.8 所示。

表 4.8　存储文件带来新的 ObjectHeader

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
0	0	500	0	shrink/delete（压缩 / 删除）	过时的 ObjectHeader，最初为空文件建立，文件大小为 0
0	1	500	1	live（正常）	第一个子块的数据
0	2	500	2	live（正常）	第二个子块的数据
0	3	500	3	live（正常）	第三个子块的数据
1	0	500	0	live（正常）	新的 ObjectHeader，文件大小为 3

需要理解的关键点是 YAFFS2 无法直接在最初的 ObjectHeader 中更新文件大小、子块的数据等信息。取而代之的是，它必须写入一个新的 ObjectHeader，内含所更新的文件的元数据。在 YAFFS2 中，新的 ObjectHeader 会获得一个更大的序号，成为当前的 ObjectHeader，YAFFS2 则就直接放弃前一个 ObjectHeader（虽然它依然保留在存储器上）。

下一步，文件被打开及读 / 写，第一子块内的数据被改变。随后，此文件被更新后关闭，结果是更多的数据被写入到 NAND 闪存中，如表 4.9 所示。

同样，由于最初的第一个子块的数据无法直接被改变，一个新的数据子块会被写入到 NAND 闪存中，使原先的数据子块变为过时。这也造成一个新的 ObjectHeader 被写入到 NAND 闪存中，并指向文件的第一个子块。下一步，我们清除文件内容，使其变成一个大小为 0 的文件，表 4.10 展示由此造成的 NAND 闪存上的改变。

更多资源请访问稀酷客(www.ckook.com)

表 4.9 存储新的数据及 ObjectHeader

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
0	0	500	0	shrink/delete（压缩 / 删除）	过时的 ObjectHeader，最初为空文件建立，文件大小为 0
0	1	500	1	shrink/delete（压缩 / 删除）	过时的第一个子块的数据
0	2	500	2	live（正常）	第二个子块的数据
0	3	500	3	live（正常）	第三个子块的数据
1	0	500	0	shrink/delete（压缩 / 删除）	过时的 ObjectHeader，文件大小为 3
1	1	500	1	live（正常）	新的第一个子块的数据
1	2	500	2	live（正常）	新的 ObjectHeader，文件大小为 3

表 4.10 清除文件与写入新的 New ObjectHeader

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
0	0	500	0	shrink/delete（压缩 / 删除）	过时的 ObjectHeader，最初为空文件建立，文件大小为 0
0	1	500	1	shrink/delete（压缩 / 删除）	过时的第一个子块的数据
0	2	500	2	shrink/delete（压缩 / 删除）	第二个子块的数据
0	3	500	3	shrink/delete（压缩 / 删除）	第三个子块的数据
1	0	500	0	shrink/delete（压缩 / 删除）	过时的 ObjectHeader，文件大小为 3
1	1	500	1	shrink/delete（压缩 / 删除）	新的第一个子块的数据
1	2	500	2	shrink/delete（压缩 / 删除）	新的 ObjectHeader，文件大小为 3
1	3	500	3	live（正常）	新的 ObjectHeader，文件大小为 0

由于整个文件的内容已经被清除，第 0 块中的所有子块不再继续被使用。在下一次写入时，此块可以作为垃圾被收集。由于垃圾收集依赖于在下一次进行写入操作时完成，这种方式被称为懒惰的垃圾收集方式。与从前一样，一个新的 ObjectHeader 被写入到 NAND 闪存上，以此来反映整个文件内容已经被清除。

最后，我们重新命名此文件，NAND 闪存上的结果如表 4.11 所示。

表 4.11 文件重命名与写入新的 ObjectHeader

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
0	0				Erased（擦除）
0	1				Erased（擦除）
0	2				Erased（擦除）
0	3				Erased（擦除）
1	0	500	0	shrink/delete（压缩 / 删除）	过时的 ObjectHeader，文件大小为 3
1	1	500	1	shrink/delete（压缩 / 删除）	新的第一个子块的数据
1	2	500	2	shrink/delete（压缩 / 删除）	新的 ObjectHeader，文件大小为 3

更多资源请访问稀酷客(www.ckook.com)

（续）

块	子块	ObjectID	ChunkID	状态（正常或压缩 / 删除）	说明
1	2	500	2	shrink/delete（压缩 / 删除）	新的 ObjectHeader，文件大小为 0
2	0	500	0	live（正常）	新的 ObjectHeader，文件大小为 0

在此周期中，第 0 块被作为垃圾完成了收集，此后它就可以重新用于写入新的数据。由于文件被重新命名，一个新的 ObjectHeader 被写入到 NAND 闪存中。由于第 1 块中所有的子块都已经过时，它们也都可以作为垃圾被收集。但是有一点必须切记，由于 NAND 闪存有限的持久性，写入 / 擦除周期被免除了。

对取证隐含的意义是，除非作为垃圾被收集，所有 ObjectHeader 及数据子块的历史数据都将存留在 NAND 闪存中。使用恰当的软件，通过对 NAND 闪存进行扫描可以重新构建文件系统的整个历史，这不但包括所有的时间信息，而且还有可能恢复文件的实际状态。在实际应用中，YAFFS2 分区的状态当然不会那么简单，但是基本原理依然适用。

4.5　挂载的文件系统

我们已经详细讨论了 Android 系统中关于内存的很多部分。我们不但探讨了随机访问内存（RAM）和 NAND 闪存，而且详细讨论了很多文件系统。这些背景信息会极大地帮助对 Android 终端设备进行取证和安全分析。为了对这些偏学术上的信息有更好的理解，我们下面讨论 Android 终端设备上挂载的文件系统。

挂载的文件系统

我们从 HTC Incredible 上挂载的文件系统开始：

```
ahoog@ubuntu:~$ adb shell mount
rootfs / rootfs ro,relatime 0 0
tmpfs /dev tmpfs rw,relatime,mode=755 0 0
devpts /dev/pts devpts rw,relatime,mode=600 0 0
proc /proc proc rw,relatime 0 0
sysfs /sys sysfs rw,relatime 0 0
none /acct cgroup rw,relatime,cpuacct 0 0
tmpfs /mnt/asec tmpfs rw,relatime,mode=755,gid=1000 0 0
none /dev/cpuctl cgroup rw,relatime,cpu 0 0
/dev/block/mtdblock3 /system yaffs2 rw,relatime 0 0
/dev/block/mmcblk0p1 /data ext3 rw,nosuid,noatime,nodiratime,
errors=continue,data=writeback 0 0
/dev/block/mtdblock6 /data/data yaffs2 rw,nosuid,nodev,relatime 0 0
/dev/block/mmcblk0p2 /cache ext3
rw,nosuid,nodev,noatime,nodiratime,errors=continue,data=writeback 0 0
tmpfs /app-cache tmpfs rw,relatime,size=8192k 0 0
/dev/block/vold/179:9 /mnt/sdcard vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=0702,
allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
```

更多资源请访问稀酷客(www.ckook.com)

```
errors=remount-ro 0 0
/dev/block/vold/179:9 /mnt/secure/asec vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=0702,
allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
tmpfs /mnt/sdcard/.android_secure tmpfs ro,relatime,size=0k,mode=000 0 0
/dev/block/vold/179:3 /mnt/emmc vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=0702,
allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
```

当不使用任何参数运行 mount 命令（挂载命令）时，结果将显示已经挂载的文件系统列表以及它们所具有的选项。表 4.12 描述的是输出结果中的几个条目。然而，需要注明的是，在表中为了节省空间，结束每一个条目的“0 0”均被略去。此“0 0”字符串表达的是此文件系统是否通过 dump 命令完成以及在启动时，文件系统检查程序（fsck）对终端设备 / 分区进行错误检查中此文件系统的优先排序号。在大多数台式机和 Linux 服务器上，root 文件系统的优先排序号为 1，使它排列在其他文件系统之前先接受错误检查。

表 4.12 Mount 命令输出总览

设备名称	挂载点	文件系统类型	选项	说明
rootfs	/	rootfs	ro,relatime	这是一个挂载在“/”、只允许读的 root 文件系统
tmpfs	/dev	tmpfs	rw, relatime, mode=755	将设备目录 /dev 作为 tmpfs 文件类型挂载，访问权限设为 755，即 root 用户具有读、写和运行权限（rwx），其他任何用户只拥有读和运行权限（rx）
/dev/block/mtdblock6	/data/data	yaffs2	rw, nosuid, nodev,relatime	虽然 /data 目录的类型为 ext3，用于存储应用数据的 /data/data 目录设为一个 YAFFS2 类型的文件系统，挂载后允许对其进行读和写，不允许执行 setuid 命令（此命令允许用户在执行程序时拥有程序文件拥有人的权限），不允许将任何文件按照特殊块设备进行使用，并且会在访问时间早于修改时间的情况下对访问时间进行更新
/dev/block/vold/179:9	/mnt/sdcard	vfat	See SD card numbered list	请查看 SD 卡带编号的列表

/mnt/sdcard 拥有很多选项，包括如下：

1）rw：挂载后允许读 / 写；

2）dirsync：所有对目录的更新都同步完成；

3）nosuid：不允许 setuid（将会允许其他用户使用文件拥有者的访问权限运行程序）；

4）nodev：不要将任何文件解析为特殊的块终端设备；

5）noexec：不允许从文件系统内执行所有文件；

6）relatime：更新访问时间如果修改时间更新；

7）uid=1000：将所有文件的拥有者设置为 1000；

8）gid=1015：将所有文件的组设置为 1015；

更多资源请访问稀酷客(www.ckook.com)

9）fmask=0702：设置 umask 只适用于文件（设置访问权限 - - -rwxr-x，即 user=none, group=read/write/execute，other=read/execute）；

10）dmask=0702：设置 umask 只适用于目录（文件夹）（设置访问权限 - - -rwxr-x，即 user=none，group=read/write/execute，other=read/execute）；

11）allow_utime=0020：控制 mtime/atime 的访问权限检查；

12）codepage=cp437：设置代码页，用于在 FAT 和 VFAT 文件系统中进行短名字符转换；

13）iocharset=iso8859-1：在 8 位字符和 16 位 Unicode 字符之间进行转换时使用的字符集。默认字符集为 iso8859-1。磁盘上的长文件名使用 Unicode 格式；

14）shortname=mixed：定义如何创建和显示满足 8.3 格式文件名的行为。长文件名是首选的显示方式。短文件名按照原名混合显示，而当短文件名不全部是大写字母时，保存长文件名；

15）utf8：将 CD 上的 16 位 Unicode 字符转换为 UTF-8；

16）errors=remount-ro：定义出错时的行为；在此设置中，重新挂载文件系统，并设置为只允许读。

所有 mount 命令的选项都在手册中加以解释（man 8 mount）。然而，在大多数情况下，快速浏览一下就可以让检查者获得需要的信息，如挂载点、类型以及文件系统的访问权限。

使用 df 命令可以获得已经挂载的文件系统中的可用空间信息：

```
ahoog@ubuntu:~$ adb shell df
/dev: 211600K total, 0K used, 211600K available (block size 4096)
/mnt/asec: 211600K total, 0K used, 211600K available (block size 4096)
/system: 253952K total, 243724K used, 10228K available (block size 4096)
/data: 765992K total, 129840K used, 636152K available (block size 4096)
/data/data: 152576K total, 52048K used, 100528K available (block size 4096)
/cache: 198337K total, 10790K used, 187547K available (block size 1024)
/app-cache: 8192K total, 7140K used, 1052K available (block size 4096)
/mnt/sdcard: 1955520K total, 245664K used, 1709856K available (block size 32768)
/mnt/secure/asec: Permission denied
/mnt/emmc: 6920512K total, 233152K used, 6687360K available (block size 32768)
```

可以看到，当 shell 用户使用 df 命令查看可用的磁盘空间时，其中一个挂载点（/mnt/secure/asec）返回的却是拒绝访问。查看一下它的父目录（即所属的文件夹），可以看到只有 root 才有对此目录进行访问的权限。

```
ahoog@ubuntu:~$ adb shell ls -l /mnt
d---rwxr-x system   sdcard_rw          1969-12-31 18:00 emmc
drwxr-xr-x root     system            2011-01-31 15:42 asec
drwx------ root     root              2011-01-31 15:42 secure
d---rwxr-x system   sdcard_rw          2011-02-01 17:49 sdcard
```

有趣的是，其中两个文件系统 / 目录可以通过使用 USB 大容量存储（UMS）选项暴露出来，当此情况发生时，相应目录的访问权限会改变。如果不允许使用 UMS 选项，Android 终端设备完全可以访问这些文件系统，正如 /mnt/emmc 和 /mnt/sdcard 所显示的那样。

更多资源请访问稀酷客(www.ckook.com)

但是，当允许 UMS 选项时，连接的工作站就可以使用这两个文件系统，访问权限改变为：

```
ahoog@ubuntu:~$ adb shell ls -l /mnt
d--------- system   system            2011-01-23 10:08 emmc
drwxr-xr-x root     system            2011-01-23 10:08 asec
drwx------ root     root              2011-01-23 10:08 secure
d--------- system   system            2011-01-23 10:08 sdcard
```

如上所述，去除 /mnt/emmc 和 /mnt/sdcard 的所有访问权限，则 Android 终端设备就无法直接从手机上再继续访问 /mnt/emmc 和 /mnt/sdcard（也就是说，此种共享访问是与所连接的工作站互斥的）。

4.6 本章小结

Android 终端设备中的物理内存、文件系统以及相应的数据结构是数据存储的基本构成要素。对于这些结构的深入理解不但能够加深对 Android 终端设备的理解，而且有助于对将来出现的新的文件系统和数据结构进行研究和开发。

4.7 参考资料

Android developers blog: Saving data safely. (n.d.). Retrieved February 2, 2011, from http://android-developers.blogspot.com/2010/12/saving-data-safely.html.

Android reverse engineering. (n.d.). thomascannon.net. Retrieved January 29, 2011, from http://thomascannon.net/projects/android-reversing/.

Data storage. (n.d.). Android Developers. Retrieved March 13, 2011, from http://developer.android.com/guide/topics/data/data-storage.html#netw.

How YAFFS works: the internals. (n.d.). YAFFS. Retrieved February 4, 2011, from http://www.yaffs.net/how-yaffs-works-internals.

Nand Faq. (n.d.). *Memory technology device (MTD) subsystem for Linux*, Retrieved March 13, 2011, from http://www.linux-mtd.infradead.org/faq/nand.html.

The sysfs filesystem. (n.d.), Retrieved February 1, 2011, from http://www.kernel.org/pub/linux/kernel/people/mochel/doc/papers/ols-2005/mochel.pdf.

Way, T. (n.d.). *Android will be using ext4 starting with Gingerbread. Thoughts by Ted. Welcome to thunk.org*, Retrieved February 2, 2011, from http://thunk.org/tytso/blog/2010/12/12/android-will-be-using-ext4-starting-with-gingerbread/.

Licence FAQs, Y. A. F. F. S. (n.d.). *YAFFS*, Retrieved February 4, 2011, from http://www.yaffs.net/yaffs-licence-faqs.

YAFFS: the NAND-specific flash file system—Introductory Article. (n.d.). Retrieved February 2, 2011, from http://www.yaffs.net/yaffs-nand-specific-flash-file-system-introductory-article.

更多资源请访问稀酷客(www.ckook.com)

第5章

Android终端设备、数据与应用安全

本章要点

- 数据窃取目标和攻击向量
- 安全考虑
- 个人安全策略
- 企业安全策略
- 应用开发安全策略

5.1 绪论

取证分析师和隐私支持者之间存在着一种很微妙的平衡。一方面，如果一个终端设备被完整安全保护起来，则取证调查不会得到任何信息。另一方面，在没有任何安全措施的情况下，从终端设备中获取信息基本上不需要具备任何取证方面的技能。

移动取证的主要用户为执法者和政府机构。他们通过移动终端设备使用大量数据，并对这些信息进行安全保护，同时他们具有调查犯罪案件的责任和权利。他们不但依赖数字取证分析技术，而且通过搜查令、传票等行使他们的权利，并能够要求大多数机构与他们进行合作，按照要求提交需要的信息，如财务记录、电子邮件、互联网服务提供商运行日志等。

同样，企业需要对内部敏感数据进行保护，并不时地进行内部调查以确保安全。虽然企业在这方面的权利通常局限在企业内部，然而在美国，企业可以拥有更加广泛的权利使他们能够对所拥有的终端设备进行检查。

最后，个人拥有访问他们所拥有的数据的权利。不管他们行使此权利的目的是支持民事案件，或者基于其他目的，他们都拥有对所拥有的终端设备做这些事情的权利。

对于个人和公司而言，当事人通常没有必要获取他们有权利进行调查的终端设备中信用卡、银行信息或者密码方面的信息。在内部调查中，公司不会寻求获得个人的信用卡数据，但是公司具有访问公司邮件和修改密码的能力。对于个人而言，他们已经具备了访问个人的财务记录、电子邮件以及其他敏感数据的能力。对于执法者和政府机构而言，他们可以通过使用传票和搜查令等手段来获取他们寻求的数据。

因此，若对存储在移动终端设备上的敏感数据没有进行有效安全保护，最后受益的就只剩下网络罪犯。在对个人、公司以及犯罪案件所涉及的终端设备进行调查的过程中，我们发现了大量与案件无关紧要的个人敏感信息。但是，如果网络罪犯能够访问这些终端设

更多资源请访问稀酷客(www.ckook.com)

备，不论是通过实际获得这些终端设备，还是通过网络远程获取这些终端设备中的数据，对于消费者来说都代表着巨大的风险。

同样，企业也已经成为商业间谍、金融盗窃、知识产权窃取以及其他多种攻击的目标。很多公司向员工持有终端设备的方向发展，甚至失掉了对这些终端设备的控制和有效管理，这给公司的数据带来巨大的风险。

最后，移动安全问题对执法机构和政府机构造成负面的影响。作为普通的用户，这些机构中的个人要分担相同的数据泄漏风险。与公司一样，这些机构也是被攻击的对象，攻击的目标不仅是能危害到正在进行的调查或者是损害机构形象的敏感数据，而且可能是以国际间谍活动为目标的攻击。对执法机构和政府机构独特的挑战是太多涉及终端设备的犯罪需要调查，使得本就超负荷工作的刑事案件调查部门和数字取证实验室更加紧张。

基于上述原因，移动终端设备的安全成为个人、企业、执法机构和政府机构日益关心的问题。

5.2 数据窃取目标和攻击向量

大家都知道，在智能手机里，特别是 Android 终端设备里，存储着混合了个人和公司数据的大量信息。Android 终端设备可能成为数据窃取的目标，也可能成为数据窃取的工具（即攻击向量）。了解各类威胁以及攻击场景能够使安全工程师和开发者通过设计合适的控制来降低所面临的风险。虽然本章不能确保完整介绍这样一个涉及面非常广的题目，但是将讨论一些具体的安全威胁以及降低风险的策略，安全专业人士可以将本章介绍内容作为一个切入点。

5.2.1 以 Android 终端设备作为目标

移动安全的研究、应用和文献的重点主要集中在将智能手机作为数据窃取的目标，这样做完全正确。对消费者、企业和政府机构所构成的风险真实存在，因此，安全专家们一致认为，面向移动终端设备的恶意软件的数量在持续增加，也将是网络犯罪关注的重点。

移动终端设备中包含大量的个人和公司数据，并且以非常集中、容易移植的方式存在。犯罪者通常都很现实，网络犯罪者也是一样。如果网络犯罪者从终端设备上不但能够获取个人用户的用户名、密码及敏感数据，而且能够获取关于这些个人用户的雇主的以上信息，他们肯定不会放过这个机会。同时，虽然从一个移动应用中获取的数据也许不足以让犯罪者达到特定的目的，但是将从多个移动应用中获取的信息进行组合通常能够勾画出终端设备拥有者的全貌。

移动终端设备不但很容易丢失，也很容易被盗取，而且通常使用寿命也比较短，因为消费者也许很快就希望更换使用最新款式的终端设备。以上的现实情况造成了大量的二手智能手机可以通过诸如 eBay、Craigslist 这样的渠道购买，同时很多购买到的终端设备内的个人数据没有完全擦除干净，并且继续存留。另外，越来越多的恶意代码、恶意软件和远程攻击也以移动终端设备作为攻击目标，历史经验表明，这些攻击也会造成大量的数据泄露。这些攻击的有效性需要通过时间来衡量，但是，操作系统开发的紧迫步伐（在这种

更多资源请访问稀酷客(www.ckook.com)

情况下就是 Android）和移动应用的快速普及使我们有充分的理由相信，很多面向终端设备的攻击都将会成功并且有效。

1. 静态数据

静态数据（data at rest）是指那些存储在非易失性存储设备中的数据，因此，这些数据不包含在随机访问内存（RAM）中的数据，也不包含正在网络中传输的数据（蜂窝网、数据网及其他任何形式的网络）。静态数据这一术语通常用于法律法规中，它定义了数据安全保护的一个核心状态。

在本书的讨论中，我们会突出有关 Android 终端设备中数据的不同实例。在这里，我们首先考虑以下两个假想的场景：一个侧重于企业用户，另一个则侧重于个人用户。

对于个人用户而言，Android 终端设备主要用于个人通信、个人财务、娱乐以及一般性的互联网信息浏览。通过检查这些手机内的数据，我们通常可以发现存在以下信息：

- 短信 / 彩信：所有未删除的短信和彩信都可以获得，同时很多已经删除的短信和彩信也可以恢复（虽然已经删除，但是仍然存留在 NAND 闪存中）。获取或恢复的不但是这些信息本身，而且还包括这些信息所附带的图像、视频和音频文件以及联系人（.vcf 文件）、日历中的日程安排信息等。很多第三方开发的短信 / 彩信应用软件中还可能包含其他功能。
- 通话日志（call log）：这一类信息也可以从移动服务提供商那里获得，并且可以恢复完整的通话日志。恢复的日志中包括失败的呼叫，即对于服务提供商来说没有成功连接的通话，以及其他可以进行相关联的元数据，如地点（也可以基于进行通话连接的蜂窝基站进行估测）以及其他在通话前发生的事件。
- 语音信箱（voice mail）：如果设备具备留言服务，如 Verizon 公司的可视化留言服务，这些留言作为语音文件（.OGG）通常都是可以恢复的。有些已删除的留言也可能恢复。
- 财务应用：虽然没有固定的模式，但是大多数财务应用软件都会在终端设备上存储一些数据，也会缓存一些访问过的网页。在某些情况下，恢复的数据可能包括用户登录信息、密码、账号以及交易的详细信息。例如，在我们的测试中，Mint.com 应用软件保存了用于访问本地应用软件的用户名和个人识别号（PIN）以及某些涉及银行账号的交易信息。
- 个人邮件：大多数安装使用的电子邮件应用软件将个人邮件以明文的方式进行保存，包括邮件头（发送者和接收者的电子邮件地址）。在某些情况下，用于证明用户的信息也可以进行恢复，如登录信息和密码。例如，在通过 POP3 连接 Hotmail 时，我们发现 Android 邮件应用软件会将密码以明文的方式进行保存。
- Web 浏览记录：在分配的区域中存在的 Web 浏览记录包括访问过的 URL、记录的用户信息以及被缓存的网页都是可以获取的。在被回收（或被删除）的区域中也可能存在更多的 Web 浏览信息。
- 谷歌搜索记录：谷歌的 URL 中所包含的搜索关键词。
- YouTube：所观赏过的视频的 URL。

更多资源请访问稀酷客(www.ckook.com)

- 图像和视频：用户所拍摄的保存在终端设备中的图像和视频，也许与具体的应用相关。
- 地理位置：图像中的全球定位系统（GPS）坐标信息以及其他的标识信息。
- 游戏记录及交互信息。

在以上的假想场景中，经验丰富的取证检查师可以从移动终端设备中获取或恢复大量的个人信息。

以下针对企业的举例将建立在上述针对个人的基础之上，这是因为大多数终端设备中都融合了个人和公司的数据。以下是能够从企业拥有的终端设备中获得或恢复的其他数据：

- 公司的电子邮件及附件；
- 通过电子邮件发送的留言和传真；
- 用户名、密码及所在区域信息；
- Wi-Fi 接入点的相关信息及密码；
- 日历中的日程安排；
- 与员工的即时通信及其他方式的通信；
- 为了使用方便而存储在终端设备上的公司文件。

可以看到，当攻击者寻找渗入公司的有效方法时，一个公司员工所使用的终端设备（多个员工所使用的终端设备则更好）能够给攻击者提供很多公司的内幕及获取途径，更不用说攻击者还可以从终端设备中直接获得属于公司的敏感信息。

Android 中自带的电子邮件应用在用户使用微软公司的 exchange ActiveSync（EAS）协议时保存电子邮件账号中核心信息的方式，这是一个足以让企业安全经理引起警觉的实际例子。这些用于向 EAS 进行认证的核心信息包括用户当前活动目录域中的用户名和密码。很多企业已经将认证、授权和审计（authentication，authorization，and accounting，AAA）三大服务整合在同一个活动目录域中进行集中管理，以实现单点登录，并易于管理。总体来说，这样的整合有益于实现更有效的安全性。在 Android 系统的电子邮件应用中（com.android.mail），用户的 EAS 以明文的方式存储在一个明确定义的地方。整个数据库存储在 /data/data/com.android.email。以下是相关文件夹以及文件的概况：

```
ahoog@ubuntu:~$ tree com.android.email/
com.android.email/
├── cache
│   └── webviewCache
├── databases
│   ├── 1.db_att
│   │   ├── 1
│   │   ├── 2
│   │   └── 3
│   ├── EmailProviderBody.db
│   ├── EmailProvider.db
│   ├── webviewCache.db
│   └── webview.db
├── files
│   └── deviceName
├── lib
└── shared_prefs
    └── AndroidMail.Main.xml
```

更多资源请访问稀酷客(www.ckook.com)

密码保存在 EmailProvider.db 数据库的 HostAuth 表中的一列，可以方便地对密码命名。

```
ahoog@ubuntu:~$ sqlite3 com.android.email/databases/EmailProvider.db
SQLite version 3.6.22
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> .mode line
sqlite> select * from HostAuth;
       _id = 1
  protocol = eas
   address = owa.CorpExchangeServerExample.com
      port = 0
     flags = 5
     login = thisIsTheirUserNameInPlainText
  password = thisIsTheirPasswordInPlainText-Seriously
    domain = NeverHurtsToHaveTheDomainInfoToo
accountKey = 0
```

除了账号中的用户名、密码和活动域，电子邮件的主题和全部邮件内容都保存在数据目录中，该目录中还包括邮件附件、各种设置及其他信息。所有的数据都是以明文的方式保存（令人想起银制磁盘机）。

攻击者主要使用两种方式对静态数据进行访问。第一种方式是实际得到终端设备并对其中的数据进行访问，使用与取证检查师基本一样的各种技术，这些技术将在第 6 章中进行探讨。然而，实际得到终端设备不是一件很容易做到的事情，我们在本章开始就特别提出了其他的可能情景，包括丢失或被偷窃的手机以及没有安全清理干净的被淘汰的旧手机。此外，在国际旅行中，主要是公司的高层管理人员，手机在入关的边境检查中可能会短暂地被海关人员检查。在此场景中，海关人员可以实际获得终端设备，并访问数据。

攻击者使用的另外一种对静态数据进行访问的方式（技术）是远程控制及恶意软件。在此场景中，攻击者通过编程技术或社会工程技术获得更多的访问权限。这些技术包括利用 Linux 内核及 Android 共享程序库中存在的漏洞、网络钓鱼攻击以及利用应用软件中存在的漏洞。此外，用户可能会下载安装应用软件，并授予这些应用软件不必要的访问权限，使恶意软件能够对静态数据进行访问。最后，社会工程技术一直是对系统实施攻击的一非常有效的方法，移动终端设备对此种技术没有任何免疫力。用户更习惯于下载安装各类应用软件，同时对应用软件提供者了解甚微，通常又非常信任并自由地点击通过电子邮件和短信收到的链接，这些都给基于社会工程技术实施的攻击提供了多种渠道。

2. 传输中的数据

传输中的数据（有时又叫做“运动中的数据”）这一术语是指在网络（蜂窝网、Wi-Fi 或其他网络）中传输的数据，也指存在于随机访问内存（RAM）中的数据。这一术语通常用于法律法规，它定义了数据安全保护的另外一个关键状态。

一般来说，大多数存储在终端设备中的信息（以上介绍过）在某些时间点上都会通过网络进行传输。除数据以外，其他很多不属于终端设备本身的信息也会在网络中进行传输，并且必须得到保护。以下是部分此类信息：

更多资源请访问稀酷客(www.ckook.com)

- 密码：很多应用软件不将密码保存在终端设备中，而要求用户在每次运行应用软件时都与服务器端进行认证。因此，密码只被传输，不被保存。
- 双方式认证。
- 密码重置的安全回复。
- 在应用软件中显示但是却没有被保存或缓存在非易失存储器中的数据（如账号、余额等）。

接下来通过实例演示数据如何在网络中进行传输。在此实例中，一个运行 BackTrack 4（一个基于 Linux 的渗透测试套间）的计算机连接到一个网络集线器，此网络集线器同时提供一个 Wi-Fi 接入点。计算机上的网络接口被设置成在监听模式下运行，使此计算机能够对所有通过网络集线器的网络数据包进行监控，即使这些网络数据包不是传输给此计算机。计算机上运行的 urlsnarf 程序对网络接口（eth0）的数据包进行侦听，并对数据包进行分析，寻找数据包中存在的 URL。对于找到的 URL，将其在计算机屏幕上进行显示。需要记住的是，所有的数据包都会被侦听，因此，所有数据包中未进行加密保护的数据，如用户名和密码，都能够被获取并进行显示。

```
root@bt:~# sudo urlsnarf
urlsnarf: listening on eth0 [tcp port 80 or port 8080 or port 3128]
10.1.10.11 - - [17/Mar/2011:09:26:19 -0500] "GET
http://api.twitter.com/1/statuses/mentions.json?include_entities=true&count=100&
since_id=45865608952295424 HTTP/1.1" - - "-" "TwitterAndroid/2.0.1 (122)
ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:26:19 -0500] "GET
http://api.twitter.com/1/statuses/retweets_of_me.json?include_entities=true&
count=100&since_id=15855940804812800 HTTP/1.1" - - "-" "TwitterAndroid/2.0.1
(122) ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:26:20 -0500] "GET
http://api.twitter.com/1/statuses/home_timeline.json?include_entities=true&
count=100&since_id=48383245812895746 HTTP/1.1" - - "-" "TwitterAndroid/2.0.1
(122) ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:26:20 -0500] "GET
http://api.twitter.com/1/account/rate_limit_status.json HTTP/1.1" - - "-"
"TwitterAndroid/2.0.1 (122) ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:26:20 -0500] "GET
http://api.twitter.com/1/direct_messages.json HTTP/1.1" - - "-"
"TwitterAndroid/2.0.1 (122) ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:26:20 -0500] "GET
http://api.twitter.com/1/account/rate_limit_status.json HTTP/1.1" - - "-"
"TwitterAndroid/2.0.1 (122) ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:26:21 -0500] "GET
http://api.twitter.com/1/direct_messages/sent.json HTTP/1.1" - - "-"
"TwitterAndroid/2.0.1 (122) ADR6300/8 (HTC;inc;verizon_wwe;inc;)"
10.1.10.11 - - [17/Mar/2011:09:27:35 -0500] "GET http://goo.gl/4G7Bx HTTP/1.1"
- - "-" "Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300 Build/FRF91)
AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:51 -0500] "GET http://mrkl.it/ HTTP/1.1"
- - "-" "Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300 Build/FRF91)
AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:51 -0500] "GET http://mrkl.it/
-/media/static/main.css HTTP/1.1" - - "http://mrkl.it/" "Mozilla/5.0 (Linux; U;
Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko)
Version/4.0 Mobile Safari/533.1"
```

更多资源请访问稀酷客(www.ckook.com)

```
10.1.10.11 - - [17/Mar/2011:09:28:51 -0500] "GET http://mrkl.it/
-/media/static/fancybox/jquery.fancybox-1.3.4.pack.js HTTP/1.1"
- - "http://mrkl.it/" "Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300
Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile
Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:51 -0500] "GET
http://ajax.googleapis.com/ajax/libs/jquery/1.5.1/jquery.min.js HTTP/1.1"
- - "http://mrkl.it/" "Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300
Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile
Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:51 -0500] "GET http://mrkl.it/
-/media/static/fancybox/jquery.fancybox-1.3.4.css HTTP/1.1"
- - "http://mrkl.it/" "Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300
Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile
Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:52 -0500] "GET http://mrkl.it/
-/media/static/misc/screenshot-demo.png HTTP/1.1" - - "http://mrkl.it/"
"Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300 Build/FRF91)
AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:52 -0500] "GET http://mrkl.it/
-/media/static/main.js HTTP/1.1" - - "http://mrkl.it/" "Mozilla/5.0 (Linux; U;
Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko)
Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:52 -0500] "GET http://mrkl.it/
-/media/static/sign-in-with-twitter-l.png HTTP/1.1" - - "http://mrkl.it/"
"Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300 Build/FRF91)
AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:53 -0500] "GET
http://www.youtube.com/embed/ptq21VOfgfs?rel=0&hd=1 HTTP/1.1"
- - "http://mrkl.it/" "Mozilla/5.0 (Linux; U; Android 2.2; en-us; ADR6300
Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko) Version/4.0 Mobile
Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:53 -0500] "GET http://www.
google-analytics.com/__utm.gif?utmwv=4.8.9&utmn=722592738&utmhn=mrkl.it&
utmcs=UTF-8&utmsr=800x1183&utmsc=32
-bit&utmul=en&utmje=0&utmfl=10.1%20r92&utmdt=Welcome&utmhid=414400362&utmr=-&
utmp=%2F&utmac=UA-21535947
-2&utmcc=__utma%3D15198314.1123929794.1300372147.1300372147.1300372147.1%3B%2B
__utmz%3D15198314.1300372147.1.1.utmcsr%3D(direct)%7Cutmccn%3D(direct)%7Cutmcmd%
3D(none)%3B&utmu=q HTTP/1.1" - - "http://mrkl.it/" "Mozilla/5.0 (Linux; U;
Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko)
Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:54 -0500] "GET http://mrkl.it/
-/media/static/favicon.ico HTTP/1.1" - - "http://mrkl.it/" "Mozilla/5.0 (Linux;
U; Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like
Gecko) Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:54 -0500] "GET
http://s.ytimg.com/yt/swfbin/watch_as3-vflFkxRDW.swf HTTP/1.1"
- - "http://www.youtube.com/embed/ptq21VOfgfs?rel=0&hd=1" "Mozilla/5.0 (Linux;U;
Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like Gecko)
Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:57 -0500] "GET
http://i1.ytimg.com/crossdomain.xml HTTP/1.1"
- - "http://s.ytimg.com/yt/swfbin/watch_as3-vflFkxRDW.swf" "Mozilla/5.0 (Linux;
U; Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like
Gecko) Version/4.0 Mobile Safari/533.1"
10.1.10.11 - - [17/Mar/2011:09:28:57 -0500] "GET
http://i1.ytimg.com/vi/ptq21VOfgfs/hqdefault.jpg HTTP/1.1"
- - "http://s.ytimg.com/yt/swfbin/watch_as3-vflFkxRDW.swf" "Mozilla/5.0 (Linux;
U; Android 2.2; en-us; ADR6300 Build/FRF91) AppleWebKit/533.1 (KHTML, like
Gecko) Version/4.0 Mobile Safari/533.1"
```

更多资源请访问稀酷客(www.ckook.com)

攻击者不但可以使用一些广为人知的技术对传输中的数据进行攻击，还可以使用一些其他的由安全研究者提出的新技术或者由实践者根据实际经验发现的技术。以下是一些广为人知的技术：

- 中间人（Man-in-the-middle，MITM）攻击。
- 中间人安全套接层（MITM Secure Sockets Layer）攻击。
- 域名服务欺骗攻击（包括 /etc/hosts）。
- TMSI 溢出基带攻击。

基带攻击是一个面向无线调制解调器（基带）固件的非常新的攻击技术。根据 Ralf-Philipp Weinmann 在 DeepSec 2010 国际会议上所做的如何发起此类攻击的介绍，基带固件的代码是在 20 世纪 90 年代编写的。直到近期，人们还没有很好地理解 GSM 网络所基于的技术。然而，随着时间的推移，研究者们逐渐将 GSM 相关网络协议和硬件的细节公诸于世，现在，已经有可能使用非常容易得到的硬件和开源软件构造一个伪装的 GSM 基站。通过对 GSM 网络实施控制，攻击者可以发起 TMSI 溢出攻击，这可以造成使用苹果 iOS 4.2 之前版本的终端设备在 GSM 基带栈中产生堆溢出。此攻击导致在基带处理器上使用远程方式执行代码（Ralf-Philipp Weinmann, n.d.）。

以上介绍的基带攻击代表了最新的安全研究成果，而中间人攻击已经存在很长时间并广为人知，同时也非常容易实施。通常，这一类攻击技术要求攻击者完全控制一个置于移动终端设备和需要连接并进行通信的目标结点之间的计算机设备。此外，攻击者也许还控制着关键的网络服务或设备，如网络交换机、Wi-Fi 接入点或 DNS，以帮助攻击者实施网络攻击。

一旦将所控制的计算机设备置于 Android 终端设备和最终通信目标之间，攻击者就可以发起攻击。在此例中，我们假设一个 Android 终端设备已经连接到一个 Wi-Fi 网络，但是没有意识到此网络已经被一个恶意攻击者所控制。Android 终端设备用户开始浏览互联网，直至决定使用 Twitter 账号。为了保持以下说明的简易性，我们假设 Twitter 应用软件没有使用安全套接层（SSL）协议，其实在实际应用中，大多数应用软件都没有使用 SSL，虽然近期正式发布的应用软件已经在向使用 SSL 的方向转变。

当用户启动 Twitter 应用软件时，终端设备将被接连到 Twitter 的官方网站，在通过认证后，用户就可以使用他们的账号。当然，在以上过程中，攻击者获取所有的数据包，并且获得用户的 Twitter 用户名和密码。即使此应用软件使用 SSL，但是没有正确地对 SSL 证书进行验证，仍然会受到中间人 SSL 攻击的威胁，如图 5.1 中所展示。

虽然此攻击只能获取一个密码，但是大多数的用户在很多网站上重复使用相同用户名和密码。一旦攻击者得到了一个用户名和密码，攻击者就可以在网上搜索更多的关于消费者的信息，通常也可以找到其他可以进行访问的系统。

另外一个非常知名的攻击方式是通过获取 HTTP 对话中的 cookies 在网站上截获另外一个用户的认证过程，并以这个用户自居。Eric Butler 通过针对 Firefox（http://codebutler.com/firesheep）浏览器所制作的 Firesheep 扩展展示出这一类攻击非常容易实

更多资源请访问稀酷客(www.ckook.com)

施。虽然采取对 Wi-Fi 实施 WPA 级别的高强度加密以及其他措施能够提高实施攻击的难度，但是只要网站不要求端到端的 HTTPS，这类攻击仍然可行。

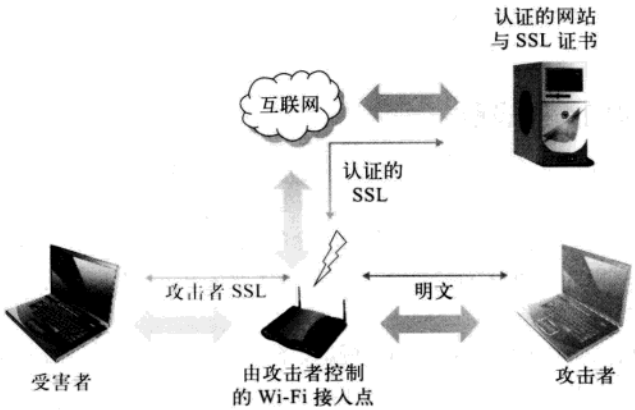


图 5.1　中间人攻击 SSL

5.2.2　以 Android 终端设备作为攻击向量

当媒体和很多安全研究人员主要关注针对移动终端设备实施的攻击时，一个日益受到关注的问题是将 Android 终端设备用作一个攻击向量，特别是在保存敏感数据的环境中。常见的情景是企业的商业机密、知识产权和其他需要得到保护的数据。企业通常会设计非常完善的系统来预防窃取数据，至少能够做到检测。这些系统不但非常昂贵，而且通常要求经验丰富的人员进行维护、监控，并对发生的意外情况及时响应。

直到近期，大多数的数据保护系统主要侧重点还是为企业建立一个安全保护边界，将攻击者阻挡在被保护的区域之外。随着时间的推移，已开发出针对内部威胁的安全保障系统。最新的系统甚至深入到网络和基础设施的内部以抵御各种攻击。这些系统注重的方面包括数据损失预防（DLP）、网络访问控制（NAC）及网络取证。然而，目前可用的解决方案无法完全应对移动终端设备所面临的安全威胁。

显而易见，智能手机非常普遍，大多数用户也是按照需要进行使用。这些终端设备通常由个人拥有，随时使用，因此，要求任何人放弃所使用的终端设备都是一个带有挑衅性的请求。基于以上理由，Android 终端设备及其他智能手机都用于保存敏感信息，却很少有人对此显示出任何疑虑。但是，如果某人购买了一台数字照相机、留言机、摄像机、移动硬盘或者个人网络设备，也许就会吸引他人的眼球。当然，Android 终端设备基本上包含了上述电子产品的所有功能，而且是一个非常轻便且外表美观的终端设备。

5.2.3　数据存储

也许如何使用 Android 终端设备去偷窃信息的一个最简单的实例就是将它看做一个

更多资源请访问稀酷客(www.ckook.com)

USB 大容量存储终端设备。直到近期，智能手机内的数据存储空间都还很小。然而，随着 NAND 闪存技术的逐步成熟，制造商认识到他们可以生产出价格低廉、具有 GB 级别数据存储能力的终端设备。今天的 Android 终端设备可以很容易地实现在 NAND 闪存上存储 8 ～ 16GB 的数据，很多终端设备还可以外接 SD 卡，用于存储额外的 16GB 或更多数据。

5.2.4 用于记录的终端设备

Android 终端设备也配备了对周围任何场景进行录音录像的功能。它们可以用于对敏感仪器和文件进行拍照，也可以用于录像，以录制通过一栋大楼所经过的路径，包括周围的声音、安全检查站、门窗、楼梯以及其他沿途的物体。最后，Android 终端设备也能够用于进行录音。所有以上这些都可以在不引起任何人注意的情况下完成。

绕开网络监控

也许一个更加危险的功能是 Android 终端设备能够为另外一台计算机或其他的终端设备提供无线通信的能力。实现此功能有以下几种方式。场景之一是攻击者（也许是一个不满的公司内部员工）可以将 Android 终端设备连接到一台工作站。随意的观察也许不会由此引起任何警觉，因为很多人都通过这种方式在工作时间内给手机充电。但是，如果在 Android 终端设备和工作站上都安装一个小程序，这样做将产生一个新的与工作站的连接。在此情况下，由工作站通过 Android 终端设备发出去的数据报将完全不受所在环境的监控。此连接可以完全绕开防火墙、网络访问控制、数据损失预防以及其他所有部署在公司环境中的安全控制。

最新的 Android 终端设备已经不再需要在以上实例中所使用的 USB 连接线以及软件包。相反，Android 终端设备成为一个功能完整的无线访问热点，使计算机可以与其实现连接。这种场景不但同样可以有效绕开部署的所有网络安全控制，并且可以将终端设备隐藏在公文包或口袋内。

真正阴险的技术

如果有人认为网络犯罪或其他攻击没有多少智慧或创新，只能说明这些人在判断力上存在很大差距。那么，让我们通过应用以上介绍的技术进行认真反思。

在此场景中，攻击者是一名公司的员工，而公司拥有很多商业机密和知识产权。（攻击者并不一定必须是公司的员工，在这里只是为了使我们的说明更加简单。）很多能够访问这些信息的公司高层管理人员也使用笔记本电脑和智能手机。这些公司高层管理人员中的很多人会在公司外面的咖啡馆里喝咖啡，而咖啡馆通常提供免费的 Wi-Fi 无线上网服务。这些咖啡馆里的无线接入点叫做“Indigo”，而由于不要求使用密码，这些接入点对外开放，也没有任何加密保护。在公司高层管理人员品尝咖啡、就餐或开会期间，他们的智能手机和笔记本电脑与 Wi-Fi 保持着连接。

在他们返回办公室后，他们的智能手机和笔记本电脑依然处于无线工作状态。这名心怀不满的员工当然知道咖啡馆里的无线接入点，并决定开启他个人 Android 终端

更多资源请访问稀酷客(www.ckook.com)

设备上的无线接入点功能。他自然拥有个人终端设备上的 root 访问权限，因此，他可以在上面安装数据包截获软件。当任何公司高层管理人员接近他的 Android 终端设备时，智能手机或笔记本电脑就连接到他的接入点。然后他就可以截获所有的数据包、获取敏感文件、监听敏感通信，或许还可以获取用于访问其他核心网络资源的关键信息。

虽然攻击者需要克服一些困难后才能够实现以上场景，但是这确实非常诱人，也清楚地展示出将 Android 终端设备用作一个攻击向量所带来的风险。

5.3　安全考虑

与开发一样，安全也是一门艺术。也许有人不同意以上的说法，但是这些人基本上不会去阅读取证和安全方面的书籍。安全是一个艺术的过程的含义在于艺术是“人类创新的产物”，也是“通过学习、实践和观察才能够掌握的高超技能”（WordNet Search, n.d.）。那么，所说的这一切有什么关联性呢？成功的安全战略家是经验、判断力、风险评估、创造能力、观察力、技能，以及也许一点运气的综合和平衡。这一节的目标是介绍一些与 Android 和安全相关的一般概念。

5.3.1　安全的哲学原理

安全基本上是一种妥协，是攻击的风险与排除风险的成本（金钱或其他成本）之间的一个权衡。虽然在理论上，完全安全的系统是可以做到的，但是在现实中，它却是基本上不可能做到的，尤其是面对社交手段实施的攻击。为系统提供安全的任务十分困难，即使当安全专业人员能够实际控制对终端设备的访问，包括实际拥有以及远程访问。

移动终端设备比传统的系统更加难以进行安全保护。企业的安全工程师对计算机系统拥有的控制大多数对 Android 终端设备都不可能实现。首先，构成 Android 终端设备的硬件和软件由大量、复杂、范围广的人员进行生产和维护，包括 Android 核心团队、源于制造商和开发商生产的硬件和软件、源于无线通信服务提供商生产的软件和硬件。此外，终端设备的拥有者还可以安装个性化应用软件，如果拥有足够的权限，甚至能够对终端设备进行重大改变。最后，终端设备使用很多网络，而没有任何一个网络可以被完全信任。归根结底，对移动终端设备提供安全保护是一个巨大的挑战。

尽管具有以上这些挑战，一定的安全控制还是可以在终端设备中实现。如果要对总体策略和哲学进行总结的话，以下术语非常贴切：

安全迫使攻击者知难而退。

这个概念来源于 1989 年由环球影视拍摄的电影《梦境》，影片中有一个声音鼓励电影主角说在他的爱尔华农场建造一个棒球场是很简单的一件事情。在影片中，这个声音反复出现：“安全迫使攻击者知难而退”（Ten, n.d.）。最终，电影主角修建了球场，可是事实上，神秘的访客还是出现了。但是，需要去观看这部电影才能了解全部故事情节。

那么，以上的故事与移动安全有何关系？在理想化的加密安全系统和将所有数据以明

更多资源请访问稀酷客(www.ckook.com)

文方式进行保存的系统之间存在着一个巨大的断层，虽然如在先前的 com.android.email 实例中介绍的那样，后者有着组织良好和清楚的结构。由于移动终端设备所面临的威胁来自于四面八方，即使是加密过的数据也有可能被破解。但是，提供一定程度的难度或加密会使攻击者破解数据的过程变得更加复杂。虽然这种方式无法保证 100% 的安全，但是攻击者也是非常现实的，他们倾向于将容易获取的数据作为攻击的目标。

对以上方式的一个反对意见是，可以克服的困难和破解的加密只会给用户造成一个错觉，导致用户在使用终端设备时更加不在意，因为他们认为对数据提供的安全保护是牢不可破的。如果将数据以明文的方式进行存储，而用户又充分了解所面临的安全风险，他们在使用终端设备时会格外小心。

归根结底，个人和企业的移动终端设备消费者通过他们的购买决定会对移动安全生态系统的发展提供一个导向。这当然也取决于消费者的教育程度。消费者必须充分了解数据所面临的安全风险以及行之有效的安全解决方案。

虽然很多潜在的安全解决方案不是建立在提供加密安全系统的基础上，但是却能够通过提高成功实施攻击的复杂程度来改进数据的安全性。以下介绍几个相关的概念。

1）要求用户在系统启动后第一次执行一个应用软件时提交密码，并将密码保存在内存中。在此情况下，密码没有以明文方式存储在终端设备中。然而，密码还是在内存中，因此，具有足够权限的攻击者还是有可能获得此密码。

2）进一步，通过对密码进行加密而对以上的概念提供安全保障，加密的密钥则基于时间、随机数据等。采用这种方式，攻击者必须首先具有终端设备的 root 访问权限，同时攻击者不再只是简单地导出进程在内存中的数据，而是必须设法找到加密密钥、加密后的密码以及破解密码的算法。

3）再进一步，对内存中的密码设置超时机制，或者改变所使用的加密密钥。

4）如果密码保存在 NAND 闪存上，使用一个加密术语对密码进行加密，加密术语要求在终端设备重启后才能输入。

虽然没有任何方法能够提供一个完美的安全系统，但是破解数据所需要付出的努力及拥有的权限比获得以明文方式存储的数据要高出很多个数量级，因此可以有效遏制大多数的攻击。

5.3.2 美国的计算机犯罪法律与规定

美国已经制定了一系列面向静态和传输中的数据安全的联邦法律。本节的目的是列举几个与之密切相关的法律，并对相关背景进行简要介绍。在美国，已经有 40 多个联邦法律用于对与计算机相关的犯罪进行起诉（Country, n.d.）。此外，每一个州通常还有各自的关于计算机犯罪的法律和法规。

在联邦层面，美国司法部（Department of Justice，DoJ）将计算机犯罪划分为 3 个领域，其中有两个广泛适用于与移动终端设备中数据安全相关的各类犯罪。这两个领域是（Country, n.d.）：

更多资源请访问稀酷客(www.ckook.com)

1）直接面向计算机网络或终端设备实施的犯罪，包括黑客、病毒、蠕虫、恶意软件、监听器等。

2）使用计算机网络或终端设备实施的犯罪，如人为诈骗、身份窃取、商业间谍等。

计算机诈骗和滥用法（The Computer Fraud and Abuse Act，CFAA）的重点是针对政府机构和金融机构中的计算机实施的攻击，以及针对用于进行州际或国际贸易的计算机实施的攻击。此法律适用于比较具体的领域，如在没有获得授权的情况下从计算机系统中获取涉及国家安全的数据；也有更加广泛的条款，如在没有获得授权的情况下使用计算机进行犯罪或获取有价物品。此法律通过其后制定的国家信息基础设施保护法（National Information Infrastructure Protection Act）加以修订，增加了新的滥用方式，也考虑了犯罪的企图。

电子通信隐私法（The Electronic Communications Privacy Act，ECPA）是另外一部有关计算机犯罪的法律，它明确了在没有得到授权的情况下对存储或传输的数据进行侦听属于非法。此部法律涵盖了以下几个核心领域：

- 传输过程中的通信，包括通过口头、有线网络或电子通信（电话监听）。
- 静态数据（存储通信法），目的是保护存储在非易失性内存中的数据。
- 收集通信元数据，如电话号码、IP 地址以及其他用于连接通信方的数据（不包括通信数据本身）。这些通称为“笔记数器以及设陷与跟踪设备”，是指用于截获信息的设备和技术。

最后一个值得提出的法律是 1996 年通过的经济间谍法（Economic Espionage Act），主要注重在窃取商业机密方面。在此法律生效之前，对经济或企业间谍进行起诉难度非常大。通过定义商业机密，并要求信息的拥有者采取合理的措施保护商业机密，现在，窃取知识产权也可能属于犯罪行为。

在此节中，我们只是简单地接触了用于对计算机犯罪进行诉讼的法律方面的主要工作。然而，十分清楚的是现在已经设计了相关的法律，对静态数据和传输中的数据进行保护。

除了联邦和州制定的计算机犯罪方面的法律，还有很多规章制度制定机构对在运行中会涉及敏感数据的公司进行监管。很多相关规定不但为企业提供必须遵守的具体指南和要求，而且包含民事和刑事条款，即有金钱方面的惩罚，对于非常严重的情况，甚至可能涉及监禁。以下是部分比较知名的条例规定：

- 支付卡行业数据安全标准（PCI）
- 健康保险携带与责任法（HIPAA）
- 高科技法临时最后规则（对 HIPAA 的补充）
- 联邦信息安全管理法（FISMA）
- 家庭教育权利与隐私法规 1974（FERPA）
- Gramm-Leach-Bliley 金融服务现代化法规 1999（GLBA）
- Sarbanes Oxley（SOX）

很清楚的一点是，美国国会已经认识到数据与计算机安全的重要性，也提供了一系列的法律、法规及其他手段，用于强制执行为保证商业系统正常运行而必须采取的安全措施。

更多资源请访问稀酷客(www.ckook.com)

5.3.3 开放源码与封闭源码

对于开源软件是否比不开源软件更加安全，在互联网上进行着激烈的讨论。就像其他的长期争论那样，争论的双方都各执己见，因此，争论一直在继续。

坚持认为开源软件更加安全的一方的基本理由是代码可以被任何人检查，可以迅速改正发现的问题。此信念隐含着以下两条假设：

1）开发者会对开源项目中的代码进行检查。

2）软件的维护者会快速修复发现的安全问题。

当然，如果没有以上两个步骤，当开源软件中存在安全漏洞，而且没有得到及时修复，攻击者就拥有了如何利用此漏洞的详细信息。然而，如果以上两个步骤都能够很好地执行，代码的安全将受益于众多的代码检查者以及相应的对软件漏洞的修复。

相比之下，不开源模型建立在由公司进行安全代码开发、代码安全检查、安全漏洞修复以及及时地发布更新后的软件。负责安全运行微软 Windows 平台的人都非常清楚会连续不断地在每个月的第二个周二收到新的更新，因而获得了一个“更新星期二”的绰号。微软通常也会针对严重的软件问题在固定的周二之外发布软件更新。

在一个具有较高知名度的案例中，微软承认（Microsoft Security Advisory, n.d.）公司的安全研究员 Tavis Ormandy 发现了自 1993 年以来发布的所有微软 Windows 的 32 位操作系统中已经存在了 17 年的一个安全问题（Windows NT 至 Windows 7）（Johnston, n.d.）。Ormandy 在 2009 年 6 月 12 日向微软报告了发现的此问题，然而在 7 个月之后才将所有的细节公布在 Full Disclosure 邮件名单中（[Full-disclosure] Microsoft, n.d.））。需要指出的是，在 Ormandy 将此问题公布于众后，微软才在其官方咨询中向微软的客户予以证实，并在一天之内承认了存在上述安全问题。

在另一个更近的案例中，信息安全公司 Matta Consulting 发现了思科公司的统一视频会议（Unified Videoconferencing）平台中存在大量安全问题。由于这些安全问题的存在，一个恶意的第三方能够完全控制此视频会议设备、获得用户密码，并能够对其他用户发起攻击（Cisco Unified Video, n.d.）。思科在对客户的咨询中承认存在这些安全问题，并承认将密码直接编写在源代码中是最危险漏洞的代表：

> Linux 影子密码文件中有 3 个直接指定的用户名和密码。这些密码无法修改，相关账号无法删除。攻击者可以利用这些账号通过使用许可的远程访问协议对远程设备进行访问。
>
> （Cisco Security Advisory, n.d.）

总之，软件开发是一项创造性的工作，无论多么努力、多么执着，还是会存在漏洞。软件和系统的整体安全直接取决于存在的漏洞和错误能够被尽快发现并被及时修复。虽然已经经过了 8 年，David A. Wheeler 在编著的《Secure Programming for Linux and Unix HOWTO》一书中对开源与非开源的争论进行了很好的概述，充分反映了双方的观点和立

更多资源请访问稀酷客(www.ckook.com)

场。Wheeler 引用了 Elias Levy（除了是 SecurityFocus 公司的首席技术官及创建者之一，也被叫做 Aleph One，同时也是 Bugtraq 完全知情列表的协调人）的以下论述：

> 对于安全漏洞而言，这些都表明开源软件没有比非开源软件更加具有优势？不对，开源软件具有比非开源软件更加安全的潜力。但是，千万不要搞错了，单纯的开源并不能确保安全。
>
> (Secure Programming, n.d.)

在此书的结尾，Wheeler 总结道，在以下条件下，开源软件能够更加安全：

- 如果代码由非开源变为开源，随着时间的推移，安全性将会得到逐步改善；
- 必须评审代码；
- 代码检查者和开发者必须知道如何编写安全的代码；
- 一旦发现问题，就必须迅速修改并发布。

归根结底，还是要由每一个人自己决定他们相信开源软件还是非开源软件会更加安全。由于 Android 系统的大部分的确都是开源的，极有可能的就是在初期会发现存在着大量的安全漏洞，但是随着时间的推移，这些安全问题都会得到解决，直至最后产生出一个更加安全的系统。

5.3.4　NAND 闪存加密

一般来说，面向传输中的数据的安全技术和策略已经比较成熟，也比用于对静态数据提供安全保护的技术更加安全。对传输中的数据能够比较容易地提供安全保护的原因在于信息需要得到保护的时间周期比较短，而且能够清楚地定义。在数据传输结束后，用于保护数据的密钥就可以丢弃。相比之下，静态数据保存在非易失存储设备中，必须保证任何时候都能够为用户所用。这就意味着用于解密的密钥必须在存储设备中（否则就必须要求用户每一次都输入密钥，然而这是非常不现实的），因此它们对于攻击者来说也成为可用的。

Bruce Schneier 是一个受到尊敬的安全技术师、密码学家，也是专著的作者，他总结了使用加密技术对静态数据实施安全保护所面临的困难：

> 加密学的发明是为了保护数据通信：传输中的数据。这就是在整个发展史中加密学的应用方式，这也是所有国家的军方发展和探索科学的方式。Alice 是发送者，Bob 是接收者，而 Eve 是一个窃听者。即使当将加密学用于保护存储中的数据时，静态数据也被当做一种类型的通信来对待。在《应用密码学》（Applied Cryptography）一书中，我对加密的存储数据的理解是："存储的数据包是一个人实现与自己长期通信的方法。"数据存储器成为数据通信的一个组成部分。
>
> 在现代网络中，两者之间的区别更加显著。通信是迅速和即时的。加密密钥可以是短暂的，在类似于 STU III 电话系统这一类的系统设计中，加密密钥在通话开始时产生，在通话结束时便被立即销毁。另一方面，数据存储持续很长时

更多资源请访问稀酷客(www.ckook.com)

> 间。只要被加密的数据还存在，加密密钥就必须继续存在。保存这些密钥与保存未加密过的数据一样重要。总之，加密没有减少需要进行安全存储的机密的数量，只不过是使机密性小了许多。
>
> 在历史上，之所以面向存储器中的数据的密钥管理能够正确工作是因为将密钥保存在一个安全的区域里：人脑中。用户会记住这些密钥，并且在不遭受外界或情感攻击的情况下不会泄露这些密钥。也就是说，这些密钥被保存在一台没有连接任何网络的“计算机”中。在那里，这些密钥是安全的。
>
> 以上的模型在互联网中完全破碎。人们只不过附带性地使用互联网中的很多数据，而这些数据主要由其他计算机使用。这样就出现了问题。密钥无法继续保存在人脑中。这些密钥必须保存在存储数据的计算机上，至少是在网络中。这样就极大地提高了风险。
>
> (Schneier on, n.d.)

在其他的一些平台上，尤其是在近期苹果公司的 iPhone 上，数据加密已经在 NAND 闪存中的用户区域部分实现。然而，对于苹果的 iOS 操作系统 3.x 版本，之所以快速破解加密的数据，是因为应用了上述原理。截至 2011 年 3 月，iOS 4.x 内的数据加密还没有被破解。但是，这只是迟早的事而已。

针对数据加密的 NAND 闪存所采取的取证策略与非加密所采取的取证策略有所不同。如果 NAND 闪存没有加密，存储的内容可以通过获取芯片的方式获得，也可以通过 JTAG 过程再进行解码的方式获得。但是，对于加密的 NAND 闪存，此技术就不适用了。要获得解密后的数据，以上进程必须在终端设备运行时同时运行。在取证学中，这种技术称为即时获取技术，它也可以用于其他场景中，如工作站或服务器上的存储在硬盘或其他存储器中加密数据的场景。

对于保护静态数据以及传输中的数据，加密将扮演一个越来越重要的角色。然而，处于静态中的数据基本上总是面临着遭受攻击的威胁。在此种情况下，安全设计者必须在基于加密的安全系统与完全没有任何安全保护的系统之间找到一个平衡点。获得的折中方案应该能够做到对静态数据和传输中的数据提供一定程度的安全保护，同时又不会给用户带来太多的使用不便，以导致此终端设备无法有效使用。

5.4 个人安全策略

至此，本章的大部分内容一直在关注与移动终端设备和数据安全相关的问题，有效的用户实践也能够帮助降低安全风险。正如前面所讨论的那样，以下的列表也许不是很完整，但是仍然可以作为终端设备安全的稳固基础。

1）总是使用可信的数据网络。对于移动终端设备，这些网络包括无线服务提供商的数据网络以及公司、居家和可信地点提供的 Wi-Fi 连接。这样就可以确保用于进行数据传

更多资源请访问稀酷客(www.ckook.com)

输的网络没有恶意，也无法被攻击者用来获取所传输的敏感数据。虽然网络服务提供商的网络也可能出现安全问题，但是大公司的网络比小型网络具有更多的安全保障。同时，虽然获得、设置和管理假冒的 Wi-Fi 连接点比较容易，但是实现假冒的蜂窝数据连接则复杂很多。因此，一般来说，使用由无线服务提供商提供的蜂窝数据连接能够有效降低遭受攻击的风险。

2）总是在 Android 终端设备上设置一个密码，以抵御一般的攻击者直接访问并获得终端设备中的敏感数据。最理想的情况是，Android 终端设备在收到一定次数的错误密码后就会自动地将所有的用户数据全部清除。这样的话，攻击者就无法通过穷举的方式猜测密码。同时，如果允许的话，使用字母与数字相结合的密码，而不是四位的个人识别码（PIN）或者固定模式锁。允许使用复杂的密码组合也会取得更好的安全保障。选择一个与计算机上所使用的密码相近的密码。实在很难想象（也许只是对安全研究者如此）任何一个有计算机的人没有设置密码。

3）连接到 appWatchdog 服务网站（http://viaforensics.com/appwatchdog/）去确认所使用的应用软件通过了基本的安全测试。appWatchdog 对移动应用软件进行跟踪，确定程序能否安全地保存用户名、密码以及敏感的应用数据。此网站对所测试的每一个方面都提供通过、警告、不通过的结果以及所发现的所有信息细节。目前，appWatchdog 的移动应用版正在开发中，最终的软件可以直接安装在 Android（以及 iPhone）终端设备上，用以对安装的应用软件的运行状态进行跟踪。此软件也将允许用户与应用软件的开发商直接进行联系，请求对在跟踪过程中发现的问题以及其他弹出信息的解决方案。

4）不要点击短信中的任何链接，理想的情况是，也要试图避免点击电子邮件中的链接，特别是简短的链接，如 bit.ly 或 goo.gl。根据信息安全公司 Trusteer 的研究，智能手机和平板电脑的用户遭受钓鱼网站攻击的可能性是普通台式电脑或笔记本电脑用户的 3 倍（Donohue, n.d.）。此研究还发现，在智能手机上识别诈骗网站更加困难，这是由于智能手机具有较小的显示屏幕，同时也缺乏保护用户应对钓鱼网站的软件。确保访问正确网站的最好方法是直接输入网站域名，或者是（也许是更好的方法）使用一个可信的搜索引擎来寻找到想要的网站。这样结合在一起使用就可以让用户先直接输入（或错误地输入）想要访问的公司名称，再由搜索引擎帮助找到正确的网址。某些搜索引擎，如谷歌，正在朝着抵御恶意或者被攻破网站的方向努力，这给智能手机用户提供了另外一层安全保障。

5）考虑在 Android 终端设备上使用另外一款 Web 浏览器。正像前面所讨论过的，网络罪犯非常现实，这就是为什么这么多年来他们一直致力于攻击微软的 Windows 操作系统，而基本上忽略了像 Linux 和 Mac OS 这一类操作系统。在移动通信环境中，很多自带的浏览器都是基于开源的 WebKit 项目。因此，非常有可能的就是最初的针对 Android 和其他终端设备而面向 Web 的网络攻击会集中在使用 WebKit 的浏览器上。通过使用另外一款浏览器，可能就会面临较少的攻击。然而，这种方法也只能在名义上短时间内提高安全性。

6）当从 Android 市场上下载和安装应用软件时，确保只赋予应用软件运行所需要的最少访问权限。如果安装的是一个增强版的闹钟应用软件，而此应用软件要求获得访问短

更多资源请访问稀酷客(www.ckook.com)

信和历史浏览数据的权限，应该予以拒绝。虽然这样做并不能保证不受到所有恶意应用软件的侵袭，但这的确是 Android 安全性的重要一层，也是用户应该负起责任的方面。

随着时间的推移，不但以上的列表会扩展、演变，而且也会推出新的安全控制，这些都将有利于对移动终端设备和敏感数据提供有效的安全保障。

5.5 企业安全策略

企业通常会有比个人更加复杂的安全需求，这是因为企业的责任是对整个企业进行保护，不仅要抵御来自外部的攻击，而且要抵御来自内部的攻击。此外，企业也许属于某些被监管的行业，因此他们必须按照本章前部所列出的某些相关指南运行。基于以上原因，需要对包括移动终端设备在内的资产进行更加细化的管理和控制。

5.5.1 安全策略

在企业中，对移动终端设备管理的一个重要方面首先就是认真检查当前正在实行的政策，确保更新相关政策，以适应由移动终端设备引起的新情况。大多数的当前政策没有考虑智能手机和平板电脑终端设备以及它们的应用所带来的新情况。对企业的相关政策进行全面审查是完全必要的，以下是需要引起特别关注的几个政策：

- 可以接受的应用方式（公司的资源现在需要包括移动终端设备）；
- 数据安全（很明显需要围绕移动终端设备制定相关政策）；
- 备份与数据保留（可能受到影响，尤其是从电子发现的角度来看）。

虽然更新政策不会直接改进通过信息技术提供的安全，但却能够为如何改进指出一个很明确的方向，不仅对于企业内的一般员工如此，而且对于企业内的安全架构师以及从事与内部调查和纪检相关工作的人员也是如此。如果实施的政策过于陈旧，将会对企业带来不利影响。例如，只要将移动终端设备简单地加入到可以接受的使用方式政策中，使其成为被涵盖的资源，那么所有允许企业检查其所拥有的终端设备的政策都非常明确地定义清楚了。否则，如果卷入到一个法律诉讼中，对方的辩护律师会申明移动终端设备不在相关政策所覆盖的范围内，从而在终端设备发现的证据（如公司的保密文件）是无法被法庭采纳的。虽然公司的律师团可能有能力驳倒以上论点，简单地将相关政策覆盖移动终端设备是非常便宜和更加有效的方式。

5.5.2 密码、模式和个人识别号锁

密码、模式和个人识别号锁在 Android 终端设备中的实现既没有保持一致，又不是很有效。然而，它们在一定程度上提供了安全。自首次使用以来，这些功能有所改进，并将随着时间的推移继续得到改进。随着处理能力呈几何级数的增长以及同期中成本的降低，使用最简单的技术攻破安全机制已经成为可能，而且逐年得到普及。然而，如果终端设备丢失或被偷窃，这些机制可以提供最基本的安全保护，对于抵御业余犯罪具有一定的效果。

更多资源请访问稀酷客(www.ckook.com)

并不是所有的密码都等同。第一种有效的保护方式是使用最有效的密码，这种方式允许或要求使用字母与数字相结合的密码。虽然用户输入这些更加有效的密码会麻烦些，但这样构成密码会大幅提高密码破解所需要付出的努力。第二种有效的保护方式是使用模式锁，自第一款 Android 终端设备之后就一直存在。模式技术提供了一个新的锁住终端设备的方式，因此要求用户学习使用此项新技术，结果是产生更加有效的锁。不再依赖于重复使用旧的方式（如个人识别号），用户需要提出新的模式，因此使攻击者使用与用户相关的信息去猜测模式锁更加困难。也许是作为一个更加为用户所熟悉的方式，很多 Android 终端设备中依然支持脆弱的个人识别号（PIN）。虽然使用个人识别号比不用任何保护措施要好，但是个人识别号也许是最容易被击破的锁方式。个人识别号通常由有限位的数字组成（最常见的是 4 位），这种方式可使恶意攻击者很容易猜测所有可能的组合，带来巨大的危险。同时，作为安全措施，个人识别号已经应用在很多其他系统中，最知名的应用就属自动提款机（Automated Teller Machine，ATM）卡。人们倾向于重复使用个人识别号，并且通常基于与他们个人相关、很容易得到的简单情况选择个人识别号。

正如前一节在个人安全策略中所建议的那样，密码锁也只应该允许有限次的尝试，当尝试的次数超过最大允许的次数时，终端设备应该重置为出厂设置，并消除所有的用户数据分区。这样做可以抵御针对容易被破解的密码（如四位的个人识别号）所发起的攻击，为系统安全提供另一层的保护。

5.5.3　终端设备远程清除

企业中的安全管理人员最希望得到的安全功能之一就是能够远程清除智能手机里的数据内容。毫无疑问，此功能非常重要，也非常强大。但是，这项功能非常脆弱，并且安全管理人员对其抱有过高的信心。

远程清除最基本的前提就是公司可以对终端设备发送一条命令，将终端设备中的数据全部清除，并恢复出厂时的设置。很多智能手机中都有允许实施这种控制的机制。但是，此机制不是一开始就设计在系统中，而且无法提供足够的可靠性来确保高度安全。

虽然提供的远程清除机制是有效的，但是抵御接收远程清除命令也很容易，如将终端设备置于飞行状态、将 SIM 卡取出等一系列方式。实际上，某些公司可能首先将即将离开公司的员工手机账号停机，然后发送远程清除命令。在大多数情况下，这样做他们却在无意间失去了远程清除的能力。虽然这时的终端设备已经不再与网络连接，但是依然可以访问存储在终端设备上的数据。比较懂的终端设备拥有者还可能使用他们的 root 访问权限过滤远程清除命令，简单地选择忽略。总之，最主要的问题还是远程清除功能不成熟。

在摩托罗拉 Droid 上运行 Android 2.1 版本时，通过微软的 Exchange ActiveSync 进行驱动的远程清除功能不会造成终端设备清除所有的数据。然而，当在摩托罗拉 Droid 上运行 Android 2.2 版本上测试此功能，远程清除顺利完成。因此，如果依靠远程清除来确保数据的安全，则必须确认远程清除功能在所有的 Android 硬件平台以及所有安装的 Android 版本上都能够正常完成。很不幸，对于很多 IT 部门来说，这是一项难度很大的工

更多资源请访问稀酷客(www.ckook.com)

作，原因在于终端设备高度分布，同时目前还缺少一个集中管理工具（在后续章节中会有更多相关讨论）。

为了克服远程清除的局限性，一个简单的技术是在终端设备中安装一个应用软件，此应用软件在检测到终端设备无法顺利与企业系统相连接已经超过了规定时间后就会自动清除终端设备上的所有数据。使用此技术（通常被称作为看门狗或倒计时应用）能够极大地提高数据的安全性，这是因为在无法与公司的系统实现连接超过规定的时间后，终端设备会自动将其中的数据清除。所以，即使终端设备处于离线状态，安全管理人员也可以确信，所有在终端设备上的公司数据在规定的时间内就会被清除。当然，如果用户很长时间都没有将智能手机与网络相连接是出于合理合情的原因，造成的后果也是智能手机上的所有数据被清除，因此，对用户进行适当的培训，使其了解此功能及其带来的后果非常重要。目前，还没有发现（也许存在）支持这一功能的应用软件。

还有另外一项面向存储在 NAND 闪存上数据的保护技术。极有可能（实际上已经存在）开发出一款应用软件来定期清除用户终端设备上数据存储分区中无用（未分配）空间里的数据。得到的好处就是（大多数）被删除的数据永远不可能恢复，即使能够获得存储空间的物理映像。清除功能可以通过让一个应用软件简单地将一个内容全部为 0xFF 的文件写入此分区的方式来实现，直至整个分区都被覆盖。在以上操作完成后，再删除此文件，因此，整个分配的空间都被覆盖。当然，此方式也会带来一些负面影响，包括缩短了 NAND 闪存的使用寿命、导致在写文件过程中终端设备无法及时响应、耗费电池中的能量、造成其他应用软件的执行非正常停止或数据丢失，后种情况发生在当这些应用软件试图访问数据分区，但是数据分区已经被全部占用的情况下。

5.5.4 升级到最新版软件

虽然智能手机市场更新换代非常快，但是企业对核心通信基础设施进行升级却非常困难，如对微软 Exchange 环境进行更新升级。好消息是，微软 Exchange 最新的两个版本提供改进后的移动管理和安全机制。当将 Exchange 升级到最新的版本（目前为 Exchange 2010）时，会发现有更多的终端设备安全管理选项可供使用。确保企业的通信基础设施及时更新换代不但能够改进整体的网络管理性，而且还能够全面改进所连接的移动终端设备的安全性。

同样，移动终端设备也要及时进行更新。虽然在新的版本中可能会出现新的软件错误和安全漏洞，一般来说，更新后的软件不但会修复之前发现的软件错误和安全漏洞，而且会增加新的管理和安全功能。在 5.5.3 节中所列举的实例是一个很好的证明，在摩托罗拉 Droid 上运行 Android 2.1 版本时，远程清除没有正确执行，但是当运行 Android 2.2 版本时，此问题得到解决。

5.5.5 终端设备的远程管理功能

企业客户有对连接到公司通信基础设施的移动终端设备进行管理的需求，这种需求日益增长。虽然这些需求因公司而异，但一般性的需求包含以下条目：

更多资源请访问稀酷客(www.ckook.com)

- 远程部署终端设备；
- 远程清除终端设备；
- 强制执行公司的 IT 策略，如密码、数据加密、最低版本操作系统要求、更新要求、许可 / 不许可的应用软件等；
- 远程安装 / 升级应用软件。

Android 2.2 版本引入了新的终端设备管理应用编程接口（API），能够将上述管理要求与应用软件进行集成。具体来说，它能够支持（Device Administration, n.d.）以下功能：

- 启用密码：要求终端设备请求用户输入个人识别号或密码。
- 密码最小长度：要求密码长度满足的最少位数。例如，要求个人识别号或密码的长度至少为 6 位字符。
- 要求密码同时具有字母和数字：要求密码是字母和数字的组合，也可以包含其他字符。
- 密码失败最大次数：设置终端设备清除数据前用户输入错误密码的最大次数。终端设备管理应用编程接口也允许系统管理员通过远程方式将终端设备初始化成出厂时的设置。此方式在终端设备丢失或被偷窃时可以确保数据安全。
- 无操作后锁住屏幕所需的最长时间：设置在用户最后一次触摸显示屏或按键后锁住屏幕所需要的最长时间。当屏幕被锁住后，用户需要输入个人识别号或密码才能继续使用终端设备，访问设备中的数据。设定的时间可能在 1 ～ 60 分钟。
- 提示用户设置新密码。
- 立即锁住终端设备。
- 清除终端设备中的数据（即将终端设备重新设置为出厂时的默认设置）。

在 developer.android.com（Device Administration, n.d.）网站上的 Device Administration（终端设备管理）页面展示了几个此应用编程接口的实例，如图 5.2 所示。

当用户使用新的管理接口安装一个新的应用软件时，展现在眼前的是一个 Enable Device Admin（允许终端设备管理）屏幕，如图 5.3 所示。

图 5.2　终端设备管理界面实例

图 5.3　开启终端设备管理

更多资源请访问稀酷客(www.ckook.com)

虽然与无终端设备管理的情况相比，这已经是一个改进，但是仍然无法满足企业客户的需求。此外，虽然提供了应用编程接口，但是开发者还是必须将其与应用软件进行集成。

有一些公司正在尝试设计完整的终端设备功能管理。然而，针对智能手机的解决方案还是非常不成熟。将希望寄托在第三方提供的解决方案上也许只能满足短期的需要。但是，随着时间的推移，Android、制造商以及终端设备生产商一定会开发出更加成熟的管理功能和工具。同样，很多公司里的终端设备基于微软的 Exchange ActiveSync（EAS）协议实现与电子邮件系统的同步，这个平台上的管理功能一直在进步和成熟。因此，如果当前的安全要求需要更好的远程终端设备管理，企业可以考虑使用第三方提供的安全管理工具，以逐步改进安全性。但是，采取此方法的风险是这些安全管理工具无法满足或优于制造商提供的相应工具，因此，相关技术和所投入的人力资源很难证明是物有所值。

由于很多在终端设备中的企业数据主要以电子邮件为主，将安全保护的重点放在此方面会获得显著的效果。很多公司已经意识到这种情况，进而开发出公司专用的电子邮件客户端，其中包括了很多其他的安全保障机制，如双元素用户认证、加密的数据存储以及更多的管理界面。但是，安全意识强的企业应该对软件进行审验，以确保这些软件正确地实现所有号称能够得到支持的功能。

很明显，远程终端设备管理对于企业客户来说非常重要，虽然这方面的功能还不是很成熟、完善，可以预计在今后版本的 Android 中将会出现重大的改进。

5.5.6 应用软件与终端设备审计

如果对处于安全威胁的那些信息缺乏全面了解，很难做到对终端设备以及设备中的数据进行有效的安全保护。企业应该对移动终端设备进行一个移动安全与风险排解方面的审计，对终端设备中的数据进行评估。通过分析企业中所使用的终端设备和操作系统，就可以确定终端设备中存储着哪方面的数据、风险从何处而来（针对终端设备中的静态数据、传输中的数据以及备份数据等），从而制定降低风险的具体策略、流程，甚至软件实现。

典型的审计必须包括对如下方面的评估：

- 在终端设备上允许存储什么类型的数据；
- 这些数据存储在哪里；
- 允许安装哪些应用，又允许哪些访问公司的数据；
- 远程数据清除及终端设备管理的能力和有效性；
- 执行企业安全策略的能力；
- 密码、认证信息和其他敏感信息的安全存储与传输；
- 加密的有效性。

在对所支持的终端设备进行全面了解后，确保静态数据和传输中的数据安全的任务并不是很难，并且要制定具体的策略，以降低不可接受的风险。

与其他领域的信息安全相同，需要通过制定策略以及执行不间断的管理过程来维护安全措施和评估风险。

更多资源请访问稀酷客(www.ckook.com)

5.6　应用开发安全策略

最后一个移动安全中极为重要的方面就是开发和安装的应用。这些不但包括第三方开发商开发的应用软件，而且包括由操作系统开发商（此刻就是 Android）、终端设备生产商和移动通信服务提供商发布的应用软件。这些应用软件是用户与其终端设备实现交互、访问感兴趣信息的主要机制。通常，这些信息具有敏感性，提供关于用户隐私的很多细节。

5.6.1　移动应用安全测试

在从事取证工作的过程中发现了不计其数的移动应用安全漏洞后，本书的作者以及在 viaForensics 公司的同僚们开始对流行的应用中的数据安全进行跟踪，并将发现的结果在该公司的网站上对外公布。这一项称为 appWatchdog 的免费服务的目标是改进移动应用中的数据安全，保护消费者。随着消费者对数据安全风险意识的提高，开发者最好在发布应用软件之前认真、深入地对应用软件进行评测，以达到更高级别的安全。所有的发现都可以在 http://viaforensics.com/appwatchdog/ 网站上浏览。viaForensics 公司计划开发一款移动软件，用于检查安装在移动终端设备上的应用软件，并向 appWatchdog 提供跟踪检查的结果。此移动软件也允许消费者提交用于进行评估的应用，并在有任何顾虑的情况下，能够与应用软件开发商取得联系。

appWatchdog 服务运用取证与安全评测技术确定用户名、密码、信用卡号码以及其他应用数据是否安全存储。整个过程包括安装并正常运行应用软件。然后就会按照取证的方式和要求获取终端设备中的映像，并分析其中的个人信息和应用数据。所有的发现首先会反馈给应用软件开发商，然后会公开披露，目的是将这些信息直接提供给消费者。用户就可以在拥有充分信息的情况下决定是否继续使用此款应用软件，或者等待开发商提供一个更加安全的版本。

对每一个应用软件的评测都是建立在一定条件的基础上，这些条件取决于此款应用软件的具体应用。例如，对于移动支付应用软件，针对它的分析将侧重于用户名、密码、应用数据及信用卡号码。但是，对于其他的应用软件，也许根本没有与信用卡号码关联的任何信息。以下进一步解释了某些条件，其中，前 3 条与大多数的应用软件相关：

- 是否安全地保存密码？如果存储在终端设备上的任何形式的密码没有经过加密，应用软件将得到“不通过”。
- 是否安全地保存用户名？检查应用数据，以确定保存在终端设备上的用户名是否经过加密。
- 是否安全地保存应用数据？对每一项应用进行与应用数据相关的分析。例如，对于金融应用，查找账号、余额及转账信息。在其他应用软件中，也许会保存其他方面的用户个人数据，如电子邮件地址、电话号码、地址等。
- 是否安全地保存信用卡信息？对于会处理信用卡信息的应用，对数据进行检查，以

更多资源请访问稀酷客(www.ckook.com)

确定保存在终端设备上的完整信用卡号码经过加密，同时还包括其他与信用卡号码相关的信息，如信用卡的有效截止日期以及打印在信用卡背面的安全数字。

- 其他安全测试：这些测试包括截获移动终端设备传送的无线数据，检查这些数据中是否包括用户名、密码、个人识别号以及任何与安全相关的应用数据。这一类的安全测试通常非常耗时，因此通常只用于对应用软件进行深入的安全评测。

appWatchdog 服务只是关于移动应用软件是否实现了安全的一个基本标识。只有与更先进的移动取证和安全技术相结合，才能够对移动应用软件进行更加深入的安全分析。以下列出的是确定移动应用软件是否安全而需要进行评估的一些条件：

- 应用软件如何处理 Web 浏览记录以及是否缓存？
- 应用软件是否安全地传输登录数据？
- 应用软件是否能够避免中间人攻击？
- 应用软件是否安全地传输敏感数据？
- 应用软件是否得到防会话劫取攻击保护？
- 应用软件是否能够永久性地删除数据以及防止在终端设备上存储数据？
- 应用软件是否安全地处理中断？
- 应用软件是否对备份数据提供合适的安全保护？

深入全面的应用软件审计充分利用先进的取证和安全测试技术发现安全漏洞，以保护开发者和用户。viaForensics 提供这些测试服务以及称为 appSecure 的认证。类似的测试也可以由企业内部的取证和安全小组有效实施，只要这个小组拥有相关的工具、具有相应的能力以及完成这些测试的经费和时间。

5.6.2 应用安全策略

appWatchdog 和 appSecure 取得的结果帮助形成了一些指南，提供给移动应用开发商在设计、开发和测试应用软件中进行参考。与其他指南一样，此指南中所列出的条目不一定完整，却能够提供一些值得参考的概念。

1. 用户名

避免在终端设备中以明文的方式保存用户名。为了确保更好的可用性，可以将用户名缓存在终端设备中，从而消费者就不需要每一次都输入用户名。但是，可以考虑屏蔽部分用户名，同时依然有足够的信息使用户能够识别用户名，而没有足够的信息使攻击者获得完整的用户名。用户名是用户进行账号登录的一个组成部分，因此，攻击者知道得越少，攻击的有效性就越差。

例如，假设一个访问敏感健康信息的应用软件要求用户提供用户名。用户创建了一个叫做“andrewhoog”的用户名。然而，在初次登录后，如果此应用软件只保存“andr＊＊＊＊”并显示给用户，很清楚的是登录成功完成。然而，一个攻击者只能获得用户名的前4个字符。更进一步，如果字符屏蔽（此例中使用的是“＊”，也可以其他方式呈现）不显示出用户名的长度，则使攻击者成功猜测用户名变得更加困难。

更多资源请访问稀酷客(www.ckook.com)

最后，越来越多的网络在线服务要求使用更加复杂的用户名，如要求最小的长度以及使用大小写字母与数字的组合。因此，“andrewhoog”也许比较容易就可以被正确地猜测到，AndrewHoog6712 则非常困难。某些网站对用户名甚至增加了更多的限制，其中不可以包含与个人身份相关的任何信息，如用户的真实姓名。

2. 密码

也许更加令人担忧的情况是将密码以明文形式进行保存的应用软件。目前已经有了一些避免这一严重问题的策略。首先，正如在 5.3.1 节中所讨论的那样，可以强迫用户每次运行应用软件时都要输入密码。如果从家用计算机中登录银行的网站，则每一次都必须登录。同时，在成功登录银行网站后的会话过程中，一旦会话过程中的无活动状态超时（或者用户登出），则无法继续访问被保护的网站，除非重新使用用户名和密码成功登录。

另外一个可以考虑的方法是使用安全标记，以避免在终端设备上保存用户名和密码。如果用户对互联网上一个被保护的资源成功地进行了认证，将产生一个安全标记，此标记不但在经过一定时间后会失效，而且对于终端设备是唯一的。虽然任何实际得到此智能手机的人都可以访问受保护的资源，但是此访问只能持续到目前的安全标记到期那一刻，此后就再次需要使用用户名和密码。同时，如果此安全标记对终端设备来说是唯一的，即使通过远程方式截获，也无法用于访问受保护的资源。使用安全标记的方式也不会将网上其他的受保护资源置于攻击风险之中，即使账号的用户使用同样的用户名和密码（这是很常见的事情）。

基于安全凭证实现的认证机制比简单地基于用户名和密码的机制要复杂很多，没有得到广泛应用，也没有得到开发者的广泛了解。然而，一定数量的应用编程接口提供这个认证功能，而且也在不断完善和成熟。其中一个叫做 OAuth 的框架得到很多谷歌服务的支持。还有其他的一些应用编程接口，但是却倾向于侧重在某些特定的服务上。它们之间有一定的相似性，以下是 OAuth 网站描述它与其他服务之间的关系：

> OAuth 是一个标准化的产物，它集中了众多成熟的行业协议中的精髓。它与其他目前得到广泛使用的协议相似（如谷歌 AuthSub、AOL OpenAuth、雅虎 BBAuth、Upcoming API、Flickr API、亚马逊 Web Services API 等）。每个协议都对交换用户身份信息以获得访问凭证或票据提供专有的方法。OAuth 是在认真研究了这些协议、提取了各自的优点以及共同点的基础上提出的，以支持新的实现以及将目前的服务平稳地过渡到支持 OAuth 。
>
> 与其他某些协议相比，OAuth 更加具有优势的一个方面是它具备直接处理非网站服务的能力。OAuth 用于支持台式计算机应用、移动终端设备、机顶盒以及网站。在很多当前使用的协议中，用于进行通信的共享机密被直接编写在软件中，这样的话，如果用于访问隐私数据的服务是基于开源软件就会成为一个问题。
>
> (IntroductiondOAuthn, n.d.)

更多资源请访问稀酷客(www.ckook.com)

由于正在向 OAuth 进行过渡，谷歌提供 OAuth 应用编程接口的详细信息以及如何使用这些应用编程接口。谷歌代码（Google Code）网站上具有以下对 OAuth 过程的基本概述（Default, n.d.）：

1）应用程序请求访问，从谷歌的认证服务器获得一个非授权的请求凭证；

2）谷歌请求用户对应用程序要求访问的数据进行授权；

3）应用程序从认证服务器获得一个授权的请求凭证；

4）应用程序使用授权的请求凭证换取一个访问凭证；

5）应用程序使用访问凭证向谷歌的服务访问服务器要求获得数据。

从以上的步骤可以看出，用户名和密码只被发送到认证服务器（此例中就是谷歌），没有保存，也没有与请求的应用程序进行共享。请求的应用程序获得的是各种凭证，用于在整个过程中进行协商，但是仍然需要保存最后获得的访问凭证，然而它的有效时间以及有效范围都是非常有限的。

虽然在 OAuth 的典型场景中允许存在一个第三方服务（如一个社交媒体网站）以及时间与范围受限、基于凭证的认证，以达到保护信息的目的（如 Gmail 中的联系人列表），需要指出的是，应用程序开发商可以使用 OAuth 服务对自己进行认证。使用这种方式，应用开发商不但可以以标准的方式对所提供的服务实现安全、基于凭证的访问，而且还具备了基础设施允许获得许可的第三方应用访问这些数据，只要账号的拥有人对这些访问予以授权。

目前还有很多不同的用于对用户进行安全认证的方法和技术不需要移动应用软件在移动终端设备上以明文的方式保存用户名和密码。应用软件开发商必须逐渐过渡到使用这些更加安全的方法，以便能够对客户提供更加有效的安全保护。

3. 信用卡数据

信息安全领域的大多数人对支付卡行业（Payment Card Industry，PCI）数据安全标准（Data Security Standard，DSS）非常熟悉，此标准是一个保护信用卡数据的安全标准。在支付卡行业信息安全标准委员会（PCI Security Standards Council，SSC）成立之前，主要的信用卡发行商都已经制定了各自的信用卡数据保护标准。支付卡行业信息安全标准委员会于 2006 年由以下几个国际支付卡品牌发起组成：

- 美国运通（American Express）；
- Discover 金融服务（Discover Financial Services）；
- JCB 国际（JCB International）；
- MasterCard 全球（Mastercard Worldwide）；
- Visa 公司（Visa Inc.）。

以上这些支付卡品牌都对降低支付卡行业中的犯罪表现出足够的兴趣。在 2010 年 10 月发布的“需求与安全评估程序”（Requirements and Security Assessment Procedures）2.0 版本包含了支付卡行业数据安全标准的具体要求。以下是这些具体要求中有代表性的很小一部分，主要反映对移动终端应用中使用到的信用卡数据进行保护（Documents Library, n.d.）：

更多资源请访问稀酷客(www.ckook.com)

- 3.2 在授权后不再保存敏感的用于授权的数据（即使经过加密）。敏感的用于授权的数据包括要求 3.2.1 至要求 3.2.3 中提到的数据。
- 3.2.1 不要保存任何磁道中的全部内容（信用卡背面磁条上的内容或者是存储在芯片上的同等内容等）。这些数据被称作全部信道、信道、信道 1、信道 2 或磁条数据。
- 3.2.2 不要保存信用卡验证码或数值（打印在信用卡正面或者背面的一个三位或四位数），验证码用于在无法出示卡的交易中对持卡人进行验证。
- 3.2.3 不要保存个人识别号（Personal Identification Number，PIN）或者经过加密的 PIN 加密块。
- 3.3 当显示信用卡号码时，遮挡 PAN（信用卡号码的前 6 位或后 4 位是能够用于显示出的最多位数）。
- 3.4.1 如果对整个磁盘进行加密（而不是在文件级或者列级的数据库加密），高层访问控制的管理必须独立于底层操作系统中的访问控制机制（例如，不要使用本地的用户账号数据库）。用于数据解密的密钥不能与用户账号绑定在一起。

虽然对支付卡行业数据安全标准也有一些批评，但此标准已经相对比较成熟，其目标是对海量的金融交易进行保护。对这些年来某些支付卡行业数据安全标准进行比较非常有趣，它涉及对加密进行要求、对保存敏感信息进行限制以及对显示在屏幕上的信用卡敏感信息进行遮挡。

不用说，将信用卡信息用明文的方式存储在移动终端设备上不但违反支付卡行业数据安全标准，而且在终端设备出现安全问题时，也会使信用卡用户面临巨大风险。也许并不奇怪，以上介绍的 appWatchdog 服务对存储明文数据的信用卡进行了验证，确认在某些应用中能够获取这些数据。

如果一个应用需要涉及信用卡处理，建议这个应用与成熟的在线支付服务商（如 PayPal 或 Google Checkout）进行集成，或者与一个支付提供商进行合作，以实现安全支付应用。需要指出的是，不但从开发而且从测试的角度来说，这都不是一项简单的工作，同时还要求有一个连续不断的安全审计流程。

4. 敏感的应用数据

很多移动应用都会涉及用户不想失去控制的敏感数据。数据的敏感度也有所不同。以下通过简单的列表对这些不同敏感度的数据进行说明：

1）没有任何用户数据，例如，计算器应用软件不会涉及任何敏感的用户数据。

2）没有任何用户数据，但是包含某些元数据，例如，气象应用软件不会涉及任何敏感的用户数据，但是在运行时可能涉及全球定位系统（GPS）坐标数据以及日期 / 时间信息。

3）包含无敏感性的用户数据，某些应用软件的目标就是为公众服务，如 Twitter 中用户共享的消息。如果一个用户被告知关于他的所有消息都能够被世界上任何人阅读，大多数用户不会对此表示关心（除了想要保护自己的消息的那些用户）。

4）包含用户的敏感数据，多数应用软件都属于这种类型，会涉及用户的敏感数据，如电子邮件地址、短信和彩信消息、留言、通话记录等。

更多资源请访问稀酷客(www.ckook.com)

5）高度敏感的用户数据，这是一个特殊的类型，包含的应用软件会涉及金融信息、健康信息、密码等，也包含其他当安全出问题时会使消费者遭受巨大安全威胁的应用软件。

对应用软件所涉及的用户数据的敏感性进行区分有助于对应用软件和数据提供不同层次的安全保护。如果一个计算器应用软件要求有双因素认证以及使用 AES-256 进行加密，用户肯定会感到反感而决定在纸上去计算 15% 小费的数额（一旦写在纸上也许会遭受到不同的直接攻击及社交攻击的威胁，但是我们不同意这个观点）。

但是，appWatchdog 服务经常发现应用软件将涉及的第 4 层和第 5 层的数据以明文的方式存储在 NAND 闪存上。如果开发的一个应用软件所涉及的数据属于这些层次，这些数据应该受到一定程度的安全保护。以下是一些可选的做法：

1）不要保存数据，这是最简单的方法，能够应对任何对 NAND 闪存进行的授权攻击。前面已经讨论过，加密算法对传输中的数据保护的效果大大优于对静态数据保护的效果（至少对移动终端设备如此）。大多数的终端设备总是连接到互联网上，因此，没有必要在终端设备上缓存数据。当然，缓存数据有其优点，比如在终端设备离线状态下仍然能够访问数据以及能够缩短应用软件执行的响应时间等。

2）对数据进行加密，正如前面讨论过的那样，对静态数据进行加密的其中一个要求是将加密所用的密钥保存在终端设备上，因此，很难做到完美的安全。但是，如果很难发现加密密钥，此方法仍然是一个比保存明文安全度要高很多的方案。另外，随着远程攻击数量的增加，攻击可能只会造成对 NAND 闪存区域的访问，而不是对其他保存密钥的区域，如内存。如果数据以明文的方式保存，肯定会有很大的风险；但是如果数据被加密，在以上这种情况下，用户的数据能够得到有效的保护。

保护敏感数据对于保护消费者的利益非常关键，行业必须成熟起来，并且将安全作为对应用软件的一个要求。

5. SSL/TLS

最后一个讨论的话题是安全套接字层（Secure Sockets Layer，SSL）和传输层安全（Transport Layer Security，TLS）的实现，用于保障传输中数据的安全。正确地实现 SSL/TLS 非常关键，包括对数字证书进行全面验证，以抵御中间人（MITM）攻击。虽然 SSL/TLS 对保护数据传输非常有效，但是如果没有正确实现，仍然会使用户面临攻击的危险。在使用 appWatchdog 对应用软件进行测试时，一个测试攻击机器会将其他不相关的应用软件的用户名和密码按照一定规律显示在显示屏上，这是由于这些应用软件会自动运行（例如，某些应用软件会周期性地登录，来查看是否有新的消息），自然就成为受害者。

在确保 SSL/TLS 正确地实现并在移动应用软件中予以验证的基础上，还必须确保 SSL/TLS 在服务器端的安全配置。SSL 自从 20 世纪 90 年代以来就被用于在不可信网络中安全地传输数据。早期的 SSL 实现中存在安全漏洞，因此必须修复，以确保安全数据传输。随着时间的推移，对此协议中的算法进行了改进，以应对简单的蛮力攻击。最新的技术就是 TLS，它与 SSL 的基本原理相同，因此很多人仍然简单地使用 SSL 来代表这两种协议。

更多资源请访问稀酷客(www.ckook.com)

一个常见的问题是，负责服务器端 SSL/TLS 实现的人不一定是安全工程师，因此倾向于更加关注服务器基础设施。他们实现并测试过的 SSL/TLS 表面看似安全，但是仍然可能存在安全漏洞。例如，MD5 是一个广泛应用的加密算法，使用时间已经超过 10 年。虽然仍然认为 MD5 确保文件的完整性有效，但是官方已经表明 MD5 不再用于保护通信数据的安全。美国国土安全部（The US Department of Homeland Security）的 CERT 小组在漏洞通知（Vulnerability Note）#836086 中做了以下陈述："软件开发者、认证权威机构、网站拥有者以及用户应该在任何场合下避免使用 MD5 算法。正如在先前的研究中所展示的那样，此算法的加密性可以被破解，因此不适合在未来继续使用"（US-CERT Vulnerability Note VU#836068, n.d.）。实时地对此算法进行破解依然是不现实的，但是主要的 Web 浏览器和移动终端设备已经支持更强大的加密功能。

另外一个更为严重的问题是，在服务器端的 SSL/TLS 设置中允许接受未加密或者较弱的允许出口的加密算法。过去的几年中，美国对具有加密技术的终端设备和软件设置了出口限制，因此，在销往美国以外的软件中，只能实现强度较弱的加密算法，算法通常只使用较短的 40 位密钥，与 128 位或者 256 位的加密算法相比较，可以非常容易地破解。

在 SSL/TLS 的使用中，还有一个问题就是实现的版本。TLSv1 已经使用超过 10 年，在 2009 年被发现存在"再解协商攻击"（renegotiation attack）问题。在此攻击中，服务器将客户端的最初 TLS 握手按照再协商对待，因此可能将攻击者传输的最初数据误认为来自于随后会传输数据的客户端（US-CERT Vulnerability Note VU#120541, n.d.）。安全咨询师 Thierry Zoller（2009 年 11 月）完善地总结了这一过程，其中提供了步骤的描述。并且在该总结中对很多不同的攻击进行了解释，图 5.4 是用于解释攻击的第一个实例。

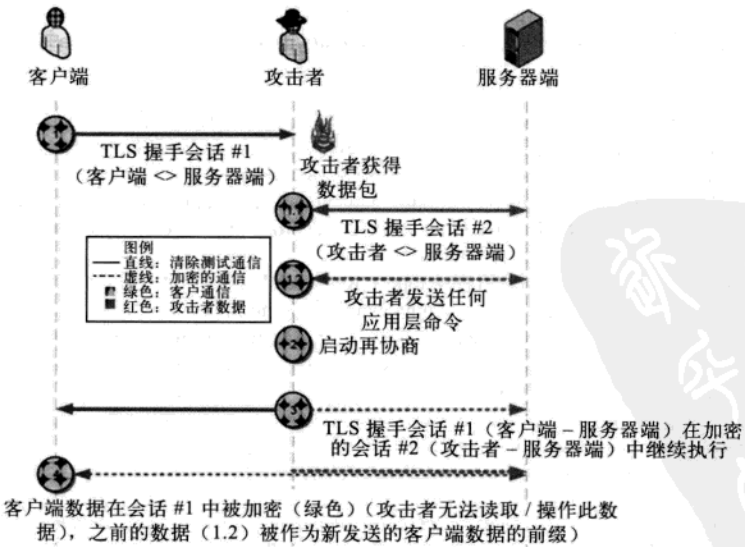


图 5.4　通用的 TLS 再协商前缀注入风险

更多资源请访问稀酷客(www.ckook.com)

大多数使用 TLSv1 的服务器都已经使用了去除此风险的补丁。然而，TLSv1 协议已经得到更新，目前的 TLSv1.2 提供最新的技术以及更强壮的数据加密。旧的包含 SSLv2 和 SSLv3 的规范依然在广泛使用，然而如果使用合适的加密技术和长度足够的密钥还是可以确保一定程度的安全，但是安全性仍然略逊于 TLS。

对移动终端设备的数据传输提供安全保障需要协调、勤奋以及对 SSL/TLS 的深入了解，不但要求应用软件开发者做到这一点，而且要求对参与安全通信的服务器进行维护的小组成员也要做到这一点。虽然 SSL/TLS 已经存在了相当长的时间，也得到广泛采纳，重要的依然是对系统的正确实现以及全面测试。

5.7 本章小结

Android 终端设备可以是恶意攻击的目标，也可以成为实施这些恶意攻击的工具。个人用户以及企业用户必须意识到存在的安全风险，并且采取有效、具体的措施来抵御恶意滥用。应用软件开发者们也必须要提高他们对安全问题的重视程度，并且负责保护用户数据。虽然实现本章所讨论的基本安全措施无法提供全面的安全保障，但是至少可以对大多数恶意攻击起到屏障的作用。

5.8 参考资料

Country. (n.d.). Computer crime law—guide to computer crimes law. Retrieved February 10, 2011, from http://www.hg.org/computer-crime.html.

Cisco unified video conferencing multiple vulnerabilities. (n.d.). Matta Consulting. Retrieved February 11, 2011, from www.trustmatta.com/advisories/MATTA-2010-001.txt.

Cisco Security advisory: multiple vulnerabilities in Cisco unified video conferencing products—Cisco Systems. (n.d.). Retrieved February 10, 2011, from http://www.cisco.com/en/US/products/products_security_advisory09186a0080.

default. (n.d.). OAuth for installed applications—authentication and authorization for Google APIs—Google Code. Retrieved February 12, 2011, from http://code.google.com/apis/accounts/docs/OAuthForInstalledApps.htm.

Device administration. (n.d.). Android Developers. Retrieved February 11, 2011, from http://developer.android.com/guide/topics/admin/device-admin.html.

Donohue, B. (n.d.). Mobile device users more susceptible to phishing scams. Retrieved February 10, 2011, from http://threatpost.com/en_us/blogs/mobile-device-users-more-susceptible-phishing-scams-010511.

Documents Library. (n.d.). PCI security standards documents: PCI DSS, PA-DSS, PED standards, compliance guidelines and more. Retrieved February 12, 2011, from https://www.pcisecuritystandards.org/security_standards/documents.php?agreements=pcidss&assocation=PCI%20DSS.

[Full-disclosure] Microsoft Windows NT #GP trap handler allows users to switch kernel stack. (n.d.). Retrieved February 10, 2011, from http://lists.grok.org.uk/pipermail/full-disclosure/2010-January/072549.html.

更多资源请访问稀酷客(www.ckook.com)

Introduction—OAuthn. (n.d.). OAuth Community Site. Retrieved February 12, 2011, from http://oauth.net/about/.

Johnston, S. J. (n.d.). Microsoft warns about 17-year-old windows bug. Retrieved February 10, 2011, from http://www.esecurityplanet.com/features/article.php/3860131/article.htm.

Microsoft security advisory (979682) vulnerability in Windows kernel could allow elevation of privilege. (n.d.). Retrieved February 10, 2011, from http://www.microsoft.com/technet/security/advisory/979682.mspx.

Ralf-Philipp Weinmann. (n.d.). CryptoLUX. Retrieved February 10, 2011, from https://cryptolux.org/Ralf-Philipp_Weinmann.

Schneier on security: data at rest vs. data in motion. (n.d.). Retrieved February 9, 2011, from http://www.schneier.com/blog/archives/2010/06/data_at_rest_vs.html.

Secure programming for Linux and Unix HOWTO. (n.d.). David A. Wheeler's personal home page. Retrieved February 10, 2011, from http://www.dwheeler.com/secure-programs/Secure-Programs-HOWTO/open-source-security.html.

Ten, T. T. (n.d.). Field of dreams (1989)—Memorable quotes. The Internet movie database (IMDb). Retrieved February 10, 2011, from http://www.imdb.com/title/tt0097351/quotes?qt0314964.

US-CERT Vulnerability Note VU#836068. (n.d.). CERT Knowledgebase. Retrieved February 13, 2011, from http://www.kb.cert.org/vuls/id/836068.

US-CERT Vulnerability Note VU#120541. (n.d.). CERT Knowledgebase. Retrieved February 13, 2011, from http://www.kb.cert.org/vuls/id/120541.

WordNet search—3.0. (n.d.). Retrieved February 10, 2011, from wordnetweb.princeton.edu/perl/webwn?s=art.

Zoller, T. (2009, November 29). TLS & SSLv3 renegotiation vulnerability explained. SANS Internet Storm Center; Cooperative Network Security Community—Internet Security. Retrieved February 13, 2011, from http://isc.sans.edu/diary.html?storyid=7582.

更多资源请访问稀酷客(www.ckook.com)

第 6 章

Android 取证技术

本章要点

- 操作 Android 终端设备的程序
- Android USB 大容量存储终端设备映像
- 逻辑技术
- 物理技术

6.1 绪论

在具体讨论 Android 取证技术之前，首先需要考虑一些会影响取证分析师使用何种技术的因素。在本节中，我们将讨论不同类型的取证调查、逻辑技术与物理技术的区别以及如何避免或者限制对终端设备进行修改。

6.1.1 取证调查的类型

从 Android 取证调查中所获得的收益量非常大。对于这些情况，虽然所运用的取证技术具有共性，但具体到每一种取证技术时，还是需要使用不同的程序、参考资料以及有不同的侧重点。

第一种情况是，人们通常认为调查是判别刑事和民事诉讼案件的一个重要部分。在这些情况下，需要考虑以下几个重要因素：

- 接受监管的次序；
- 详细的同期说明以及最后的报告；
- 通过不同的工具或调查者对结果进行验证；
- 基于证词的事实或意见。

另外一种常见的情况是公司内部进行的调查。这些调查也许最终会导致法律诉讼，但是通常是为了判定某些问题的根源（是否由系统本身造成，或由外部攻击造成，还是由内部员工造成），最终导致公司采取一些针对员工的惩戒措施。公司的内部调查可能涉及很多领域，通常包含以下情况：

- 偷窃公司的知识产权或数据；
- 对公司的资源使用不当；

更多资源请访问稀酷客(www.ckook.com)

- 对公司系统发起的未遂或成功的攻击；
- 与公司员工相关的调查，如歧视、性骚扰等；
- （随机的或有目标的）安全审计。

在与家庭相关的事项中也有取证的需求。最常见的情况包括：

- 离婚；
- 孩子的监护权；
- 遗产纠纷。

最后一个对取证调查来说具有巨大价值的领域是政府部门的安全与运行。政府通常是一个国家内的最大雇主，美国就是一个很好的实例。根据美国人口普查局 2009 年所作的年度公共就业与薪金普查（2009 Annual Survey of Public Employment and Payroll）数据，美国联邦政府所有部门雇用的员工超过 300 万人，同时各州及地方政府雇用的全职员工达到 1660 万人（Government Employment & Payroll, n.d.）。

除了与员工有关的事项，各个国家也是其他外国政府从事攻击及情报收集的潜在目标。在挫败针对国家的攻击、调查成功的攻击、反击情报收集以及为治理国家提供有价值的情报方面，取证均起着关键作用。

6.1.2　逻辑技术与物理技术的区别

Android 取证技术从本质上分为逻辑技术和物理技术两类。逻辑技术对所分配的数据进行挖掘，通常通过访问文件系统的手段来实现。所分配的数据是指那些没有被删除的数据，可以通过访问文件系统获得。这个定义的一个例外是存储诸如 SQLite 数据库的某些文件，虽然也属于所分配这一类型，但是其中仍然包含一些被删除的数据库记录。虽然需要使用一些特殊的工具与技术来恢复这些已删除的数据，但是仍极有可能从所获取的逻辑数据中恢复这些已删除的数据。

另一方面，物理技术直接将存储数据的物理介质作为目标，而不是依赖于文件系统来访问这些数据。物理技术有其优点，最重要的就是可以访问到大量被删除的数据。正如在第 4 章中所讨论的，文件系统通常会将数据简单地标识为已被删除或已过时，而不是真正地将数据从存储介质上实际擦除，除非真正需要这样去做。由于物理取证技术对存储介质直接进行访问，就使恢复被分配的数据以及未被分配（被删除或已过时）的数据成为可能。

当然，使用 Android 物理数据获取技术通常会困难很多，也十分耗时。而且，具体运用物理技术也存在很多困难，任何使用不当都可能会造成终端设备无法继续访问。

在 Android 取证中，与传统的逻辑技术相比较，最常用的逻辑技术并不是直接访问文件系统，而是在一个更加抽象以及效果更差的层面上进行，通常能够直接从文件系统中获得所有未被删除的数据。这种依赖于内容提供商内置在 Android 平台或软件开发套件（SDK）中的技术，对于产生某些重要的取证数据非常有效，但是却只占系统所能够提供数据的一小部分。

更多资源请访问稀酷客(www.ckook.com)

6.1.3 修改目标终端设备

对于任何取证调查来说，其中一个指导原则是避免对被分析的目标终端设备造成任何修改或改变。在很多情况下，这一点是可以做到的。例如，对于得到的一台没有开机的笔记本电脑，并被告知此笔记本电脑是从一个嫌疑犯处获得，需要对其进行取证调查。在此情况下进行此项工作比较容易，因为在监管之下的笔记本电脑中的数据不会发生任何改变。一个典型的调查会包括记录计算机的全部信息、移出硬盘并将其连接到一个物理的写阻止器，然后再获得硬盘中可以用于取证的所有磁盘映像。取证调查就针对这些获得的取证映像展开，终端设备本身不会再有任何改变。

随着计算机技术和功能的不断增强，理想的情况变得越来越困难。首先，如果假设你被召唤到一个取证调查的现场，那里有一台计算机，但是这台计算机正在运行中。任何与此计算机进行的互动，不论是简单地移动此计算机，或者是拔掉电源线，都会在某种程度上对这台计算机造成改变。虽然很多取证分析师提倡简单地拔掉电源线，但是将计算机断电仍然会对计算机产生影响，因为计算机内随机访问内存（RAM）中的内容、网络连接以及其他很多东西（所有这些可能都对取证调查非常有价值）都会永久地失去。

如果对运行中的终端设备进行检查，所有的交互都会对终端设备造成改变。再考虑调查中更加复杂的情况，很有可能此台计算机使用了加密技术，因此，对于正在运行的计算机，则可能获得被加密保护的数据。但是，如果此台计算机被断电，同时又没有加密所使用的密钥，则会永久地丧失恢复计算机内数据的能力。

另外一个复杂的因素是服务器中存在的特殊硬件、复杂的设置，同时这些服务器还不能简单地断电，因为会对其他系统和人员造成重大的影响。这些复杂设置的实例包括非常复杂的 RAID 设置、依赖基于网络的存储局域网（SAN）的设置以及无法支持的硬件。在这些情况下，取证分析师必须在终端设备运行状态下直接与其进行交互，即使这些交互行为会对终端设备造成改变。

当然，对移动终端设备，尤其是对 Android 终端设备，进行取证分析时不对终端设备本身造成任何影响几乎是不可能的。与台式机电脑、笔记本电脑和服务器不同，在 Android 终端设备上的部分存储器无法轻易去除。同时，在终端设备开机的情况下，强行关机或者是取出电池都会对终端设备造成改变。

当移动电话首次出现在取证调查中时，能够从终端设备中获取的数据非常有限。很多取证调查应用的都是传统的方式，如通过法院发出的搜查令从移动服务提供商处获得通话的详细记录。另外一种可能就是摘取 GSM 终端设备中的 SIM（subscriber identity module）卡，获得卡上的数据。随着电话中存储的数据越来越多，对于如何获取取证数据产生了巨大的分歧，一方提倡传统的方式（对终端设备造成很小的影响，但是只能获取非常少量的数据），另一方则提倡对终端设备进行全面利用和开发。由于这些全面利用和开发的技术肯定会对终端设备造成改变，因此引起持续不断的争论。

截至 2011 年，移动终端设备所存储的大量数据需要使用更加带有侵入性的技术才能获得，以上这些争论基本上得到平息。英国警察总监协会（The Association of Chief

更多资源请访问稀酷客(www.ckook.com)

Police Officers in the United Kingdom）提出的一个指南对这个问题做了非常清楚的阐述。这个指南（Good Practice Guide for Computer-Based Electronic Evidence，针对基于计算机的电子证据的实践指南）（ACPO Good Practice Guide, n.d.）对于基于计算机的电子证据提出了以下 4 个原则：

1）执法部门以及具体执行人员的任何行为都不能改变存储在计算机内或存储介质上的数据，这些数据可能在后续阶段为法院所使用。

2）在任何人认为有必要访问存储在计算机内或存储介质上数据的情况下，此人必须能够胜任此项工作，而且能够清楚地提出证据，用于解释其行为的关联性以及涵义。

3）对所有基于计算机的电子证据的过程都要创建审计记录，并予以保留。任何一个独立的第三方都要能够对这些过程进行检查，并能够取得同样的结果。

4）负责调查的人（即调查官员）对确保法律以及以上这些原则得到遵循负有全部责任。

移动终端设备明显属于需要对终端设备本身进行直接访问这种情况，也应该是允许的，只要取证分析师接受过足够的培训、能够对所采取的方法给出充足的理由，并且保存一份很清楚的审计记录，使第三方能够重复检查师所采取的所有步骤。这无疑是一个很好的建议，能够对针对移动终端设备所展开的取证调查打下一个坚实的基础。

6.2　操作 Android 终端设备的程序

取证分析师所面临的一个主要挑战就是在直接获得终端设备前设计一个妥善处理终端设备的计划。对取证分析师来说，这本身不是一个新出现的问题，因为在调查中还会涉及诸如计算机和笔记本电脑之类的其他电子设备。然而，相对来说，移动终端设备第一个响应者常常无法做到正确地予以处理，而是对终端设备立即着手进行检查，这样不可避免地会造成对数据的修改以及可能失去对终端设备进行访问的能力。

6.2.1　终端设备的安全保护

针对很多部门以及第一个响应者都已经建立了保护数据的规则。接下来内容的目的是对现有的程序进行补充，而不是进行替代。当然，以下所讨论的内容代表着特殊的程序，以及对第一个响应者进行指导，这些参与者同时还有着其他很多非常具有挑战性的责任。

口令程序

个人和企业安全防范意识的提高使应用口令锁住终端设备的技术日益普及。在下一节中，我们会讨论一些如何绕开口令的具体技术。但是，这样做并不是总是可行。在从终端设备获取数据时，第一个需要考虑的因素为是否有机会可以立即禁止使用口令或者绕过口令。

如果你遇到的 Android 终端设备的触摸屏处于运行状态，应该马上考虑检查并改变终端设备的设置。对于使用口令的终端设备，在重新输入口令之前，只有在一个非常短暂的时间（从不足 1 分钟到 15 分钟）内对终端设备进行访问是可行的。如果终端设备处于此状态，需要考虑的步骤如下：

更多资源请访问稀酷客(www.ckook.com)

1）提高触摸屏的超时设置，以预防或延缓触摸屏被锁住。改变这一设置对于不同版本的 Android 以及不同的终端设备没有一致的设计。例如，在一个运行 Android 1.5 的 G1 上，超时设置通过触摸 Menu（菜单）（在主屏幕上），然后选择 Settings（设置）→ Sound & display（声音和显示）→ Screen timeout（屏幕超时），然后再选择 Never timeout（从不超时）。在一个运行 Android 2.3 的 HTC Incredible 上，触摸 Menu（在主屏幕上），然后选择 Settings（设置）→ Security（安全）→ Lock phone after（锁定手机后），最后再选择“15 minutes”（15 分钟）。只要终端设备在超时所设置的时间内执行任何操作，就能够被访问；

2）开启 USB 程序调试及 Stay awake（保持唤醒）设置。在终端设备上，这个设置的位置一直保持一致，可以通过触摸 Menu（在主屏幕上），然后是 Settings（设置）→ Applications and Development（应用和开发）。此后，就可以选择 USB debugging and Stay awake（USB 调试和保持唤醒）。如果在选择 Stay awake（保持唤醒）设置后，将其终端设备接上电源，终端设备永远不会进入休眠状态，这将有效防止屏幕被锁住。通过开启 USB 程序调试，对终端设备的访问可以通过 USB 接口进行，实现数据获取。

当然，以上这些步骤会对终端设备造成改变，因此必须在案例说明中详细记录，包括描述终端设备的状态、进行改变的理由以及每一项改变的后果。这样做不但能够帮助接下来要做的报告撰写，而且一旦当初所做对终端设备造成改变的决定在法庭上受到质疑时，这些记录还极有可能成为一个很重要的考虑因素。

如果要给所做的事情增加难度，那么重要的一点就是要在屏幕激活的状态下尽量减少接触触摸屏。正如我们马上将要讨论的那样，有时是有可能通过放大终端设备触摸屏上图像的方式确定终端设备的图案锁。

6.2.2 网络隔离

正如很多取证分析师可能已经知道的那样，尽快将终端设备与网络实现隔离非常重要。在最坏的情况下，可以通过网络向终端设备发出一个远程清除命令，而且一旦成功，将防止对任何数据进行恢复。虽然大多数的远程清除都是通过数据网络来完成，有时也可以通过短信的方式实现，因此，要确保终端设备与所有的网络都实现隔离，才能彻底防止远程清除。在其他状况下，终端设备可能会继续收到更多的消息，甚至在不受控制的情况下被删除。由于取证映像的目的是为了进行更多的分析而保存终端设备的状态，应该避免对终端设备的任何改变。

目前已经有数种将终端设备与网络实现隔离的方法，每一种方法都有各自的优缺点。表 6.1 总结了这些方法所具有的优缺点。

可以看到，将终端设备与网络进行物理隔离也不是一件很容易的事情，每一个实现物理隔离的方式都有它的优缺点。采取何种方式和步骤应该由取证分析师个人或者是由他们所属的机构来决定，最好的选择应该是将终端设备置于飞行状态。虽然做到这一点的方法会由于 Android 终端设备的不同以及 Android 版本的不同存在微小的差别，然而总体方法基本相同：

更多资源请访问稀酷客(www.ckook.com)

表 6.1　终端设备隔离技术

技　术	优　点	缺　点
将终端设备置于飞行状态。此方法要求具有对 Settings 菜单进行访问的权限	终端设备继续运行，与时间有关的数据不受任何影响。中断与蜂窝网络和 Wi-Fi 网络的连接	改变了终端设备的设置。只有在对终端设备具有所有的访问权限时才可行
如果终端设备是一台 GSM 手机，将 SIM 卡取出	很容易做到，能够有效地禁止所有的无线通话、短信及数据通信	无法禁止通过 Wi-Fi 以及其他网络进行通信。对非 GSM 手机无效，包括 CDMA 或 iDEN 手机
暂停无线服务提供商处的账号	能够有效地禁止所有的语音通话、短信以及数据通信服务，对任何手机均有效	整个过程需要花费一定的时间，需要得到法院的许可。无法禁止 Wi-Fi 及其他的网络通信
将终端设备放置于一个被屏蔽的袋子、盒子、帐篷或房间内	法拉第屏蔽会防止各种形式的网络通信，在无法使用上面讨论的方法的情况下，可以作为一个有效的方法使用	对携带式法拉第屏蔽物的有效性，特别是法拉第手提袋的有效性存在一些争议。另外，虽然数据传输被阻断，但由于终端设备会一直试图与无线蜂窝网络进行连接，因此会很快耗尽电池。因为会传输信号，电源线不能置于包裹物中。专门用于进行取证检查的被屏蔽的房间是最理想的方式，但是建造和维护却非常昂贵
关闭终端设备	对防止任何网络通信都完全有效	终端设备的状态被改变，与时间有关的数据丢失。可以使用口令和重启限制设置对终端设备的访问

1）按住 Power off（关机）按钮，并选择 Airplane（飞行）模式；

2）在主屏幕上，选择 Menu（菜单），然后选择 Settings（设置），然后再选择 Wireless（无线）选项，这些按钮通常处于触摸屏的顶端区域内。可能看到的包括 Wireless controls（无线控制）或者是 Wireless and networks（无线与网络）。在下一屏的菜单中就应该会呈现出 Airplane（飞行）模式。

图 6.1 展示的是使用关机方法的截屏，图 6.2 展示的是使用无线设置方法的选项。

图 6.1　通过关机按钮设置飞行模式

图 6.2　通过无线与网络菜单设置飞行模式

更多资源请访问稀酷客(www.ckook.com)

不论最终决定使用何种方法，主要目的都是要在最短的时间内实现终端设备与网络的物理隔离。

1. 电源与数据连接线

虽然大多数取证分析实验室都有对终端设备进行充电的电源线以及与终端设备进行连接的连接线，但是直接缴获现场的电源线和数据线依然是最明智的做法。极有可能发生的情况是获取了一台更新款式的终端设备，而取证工具箱里还没有相应的连接线。例如，一个新制定出的称为便携式数字媒体接口（portable digital media interface，PDMI）的规范已经被集成进 Dell Streak 和 Samsung Galaxy Tab 这两款 Android 平板终端设备中。此便携式数字媒体接口（portable digital media interface，PDMI）不但提供电源和高清晰度视频输出，而且支持 USB 3.0。在这种情况下，设法去获得相应的连接线就会延误对终端设备的检查，如果在此期间终端设备需要充电，失去电源将意味着丢失与时间相关联的数据。

2. 已关机的终端设备

如果在得到终端设备时，终端设备已经处于关机状态，最好的方式则是启动终端设备并进入修复模式，以测试连接性以及 root 访问权限。终端设备的拥有者可能已经开启了 USB 程序调试或者已经将终端设备设置为具有 root 访问权限，这时就没有必要开启终端设备并进入正常运行模式，而是可以直接访问终端设备中的数据。

这种方法与面向一台标准的计算机硬盘做取证分析十分类似。任何一位有经验的取证分析师都不应该去开启计算机，以确定所安装的操作系统。取而代之的是将硬盘取出，并连接到一个写阻止器上进行映像，以防止对数据造成任何改变。同样，如果一台终端设备不需要被开启并进入正常工作状态，就没有必要去开机，因为这样可能对终端设备造成改变。关于如何在修复模式中对终端设备进行测试，以决定是否具有足够的访问权限的具体步骤将在本章的后续章节中进行讨论。

6.2.3 如何绕过口令

具有在 Android 终端设备上绕过口令的能力变得越来越重要，这是因为绕过口令的方法被使用得越来越频繁，而在大多数情况下，这并不允许进行数据抽取。虽然没有任何万无一失的方法，但的确存在很多在某些情况下十分有效的技术。

正如前面所讨论过的，Android 终端设备目前主要支持 3 类口令。第一类叫做图案锁，为最初的 Android 终端设备上的默认方式。要想访问终端设备，用户在被锁住的手机上绘制一个图案，如果图案正确，则手机的锁被解开。图 6.3 是在一台 HTC Incredible 上使用图案锁的实例。

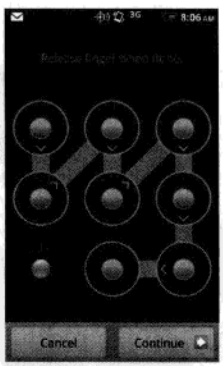


图 6.3 Android 图案锁定

第二类口令就是使用简单的个人识别号（PIN），也常使用于其他的移动终端设备上。图 6.4 是一个 PINenabled HTC Incredible 的实例。

最后一类目前使用在 Android 终端设备上的口令就是使用完整的数字字符码，如图 6.5 所示。

更多资源请访问稀酷客(www.ckook.com)

图 6.4　Android 个人识别号锁定

图 6.5　Android 字母数字键盘锁定

正如在第 4 章中所讨论的那样，并不是所有口令的效果都是等同的。最有效的口令应该是允许或要求使用由数字和字母组合构成的密码，因为这样的口令很难破解。

1. 在 USB 程序调试开启状态下使用 ADB

如果手机目前处于开机状态，第一个应该尝试使用的技术是通过 USB 将手机连接至 Android 程序调试桥（Android Debug Bridge，ADB），这已经在第 3 章中进行了深入讨论。虽然只有部分 Android 终端设备会允许通过设置 USB 程序调试而实现 ADB 连接，但是这肯定是一个值得尝试的方法，因为如果成功将可以访问到足够的数据。以下是用户开启 USB 程序调试最可能的原因：

- 应用软件开发与测试；
- 某些应用软件要求此设置，如 PDAnet，此设置允许终端设备通过 USB 接口提供到另外一台受限（tethered device）终端设备的互联网访问；
- 定制的 ROM；
- 开发者使用的手机，如谷歌的 Android 开发者手机（Google's Android Developer Phone，ADP1）；
- 终端设备破解。

如果使用 Ubuntu 虚拟机或者一台正确安装并配置了软件开发套件的取证工作站，则确定 USB 程序调试是否已开启非常简单。如果手机处于正常运行的状态下，将其插入 Ubuntu 虚拟机，在看到命令提示符时，输入 adb devices 命令。如果 USB 程序调试已经开启，ADB 后台进程（daemon）会返回一个序列号以及目前手机所处的运行模式。

```
ahoog@ubuntu:~$ adb devices
List of devices attached
HT08XHJ00657    device
```

如果 USB 程序调试没有开启，在输入 adb devices 命令后，ADB 后台进程不会返回任何信息。

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ adb devices
List of devices attached

ahoog@ubuntu:~$
```

如果在一个虚拟工作站上运行上述命令，需要注意的是要将终端设备提交给虚拟机。如果虚拟机无法看到终端设备，在任何情况下，都总会得到与 USB 程序调试没有开启情况同样的结果。在确认 USB 连接已经提交给 Ubuntu 虚拟机后，可以通过执行 lsusb 命令来验证操作系统已经知晓此连接的存在。

```
ahoog@ubuntu:~$ sudo lsusb -v
[sudo] password for ahoog:

Bus 001 Device 005: ID 0bb4:0c9e High Tech Computer Corp.
Device Descriptor:
  bLength                18
  bDescriptorType         1
  bcdUSB               2.00
  bDeviceClass            0 (Defined at Interface level)
  bDeviceSubClass         0
  bDeviceProtocol         0
  bMaxPacketSize0        64
  idVendor           0x0bb4 High Tech Computer Corp.
  idProduct          0x0c9e
  bcdDevice            2.26
  iManufacturer           1 HTC
  iProduct                2 Android Phone
  iSerial                 3 HT08XHJ00657
  bNumConfigurations      1
<snip>
    Interface Descriptor:
      bLength                 9
      bDescriptorType         4
      bInterfaceNumber        1
      bAlternateSetting       0
      bNumEndpoints           2
      bInterfaceClass       255 Vendor Specific Class
      bInterfaceSubClass     66
      bInterfaceProtocol      1
      iInterface              4 ADB
<snip>
```

在以上这个实例中，有好几个地方都清楚地显示出 Android 终端设备已经顺利连接，以及在这些情况下，可以看到一个凸现出的 ADB 接口。如果终端设备是连接上的，但是连接无法通过 ADB 实现，则应该停止运行本地的 ADB 后台进程，并重新启动此 ADB 后台进程。按照以下步骤，这一点也很容易做到：

```
ahoog@ubuntu:~$ adb kill-server
ahoog@ubuntu:~$ adb devices
* daemon not running. starting it now on port 5037 *
* daemon started successfully *
List of devices attached
HT08XHJ00657    device
```

如果 USB 程序调试已经开启，取证分析师就可以使用这个接口对终端设备进行访问，

更多资源请访问稀酷客(www.ckook.com)

完成对终端设备上数据的逻辑恢复。具体细节将在本章的后续章节中进行讨论。

2. 污点攻击

Android 终端设备最初使用图案锁作为口令保护方式，而不是使用数字或者是更完善的数字与字母组成的密码。在最近发表的一篇由美国宾夕法尼亚大学计算机与信息科学系撰写的论文《Smudge Attacks on Smartphone Touch Screens》中，作者提出了一个通过增强屏幕图案的方式访问使用图案锁住的 Android 终端设备的技术（Aviv, Gibson, Mossop, Blaze, Smith, n.d.）。论文在总结中做了以下陈述：

> 我们的图像实验表明，一个十分干净的触摸屏基本上，但并不完全，能够反射，同时，任何污点基本上，但并不是完全，能够传播。我们发现，基本上任何定向光源，只要不是与照相机处于一个准确的互补角度的位置上，都会返回一个对污点能够修复的映像。基本上不需要对影像做任何调整就可以看到图案，但是最佳的映像通常来自于对相片进行两至三次的过度曝光（4～8 次的“正确曝光”）。

如果第一个响应者处理得十分仔细，尽量避免接触终端设备的触摸屏，这个修复技术也许是可行的。作为可能情况的一个实例，图 6.6 展示的是针对同样一 个 Android 终端设备的不同图像的对比。两个图像都是基于同样一幅照片，然而右边的图像显示的是作为污点攻击（Smudge Attack）过程的一部分而经过增强处理后的结果，以突出触摸点。

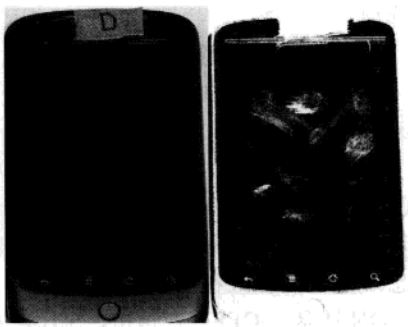


图 6.6　展示污点攻击的增强型图像

3. 修复模式

某些用户会安装定制的只读内存（ROM），通常就会通过一个修改过的修复模式对终端设备开启 root 访问权限。大多数定制的只读内存都会安装一个修改过的修复分区，目的是简化用于安装定制的只读内存的整个过程。目前有几个比较流行的修复分区，主要与定制的只读内存配合使用，它们在修复终端内都对 shell 会话过程访问提供 root 级别的访问权限。由于手机不是启动并运行在正常模式中，可以绕开口令，也可以以只读方式装入用户数据的分区，因此可以避免改变任何用户数据。

如果终端设备处于关闭状态，取证分析师在取得对终端设备的控制后，应当尽量将终端设备开启并运行在修复模式下。然而，如果终端设备处于正常运行状态，同时口令也已获得，取证分析师首先应该试图通过 ADB 进行连接并充分考虑污点攻击。如果以上两条均不成立，则应该试图重启终端设备，并将其运行在修复模式下。像其他很多技术一样，根据终端设备制造商的不同以及所生产的模型的不同，进入修复模式也需要采取不同的方法。表 6.2 罗列出在本书中所引用的手机上进入修复模式的按键组合。对于每一种组合，假设终端设备已经处于关机状态。

更多资源请访问稀酷客(www.ckook.com)

表 6.2 启动终端设备时进入修复模式的按键组合

终端设备	按键组合
HTC G1	按住主按键，再按开机按键。使用音量按键向下移动鼠标去选择 RECOVERY（修复），再按开机按键
谷歌 Nexus One	按住音量按键，再按开机按键
摩托罗拉 Droid	按住 X 键，再按开机按键
HTC Incredible	按住音量按键，再按开机键。使用音量按键向下移动鼠标去选择 RECOVERY（修复），再按开机按键

在进入修复模式后，就可以将终端设备与 Ubuntu 工作站实现连接，并尽量使用 ADB 进行连接。如果终端设备运行在一个没有被修改过的修复模式下，此连接将会失败。在这种情况下，触摸屏上通常会显示出一个三角形图案，三角形图案中会有一个感叹号，感叹号的一侧还会显示出一个很小的 Android 终端设备图案。在其他的终端设备上，则会显示出一个具有一定知名度的滑板上方有三个 Android 字样的图案。最后，其他的修复模块也会清楚地显示出使用的是经过修改后的修复模式代码，并会提供大量不同的终端设备选项。

4. 闪建新的修复分区

对于有经验的取证分析师来说，目前已经存在很多协议、应用程序以及终端设备为修改后的映像在终端设备上闪建一个分区。

第一个可以用于支持此需求的方法是快速启动（fastboot）。快速启动是一个 NAND 闪存更新协议，通过 USB 在启动加载程序模式下运行。大多数终端设备在出厂时都开启了启动加载程序保护，禁止使用快速启动协议。但是，此保护也有可能已经被禁止。要想确定启动加载程序保护是否开启，必须访问启动加载程序，查看签字信息中标明的是 S-ON 还是 S-OFF。在这里，S 代表安全，因此，S-ON 代表开启了安全保护，也是出厂时的默认值；S-OFF 则代表安全没有被开启。有些终端设备出厂时的设置为 S-OFF，如谷歌的 Nexus One，也是因为此终端设备中预装了谷歌的 Engineering SPL/Bootloader。还有其他的基于超级用户权限（root）的技术可以禁止这个保护，因此，在使用口令进行保护的终端设备上检查此保护状态能够得到相应的结果。对启动加载程序进行访问可以使用表 6.2 中按键组合的第一部分。

快速启动不需要 USB 程序调试就可以访问终端设备。因此，像修复模式一样，它可以用于对终端设备中的数据进行访问。一旦这个新的修复分区建立起来，就可以重新启动终端设备，进入修复模式，然后就会发生建立取证映像。

还有其他的技术可以用于为修复分区闪建一个新的映像。以下是一些这方面的例子：

- 摩托罗拉的 RSD Lite
- sbf_flash
- 三星的 Odin Multiloader

虽然以上这些应用程序和协议最终能够给取证分析师所需要的访问权限，但还是需要

更多资源请访问稀酷客(www.ckook.com)

大量的努力才能够找到并测试这些技术，并且需要对它们有深入透彻的理解才能在取证调查中真正使用。

5. 触摸屏锁规避应用程序

最近，安全研究者 Thomas Cannon 提出了一项能够规避触摸屏锁的技术，实现的方法就是从新的基于 Web 的 Android 市场网站上下载一个应用程序（Cannon, T., n.d.）。此方法利用了基于 Web 的 Android 市场中一个新的功能，此功能允许从 Android 市场网站上下载应用程序，并直接进行安装。为了做到这一点，用户必须使用终端设备上的 Gmail 用户名和密码访问 Android 市场网站，才可以从用户的个人计算机上去实现。另外一个方法是使用用户名和密码访问 Android 市场网站，并拥有足够的访问权限。在后一种情况下，不允许修改用户的 Gmail 密码。

Cannon 在网站上对此技术进行了解释，其原理在下一节中进行介绍（Cannon, T., n.d.）

6. 工作原理

过程的确非常简单。Android 通过无线方式对外广播很多消息，如收到的短信、Wi-Fi 连接中断等，任何应用均可以收到这些消息。应用程序只需要注册接收器就可以收到这些广播消息，注册可以在运行状态下完成，对于接收某些消息，也可以在安装时完成。当一个相关的消息产生后，此消息将会被发送给注册过的应用程序，而如果一个注册的应用程序没有处于运行状态，此应用程序会被自动启动。

在对各类广播消息进行测试后，作者认为针对这一应用的最好的一个消息便是 _android.intent.action.PACKAGE_ADDED。这个消息存在于所有的作为第一版的应用编程接口（API）中，当安装一个应用程序时便会产生。因此，为了达到远程执行一个应用程序的目的，首先将其应用程序通过 Android 市场网站部署到终端设备上，然后再通过 Android 市场网站部署另外一款应用程序的方式去启动前一个应用程序。

一旦启动并开始执行，主要的问题就变成要在 KeyguardManager 中调用 disable-Keyguard()。这是一个合法的应用编程接口（API），允许应用程序在检测到一个未能成功连接的通话时禁止触摸屏锁。当通话结束时，应用程序应该重新开启触摸屏锁，但是却可以设计成继续禁止触摸屏锁。

如果能够正常地访问 Android 市场，此技术的确值得考虑。

7. 使用 Gmail 用户名 / 密码

在大多数的 Android 手机上，如果知道在终端设备上注册的 Gmail 用户名和密码，就可以绕过口令。在登录失败数次后（在 G1 上是 10 次），终端设备上将显示出一个屏幕，询问是否忘记了口令。此时，可以输入 Gmail 用户名和密码，用户将被要求重置口令。此技术不需要手机在与网络连接时才能够实现，因为以上的操作和判断完全使用缓存在手机上的关键用户信息。

如果不知道当前的 Gmail 用户名和密码，但是却具有足够的权力（如从法院获得的命令）重置密码，可以尝试强迫谷歌对账号的密码进行重置。此时，必须将 Android 终端设备与网络实现连接，以获得对终端设备的访问。这样做会遇到很多挑战，包括将终端设备

更多资源请访问稀酷客(www.ckook.com)

与网络相连，使终端设备面临被远程清除的危险，还包括对终端设备造成改变。各式各样执法电子邮件列表中的报告表明，此技术有时不奏效。

如果尝试以上技术，则需要进行更多的研究。特别要指出的是，必须要非常仔细地控制终端设备与互联网的连接，很有可能是通过 Wi-Fi 实现的。然后，要对通过此连接对终端设备的访问进行严格限制，只允许传输与谷歌服务器进行认证所需要的信息。另外，必须对此终端设备的详细网络访问记录以及对终端设备造成的改变进行认证分析。

8. 联合测试行动小组（JTAG）与芯片摘取（chip-off）

目前，大多数 Android 终端设备对 NAND 闪存上的内容不做任何加密保护，导致在口令开启的情况下，直接对存储芯片进行访问并解码成为一个潜在的应变方案。目前主要有两种直接访问存储芯片的技术。每一种技术都很具有挑战性。以下是这两种技术：

- 联合测试行动小组（Joint test action group，JTAG）；
- 物理获取（芯片摘取，chip-off）。

每一种技术都很具有挑战性，需要对终端设备进行部分或全部解体，而且在获取数据后需要进行大量的分析，以重新构建文件系统。由于这些原因，JTAG 和芯片摘取技术很有可能是征服被锁住的终端设备不得已的最后一项选择。

应用 JTAG 时，直接与终端设备中的中央处理器（CPU）相连接，需要在印刷电路板（PCB）上与特定的 JTAG 接线头进行连接。之后就可以使用 JTAG 软件转储出整个 NAND 闪存中的全部二进制数据内容，对某些分区进行修改以达到允许 root 访问的目的，或者是将整个口令予以清除。

在芯片摘取的程序中，使用加热与通风法，将 NAND 闪存芯片从印刷电路板上予以摘取。这个芯片通常是在一个很小的球状格矩阵（BGA）包装内，需要将 BGA 的连接头插入到特殊硬件中，将这个特殊硬件与芯片相连接，并读取 NAND 闪存中的内容。

以上技术的优点是它们适用于任何情况，只要 NAND 闪存的内容没有受到加密保护。然而，实际使用这些技术要求进行大量的研究、开发、测试以及拥有丰富的经验。

6.3 Android USB 大容量存储终端设备映像

目前所有的 Android 终端设备都配备有外接的安全数字（SD）卡，或者是内置的多媒体卡（eMMC），给用户提供所需要的大容量存储空间。需要这些存储终端设备的理由是基于安全与隐私保护的需要，通常存储在 /data/data 中的用户应用数据都是被隔离开的。然而，用户希望在 Android 终端设备和个人电脑之间来回拷贝音乐、图像、视频或者其他各类文件，这些大容量的 FAT 文件系统分区可以解决这些问题。用户的敏感数据依然能够得到保护，同时，用户又能够访问大尺寸、便携式的文件。

最初，对外接存储器进行映像的方式就是简单地将这些存储器脱离 Android 终端设备，然后使用一个 USB 写阻止器进行映像。但是，随着时间的推移，存在以下问题或挑战：

- 使用 eMMC 存储就意味着大容量存储不再便携。

更多资源请访问稀酷客(www.ckook.com)

- 应用软件可以直接从安全数字（SD）卡上运行，在此情况下，.apk 文件是加密的。如果获取解密版的应用软件对取证调查非常重要（例如，涉及对恶意软件进行分析的情况或对特洛伊木马进行抵御），SD 卡必须始终连接在 Android 终端设备上。
- 新的终端设备越来越多地使用随机访问内存（RAM）保存用户数据，这样做对取证调查也许会有帮助。通常要求将终端设备关机，并将电池取出后才能拔出 SD 卡，因此会失去修复与时间相关联数据的能力。

由于以上的原因，建议对 Android 上的 USB 大容量存储设备（UMS）进行映像的方式不再是将 SD 卡拔出，取而代之的是通过 UMS 接口进行映像。

6.3.1　SD 卡与 eMMC

SD 卡与 eMMC 没有太大的区别。主要的区别当然还是在于 SD 卡是便携式的，可以很容易地从一台终端设备上转移到另外一台终端设备上。它们都使用 NAND 闪存技术，都是基于多媒体卡（MultiMediaCard，MMC）技术规范，嵌入有存储控制器，因此，任何外部操作系统都不需要依赖于系统中的 MTD 就可以对它们进行读取。

至今，Android 所接受的微 SD 卡通常为 2 ～ 16GB。然而，更大容量的卡也是可能的。取决于不同款的 Android 终端设备，SD 卡也许能够很容易地从正在运行的终端设备上接触到并取出。但是，很多终端设备还是要求将终端设备关机，以便能够取出电池。

对于嵌入在终端设备内部的存储装置，有几个厂商已经开始使用 eMMC，在其构成中有一个嵌入式的存储器以及一个与终端设备的印刷电路板直接进行集成的多媒体卡（MMC）接口。这个标准通过使用标准化的 eMMC 协议简化了对 NAND 闪存进行访问，也有能力支持与 NAND 闪存不兼容的文件系统。这并不完全表明这些文件系统对 NAND 闪存寿命的保持程度以及成熟性与诸如 YAFFS2 这样完全支持 NAND 闪存对其寿命的保持程度以及成熟性处于同一水平。但是，Android 终端设备的一般使用年限肯定在缩短，对大多数用户来说并不构成一个问题。

6.3.2　如何获得 SD 卡或 eMMC 的取证映像

在不需要从终端设备中取出任何卡的前提下，目前从 SD 卡或 eMMC 中获取取证映像的方法主要有两种。在本节中讨论的第一个方法中，将 UMS 终端设备接口暴露给取证工作站，然后使用任何可选择的取证工具获取映像。在第二种方法中，不将 UMS 暴露给取证工作站使用，而是在终端设备上使用 dd 命令。做到这一点要求 adb 端口转发，我们将在本章的后序章节中讨论物理技术时进行详细介绍。

虽然我们的 Ubuntu 虚拟机中已经装有 dd，但是还是建议下载、编译并安装由（美国）国防部的网络犯罪中心（Cyber Crime Center）维护的一个 dd 更新版本。这个叫做 dc3dd 的软件是对 GNU dd 软件的升级版本，包含了很多对计算机取证非常有用的功能（dc3dd, n.d.），部分如下：

- 面向成对或者全部的散列：支持 MD5、SHA-1、SHA-256 及 SHA-512。

更多资源请访问稀酷客(www.ckook.com)

- 自动针对输入 / 输出文件大小的进度指示表。
- 将散列与错误结合在一起的记录。
- 错误分组：对前后发生的同样错误，产生一个相同的错误消息。
- 验证模式：能够对输出文件进行散列运算，并与所得到的散列值进行比较。
- 能够将输出进行分块，并对文件名赋予数字或字母数字后缀。
- 能够同时写入多个输出文件。

这个程序是一个开源的软件，在 GNU 公共许可版本 3（GPLv3）范围内，通过 SourceForge 在网上发布，并且在 2010 年 8 月更新为版本 7.0。在撰写本书的时候，应该可以很顺利地对此程序进行编译，并安装所需要的工具，以下是简化过的步骤。

```
mkdir -p ~/src
cd ~/src
curl http://cdnetworks-us-2.dl.sourceforge.net/project/dc3dd/
dc3dd/7.0.0/dc3dd-7.0.0.tar.gz
> dc3dd-7.0.0.tar.gz
tar xzf dc3dd-7.0.0.tar.gz
cd dc3dd-7.0.0/
./configure
make
sudo make install
```

安装之后就可以进行映像。然而，每一次敲入整个 dc3dd 命令不但很烦琐，而且还容易造成打字错误，以造成不可逆转的损坏。因此，可以为此创建一个 shell 脚本程序，其中所做的工作不但要获取终端设备，而且还要记录各种系统参数、日期 / 时间戳，并且创建审计文件，以帮助随后撰写报告。

我们将所创建的 shell 脚本程序放置在 /usr/local/bin 目录中，这样就可以在任何目录中都很容易地运行此 shell 脚本程序，这是因为 /usr/local/bin 是系统中默认的程序执行路径：

```
ahoog@ubuntu:~$ sudo nano -w /usr/local/bin/acquire-disk.sh
```

下一步，将以下部分复制到此 shell 脚本程序中，通过按 Ctrl+O 组合键进行保存，再按 Ctrl+X 组合键退出：

```
#!/bin/bash

CLIENT="${1}"
CASE="${2}"
TAG="${3}"
SERIALNO="${4}"
SOURCEDEV="${5}"
DESTPATH="${6}"

OUTPUTPATH=$DESTPATH/$CLIENT/$CASE/$TAG-$SERIALNO
LOGFILE=$OUTPUTPATH/log/$TAG-$SERIALNO.log
STDERRLOG=$OUTPUTPATH/log/$TAG-$SERIALNO.stderr.log
SEPERATOR="-----------------------------------------\r"

if [ "$#" != 6 ]; then
        echo "Usage: acquire_disk.sh CLIENT CASE TAG SERIALNO SOURCEDEV
DESTPATH"
        exit 2
```

更多资源请访问稀酷客(www.ckook.com)

```
fi

# check directories, created if needed
if [ ! -d "$DESTPATH" ]; then
        echo "Destination path [$DESTPATH] does not exist, exiting"
        exit 1
fi

if [ -d "$DESTPATH/$CLIENT/$CASE/$TAG-$SERIALNO" ]; then
        echo "$DESTPATH/$CLIENT/$CASE/$TAG-$SERIALNO already exists, can't
overwrite evidence"
        exit 1
fi
GOTROOT=`whoami`

if [  "$GOTROOT" != "root" ]; then
       echo "must be root to execute"
       exit 1
fi

mkdir -p $OUTPUTPATH/log

echo -e "Start date/time" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`/bin/date`\n" >> $LOGFILE 2>> $STDERRLOG

echo -e "uname -a" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`uname -a`\n" >> $LOGFILE 2>> $STDERRLOG

echo -e "dmesg | tail -50" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`dmesg | tail -50`\n" >> $LOGFILE 2>> $STDERRLOG

echo -e "lshw" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`lshw`\n" >> $LOGFILE 2>> $STDERRLOG

VERSION=`fdisk -v`
echo -e "fdisk -l $SOURCEDEV [$VERSION]" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`fdisk -l $SOURCEDEV`\n" >> $LOGFILE 2>> $STDERRLOG

VERSION=`mmls -V`
echo -e "mmls $SOURCEDEV [$VERSION]" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`mmls $SOURCEDEV`\n" >> $LOGFILE 2>> $STDERRLOG

VERSION=`fsstat -V`
echo -e "fsstat $SOURCEDEV [$VERSION]" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`fsstat $SOURCEDEV`\n" >> $LOGFILE 2>> $STDERRLOG

VERSION=`dc3dd --version 2>&1 | grep dc3dd`
echo -e "dc3dd [$VERSION]" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "dc3dd if=$SOURCEDEV of=$OUTPUTPATH/$TAG-$SERIALNO.dc3dd verb=on
hash=sha256 hlog=$OUTPUTPATH/log/$TAG-$SERIALNO.hashlog
log=$OUTPUTPATH/log/$TAG-$SERIALNO.log rec=off\n" >> $LOGFILE
dc3dd if=$SOURCEDEV of=$OUTPUTPATH/$TAG-$SERIALNO.dc3dd verb=on hash=sha256
```

更多资源请访问稀酷客(www.ckook.com)

```
hlog=$OUTPUTPATH/log/$TAG-$SERIALNO.hashlog log=$OUTPUTPATH/log/
$TAG-$SERIALNO.log rec=off

echo -e "ls -lR $DESTPATH/$CLIENT/$CASE/$TAG-$SERIALNO" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`ls -lR $DESTPATH/$CLIENT/$CASE/$TAG-$SERIALNO`\n" >> $LOGFILE

echo -e "End date/time" >> $LOGFILE
echo -e "$SEPERATOR" >> $LOGFILE
echo -e "`/bin/date`\n" >> $LOGFILE

#sha256sum all log files
cd $OUTPUTPATH/log/
sha256sum * > $TAG-$SERIALNO.sha256.log
```

下一步，必须修改访问权限，才能够执行此 shell 脚本程序以及不使用任何参数执行此 shell 脚本程序，以查看相关的使用帮助信息：

```
ahoog@ubuntu:~$ sudo chmod 755 /usr/local/bin/acquire-disk.sh
ahoog@ubuntu:~$ sudo /usr/local/bin/acquire-disk.sh
Usage: acquire_disk.sh CLIENT CASE TAG SERIALNO SOURCEDEV DESTPATH
```

这个 shell 脚本程序的好处，以及开源的整体好处，在于能够按照实际需要进行修改。如果不想对用户的姓名进行跟踪，只需要简单地从此 shell 脚本程序中删除相关部分的代码。

下一步，需要将 UMS 终端设备挂载到 Ubuntu 工作站上。在第 1 章中讨论 Ubuntu 虚拟机设置时提到，在工作站上禁止自动挂载非常关键。如果这一点还没有做到，请回顾一下相应的步骤并完成，然后将 UMS 终端设备呈现给虚拟机。

此外，理想的情况是首先将 Android 终端设备连接到一个基于硬件的 USB 写阻止器上。然而，如果被连接的终端设备显示出具有多于一个的终端设备识别号（ID），对于某些写阻止器来说可能会有问题。因此首先需要试验 USB 写阻止器，以确认一切都工作正常。

> **提示：Tableau UltraBlock USB**
>
> 自从 2009 年 8 月 9 日以来就运行最新固件的 Tableau UltraBlock USB 模型 T8 在我们所参考的 HTC Incredible 上只能作为第一个 USB 终端设备，因此，我们无法将它作为终端设备进行分析。Tableau 有一款新的 UltraBlock USB 终端设备也许可以使用，但是，我们还没有对此进行验证。取证分析师应该对所使用的各种 USB 写阻止器进行兼容性测试。

下一步，需要确定 UMS 映射到哪些终端设备。此信息显示在内核的记录中，可以通过用 dmesg 命令很轻易地获得：

```
ahoog@ubuntu:~/$ dmesg
<snip>
[327202.720222] usb 1-1: new high speed USB device using ehci_hcd and address 12
[327203.032759] scsi11 : usb-storage 1-1:1.0
[327204.039549] scsi 11:0:0:0: Direct-Access     HTC      Android Phone    0100
PQ: 0 ANSI: 2
[327204.044572] scsi 11:0:0:1: Direct-Access     HTC      Android Phone    0100
```

更多资源请访问稀酷客(www.ckook.com)

```
PQ: 0 ANSI: 2
[327204.047208] scsi 11:0:0:2: CD-ROM            HTC      Android Phone    0100
PQ: 0 ANSI: 2
[327204.049854] sd 11:0:0:0: Attached scsi generic sg2 type 0
[327204.052640] sd 11:0:0:1: Attached scsi generic sg3 type 0
[327204.066738] sr1: scsi3-mmc drive: 0x/0x caddy
[327204.066817] sr 11:0:0:2: Attached scsi CD-ROM sr1
[327204.066892] sr 11:0:0:2: Attached scsi generic sg4 type 5
[327204.082001] sd 11:0:0:0: [sdb] Attached SCSI removable disk
[327204.091070] sd 11:0:0:1: [sdc] Attached SCSI removable disk
```

正如以上所讨论的，除了 ADB，HTC Incredible 暴露出 3 个 USB 接口：

- 用于安装终端设备驱动程序的 CD-ROM（sr1）；
- eMMC UMS 终端设备（sdb）；
- SD 卡 UMS 终端设备（sdc）。

但是，/dev/sdb 与 /dev/sdc 之间的区别不是很明显，在终端设备上必须要等到 UMS 或 Disk 驱动功能都开启以后才能确定。一旦开启，就可以再一次查看 dmesg 的输出。

警示：使用基于软件的写阻止器

虽然在 Ubuntu 工作站上已经禁止了自动挂载功能，非常关键的一点是取证分析师必须确认 Android 终端设备只能通过一个基于硬件的写阻止器与工作站实现连接，以确保对终端设备不会造成任何改变。所有的硬件在真正用于取证分析之前都应该进行严格、全面的测试。

```
ahoog@ubuntu:~/$ dmesg
<snip>
[327520.269248] sd 11:0:0:1: [sdc] 3911680 512-byte logical blocks:
(2.00 GB/1.86 GiB)
[327520.298549] sd 11:0:0:1: [sdc] Assuming drive cache: write through
[327520.304747] sd 11:0:0:1: [sdc] Assuming drive cache: write through
[327520.304757]  sdc: sdc1
[327522.267959] sd 11:0:0:0: [sdb] 13844464 512-byte logical blocks:
(7.08 GB/6.60 GiB)
[327522.271097] sd 11:0:0:0: [sdb] Assuming drive cache: write through
[327522.277187] sd 11:0:0:0: [sdb] Assuming drive cache: write through
[327522.277202]  sdb:
```

现在很清楚，/dev/sdb 是一个 7GB 的存储设备（就是这个 eMMC），而 2GB 的 SD 卡被映射到 /dev/sdc。现在就可以使用上面的 shell 脚本程序或者任何取证工作站上能够使用的映像工具来获得终端设备中的数据。这个 shell 脚本程序有以下 6 个参数：

1）client（客户端）：此参数帮助创建目录结构，实例包括“sheriffs-office”或者是一个客户名称，如“viaforensics”。

2）case（事件）：此参数提供一个事件名称，如 af-book。

3）tag（标记）：此参数是对一个证据进行取证映像所赋予的标记号，在我们的实例中为 item001。

4）serialno（序列号）：此参数赋予终端设备、硬盘、SD 卡等一个序列号。如果无法得到序列号，则可以输入任何文字，如 unknown-serialno。

更多资源请访问稀酷客(www.ckook.com)

5）sourcedev（源终端设备）：此参数指明需要获取数据的终端设备，如 /dev/sdb、/dev/sdc 等。可以使用 dmesg 命令来确定，后面内容会进行解释。

6）destpath（目的路径）：此参数指出用于存放目录的顶层目录，可以为主目录（~），也可以是一个叫做 clients 的目录（~/clients）。

在此例中，在主目录中创建一个叫做 sd-emmc 的目录，然后在拥有 sudo 访问权限的条件下执行用于获取的 shell 脚本程序。

```
ahoog@ubuntu:~$ sudo acquire-disk.sh viaforensics af-book item001
unknown-serialno /dev/sdc ~/sd-emmc
Cannot determine file system type

dc3dd 7.0.0 started at 2011-02-22 04:36:05 -0600
compiled options:
command line: dc3dd if=/dev/sdc of=/home/ahoog/sd-emmc/viaforensics/
af-book/item001-unknown-serialno/item001-unknown-serialno.dc3dd
verb=on hash=sha256 hlog=/home/ahoog/sd-emmc/viaforensics/af-book/
item001-unknown-serialno/log/item001-unknown-serialno.hashlog
log=/home/ahoog/sd-emmc/viaforensics/af-book/item001-unknown-serialno/
log/item001-unknown-serialno.log rec=off
device size: 3911680 sectors (probed)
sector size: 512 bytes (probed)
2002780160 bytes (1.9 G) copied (100%), 808.727 s, 2.4 M/s

input results for device '/dev/sdc':
   3911680 sectors in
   0 bad sectors replaced by zeros
   fc8f3d6dc7e659c3124a4113d2d0ebe87466b497038aedf9f7a1b89c44eda8b9 (sha256)

output results for file '/home/ahoog/sd-emmc/viaforensics/af-book/
item001-unknown-serialno/item001-unknown-serialno.dc3dd':
   3911680 sectors out

dc3dd completed at 2011-02-22 04:49:34 -0600
```

然后就可以使用同样的通用命令对 eMMC 进行映像，但是需要改变参数，对于我们的终端设备就是 /dev/sdb。在这些命令执行结束后，取证映像以及记录文件都在 ~/sd-emmc 目录下，其目录结构如下：

```
hoog@ubuntu:~$ tree -h sd-emmc/
sd-emmc/
└── [4.0K]  viaforensics
    └── [4.0K]  af-book
        ├── [4.0K]  item001-emmc-unknown-serialno
        │   ├── [6.6G]  item001-emmc-unknown-serialno.dc3dd
        │   └── [4.0K]  log
        │       ├── [ 686]  item001-emmc-unknown-serialno.hashlog
        │       ├── [ 39K]  item001-emmc-unknown-serialno.log
        │       ├── [ 374]  item001-emmc-unknown-serialno.sha256.log
        │       └── [   0]  item001-emmc-unknown-serialno.stderr.log
        └── [4.0K]  item001-sd-unknown-serialno
            ├── [1.9G]  item001-sd-unknown-serialno.dc3dd
            └── [4.0K]  log
                ├── [ 726]  item001-sd-unknown-serialno.hashlog
                ├── [ 39K]  item001-sd-unknown-serialno.log
                ├── [ 296]  item001-sd-unknown-serialno.sha256.log
                └── [   0]  item001-sd-unknown-serialno.stderr.log

6 directories, 10 files
```

更多资源请访问稀酷客(www.ckook.com)

对于每一个取证映像过的 UMS 终端设备，我们不但有经过验证的映像，而且还有一个 dd 映像的散列记录、一个具有日期、时间、系统信息和运行的命令的记录文件、一个错误记录，最后还有一个所有记录文件的清单以及每一个文件的 SHA256 散列值。这样就可以确保提供关于映像过程足够的细节。

> **提示：SD 卡上加密的应用程序**
>
> 如果一个应用程序安装在 SD 卡上，同时受加密保护，然后程序中的文件在 SD 卡的映像中被检查，则无法读取这些文件。但是，如果在 SD 卡还没有被挂载到取证工作站时，未加密的 .apk 文件被挂载到 /mnt/asec。如果取证调查基于应用程序的 .apk 文件展开，则确保也要得到一份未加密的文件。

6.4　逻辑技术

在本章的开始讨论到，逻辑取证技术基于抽取已经分配的数据。达到此目的通常通过访问文件系统来实现。逻辑技术通常是取证分析师首先使用的技术，这是因为这一类技术不但容易使用，而且通常可以为案件提供足够的数据。Android 取证的物理技术可以提供更多的数据。但是，这一类技术更难成功运用，也需要付出更多的努力进行分析。

逻辑技术具有普适性的优点，这是因为使用此技术的唯一要求就是开启 USB 程序调试功能。换句话来说，Android 取证逻辑技术不需要 root 访问权限。

在本节中，我们首先讨论可以免费使用的技术（虽然 AFLogical 技术只对执法机构和政府机关免费），然后对可用的商业软件进行概述。

6.4.1　adb pull

在第 4 章中，我们反复展示了几次递归的 adb pull 命令，用于将文件系统的不同部分复制到 Ubuntu 工作站中进行进一步分析。除非对 Android 终端设备具有 root 访问权限，或者运行的是一个定制的只读内存（ROM），否则，作为递归复制版本的代理，运行在终端设备上的 adb 后台进程只能在拥有 shell 访问权限的状态下运行，因此，它无法访问某些与取证更加相关的文件。但还是可以访问到一定数量的文件。

如果需要对某些 shell 用户没有访问权限的文件进行访问，很简单，这些文件不会被复制出来：

```
ahoog@ubuntu:~$ adb pull /data adbpull
pull: building file list...
0 files pulled. 0 files skipped.
```

但是，如果拥有足够的访问权限（在下一个例子中就是 root），这个方法就非常简单，也非常有效：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ adb pull /data adbpull/
pull: building file list...
<snip>
pull: /data/miscrild_nitz_long_name_31026 -> data/misc/rild_nitz_long_name_31026
pull: /data/misc/akmd_set.txt -> data/misc/akmd_set.txt

712 files pulled. 0 files skipped.
963 KB/s (208943249 bytes in 211.671s)
```

从上面的例子可以看出，整个 /data 分区在 3.5 分钟的时间内都被复制到一个本地目录中。在复制过程中，目录结构依然被保留，因此随后就可以在工作站上直接浏览并对感兴趣的文件进行分析。

由于大多数的手机都没有 root 访问权限（至少在默认情况下），此技术显得没有什么价值。然而，这是一个非常强大的、需要很好理解的工具，也有一些场景非常适合应用此技术。这些场景包括：

- 对于没有 root 访问权限的终端设备，adb pull 仍然可以用来访问非常有用的文件（如未加密的应用程序）、大多数可能包含诸如浏览记录这一类用户数据的 tmpfs 文件系统以及存在于 /proc、/sys 及其他可读目录下的系统信息。
- 对于具有 root 访问权限的终端设备，获取几乎所有的目录非常简单，某些存在于 /data 下的文件和目录将是有用的。
- 当使用物理技术时，挂载诸如 YAFFS2 之类用于取证的文件系统不一定总是可行的。如果 adbd 运行在 root 访问权限下，则可以使用 adb pull 快速地抽取文件系统的一个逻辑副本。

由于 adb 不但在 Android 软件开发套件（SDK）中是一款免费的应用程序，而且它的用途还非常广泛，因此，adb 应该作为使用的终端设备上的主要逻辑工具之一。

> **警示：adb pull 的问题**
>
> 某些使用 adb 的递归拉取可能在数据传输过程中失败，可能由于访问权限问题或者其他问题。因此，对于执行命令的结果要进行密切监控，以确定是否遇到任何问题。将对一个大目录的递归拉取分解为若干个小的数据拉取也许会产生更好的结果。

6.4.2 备份分析

在 Android 第一次被推出时，Android 没有提供任何备份个人数据的机制。后果就是，很多人开发出备份应用软件，并在 Android 市场上进行发布。对于使用定制的只读内存（ROM）的用户，甚至开发出了一款更加强大的叫做 nandriod 的备份应用软件。

很多这些备份软件都有一个“Save to SD Card”（保存在 SD 卡上）的选项（用户认为此功能非常方便），还有几个保存在“the cloud”（云端）的选项。无论采用何种方式，用户需要亲自对他们的终端设备进行备份，在需要时又将备份的数据进行修复。这不但是

更多资源请访问稀酷客(www.ckook.com)

一个用户保护数据不受损失的方法，而且还是一个取证分析师可以获取大量数据的来源。

其中一个很流行的备份应用程序是 RerWare 公司的 My Backup Pro，用于将终端设备上的数据在内容服务提供商处进行备份，如果具有 root 访问权限，还可以将整个 /data/data 中的文件进行备份。用户可以选择是将数据保存在 SD 卡上，还是保存在 RerWare 的服务器上。此款应用软件可以支持如下功能（RerWare, LLC, n.d.）：

- 应用程序安装文件（如果手机具有 root 访问权限，将包括应用、数据以及市场链接）；
- 通讯录；
- 通话记录；
- 浏览器收藏夹；
- 短消息服务（文字消息）；
- 多媒体服务（消息中的附件）；
- 系统设置；
- 主屏幕（包括 HTC Sense UI）；
- 闹钟；
- 字典；
- 日历；
- 音乐播放列表；
- 集成的第三方应用软件。

最后一点“集成的第三方应用软件”主要是指为数据备份提供与 RerWare 挂钩的公司。至少在最初，RerWare 对将 RerWare 备份支持包含在应用软件中的开发者支付一定的费用。

很有趣的是，这一款应用软件不但可以在 Android 上运行，而且还可以在 Windows Mobile、黑莓上运行，并且很快还将包括 Symbian 操作系统。用户可以将数据备份在一个平台上，然后在另外一个完全不同的平台上进行恢复。如果将终端设备的备份保存在本地，RerWare 会在 SD 卡上保存一个 SQLite 文件。

在最近推出的 Android 版本中，都有一个新的备份应用编程接口（API）可以使用。开发者可以简单地将此应用编程接口集成到他们的应用软件中去，备份所需要的其他工作就完全由 Android 和谷歌来处理。这样就给用户提供了安全、基于云的备份服务，并且对于所有的应用程序都具有一致性，也极有可能成为备份的实际应用标准。不幸的是，目前的研究还没有从安装在 Android 终端设备中这个新的备份应用编程接口中发现其他更新有用方面。

不论是否有备份应用软件，取证分析师都必须确定是否安装了任何备份应用软件，而如果有的话，备份数据保存在何处。与其他的终端设备如个人计算机或笔记本电脑一样，SD 卡也需要接受检查。很明显，保存在备份中的数据对于取证检查来说具有巨大的价值。

6.4.3　AFLogical

AFLogical 是 Android 取证中的一个逻辑技术，免费地提供给执法机构和政府机关使

更多资源请访问稀酷客(www.ckook.com)

用。此应用软件由 viaForensics 公司开发，使用内容提供商（Content Providers）抽取数据，这是 Android 平台的一个核心功能。它与使用在商业取证工具的技术一样，用于进行逻辑取证。

在此提醒的一点是，除了极少的情况，Android 的安全模型用于限制对应用数据进行访问非常有效。以下是对 Android 安全模型关键组件的一个快速小结：

- 每一个应用都被赋予一个唯一的 Linux 用户名和组名；
- 应用程序使用具体的用户名，运行在一个专门的进程和 Dalvik 虚拟机中；
- 每一个应用程序都分配有专门的存储空间，通常在 /data/data 中，只能由本应用程序进行访问。

但是，这个 Android 架构还是提供了应用程序可以进行数据共享的机制。一个应用软件开发者可以把对内容提供商的支持包含在开发的应用程序中，这样就能够允许这些应用程序与其他的应用程序实现数据共享。开发者决定哪些数据可以暴露给其他的应用程序。在安装应用软件的过程中，用户可以控制是否允许应用软件访问所要求的内容提供商。

内容提供商的一些实例包括：

- 短信服务 / 多媒服务；
- 通讯录；
- 日历；
- Facebook；
- Gmail。

当然还有更多的内容服务。

AFLogical 充分利用内容提供商架构提供的优势，对保存在终端设备上的数据进行访问。与商业版的 Android 逻辑工具相同，只有在终端设备必须开启 USB 程序调试的条件下，AFLogical 才能进行数据抽取。当前的版本，即 1.5.1，从 41 个内容提供商处抽取数据，使用 CSV 格式以及 info.xml 文件的方式，将输出信息提供给 SD 卡，其中包含终端设备的详细信息以及已经安装的应用软件。AFLogical 支持运行 Android 1.5 及更新版本的终端设备，也已经明确地更新到能够支持抽取大量的数据，如抽取一个拥有超过 3.5 万条短消息的短消息服务数据库。目前所支持的内容提供商包括：

1）浏览器收藏夹；
2）浏览器搜索；
3）日历；
4）日历参与者；
5）日历事件；
6）日历扩展功能；
7）日历提醒；
8）通话记录通话；
9）通讯录联络方式；
10）通讯录扩展；

更多资源请访问稀酷客(www.ckook.com)

11）通讯录组；
12）通讯录机构；
13）通讯录手机号；
14）通讯录设置；
15）外部媒体；
16）外部映像媒体；
17）外部映像图标媒体；
18）外部视频；
19）即时消息账号；
20）即时消息所有账号；
21）即时消息交谈；
22）即时消息通讯录提供商（即时消息通讯录）；
23）即时消息邀请；
24）即时消息信息；
25）即时消息提供商；
26）即时消息提供商设置；
27）内部映像媒体；
28）内部映像图标媒体；
29）内部视频；
30）地图 – 朋友；
31）地图 – 朋友其他功能；
32）地图 – 朋友通迅录；
33）多媒体服务；
34）多媒体服务部分提供商（MMSParts）；
35）便笺；
36）人物；
37）删除的人物；
38）手机存储（HTC Incredible）；
39）搜索记录；
40）短消息服务；
41）社交联络活动。

现在让我们在终端设备上一步一步地运行 AFLogical。首先，确认已经下载了 AFLogical，这需要与 via-Forensics 进行登记并获得批准。（可以通过 http://viaforensics.com/products/tools/aflogical/ 链接访问 AFLogical。）下一步是用自己能够控制的 SD 卡替换用户的 SD 卡，并确认在终端设备上已经开启 USB 程序调试。然后，将 Android 终端设备连接到 Ubuntu 工作站，并确认是通过 USB 连接到虚拟机上。

更多资源请访问稀酷客(www.ckook.com)

警示：替换用户的 SD 卡

目前版本的 AFLogical 将内容直接写入到 SD 卡上，因此，将用户的 SD 卡取出，而用检查师的 SD 卡替代非常重要。没有这样做的话，数据会直接写入用户的 SD 卡，或者是由于无法写入用户的 SD 卡而导致 AFLogical 的执行失败。此款软件的商业版最终将取代直接写入 SD 卡，而更加倾向于使用通过 adb 的端口转发方式。

从一个终端会话窗口，确认能够看到终端设备：

```
$adb devices
List of devices attached
0403555511112222F           device
```

假设 AFLogical 应用程序被保存在主目录中，那么就可以使用以下命令进行安装：

```
ahoog@ubuntu:~$ adb install ~/AndroidForensics.apk
523 KB/s (31558 bytes in 0.058s)
        pkg: /data/local/tmp/AndroidForensics.apk
Success
```

值得注意的是，如果 AFLogical 已经安装在终端设备的话，就会出现一个错误信息，那么就必须卸载现有的应用程序，然后再安装这个新版本。如需卸载，执行以下命令：

```
ahoog@ubuntu:~$ adb install /opt/via/AFLogical/AndroidForensics.apk
824 KB/s (31558 bytes in 0.037s)
        pkg: /data/local/tmp/AndroidForensics.apk
Failure [INSTALL_FAILED_ALREADY_EXISTS]

ahoog@ubuntu:~$ adb uninstall com.viaforensics.android
Success
```

在应用程序成功安装后，可以直接通过 Android 终端设备或者是通过命令行来执行此程序。如果通过命令行来执行此程序，则可以简单地开始应用程序的运行，然后使用终端设备结束运行，或者是让它自动运行。如果自动运行并进行数据抽取，则执行以下命令：

```
ahoog@ubuntu:~$ adb shell am start -n
com.viaforensics.android/com.viaforensics.android.ExtractAllData
Starting: Intent { cmp=com.viaforensics.android/.ExtractAllData }
```

此程序马上开始运行，并开始从所有支持的内容提供商处抓取数据。从屏幕上看，则可以看到类似于图 6.7 中所显示的图像。

或者是简单地使用以下命令开始应用程序的运行：

```
ahoog@ubuntu:~$ adb shell am start -n
com.viaforensics.android/com.viaforensics.android.ForensicsActivity
Starting: Intent { cmp=com.viaforensics.android/.ForensicsActivity }
```

然后，使用显示在图 6.8 中的终端设备屏幕完成数据抓取。

更多资源请访问稀酷客(www.ckook.com)

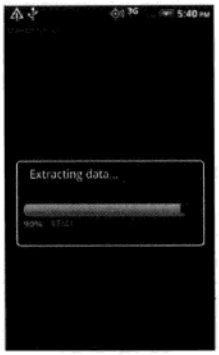


图 6.7　AFLogical，从命令行抽取全部数据

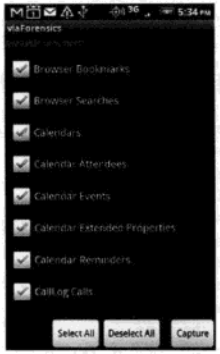


图 6.8　AFLogical，从命令行执行

否则，从所显示的 All apps（所有应用）屏幕上直接运行 AFLogical，如图 6.9 所示。首先，访问 Android 应用程序（Android app）菜单，找到一个叫做 viaForensics 的程序，然后点击相应图标，就开始执行此应用程序。

之后就将看到 AFLogical 数据抽取屏幕。可以在此屏幕上选择或者不选择每一个出现的内容提供商，或者是选择所有的内容提供商。然后，点击 Capture（抓取）将开始数据收集过程，如图 6.10 所示。

当数据收集完成后，将收到图 6.11 中所显示的相应消息。

抽取到的数据都保存在终端设备的 SD 卡上，在 forensics 目录下（文件夹中）一个使用日期命名的子目录下，子目录的名称格式为 YYYYMMDD.HHMM。在此例中，我们使用 adb pull 将文件从 SD 卡上转移到一个在本地文件系统中的 AFLogical 目录下。去查看此文件夹将看到：

图 6.9　在 All apps 列表中的 AFLogical

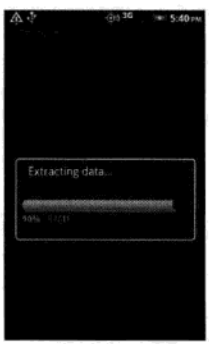


图 6.10　AFLogical 抓取数据

图 6.11　AFLogical，数据抽取完成

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ ls AFLogical/
20110221.1708
```

这个目录下包含的将是抽取到的数据：

```
ahoog@ubuntu:~$ ls AFLogical/20110221.1708/
Browser Bookmarks.csv            IM Providers.csv
Browser Searches.csv             IM ProviderSettings.csv
CallLog Calls.csv                info.xml
Contacts ContactMethods.csv      Internal Image Media.csv
Contacts Extensions.csv          Internal Image Thumb Media.csv
Contacts Groups.csv              Internal Videos.csv
Contacts Organizations.csv       Maps-Friends contacts.csv
Contacts Phones.csv              Maps-Friends.csv
Contacts Settings.csv            Maps-Friends extra .csv
External Image Media.csv         MMS.csv
External Image Thumb Media.csv   MMSParts.csv
External Media.csv               People.csv
External Videos.csv              PhoneStorage (HTC Incredible).csv
IM Account.csv                   sanitize.sh
IM Accounts.csv                  Search History.csv
IM Chats.csv                     SMS.csv
IM Contacts.csv                  Social Contracts Activities.csv
IM Invitations.csv
```

可以使用任何编辑器或电子表格处理软件查看 CSV 文件。在此目录下还有一个叫做 info.xml 的文件，此文件内包含终端设备的信息，包括 IMSI、IMEI、Android 版本、网络提供商等，还有一个已经安装的应用软件列表。

```
ahoog@ubuntu:~$ less info.xml
<android-forensics>
<date-time>20110221.1708</date-time>
<IMSI>removed</IMSI>
<IMEI>removed</IMEI>
<build>
        <version.release>2.2</version.release>
        <version.sdk>8</version.sdk>
        <version.incremental>264707</version.incremental>
        <board>inc</board>
        <brand>verizon_wwe</brand>
        <device>inc</device>
        <display>FRF91</display>
        <fingerprint>verizon_wwe/inc/inc/inc:2.2/FRF91/264707:user/release-
keys</fingerprint>
        <host>HPA003</host>
        <id>FRF91</id>
        <model>ADR6300</model>
        <product>inc</product>
        <tags>release-keys</tags>
        <time>1285855309000</time>
        <type>user</type>
        <user>root</user>
</build>
<applications>
        <app>
                <label>Network Location</label>
                <className>null</className>
                <dataDir>/data/data/com.google.android.location</dataDir>
                <descriptionRes>0</descriptionRes>
```

更多资源请访问稀酷客(www.ckook.com)

```
                <flags>48709</flags>
                <manageSpaceActivityName>null</manageSpaceActivityName>
                <name>null</name>
                <packageName>com.google.android.location</packageName>
                <permission>null</permission>
                <processName>system</processName>
                <publicSourceDir>/system/app/NetworkLocation.apk
                </publicSourceDir>
                <sourceDir>/system/app/NetworkLocation.apk</sourceDir>
                <taskAffinity>com.google.android.location</taskAffinity>
                <uid>1000</uid>
                <enabled>true</enabled>
                <description>null</description>
                <packageinfo>                    <versionCode>8</versionCode>
                        <versionName>2.2</versionName>
                </packageinfo>  </app>
        <app>
                <label>IMDb</label>
                <className>null</className>
                <dataDir>/data/data/com.imdb.mobile</dataDir>
<snip>
```

取证分析师现在就可以对这些数据进行分析，也可以与他人分享。

警示：卸载 AFLogical

不要忘记卸载 AFLogical。没有成功卸载此应用程序的后果是当 Android 终端设备返还给拥有者时，取证代理依然能够被访问。如要卸载 AFLogical，使用以下的命令：

adb uninstall com.viaforensics.android

执行此命令将返回成功提示。另外一种方式是在主屏幕上单击 Menu（菜单），然后选择 Applications → Manage Applications → viaForensics，最后选择 Uninstall（卸载）。

6.4.4　供应商

Android 现在已经得到很多商业的移动取证软件商家的支持。到目前为止，取证软件只能支持对 Android 终端设备进行逻辑检查，使用的是与 AFLogical 相同的内容提供商技术。对取证检查师来说，理解每一个取证软件商家如何实现对 Android 的支持，这对工作的开展很有帮助。

每一个软件公司都会提供一份他们所销售软件的试用版本以及对相关平台的概述，这些都包括在本节的开始。以一个运行 Android 2.2 版本的摩托罗拉 Droid 来介绍。本节的目的不是要详细介绍每一个平台，而是提供一个有帮助性的总体概括。以下的介绍包括下列取证软件包（按照字母排列先后顺序）：

- Cellebrite 公司的 UFED；
- Compelson 公司的 MOBILedit!；
- EnCase 公司的 Neutrino；

更多资源请访问稀酷客(www.ckook.com)

- Micro Systemation 公司的 XRY；
- Paraben 公司的 Device Seizure；
- viaForensics 公司的 viaExtract。

对另外两款取证软件包也进行过测试。然而，测试遇到了一些问题，导致这两款取证软件包最终没有被列入以上的列表。这些开发商的确为 Android 提供了取证方案，如果感兴趣的话，可以单独对他们的产品进行评估。这两款被排除在外的软件包是 Oxygen Forensic Suite 2010 和 Logicube 公司的 CellDEK。

对这些软件包进行评估中遇到的挑战是取证软件的频繁更新，也许已经有了新的版本。对某一款软件感兴趣的取证检查师和信息安全工程师应该查看相应商家的网站，或者是直接与商家取得联系。

1. Cellebrite 公司的 UFED

以下对 Cellebrite 的介绍资料由开发商提供。

Cellebrite UFED 取证系统是一个独立的终端设备，能够从大约 1600 款移动终端设备上获取数据，并将获取的数据存储在 USB 设备、SD 卡或者是 PC 上。UFED 内还有一个内置的 SIM 读卡器以及一个 SIM 克隆卡。对 SIM 卡进行克隆是一个非常强大的功能，可以对原版 SIM 卡进行克隆，并将克隆 SIM 卡插入手机中正常使用。然而，它无法在移动服务提供商的网络中实现注册，因此，免除了使用法拉第屏蔽包的需要以及手机中的数据被更改（或被擦除）的可能性。UFED 出厂时附带有大约 70 条电缆，用于与目前市面上大多数移动终端设备进行连接。所支持的连接协议包括串行线、USB、红外线及蓝牙。

Cellebrite 同时提供 UFED 报告管理程序（UFED Report Manager），此管理程序提供直观的报告界面，允许用户将数据 / 报告输出到 Excel、MS Outlook、Outlook Express 及 CSV，也可以直接打印报告。

UFED 设备完全支持 Unicode，因此可以支持手机上使用的任何语言。另外，UFED 能够抽取的数据类型如下：

- 通讯录；
- 文本消息；
- 通话记录（已接来电、已拨电话、未接来电）；
- SIM ID 克隆；
- SIM/USIM 卡中已经删除的文本消息；
- 语音留言；
- 视频；
- 相片；
- 通话详细情况（IMEI/ESN 电话号码）。

（1）安装

UFED 系统是一个独立的单元，包装在一个软盒子内，盒内包括 UFED 终端设备、用户使用手册、软件 CD-ROM、USB 蓝牙无线电（Cambridge Silicon Radio Ltd 生产）、

更多资源请访问稀酷客(www.ckook.com)

250MB USB 驱动器以及大约 72 条用于与所支持的终端设备进行连接的连接线。

UFED 系统为固件和软件更新升级提供了若干种方式。在设置了日期和时间后，检查师可以很容易地将 UFED 系统通过以太网（Ethernet）连接线与网络进行连接，要求是 DHCP 及 Internet 连接均正常工作。接下来，依次选择 Services（服务）→ Upgrade（更新）→ Upgrade Application Now（现在更新应用），再将 HTTP Server 选择作为源。在此次测试中，找到了最新的应用软件版本 1.1.0.5，并进行了安装。由于 UFED 系统是一个独立的解决方案，不需要安装其他任何东西。

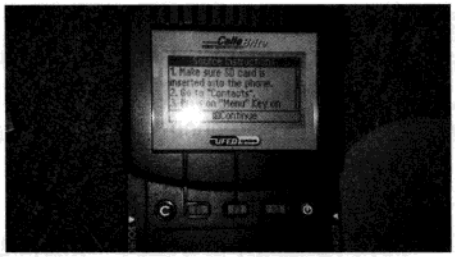


图 6.12　UFED 针对 Android 终端设备的说明

（2）数据获取

在 UFED 系统上从摩托罗拉的 Droid 中获取数据很快，也很简单。在将终端设备开机后，依次选择 Extract Phone Data（提取手机数据）→ Motorola CDMA → Moto. A855 Droid (Android) → USB disk drive (destination)，再选择所希望的内容类型（Content types）。然后，UFED 系统就会显示以下说明（见图 6.12）：

```
Moto. A855 Droid (Android):
Before starting the transaction, prepare phonebook for transfer, as follows:
1. Make sure SD card is inserted into the phone.
2. Go to "Contacts".
3. Press on "Menu" Key on the phone.
4. Select "Import/Export".
5. Select "Export to SD card" and "OK".
6. Wait for completing of Export to .vcf file.

To enable phone USB Connectivity, set Connection settings as follows:
Menu -> Settings -> Applications -> Development -> Select the checkboxes: "USB
debugging" and "Stay awake".
```

需要指出的是，如果按照以上的步骤，通讯录就会被保存在 SD 卡上。在完成了这些步骤后，点击 Continue（继续）进行数据获取。然后 UFED 系统就会提示：

```
Set USB to Mass Storage (Memory Card) mode on the SOURCE phone
```

在这一步完成后，继续进行数据获取。在数据获取完成前，系统也许会提示用户再一次设置 UMS。数据获取的整个过程大约耗时 3 分钟，然后就看到以下提示：

```
Moto. A855 Droid (Android):
Please return the Connection settings back: Menu -> Settings -> Applications ->
Development -> unmark the checkboxes: "USB debugging" & "Stay awake"
```

数据获取的结果保存在插入 UFED 的闪存驱动器上。结果是会产生一个 25MB 大小的文件夹，在此文件夹之中分别有存放视频、音频和图像的文件夹。同时还产生以下 3 个相关文件：PhoneBook 2010_11_23 (001).htm、SMSMessages 2010_11_23 (001).htm 和 Report.htm，都可以使用 Web 浏览器进行阅读。Report.htm 内是数据抽取的整个报告，内

更多资源请访问稀酷客(www.ckook.com)

容包括以下部分：手机检查报告特性（Phone Examination Report Properties）、手机检查报告索引（Phone Examination Report Index）、手机通讯录（Phone Contacts）、手机短信消息（Phone SMS Text Messages）、手机已接来电列表（Phone Incoming Calls List）、手机拨出电话列表（Phone Outgoing Calls List）、手机未接来电列表（Phone Missed Calls List）、图像（Images）、手机铃声（Ringtones）、音频（Audio）和视频（Video）。

整个数据获取过程耗费大约 10 分钟时间。在数据获取结束后，对 SD 卡的快速检查发现有一个被命名为 00001.vcf 的文件，内有来源于输出过程的联系信息。

手机信息非常清楚，也很准确。

（3）数据展现与分析

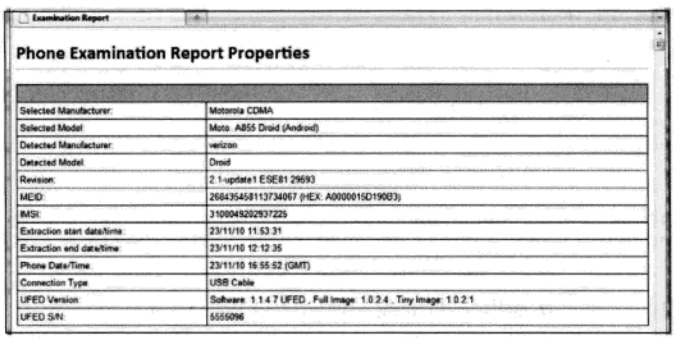
获取的数据保存在与 UFED 系统相连接的闪存驱动器上，内有一个文件夹，保存着视频、音频文件以及图像，还有以下 3 个提供报告的 HTML 文件：

1）PhoneBook 2010_11_23 (001).htm；

2）SMSMessages 2010_11_23 (001).htm；

3）Report.htm。

以上这些文件可以使用 Web 浏览器进行查看，图 6.13 是报告中的一些样本数据，从中可以看到所获取的很详细的手机信息。


Examination Report

Phone Examination Report Properties

Selected Manufacturer:	Motorola CDMA
Selected Model:	Moto. A855 Droid (Android)
Detected Manufacturer:	verizon
Detected Model:	Droid
Revision:	2.1-update1 ESE81 29693
MEID:	268435458113734067 (HEX: A0000015D190B3)
IMSI:	310004592937225
Extraction start date/time:	23/11/10 11:53:31
Extraction end date/time:	23/11/10 12:12:35
Phone Date/Time:	23/11/10 16:55:52 (GMT)
Connection Type:	USB Cable
UFED Version:	Software: 1.1.4.7 UFED , Full Image: 1.0.2.4 , Tiny Image: 1.0.2.1
UFED S/N:	5555096

图 6.13　UFED 手机信息报告

手机通讯录显示的方式如图 6.14 所示。

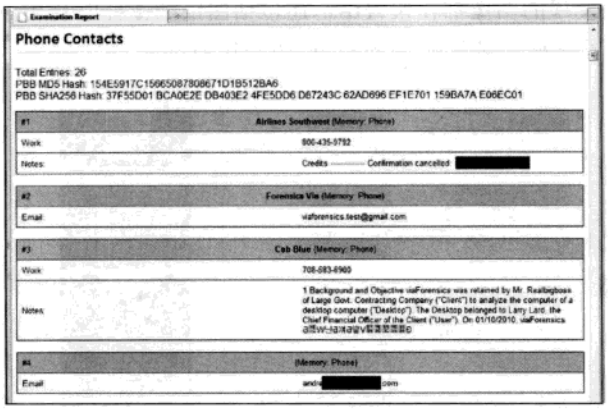
图 6.15 显示的是按照时间先后顺序排列的短信消息，包括每一条短信发送与接收的相关细节。

但是，已经被删除的短信不再被显示，被删除的多媒体消息（彩信）也不再被显示。

通话记录也按照时间先后顺序进行显示，如图 6.16 所示，其中包括每一通电话所持续的时间。这些记录被分类成为已接听（Incoming）、已拨出（Outgoing）和未接听（Missed）几部分。

通话记录中有几通电话被删除；然而，UFED 还是可以抽取并显示这几通电话的详细信息。

更多资源请访问稀酷客(www.ckook.com)



Examination Report

Phone Contacts

Total Entries: 26
PBB MD5 Hash: 154E5917C15665087808671D1B512BA6
PBB SHA256 Hash: 37F55D01 BCA0E2E DB403E2 4FE5DD6 D67243C 62AD696 EF1E701 159BA7A E06EC01

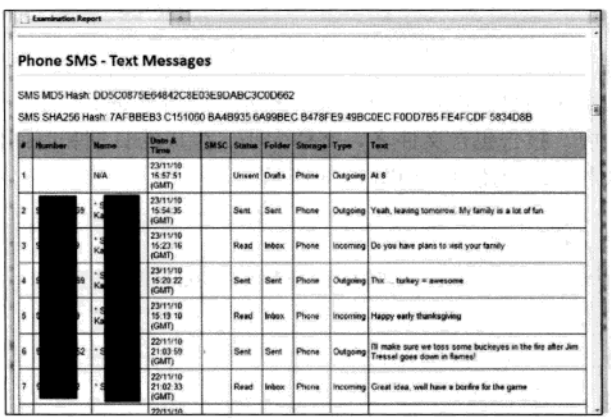
#1	Airlines Southwest (Memory: Phone)
Work:	800-435-9792
Notes:	Credits ---------- Confirmation cancelled: [redacted]

#2	Forensics Via (Memory: Phone)
Email:	viaforensics.test@gmail.com

#3	Cab Blue (Memory: Phone)
Work:	708-883-8900
Notes:	1 Background and Objective viaForensics was retained by Mr. Realbigboss of Large Govt. Contracting Company ("Client") to analyze the computer of a desktop computer ("Desktop"). The Desktop belonged to Larry Land, the Chief Financial Officer of the Client ("User"). On 01/10/2010, viaForensics [redacted]

#4	(Memory: Phone)
Email:	andr[redacted].com

图 6.14　UFED 手机通讯录报告



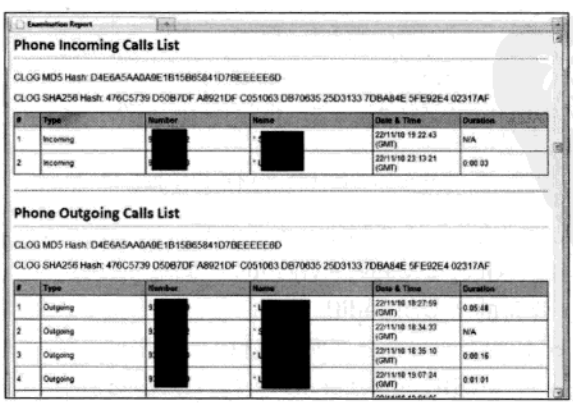
Examination Report

Phone SMS - Text Messages

SMS MD5 Hash: DD5C0875E64842C8E03E9DABC3C0D662
SMS SHA256 Hash: 7AFBBEB3 C151060 BA4B935 6A99BEC B478FE9 49BC0EC F0DD7B5 FE4FCDF 5834D8B

#	Number	Name	Date & Time	SMSC	Status	Folder	Storage	Type	Text
1		N/A	23/11/10 16:57:51 (GMT)		Unsent	Drafts	Phone	Outgoing	At 6
2	[redacted]	[redacted]	23/11/10 15:54:35 (GMT)		Sent	Sent	Phone	Outgoing	Yeah, leaving tomorrow. My family is a lot of fun
3	[redacted]	[redacted]	23/11/10 15:23:16 (GMT)		Read	Inbox	Phone	Incoming	Do you have plans to visit your family
4	[redacted]	[redacted]	23/11/10 15:20:22 (GMT)		Sent	Sent	Phone	Outgoing	Thx ... turkey = awesome
5	[redacted]	[redacted]	23/11/10 15:19:10 (GMT)		Read	Inbox	Phone	Incoming	Happy early thanksgiving
6	[redacted]	[redacted]	22/11/10 21:03:59 (GMT)		Sent	Sent	Phone	Outgoing	I'll make sure we toss some buckeyes in the fire after Jim Tressel goes down in flames!
7	[redacted]	[redacted]	22/11/10 21:02:33 (GMT)		Read	Inbox	Phone	Incoming	Great idea, well have a bonfire for the game
			22/11/10						

图 6.15　UFED 短信服务报告



Examination Report

Phone Incoming Calls List

CLOG MD5 Hash: D4E6A5AA0A9E1B15B65841D7BEEEEE6D
CLOG SHA256 Hash: 476C5739 D50B7DF A8921DF C051063 DB70635 25D3133 7DBAB4E 5FE92E4 02317AF

#	Type	Number	Name	Date & Time	Duration
1	Incoming	[redacted]	[redacted]	22/11/10 19:22:43 (GMT)	N/A
2	Incoming	[redacted]	[redacted]	22/11/10 23:13:21 (GMT)	0:00:03

Phone Outgoing Calls List

CLOG MD5 Hash: D4E6A5AA0A9E1B15B65841D7BEEEEE6D
CLOG SHA256 Hash: 476C5739 D50B7DF A8921DF C051063 DB70635 25D3133 7DBAB4E 5FE92E4 02317AF

#	Type	Number	Name	Date & Time	Duration
1	Outgoing	[redacted]	[redacted]	22/11/10 19:27:59 (GMT)	0:05:48
2	Outgoing	[redacted]	[redacted]	22/11/10 18:34:33 (GMT)	N/A
3	Outgoing	[redacted]	[redacted]	22/11/10 18:35:10 (GMT)	0:00:16
4	Outgoing	[redacted]	[redacted]	22/11/10 19:07:24 (GMT)	0:01:01

图 6.16　UFED 通话报告

更多资源请访问稀酷客(www.ckook.com)

所有在手机上发现的图像都在报告中，还包括图像的图标以及相应细节，如文件名、文件大小、生成的日期和时间以及分辨率，如图 6.17 所示。

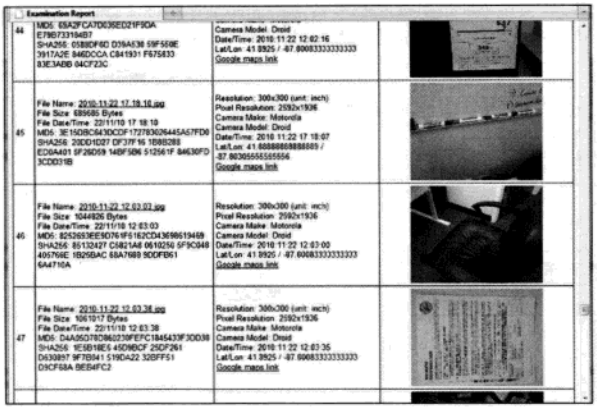


图 6.17　UFED 图像报告

已经被删除的图像不再出现，另外，看上去每一个图像还生成了一个复制件。音频和视频文件都在报告中。报告包含文件名、文件大小、生成的日期和时间以及一个指向相应文件的链接，可以点击进行播放，如图 6.18 所示。

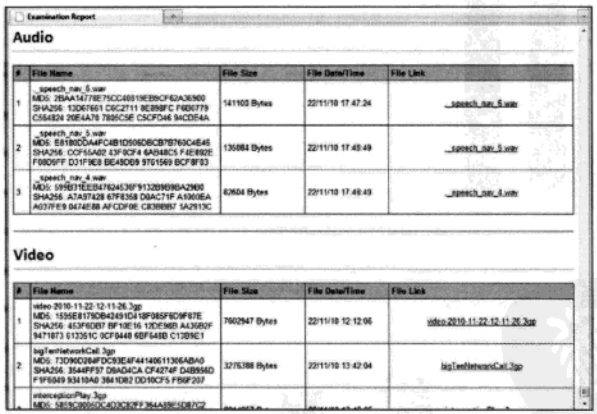


图 6.18　UFED 音频和视频报告

没有被删除的视频被恢复，上载到终端设备的音乐也不会出现在报告中。报告中的音频文件由谷歌地图导航（Google Maps Navigation）返回。

2. Compelson 公司的 MOBILedit!

以下对 MOBILedit! 的介绍资料由开发商提供。

只要轻松点击一下，MOBILedit! Forensics 就可以从移动电话上收集所有可以得到的数据，并且在 PC 上生成一个详细的、可以保存或打印的报告。此软件是一款通用的移动

更多资源请访问稀酷客(www.ckook.com)

电话解决方案，可以支持大多数 GSM 手机，所提供的开放式系统架构也能够用于支持任何手机。此系统能够用于对输出进行个性化处理，使它能够完全适应司法系统的任何需求。

MOBILedit! Forensics 能够对手机进行完整的分析，包括通讯录、最后拨出的号码、未接来电、已接来电、短信、多媒体（彩信）消息、图片、视频、文件、通话的细节、日历、便笺、任务等。

MOBILedit! Forensics 按照实际要求形成报告，报告也可以使用任何语言完成。用户可以根据自身的具体需要制做一个模板，并且可以创建所有想要出现在最终报告中的文字。此软件还可以输出 XML 格式的报告，从而可以将此系统与其他系统进行连接。XSL 将数据按照一定的格式输出给 Internet 浏览器。产生的报告可以根据需要进行发送、共享，也可以被毁掉。

MOBILedit! Forensics 无需任何手工就可以生成报告。虽然没有任何需要从 SIM 卡或手机输入或输出残存的数据，MOBILedit! Forensics 仍然提供一个手工调查模式。此模式只允许读，因此能够预防对手机造成改变，因为任何改变都意味着证据的消失。所有的条目都会使用用于提供数字签名功能的 MD5 散列码进行保护，以预防随后被修改。对于发生的任何修改，此系统也能够帮助迅速找到被修改的可能之处。

MOBILedit! Forensics 软件还频繁更新、升级，因而可以确保使用的是最新技术。对详细报告以及用户友好界面的设计让用户能够有一个非常好的体验。

（1）安装

MOBILedit!4 Forensic 应用软件可以从 www.mobiledit.com 下载，安装只需要花费几分钟时间。在完成安装后第一次运行时，系统会提醒用户去获得对软件的更新。

为了激活软件，Compelson 公司会向邮箱发送一个有“activation card（激活卡）”附件的邮件。这个 PDF 文件内有安装说明以及一个激活密钥，正常使用不会有任何问题。

（2）数据获取

为了进行数据获取，检查师首先必须将 Android 终端设备通过 USB 接口连接到取证工作站上，并确保开启 USB 程序调试。MOBILedit! 会试图检测终端设备，如图 6.19 所示。

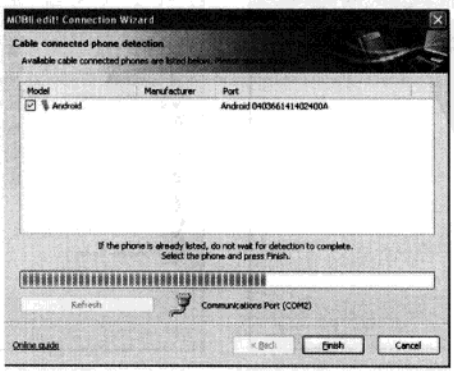


图 6.19　删除 Android 终端设备

更多资源请访问稀酷客(www.ckook.com)

在点击“Finish”（完成）按钮后，用户界面上会显示一个通知，提醒要在终端设备上安装“Connector”（连接器）应用程序，如图 6.20 所示。

在快速安装后，需要为此次调查选择一个名字，还要选择将要抽取的数据的类型。在图 6.21 所示的实例中，选择的是获得“Whole file system”（全部文件系统）的备份这一选项，之后系统就顺利地运行，没有出现任何错误，并在最后显示出成功完成状态，如图 6.22 所示。

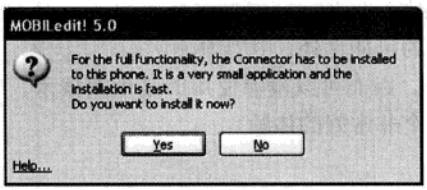


图 6.20　安装 Connector（连接器）

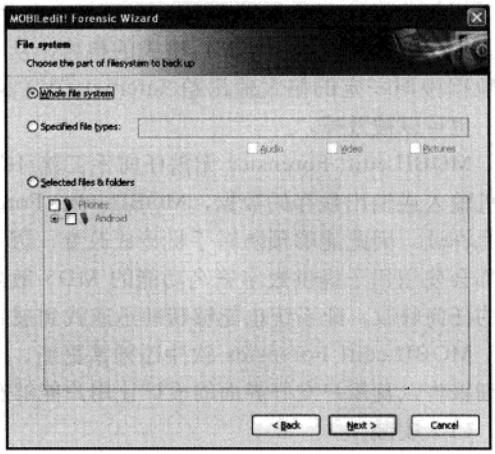


图 6.21　选择备份整个文件系统

然后检查师就可以决定是否将此次的数据加入到一个已经存在的案例中，或者是创建一个新的案例。在如图 6.23 所示的实例中是创建一个新的案例，并且选择 XLS 作为数据输出格式。

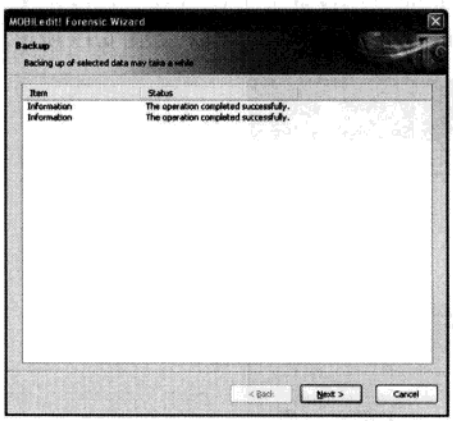


图 6.22　操作成功完成

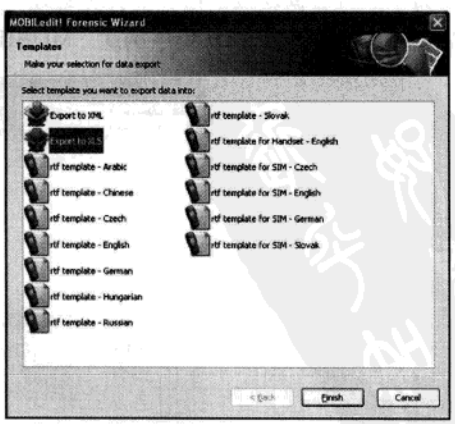


图 6.23　数据输出格式

更多资源请访问稀酷客(www.ckook.com)

（3）数据展现与分析

在完成对终端设备的数据获取后，MOBILedit 会立即显示从终端设备上所获取数据的各项统计，以及能够用于进行分析的应用数据的总揽。图 6.24 显示的是主屏幕，在此屏幕上，检查师可以看到终端设备的具体信息，包括 IMEI 号、序号以及有关手机内存、电池信号、网络信号、终端设备的内存卡上可使用空间的详细信息。

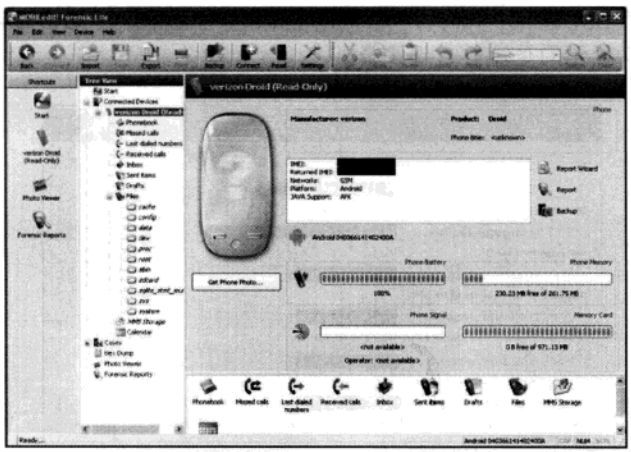


图 6.24　主屏幕

在 Tree View（树状视图）中的下一个选项就是通讯录，检查师可以在此看到所有存储在通讯录中的联系人信息，包括电子邮件地址、手机号码、昵称以及其他任何记录在通讯录内的关于联系人的文本信息，如图 6.25 所示。

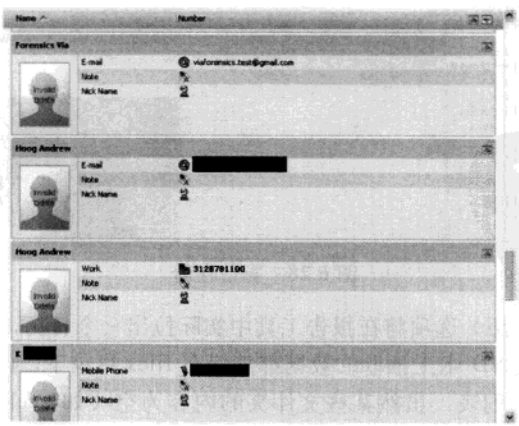


图 6.25　通讯录

下一项是通话记录，被区分为未接来电、最后拨打的电话号码以及已接来电，如

更多资源请访问稀酷客(www.ckook.com)

图 6.26 所示。

短信同样也被区分为不同类别，包括收件箱（Inbox）、已发送邮件箱（Sent items）以及草稿箱（Drafts）。每一个部分都包括消息发送（或接收）的日期和时间、消息内容以及消息的发送者。联系人姓名与通讯录相连接，因此，姓名和电话号码都会显示出来。图 6.27 显示的是短信的收件箱。

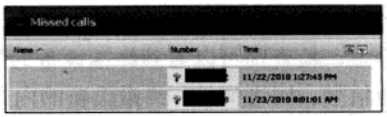


图 6.26 通话记录——未接来电

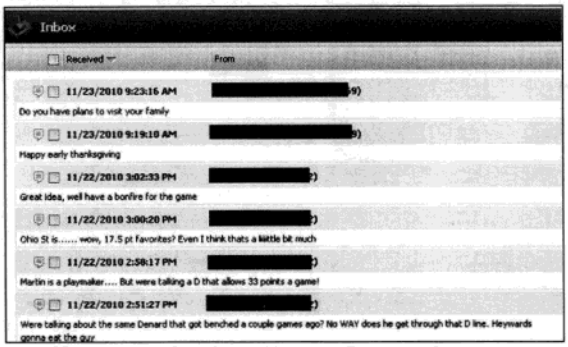


图 6.27 短信服务——收件箱

所有的彩信都在 MMS Storage 文件夹中显示，如图 6.28 所示。屏幕的左边显示的是有关消息的信息，包括主题、发送者号码、接收者号码以及日期和时间。屏幕的右边显示的是实际图片的预览。

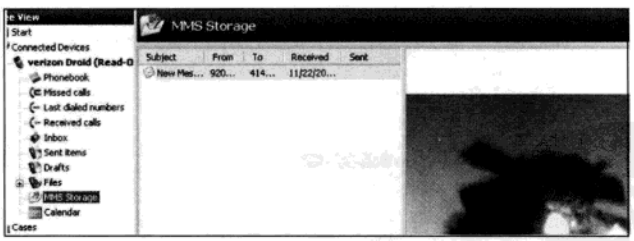


图 6.28 彩信存储

选择 Calendar（日历）选项将在报告工具中实际拉出一个日历，如图 6.29 所示。

其他从终端设备或 SD 卡上抽取的数据都放置在 files 目录下。此目录包含了一个终端设备上所有文件系统的列表。虽然某些文件夹的内容为空（如 cache、config、data），总是会有一些文件夹中包含着从终端设备上获取的原始文件。例如，在 SD 卡这个文件夹中的子文件夹 secret stuff 包含两个文件，如图 6.30 所示。

更多资源请访问稀酷客(www.ckook.com)

图 6.29　日历

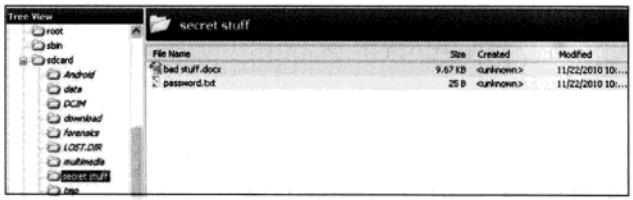


图 6.30　SD 卡上的文件

最后，此工具还具有为某些指定文件提供十六进制转储的功能。在选择 Hex Dump（十六进制转储）以及一个文件后（在此例中，选择了一个 .jpg 格式的文件），此文件的十六进制内容显示在屏幕右边的一个十六进制编辑器中，如图 6.31 所示。

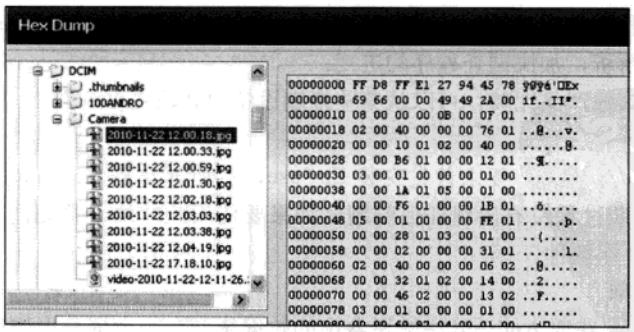


图 6.31　Hex（十六进制）转储

大多数 Android 终端设备中原始的用户数据文件都可以在 data 文件夹中找到，但是，当 MOBILedit 在 Tree View（树状视图）中创建这个文件夹时（在 Files 目录下或文件夹中），此文件夹中没有任何文件。

更多资源请访问稀酷客(www.ckook.com)

需要指出的一件事情是，当数据获取和分析完成后，MOBILedit! Connector 应用软件依然安装在终端设备上。在取证调查结束后，检查师应该认真地考虑使用手工方式将此软件从终端设备上卸载。

3. EnCase 公司的 Neutrino

以下对 Neutrino 的介绍资料由开发商提供。

EnCase 公司的 Neutrino 面向执法者、信息安全分析师以及 eDiscovery 专家而设计，要求是能够从终端设备上获取并显示符合取证要求的数据。调查者能够处理和分析移动终端设备中的数据，并与使用 EnCase 公司产品获取的其他类型的数字证据一同使用。

此公司所提供方案的特点是对硬件的支持以及解析功能，支持最常用的移动终端设备及智能手机操作系统，如 iPhone、Palm、BlackBerry（黑莓）、Android、Windows Mobile、Motorola（摩托罗拉）、Nokia（诺基亚）、Samsung（三星）等。调查者可以收集、分析并保存所有可能有关联的数据，包括：

- 终端设备的设置；
- 通讯录；
- 通话记录；
- 电子邮件；
- 图像；
- 短信多媒体消息；
- 日历；
- 其他存储在终端设备中的文件。

调查者使用 EnCase Neutrino 可以做到以下操作：

- 按照一个易于使用的指南，从各种不同种类的终端设备上收集数据；
- 将不同终端设备和计算机媒体上的数据进行关联；
- 平稳地将收集到的数据集成进 EnCase Forensic 或 EnCase Enterprise，进行分析；
- 快速数据解析，加快调查程序的进度；
- 与相近的其他产品比较，可以从某些终端设备中获取更多的数据。

（1）安装

安装 EnCase Neutrino 要求首先安装 EnCase，然后再安装 Neutrino。按照屏幕上提供的说明，软件安装过程没有出现任何问题。如果要使用此软件，首先需要从 EnCase 获得一个硬件的 USB 小插件。

（2）数据获取

使用 Neutrino 获取 Android 终端设备中的数据统一在一个屏幕上进行。首先，选择终端设备、厂商及模型。下一步是输入案例和终端设备的基本信息。最后，就是将终端设备连接到工作站，并点击“Acquire Current Item”（获取当前项）。图 6.32 显示的是数据获取屏幕。

更多资源请访问稀酷客(www.ckook.com)

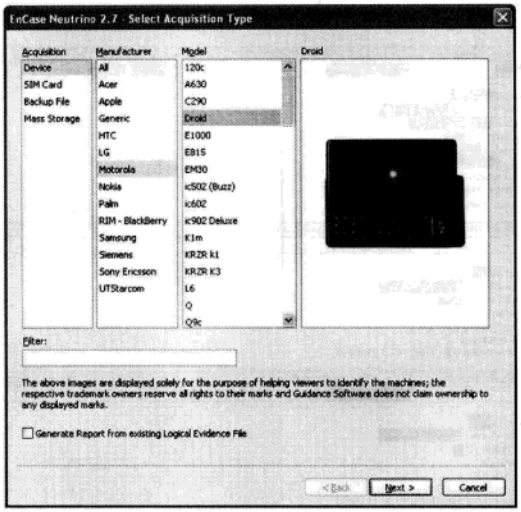


图 6.32　Neutrino 数据获取系统

数据获取在一分钟以内完成。完成后，点击 Generate Report（生成报告）就可以查看数据获取的结果。

（3）数据展现与分析

Neutrino 生成的报告可以是简短报告，也可以是详细报告，图 6.33 所示的简短报告中会包含详细报告中所有的项目，但却只提供很有限的细节信息。

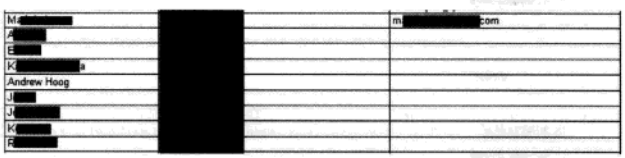


图 6.33　Neutrino 中简短版的通讯录

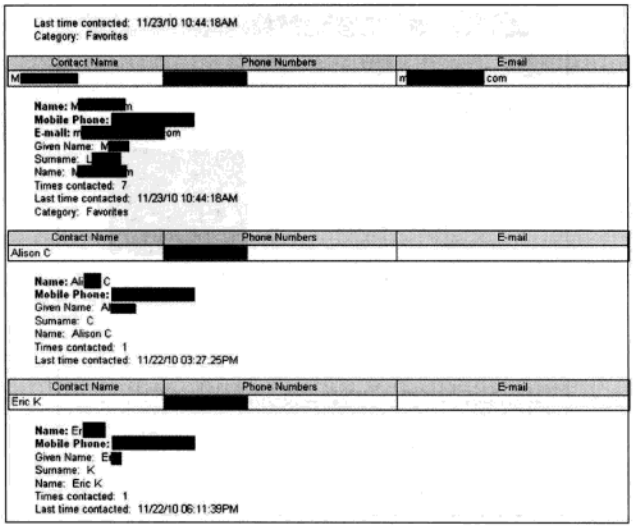
详细报告显示更多关于联系人的细节，如图 6.34 所示。

短信部分的显示也同样有一个简短和一个详细的方式。在简短报告中，只显示短信的另外一个号码、日期和时间、短信的方向（发送或接收）以及短信内容。如图 6.35 所示，详细报告包含的信息还包括与手机号码相关的名字以及短信的状态。

多媒体（彩信）消息只有详细显示方式，如图 6.36 所示。

没有被删除的短信被恢复。关于用户 Web 浏览记录的详细报告提供相当多的详细信息，如图 6.37 所示。

更多资源请访问稀酷客(www.ckook.com)


Last time contacted: 11/23/10 10:44:18AM
Category: Favorites

Contact Name	Phone Numbers	E-mail
M[redacted]	[redacted]	m[redacted]com

Name: M[redacted]n
Mobile Phone: [redacted]
E-mail: m[redacted]om
Given Name: M[redacted]
Surname: L[redacted]
Name: M[redacted]n
Times contacted: 7
Last time contacted: 11/23/10 10:44:18AM
Category: Favorites

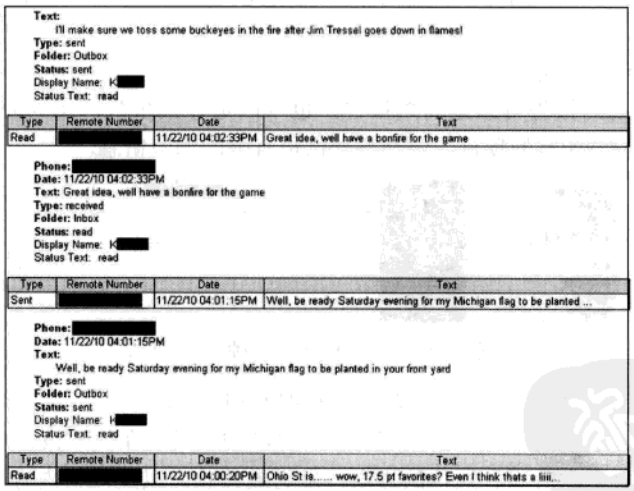
Contact Name	Phone Numbers	E-mail
Alison C	[redacted]	

Name: Ali[redacted] C
Mobile Phone: [redacted]
Given Name: Ali[redacted]
Surname: C
Name: Alison C
Times contacted: 1
Last time contacted: 11/22/10 03:27:25PM

Contact Name	Phone Numbers	E-mail
Eric K	[redacted]	

Name: Er[redacted]
Mobile Phone: [redacted]
Given Name: E[redacted]
Surname: K
Name: Eric K
Times contacted: 1
Last time contacted: 11/22/10 06:11:39PM

图 6.34　Neutrino 中详细的通讯录


Text:
I'll make sure we toss some buckeyes in the fire after Jim Tressel goes down in flames!
Type: sent
Folder: Outbox
Status: sent
Display Name: K[redacted]
Status Text: read

Type	Remote Number	Date	Text
Read	[redacted]	11/22/10 04:02:33PM	Great idea, well have a bonfire for the game

Phone: [redacted]
Date: 11/22/10 04:02:33PM
Text: Great idea, well have a bonfire for the game
Type: received
Folder: Inbox
Status: read
Display Name: K[redacted]
Status Text: read

Type	Remote Number	Date	Text
Sent	[redacted]	11/22/10 04:01:15PM	Well, be ready Saturday evening for my Michigan flag to be planted ...

Phone: [redacted]
Date: 11/22/10 04:01:15PM
Text:
Well, be ready Saturday evening for my Michigan flag to be planted in your front yard
Type: sent
Folder: Outbox
Status: sent
Display Name: K[redacted]
Status Text: read

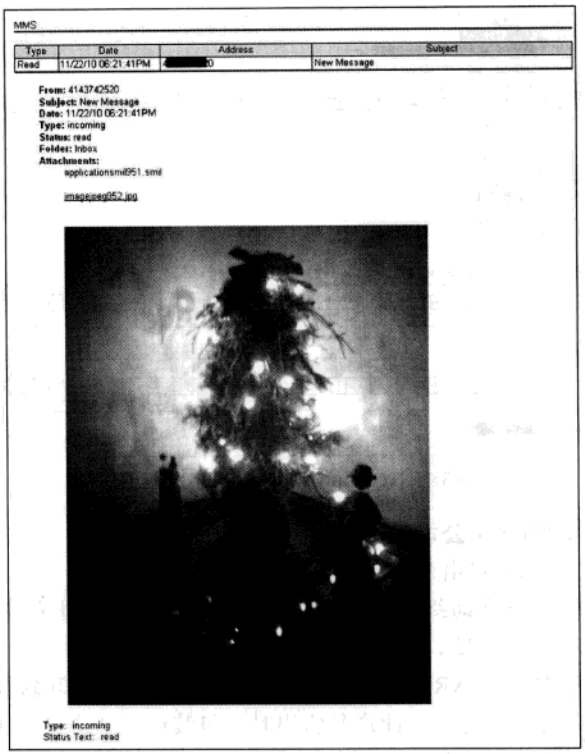
Type	Remote Number	Date	Text
Read	[redacted]	11/22/10 04:00:20PM	Ohio St is..... wow, 17.5 pt favorites? Even I think thats a liii...

图 6.35　Neutrino 中短信的详细报告

能够得到恢复的照片只有终端设备中的彩信中的图片，保存在终端设备 SD 卡上的照片和视频都无法恢复。

报告也可以用其他格式输出，如 HTML，从而使整个报告可以呈现在一个页面中，如图 6.38 所示。

更多资源请访问稀酷客(www.ckook.com)

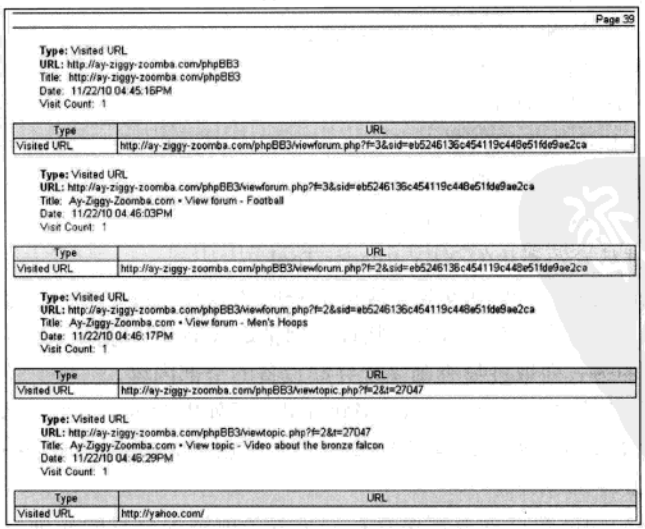

MMS

Type	Date	Address	Subject
Read	11/22/10 06:21:41PM	[redacted]0	New Message

From: 4143742520
Subject: New Message
Date: 11/22/10 06:21:41PM
Type: incoming
Status: read
Folder: Inbox
Attachments:
application/smil951.smil
image/jpeg952.jpg

Type: incoming
Status Text: read

图 6.36　Neutrino 中彩信消息的详细报告


Page 39

Type: Visited URL
URL: http://ay-ziggy-zoomba.com/phpBB3
Title: http://ay-ziggy-zoomba.com/phpBB3
Date: 11/22/10 04:45:16PM
Visit Count: 1

Type	URL
Visited URL	http://ay-ziggy-zoomba.com/phpBB3/viewforum.php?f=3&sid=eb5246136c454119c448e51fde9ae2ca

Type: Visited URL
URL: http://ay-ziggy-zoomba.com/phpBB3/viewforum.php?f=3&sid=eb5246136c454119c448e51fde9ae2ca
Title: Ay-Ziggy-Zoomba.com • View forum - Football
Date: 11/22/10 04:46:03PM
Visit Count: 1

Type	URL
Visited URL	http://ay-ziggy-zoomba.com/phpBB3/viewforum.php?f=2&sid=eb5246136c454119c448e51fde9ae2ca

Type: Visited URL
URL: http://ay-ziggy-zoomba.com/phpBB3/viewforum.php?f=2&sid=eb5246136c454119c448e51fde9ae2ca
Title: Ay-Ziggy-Zoomba.com • View forum - Men's Hoops
Date: 11/22/10 04:46:17PM
Visit Count: 1

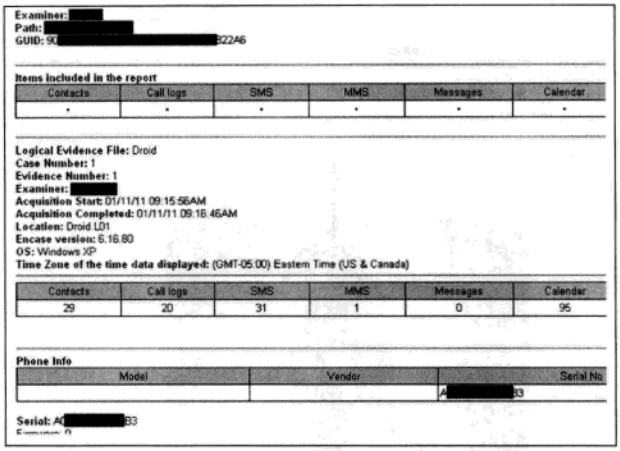
Type	URL
Visited URL	http://ay-ziggy-zoomba.com/phpBB3/viewtopic.php?f=2&t=27047

Type: Visited URL
URL: http://ay-ziggy-zoomba.com/phpBB3/viewtopic.php?f=2&t=27047
Title: Ay-Ziggy-Zoomba.com • View topic - Video about the bronze falcon
Date: 11/22/10 04:46:29PM
Visit Count: 1

Type	URL
Visited URL	http://yahoo.com/

图 6.37　Neutrino 中详细的网页浏览记录报告

更多资源请访问稀酷客(www.ckook.com)



Examiner: [redacted]
Path: [redacted]
GUID: 9[redacted]822A6

Items included in the report

Contacts	Call logs	SMS	MMS	Messages	Calendar
*	*	*	*	*	*

Logical Evidence File: Droid
Case Number: 1
Evidence Number: 1
Examiner: [redacted]
Acquisition Start: 01/11/11 09:15:56AM
Acquisition Completed: 01/11/11 09:16:46AM
Location: Droid L01
Encase version: 6.16.80
OS: Windows XP
Time Zone of the time data displayed: (GMT-05:00) Eastern Time (US & Canada)

Contacts	Call logs	SMS	MMS	Messages	Calendar
29	20	31	1	0	95

Phone Info

Model	Vendor	Serial No
		A[redacted]33

Serial: A[redacted]33

图 6.38　Neutrino 以 HTML 格式输出报告

4. Micro Systemation 公司的 XRY

以下对 XRY 的介绍资料由开发商提供。

XRY 是一款专用于移动终端设备取证的工具，由位于丹麦斯德哥尔摩的 Micro Systemation (MSAB) 公司开发。

XRY 于 2002 年推出，“XRY Complete”是一款包含了硬件和软件的产品，可以用于对终端设备进行逻辑和物理分析。此产品售出时，包装在一个非常轻便、内部定制的盒子中，包含所有必需的硬件，清单如下：

- XRY 取证包软件许可码（XRY Forensic Pack Software License Key）；
- 用于 USB、蓝牙和红外连接的通信集线器；
- SIM 卡 ID 克隆设备（cloner device）；
- 一包 SIM 克隆卡（clone）；
- 写保护的通用内存卡只读器；
- 用于逻辑和物理数据获取的整套连接线；
- XACT Hex Viewer 软件应用程序；
- 向第三方进行发布所用的 XRY Reader Tool。

XRY 在设计和改进过程中吸取了取证调查师的建议，提供一个对整个取证检查过程进行帮助的指南。新的一体化的逻辑 / 物理数据抽取 Wizard（指南）以及所生成的报告以非常简洁和专业的方式向取证检查师呈现终端设备的全部内容。

XRY 的一个独特功能就是 Device Mannual，罗列出对每一款终端设备完整与详细的支持信息：指出能够获取哪些数据以及哪些数据是无法被恢复的，这些时常是取证调查者想要知道的信息。

所有的逻辑和物理数据抽取都保存在一个专为取证安全目的而创建的 XRY 文件中。以这个 XRY 文件作为基础，可以生成所需要的各类报告，报告格式可以是 Word、Excel、

更多资源请访问稀酷客(www.ckook.com)

Open Office 或 PDF。还可以在报告中加入与案例有关的数据以及参考资料，选择哪些数据需要包含在报告中，并将报告发送给取证调查的其他参与方；律师、检察官以及其他的取证调查方。MSAB 提供一个免费的 XRY 阅读器，因此也可以将此阅读器一同提供给第三方，使他们能够在报告上做笔记，同时仍然能够确保原始取证数据的完整性。

此产品还包含一个 XCAT Hex Viewer 应用软件，能够用于对所恢复的原始数据进行更深入的检查，帮助进行查找及手工解码，对 XRY Physical 中提供的自动解码是一个补充。

于 2010 年 6 月 28 日推出的 XRY Forensic Pack v5.1 开始支持苹果公司的 iPad。

（1）安装

XRY 是一款基于 Windows 开发的应用软件，使用开发商提供的一款设置程序进行安装。设置过程包括安装指南、在线软件更新检查，完成整个过程大约需要 15 分钟。操作软件要求使用一个基于硬件的保护小插件。

（2）数据获取

在系统安装完成后，运行此软件，选择 Extract Data（抽取数据），然后再选择从一个手机上抽取数据。之后，必须指出终端设备的类型，可以通过不同的几种方式做到。在此例中，通过选择 Name Search（名称搜索）进行查询，以查找到 Droid，然后选择 Motorola Droid A855，再选择 Next。以上这些步骤如图 6.39 所示。

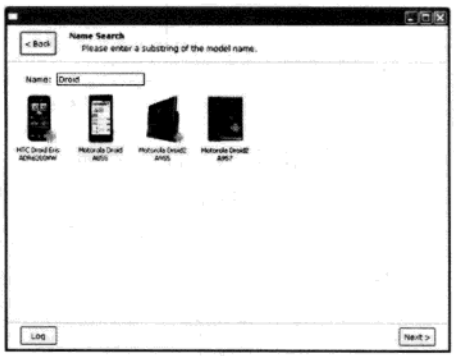


图 6.39　XRY——查询终端设备类型

虽然 MSAB 取证套件支持对某些终端设备进行物理数据获取，但是对于 Android 终端设备，却只支持逻辑数据获取。在选择了逻辑数据获取后，此软件显示能够进行获取的数据，如图 6.40 所示。

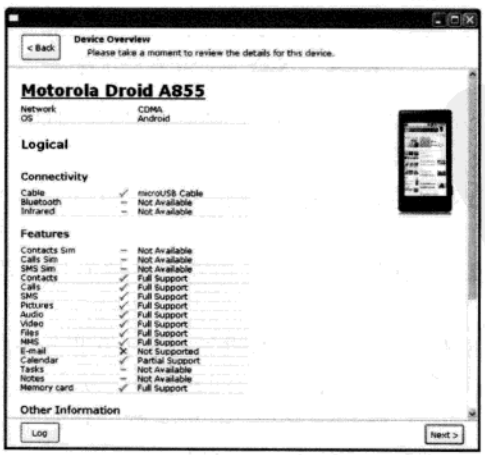


图 6.40　XRY 可以从 Droid 中获取的数据类型

更多资源请访问稀酷客(www.ckook.com)

单击 Next 按钮，然后选择通过数据线抽取数据这一选项以及完整读取，最后点击 Next 开始数据抽取。当数据抽取全部结束，数据抽取结束屏将显示出来，如图 6.41 所示。

图 6.41　XRY 数据抽取完成

（3）数据展现与分析

在完成数据抽取后，此应用程序将显示程序内部数据，并提供了易用的导航功能。通讯录中的联系人列表不但包含联系人的详细情况，而且还标出联系人信息保存的位置，如图 6.42 所示。

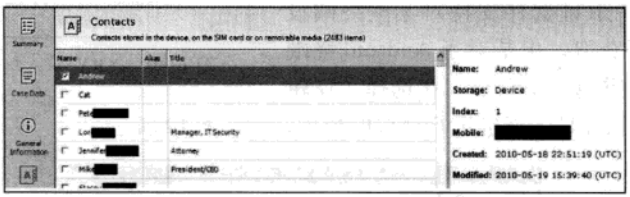


图 6.42　XRY 中的联系人列表

通话记录提供通话方式（拨打或接听）、姓名、号码、时间、通话持续时间以及保存的地方，如图 6.43 所示。

图 6.44 所显示的是短信消息，包括号码、姓名、短信、时间、保存的地方、索引以及此消息所在的文件夹。

图 6.45 所显示的数据说明从终端设备中获取的所有图像均来源于多媒体（彩信）消息。

所获取的还有其他数据。但是，这些数据没有显示在报告中的专设区域内。例如，可以在报告中的 Log 区域查看 Web 浏览记录、网站收藏夹以及谷歌搜索记录。

MSAB 还提供一个叫做 XCAT 的工具，用于查看特定部分的十六进制码。例如，图 6.46 显示的是一个短信消息内容的十六进制码。

更多资源请访问稀酷客(www.ckook.com)

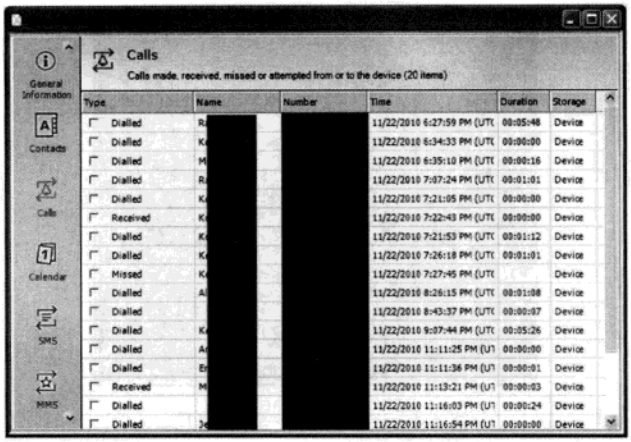


图 6.43　XRY 中的通话记录

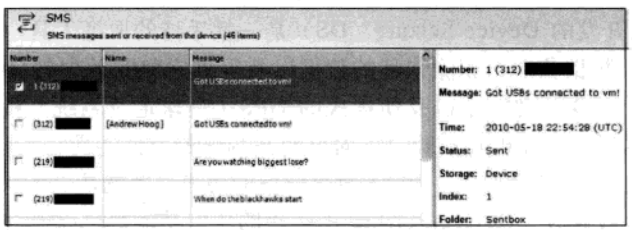


图 6.44　XRY 中的短信消息

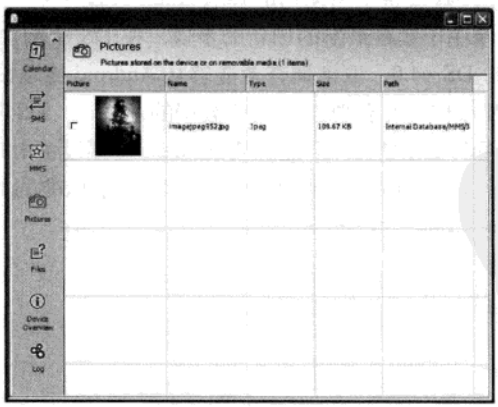


图 6.45　XRY 中的图像

更多资源请访问稀酷客(www.ckook.com)

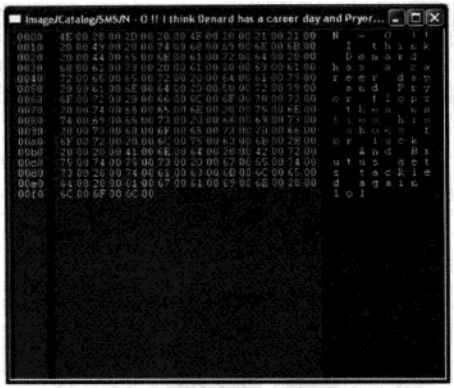


图 6.46 XACT——短信消息

5. Paraben 公司的 Device Seizure

以下对 Device Seizure 的介绍资料由开发商提供。

Paraben 公司开发的 Device Seizure（DS）是一款手持的取证工具，能够让取证调查师进行逻辑和物理数据获取、删除恢复的数据以及得到完整的数据转储内容，可以用于大约 2400 款手机、PDA/ 智能手机及便携式的 GPS（全球定位系统）机。在 2/3 的所支持的终端设备上，通常也能够在删除之处对已经被删除的数据实现物理数据获取。另外，DS 已经被证明，也已经通过验证，100% 满足取证的要求，也就是说所获取的数字证据在任何情况下都不会被修改。Device Seizure 所提供的所有功能都可以通过使用一条标准的 USB 数据连接线与任何 PC 相连接来实现。

在过去两年里，移动终端设备中所使用的谷歌 Android 操作系统对行业的发展产生了巨大的影响。Paraben 公司专注于紧跟最新的创新的步伐，在 DS 中增加了对 Android 操作系统的支持。使用已经发布的 DS v4.0，取证调查者能够从 Android 终端设备获取最需要的数据，如通话记录、通讯录以及短信。在获取以上这些数据的基础上，DS 还能够用于获取多媒体（彩信）消息、图像、视频以及音频文件。能够从 Android 终端设备中获取的数据类型如下：

- 通讯录，包括联系人组、机构以及通讯录设置，除了标准的姓名、电话号码及地址信息；
- 短信消息；
- 多媒体（彩信）消息；
- 通话记录；
- 联系方式；
- 浏览记录；
- 外来图像媒介（元数据）；
- 外来图像的图标媒介（元数据）；
- 外来的媒体、音频及多种其他数据（元数据）；

更多资源请访问稀酷客(www.ckook.com)

• 外来的视频（元数据）；

• 安装在终端设备上所有应用软件及版本列表。

（1）安装

Paraben 公司的 DS v. 4.1.3971.37683 已经安装在一台 Windows 取证工作站上。整个设置过程还要求安装很多必要的驱动软件，需要耗费大量的时间。软件在使用前必须先注册，可通过使用一个硬件小插件或者使用由 Paraben 公司提供的一个软件码文件来完成。如果要安装软件码文件，只需简单地将此文件拷贝到 DS 的安装目录下，或许就是 C:\Program Files\Paraben Corporation\Device Seizure。

（2）数据获取

如果要开始对 Android 终端设备进行数据获取，则首先需要打开一个新的案例，并在案例信息部分按照要求完成填写。选择“Data Acquisition”（数据获取），再选择 Android（安卓），此时，系统会显示以下说明：

```
Android based cell phones must be placed into "debugging" mode*. Follow the
instructions below:
1. On the cell phone, navigate to Settings>Application Settings and select the
Unknown Sources option.
2. On the cell phone, navigate to Settings>Application Settings>Development and
select the USB debugging option.
3. Install drivers for your cell phone. These drivers are provided in the
Device Seizure Drivers Pack that can be downloaded on www.paraben.com.
4. Connect the cell phone to the USB port on your computer.
* Please note that AT&T and Motorola have taken the "Unknown Sources", found on
step 1, out of Android devices. Device Seizure does not support these models.
```

从以上说明可以看到，此软件不支持摩托罗拉手机。但是，对摩托罗拉 Droid 手机进行数据获取却获得了成功。

按照说明，点击“Next”，DS 就会试图去识别是哪一款手机。确认识别出的终端设备信息是准确的，然后再点击“Next”，如图 6.47 所示。

下一个屏幕中将显示 DS 能够从终端设备上所获取的数据类型。所有的数据类型都被选中，其中包括能够从 SD 卡上读到的文件系统，因此造成整个数据获取过程非常耗时。图 6.48 显示 DS 数据获取的进度。

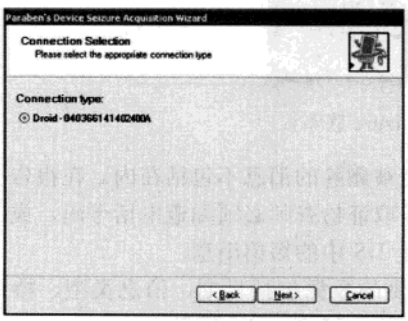


图 6.47　Device Seizure 识别终端设备

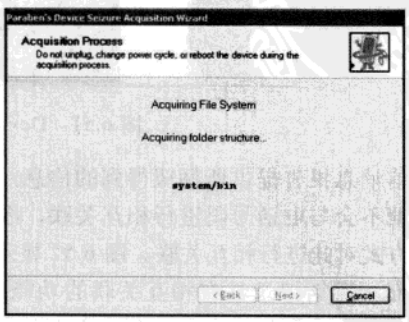


图 6.48　Device Seizure 数据获取进度

更多资源请访问稀酷客(www.ckook.com)

数据获取在大约两个小时后完成，并选择接受对文件进行分类。对文件进行分类也耗费了很长的时间，但是少于两小时。此时，数据获取整个过程才算全部结束。图 6.49 显示 DS 数据获取完成时的输出。

（3）数据展现与分析

Device Seizure 将获取的数据显示在应用中，使用了一个很容易浏览及导航的结构。所获取的数据的目录结构如图 6.50 所示。

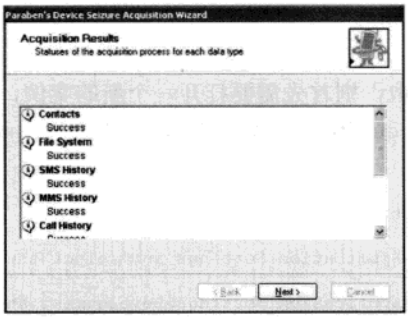


图 6.49 Device Seizure 数据获取完成

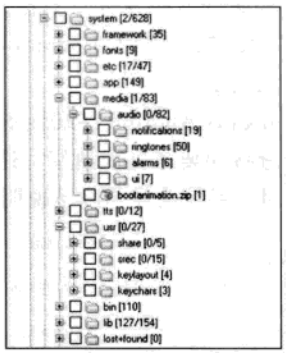


图 6.50 Device Seizure 针对 Droid 所获取的数据的目录结构

联系人部分不但提供姓名、笔记、电话号码以及电子邮件，而且还提供很有用的信息，如联系的次数、最后一次联系的时间，如果有的话，还包括照片，如图 6.51 所示。

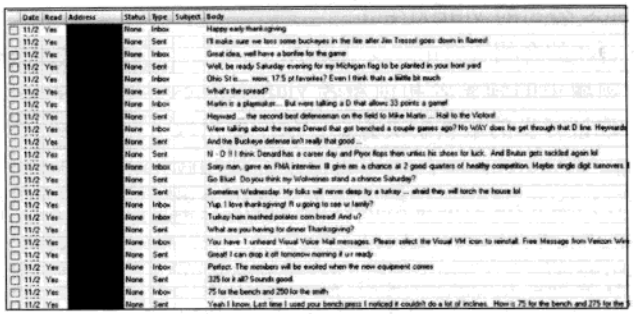
图 6.51 Device Seizure 联系人

短信消息报告提供所期望得到的信息，但是被删除的消息不包括在内。在报告中，联系人数据不会与电话号码进行相互关联，因此，取证检查师必须知道电话号码，或者是通过手工方式对此进行相互关联。图 6.52 显示的是 DS 中的短信消息。

然而，通话记录具有相互关联的功能，提供的数据包括日期、信息类型、持续的时间、电话号码、号码类型、姓名以及此次通话是否为新的通话（也许此电话号码是否第一

更多资源请访问稀酷客(www.ckook.com)

次出现在通话记录中）。图 6.53 显示的是通话记录。



Date	Read	Address	Status	Type	Subject	Body
11/2	Yes	[redacted]	None	Inbox		Happy early thanksgiving
11/2	Yes		None	Sent		I'll make sure we toss some buckeyes in the fire after Jim Tressel goes down in flames!
11/2	Yes		None	Inbox		Great idea, well have a bonfire for the game
11/2	Yes		None	Sent		Well, be ready Saturday evening for my Michigan flag to be planted in your front yard
11/2	Yes		None	Inbox		Ohio St is..... wow, 17.5 pt favorites? Even I think thats a little bit much
11/2	Yes		None	Sent		What's the spread?
11/2	Yes		None	Inbox		Martin is a playmaker... But were talking a D that allows 33 points a game!
11/2	Yes		None	Sent		Hayward ... the second best defenseman on the field to Mike Martin ... Hail to the Victors!
11/2	Yes		None	Inbox		Were talking about the same Denard that got benched a couple games ago? No WAY does he get through that D line. Heyward
11/2	Yes		None	Sent		And the Buckeye defense isn't really that good ...
11/2	Yes		None	Sent		N - O !! I think Denard has a career day and Pryor flops then unless his shoes fix luck. And Brutus gets tackled again lol
11/2	Yes		None	Inbox		Sorry man, gave an FNA interview. Ill give em a chance at 2 good quarters of healthy competition. Maybe single digit turnovers.
11/2	Yes		None	Sent		Go Blue! Do you think my Wolverines stand a chance Saturday?
11/2	Yes		None	Sent		Sometime Wednesday. My folks will never sleep by a turkey ... afraid they will torch the house lol
11/2	Yes		None	Inbox		Yup, I love thanksgiving! R u going to see yr family?
11/2	Yes		None	Inbox		Turkey ham mashed potatoe corn bread! And u?
11/2	Yes		None	Sent		What are you having for dinner Thanksgiving?
11/2	Yes		None	Inbox		You have 1 unheard Visual Voice Mail messages. Please select the Visual VM icon to reinstall. Free Message from Verizon Wire
11/2	Yes		None	Sent		Great! I can drop it off tomorrow morning if ur ready
11/2	Yes		None	Inbox		Perfect. The members will be excited when the new equipment comes
11/2	Yes		None	Sent		325 for it all? Sounds good.
11/2	Yes		None	Inbox		75 for the bench and 250 for the smith
11/2	Yes		None	Sent		Yeah I know. Last time I used your bench press I noticed it couldnt do a lot of inclines. How is 75 for the bench and 275 for the

图 6.52　Device Seizure 中的短信消息

Date	Type	Duration	New	Number	Number Type (Cached)	Name (Cached)
11/22/2010 12:27:59 PM	Outgoing	05:48	Yes	[redacted]	Mobile	Rach L
11/22/2010 12:34:33 PM	Outgoing	00:00	Yes		Mobile	K
11/22/2010 12:35:10 PM	Outgoing	00:16	Yes		Mobile	M
11/22/2010 1:07:24 PM	Outgoing	01:01	Yes		Mobile	F
11/22/2010 1:21:05 PM	Outgoing	00:00	Yes		Mobile	K
11/22/2010 1:21:53 PM	Outgoing	01:12	Yes		Mobile	K
11/22/2010 1:22:43 PM	Incoming	00:00	Yes		Mobile	K
11/22/2010 1:26:18 PM	Outgoing	01:01	Yes		Mobile	K
11/22/2010 1:27:45 PM	Missed	00:00	No		Mobile	K
11/22/2010 2:26:15 PM	Outgoing	01:08	Yes		Mobile	A
11/22/2010 2:43:37 PM	Outgoing	00:07	Yes			
11/22/2010 3:07:44 PM	Outgoing	05:26	Yes		Mobile	K
11/22/2010 5:11:25 PM	Outgoing	00:00	Yes		Work	A
11/22/2010 5:11:36 PM	Outgoing	00:01	Yes		Mobile	E
11/22/2010 5:13:21 PM	Incoming	00:03	Yes		Mobile	M
11/22/2010 5:16:03 PM	Outgoing	00:24	Yes			
11/22/2010 5:16:54 PM	Outgoing	00:00	Yes		Mobile	J
11/22/2010 5:17:06 PM	Outgoing	00:00	Yes		Mobile	F
11/22/2010 5:17:17 PM	Outgoing	00:00	Yes		Mobile	K
11/23/2010 8:01:01 AM	Missed	00:00	No		Mobile	M

图 6.53　Device Seizure 中的通话记录

对于 Web 浏览，可以得到一个完整的记录，并对记录进行了解析，包括对网页访问的次数及网站收藏夹。但是，数据显示非常冗长，因此，图 6.54 中只显示出了开头部分的信息。

		Ay-Ziggy-Zoomba.com • View topic - Video about the bronze falcon	http://ay-ziggy-zoomba.com/p
		Ay-Ziggy-Zoomba.com • View forum - Football	http://ay-ziggy-zoomba.com/p
		Ay-Ziggy-Zoomba.com • View forum - Men's Hoops	http://ay-ziggy-zoomba.com/p
	SQ	Ay-Ziggy-Zoomba.com • Index page	http://ay-ziggy-zoomba.com/p
		http://ay-ziggy-zoomba.com/phpBB3	http://ay-ziggy-zoomba.com/p

图 6.54　Device Seizure 中网页浏览记录

Device Seizure 允许取证检查师选择任何文件，并将其抽取到取证工作站上进行更深入的分析。这一点对于查看和分析在 DS 上没有得到支持的文件类型很有帮助。如果在数据获取阶段选择了对文件进行分类这个选项，则每一个获取的文件都会按照类型进行识别

更多资源请访问稀酷客(www.ckook.com)

并组合，因此就能够快速找到感兴趣的文件。这一点显示在图 6.55 中。

6. viaForensics 公司的 viaExtract

以下对 viaExtract 的介绍资料由开发商提供：

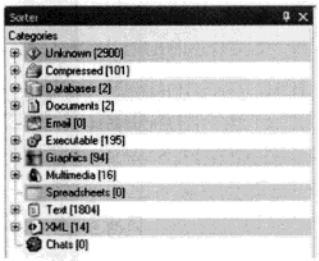


图 6.55 Device Seizure 中的文件排序器

viaExtract 是领域中的领先者和创新者 viaForensics 公司面向 Android 取证开发的最新解决方案。除了提供取证白皮报告和书籍，viaForensics 公司还为执法机构和政府机构免费提供面向 Android 的取证解决方案，解决方案就是 AFLogical。

建立此经验的基础上，viaForensics 公司还开发出了 viaExtract 产品，用于抽取、分析和报告 Android 终端设备上的数据。viaExtract 是一个模块化的设计方案，基于公司的研究和开发工作，很快将提供进行 Android 取证的物理技术。最新的关于 viaExtract 的信息都可以从 http://viaforensics.com/products/viaextract/ 网站上获得，包括对 Android 取证物理技术的支持、对移动平台的更多支持以及先进的取证修复技术。

viaForensics 是一家取证与信息安全公司，积极参与对移动终端设备和传统的计算机进行取证调查商业活动。由于具有直接作为取证检查师的经历，公司已经开发出了特别适合取证检查师使用的工具。开发出的工具适合于频繁地进行更新，这是由于移动取证领域是一个快速发展和变化的领域。与 viaExtract 集成在一起的一个独特的程序调试与报告系统能够简化向 viaForensics 公司发送程序错误及数据清洗的整个过程，极大地帮助设计与改进 viaExtract，使其能够在各种各样的 Android 生态系统上正确运行。

（1）安装

viaExtract 软件以一个虚拟机的方式发布，因此，它可以运行在微软的 Windows、苹果的 Apple OS X、Linux 以及其他任何能够运行所支持的虚拟机的操作系统上。此软件拥有全部的配置，包括所需要的所有驱动程序及支持所用的共享程序库，这样就可以极大地简化安装过程。目前已经支持的几个免费虚拟机软件包包括：

- Oracle 的 VirtualBox；
- VMWare Player。

还有一些商业用的软件包可以使用。虚拟机软件可以从 viaForensics 公司的网站上下载，并导入所支持的软件中。取证检查师可以使用内置在虚拟机软件中的功能，如获取即时状态，以便将软件运行的状态在完成一个案例后又恢复到从前的状态，或者是将此功能直接集成到本地操作系统中，以实现共享数据存储空间以及其他有用的功能。

（2）数据获取

在将 viaExtract 导入到主机的虚拟机软件中，并开始运行后，取证检查师需要登录到 Ubuntu 虚拟机中，并执行 viaExtract，如图 6.56 所示。

然后检查师就可以开始一个新的案例，或者是打开一个过去的案例，如图 6.57 所示。

更多资源请访问稀酷客(www.ckook.com)

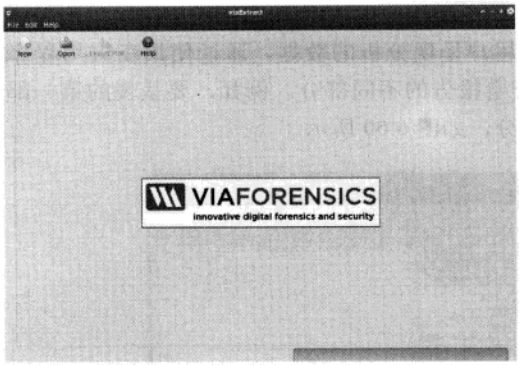


图 6.56　viaExtract 软件

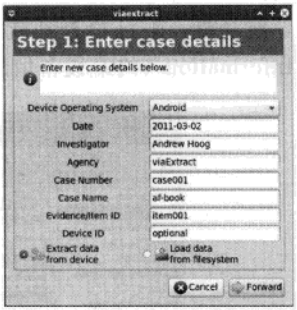


图 6.57　viaExtract——建立新案例

在输入有关案例的细节后，检查师就可以选择从一个 Android 终端设备上直接抽取数据，或者是从存在于文件系统中一个保存抽取数据的文件中载入数据。后一种功能非常有用，适用于检查师在过去已经使用 viaForensics 公司的免费 AFLogical 软件从 Android 终端设备中获取到了数据。此功能也适用于让检查师使用过去获取到的终端设备数据生成新的取证报告，这一类应用非常有用，因为新的报告功能随时被开发出来并加入到软件中。

在此例中，我们通过点击 Forward 来开始从 Android 终端设备中获取数据。我们然后就会看到屏幕上显示出 Load data（载入数据），并对开启 USB 程序调试进行说明。在点击 OK 就开始进行数据获取，如图 6.58 所示。

在数据抽取完成后，检查师会看到所获取数据的列表，并且能够选择所希望包含在取证分析与报告中的数据，如图 6.59 所示。

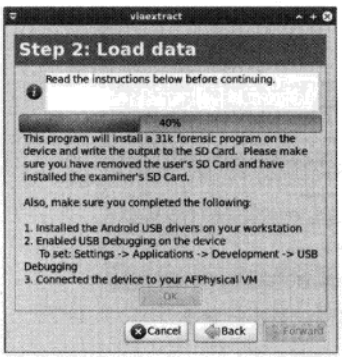


图 6.58　viaExtract——数据抽取

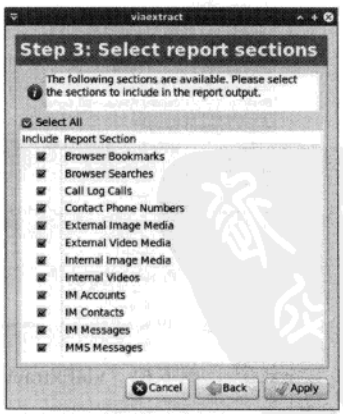


图 6.59　viaExtract——取证分析与报告

在选择完成后，检查师点击 Apply，报告部分的程序开始运行，数据抽取全部结束。

更多资源请访问稀酷客(www.ckook.com)

（3）数据展现与分析

在数据抽取完成后，viaExtract 向用户呈现分析的数据。通过使用在应用程序左边部分的导航功能，取证检查师就可以查看报告的不同部分。例如，要呈现的第一部分是 Device Information（终端设备信息）部分，如图 6.60 所示。

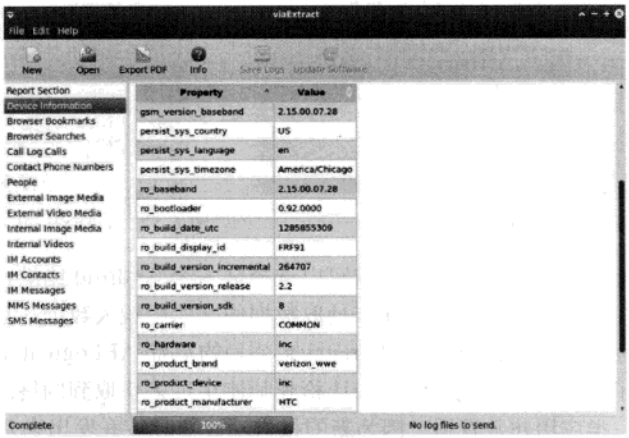


图 6.60　viaExtract——终端设备信息

下一个，图 6.61 显示的是报告中的网页浏览记录及收藏夹部分。

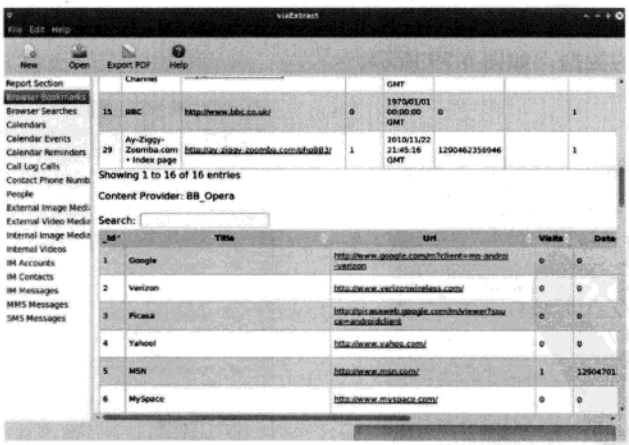


图 6.61　viaExtract——网页浏览记录及收藏夹

在此例中，可以看到报告展示中的几个特点，包括：

- 能够随意地对报告中的任何部分进行过滤；
- 能够对报告中任何的列按照前后顺利进行排序。

更多资源请访问稀酷客(www.ckook.com)

在下一例中，使用 viaforensics.com 关键词对 29 条用户记录进行过滤，结果只剩下 2 条。但是，还有很多记录中的域及数据显示在如图 6.62 所示的屏幕截图的右面。

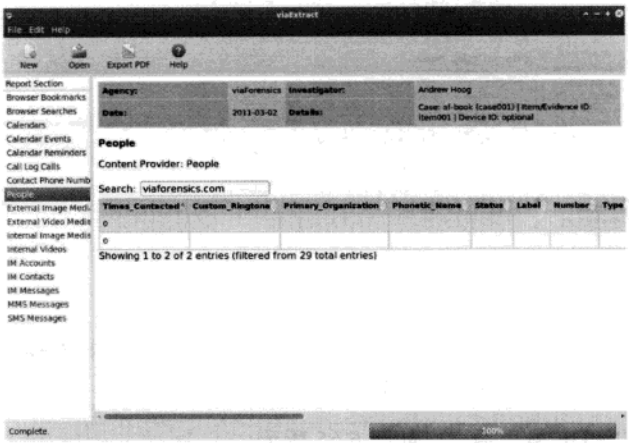


图 6.62　viaExtract—基于人名进行过滤。

viaExtract 目前共支持超过 41 个内容提供商。但是，在下一版软件中，将有大约 100 个内容提供商会得到支持。如果终端设备对内容提供商有所响应，数据抽取以及接下来的报告就会成功。图 6.63 显示的是一个通话记录的实例。

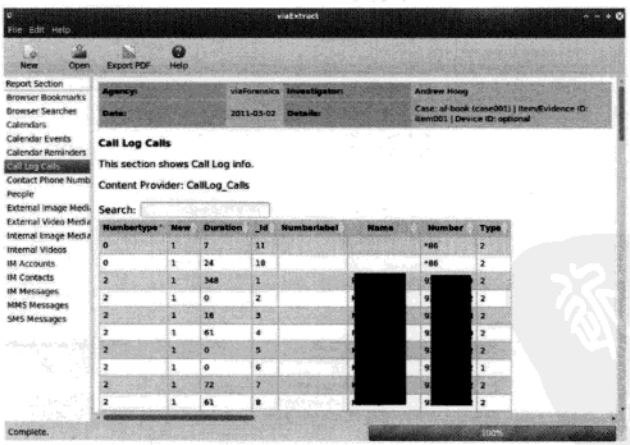


图 6.63　viaExtract——通话记录

图 6.64 显示的是视频媒介元数据。

更多资源请访问稀酷客(www.ckook.com)

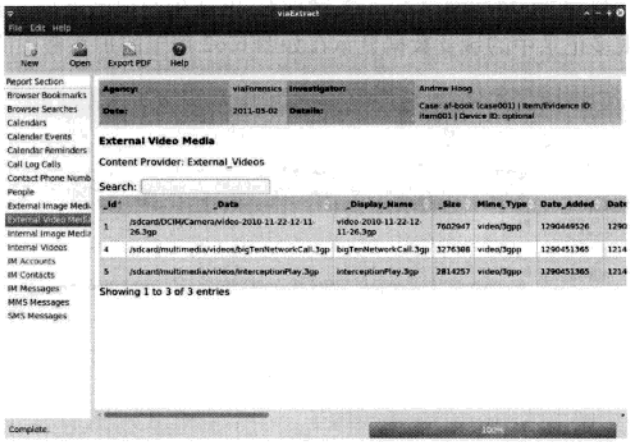


图 6.64　viaExtract——视频媒介元数据

报告还可以以 PDF 格式输出，如图 6.65 所示。

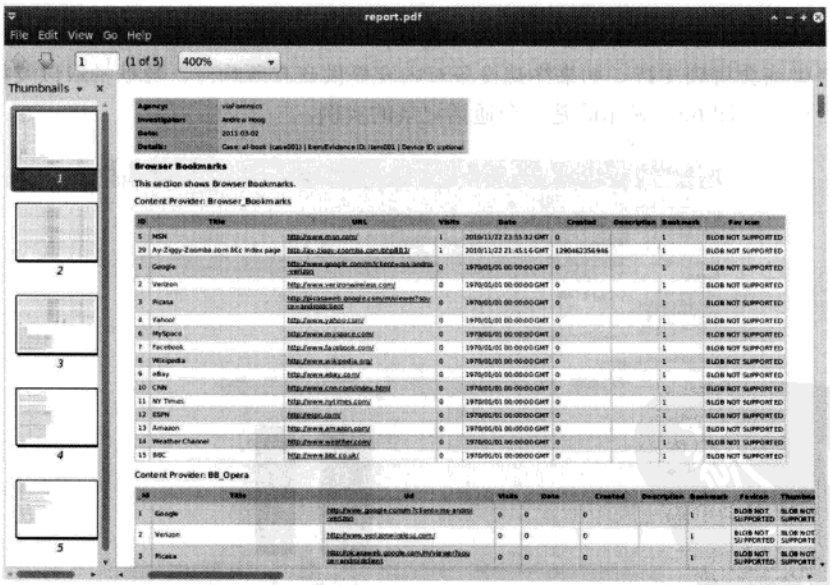


图 6.65　viaExtract—以 PDF 格式输出

6.5　物理技术

获取目标数据存储的物理映像这一类取证技术，通常会产生被修复的数据按照指数

更多资源请访问稀酷客(www.ckook.com)

数量级增加，通常还能够绕过口令的保护。这一类技术不但能够访问到已经被删除的数据，而且还能够访问到系统认为不再需要而丢弃的数据。例如，某些系统跟踪网站最后一次被访问的时间，而当一个网站再一次被访问时，最后一次被访问的时间就被更新。之前的访问日期和时间并没有被实际删除，只是系统不再继续跟踪而已。在使用 YAFFS2 的 Android 终端设备中，只要垃圾回收还没有发生，这些之前的数据还可以被恢复出来。因此，物理技术不但能够提供对系统中已经被删除的数据进行访问，而且还能够提供对系统中已经过时的数据进行访问。

Android 取证物理技术大体上可以划分为以下两类：

- 硬件：将硬件与终端设备实现连接的方法，或者是物理上实际获取终端设备部件的方法；
- 软件：将终端设备中的软件在具有 root 访问权限条件下运行的技术，以获得数据分区完整的物理映像。

基于硬件的方法需要专业的、昂贵的仪器以及对取证人员培训，但是对于无法获得 root 访问权限的终端设备非常有效。基于软件的物理技术是一个更加直接的数据获取途径，因而通常也是最好的开始点。当然，必须首先获得终端设备上的 root 访问权限，基于软件的技术才成为可行。

6.5.1　基于硬件的物理技术

两项基于硬件的物理技术就是 JTAG 和芯片摘取，在本节中对这两项技术进行简要介绍。

1. JTAG

JTAG 技术在 1980 年提出，目的是为印刷电路板（Printed Circuit Boards，PCB）布线和相互连接的测试制定一个标准。到 1990 年，此标准完成了制定，正式成为电机与电子工程师协会（Institute of Electrical and Electronics Engineers，IEEE）的标准，具体来说就是 IEEE 1149.1-1990（IEEE SA, n.d.），随后又在 2001 年进行了更新，标准也变为 IEEE 1149.1-2001。此标准得到广泛接受，目前，大多数印刷电路板都有 JTAG 测试访问接口（JTAG test access ports，TAPs），支持对中央处理器进行访问。

每一个 JTAG 测试访问接口都会向外呈现不同的信号，大多数的移动终端设备都拥有以下接口：

1）TDI Test Data In
2）TDO Test Data Out
3）TCK Test Clock
4）TMS Test Mode Select
5）TRST Test Reset
6）RTCK Return Test Clock

JTAG 的一个主要障碍就是在印刷电路板上找到 JTAG 测试访问接口，并追踪这些

更多资源请访问稀酷客(www.ckook.com)

接口到中央处理器，以确定哪一个连接点具体对应到哪一个测试功能。即使制造商已经提供了关于中央处理器的部署图，以上这些还是很难做到的。此外，在芯片上跟踪 JTAG 功能也是非常困难的，因此这需要先从印刷电路板上取下中央处理器。终端设备制造商对 JTAG 都有各自的定义，但是这些都被认为是公司的机密，只提供给经过授权的服务中心。

另外一种确定连接点功能的方式是先读取每一个连接点上的电压，再根据参考电压数值来确定连接点的功能。在某些情况下，JTAG 的外接点由组装箱的制造商发布，或者由各种在线团体提供。图 6.66 展示的是 T-Mobile HTC G1 上使用的 JTAG 的外接点。6 个外接点用小的白色圈标示出，在右边的说明中提供了详细的信息。

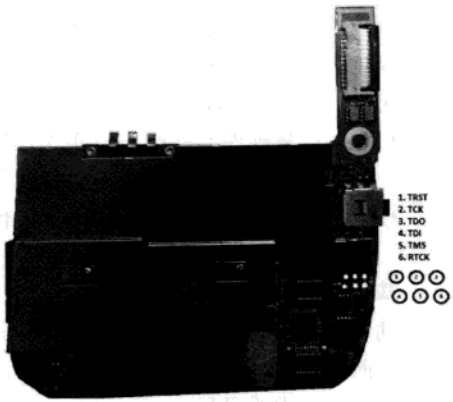


图 6.66　T-Mobile HTC G1 印刷电路板

在大多数情况下，将线头焊接到印刷电路板的连接点上，另外一头则连接到一个特殊设备（组装盒），这个特殊设备再通过软件对中央处理器进行管理。有些公司使用特制的连接头，以支持某些特殊设备，通过将印刷电路板放置在两个具有高翘起接头的夹板中间的方式来简化与连接点进行连接。这些 pogo 连接头与印刷电路板上 JTAG 的连接点相接触，从而可以很容易地与组装盒相连接。然而，有经验的工程师们会发现，将连接头直接与中央处理器进行焊接是一个更加稳定的连接方式。

一旦这些连接头与对应的连接点相连接后，必须向电路板供应电源，以开启中央处理器。每一个中央处理器制造商都公布其硬件使用的参考电压，供应的电压绝对不能超过此参考电压值。某些组装盒提供调节供应的电压这一选项，但是通常应该通过使用外部带有数字电压指示仪的方式予以管理，以确保电压的精确度。一旦向电路板供应了电源，组装盒软件就能够对 NAND 闪存中的数据进行二进制内存转储。但是，由于是串行连接，会需要大量的时间才能完成。尽管有以上这么多的复杂性，只要正确地使用 JTAG 技术，手机可以重新组装，并在没有任何数据损失的情况下正常运行。

虽然 JTAG 是从 Android 终端设备的 NAND 闪存中抽取数据的一种途径，但是会非常困难，只应该由接受过足够的培训以及具有处理很小的印刷电路板焊接点经验的人员去做。在处理 JTAG 焊接点过程出现任何差错，或者是给电路板供应错误的电压不但会造成 JTAG 无法再使用，而且会对终端设备造成严重的损坏。因此，JTAG 通常不是物理获取 Android 终端设备取证映像的首选。

2. 芯片摘取

芯片摘取（chip-off）技术实际上就是将 NAND 闪存从终端设备中取出，在外部对其进行检查。此技术不但可以用于修复已经遭到毁坏的终端设备中的数据，也可以绕过使用

更多资源请访问稀酷客(www.ckook.com)

口令保护的终端设备。将 NAND 闪存取出的过程将是毁灭性的，将 NAND 闪存重新粘连到印刷电路板上并使其正常运行是非常困难的。

芯片摘取技术涉及以下 3 个主要步骤：

1）将 NAND 闪存从终端设备中完全地取出，可以通过断开焊接点或者使用特殊仪器设备通过喷出热气及吸取的方式将芯片取出。还有一些技术是将芯片加热到特定温度。在这个过程中，NAND 闪存很容易遭到损坏，也有特殊硬件，甚至控制软件，可以用于进行数据抽取。

2）取出芯片的过程常常会损坏芯片底部的连接头，因此芯片必须首先经过清洗，然后再进行修复。修复芯片底部的导电球这个过程被称为重建导电球。

3）取出的芯片被插入到一个特殊的硬件设备中，以对芯片进行读取。这些特殊设备通常必须为具体的 NAND 闪存通过程序进行设置，并且已经能够支持若干流行的芯片。

此时就获得了存储在 NAND 闪存芯片中数据的物理映像。

虽然芯片摘取过程非常有效，仍然还存在一个很大的入门障碍。仪器设备和工具昂贵得难以置信，取证检查师必须具有非常专业化的训练和技能。芯片摘取技术，通常是在将芯片从印刷电路板上取出时，总是会存在损坏 NAND 闪存芯片的风险。最后，还期望拥有干净并受到电磁场辐射保护的房间。虽然本地，甚至州一级的执法机构及取证分析公司都会认为，为芯片摘取付出的价钱高得无法接受，但是它仍然是一个有用的技术，更高级别的执法机构会认为它是非常有用的取证技术之一。

6.5.2　基于软件的物理技术和权限

基于软件的物理技术与基于硬件的物理技术相比较具有很多优势。基于软件的技术：

1）更容易执行；

2）通常直接对文件系统进行访问，能够完整地拷贝出所有的逻辑文件（简化某些分析）；

3）几乎没有损坏终端设备或丢失数据的风险。

要使用基于软件的物理技术，首先则必须获得 root 访问权限，然后才能运行数据获取程序。

很不幸的是，Android 终端设备上的 root 访问权限并不能够默认地获得。然而，在某些特定情况下得到 root 访问权限还是很有可能的，我们将会在下面讨论这些情况。值得注意的是获得 root 访问权限面临以下现实的挑战。

1）在很多情况下，获得 root 访问权限会改变终端设备。

2）root 访问权限技术不但随着制造商以及终端设备的不同而不同，而且还随着不同版本的 Android 系统，甚至随着系统所使用的 Linux 核的不同而不同。只是针对目前已经开发出的 Android 终端设备以及版本而言，就已经有数千种可能的组合。

3）获得 root 访问权限的很多技巧在网上已经得到广泛讨论，还常常包含不准确的信息。

综合以上，获得 root 访问权限也许非常困难，而且还总是让人很沮丧。

更多资源请访问稀酷客(www.ckook.com)

通常主要存在 3 种类型的 root 访问权限：

1）利用一些特殊技巧获得的临时的 root 访问权限，通常在系统重启后就不再有效。在这种情况下，adb 后台进程通常在没有 root 访问权限的条件下运行。

2）通过一个定制的只读内存（ROM）或者是利用对 root 持续有效的技巧。定制的只读内存（ROM）通常将 adb 后台进程在具有 root 访问权限的条件下运行，而大多数对 root 持续有效的技巧则不然。

3）通过闪建一个特定的修复分区或者是闪建定制的只读内存（ROM）的部分内容而获得的修复模式 root。定制的只读内存（ROM）通常将 adb 后台进程运行在具有 root 访问权限的条件下，大多数被修改过的修复分区也如此。

如果想要获得 root 访问权限，Android 的追随者们通常只对完整的、可持续的 root 访问权限发生兴趣。然而，从取证分析的观点来看，则更加希望获得临时的 root 访问权限或者是通过特定的修复模式而获得的 root 访问权限。

如果需要获得对一个新的终端设备或者是一个新版本的访问，必须需要一个独立的终端设备用于测试，以确定整个过程运行正常，也没有数据丢失。虽然非常耗费时间，在这种情况下，测试是一个非常重要的步骤。

在以下小节中详细讨论每一个步骤。

1. su

取证检查师需要做的第一件事情就是确定终端设备是否已经具有 root 访问权限。这是已经讨论过的技术中最容易做的一件事情，然而，花上一点点时间去做此测试却是非常值得的。终端设备必须开启 USB 程序调试，即使终端设备被锁住，仍然需要检查一下。如果终端设备没有通过使用口令锁住，确认 USB 程序调试已经开启，这些在第 3 章中已经讨论过。

下一步，将终端设备连接到工作站上，并且尝试通过执行以下的“su”命令来请求获得超级用户访问权限，以获得 root 访问权限：

```
ahoog@ubuntu:~$ adb shell su
su: permission denied
```

在以上这种情况下，没有获得 root 访问权限，这是执行此命令的常见结果。但是，以下结果就表明已经获得了终端设备上的 root 访问权限：

```
ahoog@ubuntu:~$ adb shell su
#
```

当没有收到访问权限拒绝错误回复时就表明授予了 root 访问权限。这一点通过一个新的 # 命令提示符来表明。有些终端设备允许 root 访问权限，但是却要求用户在显示在终端设备上的一个提示符上点击 OK，以授予 root 访问权限。如果此终端设备没有使用任何口令加以保护，就应该查看此命令提示符是否已经显示出来。

2. 研究获得 root 访问权限的技巧

如果终端设备还没有 root 访问权限，则可以在网上寻找可以使用的技术。这个过程也

更多资源请访问稀酷客(www.ckook.com)

许会令人万分沮丧，从非常多缺乏经验的人在各种各样讨论版块上寻求帮助就可以看到这一点。然而，虽然存在着大量的不准确信息，仍然有很多具有丰富知识的资源和技术能够提供有效帮助。

虽然很多可以参考的网站上都在讨论获得 Android root 的技巧，有一个的确比其他都好很多。这个叫做 xda-developers 的网站是一个非常知名与活跃的网站，自称为是“关于 Android 和 Windows 的最大的智能手机追随者和开发者 Internet 社区”（Android & Windows Phone, n.d.）虽然很多其他的网站上都会上传各式各样的 root 技巧，但是一般还都会链接到 xda-developers 网站的讨论区里来。

研究 root 技巧的最好方式常常也就是使用喜欢的搜索引擎在 Internet 上进行搜索，拥有一台测试设备，并且要有足够的耐心。

3. 修复模式

修复模式是 Android 系统中的一个运行模式，用于系统更新、设备格式化以及对设备进行维护。在大多数终端设备上，修复模式非常基本，只提供有限的功能，很确定在 shell 会话中并不提供 root 访问权限。

另一方面，特殊的修复分区几乎总是通过 shell 提供 root 访问权限。这些新的修复分区通常在终端设备具有 root 访问权限时由用户进行安装，用以提供各式各样的功能，以简化定制的只读内存（ROM）所需要的系统备份和更新程序。

与研究 root 技巧一样，检查师在安装一个特殊的修复分区时要极为小心，因为这个过程经常包含对内核和无线电进行更新，在终端设备、内核和无线电固件存在不一致的情况下，很容易造成终端设备无法继续使用（经常被称为“被砖砸了”）。首先必须在一个实验室里的终端设备上进行全面测试，以确保不会出现任何问题。取证检查师们还需要充分了解在终端设备上安装特殊的修复固件时，系统中有什么地方会被修改。

驱动修复模式的软件存储在一个特定的分区中，也非常小。在很多终端设备上，可以通过查看 /proc/mtd 来观看这个修复分区的详细信息：

```
ahoog@ubuntu:~$ adb shell cat /proc/mtd
dev:    size   erasesize  name
mtd0: 00040000 00020000 "misc"
mtd1: 00500000 00020000 "recovery"
mtd2: 00280000 00020000 "boot"
mtd3: 04380000 00020000 "system"
mtd4: 04380000 00020000 "cache"
mtd5: 04ac0000 00020000 "userdata"
```

以上这个列表源自 T-Mobile HTC G1，从中可以看到，这个修复分区的大小为 0x500000 字节，也就是 5MB（0x500000=5 242 880，然后再除以 1024 两次以转换到 KB，最后再到 MB）。以下是本书中所参考的其他手机内此分区的大小：

- T-Mobile HTC G1: 5 MB；
- HTC Incredible: 4 MB；
- 摩托罗拉（Motorola）Droid: 4 MB；

更多资源请访问稀酷客(www.ckook.com)

• 谷歌（Google）Nexus One: 4 MB。

了解这些信息对于我们在下一节中要讨论的替换这个很小但却又很重要的修复分区技术很有帮助。

在前面章节中，我们讨论了如何绕过口令保护的终端设备的技术，其中提到，利用修复模式是一个可行的解决方案。用同样的方式，还是建议要查看一下修复分区的 root 访问权限，因为它将使基于软件的物理技术成为可能。首先，开启终端设备，并进入修复模式运行，可以按照表 6.2 中所罗列的方式，也可以在 Internet 上查找所拥有的终端设备的具体按键组合。一旦终端设备进入修复模式，将其连接到 Ubuntu 虚拟机上，并按照以下方式执行 adb：

```
ahoog@ubuntu:~$ adb devices
List of devices attached
0403555551222244F       recovery
```

在此例中，adb 发现了一个运行在修复模式下的终端设备。但是，很多终端设备在修复模式中根本就不允许 adb 访问，特别对于库存的终端设备更是如此。在这种情况下，可以按照以下方式确定 shell 是否拥有 root 访问权限：

```
ahoog@ubuntu:~$ adb shell
#
```

就像我们前面看到的那样，如果出现的是一个 # 提示符，则表明拥有 root 访问权限。反之，如果出现的是一个 $ 提示符，则表明没有 root 访问权限。然而，接下来就应该至少通过执行 su 命令去尝试得到 root 访问权限。

4. 启动加载程序

在第 2 章中已经讨论过，启动加载程序是一个很小的程序，只在 Android 启动的最初运行，负责选择并载入主内核以及其他的一些功能。在某些终端设备上还存在有由制造商开发的特殊软件，可以与启动加载程序进行交互。此软件能够向终端设备上的 NAND 闪存内写入新的映像。制造商使用此软件来修复无法正常运行的终端设备，此软件也可能使用在其他场合，如开发及测试。取证检查师们也可以使用此软件来闪存一个共享应用程序或者来利用终端设备上的 NAND 闪存，以此来获得 root 访问权限。但是，大多数终端设备上的启动加载程序在出厂向外销售时都处于锁住状态，以防止以上这些对其的更新。

这一类软件的一个具体实例是一款叫做 RSD Lite 的软件，由摩托罗拉开发，可以与摩托罗拉生产的 Android 终端设备进行交互。RSD Lite 是一款不开放的软件，看起来只提供给摩托罗拉服务中心用于终端设备维修。假设任何使用此款软件的人都获得了使用授权，在此对这款软件的介绍也只是作为一个实例来说明在某些 Android 终端设备中是如何进行闪存的。

已经有很多网站讨论 RSD Lite，并对此款软件提供使用指南。在一个叫做 modmymobile.com 的网站上的一篇文章“[Guide] Flashing Linux Motorola's with RSD Lite

更多资源请访问稀酷客(www.ckook.com)

Versions”中有如何使用此软件的一步一步详细说明（[Guide] Flashing Linux, n.d）。

假设这个终端设备得到支持，并且启动加载程序也已经解锁，那么就可以将此终端设备连接到取证工作站上，并运行此软件，软件运行中将识别出此手机。然后就必须要提供正确的 .sbf 文件，并点击 Start（开始）按钮来对终端设备进行闪存，如图 6.67 所示。

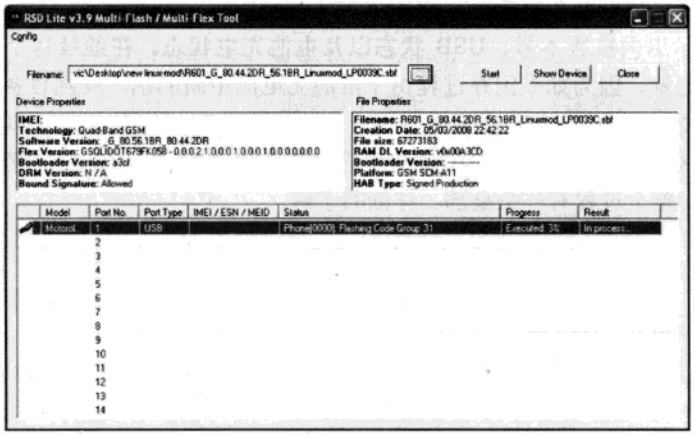


图 6.67　RSD Lite

在此过程结束后，重新开启终端设备，一个新的修复分区（以及其他被所使用的 .sbf 文件修改的区域）就可以使用了。RSD Lite 为取证分析师们提供一个很好的选择，只要这些取证分析师们获得了使用 RSD Lite 的授权。

5. sbf_flash

与摩托罗拉公司的 RSD Lite 类似的另外一款软件叫做 sbf_flash，但是却没有 RSD Lite 所具有的许可和使用限制。此应用软件由一个 Android 追随者开发，并发布在网上，虽然可以从很多网站上得到，但是最好还是从作者的微博（OPTICALDELUSION）去下载，同时当有新的版本时还会进行更新。此软件最初在 Linux 上开发，目前也可以在 OS X 上运行，因而可以极大地简化通过一个未锁住的启动加载程序在 NAND 闪存上对数据进行闪存。目前最新的版本是 sbf_flash v.1.15，提供以下的支持：

```
ahoog@ubuntu:~$ ./sbf_flash -h
SBF FLASH 1.15 (mbm)
http://opticaldelusion.org

Usage: ./sbf_flash <filename>

sbf_flash [options] [sbf file]
    -f               - force; attempt to continue on error
    -v               - verbose output (of CDT)
    -r               - read CDT information from the phone
    -x               - extract sbf file
    -d               - download cg from phone
    --cgname [file]  - upload/download cgname
```

更多资源请访问稀酷客(www.ckook.com)

```
                    matches any cgname shown with -r
                    optional file arg for contents
```

在使用 sbf_flash 之前，首先必须确认它当前所支持的终端设备。在此例中，我们使用的是摩托罗拉的 Droid；当然，还有其他的终端设备也被支持。终端设备必须被置于启动加载程序模式中，实现的方法是在 D 板上按住向上方向的箭头键，同时再按下电源键。启动下载程序可以很容易在摩托罗拉的 Droid 上得到识别，因为会在一个全黑的屏幕中用白颜色文字显示出内核版本号、USB 状态以及电池充电状态。在继续这个过程前，确认电池已经充满电量，因为如果闪存过程由于电池无电而中断的话，终端设备很容易遭到毁坏。另外，还要确认 SBF 文件必须已经保存在取证工作站上，从而才可以将它闪存到终端设备中去。这就是为什么在面向终端设备的工作开始前需要进行大量的测试工作，以确保兼容性以及对整个过程有一个全部、详细的了解。

当 Droid 工作在启动加载程序模式时，我们可以使用 sbf_flash 向终端设备发出请求，如下所示：

```
ahoog@ubuntu:~$ ./sbf_flash -r
SBF FLASH 1.15 (mbm)
http://opticaldelusion.org
 >> waiting for phone: Droid found.
 CG63 0xC0000000-0xC001FFFF mbmloader.img
 CG30 0xC0020000-0xC00BFFFF mbm.img
 CG55 0xC00C0000-0xC015FFFF mbmbackup.img
 CG31 0xC0160000-0xC01BFFFF cdt.bin
 CG38 0xD01CE000-0xD0359FFF pds
 CG34 0xC035A000-0xC03BCFFF lbl
 CG57 0xC03BD000-0xC041FFFF lbl_backup
 CG41 0xC0400000-0xC057FFFF sp
 CG42 0xC0580000-0xC061FFFF logo.bin
 CG44 0xC0620000-0xC067FFFF misc
 CG35 0xC0680000-0xC09FFFFF boot
 CG47 0xC0A00000-0xC0E7FFFF recovery
 CG39 0xD0EF4000-0xD9FB6FFF system
 CG40 0xD9FB7000-0xDFF3BFFF cache
 CG37 0xDFF3C000-0xF0D29FFF userdata
 CG53 0xDFD40000-0xDFF3FFFF kpanic
 CG54 0xDFF40000-0xDFFFFFFF rsv

Usage: ./sbf_flash <filename>
```

sbf_flash 程序找到一个目前正处于启动加载程序模式下的终端设备，并立即将映像文件闪存到 NAND 闪存中去。更新过程的状态显示在屏幕上，然后，Droid 被重新启动。

```
ahoog@ubuntu:~$ sbf_flash SPRecovery.sbf
=== SPRecovery.sbf ===
00: RDL03 0x80500000-0x8054CFFF DECE AP
01:  CG47 0xC0A00000-0xC0D5C7FF 02C0 AP

 >> waiting for phone: Droid found.
 >> uploading RDL03
Uploading: 100% OK
 >> verifying ramloader
```

更多资源请访问稀酷客(www.ckook.com)

```
 -- OK
 >> executing ramloader
Droid found.
 >> sending erase
 >> uploading CG47
Uploading: 100% OK
 >> verifying CG47
 -- OK
 >> rebooting
```

也应该要准备好使启动起来的终端设备立即进入修复模式，这是因为在摩托罗拉 Droid 固件的近期版本实现了一个验证现有的修复分区的散列签名，与同一 Android 版本原来的修复分区进行比较。如果出现不一致，系统就会在启动过程中重建原来的修复分区，因此也就将修改过的修复分区进行覆盖。

一旦这个新的 SBF 文件完成了被闪存，终端设备运行在修改后的修复模式下，检查师就获得了 root 访问权限，也就可以继续使用基于软件的物理技术。

6. 快速启动

快速启动（fastboot）是另外一个可以通过 USB 接口将映像闪存到 NAND 闪存中去的应用程序。快速启动的源代码包含在 AOSP 中，因此，一旦对 AOSP 代码进行编译，此应用程序就得以构造。与 sbf_flash 一样，要求启动加载程序必须要支持快速启动，这就不但需要使用一款兼容的启动加载程序，而且此款启动加载程序的安全还必须置于关闭（即 S-OFF）。

快速启动最初被用在谷歌 Android 开发者手机上（ADP），由 HTC 公司生产。因此，在大多数关于快速启动的文档和参考文献中引用的都是 ADP，同时，HTC 对此应用程序也有一个非常有帮助的参考网页（HTC Developer Center, n.d.）。此网页不但提供 ADP 终端设备中的各种原来的 NAND 闪存映像文件，还提供使用快速启动以及访问终端设备上合适模式的指导性说明（HTC Developer Center, n.d.）。

如果要想进入快速启动模式，在按住 BACK 按键的同时，开启终端设备的电源（或重新启动）。按住 BACK 按键，一直等到启动加载程序屏幕出现并显示出“FASTBOOT”。终端设备现在就进入了快速启动模式，等待接受快速启动模式下的命令。如果此时想要退出快速启动模式，按住以下几个按键 MENU+SEND+END（在 ADP 上，SEND 是“Call”按键，而 END 是“End call”按键）。

需要注意的是，启动加载程序的屏幕在不同的终端设备上可能有所不同。对于 ADP 终端设备，启动加载程序显示的是一个站在滑雪板机器人的图像。其他款的终端设备可能显示不同的图像，或者是不同的颜色图案。但是在任何情况下，启动加载程序在快速启动模式下都会显示出“FASTBOOT”文字。同时，启动加载程序还会显示出无线电版本。

一旦在快速启动模式中，使用以下命令来验证快速启动找到了终端设备：

```
ahoog@ubuntu:~$ ./fastboot devices
HT08XHJ00657    fastboot
```

快速启动还提供很多选项，执行快速启动命令并将 help 作为参数就会看到所有细节，

更多资源请访问稀酷客(www.ckook.com)

如下：

```
ahoog@ubuntu:~$ ./fastboot --help
usage: fastboot [ <option> ] <command>

commands:
  update <filename>                        reflash device from update.zip
  flashall                                 flash boot + recovery + system
  flash <partition> [ <filename> ]         write a file to a flash partition
  erase <partition>                        erase a flash partition
  getvar <variable>                        display a bootloader variable
  boot <kernel> [ <ramdisk> ]              download and boot kernel
  flash:raw boot <kernel> [ <ramdisk> ]    create bootimage and flash it
  devices                                  list all connected devices
  reboot                                   reboot device normally
  reboot-bootloader                        reboot device into bootloader

options:
  -w                                       erase userdata and cache
  -s <serial number>                       specify device serial number
  -p <product>                             specify product name
  -c <cmdline>                             override kernel commandline
  -i <vendor id>                           specify a custom USB vendor id
```

可以看到，一旦在快速启动模式中，很容易将修改过的修复分区进行闪存：

```
ahoog@ubuntu:~$ fastboot flash recovery modified-recovery-image.img
```

在这个过程完成后，就可以将手机重启并进入修复模式，然后就可以继续使用基于软件的物理映像技术。

6.5.3 AFPhysical 技术

AFPhysical 技术由 viaForensics 公司开发，用于获取 Android 终端设备中 NAND 闪存分区的物理映像。使用 AFPhysical 技术需要对终端设备拥有 root 访问权限，此技术应该能够支持任何 Android 终端设备。然而，使用 AFPhysical 技术并不是一个简单的过程，取证分析师需要对此技术做相应的调整，使其能够适用于被调查的目标终端设备。这主要是由于 Android 终端设备之间存在着相当多的不同之处，不但在不同的制造商之间如此，即使在运行不同 Android 版本的终端设备之间也如此。

AFPhysical 使用的总体过程还是比较简单：

1）获得目标 Android 终端设备上 root 访问权限；

2）识别 NAND 闪存中需要被映像的数据分区；

3）将取证共享应用程序库上载到目标 Android 终端设备中；

4）获取 NAND 闪存中数据分区的物理映像；

5）删除所有存储在非易失性存储空间中的取证共享应用程序库的程序。

不管如何使用此技术，都要求必须已经获得终端设备上的 root 访问权限。在此例中，我们的取证目标是一台摩托罗拉 Droid 手机。由于我们能够在摩托罗拉 Droid 中闪存一个

更多资源请访问稀酷客(www.ckook.com)

经过修改的修复分区，即使终端设备使用口令加以保护，此技术也可以正常使用。

在我们完成将修改过的修复分区进行闪存，重启终端设备并进入修复模式后，我们将此终端设备连接到 Ubuntu 虚拟机上，并通过执行 adb devices 命令确认 adb 能够找到此终端设备：

```
ahoog@ubuntu:~$ adb devices
List of devices attached
040363260C006018        recovery
```

然后，进入 shell 并确认已经具有 root 访问权限：

```
ahoog@ubuntu:~$ adb shell
/ #
```

此时，我们需要更好地了解此终端设备，使我们能够决定对哪些分区进行物理映像。如果有任何挂载的文件系统，首先查看的就是这些挂载的文件系统：

```
/ # mount
rootfs on / type rootfs (rw)
tmpfs on /dev type tmpfs (rw,mode=755)
devpts on /dev/pts type devpts (rw,mode=600)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/block/mtdblock7 on /cache type yaffs2 (rw,nodev,noatime,nodiratime)
```

从以上结果可以看出，终端设备使用 MTD 对 NAND 闪存及 YAFFS2 进行访问。执行以下命令可以确定使用 MTD 可以看到的分区：

```
/ # cat /proc/mtd
dev:    size   erasesize  name
mtd0: 000a0000 00020000 "mbm"
mtd1: 00060000 00020000 "cdt"
mtd2: 00060000 00020000 "lbl"
mtd3: 00060000 00020000 "misc"
mtd4: 00380000 00020000 "boot"
mtd5: 00480000 00020000 "recovery"
mtd6: 08c60000 00020000 "system"
mtd7: 05ca0000 00020000 "cache"
mtd8: 105c0000 00020000 "userdata"
mtd9: 00200000 00020000 "kpanic"
```

取证检查师应该对所有的 MTD 分区进行映像。但是，在此例中，我们将重点放在 mtd8 这个用户数据分区上。

在我们准备获取终端设备上的数据时，如果对有些术语和数据结构还不是很清楚的话，再回顾一下第 4 章中讨论过的 NAND 闪存以及文件系统很有帮助。一旦获得了终端设备上的 root 访问权限，就可以对 Android 物理数据获取采取以下 4 种策略：

1）将所有 NAND 闪存上的分区全部进行转储（nanddump），包括数据及 OOB（优先考虑）；

2）只对分区进行 dd 映像，这样做只获得数据，不能获得 OOB；

3）使用 tar 对文件进行逻辑获取；

更多资源请访问稀酷客(www.ckook.com)

4）使用 adb 对文件进行逻辑获取。

此外，主要有两种方式保存从终端设备上获取的数据：

1）使用 adb 端口转发（adb port forward）技术通过 USB 在 Ubuntu 工作站与 Android 终端设备之间建立一个网络；

2）将一个 SD 卡插入终端设备，完成挂载，在本地保存数据。

以上两个方法各有优点。使用 adb 端口转发不需要插入用于数据存储的设备，可以直接在工作站上创建文件。将数据文件保存在 SD 卡上则会大大加快数据获取的速度。两种方法都是行之有效的，将在下面的讨论中进行介绍。

我们首先讨论将用户数据分区通过 nanddump 全部进行转储，因为这样可以获得所有的取证数据。为了实现 nanddump 的目的，首先必须在 ARM 平台上将某一个版本的 nanddump 进行编译。将 nanddump 进行交叉编译不在本书的讨论范围之内。然而，可以在互联网上找到这样的编译程序，或者是按照网上的在线指导中的说明去做。

提示：为 ARM 交叉进行编译

对源代码进行交叉编译，然后在 ARM 平台上运行可能会非常困难，这方面的在线支持信息也非常稀少。一种可能的方案就是使用 Android 的 Native Development Kit（NDK）来构建一个兼容的共享程序库。另外一种可能就是使用 Linux 并安装一个交叉编译器，如专为 ARM GNU/Linux 的 Code Sourcery 的 G++ Lite 2009q3-67，可以从 http://www.codesourcery.com/sgpp/lite/arm/portal/release1039 网站上下载。在安装了交叉编译器后，必须修改源代码中的 Makefile，明确交叉编译选项。此外，可以登录到本书的网站 http://viaforensics.com/education/android-forensics-mobile-security-book/ 上去获得所有的更新。

为了避免向 NAND 闪存写入任何数据，我们再一次使用 mount 命令检查一下其结果，并且需要注意挂载到 /dev 目录的是 tmpfs，因而其中的数据被保存在 RAM 中。我们因此可以将取证应用共享程序库到 /dev 中：

```
ahoog@ubuntu:~$ adb push AFPhysical/ /dev/AFPhyiscal
push: AFPhysical/tar -> /dev/AFPhyiscal/tar
push: AFPhysical/md5sum -> /dev/AFPhyiscal/md5sum
push: AFPhysical/nanddump -> /dev/AFPhyiscal/nanddump
push: AFPhysical/nc -> /dev/AFPhyiscal/nc
4 files pushed. 0 files skipped.
1003 KB/s (2803303 bytes in 2.727s)
```

下一步需要做的是在终端设备上执行程序。为了做到这一点，我们使用 chmod 命令来改变文件的访问权限，包括执行标志位。我们设置让所有的文件都可以被任何用户读取和执行：

```
ahoog@ubuntu:~$ adb shell
/ # cd /dev/AFPhyiscal
/dev/AFPhyiscal # ls -l
```

更多资源请访问稀酷客(www.ckook.com)

```
-rw-rw-rw-    1 0        0          711168 Jan 24  2011 md5sum
-rw-rw-rw-    1 0        0          669799 Jan 24  2011 nanddump
-rw-rw-rw-    1 0        0          711168 Jan 24  2011 nc
-rw-rw-rw-    1 0        0          711168 Jan 24  2011 tar
/dev/AFPhyiscal # chmod 755 *
/dev/AFPhyiscal # ls -l
-rwxr-xr-x    1 0        0          711168 Jan 24  2011 md5sum
-rwxr-xr-x    1 0        0          669799 Jan 24  2011 nanddump
-rwxr-xr-x    1 0        0          711168 Jan 24  2011 nc
-rwxr-xr-x    1 0        0          711168 Jan 24  2011 tar
```

从以上可以看出，在对程序执行完“chmod 755”命令后，所有程序的执行位现在都已被设置，访问权限中相应位的 x 表达出了这一设置。

如果想要将 nanddump 的数据保存在 SD 卡上，需要确认将一个格式化后的 SD 卡插入终端设备，并正确地将其挂载到系统。然后，我们就可以按照以下方式执行 nanddump 命令：

```
/ # /dev/AFPhyiscal/nanddump /dev/mtd/mtd8ro > /sdcard/af-book-mtd8.nanddump
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 1
Number of bbt blocks: 0
Block size 131072, page size 2048, OOB size 64
Dumping data starting at 0x00000000 and ending at 0x105c0000...

/ # ls -l /sdcard/af-book-mtd8.nanddump
-rwxrwxrwx    1 0        0       283041792 Jan  1 00:12 /sdcard/
af-book-mtd8.nanddump
```

最后，就可以使用 adb pull 将数据传送给 Ubuntu 虚拟机，也可以取出 SD 卡，直接使用 USB 复制数据，后一种方式在速度上会快很多。

提示：MD5 散列

虽然在数据获取过程中，用户的数据分区并没有挂载到终端设备上，但“/dev/mtd/mtd8ro”的 md5sum 散列签名仍然会改变，即使没有写入任何数据。这些都由 NAND 闪存的特殊性质而决定，即操作系统及内存总是处于一个几乎常态化改变的状态下，这是由于层级磨损、损坏数据块管理以及其他即使在没有任何用户数据改变的情况下也会发生的机制。最好的方法就是对生成的 NAND 闪存文件完成一次 md5sum，以确保文件随后的完整性。

第二种保存 NAND 闪存文件或者其他映像文件的方法是使用 netcat，这也是一款应用共享程序，用于将一个命令的输出转发到网络上。使用此方法需要两个正在运行的用户终端或 ssh 会话界面，在此我们用 Session0 和 Session1 来表示。所有的 Session0 命令都将在 Ubuntu 虚拟机上执行，因此我们无法从 Session0 进入 Android 终端设备的 shell。所有需要在 Android 终端设备上的 shell 中执行的命令都在 Session1 中发生。

我们首先使用 adb 端口转发功能来实现网络中两个节点之间的连接：

更多资源请访问稀酷客(www.ckook.com)

```
SESSION0
--------------
ahoog@ubuntu:~$ adb forward tcp:31337 tcp:31337
```

以上这个命令实质上使用端口 31337 将 Android 终端设备与 Ubuntu 虚拟机进行连接。然后，我们在 Android 终端设备上执行 nanddump 命令，并将得到的结果输送给 netcat 作为输入：

```
SESSION1
--------------
ahoog@ubuntu:~$ adb shell
/ # /dev/AFPhyiscal/nanddump /dev/mtd/mtd8ro | /dev/AFPhyiscal/nc -l -p 31337
ECC failed: 0
ECC corrected: 0
Number of bad blocks: 0
Number of bbt blocks: 0
Block size 131072, page size 2048, OOB size 64
Dumping data starting at 0x00000000 and ending at 0x105c0000...
```

现在，Android 终端设备将 nanddump 的数据通过 netcat 发送过来，我们需要在 Ubuntu 虚拟机端接收这些数据：

```
SESSION0
--------------
ahoog@ubuntu:~$ nc 127.0.0.1 31337 > af-book-mtd8.nanddump
```

nanddump 命令结束后直接退出，不遗留任何多余的数据，对于 Ubuntu 虚拟机来说，netcat 也如此。我们可以使用 ls 命令来验证工作站端的确收到了 nanddump 的数据：

```
SESSION0
--------------
ahoog@ubuntu:~$ ls -lh af-book-mtd8.nanddump
-rw-r--r-- 1 ahoog ahoog 270M 2011-02-26 20:58 af-book-mtd8.nanddump
```

此刻就可以继续对取证调查所需要的 MTD 分区进行物理映像，至少要获取用户数据分区和缓存分区。

在第 7 章中，我们会展示一个用于从 nanddump 抽取 OOB 数据的程序，以帮助对取证数据进行处理，如文件雕复。由于可以使用此方法生成 dd 映像，就不再需要使用 Android 终端设备来获取 dd 映像。然而，dd 属于 Android 系统的一部分，因此，我们提供的以下实例与使用 nanddump 的情况类似，不同的是使用 dd，因而也无法用于获取 OOB 数据。以下实例针对的是参考的 HTC Incredible：

```
SESSION0
--------------
ahoog@ubuntu:~$ adb forward tcp:31337 tcp:31337

SESSION1
--------------
ahoog@ubuntu:~$ adb shell
$ su
# cat /proc/mtd
dev:    size   erasesize  name
```

更多资源请访问稀酷客(www.ckook.com)

```
mtd0: 000a0000 00020000 "misc"
mtd1: 00480000 00020000 "recovery"
mtd2: 00300000 00020000 "boot"
mtd3: 0f800000 00020000 "system"
mtd4: 000a0000 00020000 "local"
mtd5: 02800000 00020000 "cache"
mtd6: 09500000 00020000 "datadata"
# dd if=/dev/mtd/mtd6 bs=4096 | /dev/AFPhyiscal/nc -l -p 31337
38144+0 records in
38144+0 records out
156237824 bytes transferred in 182.898 secs (854234 bytes/sec)

SESSION0
--------------
ahoog@ubuntu:~$ nc 127.0.0.1 31337 > dd of=htc-datadata.dd bs=4096
```

由于 Android 终端设备、MTD、YAFFS2 以及其他方面存在着的不同，挂载获取的 nanddump 映像并抽取逻辑文件不可能总是可行。由于已经有了足够的访问权限，最好就是去抽取希望得到的逻辑数据，可以使用递归 adb pull 去做到，这是因为在终端设备上运行的 adb 后台进程已经具有 root 访问权限。另外也可以使用类似 tar 这样的应用共享程序将所有数据都复制到一个档案文件中去。在以上任何一种情况下都需要确认已经挂载了正确的文件系统。有些修改后的修复分区给用户提供了一个挂载文件系统的接口。然而，也可以在命令行完成以上工作，将文件系统以只读的方式挂载。在所参考的摩托罗拉 Droid 手机上，执行以下命令：

```
SESSION1
--------------
/ # mount -o ro -t yaffs2 /dev/block/mtdblock8 /data
/ # mount -o ro,remount -t yaffs2 /dev/block/mtdblock7 /cache
/ # mount
rootfs on / type rootfs (rw)
tmpfs on /dev type tmpfs (rw,mode=755)
devpts on /dev/pts type devpts (rw,mode=600)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/block/mtdblock7 on /cache type yaffs2 (ro)
/dev/block/mmcblk0p1 on /sdcard type vfat
(rw,nodev,noatime,nodiratime,fmask=0000,dmask=0000,allow_utime=0022,
codepage=cp437,iocharset=iso8859-1,errors=remount-ro)
/dev/block/mtdblock8 on /data type yaffs2 (ro)
```

第一个命令将 /data 分区以只读的方式挂载。第二个命令将已经挂载的“/cache”目录重新以只读的方式挂载。然后就可以执行 adb pull 命令：

```
ahoog@ubuntu:~$ adb pull /data/data/com.android.providers.telephony sms
pull: building file list...
pull: /data/data/com.android.providers.telephony/databases/mmssms.db ->
sms/databases/mmssms.db
pull: /data/data/com.android.providers.telephony/databases/telephony.db ->
sms/databases/telephony.db
2 files pulled. 0 files skipped.
137 KB/s (44032 bytes in 0.311s)
```

更多资源请访问稀酷客(www.ckook.com)

最后一项工作就是将文件和目录使用 tar 应用共享程序打包在一个存档文件中，此存档文件通常也被称为 tarball：

```
SESSION1
--------------
ahoog@ubuntu:~$ adb shell
/ # /dev/AFPhyiscal/tar cpv -f - /data/data/com.android.providers.telephony
/cache | /dev/AFPhyiscal/nc -l -p 31337
tar: removing leading '/' from member names
data/data/com.android.providers.telephony/
data/data/com.android.providers.telephony/lib/
data/data/com.android.providers.telephony/databases/
data/data/com.android.providers.telephony/databases/telephony.db
data/data/com.android.providers.telephony/databases/mmssms.db
cache/
cache/recovery/
cache/recovery/log
cache/lost+found/

SESSION0
--------------
ahoog@ubuntu:~$ nc 127.0.0.1 31337 > af-book-droid-files.tar
```

在此例中，我们向 tar 提交了两个我们希望存档的目录：在“/data/data”目录下用于保存短信消息 / 多媒体（彩信）消息的目录以及“/cache”目录。我们将此存档文件通过网络发送给了 Ubuntu 虚拟机。然而，我们也可以简单地将这个存档文件保存在 SD 卡上。

在拥有了 Android 终端设备上的 root 访问权限，并且充分了解了终端设备的体系架构后，就可以使用 nanddump、dd、tar、netcat 以及 adb 来生成取证映像，或者更简单地去复制数据，用于进行取证分析。

6.6 本章小结

目前已经有好几项技术能够用于对 Android 终端设备进行取证数据获取。如果终端设备使用口令进行保护，则必须设法攻克或绕过这一层保护，才能获取数据。虽然已经有很多种技术能够用于攻克口令，然而在任何情况下都取得成功是不可能的。一旦获得对终端设备的访问，取证分析师就可以选择主要面向未被删除而通过内容提供商访问数据的逻辑技术或者是更加深入但却在技术上更加富有挑战性的物理技术。虽然物理获取能够产生更多的数据，此方法通常需要更加成熟的分析技术，这些将在第 7 章中进行讨论。

6.7 参考资料

ACPO Good Practice Guide for Computer-Based Electronic Evidence—7Safe Information Security. (n.d.). Retrieved February 19, 2011, from http://7safe.com/electronic_evidence/index.html#.

Android & Windows Phone: Tablets, Apps, & ROMs @ xda-developers. (n.d.). Retrieved February 23, 2011, from http://www.xda-developers.com/.

更多资源请访问稀酷客(www.ckook.com)

Aviv, Gibson, Mossop, Blaze, & Smith. (n.d.). Smudge attacks on smartphone touch screens. Retrieved February 21, 2011, from http://www.usenix.org/events/woot10/tech/full_papers/Aviv.pdf.

Cannon, T. (n.d.). Android lock screen bypass. Retrieved February 21, 2011, from http://thomascannon.net/blog/2011/02/android-lock-screen-bypass/.

dc3dd. (n.d.). Retrieved February 22, 2011, from http://dc3dd.sourceforge.net/.

Government Employment & Payroll. (n.d.). Retrieved February 19, 2011, from http://www.census.gov/govs/apes/.

[Guide] Flashing Linux Motorola's with RSD Lite Versions. (n.d.). Retrieved February 24, 2011, from modmymobile.com/forums/8-guides-downloads-forum-suggestions/218651-guide-flashing-linux-motorolas-rsd-lite-versions.html.

HTC—Developer Center. (n.d.). Retrieved February 28, 2011, from http://developer.htc.com/adp.html.

IEEE SA—1149.1—1990—IEEE Standard Test Access Port and Boundary-Scan Architecture. (n.d.). Retrieved February 23, 2011, from http://standards.ieee.org/findstds/standard/1149.1-1990.html.

RerWare, LLC: Android Backup and BlackBerry Backup—MyBackup Pro. (n.d.). Retrieved February 22, 2011, from http://www.rerware.com/.

更多资源请访问稀酷客(www.ckook.com)

第 7 章

Android 应用与取证分析

本章要点

- 分析技术
- FAT 取证分析
- YAFFS2 取证分析
- Android 应用分析与参考信息

7.1 绪论

本书的前几章已经讨论了很多内容，不但有关于 Android 终端设备的历史和架构方面的内容，还有关于文件系统、设备安全方式以及从终端设备上获取数据的方法论方面的详细内容。但是，对数据不进行上下文关联与分析就只是噪声。很多应用于传统取证调查的技术对 Android 取证分析也是有效的。

7.2 分析技术

本节首先对取证分析技术进行概述，然后再展示对具体的文件系统进行分析的整个过程。

7.2.1 时间序列分析

时间序列分析是任何取证调查中的一个关键部分，这是因为任何事件的发生时间几乎总是有意义的。已经有很多方法可以用于构造出一个取证的时间序列。但是，除非使用特殊的软件进行构造，否则这个过程会很烦琐与乏味。本章的后续内容会详细介绍几种软件技术，包括 Sleuth Kit 和 log2timline 提供的免费、开源的取证应用程序。还有其他的取证工具也可以用来构造时间序列。

对于支持的文件系统（如 SD 卡以及嵌入式多媒体卡（eMMC）上的 FAT16/FAT32 文件系统），已经存在数种可以用于构建时间序列的工具。然而，目前没有任何分析工具支持 YAFFS2，因而构建一个时间序列需要大量的人工分析。

时间序列信息主要来源于文件系统的元数据，包括修改（文件元数据）、访问、改变

更多资源请访问稀酷客(www.ckook.com)

（文件内容）及创建。这些元数据经常被称为媒介访问控制（MAC）时间，有时会被称为 MACB 时间，其中“B”代表文件在什么时间被创建（生成）。文件系统跟踪不同的时间戳，因此在取证分析的过程中需要考虑细微的差别。

例如，微软公司的 FAT 文件系统已经成为许多取证调查和分析的对象。微软开发者网络（Microsoft Developers Network，MSDN）在 FAT 和 NTFS 文件系统中都对文件时间提供了详细的介绍，声明如下：

> 并不是所有的文件系统都可以记录创建时间和最后一次访问时间，也并不是所有的文件系统都以同样的方式记录这些时间。例如，FAT 文件系统中的创建时间分辨率为 10 毫秒（即精确到 10 毫秒），但是写入时间分辨率为 2 秒，访问时间分辨率为 1 天，因此，实际数据是访问时间。NTFS 文件系统中将对上一次访问文件时间的更新延迟最长可达到 1 小时。
>
> (File Times, n.d.)

Andre Ross 在个人的 digfor（DIGital FORensics）博客中使用图形表达对以上延迟与精确度的情况进行了一个有用的说明（http://digfor.blogspot.com/2008/10/time-and-timestamps.html），如图 7.1 所示（Ross, A., n.d.）。

为了进一步说明 FAT 文件系统中时间戳的工作原理，数字取证与突发事件响应公司 cmdLab 发表了一篇题目为“Misinterpretation of File System Timestamps”的博文（Casey, E., n.d.），该文章对 FAT 文件系统中的时间戳进行了详细说明。

很容易混淆的地方是，FAT 文件系统表达创建时间戳和最后一次写入时间戳的方式稍有不同。最后一次写入时间戳使用 32 比特位的小尾端值来表达，含义如下：

```
24                  16                  8                   0
+-+-+-+-+-+-+-+-+ +-+-+-+-+-+-+-+-+ +-+-+-+-+-+-+-+-+ +-+-+-+-+-+-+-+-+
|Y|Y|Y|Y|Y|Y|Y|M| |M|M|M|D|D|D|D|D| |h|h|h|h|h|m|m|m| |m|m|m|s|s|s|s|s|
+-+-+-+-+-+-+-+-+ +-+-+-+-+-+-+-+-+ +-+-+-+-+-+-+-+-+ +-+-+-+-+-+-+-+-+

\_____________/\_________/\_________/ \_________/\_____________/\_________/
year          month       day         hour        minute        second
```

以下面这个 FAT 文件系统文件夹条目为例，最后一次的写入时间用黑体字表达：

```
$ icat /dev/sdb1 353884 | xxd
0000000: 2e20 2020 2020 2020 2020 2030 004f b079 . 0.O.y
0000010: 763a 763a 0000 b579 763a a502 0000 0000 v:v:…yv:……
0000020: 2e2e 2020 2020 2020 2020 2010 004f b079 .. ..O.y
0000030: 763a 763a 0000 b079 763a 6605 0000 0000 v:v:…yv:f….. .
0000040: 4173 0061 006c 0076 0065 000f 009e 6e00 As.a.l.v.e….n.
0000050: 6500 7700 3400 2e00 6700 0000 6900 6600 e.w.4…g…i.f.
0000060: 5341 4c56 454e 7e31 4749 4620 0075 78b9 SALVEN~1GIF .ux.
0000070: 753a 763a 0000 78b9 753a 9212 c1d4 0000 u:v:..x.u:……
0000080: 4269 0066 0000 00ff ffff ff0f 0014 ffff Bi.f……
0000090: ffff ffff ffff ffff ffff 0000 ffff ffff ……
```

将这个小尾端值转换为大尾端 3a 75 b9 78，以二进制表达的值为：

更多资源请访问稀酷客(www.ckook.com)

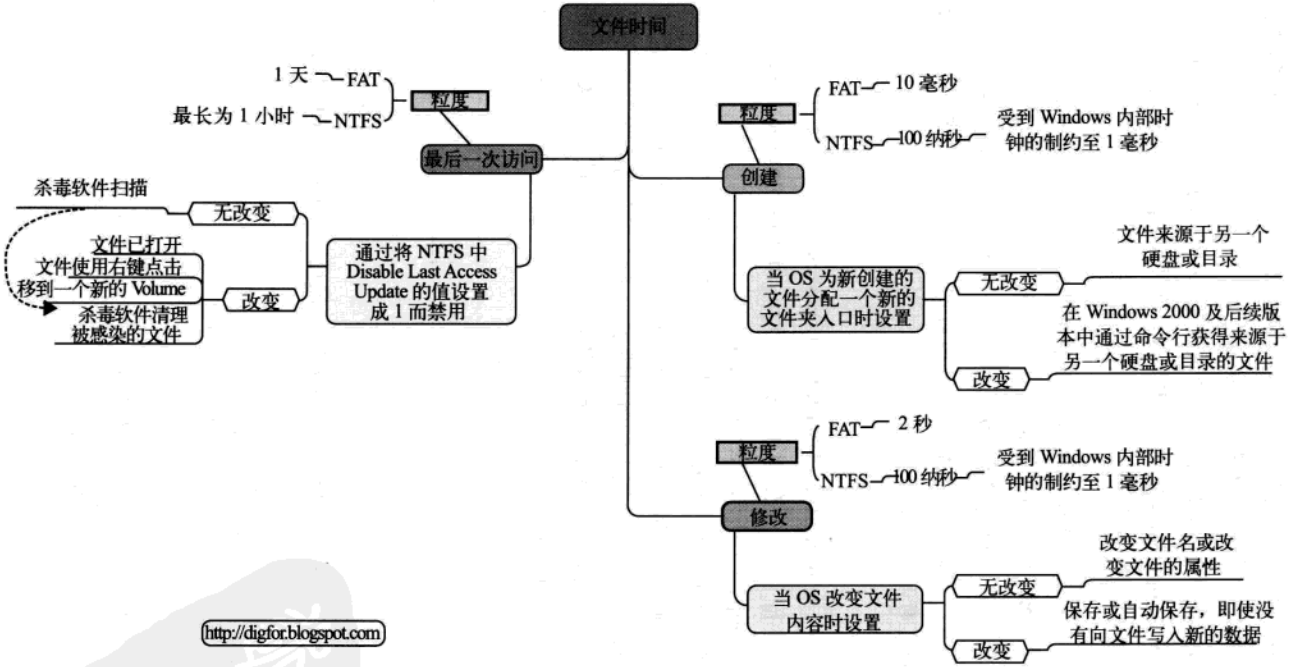


图 7.1　FAT 与 NTFS 中的时间及时间戳

更多资源请访问稀酷客(www.ckook.com)

00111010 01110101 10111001 01111000

以上的值转换为时间戳后为 2009.03.21 23:11:48，具体解释如下：

- 7 位 = 0011101 = 29 年（自 1980 年以来）；
- 4 位 = 0011 = 3 月；
- 5 位 = 10101 = 21 天；
- 5 位 = 10111 = 23 时；
- 6 位 = 001011 = 11 分；
- 5 位 = 11000 = 24 = 48 秒。

需要注意的是，5 位长度不够存储 60 秒，因此，最后写入时间戳只能以 2 秒为单位进行递增，也总是偶数秒。

虽然创建时间也是按照同样的方式进行计算，此时间里使用了另外 8 位来表达百分之一秒。在同样的以上列表中，创建时间戳与最后一次修改时间戳完全相同，只是额外多出了一个字节（75 78 b9 75 3a）。这个额外字节的数值等于 1/117 秒，使创建时间变为 2009.03.21 23:11:49.17。因此，创建时间可能是奇数秒，分辨率为 10 毫秒。表 7.1 列出了 TSK 在所支持的系统中对 MACB 时间戳的定义（Carrier, B., n.d.）。

本书所用的时间序列分析工具是 Sleuth Kit（TSK），该工具支持几种文件系统。表 7.1 展示了 TSK wiki 提供的信息，以定义所支持的文件系统的 MACB 时间戳的意义。

不幸的是，至少到 TSK 3.2.1 版本为止，TSK 还无法完全支持 YAFFS2。然而，可以使用其他的一些方式来获得此方面的信息。对 Android 来说，TSK 能够用于对某些终端设备中使用的 FAT 文件系统和 EXT3 文件系统进行有效分析。TSK 还无法完全支持 EXT4，但是此软件更新频繁，取证检查师应该经常登录到 http://www.sleuthkit.org/ 网站上去跟踪最新的进展情况。

7.2.2　文件系统分析

正如贯穿于全书的讨论中所指出的，取证调查的重点对象就是 Android 文件系统中的目录和文件。本章的最后一节以及《安卓应用分析与参考》一书将对 Android 应用进行详细的分析。将通过分析获得的结果与第 4 章中展示的技术结合在一起会为取证调查提供最关键的结果。

表 7.1　MAC 在不同系统中的含义

文件系统	m	a	c	b
Ext 2/3	已修改	已访问	已改变	不适用
FAT	已写入	已访问	不适用	已创建
NTFS	文件已修改	已访问	MFT 已修改	已创建
UFS	已修改	已访问	已改变	不适用

在取证调查中，很多目录需要被检查。随着时间的推移，由于 Android 终端设备技术的快速发展，取证检查师需要对检查列表进行扩充，收录进新的目录和文件。应对此

更多资源请访问稀酷客(www.ckook.com)

问题的方法就是首先运行以下的命令，以确定系统中都挂载了哪些文件系统、挂载在哪些点上以及文件系统的类型。为了展示的目的，我们使用 4 种不同的 Android 手机，包括 T-Mobile/HTC G1、谷歌 Nexus One、摩托罗拉 Droid 以及 HTC Incredible 手机各一部。

首先，我们看一下在运行 Android1.5 版本的 G1 手机上执行挂载命令的输出结果：

```
ahoog@ubuntu:~$ adb shell mount
rootfs / rootfs ro 0 0
tmpfs /dev tmpfs rw,mode=755 0 0
devpts /dev/pts devpts rw,mode=600 0 0
proc /proc proc rw 0 0
sysfs /sys sysfs rw 0 0
tmpfs /sqlite_stmt_journals tmpfs rw,size=4096k 0 0
/dev/block/mtdblock3 /system yaffs2 ro 0 0
/dev/block/mtdblock5 /data yaffs2 rw,nosuid,nodev 0 0
/dev/block/mtdblock4 /cache yaffs2 rw,nosuid,nodev 0 0
```

这一部 G1 手机运行着没有经过改动的 stock 固件，没有插入任何 SD 卡，从而我们面对着 4 个文件系统需要进行取证检查，这 4 个文件系统在以上的输出中用黑体进行了标注。

同样，我们可以看一下在运行 Android2.1-update1 版本的 Nexus One 手机上执行挂载命令的输出结果：

```
ahoog@ubuntu:~$ adb shell mount
rootfs on / type rootfs (ro,relatime)
tmpfs on /dev type tmpfs (rw,relatime,mode=755)
devpts on /dev/pts type devpts (rw,relatime,mode=600)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
tmpfs on /sqlite_stmt_journals type tmpfs (rw,relatime,size=4096k)
none on /dev/cpuctl type cgroup (rw,relatime,cpu)
/dev/block/mtdblock3 on /system type yaffs2 (ro,relatime)
/dev/block/mtdblock5 on /data type yaffs2 (rw,nosuid,nodev,relatime)
/dev/block/mtdblock4 on /cache type yaffs2 (rw,nosuid,nodev,relatime)
/sys/kernel/debug on /sys/kernel/debug type debugfs (rw,relatime)
/dev/block//vold/179:1 on /sdcard type vfat
(rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,
dmask=0702,allow_utime=0020,codepage=cp437,iocharset=iso88591,shortname=mixed,
utf8,errors=remount-ro)
```

从输出结果可以看到，此部 Nexus One 手机中有 5 个文件系统，都使用黑体字进行了标注，这些文件系统应该成为最初的取证检查重点。另外一个所挂载的文件系统是 SD 卡。

接下来，我们看一下在运行 Android2.2.1 版本的摩托罗拉 Droid 手机上执行挂载命令的输出结果：

```
ahoog@ubuntu:~$ adb shell mount
rootfs / rootfs ro,relatime 0 0
tmpfs /dev tmpfs rw,relatime,mode=755 0 0
devpts /dev/pts devpts rw,relatime,mode=600 0 0
proc /proc proc rw,relatime 0 0
sysfs /sys sysfs rw,relatime 0 0
none /acct cgroup rw,relatime,cpuacct 0 0
tmpfs /mnt/asec tmpfs rw,relatime,mode=755,gid=1000 0 0
none /dev/cpuctl cgroup rw,relatime,cpu 0 0
/dev/block/mtdblock4 /system yaffs2 ro,relatime 0 0
/dev/block/mtdblock6 /data yaffs2 rw,nosuid,nodev,relatime 0 0
```

更多资源请访问稀酷客(www.ckook.com)

```
/dev/block/mtdblock5 /cache yaffs2 rw,nosuid,nodev,relatime 0 0
/dev/block/mtdblock0 /config yaffs2 ro,relatime 0 0
/dev/block/vold/179:1 /mnt/sdcard vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=
0702,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
/dev/block/vold/179:1 /mnt/secure/asec vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=
0702,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
tmpfs /mnt/sdcard/.android_secure tmpfs ro,relatime,size=0k,mode=000 0 0
```

这部摩托罗拉 Droid 手机上有 7 个相关的文件系统。但是，这些文件系统与 Nexus One 上的文件系统存在一些微小的差别。

最后，我们看一下在运行 Android2.2 版本的 HTC Incredible 手机上执行挂载命令的输出结果：

```
ahoog@ubuntu:~$ adb shell mount
rootfs / rootfs ro,relatime 0 0
tmpfs /dev tmpfs rw,relatime,mode=755 0 0
devpts /dev/pts devpts rw,relatime,mode=600 0 0
proc /proc proc rw,relatime 0 0
sysfs /sys sysfs rw,relatime 0 0
none /acct cgroup rw,relatime,cpuacct 0 0
tmpfs /mnt/asec rw,relatime,mode=755,gid=1000 0 0
none /dev/cpuctl cgroup rw,relatime,cpu 0 0
/dev/block/mtdblock3 /system yaffs2 rw,relatime 0 0
/dev/block/mmcblk0p1 /data ext3
rw,nosuid,noatime,nodiratime,errors=continue,data=writeback 0 0
/dev/block/mtdblock6 /data/data yaffs2 rw,nosuid,nodev,relatime 0 0
/dev/block/mmcblk0p2 /cache ext3
rw,nosuid,nodev,noatime,nodiratime,errors=continue,data=writeback 0 0
tmpfs /app-cache tmpfs rw,relatime,size=8192k 0 0
/dev/block/vold/179:9 /mnt/sdcard vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=070
2,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
/dev/block/vold/179:9 /mnt/secure/asec vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=070
2,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
tmpfs /mnt/sdcard/.android_secure tmpfs ro,relatime,size=0k,mode=000 0 0
/dev/block/vold/179:3 /mnt/emmc vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,dmask=070
2,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,
errors=remount-ro 0 0
```

与其他几个文件相比较，HTC Incredible 中相关的文件系统最多，共有 9 个。可以看出，对于不同的 Android 终端设备以及不同的 Android 版本，相关的文件系统都会有所不同。我们将以上的分析结果列在表 7.2 中，为下一步对文件系统进行取证检查打下一个良好基础。

从正面来看，在这些文件系统被分配以后，即使对不支持的文件系统，如 YAFFS2，文件都可以复制到另外一个媒介上，取证检查师再使用已有并且了解的取证分析工具与技术来检查。例如，“/data/data”下的数据可以首先从一个 Android 终端设备上复制到取证服务器上，然后再对这些数据直接进行检查。

更多资源请访问稀酷客(www.ckook.com)

表 7.2 在取证调查中涵盖的文件系统

挂载点	文件系统类型	相关程度
/proc	proc	使用“cat”命令在手机上查看。查询与系统相关的元数据，如文件系统统计数据
/data/data (on older systems, entire /data is 1 partition/file system)	YAFFS2	几乎所有的应用数据
/data (on newer phones /data can be further segemented)	EXT3/EXT4/YAFFS2	应用与系统数据，除去存储在 /data/data 下的应用数据
/cache	YAFFS2/EXT3	在某些应用和系统中使用的缓存文件系统
/mnt/asec	tmpfs	解密后的应用 .apk 文件，以加密的方式存储在 SD 卡上，但是在解密后存储于此，使运行系统能够访问与使用
/app-cache	tmpfs	临时文件系统，com.android.browser（在 HTC Incredible 上）将此作为缓存空间，随着时间的推移，其他文件系统也可能会使用此目录
/mnt/sdcard	vfat	可取出的 SD 卡上的 FAT32 文件系统
/mnt/emmc	vfat	eMMC 卡上的 FAT32 文件系统

7.2.3 文件雕复

文件雕复（file carving）是指对具体的文件类型进行查找与获取的过程，在此过程中，文件的内容被作为二进制数值对待，得到的结果常常是整个磁盘或分区的取证映像。文件雕复的工作原理是基于已知的文件头信息，通过检查二进制数据而对文件进行识别。如果从文件的格式定义中知晓文件尾，文件雕复将从文件头开始扫描，直至找到文件尾（或者配置文件中设置的文件最大长度），然后将雕复的文件保存在磁盘上，以备随后对其进行进一步检查。

传统的文件雕复技术要求映像中的数据按照顺序进行存储，因此在文件分段存储的情况下无法获得整个文件。将文件分段存储的理由很多，这是由于将文件保存在非易失存储器的过程随着文件系统类型的不同而变化，而且还受到不同类型内存的影响很大，如 NAND 闪存。这也就意味着非常大的文件（如视频文件）很难恢复。

新一代的文件雕复技术正在研究及开发中，以应对由碎片文件所产生的制约。地处美国纽约的取证方案公司 Digital Assembly 开发出的一项名为 SmartCarving 的技术。此技术对几个常见的文件系统（不幸的是，YAFFS2 不属于此列）中的碎片特性进行描述，再基于这些特性对分段存储的相片完成雕复。此公司的产品 Adroit Photo Forensics 甚至还能够对未知的文件系统进行雕复（Digital Assembly, n.d.）。

scalpel 由 Golden G. Richard III 开发，是一款非常流行、开源和高性能的数据文件雕复工具（Scalpel, n.d.）。Saclpel 从配置文件中读取满足要求的文件头和文件尾定义，使它能够从原始映像中抽取出文件。Scalpel 工具独立于任何文件系统，可以应用于 FATx、NTFS、EXT2/EXT3、HFS 或原始分区。Scalpel 用 C 语言编写，可以运行在 Linux、

更多资源请访问稀酷客(www.ckook.com)

Windows、OS X 以及其他能够编译 C 代码的操作系统上（Scalpel, n.d.）。

有两种方式可以获得 Scalpel。第一种方式是在取证工作站上使用 apt-get 进行安装：

```
ahoog@ubuntu:~$ sudo apt-get install scalpel
```

这样就可以安装最新的版本。另外一种方式是直接对原代码进行编译，这样就可以在 Linux 或其他平台上安装最新版本，而不需要等待具体的平台维护者更新预先打包的版本。

```
cd ~
wget http://www.digitalforensicssolutions.com/Scalpel/scalpel-1.60.tar.gz
tar xzvf scalpel-1.60.tar.gz
cd scalpel-1.60/
make
```

scalpel 的可执行程序现在就在“~/scalpel-1.60”目录下，文件名为 scalpel。此外，在同一个目录下还存在一个样本的 scalpel.conf 文件，scalpel 的运行以及扩展对文件定义的支持都依赖于此配置文件。以下是 Android 终端设备上一个简单的 scalpel.conf 文件：

```
#ext  case   size    header    footer
gif   y   5000000   \x47\x49\x46\x38\x37\x61   \x00\x3b
gif   y   5000000   \x47\x49\x46\x38\x39\x61   \x00\x3b
jpg   y   200000000   \xff\xd8\xff\xe0\x00\x10   \xff\xd9
jpg   y   5000000   \xff\xd8\xff\xe1   \x7f\xff\xd9

png   y   102400      \x50\x4e\x47?   \xff\xfc\xfd\xfe
png   y   102400      \x89PNG

db    y   409600   SQLite\x20format

email y   10240   From:

doc   y   10000000  \xd0\xcf\x11\xe0\xa1\xb1\x1a\xe1\x00\x00

\xd0\xcf\x11\xe0\xa1\xb1\x1a\xe1\x00\x00 NEXT
doc   y   10000000  \xd0\xcf\x11\xe0\xa1\xb1

htm   n   50000   <html          </html>

pdf   y   5000000   %PDF  %EOF\x0d   REVERSE
pdf   y   5000000   %PDF  %EOF\x0a   REVERSE

wav   y   200000   RIFF????WAVE
amr   y   200000   #!AMR

zip   y   10000000   PK\x03\x04   \x3c\xac

java   y   1000000   \xca\xfe\xba\xbe
```

可以看到，在这个配置文件的头中定义了文件扩展部分或文件类型（如果大小写敏感的话）、需要雕复的最大长度、文件头定义（使用 ASCII、十六进制或其他可支持的表达方式）以及文件尾（如果存在的话）。需要被雕复的目标文件类型不需要对每一项设置都进行定义。更多的信息可以在所下载的源程序文件的一个实例配置文件中获得，因为其中还包含很多功能强大的选项。现在，Ubuntu 工作站上已经有了文件雕复所需要的软件，

更多资源请访问稀酷客(www.ckook.com)

更多的内容将在 FAT32 与 YAFFS2 章节中讨论。

需要指出的是，大量的文件签名已经形成。数字取证领域的独立咨询师与实践者 Gary Kessler 在个人网站上主动地维护着一份文件签名列表（Kessler, G., n.d.）。这个他称为“神奇文件”的列表安装在大多数 UNIX 系统的 Ubuntu 工作站上，位于“/usr/share/file/magic”。在工作站上，可以执行将文件名作为参数的“file”命令，基于神奇文件中的签名来确定文件的类型。

一个简单的实例是来确定一下在“/mnt/emmc/.Trashes”目录下一个文件名为“._501”的文件类型，因为从文件名上无法直观地看出此文件的类型（当然，有些人会通过改变文件名的扩展名来隐藏文件类型；然而，通过查看文件签名，就可以很容易确定此文件的类型）：

```
#ahoog@ubuntu:~/htc-inc/mnt/emmc$ file ./.Trashes/._501
./.Trashes/._501: AppleDouble encoded Macintosh file
```

因此，在 eMMC 上的 Trash（回收站）中有一个 OS X 文件，告诉我们曾有人将此 Android 终端设备连接到一台 Mac 计算机上。

7.2.4 strings 命令

在 Ubuntu 工作站执行 strings 命令的默认结果是，从任何一个文本或二进制文件中获取长度至少为 4 个字符的可打印 ASCII 字符串。虽然这个技术并不是十分简洁与完善，它却能够非常有效地用于对二进制数据进行快速搜索，以确定文件中是否包含令人感兴趣的信息。

strings 命令中有一些选项会对执行此命令得到的输出结果产生极大的影响。首先，我们看一下此命令手册页（手册）中的概要解释部分：

```
STRINGS(1)                  GNU Development Tools                  STRINGS(1)

NAME
       strings - print the strings of printable characters in files.

SYNOPSIS
       strings [-afovV] [-min-len]
               [-n min-len] [--bytes=min-len]
               [-t radix] [--radix=radix]
               [-e encoding] [--encoding=encoding]
               [-] [--all] [--print-file-name]
               [-T bfdname] [--target=bfdname]
               [--help] [--version] file
```

这里有几个选项是在使用此命令时总会用到的。首先，“--all”选项将使此命令对整个文件进行检查（对于某些文件，还是只对文件的特定部分进行检查）。其次，“--radix=”选项使此命令将所发现的字符串在文件中的相对偏移值打印出来。这些对于将字符串进行组合以及使用十六进制编辑器在文件中查看发现的证据时都非常重要。这个 radix 选项可以用于将偏移值以八进制（--radix=o）、十六进制（--radix=x）或十进制（--radix=d）的方式打印出来。对于大多数的十六进制编辑器来说，应该考虑使用十六进制或者十进制偏

更多资源请访问稀酷客(www.ckook.com)

移值。

strings 的另外一个非常重要的选项是对字符的编码进行控制，以支持 Unicode 字符从小到大的排列顺序或者从大到小的排列顺序：

```
--encoding=encoding
    Select the character encoding of the strings that are to be found.
    Possible values for encoding are: s = single-7-bit-byte characters
    (ASCII, ISO 8859, etc., default), S = single-8-bit-byte characters,
    b = 16-bit bigendian, l = 16-bit littleendian, B = 32-bit
    bigendian, L = 32-bit littleendian.  Useful for finding wide
    character strings. (l and b apply to, for example, Unicode
    UTF-16/UCS-2 encodings).
```

这一点也非常重要，这是因为 Android 不但本身就支持 Unicode，而且还允许对手机的默认语言要求以 Unicode 进行表达的手机进行调查。以下是几个源于前面那个 dd 映像的实例：

```
ahoog@ubuntu:~$ strings --all --radix=x htc-datadata.dd | less
   880a htcchirp.db
   891d ^XMp4XM
   900a unlinked
   980a deleted
   a00a htcchirp.db-journal
```

在此例中，第一行的结果被忽略了。但是可以看到，dd 映像对 htcchirp.db 文件中偏移 0x880A 进行了引用，并且在此之后，很快又可以看到 unlinked (0x900A)、deleted (0x980A)，最后是 htcchirp.db-journal (0xA00A)。那么这些是什么意思呢？看上去 htcchirp.db 数据库被修改。在修改过程中，生成了一个特殊的文件（htcchirp.db-journal，是一个期刊文件），用以对更新进行管理，并且在修改没有成功完成的情况下允许废弃所有的改变。此期刊文件在没有继续存在的必要后就被删除。这些信息对于了解发生的事情以及被删除的数据可能存在何处非常重要（这个期刊文件记录了被更新的 SQLite 页面的完整信息（快照），因此，从前的数值被保存在 NAND 闪存上）。

现在，我们来改变编码参数，并对以下进行查询：

```
ahoog@ubuntu:~$ strings --all --radix=x --encoding=b htc-datadata.dd | less
 c42404 xt=\"chicago tribune\"><b>chicago</b> tribune
</div>",259200,604800],["<div class=\"sg_g\"
        sg_text=\"chicago bears\"><b>chicago</b> bears
</div>",259200,604800],["<div class=\"sg_g\"
        sg_text=\"chicago weather\"><b>chicago</b> weather
</div>",259200,604800],["<div class=\"sg_g\"
        sg_text=\"chicago sun times\"><b>chicago</b> sun times
</div>",259200,604800],["<div class=\"sg_g\"
        sg_text=\"chicago public library\"><b>chicago</b> public library
</div>",259200,604800],
        ["<div class=\"sg_g\" sg_text=\"chicago bul

1943020 gt_bearsh[1295219345,["<div class=\"sg_n\"
sg_url=\"/url?ct=res&amp;oi=s
        uggest_nav&amp;q=http://www.bearshare.com/&amp;sa=X&amp;
source=suggest&amp;usg=AFQjCNGP71yyDMU
        jIhmvN-DN2Tm7yPYCAA\"><span style=\"display:inline;\">Free Music
```

更多资源请访问稀酷客(www.ckook.com)

```
Downloads - Download Free MP3
        Music - BearShare.com Music</span> <br/><span
style=\"display:inline;\">www.bearsha
```

在此例中，我们对十六位的大尾端字符进行查询。对于此查询，在结果中存在两例。在偏移值为 0xC42404 的第一例中，可以清楚地看到发生了与芝加哥有某种关联的活动。取证检查师就可以打开一个十六进制的编辑器，找到偏移值 0xC42404，然后对此项目前后的数据进行进一步分析，试图更好地了解发生的事情。

同样有趣的是，下一个偏移值为 0x1943020 的项目，此项目中引用 bearshare.com 网站，还有一个看上去像是时间戳的数据。通过对数据 1295219345 进行基于 UNIX 格式的日期 / 时间转换，我们可以得到格林威治标准时间 2011 年 1 月 16 日（星期天）23 时 09 分 05 秒。同样，取证检查师需要对数据进行更进一步的分析，才可以对任何发现进行确认，但是一个很好的假设是在 Android 终端设备上此时发生了某些与 Web 相关的活动。

最后的一个例子（其他的编码就留给读者作为练习）揭示了以下情况：

```
ahoog@ubuntu:~$ strings --all --radix=x --encoding=l htc-datadata.dd | less
18451a8 rgc:0:lat41.8786
18451d2 rgc:0:lon-87.6359
18451fc rgc:1:last1288470018632
1845230 rgc:0:rgcOak Park, IL
1845260 rgc:1:acc912
```

这是一个非常好的例子，因为它不但包括了一个（有效的）经度和纬度，而且还包括了一个时间戳（自 1970 年以来所经历的毫秒，而不是秒），转换为格林威治标准时间：2010 年 10 月 30 日（星期六）20 时 20 分 18 秒。

strings 是一个功能非常强的命令，当与查询和过滤结合使用时，可以快速地确定电话号码、姓名、地点、全球定位系统（GPS）坐标、日期以及很多其他信息是否能够很容易地从数据文件中被抽取出来。

7.2.5 十六进制：取证分析师的好朋友

在很多取证调查中，基于物理获取之后的逻辑获取或逻辑文件系统分析通常可以为案件提供足够的数据。然而，某些案件还是需要更深入的分析，以找到已经被删除的文件或未知的文件结构。当标准的取证工具对文件系统提供很少或不提供支持时，这种分析也就变得非常必要，一个典型的例子就是 YAFFS2 文件系统。

可以理解的是，很多取证分析是倾向于不要进行更深入的分析，这是因为进行这种分析需要耗费大量的时间，非常烦琐，还需要对数据结构有一定程度的理解和求知欲。然而，这种分析所产生的结果通常非常令人惊讶。不但可以了解个案的一些重要信息，而且通常所学到的知识还可以应用于将来的案件中。

基于以上这些理由，每一个取证分析师在必要时都应该愉快地去使用十六进制编辑器。这样就可以使分析师准确地看到存储的是什么数据，寻找其模式，也许还可以去识别

更多资源请访问稀酷客(www.ckook.com)

被删除的数据或者是从前见到过的数据结构。

当然，还是让我们通过一个实例来更好地进行解释。首先，确认在 Ubuntu 工作站上已经安装了以下的软件包：

```
ahoog@ubuntu:~$ sudo apt-get install ncurses-hexedit
```

这是一个非常快的 curses-based（终端）的十六进制编辑器。当然，也可以使用更加熟悉的十六进制编辑器。下一步，我们使用 strings 命令来寻找 /data/data/com.android.providers.telephony/databases 目录下的 mmssms.db 文件，以确定是否存在已经被删除的文本数据包。在此例中，我们已经知道发送给 3128781100 的文本数据包已经从终端设备中被删除。首先，我们使用 strings 命令在 SQLite 文件中查找以上的电话号码：

```
ahoog@ubuntu:~$ strings --all --radix=x mmssms.db | grep 3128781100 | wc -l
417
```

在使用此命令中，我们使用了管道（“|”）操作符，将一个命令执行的输出结果发送给下一个命令作为输入。采用这种方法，我们可以将多个命令串在一起同时使用，轻易地就得到一个非常强大的分析技术。因此，以上的命令完成了以下功能：

1）对 SQLite 数据库执行 strings 命令。

2）对 strings 命令的输出结果执行 grep 命令，此命令基于提供的字符串样本对数据进行过滤。在此例中，过滤用的字符串样本为电话号码。但是，可以通过使用包括数学上的规范表达式使 grep 成为非常强大的查询工具。

3）将 grep 命令的输出结果通过管道机制发送给字数统计程序（“wc”），并指示该程序计算行数，而非字数，最终得到 grep 命令执行结果的总行数。

最后得到的结果是发现此电话号码出现了 417 次；很明显地指出终端设备的确与此电话号码进行过通信。下一步就是要进一步查看通信的内容，因此，不再将 grep 命令的输出结果发送给 wc 程序，而是直接查看 grep 的输出结果，并且还要包括此电话号码后的第一行文字内容，这一点可以通过使用 grep 命令的“-A 1”选项实现（我们还是将输出结果通过管道机制发送给“less”命令，使我们能够进行逐页翻看）：

```
ahoog@ubuntu:~$ strings --all --radix=x  mmssms.db   | grep -A 1 3128781100 |
less
  12108 3128781100
  1211b Activated my phone
--
  14080 3128781100
  14097 Have to meet CPA at 11:30.
--
  16116 3128781100
  16129 Can you try to refresh...
```

因此，我们就知道了一个具有特定号码以及相应内容的数据库。现在让我们使用 SQLite3 来帮助我们更好地了解数据库。在这里，我们将使用命令行（在第 4 章中进行过讨论），但是如果愿意，也可以使用一个带有图形显示前端的 SQLite 浏览程序。

更多资源请访问稀酷客(www.ckook.com)

提示：SQLite 语言参考资料

在本书中谈到好几个 SQLite 工具，读者可以在 http://www.sqlite.org/lang.html 上找到关于 SQLite 语言的参考资料，此网页是一个很好的资料来源。对于不熟悉 SQL 的读者来说，此网页还提供很好的概要介绍。而对 SQL 有一定了解的读者，此参考网页会提供更好的参照，正如此网页的名称所描述的那样，“通过 SQLite 来了解 SQL”。

```
ahoog@ubuntu:~$ sqlite3 mmssms.db
SQLite version 3.6.22
Enter ".help" for instructions
Enter SQL statements terminated with a ";"
sqlite> .tables
addr                incoming_msg          sms
android_metadata    incoming_msg_v2       sr_pending
attachments         part                  threads
canonical_addresses pdu                   threads_list
cbch                pending_msgs          words
drm                 qtext                 words_content
htcmsgs             rate                  words_segdir
htcthreads          raw                   words_segments

sqlite> .schema sms
CREATE TABLE sms (_id INTEGER PRIMARY KEY,thread_id INTEGER,toa INTEGER
DEFAULT 0,address TEXT,person
INTEGER,date INTEGER,protocol INTEGER,read INTEGER DEFAULT 0,status INTEGER
DEFAULT -1,type
INTEGER,reply_path_present INTEGER,subject TEXT,body TEXT,sc_toa INTEGER
DEFAULT 0,report_date
INTEGER,service_center TEXT,locked INTEGER DEFAULT 0,index_on_sim INTEGER
DEFAULT -1,callback_number TEXT,
priority INTEGER DEFAULT 0,htc_category INTEGER DEFAULT 0,cs_timestamp LONG
DEFAULT -1, cs_id TEXT,
cs_synced INTEGER DEFAULT 0, error_code INTEGER DEFAULT 0,seen INTEGER
DEFAULT 0);
<snip>

sqlite> .mode line

sqlite> select * from sms limit 1;
           _id = 5
     thread_id = 3
           toa = 0
       address = 3121111111
        person = 901
          date = 1284137437259
      protocol = 0
          read = 1
        status = -1
          type = 1
reply_path_present = 0
       subject =
          body = Did you have a chance do upload the new classes to the
website?
       sc_toa = 0
```

更多资源请访问稀酷客(www.ckook.com)

```
        report_date =
     service_center =
            locked = 0
      index_on_sim = -1
    callback_number =
          priority = 0
      htc_category = 0
      cs_timestamp = -1
             cs_id =
         cs_synced = 0
        error_code = 0
              seen = 0

sqlite> .quit
```

在以上 SQLite3 对话中，通过运行以下命令来更好地理解数据：

1）SQLite3 mmssms.db：打开数据库以进行查询。

2）.tables：列出数据库中的表。

3）.schema sms：以 sms 表为重点，询问数据库关于此表的结构（schema）。schema 会特别长，但是已经被截短。

4）.mode line：为了更好地观看，将显示模式设置为按行显示。

5）“select * from sms limit 1;”：指示 SQLite3 在屏幕上只显示 sms 表中一个记录（限于一个）的所有列。

6）.quit：退出此程序。

因此，我们现在就可以看到 sms 表中存在多个字段，在电话号码之后，还有一个个人 ID，然后再跟着时间戳。使用十六进制编辑器，我们来看看是否可以从信息中确定激活手机的日期 / 时间戳。首先，我们使用刚才安装好的十六进制编辑器来打开 mmssms.db：

```
ahoog@ubuntu:~$ hexeditor mmssms.db
```

执行此命令的结果就是显示文件的开始部分，以块状的十六进制方式显示，并且在右边的列中显示出可打印的 ASCII 字符串：

```
File: mmssms.db                    ASCII Offset: 0x00000000 / 0x00077FFF (%00)
00000000  53 51 4C 69  74 65 20 66   6F 72 6D 61  74 20 33 00   SQLite format 3.
00000010  04 00 01 01  00 40 20 20   00 00 24 4A  00 00 00 00   .....@  ..$J....
00000020  00 00 00 00  00 00 00 00   00 00 00 3C  00 00 00 01   ...........<....
00000030  00 00 00 00  00 00 00 18   00 00 00 01  00 00 00 3C   ...............<
00000040  00 00 00 00  00 00 00 00   00 00 00 00  00 00 00 00   ................
00000050  00 00 00 00  00 00 00 00   00 00 00 00  00 00 00 00   ................
00000060  00 00 00 00  05 00 00 00   17 03 8D 00  00 00 00 45   ...............E
00000070  03 FB 03 F6  03 F1 03 EC   03 E7 03 E2  03 DD 03 D8   ................
00000080  03 D3 03 CE  03 C9 03 C4   03 BF 03 BA  03 B5 03 B0   ................
00000090  03 AB 03 A6  03 A1 03 9C   03 97 03 92  03 8D 00 00   ................
000000A0  00 00 00 00  00 00 00 00   00 00 00 00  00 00 00 00   ................
000000B0  00 00 00 00  00 00 00 00   00 00 00 00  00 04 81 0C   ................
000000C0  03 07 17 15  15 01 81 7B   74 61 62 6C  65 61 64 64   .......{tableadd
000000D0  72 61 64 64  72 05 43 52   45 41 54 45  20 54 41 42   raddr.CREATE TAB
000000E0  4C 45 20 61  64 64 72 20   28 5F 69 64  20 49 4E 54   LE addr (_id INT
000000F0  45 47 45 52  20 50 52 49   4D 41 52 59  20 4B 45 59   EGER PRIMARY KEY
00000100  2C 6D 73 67  5F 69 64 20   49 4E 54 45  47 45 52 2C   ,msg_id INTEGER,
```

更多资源请访问稀酷客(www.ckook.com)

```
00000110  63 6F 6E 74  61 63 74 5F   69 64 20 49  4E 54 45 47   contact_id INTEG
00000120  45 52 2C 61  64 64 72 65   73 73 20 54  45 58 54 2C   ER,address TEXT,
00000130  74 79 70 65  20 49 4E 54   45 47 45 52  2C 63 68 61   type INTEGER,cha
00000140  72 73 65 74  20 49 4E 54   45 47 45 52  29 84 57 02   rset INTEGER).W.
00000150  07 17 13 13  01 89 15 74   61 62 6C 65  70 64 75 70   .......tablepdup
^G Help   ^C Exit (No Save)   ^T goTo Offset   ^X Exit and Save   ^W Search
```

我们可以按下 Control-T（^T）键跳往文件中的任意偏移部分。此例中，在上一个 strings 命令中包括了“-radix=x”，因此拥有了用十六进制表达的偏移值（0x12108），下面就可以开始进行查找：

```
File: mmssms.db                    ASCII Offset: 0x00000000 / 0x00077FFF (%00)
00000000  53 51 4C 69  74 65 20 66   6F 72 6D 61  74 20 33 00   SQLite format 3.
00000010  04 00 01 01  00 40 20 20   00 00 24 4A  00 00 00 00   .....@  ..$J....
00000020  00 00 00 00  00 00 00 00   00 00 00 3C  00 00 00 01   ...........<....
00000030  00 00 00 00  00 00 00 18   00 00 00 01  00 00 00 3C   ...............<
00000040  00 00 00 00  00 00 00 00   00 00 00 00  00 00 00 00   ................
00000050  00 00 00 00  00 00 00 00   00 00 00 00  00 00 00 00   ................
00000060  00 00 00 00  05 00 00 00   17 03 8D 00  00 00 00 45   ...............E
00000070  03 FB                                                 ...............
00000080  03 D3|          Goto Offset                           ...............
00000090  03 AB|                                                ...............
000000A0  00 00|          Offset:  0x12108                      ...............
000000B0  00 00|                                                ...............
000000C0  03 07|  Hint: Decimal 255 = Hex 0xFF = Octal 0377     ......{tableadd
000000D0  72 61|                                                addr.CREATE TAB
000000E0  4C 45|                                                E addr (_id INT
000000F0  45 47 45 52  20 50 52 49   4D 41 52 59  20 4B 45 59   EGER PRIMARY KEY
00000100  2C 6D 73 67  5F 69 64 20   49 4E 54 45  47 45 52 2C   ,msg_id INTEGER,
00000110  63 6F 6E 74  61 63 74 5F   69 64 20 49  4E 54 45 47   contact_id INTEG
00000120  45 52 2C 61  64 64 72 65   73 73 20 54  45 58 54 2C   ER,address TEXT,
00000130  74 79 70 65  20 49 4E 54   45 47 45 52  2C 63 68 61   type INTEGER,cha
00000140  72 73 65 74  20 49 4E 54   45 47 45 52  29 84 57 02   rset INTEGER).W.
00000150  07 17 13 13  01 89 15 74   61 62 6C 65  70 64 75 70   .......tablepdup
^G/^X/Escape Cancel   ^U Clear input
```

十六进制编辑器反应非常之快，跳到了指定的偏移值：

```
File: mmssms.db                    ASCII Offset: 0x00012108 / 0x00077FFF (%15)
00012100  01 01 00 01  01 01 01 00   33 31 32 31  31 31 31 31   ........31211122
00012110  32 32 01 2A  FC 97 C5 2C   01 FF 02 41  63 74 69 76   22.*...,...Activ
00012120  61 74 65 64  20 6D 79 20   70 68 6F 6E  65 00 00 FF   ated my phone...
00012130  33 31 32 34  34 34 33 33   33 33 00 00  FF 00 FF 00   3124443333......
00012140  7B 05 1C 00  01 01 21 00   05 01 01 01  01 01 00 81   {.....!.........
00012150  0B 01 00 00  01 01 0D 01   01 01 00 01  01 01 03 00   ................
00012160  33 33 33 35  35 35 37 37   37 37 01 2A  FC 8E 48 4B   3335557777.*..HK
00012170  00 01 FF 01  00 44 69 64   20 79 6F 75  20 68 61 76   .....Did you hav
00012180  65 20 61 20  63 68 61 6E   63 65 20 64  6F 20 75 70   e a chance do up
00012190  6C 6F 61 64  20 74 68 65   20 6E 65 77  20 63 6C 61   load the new cla
000121A0  73 73 65 73  20 74 6F 20   74 68 65 20  77 65 62 73   sses to the webs
000121B0  69 74 65 3F  00 00 FF 00   00 FF 00 00  00 81 25 04   ite?..........%.
000121C0  1C 00 01 01  3F 00 05 01   01 01 01 01  00 81 41 01   ....?.........A.
000121D0  00 00 01 01  0D 01 01 01   00 01 01 01  02 00 73 74   ..............st
```

从以上的十六进制数据中可以看到电话号码，在偏移值 0x12111 处结束。此后，我们应该看到一个个人字段，因为在表的设计中，它处于下一列（也被称作 table schema）。如果此个人字段进行了设置，它将指向联络（contact）表，在那里会提供所涉及的人的详

更多资源请访问稀酷客(www.ckook.com)

细情况。在此例中，个人字段没有进行设置，因此 SQLite 不会记录任何数据。然后，让我们看看随后的 6 个字节，它们是 0x012AFC97C52C。把这个数转换成十进制数，我们可以得到 1284138059052。最后，我们可以使用任何一项技术将这个时间数（UNIX 中使用的时间格式，以毫秒为单位）转换成更容易读的日期 / 时间。可以迅速使用的一项技术就是内置的 date 命令。但是，此命令只能处理以秒为单位的 UNIX 时间，而并不是毫秒，因此可以简单地使用除以 1000 的方式来实现（也就是说，忽略最后的 3 位数字）：

```
ahoog@ubuntu:~$ date -d @1284138059
Fri Sep 10 12:00:59 CDT 2010
```

date 命令自动地将时间在当前系统的时间区域显示出来。另外一个进行快速时间转换的方法是使用专门为转换 UNIX 时间而设计的网站 http://www.epochconverter.com/ 来实现以秒或以毫秒表达的 UNIX 时间的转换（Epoch Converter, n.d.）。在转换时，将整个时间戳复制（不需要将其除以 1000）到此网站中的文本框中，然后点击“Timestamp to Human date”（时间戳至用户日期），如图 7.2 所示。

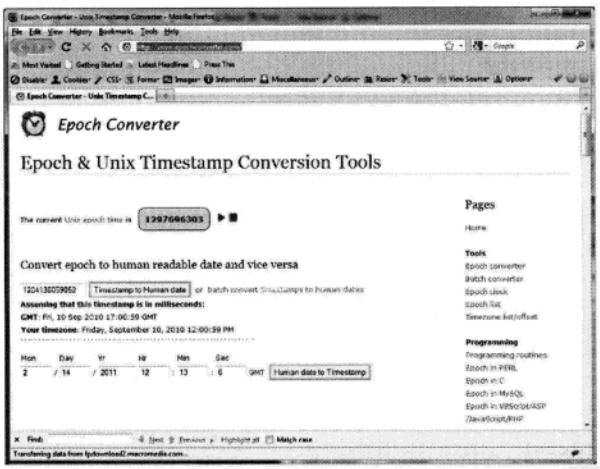


图 7.2　在 http://www.epochconverter.com/ 上进行时间转换

值得提到的最后一个时间戳转换技术就是由 Digital Detective 开发的免费应用共享程序 DCode。Digital Detective 是一家英国的数字取证软件公司。DCode 支持多种不同格式（Digital Detective, n.d.），可以对毫秒进行转换，如图 7.3 所示。

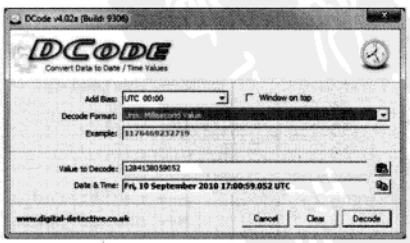


图 7.3　Digital Detective 公司开发的 DCode 时间转换应用程序

此例的目的是为了说明也许只有当取证分析师超越目前所使用的取证软件手段而对数据进行直接检查时才能够获得的那些数据的重要性。对

更多资源请访问稀酷客(www.ckook.com)

于比较熟知的文件系统以及也许是“标准”的案例，以上情况通常没有必要。但是，移动取证，尤其是 Android 取证，是一个非常带有挑战性的领域，这是由于很多文件格式、文件系统、硬件以及软件不但很新（以及没有得到很好的支持），而且还在快速地发生变化。取证分析师在使用十六进制方法后会发现与单纯地使用现有的取证软件相比较能够获得更多的数据。

7.2.6 Android 目录结构

广泛了解 Android 系统的目录结构，对于终端设备的取证分析会有非常大的帮助。为了进行此项分析，我们将 HTC Incredible 中 root 级的 5 个重要目录复制至此，并使用 tree 命令显示在本地工作站上。按照显示出的层次结构，在此对这些目录进行解释。

```
/
├── app-cache
│   └── com.android.browser
│       └── cache
│           └── webviewCache
├── cache
│   ├── lost+found
│   └── recovery
├── data
│   ├── anr
│   ├── app
│   ├── app-private
│   ├── backup
│   ├── btips
│   ├── dalvik-cache
│   ├── data
│   │   ├── com.facebook.katana
│   │   │   ├── cache
│   │   │   │   └── webviewCache
│   │   │   ├── databases
│   │   │   ├── files
│   │   │   ├── lib
│   │   │   └── shared_prefs
│   ├── dontpanic
│   ├── local
│   ├── lost+found
│   ├── misc
│   │   ├── bluetooth
│   │   ├── bluetoothd
│   │   ├── dhcp
│   │   ├── keystore
│   │   ├── lockscreen
│   │   ├── systemkeys
│   │   ├── vpn
│   │   └── wifi
│   ├── property
│   ├── system
│   │   ├── registered_services
│   │   ├── shared_prefs
│   │   ├── sync
│   │   ├── throttle
│   │   └── usagestats
│   └── tombstones
```

更多资源请访问稀酷客(www.ckook.com)

```
├── mnt
│   ├── asec
│   ├── emmc
│   │   ├── Android
│   │   │   └── data
│   │   │       └── com.android.providers.media
│   │   │           └── albumthumbs
│   │   ├── DCIM
│   │   │   └── 100MEDIA
│   │   ├── LOST.DIR
│   │   ├── MP3
│   │   │   ├── People Under the Stairs
│   ├── sdcard
│   │   ├── Android
│   │   │   └── data
│   │   │       ├── com.google.android.apps.maps
│   │   │       │   ├── cache
│   │   │       │   ├── debug
│   │   │       │   └── testdata
│   │   │       └── com.yelp.android
│   │   │           └── cache
│   │   │               └── images
│   │   ├── dcim
│   │   ├── download
│   │   ├── Downloads
│   │   ├── LOST.DIR
│   │   └── tmp
│   └── secure
│       ├── asec
│       └── staging
└── system
    ├── app
    ├── bin
    ├── customize
    │   ├── CID
    │   ├── MNS
    │   └── resource
    ├── etc
    │   ├── bluetooth
    │   ├── clockwidget
    │   ├── dhcpcd
    │   │   └── dhcpcd-hooks
    │   ├── firmware
    │   ├── iproute2
    │   ├── permissions
    │   ├── ppp
    │   ├── security
    │   ├── updatecmds
    │   ├── wifi
    │   └── wimax
    ├── fonts
    ├── framework
    ├── lib
    │   ├── bluez-plugin
    │   ├── egl
    │   ├── hw
    │   │   └── modules
    ├── lost+found
    ├── media
```

更多资源请访问稀酷客(www.ckook.com)

```
103         │   └── audio
104         │       ├── alarms
105         │       ├── notifications
106         │       ├── ringtones
107         │       └── ui
108         ├── tts
109         │   └── lang_pico
110         ├── usr
111         │   ├── keychars
112         │   ├── keylayout
113         │   ├── share
114         │   │   ├── bmd
115         │   │   └── zoneinfo
116         │   └── srec
117         └── xbin
```

第 1 行：顶层是 root 目录，在此之下创建之前提到的其他文件系统的结构及挂载点。

第 2 行：正如之前讨论的那样，在 HTC Incredible 中创建了一个“/app-cache”目录，类型为 tmpfs。在此可以看到浏览器缓存结构。随着时间的推移，其他应用程序也许会利用此目录。

第 6 ～ 8 行：从一开始，Android 终端设备中就有一个专门的“/cache”目录，但是最初好像没有使用。然而，在这里肯定不是这种情况，应该对“/cache”分区进行映像并进行全面分析。在这里可以发现的文件包括 Gmail 附件预览、浏览器 DRM、一些下载的软件（从 Android 市场及其他地方），以及从无线服务提供商处下载的更新。

第 9 行：root 级别的目录“/data”中有一系列很重要的子目录，将在下面讨论。需要注意的是，某些手机（如 HTC Incredible）针对“/data/data”子目录专门建立了一个分区。

第 10 行：“/data/anr”目录中有一个系统的原始跟踪（程序调试），通常不允许 shell 用户进行访问。但是，有一些 adb debug 命令好像会读取这里的数据。

第 11 行：“/data/app”目录中包含从 Android 市场下载的 .apk 文件。

第 12 行：“/data/app-private”目录中存放着从 Android 市场下载的受保护应用程序。

第 13 行：很多近期版本的 Android 都有一个安全云备份应用编程接口（API），开发者可以集成到他们的应用程序中的。“/data/backup”目录用于对这些备份进行排队与管理。然而，到目前为止，在此目录中没有发现很多有意义的数据。

第 14 行：“/data/btips”（德州仪器公司的蓝牙协议栈）目录中保存着在相关应用程序（com.ti.btips）崩溃时的日志文件。

第 15 行：“/data/davlik-cache”目录中保存着 Davlik 虚拟机的缓存 dex 文件，用于运行应用程序。

第 16 行：“/data/data directory”目录中存放应用程序专用数据，很容易地就知道是取证调查中需要关注的最重要区域。

第 17 ～ 23 行：为了演示的目的，在此目录层次下保存着一个应用程序。此目录根据软件包名进行命名，通常会很清楚地识别出开发者（在此例中是 Facebook）。

第 24 行：对于 HHGTTG 的崇拜者（来自于经典小说《The Hitchhiker's Guide to the

更多资源请访问稀酷客(www.ckook.com)

Galaxy》中对太空旅行者的著名忠告：不要惊慌（DON'T PANIC））来说，这里有一个很棒的目录“/data/dontpanic”，但只是用于保存系统的一些错误日志文件而已。再一次强调，开放式系统的最大好处就是能够检查代码。在此之上，我们就必须去猜测其中的目的，或者去进行大量的测试。以下源自于 AOSP：

```
# Create dump dir and collect dumps.
  # Do this before we mount cache so eventually we can use cache for
  # storing dumps on platforms which do not have a dedicated dump partition.

  mkdir /data/dontpanic
  chown root log /data/dontpanic
  chmod 0750 /data/dontpanic

  # Collect apanic data, free resources and re-arm trigger
  copy /proc/apanic_console /data/dontpanic/apanic_console
  chown root log /data/dontpanic/apanic_console
  chmod 0640 /data/dontpanic/apanic_console

  copy /proc/apanic_threads /data/dontpanic/apanic_threads
  chown root log /data/dontpanic/apanic_threads
  chmod 0640 /data/dontpanic/apanic_threads

  write /proc/apanic_console 1

  # Collect ramconsole data
  copy /proc/last_kmsg /data/dontpanic/last_kmsg
  chown root log /data/dontpanic/last_kmsg
  chmod 0640 /data/dontpanic/last_kmsg
```

第 25 行：“/data/local”目录很重要，因为 shell（允许没有 root 访问权限手机中的用户账号运行 adbd）可以对此目录进行读取 / 写入访问。在安装一个应用程序时，此应用程序首先被复制到“/data/local”中。同时，有些取证技术依赖这个目录来上传一些重要的文件，这些文件通常是二进制文件。

第 26 行：“/data/lost+found”目录在 YAFFS2 文件系统中出现多次。再为快速进行一次搜索（在下载的 YAFFS2 的源目录上使用“grep -R lost+found *.c”）将会证明，任何被发现的文件，如果没有通往 root 目录的路径，都将被放置在此文件夹中。

```
/*
 * This code iterates through all the objects making sure that they are
rooted.
 * Any unrooted objects are re-rooted in lost+found.
 * An object needs to be in one of:
 * - Directly under deleted, unlinked
 * - Directly or indirectly under root.
 *
 * This fixes the problem where directories might have inadvertently been
deleted
 * leaving the object "hanging" without being rooted in the directory tree.
 */
```

第 27 ～ 35 行：“/data/misc”目录包含着与蓝牙、dhcp、vpn、Wi-Fi 等有关系的文件。需要指出的一个重要文件是“/data/misc/wifi/wpa supplicant.conf”，此文件的内容是终端

更多资源请访问稀酷客(www.ckook.com)

设备能够连接到的 Wi-Fi 网络的列表。如果无线连接要求提供密码，此密码也以明文的方式存储在此文件中（笔测者，愿你快乐）。以下是部分内容：

```
ahoog@ubuntu:~/htc-inc/data/misc/wifi$ cat wpa_supplicant.conf
ctrl_interface=eth0
update_config=1

network={
        ssid="viaForensics"
        psk="s0rryN04cc3ss"
        priority=1
}
network={
        ssid="attwifi"
        key_mgmt=NONE
        priority=3
}

network={
        ssid="GoogleGuest"
        key_mgmt=NONE
        priority=4
}

network={
        ssid="sfo free wifi"
        key_mgmt=NONE
        priority=5
}
```

第 36 行：“/data/property”目录中包含着各种关于系统属性的内容，如时间区域、国家和语言。

第 37 行：除了一些子目录外，可以看到 /data/system 目录中有几个很关键的文件。首先，accounts.db 文件中是要求进行认证的账号列表，每一列包含账号名、类型、密码（加密过的）以及认证用的标记（在其他数据中）。在这里还有两个与终端设备的口令或 PIN 相关的文件。这两个文件是 gesture.key 和 password.key，包含着编码 / 加密过的口令的十六进制数值。

第 43 行：当一个进程崩溃时，会生成一个特殊的 tombstone 文件。此文件的内容为 ASCII 码，因此可以直接阅读。更多的信息可以在线获得，如 Crazydaks.com 上的信息发布栏（Debugging in Android, n.d.）。

第 44 行：“/mnt”目录是系统挂载各类文件系统之处，包括 SD 卡、eMMC 等。

第 45 行：“/mnt/asec”目录下是保存在 SD 卡上的未加密应用程序。当 Android 引入将应用程序保存在 SD 卡上这个功能后，出于安全保护的目的，这些应用程序被加密保护。但是，当系统运行起来后，对这些应用程序的文件进行未加密的访问非常必要，这些文件就被解密，并被挂载在“/mnt/asec”。

第 46 行：对于某些终端设备，“/mnt/emmc”目录中的是 NAND 闪存上的 FAT32 文件系统。

更多资源请访问稀酷客(www.ckook.com)

第 47 ～ 55 行：是几个 eMMC 子目录示例。

第 51 行："/mnt/emmc/DCIM directory" 保存着相册的标记。

第 52 行："/mnt/emmc/DCIM/100MEDIA" 目录中是使用 HTC Incredible 拍摄的相片和视频。

第 53 行："/mnt/emmc/LOST.DIR" 目录存在于 FAT32 分区中，它包含文件系统无法追踪的文件或存储块（与 YAFFS2 中的 lost+found 目录相同）。应该检查此目录。

第 56 行：如果存在一个实际的 SD 卡，这个 SD 卡就挂载在 "/mnt/sdcard" 处。

第 66 行：与 eMMC 相同，"/mnt/sdcard/dcim" 目录会存放使用终端设备拍摄的相片和视频。在 HTC Incredible 上，这些相片和视频存放在 "/mnt/emmc/DCIM" 中，因此，它们实际上并不是存放于 SD 卡上。

第 67 ～ 68 行："/mnt/sdcard/download" 和 "/mnt/sdcard/Downloads" 目录中存放着使用浏览器、电子邮件客户端等下载的文件。

第 72 行：正如前面讨论过的那样，"/mnt/sdcard/secure/asec" 目录是加密的，它是驻留在 SD 卡（而不是 NAND 闪存）上的应用程序保存数据之处。

第 75 行：这个 "/system/app" 目录保存着由系统提供的与应用程序相对应的 .apk 文件。这些应用程序包括谷歌 /Android 捆绑的应用程序、制造商（此例中是 HTC）提供的应用程序以及无线服务提供商（此例中是 Verizon）提供的应用程序。对于 HTC Incredible 来说，此目录中包含了 152 个 .apk 文件。对于任何一个分析案件，如果需要对应用程序进行分析，此目录就非常重要（意味着需要访问 apk 文件）。存在于 HTC Incredible 上的这些 .apk 文件（供参考）如下：

```
ahoog@ubuntu:~/htc-inc/system/app$ ls *.apk

AccountSyncManager.apk              HTCAlbum.apk                    PackageInstaller.apk
AdobeReader.apk                     htcbookmarkwidget.apk           PCSCII.apk
amazonmp3.apk                       HtcCalculatorWidget.apk         Phone.apk
ApplicationsProvider.apk            htccalendarwidgets.apk          PicoTts.apk
AppSharing.apk                      HTCCamera.apk                   PluginManager.apk
Bluetooth.apk                       HtcCarPanel.apk                 QuickLookup.apk
BrcmBluetoothServices.apk           HtcCdmaMccProvider.apk          Quickoffice.apk
Browser.apk                         HtcClockWidget.apk              QxdmLog.apk
Calculator.apk                      HtcContacts.apk                 restartapp.apk
Calendar.apk                        htccontactwidgets.apk           Rosie.apk
CalendarProvider.apk                HtcCopyright.apk                RSS.apk
CertInstaller.apk                   HtcDialer.apk                   Settings.apk
CheckinProvider.apk                 HtcFacebook.apk                 SettingsProvider.apk
CityID.apk                          HtcFMRadio.apk                  SetupWizard.apk
Clicker.apk                         HtcFootprints.apk               SlackerRadio.apk
com.htc.FMRadioWidget.apk           HtcFootprintsWidget.apk         SocialNetworkProvider.apk
com.htc.FriendStreamWidget.apk      HTC_IME.apk                     Stock.apk
com.htc.MusicWidget.apk             HtcLocationPicker.apk           Street.apk
com.htc.NewsReaderWidget.apk        HtcLocationService.apk          Superuser.apk
com.htc.StockWidget.apk             HtcLockScreen.apk               Talk.apk
com.htc.TwitterWidget.apk           htcmailwidgets.apk              teeter.apk
com.htc.WeatherWidget.apk           HtcMessageUploader.apk          TelephonyProvider.apk
ContactsProvider.apk                htcmsgwidgets.apk               TtsService.apk
CustomizationSettingsProvider.apk   HtcMusic.apk                    TVOUT.apk
CustomizationSetup.apk              HtcPhotoWidget.apk              Updater.apk
DCSImpl.apk                         HtcProfilesWidget.apk           UpgradeSetup.apk
DCSStock.apk                        HtcRingtoneTrimmer.apk          UploadProvider.apk
DCSUtility.apk                      HtcRingtoneWidget.apk           UserDictionaryProvider.apk
```

更多资源请访问稀酷客(www.ckook.com)

```
DebugTool.apk
DefaultContainerService.apk
DMPortRead.apk
DownloadProvider.apk
DrmProvider.apk
EPST.apk
Facebook.apk
FieldTest.apk
FieldTrial.apk
FilePicker.apk
Flashlight.apk
Flickr.apk
FriendStream.apk
GenieWidget.apk
Gmail.apk
GoogleCalendarSyncAdapter.apk
GoogleContactsSyncAdapter.apk
GoogleFeedback.apk
GooglePartnerSetup.apk
GoogleQuickSearchBox.apk
GoogleServicesFramework.apk
GSD.apk
HtcAddProgramWidget.apk
HtcSettingsProvider.apk
htcsettingwidgets.apk
HTCSetupWizard.apk
HtcSoundRecorder.apk
HtcStreamPlayer.apk
HtcSyncwidget.apk
HtcTwitter.apk
HtcWeatherWallpaper.apk
HTMLViewer.apk
install_flash_player.apk
LbsProvider.apk
LiveWallpapers.apk
LiveWallpapersPicker.apk
MagicSmokeWallpapers.apk
Mail.apk
Maps.apk
MarketUpdater.apk
MediaProvider.apk
MediaUploader.apk
Mms.apk
Model0Wallpapers.apk
NetworkLocation.apk
NewsReader.apk
VCast.apk
Vending.apk
VisualizationWallpapers.apk
VoiceDialer.apk
VoiceSearch.apk
VpnServices.apk
VVM.apk
VzNav.apk
VzWBAClient.apk
VzWBAService.apk
VZWInstaller.apk
VzwLBSPerm.apk
VZW_MyVerizon.apk
VZW_Skype.apk
WeatherAgentService.apk
Weather.apk
WeatherProvider.apk
WeatherSyncProvider.apk
WidgetDownloadManager.apk
WifiRouter.apk
WorldClock.apk
YouTube.apk
```

第 76 和第 117 行："/system/bin" 和 "/system/xbin" 这两个目录中包含着 Android 系统需要使用的二进制文件。通过对这些目录下的文件开展各类实验，取证分析师和安全工程师们（当然还有 Android 系统研究者们）可以发现很多非常有用，但是却没有解释文档的命令。

第 77 ～ 80 行："/system/customize" 目录包含着服务提供商专用于手机持有者的具体个性化文件，特别是用户界面。

第 81 行："/system/etc" 目录通常是 Android 保存 Linux/Unix 配置目录（/etc）之处。此目录下有数不清的配置文件，非常值得检查，多到无法在此书中进行详细讨论，然而也是随着终端设备的不同而有所不同。

还有太多的目录和文件值得探讨，但是以上的简要介绍是一个很好的开始。

7.3 FAT 取证分析

SD 卡对于取证检查师来说可能是一个金矿。所有已经同步到手机中或者使用手机上的摄像机拍摄到的多媒体数据都保存在 SD 卡上。照片、视频、通话录音、应用数据、音乐、谷歌地图数据以及将 SD 卡作为存储空间的应用程序所保存的完整的备份文件都是可以获得的。此外，取证检查师还可能找到缓存在此卡上的多媒体（彩信）图像的图标、与已删除文件相关的回收信息以及所下载的应用程序的 Android 安装包（APK）。

例如，一个典型的用户会使用谷歌地图来找到驶往本地一个购物中心的驾驶导航。通过对 SD 卡上的 "com.google.android.apps.maps/cache" 目录进行取证检查，就可以得到地图的图像分块以及导航的语音提示。这些语音提示还都附有日期和时间，因此，取证检查师就可以驾轻就熟地对终端设备的给定日期和时间进行跟踪定位。

以下是在芝加哥郊区一个短暂的旅程后所能够看到的场景：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:/mnt/readonly-fs/google_maps_navigation/cache$ ls -la
total 1184
dr-xr-xr-x 2 root root 32768 2010-11-16 15:32 .
dr-xr-xr-x 4 root root 32768 2010-11-16 13:31 ..
-r-xr-xr-x 1 root root 66476 2010-11-16 15:20 ._speech_nav_0.wav
-r-xr-xr-x 1 root root 142252 2010-11-16 15:19 ._speech_nav_1.wav
-r-xr-xr-x 1 root root 142380 2010-11-16 15:18 ._speech_nav_2.wav
-r-xr-xr-x 1 root root 73644 2010-11-16 15:15 ._speech_nav_3.wav
-r-xr-xr-x 1 root root 60460 2010-11-16 15:15 ._speech_nav_4.wav
-r-xr-xr-x 1 root root 107948 2010-11-16 15:15 ._speech_nav_5.wav
-r-xr-xr-x 1 root root 96300 2010-11-16 15:20 ._speech_nav_6.wav
-r-xr-xr-x 1 root root 6144 2010-11-16 13:31 tilecache_ImageTileStore.db
-r-xr-xr-x 1 root root 281600 2010-11-16 15:32 tilecache_VectorTileStore.db
```

还有一点需要牢记的是，通过 Android 可以将 SD 卡作为一个外部的大容量存储设备进行挂载。这样做就可以使用户在 SD 卡和个人计算机之间传送文件。

在第 6 章中，我们演示了如何在包含数据的 Android 终端设备上获取两个 FAT32 分区（包含数据）。目前已经有很多关于如何对 FAT32 文件系统进行分析的书籍和文章，此节就不再重复讨论这些细节。但是，在此节中，我们将演示一些如何在 Ubuntu 工作站上检查存在于 Android 终端设备上的 FAT32 分区的技术。

7.3.1　FAT 时间序列分析

为了基于 FAT32 映像构建一个文件系统的时间序列，我们同时使用 Sleuth 套件（The Sleuth Kit，TSK）和另外一个很棒的名为 log2timeline 的开源取证工具。这个由 Kristinn Gudjonsson 开发的名为 log2timeline 的共享应用程序是一个自动建立时间序列的框架，包括了系统中的各种日志文件以及存在于系统中的其他一些构建。log2timeline 用于很多系统，能够出色地将时间戳信息从多种格式的文件中抽取出来进行分析。

因为我们已经安装了 TSK，所以只需要再有几个步骤来完成 log2timeline 安装。首先，也许构建 Ubuntu 工作站已经有相当长一段时间，因此，有必要对已经安装的软件进行更新，升级到新的版本以及修复可能存在的安全问题：

```
sudo apt-get update
sudo apt-get upgrade -u
```

以上的第一步更新软件表，第二步将完成更新。Kristinn 已经通过对他所开发的 log2timeline 生成一个 Ubuntu 软件包的方式极大地简化了安装过程：

```
sudo add-apt-repository "deb http://log2timeline.net/pub/ maverick main"
wget -q http://log2timeline.net/gpg.asc -O- | sudo apt-key add -
sudo apt-get update
sudo apt-get install log2timeline-perl
```

以上的 4 个命令能够完成以下的功能：

1）将 log2timeline 自定义软件库添加到 Ubuntu 工作站的总表中；

2）下载用于验证此软件的公钥，并添加到接受的密钥表中；

更多资源请访问稀酷客(www.ckook.com)

3）更新软件包表；

4）安装 log2timeline。

针对要进行的分析，我们将使用第 6 章中从 SD 卡上获取的一个 2GB 的取证映像。首先，检查师需要确认此映像的散列签名与获得取证映像时得到的散列值相匹配，以确保此映像的有效性：

```
ahoog@ubuntu:~/sd-emmc/viaforensics/af-book/sdcard2-113serialno$ sha256sum
sdcard2-113serialno.dc3dd
e5dcc0af1d8a09c9af4d2db98f5f684d20a561666b9ff8df7c8b90a0b9d78770  sdcard2-
113serialno.dc3dd
```

如果还记得的话，输入设备（此例中为 /dev/sdc）的散列值是 e5dcc0af1d8a09c9af4d2db98f5f684d20a561666b9ff8df7c8b90a0b9d78770。此时，取证映像的有效性得到验证。下一步，首先让我们使用以下命令快速地看一下文件：

```
ahoog@ubuntu:/home/ahoog$ file /home/ahoog/sd-emmc/viaforensics/af-book/
sdcard2-113serialno/sdcard2-113serialno.dc3dd
/home/ahoog/sd-emmc/viaforensics/af-book/sdcard2-
113serialno/sdcard2-113serialno.dc3dd: x86 boot sector; partition 1: ID=0x6,
starthead 2, startsector 129, 3911551 sectors, extended partition table
(last)\011, code offset 0x0
```

所以，我们是在面对一个有效分区的磁盘映像。接下来，我们使用 TSK 中的 mmls 命令来进一步查看这个磁盘映像：

```
ahoog@ubuntu:~/sd-emmc/viaforensics/af-book/sdcard2-113serialno$ mmls sdcard2-
113serialno.dc3dd
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot    Start        End          Length       Description
00:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
01:  -----   0000000000   0000000128   0000000129   Unallocated
02:  00:00   0000000129   0003911679   0003911551   DOS FAT16 (0x06)
```

最后，使用 TSK 中的 fsstat 命令，但是需要注意的是必须提供一个数据为 129 的偏移值，因为此分区从那里开始：

```
ahoog@ubuntu:~/sd-emmc/viaforensics/af-book/sdcard2-113serialno$ fsstat -o 129
sdcard2-113serialno.dc3dd
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: FAT16

OEM Name:
Volume ID: 0xe0fd1813
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory):
File System Type Label: FAT16

Sectors before file system: 129

File System Layout (in sectors)
```

更多资源请访问稀酷客(www.ckook.com)

```
Total Range: 0 - 3911550
* Reserved: 0 - 0
** Boot Sector: 0
* FAT 0: 1 - 239
* FAT 1: 240 - 478
* Data Area: 479 - 3911550
** Root Directory: 479 - 510
** Cluster Area: 511 - 3911550

METADATA INFORMATION
--------------------------------------------
Range: 2 - 62577158
Root Directory: 2
CONTENT INFORMATION
--------------------------------------------
Sector Size: 512
Cluster Size: 32768
Total Cluster Range: 2 - 61111

FAT CONTENTS (in sectors)
--------------------------------------------
511-574 (64) -> EOF
575-638 (64) -> EOF
639-702 (64) -> EOF
703-766 (64) -> EOF
767-830 (64) -> EOF
831-894 (64) -> EOF
895-958 (64) -> EOF
959-1022 (64) -> EOF
1023-1086 (64) -> EOF
<snip>
```

在此例中，这个分区是一个 FAT16 分区，分区中还存储有数据。因此，我们准备首先使用 TSK 中的 fls 命令来构建时间序列：

```
ahoog@ubuntu:/home/ahoog$ time fls -z CST6CDT -s 0 -m /mnt/sdcard -f fat16 -r
-o 129 -i raw ~/sd-emmc/viaforensics/af-book/sdcard2-113serialno/sdcard2-
113serialno.dc3dd > ~/sdcard.body

real    0m55.765s
user    0m0.820s
sys     0m12.850s
```

以上命令中选项的含义如下：

- -z CST6CDT：将时间区域设置为 CST6CDT，即为美国中部标准时间；
- -s 0：如果存在的话，设置时间误差；
- -m /mnt/sdcard：当写入文件时，将此值作为文件路径的前缀；
- -f fat16：将文件系统设置为 FAT16；
- -r：递归地遍历所有目录，以构建时间序列；
- -o 129：将偏移值设为 129（假设每一块的大小为 512 字节，但是可以使用 -b 选项进行修改）；
- -i raw：设置映像的类型，在此例中是原始映像，而不是其他的取证映像格式；
- ~/sd-emmc/viaforensics/af-book/sdcard2-113serialno/sdcard2-113serialno.dc3dd：映

更多资源请访问稀酷客(www.ckook.com)

像文件；

- > ~/sdcard.body：将命令的输出转指向一个文件，而不是显示在屏幕上。

通常，知道一个命令执行需要多长时间非常有帮助（即使不为其他目的，长此以往，至少也知道什么时候可以去取一台咖啡加热器）。因此，我们将 fls 命令前加上 time 命令，这样我们就可以知道程序执行多长时间（实际时间）以及所花费的各种系统时间（用户、系统）。我们可以直接去查看这个原始文件里的内容，但是在接下来的一步，我们将会把这个原始文件转换成一个可读文件的格式。如果想要验证 fls 命令返回结果的正确性，至少总是可以确定文件中所包含内容的总行数：

```
ahoog@ubuntu:~$ wc -l sdcard.body
24399 sdcard.body
```

所以，我们有超过 24 000 个条目。下一步，我们需要将此文件系统以只读的方式挂载，这样就可以对此文件系统运行 log2timeline 命令。为了使用 dd image 来挂载此文件系统，可以使用 mount 命令以及一个被称为环回设备的特殊设备。以下是全部的命令：

```
ahoog@ubuntu:~$ mkdir -p ~/mnt/sdcard
ahoog@ubuntu:~$ sudo mount -t vfat -o loop,ro,offset=66048 ~/sd-emmc/
viaforensics/af-book/sdcard2-113serialno/sdcard2-113serialno.dc3dd ~/mnt/sdcard
```

让我们再看一下每一个选项。首先，执行这个命令需要管理员的权限，因此，我们将它与 sudo 一起执行。这些选项就是：

- -t vfat。
- -o loop,ro,offset=66068：这个选项告诉 mount 命令使用回路设备，因为我们使用的是一个实际的文件，而不是一个实际的设备。ro 将 e-mail 以只读方式挂载。最后，必须要告诉 mount 在哪里可以找到分区。在 mmls 命令处，我们回忆一下，块偏移值是 129。Mount 不知道块的大小，因此，我们可以计算一下 129×512=66 048。
- ~/sd-emmc/viaforensics/af-book/sdcard2-113serialno/sdcard2-113serialno.dc3dd：这些就是 dd 映像。
- ~/mnt/sdcard：在何处挂载映像。

我们可以通过执行不带任何选项的 mount 命令来验证文件系统已经正确挂载，在这种情况下，mount 命令会返回所有已经挂载的文件系统。我们通过管道机制将此结果发送给 grep 命令，以便将我们想要寻找的文件系统隔离出来：

```
ahoog@ubuntu:~$ mount | grep vfat
/dev/loop0 on /home/ahoog/mnt/sdcard type vfat (ro,offset=66048)
```

因此，我们可以看到，vfat 文件系统以只读方式挂载，使用回路设备“/dev/loop0”进行代表，并且位置在“/home/ahoog/mnt/sdcard”。可以使用 df 命令来查看存储空间的总体大小情况：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/sda1              19G   18G  570M  97% /
none                  366M  208K  366M   1% /dev
none                  373M  256K  373M   1% /dev/shm
none                  373M  100K  373M   1% /var/run
none                  373M     0  373M   0% /var/lock
/dev/mtdblock0         64M  1.2M   63M   2% /home/ahoog/mnt/yaffs2
/dev/loop0            1.9G  244M  1.7G  13% /home/ahoog/mnt/sdcard
```

可以看到，1.7GB 中，只有 244M 已用。现在，我们就可以使用 log2timeline 中的 timescanner 程序从文件中获取更多的时间序列信息。我们将此命令的运行结果附加到存放 fls 命令运行结果的同一个主体文件中去：

```
ahoog@ubuntu:~$ time timescanner -d /home/ahoog/mnt/sdcard -z CST6CDT >>
sdcard.body
Loading output file: mactime
[timescanner] Recursive scan completed.  Successfully extracted timestamps from
410 artifacts (either files or directories).
Run time of the script 62 seconds.

real    1m2.241s
user    0m31.810s
sys     0m13.760s
```

对 timescanner 中的选项解释如下：

- -d /home/ahoog/mnt/sdcard：指定寻找时间戳信息所要扫描的目录。
- -z CST6CDT：将时间区域设置为美国中部标准时间。
- >> sdcard.body：将输出结果附加到已经存在的 sdcard.body 文件中去。需要说明的是 >> 是用于指示 shell 去附加到一个文件中去（如果此文件不存在的话，就会创建这个文件）。如果忘记使用 >>，而是使用了“>”，则文件中已经存在的 fls 命令的输出结果就会被覆盖掉。

Timescanner 只找到了 410 个有效结果，这个数量比如果对整个硬盘进行扫描所能够获得的结果少了很多。然而，这 410 个结果对于构建终端设备的整体时间序列肯定也是非常有帮助的。

最后，我们可以为时间序列来使用 TSK 中的 mactime 命令，生成一个很容易阅读（很容易进行分享）、使用逗号来分离不同数值（csv）的文件：

```
ahoog@ubuntu:~$ mactime -b sdcard.body -z CST6CDT -d > sdcard-timeline.csv
```

以下选项在 mactime 中使用：

- -b sdcard.body：指定要转换的主体文件；
- -z CST6CDT：时间区域；
- -d：csv 格式的文件输出。

执行这个命令只用了区区几秒钟，产生了不到 25 000 项条目，但是在 csv 文件中却生成了超过 73 000 行的数据：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ wc -l sdcard-timeline.csv
73739 sdcard-timeline.csv
```

现在就可以使用制表软件来浏览时间序列，也可以将结果输入到一个数据库中进行更进一步的分析。在这里，可以指出有几个很有趣的要点。

首先，很有可能会看到很多在 1970 年 1 月 1 日附近的日期。这是由于将时间戳设置为 0 或者是根本没有设置而造成的。由于 UNIX 的标准时间设置基于自美国中部标准时间 1970 年 1 月 1 日零点以来所经历的时间，所以加上一个偏移值也许就是准确的时间。在以上的例子中，由于将时间设置为 CST6CDT，也就是在 1 月中为格林威治时间减去 6 小时。因此，有很多条目的时间戳为“Wed Dec 31 1969 18:00:00（1969 年 12 月 31 日，星期三，18 时整）”。虽然我们无法对这些条目收集到具体的时间序列数据，但是它们可能包含有重要的信息。

如果在 Ubuntu 工作站上双击 sdcard-timeline.csv 文件，Open Office 中的 Calc 程序就会显示出一个 Text Import 屏幕，如图 7.4 所示。确认将 Separated by 设置成逗号，然后按下 OK。

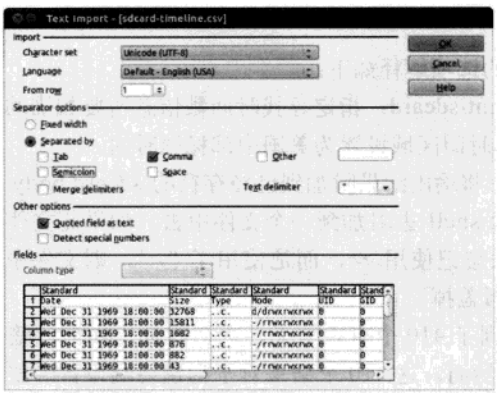


图 7.4　将 sdcard-timeline.csv 作为 Open Office 中 Calc 的 Text import

这个 Calc 程序将会被打开，然后就可以按照时间序列进行浏览，如图 7.5 所示。

可以看到，大多数的文件已经被删除。但是，使用 TSK 和其他程序仍然可以将这些文件进行修复。一个值得检查的很有趣的事件是：将一个应用程序移到 SD 卡上进行测试。这个名为 Angry Birds 的程序可以从 SD 卡上运行，在 2011 年 2 月 15 日 6 时 17 分 28 秒时进行过测试，如图 7.6 所示。

时间序列清楚地表明一个新的文件“/mnt/sdcard/.android secure”在 6 时 17 分 28 秒时被创建并修改。

最后一个需要指出的条目来源于 log2timeline。当此条目经过一个 PDF 时，元数据被提取出来。我们可以看到以下的信息：

更多资源请访问稀酷客(www.ckook.com)

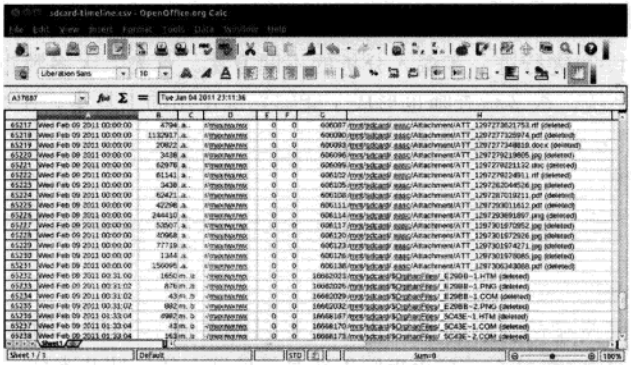


图 7.5　在 OO Calc 中查看时间序列

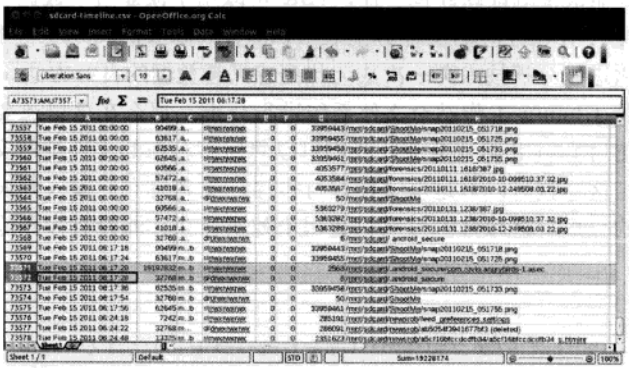


图 7.6　当应用移至 SD 卡时的时间序列

- Mon Nov 29 2010 04:44:47（星期一，2010 年 11 月 29 日，4 时 44 分 47 秒）；
- 23,159 bytes（23 159 字节）；
- File modified（文件被修改）；
- Title: (Scanned Document)（题目：扫描文件）；
- Author: [michelle]（作者：michelle）；
- Creator: [HardCopy]（创建者：HardCopy）；
- Produced by: [Lexmark X543]:（生成设备：Lexmark X543）；
- File: /mnt/sdcard/.easc/Attachment/ATT 1291219677612.pdf（文件：/mnt/sdcard/.easc/Attachment/ATT 1291219677612.pdf）。

时间序列分析非常有效，特别是当取证分析师将传统的文件系统时间戳机制与 Kristinn Gudjonsson 开发的 log2timeline 结合在一起使用时。

更多资源请访问稀酷客(www.ckook.com)

7.3.2 更多的 FAT 分析

还有很多对 FAT 进行进一步深入分析的不同方法。首先，大多数市场上的取证软件都支持 FAT 文件系统，很多检查师也形成了能够熟练使用的工具集。在取证软件之外，我们在此节的第一部分再介绍几项技术，包括：

- 文件系统分析；
- 文件雕复；
- strings；
- 十六进制分析。

即使对于不同类型的文件系统，以上这些技术也非常类似，因此我们将在下面的 YAFFS2 一节中进行详细讨论。在此我们仍然要简单地介绍一些基本概念。

首先，在 SD 卡上明显地存在着大量的已删除文件。建议使用任何一款取证工具去恢复这些数据。与其他很多取证软件包一样，TSK 可以用于恢复这些数据。此外，文件雕复技术可以用于恢复没有在分区的文件分配表（File Allocation Table）中得到引用的文件。strings 技术和十六进制分析技术再一次成为找到感兴趣数据的良好方法。

如果使用的是 Ubuntu 工作站对文件系统进行分析，那么在这里还要顺便提一下另外一项快速技术。使用 find 和 file 命令，我们可以将所有存在的文件按照文件名、路径或文件类型进行罗列。这样就可以对这些文件进行排序，或者将它们输入到表格制作程序或数据库中进行更多的分析。这项技术虽然只适用于列出存在（即没有被删除）的文件，却非常有效。

命令如下：

```
ahoog@ubuntu:~$ find ~/mnt/sdcard -type f -print0 | xargs -0 file
/home/ahoog/mnt/sdcard/.android_secure/com.rovio.angrybirds-1.asec:
data
/home/ahoog/mnt/sdcard/.footprints/footprints.db:
SQLite 3.x database
/home/ahoog/mnt/sdcard/download/Swype-Installer.apk:
Zip archive data, at least v2.0 to extract
/home/ahoog/mnt/sdcard/download/Swype-Installer-1.apk:
Zip archive data, at least v2.0 to extract
/home/ahoog/mnt/sdcard/download/Swype-Installer-2.apk:
Zip archive data, at least v2.0 to extract
/home/ahoog/mnt/sdcard/download/Download.apk:
Zip archive data, at least v1.0 to extract
/home/ahoog/mnt/sdcard/download/PdaNetA242.pkg:
xar archive - version 1
/home/ahoog/mnt/sdcard/download/hotwatch-powerpoint.pdf:
PDF document, version 1.5
/home/ahoog/mnt/sdcard/download/dinner.pdf:
PDF document, version 1.3
/home/ahoog/mnt/sdcard/download/dinner-1.pdf:
PDF document, version 1.3
/home/ahoog/mnt/sdcard/download/Swype-Installer-3.apk:
Zip archive data, at least v2.0 to extract
/home/ahoog/mnt/sdcard/download/subpoena.pdf:
PDF document, version 1.3
```

更多资源请访问稀酷客(www.ckook.com)

```
/home/ahoog/mnt/sdcard/Android/data/com.google.android.apps.genie.geniewidget.
news-content-cache/.nomedia: empty
/home/ahoog/mnt/sdcard/Android/data/com.google.android.apps.maps/cache/
cache_vts.m:data
/home/ahoog/mnt/sdcard/Android/data/com.google.android.apps.maps/cache/
cache_rgts.m:data
/home/ahoog/mnt/sdcard/Android/data/com.google.android.apps.maps/cache/
cache_rgts.0:Microsoft Document Imaging Format
/home/ahoog/mnt/sdcard/Android/data/com.google.android.apps.maps/cache/
cache_vts.0:data
/home/ahoog/mnt/sdcard/Android/data/com.google.android.apps.maps/cache/
._speech_nav_5.wav: RIFF (little-endian) data, WAVE audio,
Microsoft PCM, 16 bit, mono 16000 Hz
```

此命令在 SD 卡上共找到 4352 个文件，并对其进行了分类。以下是对此命令的每一部分所完成功能的解释：

- find ~/mnt/sdcard：在挂载 SD 卡映像的目录中查找文件；
- -type f：只针对正常文件（也就是说，不包含子目录）；
- -print0：每一个文件名都以一个 NULL 字符结束，而不是默认的回车（新行）字符，后者在文件名含有空格时会产生问题；
- “|”：使用管道机制将输出结果输送到下一个程序；
- xargs：此程序建立和执行的命令行来源于标准输入的数据（也就是说，来源于其他程序的输出）；
- -0：通知 xargs，通过管道机制得到的数据都以 NULL 字符结尾（与 find 的 -print0 选项相对应）；
- file：这是 xargs 针对 find 命令返回的每一行结果所执行的命令。

find 和 xargs 命令是对文件进行检查、与文件进行交互，或者处理大量文件的强大工具。通过将时间序列和文件系统分析、文件雕复、strings 和十六进制分析相结合使用，调查师就装备了功能强大的工具，可以获取对案件来说非常关键的信息。

7.3.3　FAT 分析师说明

对于 Android 系统上的 FAT 文件系统，在此还有几点说明。

1. SD 卡上的应用程序

首先，正如在第 6 章中所讨论的，截至 Android2.2 版本，用户可以将所支持的应用程序移到 SD 卡上，以节省“/data/data”目录下的空间，因为用户数据也保存在这个目录下。这就要求应用程序必须支持这一功能，通常这一类的应用程序都使用大量的存储空间，如游戏软件。为了测试一下应用程序是否具备此功能，我们将非常流行的由 Rovio Mobile 开发的 Angry Birds（愤怒的小鸟）游戏（Rovio - Angry Birds, n.d.）安装在 HTC Incredible 手机上。此应用程序只是简单地运行了一下就被关闭。如果想要将此应用程序移到 SD 卡，在主屏幕上选择 Settings（设置），然后是 Applications（应用程序），最后是 Manage Applications（管理应用程序），如图 7.7 所示。

更多资源请访问稀酷客(www.ckook.com)

可以看出，Angry Birds 占用了“/data/data”中的 18.40MB 存储空间，这可是非常珍贵的空间。所以，此应用程序的开发者实现了必要的支持，可以将 .apk 文件移到 SD 卡上。然后用户就可以选择此应用程序（此例中就是 Angry birds），随后就看到屏幕上显示出与此应用程序有关的信息和选项，包括“Move to SD card”（移至 SD 卡），如图 7.8 所示。

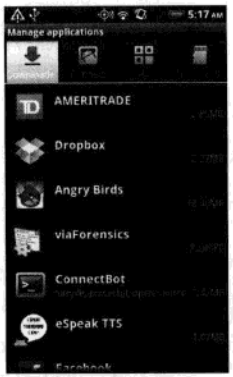


图 7.7 应用程序管理界面

在这个屏幕上可以看到此应用程序本身就占用了 18.39MB 的存储空间，而用户数据只占用了 12.00MB 的存储空间，使该应用程序成为将其 .apk 文件移到 SD 卡上的候选者。如图 7.9 所示，将此应用程序的 .apk 文件移到 SD 卡上简单到点击一下“Move to SD card”按钮并等待整个任务完成而已。

从用户的角度来看，这是非常美妙的事情。但是，这个操作对于取证检查师和安全分析师到底意味着什么？这个操作需要经历好几个步骤才能完成，以下对这些步骤进行解释，并在所罗列的代码中特别标示出来：

图 7.8 将应用程序移到 SD 卡上

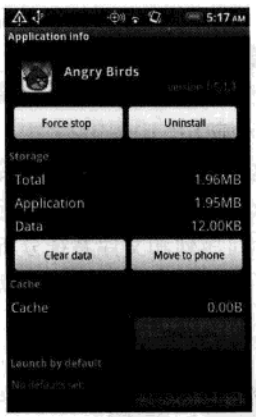


图 7.9 应用程序移动完毕

1）将 .apk 文件从“/data/app”（或“/data/app-private”）目录移到 SD 卡上。其中，SD 卡位于加密分区 /mnt/secure/asec 中，此分区只能通过 root 目录进行访问。

2）一个新的设备映射文件系统以只读的方式被挂载，来自于“/mnt/secure/asec”的这个应用程序被解密，并且放置在“/mnt/asec/com.rovio.angrybirds-1”处，使系统能够对其进行访问。系统必须要能够访问这个解密后的 .apk 文件，才能运行此应用程序。

3）用户数据仍然保存在“/data/data”处（在此例中，具体是在“/data/data/com.rovio.angrybirds”下）。

更多资源请访问稀酷客(www.ckook.com)

4）如果无法访问 SD 卡（不论是被移走、损坏或者只是为了文件共享而挂载在一个计算机上），解密的部分就会被卸载，系统无法再继续访问。这就是为什么核心的应用程序目前都无法从 SD 卡上运行。

.apk 文件在 SD 卡上又被加密，以确保其他应用程序（或进程）无法损坏或恶意地修改此应用程序。然而，应用程序的开发者们不能只依赖于加密手段来防止他人对 .apk 文件进行访问，因为如果真要想访问，那么仅将此文件从未加密的“/mnt/asec/<app-name>”处进行复制即可。以下代码中的 mount 和 ls 命令对这个结构进行了说明：

```
ahoog@ubuntu:~/sd-emmc$ adb shell mount
rootfs / rootfs ro,relatime 0 0
tmpfs /dev tmpfs rw,relatime,mode=755 0 0
devpts /dev/pts devpts rw,relatime,mode=600 0 0
proc /proc proc rw,relatime 0 0
sysfs /sys sysfs rw,relatime 0 0
none /acct cgroup rw,relatime,cpuacct 0 0
tmpfs /mnt/asec tmpfs rw,relatime,mode=755,gid=1000 0 0
none /dev/cpuctl cgroup rw,relatime,cpu 0 0
/dev/block/mtdblock3 /system yaffs2 ro,relatime 0 0
/dev/block/mmcblk0p1 /data ext3
rw,nosuid,noatime,nodiratime,errors=continue,data=writeback 0 0
/dev/block/mtdblock6 /data/data yaffs2 rw,nosuid,nodev,relatime 0 0
/dev/block/mmcblk0p2 /cache ext3
rw,nosuid,nodev,noatime,nodiratime,errors=continue,data=writeback 0 0
tmpfs /app-cache tmpfs rw,relatime,size=8192k 0 0
/dev/block/vold/179:3 /mnt/emmc vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,
dmask=0702,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,
utf8,errors=remount-ro 0 0
/dev/block/vold/179:9 /mnt/sdcard vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,
dmask=0702,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,
utf8,errors=remount-ro 0 0
/dev/block/vold/179:9 /mnt/secure/asec vfat
rw,dirsync,nosuid,nodev,noexec,relatime,uid=1000,gid=1015,fmask=0702,
dmask=0702,allow_utime=0020,codepage=cp437,iocharset=iso8859-1,shortname=mixed,
utf8,errors=remount-ro 0 0
tmpfs /mnt/sdcard/.android_secure tmpfs ro,relatime,size=0k,mode=000 0 0
/dev/block/dm-0 /mnt/asec/com.rovio.angrybirds-1 vfat
ro,dirsync,nosuid,nodev,noexec,relatime,uid=1000,fmask=0222,dmask=0222,
codepage=cp437,iocharset=iso8859-1,shortname=mixed,utf8,errors=remount-ro 0 0


ahoog@ubuntu:~/sd-emmc$ adb shell ls -l /mnt
d---rwxr-x system   sdcard_rw          1969-12-31 18:00 emmc
drwxr-xr-x root     system             2011-02-15 05:17 asec
drwx------ root     root               2011-02-14 07:59 secure
d---rwxr-x system   sdcard_rw          1969-12-31 18:00 sdcard

ahoog@ubuntu:~/sd-emmc$ adb shell ls -l /mnt/asec
dr-xr-xr-x system   root              1969-12-31 18:00 com.rovio.angrybirds-1

ahoog@ubuntu:~/sd-emmc$ adb shell ls -l /mnt/asec/com.rovio.angrybirds-1
-r-xr-xr-x system   root     17235931 2011-02-15 05:17 pkg.apk

# ls -l /mnt/secure/asec
```

更多资源请访问稀酷客(www.ckook.com)

```
----rwxr-x system   sdcard_rw 19192832 2011-02-15 05:17 com.rovio.angrybirds-1.
asec

# ls -l /data/data/com.rovio.angrybirds
drwxrwx--x app_106  app_106           2011-02-15 05:14 cache
drwxrwx--x app_106  app_106           2011-02-15 05:15 files
drwxr-xr-x system   system            2011-02-15 05:14 lib
```

如果需要对 .apk 文件进行分析（例如，调查恶意代码），那么在 SD 卡插入 Android 终端设备并且处于正常工作状态时（也就是说，没有挂载到取证工作站上以对数据进行物理获取），在获取过程中复制“/mnt/asec”中的子目录也非常重要。

2. nomedia

还有一个存在于 SD 卡和 eMMC 分区的共同事项是：存在一个文件名为 .nomedia 的空文件，其将指示 Android 系统中的媒体扫描仪跳过此目录，从而就不包括应用程序中的任何多媒体文件，如 Gallery 或 Music。如果某个目录中没有这个文件，那么就极有可能在多媒体扫描仪目录中发现那些已删除却具有图标的这些图像。这个话题将在 Android 应用程序分析与引用一节中再进行进一步的介绍。

7.4 YAFFS2 取证分析

正如在第 4 章所讨论的，YAFFS2 是一个开源的、日志结构的文件系统，特别针对 NAND 闪存而开发，因此包含了对耗损程度和纠错代码算法的支持。这不但对于手机用户来说是一个好消息，因为 YAFFS2 在 Android 系统上已经非常稳定，对于取证分析师来说也是如此，因为大量旧的（被删除的或被更新过的）数据可以恢复出来。但是，好消息仅此而已。由于 YAFFS2 是一个相对较新的文件系统，此刻还没有任何一款商用的取证软件方案能够完全支持这个文件系统。

然而，并不是一切都没有希望了。依赖于 Linux 的强大功能、YAFFS2 的开放性、有效的取证策略以及所有已经讨论过的技术（包括文件系统分析、文件雕复、strings 及十六进制分析等），取证检查师能够获取从文件系统中抽取出的大量的有用数据。

在我们开始进入 YAFFS2 分析时，牢记住 YAFFS2 的结构非常有帮助。需要回顾的一个重要概念是在 NAND 闪存上通过 MTD 如何将数据有结构地组织在一起。一般来说，YAFFS2 和 MTD 将 NAND 闪存以 128KB 为块作为单位进行组织，每一块由 64 个 2048B（2KB）子块数据加上一个 64 字节的 OOB（Out-Of-Band）数据存储区域组成（使 NAND 闪存上块的实际大小为 132KB），OOB 数据区域用于存储磁盘和文件系统的元数据，如图 7.10 所示。

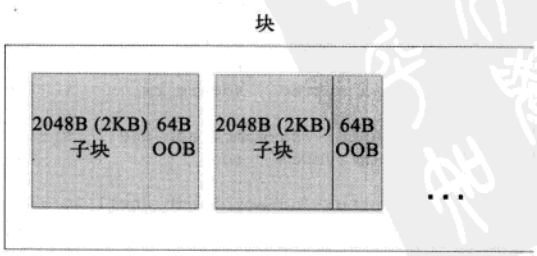


图 7.10 块（128 KB=64×2k 子块 + 带外数据存储区域）

更多资源请访问稀酷客(www.ckook.com)

当然，并不是一定要求 Android 终端设备非要用这种方式组织 NAND 闪存和 MTD，最好的查验方法就是查看“/proc/mtd”文件中的内容，因为此文件中的数据是配置信息。以下是我们所参考的 HTC Incredible 中此文件的内容：

```
ahoog@ubuntu:~$ adb shell cat /proc/mtd
dev:    size   erasesize  name
mtd0: 000a0000 00020000 "misc"
mtd1: 00480000 00020000 "recovery"
mtd2: 00300000 00020000 "boot"
mtd3: 0f800000 00020000 "system"
mtd4: 000a0000 00020000 "local"
mtd5: 02800000 00020000 "cache"
mtd6: 09500000 00020000 "datadata"
```

虽然我们需要 OOB 数据去试图将 YAFFS2 nanddump 挂载在 Ubuntu 工作站上，OOB 中 64 个字节的数据对大多数取证技术来说会造成问题，特别是文件雕复技术。为了缓解此问题，可以简单地使用程序清除 OOB 中的数据。这个程序使用 Python 编写，因此可以很容易地进行修改而适用于不同的数据块或者不同的 OOB 配置。

首先，打开一个新文件，使用 sudo 访问权限进行编辑，从而就可以将编写的程序放置在“/usr/local/bin”目录中，这也是保存个性化脚本语言和程序的最佳地方：

```
ahoog@ubuntu:~$ sudo nano -w /usr/local/bin/removeOOB.py
```

下一步，将以下的内容复制到编辑器中：

```
#!/usr/bin/env python

# Author: Andrew Hoog [ahoog at viaforensics dot com]
# Name: removeOOB.py
#

import subprocess, os, csv, getopt, sys

def usage():
   print """
Written by Andrew Hoog, remove the recurring 64 bytes of OOB data from
nanddump file with 2k chunks.  Resulting dd file in saved in current directory

Usage: removeOOB.py [-h] -o nanddump file

   -h|--help: prints this help function and exits
   -o|--oobFile: name of nanddump file to parse
   """

def main():
   try:
      opts, args = getopt.getopt(sys.argv[1:], "ho:", ["help", "oobFile="])
   except getopt.GetoptError, err:
      # print help information and exit:
      print str(err) # will print something like "option -a not recognized"
      usage()
      sys.exit(2)
```

更多资源请访问稀酷客(www.ckook.com)

```
    oobFile = None
    for o, a in opts:
        if o in ("-o", "--oobFile"):
            oobFile = a
        elif o in ("-h", "--help"):
            usage()
            sys.exit()
        else:
            assert False, "unhandled option"

    #open OOB file binary, read-only
    print "opening %s readonly" % oobFile
    try:
        yaffs2File = open(oobFile, 'rb')
    except:
        print "Unable to open file source file %s" % oobFile
        usage()
        sys.exit()

    #open .dd file binary, writable
    print "opening %s.dd r/w" % oobFile
    try:
        ddFile = open(oobFile + '.dd', 'wb')
    except:
        print "Unable to file %s" % oobFile + '.dd for output'
        usage()
        sys.exit()

    chunks=0

    data = yaffs2File.read(2048)
    oob = yaffs2File.read(64)

    print "Processing..."
    while data != "":
        ddFile.write(data)
        chunks += 1
        data = yaffs2File.read(2048)
        oob = yaffs2File.read(64)

    print "Complete.  Read %d chunks" % chunks
    yaffs2File.close()
    ddFile.close()

if __name__ == "__main__":
    main()
```

使用按键 Ctrl+O 进行保存，并使用按键 Ctrl+X 退出编辑器。然后，需要将这个 Python 文件设置为可执行，因此输入以下命令：

```
ahoog@ubuntu:~$ sudo chmod 755 /usr/local/bin/removeOOB.py
```

最后，我们就可以针对一个 nanddump 文件运行此程序：

```
ahoog@ubuntu:~$ removeOOB.py -o htcinc-mtd6-datadata.nanddump
opening htcinc-mtd6-datadata.nanddump readonly
opening htcinc-mtd6-datadata.nanddump.dd r/w
Processing...
Complete.  Read 76288 chunks
```

更多资源请访问稀酷客(www.ckook.com)

我们需要确认 removeOOB.py 的结果是我们期望的结果。根据“/proc/mtd”提供的数据，“/dev/mtd/mtd6”分区的大小为 0x9500000 字节，擦除大小（即块的大小）为 0x20000 字节。整个存储空间大小为 156 237 824 字节（只需要简单地将十六进制转换为十进制），即大约为 154MB。同样，擦除空间的大小为 2048 字节，因此，如果我们用 156 237 824 除以 2048，我们就得到 76 288 个数据块。这个数据与 removeOOB.py 返回的结果一致。但是，还有一件事情需要查验。我们知道 nanddump 中每一个 2KB 的子块后都跟随着一个 64 字节的 OOB 数据，磁盘上的 nanddump 总空间大小应该为 (76 288×64)+156 237 824。因此，总空间大小应该为 161 120 256，代表了所有的数据分块以及相对应的 OOB。我们可以使用一个简单的“ls –l”命令看到磁盘上空间的大小：

```
ahoog@ubuntu:~$ ls -ltr htcinc-mtd6-datadata*
-rw-r--r-- 1 ahoog ahoog 161120256 2011-02-13 19:34 htcinc-mtd6-datadata.
nanddump
-rw-r--r-- 1 ahoog ahoog 156237824 2011-02-13 19:36 htcinc-mtd6-datadata.
nanddump.dd
```

可以看到，全部的 nanddump 以及 .dd 映像（将 nanddump 中的 OOB 清除）与我们所期望看到的存储空间大小相对应。

在拥有了 YAFFS2 中的 nanddump 文件、dd 映像以及所有的逻辑文件后（通过 tar 或 adb pull，或者是通过挂载 YAFFS2 分区），我们现在就可以对这些数据使用各种技术。

7.4.1　YAFFS2 时间序列分析

理想情况是，本节应该介绍如何使用可用的取证软件去构建 YAFFS2 时间序列的详细内容。然而遗憾的是，还没有任何取证时间序列工具支持 YAFFS2，因此，如果检查师真的需要获得这方面的信息，就只能采取一条困难的途径。

在今后的几年中，取证行业需要针对 YAFFS2 开展更多的研究。然而，随着 Android 系统向 EXT4 过渡，很有可能这些针对 YAFFS2 的研究不会实现。但是仍然希望不要发生这种情况。

在此，我们讲解一些关于此方面的研究结果。这个研究在模拟的 NAND 闪存上完成，在第 4 章中已经进行过讨论。虽然这种方式给研究者提供了进行有效研究所需要的控制与权限，但是并不能完全模仿出实际的生产和运行环境。

正如在第 4 章中所讨论的，YAFFS2 和 MTD 都涉及将数据写入 NAND 闪存。但是不幸的是，这样做就会使分析的复杂度大幅增加，因为 MTD 可以接受来自于 YAFFS2 的数据，这个过程需要写入到 NAND 闪存中去，这样不但有能够写入 YAFFS2 数据，而且有写入其他 MTD 数据的自由度，只要 MTD 认为合适即可。对这些数据进行研究就必须同时考虑 YAFFS2 代码和 MTD 代码。此外，应用在不同的 Android 终端设备上时，YAFFS2 模块和 MTD 模块之间还存在一些微妙的差别，这又增加了其他方面的复杂性。

这些困难和问题当然不表明了解 nanddump 中的 YAFFS2 数据是不可能的。我们先在 Ubuntu 工作站上设置一个拥有 64MB 的模拟 NAND 闪存设备：

更多资源请访问稀酷客(www.ckook.com)

```
sudo modprobe mtd
sudo modprobe mtdblock
sudo insmod ~/yaffs2/yaffs2.ko
sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xa2 third_id_byte=0x00
fourth_id_byte=0x15
```

验证一下是否已经正确地设置了 nandsim：

```
ahoog@ubuntu:~$ cat /proc/mtd
dev:    size   erasesize  name
mtd0: 04000000 00020000 "NAND simulator partition 0"
```

在挂载 YAFFS2 文件系统之前，我们需要开启内建在 YAFFS2 文件系统中的各种调试功能。YAFFS2 支持很多程序调试选项（YAFFS Debugging, n.d.），但是我们只准备开启其中的 3 个选项。要想继续往下做，我们首先需要 root 访问权限，然后使用 echo 命令将这些调试的标志写入“/proc/yaffs”：

```
ahoog@ubuntu:~$ sudo -s
root@ubuntu:~# echo =none+os+write+mtd > /proc/yaffs
```

echo 命令所产生的结果是首先去除任何现有的调试选项（没有），然后再开启 os、write 和 mtd 这 3 个调试选项。可以通过查看在“/var/log/syslog”的系统日志来查看调试的结果。最理想的方式是打开另外一个终端窗口或 ssh 会话，然后使用 tail 命令来连续输出 syslog 文件中的尾部结果：

```
ahoog@ubuntu:~$ tail -f /var/log/syslog
Feb 17 18:29:34 ubuntu kernel: [ 4474.970406] new trace = 0xF0004082
```

下一步，创建一个挂载点来挂载 YAFFS2 文件系统：

```
mkdir -p ~/mnt/yaffs2
sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/yaffs2
```

此时，检查师就应该创建一系列的测试用例，从而能够在模拟的 NAND 闪存上对期望控制的数据进行检查，以做到最终确定其结构。对于这个实验，可以采取以下的步骤：

1）在“~/mnt/yaffs2”下创建一个叫做 test 的目录（mkdir ~/mnt/yaffs2/test）；

2）创建文件“~/mnt/yaffs2/test/file1.txt”，内容为“viaforensics”（nano –w ~/mnt/yaffs2/test/file1.txt）；

3）更新文件“~/mnt/yaffs2/test”，将“updated”附加到此文件中去（echo“updated” >> ~/mnt/yaffs2/test/file1.txt）；

4）读取文件“~/mnt/yaffs2/test/file1.txt”（cat ~/mnt/yaffs2/test/file1.txt）；

5）对目录和文件，改变用户和组的拥有者（chown -R pulse.rtkit ~/mnt/yaffs2/test）；

6）改变目录和文件的访问权限（chmod -R 777 ~/mnt/yaffs2/test）。

随着执行以上这些测试步骤，不但文件系统被更新，而且调试信息也被写入到 syslog 文件中去。调试信息非常详细（我们只开启了 3 个调试选项而已），因此在这里只提供两

更多资源请访问稀酷客(www.ckook.com)

个实例，对应于以下的两行：

1）创建“~/mnt/yafs2/test”目录；

2）改变 file1.txt 文件的访问权限（chmod -R 777 ~/mnt/yaffs2/test）。

```
mkdir ~/mnt/yafs2/test
-----------------------------------
Feb 17 15:53:33 ubuntu kernel: [26704.104072] yaffs_lookup for 1:test
Feb 17 15:53:33 ubuntu kernel: [26704.104076] yaffs_lookup not found
Feb 17 15:53:33 ubuntu kernel: [26704.104078] yaffs_mkdir
Feb 17 15:53:33 ubuntu kernel: [26704.104080] yaffs_mknod: parent object 1 type 3
Feb 17 15:53:33 ubuntu kernel: [26704.104082] yaffs_mknod: making oject for test, mode 41ed dev 0
Feb 17 15:53:33 ubuntu kernel: [26704.104083] yaffs_mknod: making directory
Feb 17 15:53:33 ubuntu kernel: [26704.104191] yaffs_MarkSuperBlockDirty() sb = ffff88001bf74800
Feb 17 15:53:33 ubuntu kernel: [26704.104194] nandmtd2_ReadChunkWithTagsFromNAND chunk 0 data ffff88000d30c000 tags ffff880019f479f8
Feb 17 15:53:33 ubuntu kernel: [26704.104233] packed tags obj -1 chunk -1 byte -1 seq -1
Feb 17 15:53:33 ubuntu kernel: [26704.104235] ext.tags eccres 0 blkbad 0 chused 0 obj 0 chunk0 byte 0 del 0 ser 0 seq 0
Feb 17 15:53:33 ubuntu kernel: [26704.104237] packed tags obj -1 chunk -1 byte -1 seq -1
Feb 17 15:53:33 ubuntu kernel: [26704.104239] ext.tags eccres 1 blkbad 0 chused 0 obj 0 chunk0 byte 0 del 0 ser 0 seq 0
Feb 17 15:53:33 ubuntu kernel: [26704.104243] Writing chunk 0 tags 257 0
Feb 17 15:53:33 ubuntu kernel: [26704.104244] nandmtd2_WriteChunkWithTagsToNAND chunk 0 data ffff88000d30c800 tags ffff880019f47ba8
Feb 17 15:53:33 ubuntu kernel: [26704.104247] packed tags obj 805306625 chunk -2147483647 byte 0 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.104249] ext.tags eccres 0 blkbad 0 chused 1 obj 257 chunk0 byte 0 del 0 ser 1 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.104466] nandmtd2_ReadChunkWithTagsFromNAND chunk 0 data ffff88000d30c000 tags ffff880019f47aa8
Feb 17 15:53:33 ubuntu kernel: [26704.104471] packed tags obj 805306625 chunk -2147483647 byte 0 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.104474] ext.tags eccres 0 blkbad 0 chused 1 obj 257 chunk0 byte 0 del 0 ser 0 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.104476] packed tags obj 805306625 chunk -2147483647 byte 0 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.104478] ext.tags eccres 1 blkbad 0 chused 1 obj 257 chunk0 byte 0 del 0 ser 0 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.104487] yaffs_get_inode for object 257
Feb 17 15:53:33 ubuntu kernel: [26704.104488] yaffs_iget for 257
Feb 17 15:53:33 ubuntu kernel: [26704.104492] yaffs_FillInode mode 41ed uid 0 gid 0 size 2048 count 1
Feb 17 15:53:33 ubuntu kernel: [26704.104494] yaffs_mknod created object 257 count = 1
Feb 17 15:53:33 ubuntu kernel: [26704.330210] yaffs_MarkSuperBlockDirty() sb = ffff88001bf74800
Feb 17 15:53:33 ubuntu kernel: [26704.330214] Writing chunk 1 tags 1 0
Feb 17 15:53:33 ubuntu kernel: [26704.330217] nandmtd2_WriteChunkWithTagsToNAND chunk 1 data ffff88000d30c800 tags ffff880011c3dd00
Feb 17 15:53:33 ubuntu kernel: [26704.330219] packed tags obj 805306369 chunk -2147483648 byte 0 seq 4097
Feb 17 15:53:33 ubuntu kernel: [26704.330221] ext.tags eccres 0 blkbad 0 chused 1 obj 1 chunk0 byte 0 del 0 ser 1 seq 4097
Feb 17 15:53:37 ubuntu kernel: [26708.060113] yaffs_write_super
Feb 17 15:53:37 ubuntu kernel: [26708.060118] yaffs_do_sync_fs: dirty no checkpoint
Feb 17 15:53:37 ubuntu kernel: [26708.060121] flushing obj 257
Feb 17 15:53:37 ubuntu kernel: [26708.060122] flushing obj 2
Feb 17 15:53:37 ubuntu kernel: [26708.060123] flushing obj 1

chmod -R 777 ~/mnt/yaffs2/test
-----------------------------------
Feb 17 15:59:00 ubuntu kernel: [27030.833325] yaffs_setattr of object 257
Feb 17 15:59:00 ubuntu kernel: [27030.833331] inode_setattr called
Feb 17 15:59:00 ubuntu kernel: [27030.833338] nandmtd2_ReadChunkWithTagsFromNAND chunk 11 data ffff88000d30c800 tags ffff88002e2efc38
Feb 17 15:59:00 ubuntu kernel: [27030.833356] packed tags obj 805306625 chunk -2147483647 byte 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833358] ext.tags eccres 0 blkbad 0 chused 1 obj 257 chunk0 byte 0 del 0 ser 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833360] packed tags obj 805306625 chunk -2147483647 byte 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833362] ext.tags eccres 1 blkbad 0 chused 1 obj 257 chunk0 byte 0 del 0 ser 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833366] yaffs_MarkSuperBlockDirty() sb = ffff88001bf74800
Feb 17 15:59:00 ubuntu kernel: [27030.833368] Writing chunk 12 tags 257 0
Feb 17 15:59:00 ubuntu kernel: [27030.833369] nandmtd2_WriteChunkWithTagsToNAND chunk 12 data ffff88000d30c800 tags ffff88002e2efc88
Feb 17 15:59:00 ubuntu kernel: [27030.833372] packed tags obj 805306625 chunk -2147483647 byte 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833378] ext.tags eccres 0 blkbad 0 chused 1 obj 257 chunk0 byte 0 del 0 ser 4 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833390] yaffs_setattr done returning 0
Feb 17 15:59:00 ubuntu kernel: [27030.833485] yaffs_readdir: starting at 0
Feb 17 15:59:00 ubuntu kernel: [27030.833486] yaffs_readdir: entry . ino 257
Feb 17 15:59:00 ubuntu kernel: [27030.833488] yaffs_readdir: entry .. ino 1
Feb 17 15:59:00 ubuntu kernel: [27030.833490] yaffs_readdir: file1.txt inode 258
Feb 17 15:59:00 ubuntu kernel: [27030.833493] yaffs_readdir: starting at 3
Feb 17 15:59:00 ubuntu kernel: [27030.833500] yaffs_setattr of object 258
Feb 17 15:59:00 ubuntu kernel: [27030.833501] inode_setattr called
Feb 17 15:59:00 ubuntu kernel: [27030.833504] nandmtd2_ReadChunkWithTagsFromNAND chunk 10 data ffff88000d30c800 tags ffff88002e2efc38
Feb 17 15:59:00 ubuntu kernel: [27030.833510] packed tags obj 268435714 chunk -2147483391 byte 21 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833512] ext.tags eccres 0 blkbad 0 chused 1 obj 258 chunk0 byte 0 del 0 ser 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833514] packed tags obj 268435714 chunk -2147483391 byte 21 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833517] ext.tags eccres 1 blkbad 0 chused 1 obj 258 chunk0 byte 0 del 0 ser 0 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833519] yaffs_MarkSuperBlockDirty() sb = ffff88001bf74800
Feb 17 15:59:00 ubuntu kernel: [27030.833520] Writing chunk 13 tags 258 0
Feb 17 15:59:00 ubuntu kernel: [27030.833522] nandmtd2_WriteChunkWithTagsToNAND chunk 13 data ffff88000d30c800 tags ffff88002e2efc88
Feb 17 15:59:00 ubuntu kernel: [27030.833524] packed tags obj 268435714 chunk -2147483391 byte 21 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833526] ext.tags eccres 0 blkbad 0 chused 1 obj 258 chunk0 byte 0 del 0 ser 7 seq 4097
Feb 17 15:59:00 ubuntu kernel: [27030.833533] yaffs_setattr done returning 0
Feb 17 15:59:01 ubuntu kernel: [27032.060195] yaffs_write_super
Feb 17 15:59:01 ubuntu kernel: [27032.060205] yaffs_do_sync_fs: dirty no checkpoint
Feb 17 15:59:01 ubuntu kernel: [27032.060208] flushing obj 258
Feb 17 15:59:01 ubuntu kernel: [27032.060209] flushing obj 257
Feb 17 15:59:01 ubuntu kernel: [27032.060210] flushing obj 2
Feb 17 15:59:01 ubuntu kernel: [27032.060212] flushing obj 1
```

更多资源请访问稀酷客(www.ckook.com)

调试能够提供很多有价值的信息，包括对象 ID、序列号、子块 ID 以及 YAFFS2 创建文件的详细过程。如果开启更多的调试选项，在了解 YAFFS2 过程中就会有更多的数据可以进行关联与使用。

在我们检查实际的 nanddump 之前，还有一个很有用的命令值得提一下。这个 stat 命令用于获得关于文件、目录以及其他文件系统实体的详细信息。例如，我们可以对“~/mnt/yaffs2/test”和“~/mnt/yaffs2/test/file1.txt”使用 stat，在后续的研究中使用所得到的信息：

```
root@ubuntu:~/mnt/yaffs2# stat test
  File: `test'
  Size: 2048            Blocks: 4          IO Block: 4096   directory
Device: 1f00h/7936d     Inode: 257         Links: 1
Access: (0777/drwxrwxrwx)  Uid: (  109/   pulse)   Gid: (  117/   rtkit)
Access: 2011-02-17 15:53:33.000000000 -0600
Modify: 2011-02-17 15:55:02.000000000 -0600
Change: 2011-02-17 15:59:00.000000000 -0600

root@ubuntu:~/mnt/yaffs2# stat test/file1.txt
  File: `test/file1.txt'
  Size: 21              Blocks: 1          IO Block: 4096   regular file
Device: 1f00h/7936d     Inode: 258         Links: 1
Access: (0777/-rwxrwxrwx)  Uid: (  109/   pulse)   Gid: (  117/   rtkit)
Access: 2011-02-17 15:55:02.000000000 -0600
Modify: 2011-02-17 15:56:13.000000000 -0600
Change: 2011-02-17 15:59:00.000000000 -0600
```

我们现在就得到了文件和目录的 Modified（修改）、Access（访问）和 Change（改变）特性的信息，再与其他用于创建文件的活动相结合，我们就拥有了开始进行研究的足够信息。

是查看 NAND 闪存的时候了，当然需要 root 访问权限。以下的命令用于忽略 NAND 闪存中的行，因为都是 0xFF 和 0x00，使我们很容易地就可以看到大的变化趋势。同时，为了节省空间，在此以十六进制的方式只显示很小一部分的 NAND 闪存内容：

```
nanddump -c /dev/mtd0ro | grep -v "00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00" |
grep -v "ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff" | less
0x00000000: 30 00 00 00 10 00 00 00 ff ff 47 56 37 47 00 00  |..........test..|
0x00000100: 00 00 00 00 00 00 00 00 00 00 ff ff de 14 00 00  |..............A..|
0x00000110: 00 00 00 00 00 00 00 00 dd 89 d5 d4 dd 89 d5 d4  |..........]M..]M|
0x00000120: dd 89 d5 d4 ff ff ff ff ff ff ff ff ff ff ff ff  |..]M............|
0x000001c0: ff ff ff ff ff ff ff ff ff ff ff ff 00 00 00 00  |................|
0x000001e0: ff ff ff ff ff ff ff ff 00 00 00 00 ff ff ff ff  |................|
0x000001f0: ff ff ff ff ff ff ff ff 00 00 00 00 00 00 00 00  |................|
  OOB Data: ff ff 10 01 00 00 10 10 00 03 10 00 00 08 00 00  |.........0......|
  OOB Data: 00 00 c0 a7 4f 91 30 00 00 00 30 00 00 00 ff ff  |...z............|
  OOB Data: ff ff ff ff ff ff ff ff ff ff f3 f0 ff 0f 3c ff ff  |.........?......|
  OOB Data: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff  |................|
<snip>
0x00006800: 10 00 00 00 10 10 00 00 ff ff 66 96 c6 56 13 e2  |..........file1.|
0x00006810: 47 87 47 00 00 00 00 00 00 00 00 00 00 00 00 00  |txt.............|
0x00006900: 00 00 00 00 00 00 00 00 00 00 ff ff ff 18 00 00  |................|
```

更多资源请访问稀酷客(www.ckook.com)

```
0x00006910: d6 00 00 00 57 00 00 00 63 99 d5 d4 d7 99 d5 d4  |m...u...6.]M}.]M|
0x00006920: 42 a9 d5 d4 51 00 00 00 ff ff ff ff ff ff ff ff  |$.]M............|
0x000069c0: ff ff ff ff ff ff ff ff ff ff ff ff 00 00 00 00  |................|
0x000069e0: ff ff ff ff ff ff ff ff 00 00 00 00 ff ff ff ff  |................|
0x000069f0: ff ff ff ff ff ff ff ff 00 00 00 00 00 00 00 00  |................|
  OOB Data: ff ff 10 01 00 00 20 10 00 01 10 10 00 08 51 00  |................|
  OOB Data: 00 00 51 af e2 e2 10 00 00 00 ef ff ff ff ff ff  |................|
  OOB Data: ff ff ff ff ff ff ff ff ff ff 00 3c ff 3c ff ff ff  |................|
  OOB Data: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff  |................|
```

显示的部分 NAND 闪存内容包含了以下方面：

1）创建“~/mnt/yaffs2/test”目录；

2）改变文件 file1.txt 的访问权限（chmod -R 777 ~/mnt/yaffs2/test）。

所显示的数据是 YAFFS2 中 ObjectHeaders 的一部分，包含了此文件系统的元数据。在 64 字节的 OOB 区域里可以清楚地看到文件名。关于这个数据的一个重要特性是表达 UNIX 时间戳的整数按照从小到大的字节顺序存储，意味着这个数据必须从右到左读出来。

- 写入 NAND 闪存的数：63 99 d5 d4（0x6399d5d4）；
- 从小字节顺序转换到大字节顺序：4d 5d 99 36（0x4d5d9936，以十六进制从右至左读取）；
- 将 0x4d5d9936（十六进制）转换为十进制的数为 1297979702；
- UNIX 时间戳 1297979702 以可读格式表达为 Thu Feb 17 15:55:02 CST 2011（星期四 2 月 17 日 15:55:02 中部标准时间 2011 年）（date -d @1297979702）。

运用这个信息，可以从 nanddump 中分离出很多重要的信息，如表 7.3 所示。

表 7.3　源自 YAFFS2 nanddump 的信息

偏移值	十六进制	十进制	转换为	说　明
0x00000118–11B	dd 89 d5 d4	1 297 979 613	Thu Feb 17 15:53:33 CST 2011（星期四 2 月 17 日 15:53:33 美国中部标准时间 2011 年）	这是所创建的目录的 atime（访问时间），由于目录刚建立，这个时间与修改和改变时间相同。这个时间对应于调试语句中的日期 / 时间
0x0000011C–11F	dd 89 d5 d4	1 297 979 613	Thu Feb 17 15:53:33 CST 2011（星期四 2 月 17 日 15:53:33 美国中部标准时间 2011 年）	这是目录的 mtime（修改时间），在目录刚创建时与 atime 相同
0x00000120–123	dd 89 d5 d4	1 297 979 613	Thu Feb 17 15:53:33 CST 2011（星期四 2 月 17 日 15:53:33 美国中部标准时间 2011 年）	这是目录的 ctime（元数据改变时间），在目录刚创建时与 atime 相同
Bytes 3–6 in OOB	10 01 00 00	4 097	N/A（不适用）	块的序列号
Bytes 7–10 in OOB	10 10 00 03	805 306 625	N/A（不适用）	目录测试的内部物理 ID，与调试数据一致

更多资源请访问稀酷客(www.ckook.com)

（续）

偏移值	十六进制	十进制	转换为	说　明
0x00006918–691B	63 99 d5 d4	1 297 979 702	Thu Feb 17 15:55:02 CST 2011（星期四 2 月 17 日 15:55:02 美国中部标准时间 2011 年）	文件的 atime，虽然被访问，但是由于是创建文件，此时间没有被更新
0x0000691C–691F	d7 99 d5 d4	1 297 979 773	Thu Feb 17 15:56:13 CST 2011（星期四 2 月 17 日 15:56:13 美国中部标准时间 2011 年）	文件的 mtime，与文件内容被更新保持一致
0x00006920–6923	42 a9 d5 d4	1 297 979 940	Thu Feb 17 15:59:00 CST 2011（星期四 2 月 17 日 15:59:00 美国中部标准时间 2011 年）	文件的 ctime，与调试日志中记录的对访问权限的改变保持一致
Bytes 3–6 in OOB	10 01 00 00	4097	N/A（不适用）	块的序列号，与存储在 128KB 块中的所有数据保持一致，因此，不会分配一个新的序号
Bytes 7–10 in OOB	20 10 00 01	268 435 714	N/A（不适用）	file1.txt 文件的内部物理 ID，与调试数据一致

令人满意的是，来自于调试日志、stat 命令以及模拟 NAND 闪存设备中 nanddump 的数据都是相互对应的。通过进行更多的分析，就极有可能为 NAND 闪存上的每一个文件和目录都生成 MAC 时间，使取证检查师从中受益。同时也可以从仍然存在于 NAND 闪存上的 ObjectHeaders 中获取元数据以及在针对块的垃圾回收操作没有发生之前每一个时间点上的所有与时间相关的结点。

值得指出的是，在此节所展示的有限的测试和分析中，文件的 atime（被访问的时间）看起来好像并没有随着文件的每一次被访问而改变。这一点也不让人感到惊讶，因为这就说明当一个程序访问一个文件时，一个新的 ObjectHeader 会被写入到 NAND 闪存上。这样做所带来的后果是需要对 NAND 闪存进行大量写入操作，这不但会消耗电池中宝贵的电量，而且还会由于元数据更新（大多数用户并不真正关心）而造成 NAND 闪存磨损。这种情况时常发生，这是由于微软的 Windows Vista 和 Windows 7 在默认情况下关闭 atime 更新。

需要确认的是，这种分析不是很简单。但是这样的分析可以提供有价值的信息，是对 YAFFS2 文件系统开展取证研究的基础。

7.4.2 YAFFS2 文件系统分析

对 YAFFS2 文件系统进行十六进制分析是非常耗费时间的一件事情，那么就让我们继续去研究对存在的文件进行分析的技术。很不幸的是，这样做也要面临很多挑战，因为数据从 Android 终端设备抽取出来后，将一个 YAFFS2 的 nanddump 进行挂载也可能非常困难。

更多资源请访问稀酷客(www.ckook.com)

正如在第 6 章中指出过的那样，由于这个原因，如果在 Android 终端设备上具有 root 访问权限，最好是既要获取适合的 YAFFS2 nanddump 文件，还要使用诸如 adb pull、tar 或其他的逻辑技术将系统中重要的目录都复制出来。这样的话，即使无法将 YAFFS2 nanddump 成功挂载，我们仍然有这些文件的逻辑版本。然后，可以使用 nanddump 进行时间序列的生成、十六进制分析以及文件雕复。

然而，还是有可能在 Linux 系统中挂载某些 YAFFS2 nanddump，并且随着时间的推移，期望会有更多的 nanddump 文件可以成功挂载。在这一节中，我们将重点放在从摩托罗拉 Droid 上获取 nanddump，因为它可以成功地挂载到一个安装了 nandsim 以及 YAFFS2 内核模块的 Linux 系统上。

首先，我们需要安装所需的内核模块，并创建一个容量为 1GB 的 nandsim 终端设备：

```
sudo modprobe mtd
sudo modprobe mtdblock
sudo insmod ~/yaffs2/yaffs2.ko
sudo modprobe nandsim first_id_byte=0xec second_id_byte=0xd3 third_id_byte=0x51
fourth_id_byte=0x95
```

我们现在就具有了一个 1GB 的 nandsim 终端设备，可以容纳下 Droid 中的“/data”分区。下一步是使用 nandwrite 命令将数据和 OOB 区域复制到这个模拟 NAND 闪存中去：

```
ahoog@ubuntu:~$ sudo nandwrite --autoplace --oob /dev/mtd0 droid_userdata.
nanddump
Writing data to block 0 at offset 0x0
Writing data to block 1 at offset 0x20000
Writing data to block 2 at offset 0x40000
Writing data to block 3 at offset 0x60000
Writing data to block 4 at offset 0x80000
Writing data to block 5 at offset 0xa0000
<snip>
```

以上命令中的参数指示 nandwrite 完成以下功能：

- sudo nandwrite：nandwrite 要求 root 访问权限；
- --autoplace：对 OOB 使用自动布局；
- --oob：映像中包含 OOB 数据；
- /dev/mtd0：用于写入 nanddump 的 MTD 设备；
- droid_userdata.nanddump：nanddump 文件的名称。

在一切都正常执行后，我们现在就可以使用以下命令挂载此文件系统：

```
ahoog@ubuntu:~$ mkdir -p ~/mnt/yaffs2
ahoog@ubuntu:~$ sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/yaffs2
```

在挂载过程中，可能会出现两个主要问题。第一个常见的问题是 YAFFS2 及 MTD 也许无法正确地挂载此文件系统，此时会显示以下信息：

更多资源请访问稀酷客(www.ckook.com)

```
ahoog@ubuntu:~$ sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/yaffs2
mount: wrong fs type, bad option, bad superblock on /dev/mtdblock0,
       missing codepage or helper program, or other error
       In some cases useful info is found in syslog - try
       dmesg | tail  or so
```

造成这个问题的可能原因有很多，例如：

- 终端设备中所实现的 YAFFS2 和 MTD 与取证工作站存在一些差别；
- 损坏或无效的 nanddump；
- 文件系统中的问题。

第二个常见的问题是虽然挂载完成，但是却没有任何文件：

```
ahoog@ubuntu:~/mnt/yaffs2$ ls
lost+found
```

这个问题的可能原因是终端设备中实现的 YAFFS2 和 MTD 与取证工作站之间的差别，同样也是非常难以确定。可以尝试一些方法来解决这些问题。首先，如果 mtu-utils 的版本在 20090606-1 之后，试一下用旧的版本。可以用以下方法确定目前使用的版本，并且根据具体情况进行安装：

```
ahoog@ubuntu:~$ dpkg -l | grep mtd-utils
ii  mtd-utils                          20100706-1
Memory Technology Device Utilities

ahoog@ubuntu:~$ sudo apt-get remove mtd-utils
ahoog@ubuntu:~$ wget http://mirror.pnl.gov/ubuntu//pool/universe/m/mtd-
utils/mtd-utils_20090606-1ubuntu0.10.04.1_amd64.deb
ahoog@ubuntu:~$ sudo dpkg -i mtd-utils_20090606-1ubuntu0.10.04.1_amd64.deb

ahoog@ubuntu:~$ dpkg -l | grep mtd-utils
ii  mtd-utils                          20090606-1ubuntu0.10.04.1
Memory Technology Device Utilities
```

这样我们就下载了这个版本的 MTD，然后就可以再试着去重新挂载一个 YAFFS2 的 nanddump。

另外一个可能性是，目前使用的 YAFFS2 版本与 Android 终端设备上使用的版本不是完全一致。必须使用 YAFFS2 或者 Android 开源项目（Android Open Source Project）中的源代码控制系统来得到不同版本的 YAFFS2。使用以上方法得到源代码后，就可以对源代码进行编译，删除现有的 YAFFS2 内核模块，用新的模块去替代，然后再去试。

如果以上这些步骤都顺利的话，将会看到以下结果：

```
ahoog@ubuntu:~$ ls -l ~/mnt/yaffs2/
total 25
drwxrwx--x 1 ahoog ahoog 2048 2010-10-11 15:21 app
drwxrwx--x 1 ahoog ahoog 2048 2010-10-07 13:53 app-private
drwx------ 1 ahoog ahoog 2048 2010-10-11 20:16 backup
-rw-rw-rw- 1 root  root     8 2010-10-11 20:18 cc_data
drwxrwx--x 1 ahoog ahoog 2048 2010-10-11 15:21 dalvik-cache
drwxrwx--x 1 ahoog ahoog 2048 2010-10-11 15:21 data
drwxr-x--- 1 root   1007 2048 2010-10-07 13:53 dontpanic
```

更多资源请访问稀酷客(www.ckook.com)

```
drwxrwx--x 1  2000  2000 2048 2010-10-07 13:53 local
drwxrwx--- 1 root  root  2048 2010-10-07 13:53 lost+found
drwxrwx--t 1 ahoog  9998 2048 2010-10-11 19:02 misc
drwx------ 1 root  root  2048 2010-10-11 19:38 property
drwxrwxr-x 1 ahoog ahoog 2048 2010-10-11 20:20 system
drwxr-xr-x 1 ahoog ahoog 2048 2010-10-11 14:55 tombstones

ahoog@ubuntu:~$ df -h
Filesystem            Size  Used Avail Use% Mounted on
/dev/mtdblock0        1.0G   65M  960M   7% /home/ahoog/mnt/yaffs2
```

所以，我们现在在 Ubuntu 工作站上就有了从摩托罗拉 Droid 上获得的完整的“/data”文件系统，从工作站上可以进行访问，使用熟悉的取证工具对数据进行研究和分析。在最坏的情况下，如果还是无法将 nanddump 挂载到 Ubuntu 工作站上，我们还是有在获取过程中所得到的一套完整的逻辑文件，因此也不会禁止我们对终端设备进行分析。

7.4.3　YAFFS2 文件雕复

另一个对 YAFFS2 文件系统进行分析的有用技术是文件雕复。在本章的前面部分，我们已经在 Ubuntu 工作站上安装并配置了 scalpel。在下面的举例中，我们假设在 Ubuntu 虚拟机上，在登录用户的主目录中存在一个名为 htc-datadata.dd 的文件。在同一个目录中（在所有的命令中，我们都可以使用一个 ~ 来代表主目录），已经存在一个 scalpel 配置，其中包含了 1.3 节的文件 named-scalpelandroid.conf 中的所有条目。

请注意，对于文件雕复，我们使用的是 dd 映像，而不是 nanddump。这样做的必要性在于，在 YAFFS2 数据中，每一个 2KB 子块后跟随的 OOB 数据会对 scalpel 雕复有效文件的能力产生巨大的影响。以下命令将针对 dd 映像使用 scalpel 命令，并将文件输出放置在主目录下一个叫做 htc-scalpel-test 的文件夹中：

```
ahoog@ubuntu:~$ scalpel -c ~/scalpel-android.conf ~/htc-datadata.dd -o
~/htc-scalpel-test
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/ahoog/htc-datadata.dd"

Image file pass 1/2.
/home/ahoog/htc-datadata.dd: 100.0%
|*************************************************************|  149.0 MB
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built.  Workload:
gif with header "\x47\x49\x46\x38\x37\x61" and footer "\x00\x3b" --> 16 files
gif with header "\x47\x49\x46\x38\x39\x61" and footer "\x00\x3b" --> 385 files
jpg with header "\xff\xd8\xff\xe0\x00\x10" and footer "\xff\xd9" --> 2140 files
jpg with header "\xff\xd8\xff\xe1" and footer "\x7f\xff\xd9" --> 18 files
png with header "\x50\x4e\x47\x3f" and footer "\xff\xfc\xfd\xfe" --> 0 files
png with header "\x89\x50\x4e\x47" and footer "" --> 1442 files
db with header "\x53\x51\x4c\x69\x74\x65\x20\x66\x6f\x72\x6d\x61\x74" and
footer "" --> 5453 files
email with header "\x46\x72\x6f\x6d\x3a" and footer "" --> 1183 files
```

更多资源请访问稀酷客(www.ckook.com)

```
doc with header "\xd0\xcf\x11\xe0\xa1\xb1\x1a\xe1\x00\x00" and footer
"\xd0\xcf\x11\xe0\xa1\xb1\x1a\xe1\x00\x00" --> 0 files
doc with header "\xd0\xcf\x11\xe0\xa1\xb1" and footer "" --> 0 files
htm with header "\x3c\x68\x74\x6d\x6c" and footer
"\x3c\x2f\x68\x74\x6d\x6c\x3e" --> 732 files
pdf with header "\x25\x50\x44\x46" and footer "\x25\x45\x4f\x46\x0d" -->
1 files
pdf with header "\x25\x50\x44\x46" and footer "\x25\x45\x4f\x46\x0a" -->
0 files
wav with header "\x52\x49\x46\x46\x3f\x3f\x3f\x3f\x57\x41\x56\x45" and
footer "" --> 0 files
amr with header "\x23\x21\x41\x4d\x52" and footer "" --> 0 files
zip with header "\x50\x4b\x03\x04" and footer "\x3c\xac" --> 0 files
java with header "\xca\xfe\xba\xbe" and footer "" --> 0 files
Carving files from image.
Image file pass 2/2.
/home/ahoog/htc-datadata.dd: 100.0%
|*******************************************************|  149.0 MB
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 11370, elapsed = 4 seconds.
```

在此例中，使用 scalpel 恢复了 11 370 个文件，在输出中提供了文件类型的详细信息和恢复的文件数量。图 7.11 显示的是在 Ubuntu 工作站上查看其中一个 JPG 目录的截图。

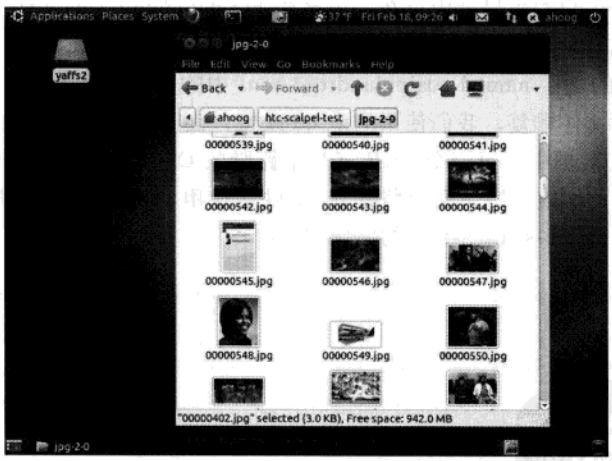


图 7.11　查看使用 scalpel 工具恢复出来的图像

此图的分辨率不是很高，但是，当直接在工作站上显示时，可以不要太过于关注显示的内容。以下是一些图像所显示出的一部分信息：

- 最上面 3 个图片是 YouTube 影视的打开框，在这里就是龙的卡通；
- 下一个图片是 Facebook 报文的一部分，询问午餐并推荐寿司；
- 剩下的图片看上去来自于各类新闻稿。

以上这个例子说明，文件雕复可以用于恢复 YAFFS2 文件系统中的重要元素，对于取证分析来说是一个非常有价值的工具。

更多资源请访问稀酷客(www.ckook.com)

7.4.4　YAFFS2 的 strings 分析

我们已经举出了一个使用 strings 命令从 Android 终端设备上找到数据的例子。但是，还是让我们通过另外一个练习来展示一下 strings 技术本身和这一类分析的力度。

在此例中，取证检查师希望在 Android 终端设备上找到位置信息。首当其冲是在 Android 终端设备上寻找 maps.google.com，这是由于谷歌地图服务已经集成在 Android 操作系统中。有个功能异常强大的查询工具 grep，此工具得到广泛应用的程度使 grep 这个词现在已经成为技术词汇中的一个动词（“你有没有 grep 这个终端设备，将它用于地图参考？”）。此工具不但已经包含在 Ubuntu 虚拟机中，而且还能够支持功能非常强大的正则表达式（常常缩写为 regex 或 regexp），在查询中允许复杂的模式匹配。目前已经有很多在线资源和信息通过实例来介绍 grep 和 regex。

在本例中，我们将使用 grep 最基本的功能，用它来查找字符串：

```
ahoog@ubuntu:~$ strings --all --radix=x htcinc-mtd6-datadata.nanddump | grep
maps.google.com | less
691e6 Nhttp://maps.google.com/?q=Naperville+Crime+Prevention&
cid=14683103809248498655 8d2e7cc
Vhttp://maps.google.com/?q=Kennedy+Space+Center+Visitor+Complex&
cid=3639318623825085463 8d30fce
http://maps.google.com/maps/gen_204?oi=miwd&sa=X&ct=miw_details&cd=1&ei=3oVUTdC
aNZn2iwPg2dyOBQ&q=Kennedy+near+Space+Center,+FL&cad=ip:174.253.2.53,client:
maps-mobile,cid:3639318623825085463,src:link
```

这个询问返回了 775 个结果，但是出于简洁与隐私的原因，我们只显示几个而已。然而，的确是很容易就可以确定发生在终端设备上的一些事情。具体来说：

- 执行了查找 Naperville Crime Prevention（在 URL 中的 q 代表询问）的活动，使用的 URL 是有效、完整的，包含有地址、电话号码、相片、方向说明等；
- 也执行了对 Kennedy Space Center Visitor Complex 的查找；
- 对 Kennedy Space Center 的询问请求被移动地图应用所查看，我们甚至可以看到 IP 地址。

我们来看看这个 IP 地址，它是最后一个查询请求的一部分。我们可以使用命令行工具或者在线 IP 地址查询服务（What Is My IP Address, n.d.）来发现更多的信息。这些结果如图 7.12 所示。

从 IP 地址查询服务返回的信息揭示出一些重要的信息：

1）这个终端设备使用的是 Verizon 无线网络提供的服务；

2）这个终端设备位于伊利诺伊、芝加哥地区（也有可能会是假冒或虚假的 IP 地址，但这不容易做到，在大多数情况下是不可能的）。

因为有这个恢复的字符串的十六进制偏移值，所以可以看看前后的数据，以确定查询发生的大致时间段。

更多资源请访问稀酷客(www.ckook.com)

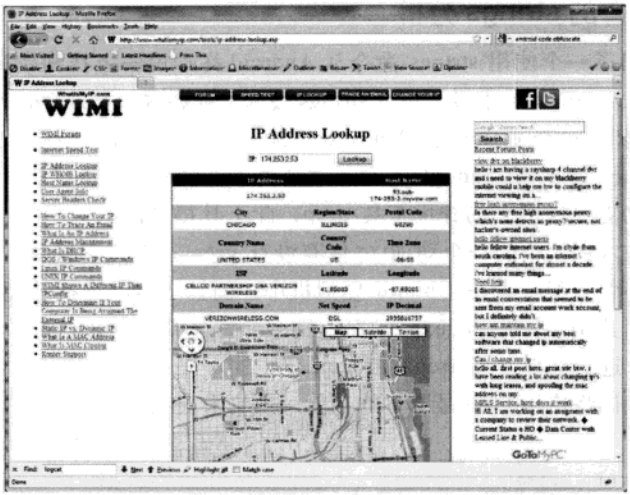


图 7.12 IP 地址查询结果

7.4.5 YAFFS2 分析师注意事项

很多的 YAFFS2 取证所需要的人工分析肯定是非常耗时的，并且也有点儿枯燥，这就使开展这些分析的门槛远远高于得到广泛了解和支持的文件系统的。然而，分析师们仍然可以获得很多资源来帮助他们去实践这个新的方法。

首先，希望本书很好地介绍了使用 Linux 进行取证分析的技术。（请向你的朋友们和同事们推荐将此书。）

也许更有帮助的是一个简单的谷歌搜索，它通常会返回发布在博客、讨论版块、邮件列表等很多在线资源的有用信息。虽然在互联网上肯定会存在不准确的信息，但是很多在这个技术领域工作的技术人员都会发布比较准确和正确的信息并且很多信息发布者对于询问和澄清请求都会及时予以回复。

接下来，执法机构和取证分析师们可以在线免费地得到非常棒的 Linux 介绍材料。网站 Linux LEO: The Law Enforcement and Forensic Examiner’s Introduction to Linux 以 PDF 格式的方式提供指导，给读者介绍使用 Linux 操作系统作为取证工具需要了解的概念（Grundy, B., n.d.）。

在使用 Linux 一段时间后，用户会发现将很多手工任务进行自动化其实很容易。自动化不但可以极好地控制系统的使用，而且还可以节省大量的时间。这是一个非常好的结合，使繁忙的检查师们能够使用先进的工具处理更多的取证数据，同时又能够很清晰地解释系统都做了一些什么工作，而不只是简单地叙述操作，然后报告就产生了。

对常见的 Linux 任务进行自动化的最简单的方式就是编写 shell 脚本程序，这是非常容易做到的，基本上就是依样在终端会话框中键入命令。

更多资源请访问稀酷客(www.ckook.com)

例如，如果我们有一系列的 .csv 文件，其包含在 Android 终端设备上使用 AFLogical 取证技术得到的结果。你想将每一个文件中的第一行发送给某人，但是又不要包含从终端设备中获取的任何数据。要做到这一点，你可以用手工方式打开每一个文件，删除所有的数据行，将修改过的文件保存到一个新的目录下，然后再将这些信息发送出去。但是，这样做需要花费很多时间。可行的方法是，编写一个简单的 shell 脚本程序。

首先，让我们打开一个新的文件进行编辑：

```
ahoog@ubuntu:~$ nano -w ~/sanitize-csv.sh
```

然后将以下内容放置在文件中：

```
#!/bin/bash

# create a new directory to store the sanitized files
mkdir sanitized

#for each file ending with .csv
for f in *.csv
do
        #read the top 1 row of the file and save it to the sanitized directory
        #with the same filename followed by -1strowonly
        head -1 "$f" > sanitized/"$f"-1strowonly
done

#create a "tar gzip" archive of the file so it is easier share
tar czvf AFlogical-sanitized.tgz sanitized/*
```

然后，我们按 Ctrl+O 来保存这个文件，再按 Ctrl+X 退出编辑器。下一步，我们需要使这个 shell 脚本程序成为可执行程序：

```
ahoog@ubuntu:~$ chmod 755 ~/sanitize-csv.sh
```

这些被清理的文件放置在用户的主目录下一个叫做 AFLogical 的目录中，正如我们都知道的，手工清理非常耗时：

```
ahoog@ubuntu:~/AFLogical$ ls
Browser Bookmarks.csv            IM Providers.csv
Browser Searches.csv             IM ProviderSettings.csv
CallLog Calls.csv                info.xml
Contacts ContactMethods.csv      Internal Image Media.csv
Contacts Extensions.csv          Internal Image Thumb Media.csv
Contacts Groups.csv              Internal Videos.csv
Contacts Organizations.csv       Maps-Friends contacts.csv
Contacts Phones.csv              Maps-Friends.csv
Contacts Settings.csv            Maps-Friends extra .csv
External Image Media.csv         MMS.csv
External Image Thumb Media.csv   MMSParts.csv
External Media.csv               People.csv
External Videos.csv              PhoneStorage (HTC Incredible).csv
IM Account.csv                   sanitize.sh
IM Accounts.csv                  Search History.csv
IM Chats.csv                     SMS.csv
IM Contacts.csv                  Social Contracts Activities.csv
IM Invitations.csv
```

更多资源请访问稀酷客(www.ckook.com)

我们现在可以很容易地处理这些文件：

```
ahoog@ubuntu:~/AFLogical$ cd ~/AFLogical
ahoog@ubuntu:~/AFLogical$ ~/sanitize-csv.sh
sanitized/Browser Bookmarks.csv-1strowonly
sanitized/Browser Searches.csv-1strowonly
sanitized/CallLog Calls.csv-1strowonly
<snip>
```

现在，在 AFLogical 目录下不但多了一个子目录 sanitized（包含更新过的文件），而且还有一个包含数据的文件：

```
ahoog@ubuntu:~/AFLogical$ ls -lh *.tgz
-rw-r--r-- 1 ahoog ahoog 2.1K 2011-02-19 03:44 AFlogical-sanitized.tgz
```

这些数据很容易地就可以通过邮件进行发送，或者是复制到另外磁盘上去。

以下是一个更加复杂一点的例子，目的是创建一个 nandsim 设备、挂载一个空的 YAFFS2 文件系统、产生几个文件、再允许用户卸载该文件系统。这是将测试 YAFFS2 所涉及的步骤进行自动化的简洁方式。

```
#!/bin/bash

read -p "Load kernel modules? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo modprobe mtd
     sudo modprobe mtdblock
     sudo insmod ~/yaffs2/yaffs2.ko
else
     exit
fi
echo ""
echo "Choose nandsim size in MiB [64, 128, 256, 512, 1024]"
echo "1) 64 MiB"
echo "2) 128 MiB"
echo "3) 256 MiB"
echo "4) 512 MiB"
echo "5) 1024 MiB"
read size
case $size in
     1) sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xa2
third_id_byte=0x00 fourth_id_byte=0x15 ;;
     2) sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xa1
third_id_byte=0x00 fourth_id_byte=0x15 ;;
     3) sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xaa
third_id_byte=0x00 fourth_id_byte=0x15 ;;
     4) sudo modprobe nandsim first_id_byte=0x20 second_id_byte=0xac
third_id_byte=0x00 fourth_id_byte=0x15 ;;
     5) sudo modprobe nandsim first_id_byte=0xec second_id_byte=0xd3
third_id_byte=0x51 fourth_id_byte=0x15 ;;
     *)
     echo "invalid nandsim size"
     exit
     ;;
esac
```

更多资源请访问稀酷客(www.ckook.com)

```
read -p "Mount yaffs2 in ~/mnt/y? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     echo ""
     echo "Mounting with sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/y"
     sudo mount -t yaffs2 /dev/mtdblock0 ~/mnt/y
else
     exit
fi

echo ""
read -p "Write viafile.txt with contents of viaforensics? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo echo "viaforensics" > ~/mnt/y/viafile.txt
else
     exit
fi

echo ""
read -p "Append .com to contents of viafile.txt? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo echo ".com" >> ~/mnt/y/viafile.txt
else
     exit
fi

echo ""
read -p "Rename viafile.txt to renamed.txt? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo mv ~/mnt/y/viafile.txt ~/mnt/y/renamed.txt
else
     exit
fi

echo ""
read -p "Delete renamed.txt? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo rm ~/mnt/y/renamed.txt
else
     exit
fi

echo ""
read -p "Create last.txt with afphysical? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo echo  "afphysical" > ~/mnt/y/last.txt
else
     exit
fi

echo ""
read -p "Unmount and remove modules? (y/n)?"
if [ "$REPLY" == "y" ] ; then
     sudo umount ~/mnt/y
     sudo rmmod yaffs2
     sudo rmmod nandsim
     sudo rmmod mtdblock
     sudo rmmod nand
     sudo rmmod mtd_blkdevs
     sudo rmmod mtd
     echo "Done"
```

更多资源请访问稀酷客(www.ckook.com)

```
else
    exit
fi
```

希望不要因为缺少支持 YAFFS2 文件系统的商用版工具而使检查师们认为没有方法获取更多的数据。很清楚，在 Ubuntu 工作站上使用现有的免费开源工具是一个进一步进行取证调查的强有力手段。在将这些技术与本章介绍的十六进制分析相结合后，检查师们应该可以发现，他们完全可以从 Android 终端设备上的 YAFFS2 分区中获取证据。当一个涉及嵌入了 Linux 和 YAFFS2 文件系统的医疗设备的案件放在桌面上时，检查师应该非常有信心地投入进去，并找出有力的证据。

7.5 Android 应用分析与参考

虽然本章介绍了很多有用的技术，用于对 Android 终端设备进行取证和安全调查，了解一下核心应用软件中数据都存储在哪里来作为参考还是非常有帮助的。当然，维护一个完整的参考几乎是不可能的，这不仅缘于应用软件的数量之大，而且还因为不同终端设备和不同 Android 系统版本之间的差异。

在以下章节中，我们从本书中所参考的手机中选择 10 个重要的应用软件进行分析，并提供以下数据：

1）应用信息；

2）文件与目录（包括 /data/data、SD 卡以及 eMMC）；

3）重要的数据库表；

4）分析师注意事项。

在对应用软件的分析中，所有的时间戳都以 UNIX 标准毫秒（即自 1970 年 1 月 1 日以来所经历的毫秒数）来计算，除非我们予以特别指出。

这些测试过的应用软件都安装和应用在终端设备上，在分析中使用一个自定义的 Python 程序对本章中介绍过的某些技术进行了自动化。虽然提供了很多信息，在这里还是要提醒读者，这还只不过是一个参考，很有可能依然不完整。分析师们依然应该使用取证和安全工具以及本书前面介绍的各项技术对所检查的终端设备进行全面分析。

7.5.1 Messaging（短信与彩信）

1. 应用信息

这是 Android 系统中的一个默认应用软件，用于处理短信和多媒体（彩信）消息。

- 应用软件名：Messaging；
- 软件包名：com.android.providers.telephony；

更多资源请访问稀酷客(www.ckook.com)

- 版本：2.2；
- 终端设备：HTC Incredible；
- 应用软件开发商：Android。

2. 目录、文件及文件类型

在 /data/data/com.android.providers.telephony 中：

```
com.android.providers.telephony/      directory
├── app_parts                         directory
│   ├── PART_1285875367786            JPEG image data, JFIF standard 1.01
│   ├── PART_1287901591761            JPEG image data, JFIF standard 1.01
│   └── PART_1293199567316            JPEG image data, JFIF standard 1.01
├── databases                         directory
│   ├── mmssms.db                     SQLite 3.x database, user version 60
│   └── telephony.db                  SQLite 3.x database, user version 524296
└── lib                               directory
```

3. 重要的数据库表与文件

短信和彩信应用中用到的重要数据库表和文件如表 7.4 所示。

表 7.4　/data/data/com.android.providers.telephony/databases/mmssms.db 中重要的数据库表与文件

数据库表 / 文件	说　明
words_content（文字内容）	消息的内容
part（部分）	彩信附件的细节，包括类型、名称、在文件系统中的位置（如果是二进制文件，为图像；如果是一般文本的话则为内容）
sms（短信）	完整的短信消息，包括接收方、发送方、人员、时间戳、阅读与状态、发送 / 接收与否状态、消息的内容

4. 分析师注意事项

对于 /data/data/com.android.providers.telephony，分析师需注意以下事项：

- app_parts 文件夹内存有多媒体附件，可能包含图像、视频以及其他所支持的数据。文件的类型没有进行维护和标示，因此，需要使用 file 命令或者其他文件签名分析工具来识别文件类型。
- 由于 telephony 数据库只包含无线服务提供商所提供服务的信息，通常不会引起分析师们太大的兴趣。
- sms 表中包含所有的消息，因此应该成为关注的重点。
- 还有其他几个表好像重复了 sms 表中的部分内容，因此应该看看 words_content 以及其他一些表的内容，以帮助更好地进行分析。
- 也可以看看 com.android.mms。

更多资源请访问稀酷客(www.ckook.com)

7.5.2 多媒体消息帮助应用

1. 应用信息

这个应用软件保存着一些多媒体消息数据，看起来像是针对 Messaging 软件提供帮助的辅助应用。

- 应用软件名：com.android.mm；
- 软件包名：com.android.mms；
- 版本：2.2；
- 终端设备：HTC Incredible；
- 应用软件开发商：Android。

2. 目录、文件及文件类型

在 /data/data/com.android.mms 中：

```
com.android.mms                                    directory
├── bufferFileForMms                               data
├── cache                                          directory
│   ├── PART_1285875367786                         PNG image, 80 x 60,
8-bit/color RGB, non-interlaced
│   ├── PART_1287901591761                         PNG image, 80 x 60,
8-bit/color RGB, non-interlaced
│   └── PART_1293199567316                         PNG image, 80 x 60,
8-bit/color RGB, non-interlaced
├── lib                                            directory
└── shared_prefs                                   directory
    ├── com.android.mms.customizationBySIM.xml     XML  document text
    ├── com.android.mms_preferences.xml            XML  document text
    ├── FIRST_EXECUTE.xml                          XML  document text
    ├── _has_set_default_values.xml                XML  document text
    └── VERY_FIRST_EXECUTE.xml                     XML  document text
```

3. 重要的数据库表与文件

- 在 /data/data/com.android.mms/cache 中的 PART 文件。

4. 分析师注意事项

对于 /data/data/com.android.mms，分析师需注意以下事项：

- 文件“bufferFileForMms”中也许有发送时的缓存数据。但是，这个文件的内容通常是 0x00。
- 缓存区域中的 PART 文件是 Messaging 软件在 /data/data/com.android.providers.telephony/app 处的图像经过缩小后的 PNG 版本。

7.5.3 浏览器

1. 应用信息

这个应用软件是 Android 系统的内置 Web 浏览器，软件代码来源于开源的 WebKit 项目。

- 应用软件名：Internet；

更多资源请访问稀酷客(www.ckook.com)

- 软件包名：com.android.browser；
- 版本：2.2；
- 终端设备：HTC Incredible；
- 应用软件开发商：Android。

2. 目录、文件及文件类型

在 /data/data/com.android.browser 中：

```
com.android.browser/directory
├── app_appcache directory
│   └── ApplicationCache.db SQLite 3.x database, user version 5
├── app_databases directory
│   ├── Databases.db SQLite 3.x database
│   ├── http_a.ringleaderdigital.com_0 directory
│   │   └── 0000000000000002.db SQLite 3.x database
│   ├── http_blogs.techrepublic.com.com_0.localstorage SQLite 3.x database
│   ├── http_b.scorecardresearch.com_0.localstorage SQLite 3.x database
│   ├── http_forumlogr.disqus.com_0.localstorage SQLite 3.x database
│   ├── http_mashable.com_0.localstorage SQLite 3.x database
│   ├── http_mashable.disqus.com_0.localstorage SQLite 3.x database
│   ├── http_m.imdb.com_0.localstorage SQLite 3.x database
│   ├── https_api.twitter.com_0.localstorage SQLite 3.x database
│   ├── http_singularityhub.com_0.localstorage SQLite 3.x database
│   ├── http_singularityhub.disqus.com_0.localstorage SQLite 3.x database
│   ├── http_twitpic.com_0.localstorage SQLite 3.x database
│   ├── http_voices.washingtonpost.com_0.localstorage SQLite 3.x database
│   ├── http_www.accuweather.com_0 directory
│   │   └── 0000000000000001.db SQLite 3.x database
│   ├── http_www.accuweather.com_0.localstorage SQLite 3.x database
│   ├── http_www.cbc.ca_0.localstorage SQLite 3.x database
│   ├── http_www.forensicfocus.com_0.localstorage SQLite 3.x database
│   ├── http_www.forumlogr.com_0.localstorage SQLite 3.x database
│   ├── http_www.google.com_0.localstorage SQLite 3.x database
│   ├── http_www.iphoneworld.ca_0.localstorage SQLite 3.x database
│   ├── http_www.linkedin.com_0.localstorage SQLite 3.x database
│   └── http_www.youtube.com_0.localstorage SQLite 3.x database
├── app_geolocation directory
│   ├── CachedGeoposition.db SQLite 3.x database
│   └── GeolocationPermissions.db SQLite 3.x database
├── app_icons directory
│   └── WebpageIcons.db SQLite 3.x database
├── app_plugins directory
│   └── com.adobe.flashplayer directory
│       ├── .adobe directory
│       │   └── Flash_Player directory
│       │       └── AssetCache directory
│       │           └── YY3JJV4U directory
│       │               ├── 3C82B2A2455B252B8595FD0113249AA19D7E8BDD.heu data
│       │               ├── 3C82B2A2455B252B8595FD0113249AA19D7E8BDD.swz data
│       │               └── cacheSize.txt ASCII text, with no line terminators
│       └── .macromedia directory
│           └── Flash_Player directory
│               ├── adobe.com directory
│               │   └── flashplayer directory
│               │       ├── #cdn.widgetserver.com directory
│               │       │   └── settings.sol data
│               │       ├── #flashapps.ifg.net directory
│               │       │   └── settings.sol data
│               │       ├── #images10.newegg.com directory
│               │       │   └── settings.sol data
│               │       ├── settings.sol data
│               │       └── #s.ytimg.com directory
│               │           └── settings.sol data
│               └── #SharedObjects directory
│                   └── GPDJW6S3 directory
│                       ├── cdn.widgetserver.com directory
```

更多资源请访问稀酷客(www.ckook.com)

```
                    ├── com.quantserve.sol data
                    ├── syndication directory
                    │   └── flash directory
                    │       └── InsertWidget.swf directory
                    │           └── wbx.sol data
                    └── wbx_cookie.sol data
                ├── flashapps.ifg.net directory
                │   └── weather directory
                │       └── weather.swf directory
                │           └── TestMovie_Config_Info.sol data
                ├── images10.newegg.com directory
                │   ├── s7_newegg.sol data
                │   └── WebResource directory
                │       └── Themes directory
                │           └── 2005 directory
                │               └── Nest directory
                │                   └── genericzoomviewer.swf directory
                │                       └── #newegg directory
                │                           └── 16%2D102%2D144%2DZ02_init.sol data
                └── s.ytimg.com directory
                    └── soundData.sol data
├── app_sharedimage directory
├── app_thumbnails directory
├── cache -> /app-cache/com.android.browser/cache broken symbolic link to '/app-cache/
   com.android.browser/cache'
├── databases directory
   ├── browser.db SQLite 3.x database, user version 24
   ├── webviewCache.db SQLite 3.x database, user version 4
   └── webview.db SQLite 3.x database, user version 10
├── lib directory
└── shared_prefs directory
    ├── com.android.browser_preferences.xml XML document text
    └── WebViewSettings.xml XML document text
```

在 /app-cache/com.android.browser/cache 中：

```
app-cache/                  directory
└── com.android.browser     directory
    └── cache               directory
        └── webviewCache    directory
            ├── 00684608    JPEG image data, JFIF standard 1.01, comment:
"CREATOR: gd-jpeg v1.0 (using IJ"
            ├── 00f02aa9    GIF image data, version 89a, 3 x 3
            ├── 0113650e    ASCII text, with very long lines
            ├── 01c6689f    ASCII C program text, with very long lines
            ├── 0249f797    PNG image, 46 x 20, 8-bit/color RGBA,
                            non-interlaced
            ├── 02d78554    GIF image data, version 89a, 468 x 60
            ├── 0305cad6    JPEG image data, JFIF standard 1.02
            ├── 0339028b    GIF image data, version 89a, 155 x 85
            ├── 036d8956    036d8956: GIF image data, version 89a, 15 x 15
            ├── 03bc67f9    ASCII text, with very long lines
            ├── 04e9f7f8    JPEG image data, JFIF standard 1.01
            ├── 056d50ab    HTML document text
            ├── 069360d7    JPEG image data, JFIF standard 1.02
            ├── 073f38ff    GIF image data, version 89a, 20 x 14
            ├── 074a5b68    ASCII text, with very long lines
            ├── 07f04406    PNG image, 200 x 52, 8-bit/color RGBA,
                            non-interlaced
            ├── 07ff1c11    PNG image, 132 x 29, 8-bit/color RGBA,
                            non-interlaced
            ├── 09fda0dd    GIF image data, version 89a, 32 x 11
            ├── 0aec9086    GIF image data, version 89a, 8 x 3
            ├── 0c24e90f    GIF image data, version 89a, 17 x 17
<snip>
```

更多资源请访问稀酷客(www.ckook.com)

3. 重要的数据库表及文件

Web 应用所使用的本地数据库和文件如表 7.5 所示。

表 7.5　/data/data/com.android.browser/app_databases/http_www.google.com _ 0.localstorage 中重要的数据库表及文件

数据库表 / 文件	说　明
ItemTable（项目表）	这个表是一个关键词 / 数值二元组的简单列表；但它可能包含了访问过的网站的有用信息，并且充分利用了本地存储的特性

缓存的地理位置数据如表 7.6 所示。

表 7.6　/data/data/com.android.browser/app_geolocation/CachedGeoposition.db 中重要的数据库表及文件

数据库表 / 文件	说　明
CachedPosition（缓存的位置）	• latitude（纬度）=41.896888 • longitude（经度）=–87.799985 • altitude（高度）= • accuracy（精度）=1368.0 • altitudeAccuracy（高度的精确度）= • heading（头）= • speed（速度）= • timestamp（时间戳）=1296479267929

地理位置访问权限数据库如表 7.7 所示。

表 7.7　/data/data/com.android.browser/app_geolocation/GeolocationPermissions.db 中重要的数据库表及文件

数据库表 / 文件	说　明
Permissions（访问权限）	此表是关于行的来源及相应访问权限的一个列表。例如，http://www.google.com 以及值 1 表明谷歌网站拥有访问地理位置的权限

表 7.8 中的说明是浏览器数据库。

表 7.8　/data/data/com.android.browser/databases/browser.db 中重要的数据库表及文件

数据库表 / 文件	说　明
Bookmarks（书签）	• _id=662 • title=http://mobile.itworld.com/device/article.php?CALL_URL=http://www.itworld.com/security/135495/ddos-attacks-made-worse-firewalls-report-finds • url=http://mobile.itworld.com/device/article.php?CALL_URL=http://www.itworld.com/security/135495/ddos-attacks-made-worse-firewalls-report-finds • visits=1 • date=1296736862801
Searches（搜索）	• _id=4 • search=fogo de chao chicago • date=1291401576968

更多资源请访问稀酷客(www.ckook.com)

表 7.9 中的说明是网站显示数据库。

表 7.9 /data/data/com.android.browser/databases/webview.db 中重要的数据库表及文件

数据库表 / 文件	说　明
cookies	• _id=3912 • name=PHPSESSID • value=25b5b5a8608795fa4ac45d2b872a20e5 • domain=mobile.itworld.com • path=/ • expires= • secure=0
formurl	• _id=95 • url=http://en.m.wikipedia.org/wiki/Dime_(United_States_coin)?wasRedirected=true
formdata	• _id=39 • urlid=95 • name=search • Value=Dime (United States coin)
httpauth	• _id=1 • host= dev-computer-forensics.sans.org • realm= SANS - Restricted Access [Area - 39] • username= your-sans-blog-username • password=your-sans-blog-password
password	• _id=2 • host= httpswww.netflix.com • username= your-netflix-username • password=your-netflix-password

表 7.10 中说明的是网站显示的缓存数据库。

表 7.10 /data/data/com.android.browser/databases/webviewCache.db 中重要的数据库表及文件

数据库表 / 文件	说　明
Cache（缓存）	• _id=464 • url=http://profile.ak.fbcdn.net/hprofile-ak-snc4/hs267.snc3/23271_145853460360_2616_q.jpg • filepath= 00684608 • lastmodity= Thu, 01 Jan 2009 00:00:00 GMT • etag= • expires=1299533415543 • expiresstring= Mon, 07 Mar 2011 21:30:00 GMT • mimetype=image/jpeg • encoding= • httpstatus=200 • location= • contentlength= 2890 • contentdisposition= • crossdomain=

更多资源请访问稀酷客(www.ckook.com)

4. 分析师注意事项

对于 /data/data/com.android.browser，分析师需注意以下事项：

- 如果寻找一个特定的网站，并且此网站有自己独特的图标，则应该查看 app_icons 中的 WebpageIcons.db。
- 具体的应用程序数据库也许包含非常有用的信息。例如，谷歌应用程序数据库内包含有上一次的搜索关键术语、带有时间戳的很多版本的 lon/lat/acc 等。
- com.adobe.flashplayer 目录中不但包含了 Flash cookies，这些文件的后缀为 .sol，可以使用 log2timeline 对其进行语法分析，而且还有部分 Flash .swf 文件。
- 浏览器数据库（databases/browser.db）包含有一个叫做 bookmarks 的表，此表通常被无线服务提供商预置了一些网址。此表同时还包含 Web 浏览历史。还有一个叫做 Searches 的表，包含在浏览器上使用谷歌所做的搜索。
- Web view 数据库（databases/webview.db）包含大量的信息，这些信息不但对取证检查师们很有用，而且对安全工程师们也是如此。任何人都可以看得到 cookies，但是大多数的 cookies 都不安全，也就是说它们无法抵御使用诸如 Firesheep 之类工具发起的 cookies 劫持攻击。Form URL 以及数据通常包含了敏感的信息，httpauth 和 password 也如此。
- Web view cache 数据库（databases/webviewCache.db）提供关于存储在缓存目录中缓存文件的元数据。
- 大多数的终端设备将 Web view cache 数据作为一个子目录保存在 /data/data/com.android.browser 下。然而，在 HTC Incredible 中，这个目录被移到一个 tmpfs（随机访问磁盘）目录下。

7.5.4　联系人

1. 应用信息

这个应用软件是 Android 提供的主要联系人应用软件。虽然还有很多其他的应用软件，此应用软件提供了核心的联系人功能。

- 应用软件名：Contacts；
- 软件包名：com.android.providers.contacts；
- 版本：2.2；
- 终端设备：HTC Incredible；
- 应用软件开发商：Android。

2. 目录、文件及文件类型

在 /data/data/com.android.providers.contacts 中：

```
com.android.providers.contacts                    directory
├── databases                                     directory
│   └── contacts2.db                              SQLite 3.x database, user version 309
├── files                                         directory
```

更多资源请访问稀酷客(www.ckook.com)

```
|   ├── thumbnail_photo_10014.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_10194.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_10199.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_10202.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_10203.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_12450.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_12827.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_12832.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_12833.jpg                    JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_9508.jpg                     JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_9509.jpg                     JPEG image data, JFIF standard 1.01
|   ├── thumbnail_photo_9566.jpg                     JPEG image data, JFIF standard 1.01
|   └── thumbnail_photo_9866.jpg                     JPEG image data, JFIF standard 1.01
├── lib                                              directory
├── shared_prefs                                     directory
|   ├── com.android.providers.contacts_preferences.xml  XML  document text
|   └── ContactsUpgradeReceiver.xml                  XML  document text
└── SNtemp                                           directory
    └── 27386_604172803_5385_n.jpg                   empty
```

3. 重要的数据库表与文件

在这里只有一个数据库，即 contacts2.db，其中却有超过 30 个表。表 7.11 中列出了几个关键的表。

表 7.11 /data/data/com.android.browser/databases/contacts2.db 中重要的数据库表及文件

数据库表 / 文件	说　明
Accounts（账号）	• accounnt_name= viaforensics • account_type= com.twitter.android.auth.login
Calls（通话）	• _id=1156 • number= 3128781100 • date= 1296780296202 • duration=142 • type= 1 • new=1 • name=viaForensics Corporate • numbertype=3 • numberlabel= • raw_contact_id=907 • city_id=
status_updates（状态更新）	• status_update_data_id=14792 • status=installed Facebook for Windows Phone • status_ts=1287597506000 • status_res_package=com.htc.socialnetwork.provider • status_label=2130968576 • status_icon=336859322
Contacts（联系人）	• _id=907 • name_raw_contact_id=907 • photo_id= • custome_ringtone= • send_to_voicemail=0 • times_contacted=19

更多资源请访问稀酷客(www.ckook.com)

（续）

数据库表 / 文件	说　明
Contacts（联系人）	• last_time_contacted=1296780451343 • starred=0 • in_visible_group=1 • has_phone_number=1 • lookup= 1598i2%3A40 • status_update_id= • single_is_restricted=0 • ext_account_Type=com.htc.android.mail.eas • ext_photo_url= • display_name=viaForensics Corporate • default_action=

4. 分析师注意事项

对于 /data/data/com.android.providers.contacts，分析师需注意以下事项：

- 此应用软件将终端设备上的通话记录保存在 calls 表中。
- 在数据库 contacts2.db 中有超过 30 个表，因此，需要进行进一步的检查。在 data 表中还有更多有关联系人的信息，raw_contacts 中也有某些联系人更多的数据。
- 此应用软件能够为很多不同的账号保存联系人信息，如 Gmail、Exchange、Facebook、Twitter 等。某些保存的数据中有来自于这些应用的信息，如 Facebook 状态信息。
- 如果有联系人的相片，这些相片被保存在 files 目录中，文件名为 thumbnail_photo_[NNNNN].jpg。在我们参考的手机 HTC Incredible 中，找到超过 200 个图像，但是也发现了一些重复的情况。

7.5.5　媒体扫描仪

1. 应用信息

这个应用软件将媒体文件的元数据进行扫描，并存储在内部或外部存储器上。

- 应用软件名：Media Store；
- 软件包名：com.android.providers.media；
- 版本：2.2；
- 终端设备：HTC Incredible；
- 应用软件开发商：Android。

2. 目录、文件及文件类型

在 /data/data/com.android.providers.media 中：

```
com.android.providers.media       directory
├── databases                     directory
│   ├── emmc-c7f80810.db          SQLite 3.x database, user version 90
│   ├── external-e0fd1813.db      SQLite 3.x database, user version 90
│   └── internal.db               SQLite 3.x database, user version 90
├── lib                           directory
└── shared_prefs                  directory
    └── ringtoneinit.xml          XML  document text
```

更多资源请访问稀酷客(www.ckook.com)

3. 重要的数据库表与文件

每一个数据库都有相似的结构，如表 7.12 所示。

表 7.12 /data/data/com.android.providers.media/databases/ 中重要的数据库表及文件

数据库表 / 文件	说　明
Images（图像）	• _id=88 • _data= /mnt/emmc/DCIM/100MEDIA/IMAG0074.jpg • _size= 873150 • _display_name= IMAG0074.jpg • mime_type= image/jpeg • title= IMAG0074 • date_added=1295368758 • date_modified=1295372358 • description= • picasa_id= • ispivate= • latitude= • longitude= • datetaken=1295372358000 • orientation=0 • mini_thumb_magic=88 • bucket_id= -942500167 • bucket_display_name=100MEDIA • favorite= • lock_screen=
Videos（视频）	Fields similar to images table
Thumbnails（图标）	• _id=88 • data= /mnt/emmc/dcim/.thumbnails/(28)890943898-s 901931-fH 274-gH 160-mode 10-AG 0.raw • image_id= id 28 • kind= 103 • width=160 • height=160

4. 分析师注意事项

对于 /data/data/com.android.providers.media，分析师需注意以下事项：

- 如果存在的话，names 数据库包含有 volume ID。例如，在我们参考的 HTC Incredible 终端设备上，eMMC FAT32 文件系统的 volume ID 是 0xc7f80810。
- 如果某个目录下有一个文件名为 .nomedia 的文件，那么，媒体存储就不会将此目录下文件的元数据进行扫描和存储。
- 在一个图像被删除后，这个图像的图标很有可能继续存在。同时，即使元数据记录也被删除，如果使用的是 YAFFS2 文件系统，元数据记录还是很有可能被恢复出来。
- 同时也可以对语音文件、相册等进行扫描。
- 可能还存在其他的扫描程序和应用软件，因此，也要查找一下。有一个应用软件将图标保存在 SD 卡上，这对已经被删除的图片和视频提供一些信息。

更多资源请访问稀酷客(www.ckook.com)

7.5.6　YouTube

1. 应用信息

YouTube 是谷歌公司拥有的一个视频观看网站，因此被开发成为 Android 系统中一个原生的应用软件。

- 应用软件名：YouTube；
- 软件包名：com.google.android.youtube；
- 版本：1.6.21；
- 终端设备：HTC Incredible；
- 应用软件开发商：谷歌。

2. 目录、文件及文件类型

在 /data/data/com.google.android.youtube 中：

```
com.google.android.youtube/          directory
├── cache                            directory
│   ├── GDataRequest.-1358025214     XML  document text
│   ├── GDataRequest.149614182       XML  document text
│   ├── GDataRequest.1718906282      XML  document text
│   ├── GDataRequest.307198247       XML  document text
│   ├── GDataRequest.-689089246      XML  document text
│   ├── GDataRequest.718990876       XML  document text
│   └── GDataRequest.-953243531      XML  document text
├── files                            directory
│   └── DATA_Preferences             data
├── lib                              directory
└── shared_prefs                     directory
    └── youtube.xml                  XML  document text
```

3. 重要的数据库表与文件

YouTube 偏好，包括终端设备密钥以及观看过的视频都在文件 /data/data/com.google.android.youtube/shared_prefs/youtube.xml 中：

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
<int name="safe_search_mode" value="1" />
<string name="StrongAuth.deviceKey">AomSH9asdnA5DMqw/8mzHUDXdsaIl5e0s=</string>
<string name="watchedVideos">osnUasdB9bUm-E,SunCPrwdsadOlNI,
qKrrHtY1231WcWA,E7ULR-asdayfNnk,aHmUZgssaoq123rIYA,JaecOddBxDFlas0</string>
<string name="StrongAuth.deviceId">AOuj_RqrF8oasdastTCOySdFNIaNV_M91X
-3MQMbBzzLassdTcAQQn8oYPMWZRK_PiTYMgB-T_rPECOyG4W2jd7zLT7TS2Q</string>
<string name="ratedVideos"></string>
<string name="MasfLogYouTubeApplicationVersion">1.6.21</string>
</map>
```

而观看过的具体影片的信息则保存在 /data/data/com.google.android.youtube/cache 中的 XML 文件中：

```
<?xml version='1.0' encoding='UTF-8'?>
<id>tag:youtube.com,2008:video:E7ULR-yfNnk</id>
<published>2006-09-10T22:22:58.000Z</published>
<updated>2010-11-30T19:06:08.000Z</updated>
```

更多资源请访问稀酷客(www.ckook.com)

```
<category scheme='http://schemas.google.com/g/2005#kind'
term='http://gdata.youtube.com/schemas/2007#video'/>
<category scheme='http://gdata.youtube.com/schemas/2007/categories.cat'
term='Entertainment' label='Entertainment'/>
<category scheme='http://gdata.youtube.com/schemas/2007/keywords.cat'
term='Mime'/>
<category scheme='http://gdata.youtube.com/schemas/2007/keywords.cat'
term='Miming'/>
<category scheme='http://gdata.youtube.com/schemas/2007/keywords.cat'
term='Robot'/>
<category scheme='http://gdata.youtube.com/schemas/2007/keywords.cat'
term='Dance'/>
<title>Cool Mime! Tyson Eberly Mime Performance Part 2</title>
<content type='video/mp4' src='http://v12.lscache5.googlevideo.com/
videoplayback?id=13b50b47ec9f3679&amp;itag=18&amp;uaopt=
no-save&amp;ip=0.0.0.0&amp;ipbits=0&amp;expire=1293757989&amp;
sparams=id,itag,uaopt,ip,ipbits,expire
&amp;signature=C3885D4913EC51106910D8E99049F7D39F7C4AA6.D809F3290B0A6E0D8C83C28
F31261D64BDF0C680&amp;key=yta1
&amp;el=videos&amp;client=mvapp-android-
verizon&amp;devKey=ATEU_r3RX2afGwq_gCqiS2UO88HsQjpE1a8d1GxQnGDm&amp;
app=youtube_gdata'/>

<snip>
```

4. 分析师注意事项

对于 /data/data/com.google.android.youtube，分析师需注意以下事项：

- 检查 cache 和 shared_prefs 目录下的 XML 文件，去寻找观看视频方面的信息；
- 使用文件雕复或其他技术通常可以在终端设备上找到一个视频的开始图像截图。

7.5.7 Cooliris 多媒体展厅

1. 应用信息

这个应用软件为谷歌的 Nexus One 手机开发，提供一个多媒体展厅及扫描仪。

- 应用软件名：Cooliris Media Gallery；
- 软件包名：com.cooliris.media；
- 版本：1.1.30682；
- 终端设备：Google Nexus One；
- 应用软件开发商：Cooliris。

2. 目录、文件及文件类型

在 /data/data/com.cooliris.media 中：

```
com.cooliris.media/      directory
└── databases            directory
    └── picasa.db        SQLite 3.x database, user version 83
```

更重要的是，图标被保存在 SD 卡上：

```
/mnt/sdcard/Android/data/com.cooliris.media                directory
└── cache                                                  directory
```

更多资源请访问稀酷客(www.ckook.com)

```
    ├── geocoder-cache                                    directory
    ├── geocoder-cachechunk_0                             data
    ├── geocoder-cacheindex                               data
    ├── hires-image-cache                                 directory
    ├── hires-image-cache-1158935264581041381_1024.cache  JPEG image data, JFIF standard
    ├── hires-image-cache1585961800385347536_1024.cache   JPEG image data, JFIF standard
    ├── hires-image-cache1588208008548304680_1024.cache   JPEG image data, JFIF standard
    ├── hires-image-cache-1695915026582362443_1024.cache  JPEG image data, JFIF standard
    ├── hires-image-cache1826788044297674713_1024.cache   JPEG image data, JFIF standard
    ├── hires-image-cache1830742186312500388_1024.cache   JPEG image data, JFIF standard
    ├── hires-image-cache2087069753732167412_1024.cache   JPEG image data, JFIF standard
    ├── hires-image-cache2114895255670203853_1024.cache   JPEG image data, JFIF standard
<snip>
    ├── hires-image-cache9169278413697037975_1024.cache   JPEG image data, JFIF standard
    ├── local-album-cache                                    directory
    ├── local-album-cachechunk_0                             data
    ├── local-album-cacheindex                               data
    ├── local-image-thumbs                                   directory
    ├── local-image-thumbschunk_0                            data
    ├── local-image-thumbsindex                              data
    ├── local-meta-cache                                     directory
    ├── local-meta-cachechunk_0                              data
    ├── local-meta-cacheindex                                data
    ├── local-skip-cache                                     directory
    ├── local-skip-cachechunk_0                              MMDF mailbox
    ├── local-skip-cacheindex                                data
    ├── local-video-thumbs                                   directory
    ├── local-video-thumbschunk_0                            data
    ├── local-video-thumbsindex                              data
    ├── picasa-thumbs                                        directory
    └── picasa-thumbsindex                                   data
```

3. 重要的数据库表与文件

对于这个终端设备，picasa.db 数据库为空，但是这些数据库应该会有一些有用的信息。

4. 分析师注意事项

对于 /data/data/com.cooliris.media，分析师需注意以下事项：

此应用软件的价值在于在终端设备上发现的多媒体都缓存在 SD 卡上的“/mnt/sdcard/Android/data/com.cooliris.media/cache”目录下，应该对此进行认真检查。

7.5.8　谷歌地图

1. 应用信息

这是内置的谷歌地图应用软件，用于观看地图、查找地点，甚至可以提供行车路线。

- 应用软件名：Google Maps；
- 软件包名：com.google.android.apps.maps；
- 版本：4.4.0；
- 终端设备：HTC Incredible；
- 应用软件开发商：谷歌。

2. 目录、文件及文件类型

在 /data/data/com.google.android.apps.maps 中：

更多资源请访问稀酷客(www.ckook.com)

```
com.google.android.apps.maps/                          directory
├── app_                                               directory
│   ├── cache                                          directory
│   │   └── cache_r.m                                  data
│   ├── debug                                          directory
│   └── testdata                                       directory
├── cache                                              directory
│   └── webviewCache                                   directory
├── databases                                          directory
│   ├── da_destination_history                         SQLite 3.x database, user version 1
│   ├── friends.db                                     SQLite 3.x database, user version 19
│   ├── LayerInfo                                      SQLite 3.x database, user version 2
│   ├── search_history.db                              SQLite 3.x database, user version 5
│   ├── webviewCache.db                                SQLite 3.x database, user version 4
│   └── webview.db                                     SQLite 3.x database, user version 10
├── files                                              directory
│   ├── DA_DirOpt_en_US                                data
│   ├── DA_LayerInfo                                   data
│   ├── DATA_LATITUDE_WIDGET_MODEL                     data
│   ├── DATA_LAYER_10                                  data
│   ├── DATA_LAYER_11                                  data
│   ├── DATA_LAYER_13                                  data
│   ├── DATA_LAYER_14                                  data
│   ├── DATA_LAYER_15                                  data
│   ├── DATA_LAYER_16                                  data
│   ├── DATA_LAYER_18                                  data
│   ├── DATA_LAYER_2                                   data
│   ├── DATA_LAYER_20                                  data
│   ├── DATA_LAYER_21                                  data
│   ├── DATA_LAYER_24                                  data
│   ├── DATA_LAYER_25                                  data
│   ├── DATA_LAYER_27                                  data
│   ├── DATA_LAYER_28                                  data
│   ├── DATA_LAYER_3                                   data
│   ├── DATA_LAYER_5                                   data
│   ├── DATA_LAYER_6                                   data
│   ├── DATA_LAYER_7                                   data
│   ├── DATA_LAYER_8                                   data
│   ├── DATA_LAYER_9                                   data
│   ├── DATA_location_history                          data
│   ├── DATA_OptionDefinitionBlock_en                  data
│   ├── DATA_Preferences                               data
│   ├── DATA_PROTO_SAVED_CATEGORY_TREE_DB              raw G3 data, byte-padded
│   ├── DATA_PROTO_SAVED_LAYER_STATE                   data
│   ├── DATA_PROTO_SAVED_RECENT_LAYERS                 data
│   ├── DATA_RemoteStringsBlock_en                     data
│   ├── DATA_Restrictions                              raw G3 data, byte-padded
│   ├── DATA_Restrictions_lock                         empty
│   ├── DATA_SAVED_BGFS_3                              data
│   ├── DATA_SAVED_BGFS_EXTRA_3                        data
│   ├── DATA_SAVED_BGSF_                               data
│   ├── DATA_SAVED_REMOTE_ICONS_DATA_BLOCK             data
│   ├── DATA_ServerControlledParametersManager.data    data
│   ├── DATA_STARRING                                  X11 SNF font data, MSB first
│   ├── DATA_SYNC_DATA                                 data
│   ├── DATA_SYNC_DATA_LOCAL                           data
│   ├── DATA_TILE_HISTORY                              data
│   ├── DATA_Tiles                                     data
│   ├── DATA_Tiles_1                                   data
│   ├── DATA_Tiles_2                                   data
│   ├── DATA_Tiles_3                                   data
│   ├── DATA_Tiles_4                                   DBase 3 data file (45375 records)
│   ├── DATA_Tiles_5                                   data
│   ├── DATA_Tiles_6                                   data
│   ├── DATA_Tiles_7                                   data
│   ├── DATA_Tiles_8                                   DBase 3 data file (60175 records)
```

更多资源请访问稀酷客(www.ckook.com)

```
│   ├── event_store_driveabout                    data
│   ├── event_store_LocationFriendService         data
│   ├── NavigationParameters.data                 data
│   ├── NavZoomTables.data                        data
│   ├── nlp_GlsPlatformKey                        data
│   ├── nlp_state                                 data
│   └── ZoomTables.data                           data
├── lib                                           directory
└── shared_prefs                                  directory
    ├── DriveAbout.xml                            XML  document text
    ├── friend_service.xml                        XML  document text
    ├── login_helper.xml                          XML  document text
    └── network_initiated_prefs.xml               XML  document text
```

此应用软件也将数据保存在 SD 卡上：

```
/mnt/sdcard/Android/data/com.google.android.apps.maps/     directory
├── cache                                                  directory
│   ├── cache_its.0                                        data
│   ├── cache_its.m                                        data
│   ├── cache_its_ter.m                                    data
│   ├── cache_r.0                                          data
│   ├── cache_r.1                                          data
│   ├── cache_rgts.0                                       Microsoft Document Imaging Format
│   ├── cache_rgts.m                                       data
│   ├── cache_r.m                                          data
│   ├── cache_vts.0                                        data
│   ├── cache_vts.1                                        data
│   ├── cache_vts_GMM.0                                    data
│   ├── cache_vts_GMM.1                                    data
│   ├── cache_vts_GMM.10                                   data
│   ├── cache_vts_GMM.11                                   data
│   ├── cache_vts_GMM.12                                   data
│   ├── cache_vts_GMM.2                                    data
│   ├── cache_vts_GMM.3                                    data
│   ├── cache_vts_GMM.4                                    data
│   ├── cache_vts_GMM.5                                    data
│   ├── cache_vts_GMM.6                                    data
│   ├── cache_vts_GMM.7                                    data
│   ├── cache_vts_GMM.8                                    data
│   ├── cache_vts_GMM.9                                    data
│   ├── cache_vts_GMM.m                                    data
│   ├── cache_vts.m                                        data
│   ├── cache_vts_tran_GMM.m                               data
│   ├── ._speech_nav_0.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
│   ├── ._speech_nav_1.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
│   ├── ._speech_nav_2.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
│   ├── ._speech_nav_3.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
│   ├── ._speech_nav_4.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
│   ├── ._speech_nav_5.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
│   └── ._speech_nav_6.wav                                 RIFF (little-endian)
data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz
├── debug                                                  directory
└── testdata                                               directory
```

3. 重要的数据库表与文件

虽然每一个数据库都应该被检查，但是在 HTC Incredible 上，其中有两个数据库含

更多资源请访问稀酷客(www.ckook.com)

有非常有用的信息。第一个是 da_destination_history 数据库，如表 7.13 所示，第二个是 search_history 数据库，如表 7.14 所示。

表 7.13 /data/data/com.google.android.apps.maps/databases/da_destination_history 中重要的数据库表与文件

数据库表 / 文件	说　明
destination_history（目的地历史）	• time=1295058395176 • dest_last=37786034 • dest_lng=-122405174 • dest_title=Coffee Bean and Tea Leaf • dest_address=773 Market Street San Francisco, CA 94103 • dest_tokenFbKRQAIdyj60-CE--yMryKRCIQ • cource_lat=37791708 • source_lng= -122410077 • day_of_week=6 • hour_of_day=18

表 7.14 /data/data/com.google.android.apps.maps/databases/search_history.db 中重要的数据库表与文件

数据库表 / 文件	说　明
Suggestions（建议）	• _id=140 • data1=the stanford court, a renaissance hotel, 905 california street, san francisco, ca 94108 • singleResult= • displayQuery=The Stanford Court, A Renaissance Hotel, 905 California Street, San Francisco, CA 94108

files 目录下还有大量的信息。例如，DATA_LAYER_24 文件中的第一部分包含了以下的字符串：

```
t,+0
XThe Stanford Court, A Renaissance Hotel, 905 California Street,
San Francisco, CA 94108
FQqpQAIdGS20-CHNignjYUhNwQ,CJ
$3.75;B
VWalking directions (beta): use caution.
Head southeast on Naglee Ave toward Cayuga Ave
3 %(
Balboa Park BART`
167 ft
37 secsRV
Turn left at Cayuga Ave
78aBo16F4yPZxvWb9KipKA
0.2 mi
3 minsRU
Turn left at Ottawa Ave
3 ((
KutrDTBAAz1iBqg0_dtz3w
167 ft
40 secsRW
Turn right at Delano Ave
paD-lLwjtwdrtiMP6pjAOw
```

更多资源请访问稀酷客(www.ckook.com)

```
0.3 mi
5 minsRV
Turn left at Geneva Ave
CHzTIqx_6bEUmrZ5e2dnBw
0.2 mi
4 minsR
#Millbrae-SFIA to Pittsburg/Baypoint
Balboa Park BART:
Powell St. BARTB
Pittsburg / Bay Point`
11 mins
Civic Center BART
us-ca-bart:CIVC
```

保存在 SD 卡上的数据用于向 Google Maps Navigation（谷歌地图导航）提供实时跟踪导航，同时这些实时跟踪导航都附有时间戳。

```
ahoog@ubuntu:~/htc-inc/mnt/sdcard/Android/data/com.google.android.apps.maps/
cache$ ls -lah | grep speech
-rwxr-xr-x 1 root root 105K 2011-01-27 14:35 ._speech_nav_0.wav
-rwxr-xr-x 1 root root  81K 2011-01-27 14:34 ._speech_nav_1.wav
-rwxr-xr-x 1 root root 127K 2011-01-27 14:34 ._speech_nav_2.wav
-rwxr-xr-x 1 root root  61K 2011-01-27 14:33 ._speech_nav_3.wav
-rwxr-xr-x 1 root root  94K 2011-01-27 14:41 ._speech_nav_4.wav
-rwxr-xr-x 1 root root  67K 2011-01-27 14:41 ._speech_nav_5.wav
-rwxr-xr-x 1 root root 112K 2011-01-27 14:41 ._speech_nav_6.wav
```

4. 分析师注意事项

对于 /data/data/com.google.android.apps.maps，分析师需要注意以下事项。

- 这个应用软件在 files 目录下存储大量关于地图、贴图、搜索方面的信息，因此需要认真检查。
- 虽然每一个单独的数据库内不一定有信息，search_history.db 和 da_destination_history 数据库都应该被认真检查。
- 虽然 shared_prefs 行车路线内含有一些信息，大多数信息对于取证检查来说都不是很有用。但是，认证标记可以被恢复，某些安全审查可能对此会感兴趣。
- 导航功能将地图数据以及存储实际行车路线的 .wav 文件缓存在 SD 卡上。如果查看一下文件的时间戳（文件的后缀是“. speech nav”），就可以得到提供的行车路线的时间，并且还可以听到实际的语音导航。

7.5.9　Gmail

1. 应用信息

谷歌为 Gmail 服务提供一个专门的客户端。

- 应用软件名：Gmail（Google Mail）；
- 软件包名：com.google.android.gm；
- 版本：2.2；
- 终端设备：HTC Incredible；

更多资源请访问稀酷客(www.ckook.com)

• 应用软件开发商：谷歌。

2. 目录、文件及文件类型

在 /mnt/sdcard/Android/data/com.google.android.apps.maps/ 中：

```
com.google.android.gm/                          directory
├── app_sslcache                                directory
│   └── android.clients.google.com.443          data
├── cache                                       directory
│   ├── download                                directory
│   │   └── .jpeg                               JPEG image data, JFIF standard 1.01
│   └── webviewCache                            directory
├── databases                                   directory
│   ├── downloads.db                            SQLite 3.x database, user version 100
│   ├── gmail.db                                SQLite 3.x database, user version 18
│   ├── gmail.db-journal                        empty
│   ├── mailstore.book@viaforensics.com.db      SQLite 3.x database, user version 56
│   ├── mailstore.personal@emailaddress.com.db  SQLite 3.x database, user version 56
│   ├── suggestions.db                          SQLite 3.x database, user version 513
│   ├── webviewCache.db                         SQLite 3.x database, user version 4
│   ├── webviewCache.db-journal                 data
│   └── webview.db                              SQLite 3.x database, user version 10
├── files                                       directory
├── lib                                         directory
└── shared_prefs                                directory
    ├── Gmail.xml                               XML  document text
    └── _has_set_default_values.xml             XML  document text
```

3. 重要的数据库表与文件

Gmail 应用在 SQLite 数据库中存储大量信息，表 7.15 显示的是一个典型的关键表。

表 7.15 /data/data/com.google.android.gm/databases/mailstore.book@viaforensics.com.db 中的重要数据库表与文件

数据库表 / 文件	说　明
conversations（通话）	• _id=1343614283601791413 • queryId=3 • subject=New Banking Trojan Discovered Targeting Businesses' Financial Accounts • maxMessageId=1347608009807593988 • snippet=Forwarded message From: "Andrew Hoog" <book@viaforensics.com> Date... • fromAddress=n 2 0 0
messages（信息）	• _id=2 • messageId= ID 1338926826441746102 • conversation=1338926826441746102 • fromAddress= "Hoog, Andrew" <book@viaforensics.com> • toAddresses="" <us12268@somegraphxcompany.org> • ccAddresses= • bccAddresses= • replyToAddresses= • dateSentMs=1276900125000 • dateReceivedMs=1276900125971 • subject=viaForensics training packet • snippet=Hello, Per our conversation with sandy, we need 11 packets by 10am monday. Th…

更多资源请访问稀酷客(www.ckook.com)

（续）

数据库表 / 文件	说　明
messages（信息）	• listInfo= • personalLevel= 0 • body= Hello, Per our conversation with sandy, we need 11 packets by 10am monday. Thanks. Andrew • bodyEmbedsExternalResources=0 • joinedAttachmentInfos= • synced=1 • error= • clientCreated=0 • refMessageId=0 • forward=0 • includeQuotedText=0 • quoteStartPos=0 • bodyCompressed= • customFromAddress=

4. 分析师注意事项

对于 /data/data/com.google.android.gm，分析师需注意以下事项：

- 每一个配制好的 Gmail 账号都有各自的 SQLite 数据库，用于存储所有的电子邮件；
- 其他一些数据库，如 downloads.db、suggestions.db 及 gmail.db，用于存储其他信息；
- 某些 SQLite 期刊文件也许可以得到修复；
- 在 cache/download 目录中存储着下载的数据；
- 同步的 Gmail 账号也在 shared_prefs 中的 Gmail.xml 文件中进行引用。

7.5.10　Facebook

1. 应用信息

这是一款官方版的 Facebook 应用。

- 应用软件名：Facebook；
- 软件包名：com.facebook.katana；
- 版本：1.2；
- 终端设备：HTC Incredible；
- 应用软件开发商：Facebook。

2. 目录、文件及文件类型

在 /data/data/com.facebook.katana 中：

```
com.facebook.katana/                    directory
├── cache                               directory
│   └── webviewCache                    directory
├── databases                           directory
│   ├── fb.db                           SQLite 3.x database, user version 58
│   ├── webviewCache.db                 SQLite 3.x database, user version 4
│   ├── webviewCache.db-journal         data
│   └── webview.db                      SQLite 3.x database, user version 10
├── files                               directory
```

更多资源请访问稀酷客(www.ckook.com)

```
│   ├── 093m                                JPEG image data, JFIF standard 1.01
│   ├── 0iC8                                JPEG image data, JFIF standard 1.01
│   ├── 0NUX                                JPEG image data, JFIF standard 1.01
│   ├── 0SSB                                JPEG image data, JFIF standard 1.01
│   ├── 0vgY                                JPEG image data, JFIF standard 1.01
│   ├── 0xKj                                JPEG image data, JFIF standard 1.01
<snip>
│   ├── vT4y                                JPEG image data, JFIF standard 1.01
│   ├── VVzz                                JPEG image data, JFIF standard 1.01
│   ├── wE7J                                JPEG image data, JFIF standard 1.01
│   ├── WHTa                                JPEG image data, JFIF standard 1.01
│   ├── X663                                JPEG image data, JFIF standard 1.01
│   ├── XzR6                                JPEG image data, JFIF standard 1.01
│   ├── y44e                                JPEG image data, JFIF standard 1.01
│   └── YLyf                                JPEG image data, JFIF standard 1.01
├── lib                                     directory
└── shared_prefs                            directory
    └── com.facebook.
        katana_preferences.xml              XML  document textvv
```

3. 重要的数据库表与文件

在这里有一个主要的数据库，如表 7.16 所示。

表 7.16 /data/data/com.facebook.katana/databases/fb.db 中重要的数据库表与文件

数据库表 / 文件	说　明
friends（朋友）	• _id=125 • user_id=removed • first_name=FName • last_name=LName • display_name=FName LName • user_image_url= http://profile.ak.fbcdn.net/hprofile-aksnc4/aa.jpg • user_image=ÿØÿà • intent=content://com.facebook.katana.provider.FriendsProvider/info_contacts/uid<snip> • birthday_month= 01 • birthday_day=01 • birthday_year=1929 • hash=-15123123123177976
user_statuses（用户状态）	• _id=21 • user_id=removed • first_name=FName • lasst_name=LName • display_name=FName LName • user_pic=http://profile.ak.fbcdn.net/hprofile-ak-snc4/aa.jpg • timestamp=1296367386 • message=geolocation, geolocation, geolocation...hmmm...
mailbox_messages（邮箱中的邮件）	• _id=13 • folder=0 • tid=1825710720339 • mid=0 • author_id=removed • sent=1290345224 • body=happy birthday! hope it was a fun day and that you have a wonderful year!

更多资源请访问稀酷客(www.ckook.com)

4. 分析师注意事项

对于 /data/data/com.facebook.katana，分析师需注意以下事项。

fb.db 包含了几乎所有的信息，但是在上面只描述了 3 个表。以下是所有表的一份清单：

- albums（相册）；
- info_contacts（信息联系人）；
- notifications（通知）；
- android_metadata（Android 元数据）；
- key_value（关键值）；
- perf_sessions；
- chatconversations（聊天交流）；
- mailbox_messages（邮箱中的邮件）；
- photos（相片）；
- chatmessages（聊天信息）；
- mailbox_messages_display（邮箱中的邮件显示）；
- search_results（搜索结果）；
- default_user_images（默认用户图像）；
- mailbox_profiles（邮箱设置）；
- stream_photos（流相片）；
- events（事件）；
- mailbox_threads（邮箱线索）；
- user_statuses（用户状态）；
- friends（朋友）；
- mailbox_users（邮箱用户）；
- user_values（用户值）。

在 files 目录中有大量来自于 Facebook 的图像。

7.5.11　Adobe Reader

1. 应用信息

以下是官方的针对 PDF 文件的 Abode Reader。

- 应用软件名：Adobe Reader；
- 软件包名：com.adobe.reader；
- 版本：9.0.1；
- 终端设备：HTC Incredible；
- 应用软件开发商：Adobe。

2. 目录、文件及文件类型

在 /data/data/com.adobe.reader 中：

更多资源请访问稀酷客(www.ckook.com)

```
com.adobe.reader/                          directory
├── cache                                  directory
│   └── cache_file.pdf                     PDF document, version 1.1
├── lib                                    directory
└── shared_prefs                           directory
    ├── AdobeReader.xml                    XML  document text
    └── com.adobe.reader.preferences.xml   XML  document text
```

3. 重要的数据库表与文件

com.adobe.reader.preferences.xml 优先文件：

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
<string name="recentFile0">/sdcard/dropbox/Android intro.pdf</string>
<string name="recentFile3">/mnt/sdcard/download/presentation-powerpoint.pdf
</string>
<string name="recentFile2">/mnt/sdcard/download/dinner-1.pdf</string>
<string name="recentFile1">/mnt/sdcard/download/file-1.pdf</string>
</map>
```

4. 分析师注意事项

对于 /data/data/com.adobe.reader，分析师需注意以下事项：

- 缓存的 PDF 文件保存在 cache 目录中；
- 最近文件的列表保存在 cache/com.adobe.reader.preferences.xml 中。

7.6 本章小结

虽然从 Android 终端设备中获取数据是很多研究、开发以及讨论中的关注点，这实际上只是 Android 取证挑战中的一半内容。使用逻辑及物理技术进行分析是非常必要的。但是，在物理上获取数据后还需要进行更加大量的分析。本章的目的是介绍一些关键技术，使取证分析师和安全工程师能够通过获取去检查和抽取数据，即使对于取证共享程序库不支持的文件系统也如此。通过使用取证共享程序库和 Linux 命令，以及在必要时直接对十六进制数据进行分析，就可以得到调查所需要的大部分数据。

7.7 参考资料

File times (Windows). (n.d.). Retrieved February 13, 2011, from http://msdn.microsoft.com/en-us/library/ms724290%28VS.85%29.aspx.

Carrier, B. (n.d.). Mactime output—SleuthKitWiki. Retrieved February 13, 2011, from http://wiki.sleuthkit.org/index.php?title=Mactime_output.

Casey, E. (n.d.). Misinterpretation of file system timestamps. Retrieved February 13, 2011, from http://blog.cmdlabs.com/2009/05/08/misinterpretation-of-file-system-timestamps/.

Debugging in Android with tombstones. (n.d.). Retrieved March 14, 2011, from http://crazydaks.com/debugging-in-android-with-tombstones.html.

Digital assembly: Adroit photo forensics—SmartCarving™. (n.d.). Retrieved February 13, 2011, from http://digital-assembly.com/products/adroit-photo-forensics/features/smartcar

更多资源请访问稀酷客(www.ckook.com)

ving.html.

Digital Detective—DCode. (n.d.). Retrieved February 14, 2011, from http://www.digital-detective.co.uk/freetools/decode.asp.

Epoch converter—epoch & unix timestamp conversion tools. (n.d.). Retrieved February 14, 2011, from http://www.epochconverter.com/.

Grundy, B. (n.d.). Linux LEO. Retrieved February 19, 2011, from http://www.linuxleo.com.

Kessler, G. (n.d.). File signatures table. Retrieved February 13, 2011, from http://www.garykessler.net/library/file_sigs.html.

Ross, A. (n.d.). digfor: Time and timestamps. Retrieved February 13, 2011, from http://digfor.blogspot.com/2008/10/time-and-timestamps.html.

Rovio—Angry Birds. (n.d.). Retrieved February 15, 2011, from http://www.rovio.com/index.php?page=angry-birds.

Scalpel: a frugal, high performance file carver. (n.d.). Retrieved February 13, 2011, from www.digitalforensicssolutions.com/Scalpel/.

What is my IP address. (n.d.). Retrieved February 19, 2011, from http://www.whatismyip.com/.

YAFFS debugging. (n.d.). Retrieved February 17, 2011, from http://www.yaffs.net/yaffs-debugging.

更多资源请访问稀酷客(www.ckook.com)